OpenStitch Studio

Numérisation open source pour broderie machine

Documentation utilisateur et technique

Version du logiciel : 0.1.0 · Version de la documentation : 1.0
Générée le 2026-10-13
Licence : Apache-2.0
Nom temporaire du projet (remplaçable, ADR-001)

Informations éditoriales

Élément	Valeur
Statut du document	Version initiale, générée automatiquement à partir des sources
Version de la documentation	1.0
Version du logiciel documentée	0.1.0 (constante `kAppVersion`, `libs/core/include/openstitch/core/app_info.hpp`)
Date de génération	2026-10-13
Licence du logiciel	Apache-2.0 (`LICENSE`)
Licence de la documentation	Apache-2.0 (même dépôt)
Dépôt	Dépôt Git local ; aucun remote public déclaré à ce jour

Historique des révisions

Version	Date	Changements
1.0	2026-10-13	Première édition complète : audit du dépôt, guides utilisateur et technique, référence des modules, format DST, format projet.

Signaler une erreur

La documentation est générée depuis les fichiers Markdown de `docs/source/`. Pour corriger une erreur, modifiez le fichier source concerné puis régénérez le PDF (voir *Système de génération de la documentation* dans `docs/README.md`). Les affirmations techniques sont tracées vers le code réel dans chaque section « Implémentation associée ».

Table des matières

Les titres sont cliquables et les numéros de page indiqués. Les lecteurs PDF affichent aussi les signets (panneau latéral) pour une navigation détaillée par section.

5 À propos de ce document
7 Introduction et présentation générale
9 Guide de prise en main
12 Installation et premier démarrage
16 Guide utilisateur détaillé
31 Ligne de commande (openstitch-cli)
36 Traitement d'image
38 Segmentation
40 Vectorisation
43 Objets de broderie
45 Numérisation automatique
52 Génération de points — point droit et fondations
55 Moteur de génération de points
67 Satin par squelette et traversées orientées
72 Colonne satin
149 Remplissage tatami
153 Remplissage directionnel
160 Lettrage (texte brodé)
162 Retouche des points et de la géométrie
165 Recherche brevets broderie
169 Fiches brevets : satin, squelette et jonctions
175 Fiches brevets : auto-numérisation (famille Goldman et Brother)
190 Fiches brevets : sous-couches, contours auto-intersectés et points cachés
196 Fiches brevets : stippling, analyse de points et appliqué
201 Palettes et fils
205 Simulation de couture
206 Analyse et validation
208 Fiche de production
210 Format DST (Tajima)
213 Formats PES, JEF et EXP
Format de projet `.osp`
219 Architecture logicielle
221 Référence des modules
224 Modèle de données
226 Algorithmes
228 Compilation et développement
230 Guide du développeur
231 Tests
241 Guide de contribution

243Dépannage
247Limitations et état des fonctionnalités
252Roadmap
254Glossaire
256Licences
258Annexe — Audit du dépôt

Partie I

Présentation et manuel utilisateur

À propos de ce document

Cette documentation décrit **OpenStitch Studio**, une application de bureau libre de numérisation pour broderie machine. Elle couvre l'usage du logiciel, les concepts de broderie qu'il met en œuvre, son architecture logicielle, ses formats de fichiers et sa compilation.

Le document s'adresse à quatre publics : l'**utilisateur débutant**, l'**utilisateur avancé en broderie**, le **développeur contributeur** et le **mainteneur**. Chaque chapitre indique clairement à qui il s'adresse en priorité.

Principe de véracité

Les descriptions de fonctionnalités ont été **rapprochées du code et des tests** présents dans le dépôt (lire le code confirme qu'une fonction existe et ce qu'elle fait, mais ne garantit pas que son résultat est de qualité — d'où la distinction des niveaux de validation, voir *Limitations*). Lorsqu'une capacité était prévue mais n'est pas encore disponible, elle est marquée explicitement :

- **Implémenté** : disponible et testé ;
- **Partiellement implémenté** : présent mais incomplet ;
- **Expérimental** : présent, non stabilisé ;
- **Prévu** : conçu dans l'architecture, non implémenté ;
- **Non implémenté** : absent.

L'annexe *Audit du dépôt* recense l'inventaire factuel qui sert de base à ce document. Chaque chapitre technique se termine par une section **Implémentation associée** listant les fichiers, classes, fonctions et tests réels.

Comment lire ce document

- Vous voulez **utiliser** le logiciel : lisez Introduction, Installation (sans compiler : binaire de la page Releases), Guide de prise en main, puis Guide utilisateur détaillé. En cas de souci : Dépannage et Glossaire.
- Vous voulez **automatiser** ou diagnostiquer : Ligne de commande.
- Vous voulez **comprendre la broderie** telle qu'implémentée : lisez *Concepts* (point droit, satin, tatami) et Numérisation automatique.
- Vous voulez **contribuer** : lisez Architecture, Référence des modules, Modèle de données, Algorithmes, Compilation et développement, Tests.
- Vous voulez **maintenir** : ajoutez Limitations, Roadmap, Licences et le Guide de contribution.

Table des matières

Utiliser le logiciel

- Introduction
- Guide de prise en main
- Installation et premier démarrage
- Guide utilisateur détaillé
- Ligne de commande (openstitch-cli)
- Dépannage
- Glossaire

Du pixel au point

- Traitement d'image
- Segmentation
- Vectorisation
- Objets de broderie
- Numérisation automatique
- Génération de points et moteur de points
- Satin par squelette, colonne satin, tatami, remplissage directionnel
- Retouche des points
- Palettes et fils, simulation, analyse et validation, fiche de production

Formats, architecture et développement

- Format DST, formats PES, JEF et EXP, format de projet `.osp`
- Architecture, référence des modules, modèle de données, algorithmes
- Compilation et développement, guide du développeur, tests, contribution
- Limitations, roadmap, licences
- Recherche brevets : synthèse, satin, auto-numérisation, sous-couches, décoratif et appliqué
- Annexe : audit du dépôt

Dans le PDF, tous ces liens sont cliquables (et la table des matières générée au début du document renvoie aussi aux chapitres).

Note : Ce document est généré automatiquement à partir des fichiers Markdown de `docs/source/`. Ne le modifiez pas directement dans le PDF.

Introduction et présentation générale

Vision du projet

OpenStitch Studio vise à offrir, **gratuitement et en logiciel libre**, la chaîne de travail qui va d'une **image matricielle** jusqu'à un **fichier de broderie machine** au format DST (Tajima). Le projet est écrit principalement en C++ moderne, fonctionne localement (sans compte, sans cloud, sans télémétrie) et sépare strictement son cœur métier de son interface graphique.

Le nom « OpenStitch Studio » est temporaire et remplaçable (décision ADR-001) : il n'apparaît en dur qu'à un seul endroit du code (`kAppName`).

Cas d'usage

- Transformer un logo ou une illustration simple en motif brodable.
- Segmenter une image en zones de couleur et les convertir en objets de broderie éditables (contours, remplissages, colonnes satin).
- Générer, prévisualiser et analyser les points avant de produire un fichier DST.
- Relire un fichier DST existant et l'inspecter.

La chaîne de travail en un coup d'œil

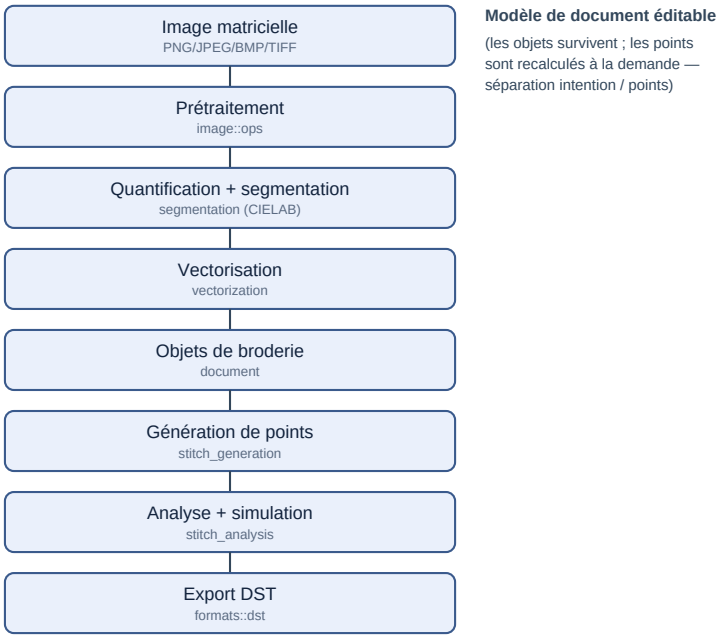


Figure 1 — Le pipeline image → broderie et les modules responsables.

L'idée centrale est la **séparation entre l'intention et le résultat** : la géométrie éditable (objets vectoriels et de broderie) est conservée dans un modèle de document ; les **points** sont recalculés à la demande à partir de cette géométrie. Modifier un paramètre régénère les points sans détruire l'objet source.

Quatre représentations distinctes

Il est essentiel de distinguer quatre choses que le logiciel manipule :

Représentation	Ce que c'est	Type/Module
Image	pixels RGBA	`image::Image`
Région	zone de couleur connexe issue de la segmentation	`segmentation::Region`
Objet vectoriel	contours (extérieur + trous), éditables	`document::VectorObject`
Objet de broderie	intention de couture (type de point + paramètres)	`document::EmbroideryObject`
Séquence de points	commandes machine (Stitch/Jump/…)	`stitch::StitchSequence`

Un DST, lui, ne contient **que** la séquence de points : il ne conserve **pas** les objets éditables (voir le chapitre *Format DST*).

Formats

- **Entrée image** : PNG, JPEG, BMP, TIFF.
- **Broderie** : DST en import et export.
- **Diagnostic** : SVG en export.
- **Projet** : `.osp` (archive contenant tout le document éditable).

Limites générales

Le logiciel constitue un **prototype fonctionnel** couvrant l'ensemble du pipeline principal. Ses composants sont **testés logiciellement**, mais la **qualité de broderie produite n'a pas encore été validée sur une machine à broder réelle**. Les heuristiques physiques (compensation de tirage, densité) sont paramétrables mais approximatives, le satin est **expérimental** et le tatami **partiel** (voir *Limitations et roadmap*). Certaines conventions DST varient aussi selon les machines.

Limitation : Certaines fonctionnalités décrites dans l'étude de cadrage initiale (palette de fils réelle, remplissages courbes, profils machine avancés, édition manuelle des points) sont **prévues mais non implémentées**. Elles sont signalées comme telles tout au long du document.

Philosophie open source

Le projet est sous licence **Apache-2.0** (permissive avec clause de brevets). Pour conserver cette distribution, aucun code sous licence copyleft incompatible (par exemple la GPL d'Ink/Stitch) ne doit être intégré directement — cela imposerait vraisemblablement une redistribution sous licence copyleft. Les idées algorithmiques restent réimplémentables indépendamment à partir de sources publiques. Voir *Licences* pour la formulation complète et les précautions.

Implémentation associée

- `libs/core/include/openstitch/core/app_info.hpp` — nom et version.
- `docs/phase0/` — étude de cadrage et 14 décisions d'architecture (ADR).
- `README.md` — résumé du projet (établi à un jalon antérieur).

Guide de prise en main

Public : utilisateur débutant. Ce tutoriel suit un flux réaliste, **adapté à ce qui est réellement implémenté**. Il n'a qu'une voie pour débuter : **image** → **numérisation automatique** → **auto-satin sur les bandes fines** → **export DST**. Le détail de chaque menu est dans le Guide utilisateur détaillé ; l'installation (sans compiler) est dans Installation.

Important : le fichier produit **n'a pas encore été validé sur une machine à broder réelle** (voir Limitations). Faites un essai sur chute de tissu avant tout ouvrage.

Vue d'ensemble

Image → préparation → segmentation en régions de couleur → **numérisation automatique** (objets éditables) → **satin automatique** pour les formes allongées → vérification (analyse, simulation) → export DST. Tout est annulable (**Ctrl+Z**) et le projet `.osp` conserve l'image, les régions et les objets éditables, ce que le DST ne fait pas.

1. Ouvrir une image

Fichier → **Ouvrir une image…** (Ctrl+O). Choisissez un PNG, JPEG, BMP, TIFF ou SVG. Un logo simple à quelques couleurs franches donne les meilleurs résultats. Les fichiers récents sont dans **Fichier** → **Récents**.

2. Choisir les dimensions physiques

Un dialogue d'import demande la **taille physique** en millimètres (largeur et hauteur), avec l'option « conserver les proportions ». La valeur par défaut suppose 96 dpi. C'est ici que se fixe la résolution de travail (mm par pixel) — elle ne changera plus ensuite.

Conseil : visez une taille qui tient dans le cadre affiché (100 × 100 mm par défaut, réglable par **Affichage** → **Taille du cadre…**) ; sinon le motif dépassera du tambour.

3. Préparer l'image

Menu **Image** : niveaux de gris, luminosité/contraste (avec aperçu), débruitage, symétries, rotations 90°, recadrage (par sélection au rectangle) et quantification des couleurs. Toutes ces opérations sont **non destructives** : l'original est conservé, chaque opération s'annule (Ctrl+Z).

4. Segmenter en régions

Segmentation → **Segmenter l'image…** : choisissez le nombre maximal de couleurs et la taille minimale de région. Le logiciel quantifie en espace perceptuel CIELAB puis extrait les **régions connexes**. Activez **Afficher la carte des régions** pour les visualiser. Cliquez une région pour la sélectionner (ses infos s'affichent dans la barre d'état) ; vous pouvez la **supprimer** (elle redevient du fond), la **recolorer** ou la **fusionner** avec une voisine (« Fusionner avec… » puis clic sur la cible).

5. Numériser automatiquement

Broderie → **Numérisation automatique** crée d'un coup les objets éditables de toutes les régions. Le dialogue propose :

- **Ignorer la plus grande région** : cochée d'office seulement si elle ressemble à un fond quasi blanc qui encadre le motif (la couleur et la part de l'image sont affichées pour vous laisser trancher) ;
- la stratégie **Formes pleines** (remplissages) ou **Contours (dessin au trait)**, pour un dessin fait de traits ;
- un curseur de détail.

Les zones remplissables deviennent des **tatamis**, les petites formes des contours cousus. Le satin n'est **pas** posé automatiquement : on le demande région par région, à l'étape suivante. Pour comprendre ce qui se passe, voir Numérisation automatique.

6. Auto-satin pour les bandes fines

Pour une forme allongée (lettre, trait épais, bordure), sélectionnez la forme puis **Broderie** → **Créer un satin automatique…**. Le logiciel construit la colonne par **squelette** de la forme, **sans que vous posiez de rails** : l'aperçu annonce le nombre de colonnes et la couverture estimée. S'il **refuse** la forme (disque, forme compacte…), il explique pourquoi et propose un tatami à la place. Réglages : espacement, compensation de tirage, sous-couche, fractionnement des traversées longues.

L'entrée voisine **Convertir automatiquement en satin (expérimental)…** utilise le même moteur avec des réglages par défaut et une simple confirmation.

Limitation — l'auto-satin par squelette est **expérimental et non validé sur machine**. Vérifiez l'aperçu et la simulation avant de broder, et préférez un tatami en cas de doute (clic droit ■ Type de points).

On change ensuite le type d'une forme par **clic droit ■ Type de points**, et on règle l'orientation d'un tatami à la souris (poignée de rotation) ou par **Orientation du remplissage…**.

7. Régler les points et les couleurs

Les points se régénèrent automatiquement à chaque changement ; l'**inspecteur** (à droite) expose les paramètres de couture de l'objet sélectionné (densité, longueur, compensation, sous-couche…). La couleur d'un objet reprend celle de sa région : le logiciel gère des couleurs **RGB libres**, pas encore des références de fils de fabricant (voir Palettes et fils).

8. Organiser l'ordre de couture

Le panneau **Ordre de couture** liste les objets ; vous pouvez les monter/descendre, les verrouiller, et appliquer une stratégie automatique (par couleur, par proximité) avec une estimation de coût.

9. Simuler et analyser

La **barre de simulation** rejoue la couture point par point (lecture/pause, curseur). **Analyse** → **Analyser le motif** (F5) liste les problèmes détectés (points trop courts/longs, sauts trop longs, hors cadre) ; un double-clic centre la vue sur le problème.

10. Exporter en DST

Fichier → **Exporter une broderie machine…** (Ctrl+E ; DST, PES, JEF ou EXP) : un résumé (dimensions, points, coupes, changements de couleur) s'affiche avant l'écriture. Le DST **ne conserve pas** les objets éditables : gardez aussi votre projet `.osp` (**Ctrl+S**). Le seuil de coupe de fil, la coupe avant changement de couleur et les points d'arrêt se règlent dans **Broderie** → **Options de génération…** ; le fichier se termine par une coupe finale. Ces coupes sont **à tester sur votre machine** (voir Format DST).

11. Vérifier le fichier

En ligne de commande : `openstitch-cli stats motif.dst` affiche points, sauts, dimensions et longueur de fil estimée ; `openstitch-cli dst2svg motif.dst apercu.svg` produit un aperçu vectoriel. Voir Ligne de commande.

Si la machine ne lit pas le fichier ou si quelque chose ne va pas : Dépannage.

Avancé : colonne satin à rails manuels

Réservé aux besoins que l'auto-satin ne couvre pas. **Cette voie est en partie archivée** (voir Colonne satin : seules certaines parties sont encore maintenues). L'outil **Colonne satin** (touche `S` dans la palette) trace deux rails à la main. Pour une colonne satin sélectionnée, **Broderie ■ Modifier la colonne satin (rails + guides)…** (`Maj+E`) affiche les nœuds des rails et les barreaux d'orientation : glissez une extrémité le long de son rail pour infléchir localement les points ; **Ajouter un guide satin** (`Maj+G`) partage le plus grand intervalle ; le logiciel garde au moins deux guides et chaque geste est annulable. Le sous-menu **Remodelage satin (avancé)** n'affiche que les rails ou que les guides.

Implémentation associée

- `apps/desktop/main_window.cpp` — tous les menus et actions ci-dessus.
- `apps/desktop/main_window_satin_auto.cpp` — « Créer un satin automatique ».
- `apps/desktop/import_dialog.cpp` — dialogue de taille physique.
- `libs/autodigitize/src/autodigitize.cpp` — numérisation automatique.
- `apps/cli/main.cpp` — `stats`, `dst2svg`.

Installation et premier démarrage

Public : utilisateur débutant (sections 1 à 4), développeur (section 5).

OpenStitch Studio se **télécharge sans rien compiler** depuis la page *Releases* du dépôt GitHub (https://github.com/ElZacho200a/OpenStitch/releases). La compilation depuis les sources ne concerne que les développeurs (section « 5. Compiler depuis les sources » plus bas et Compilation et développement).

Systèmes supportés

Système	Statut
Windows 10 / 11 (x64)	Cible principale (application complète)
Linux	Cœur + CLI uniquement (garde-fou de portabilité, vérifié en CI) — **pas** l'interface Qt à ce stade
macOS	Prévu, non vérifié

1. Installer sans compiler

Sur la page *Releases* du dépôt, trois sortes de téléchargements existent :

Pièce jointe	Quand	Pour qui
`OpenStitchStudio-Setup-X.Y.Z.exe` (installeur)	publiée sur chaque version `vX.Y.Z`	**recommandé** : installation par utilisateur (sans droits administrateur), raccourci Bureau en option, désinstallation dans « Applications installées »
`openstitch-windows-x64.zip`	release « Dernier build (main) », remplacée à chaque validation de `main`	essayer la version la plus récente sans installer
ZIP `openstitch-beta-windows-x64`	onglet *Actions* → exécution concernée → *Artifacts* (30 jours)	builds bêta, **pas forcément validés par les tests**

Avec l'installeur : lancez-le, choisissez la langue et le dossier, puis démarrez *OpenStitch Studio* depuis le menu Démarrer.

Avec un ZIP : **décompressez tout le ZIP** (clic droit → *Extraire tout* ; ne lancez pas l'exécutable depuis l'aperçu de l'archive), puis lancez `desktop\Release\openstitch.exe`. Le dossier contient les DLL et les plugins Qt dont l'application a besoin : **ne déplacez pas `openstitch.exe` seul**. L'outil en ligne de commande `openstitch-cli.exe` est dans `cli\Release\` (voir Ligne de commande).

Windows SmartScreen et « éditeur inconnu »

Les binaires ne sont **pas signés** (aucun certificat d'éditeur). Au premier lancement, Windows SmartScreen peut afficher « Windows a protégé votre PC » ou « Éditeur inconnu » : cliquez sur **Informations complémentaires**, puis **Exécuter quand même**. Si vous préférez ne pas faire confiance au binaire, compilez depuis les sources (section 5) : le code est entièrement public. Un antivirus peut aussi mettre en quarantaine un ZIP téléchargé ; rien n'est envoyé sur Internet par l'application (100 % local, sans compte ni télémétrie).

Si une DLL est introuvable

« Le programme ne peut pas démarrer car `Qt6Core.dll` (ou `opencv_*.dll`, `VCRUNTIME140.dll`…) est introuvable » signifie en général que l'exécutable a été copié **sans ses voisins** ou lancé depuis l'archive non extraite. Extrayez le ZIP en entier, ou réinstallez avec l'installeur. Pour `VCRUNTIME140*.dll` / `MSVCP140.dll`, installez le *Microsoft Visual C++ Redistributable 2015-2022 (x64)*.

2. Premier démarrage

Lancez `openstitch.exe`. La fenêtre principale s'ouvre sur un **canevas** vide, avec des règles graduées en millimètres et un **cadre** de broderie de 100 × 100 mm par défaut. L'écran d'accueil propose **Ouvrir une image**, **Ouvrir un projet** et **Importer un DST**. Continuez avec le Guide de prise en main.

3. Où l'application range ses données

Quoi	Où (Windows)	Contenu
Vos fichiers	là où vous les enregistrez	projets `.osp`, DST exportés : **jamais** ailleurs sans votre accord
Préférences (QSettings)	registre, clé `HKEY_CURRENT_USER\Software\OpenStitch\OpenStitch Studio`	disposition des panneaux, géométrie de fenêtre, thème, densité, navigation, projets récents, préférences IA, accrochage des nœuds
Sauvegarde automatique	`%APPDATA%\OpenStitch\OpenStitch Studio\autosave\` (sous `%USERPROFILE%\AppData\Roaming`)	un instantané `<empreinte>.osp` + un petit fichier `.json` (chemin d'origine, horodatage) par document modifié
Journaux	sortie d'erreur de la console uniquement	**aucun fichier de log** n'est écrit : lancez `openstitch.exe` depuis un terminal pour voir les messages

Sauvegarde automatique : toutes les 2 minutes, si le document est modifié et non vide. Elle est supprimée à la fermeture normale de l'application ; après un arrêt anormal, un dialogue **Récupération après un arrêt anormal** propose de **Récupérer** ou d'**Ignorer** au démarrage suivant.

Réinitialiser les préférences : fermez l'application et supprimez la clé de registre ci-dessus (`reg delete "HKCU\Software\OpenStitch" /f`). **Remettre la disposition** : menu Affichage ■ Panneaux ■ Réinitialiser la disposition.

4. Désinstaller

- **Installeur** : *Paramètres* → *Applications* → *Applications installées* → OpenStitch Studio → Désinstaller.
- **ZIP** : supprimez le dossier extrait.
- Dans les deux cas, supprimez si vous le souhaitez la clé de registre `HKCU\Software\OpenStitch` et le dossier `%APPDATA%\OpenStitch` (préférences et sauvegardes automatiques). Vos projets `.osp` ne sont jamais supprimés.

5. Compiler depuis les sources (développeurs)

À réserver aux contributeurs, ou pour ne pas utiliser de binaire non signé. Deux voies :

Voie rapide : `scripts\build.ps1`. Elle **installe tout ce qui manque** : le script liste d'abord les outils absents (CMake, Visual Studio Build Tools et son workload C++ — environ 5 Go —, vcpkg, Qt 6.8.3 via `aqtinstall`), **demande confirmation**, puis les installe et écrit les variables utilisateur `VCPKG_ROOT` et

`QT_ROOT` de façon permanente avant de compiler Debug et Release.

```
.\scripts\build.ps1 -Test           # confirmer l'installation, tout compiler, lancer les tests
.\scripts\build.ps1 -Yes            # confirme d'avance (aucune invite)
.\scripts\build.ps1 -SkipBootstrap  # poste déjà configuré : n'installe rien
```

Voie manuelle (équivalent détaillé de ce que fait le script) :

Outil	Version	Rôle
Visual Studio 2022 ou plus récent (charge « Développement Desktop C++ »)	MSVC v143+	Compilateur
CMake	≥ 3.27	Configuration du build
Git	récent	Récupération du code et de vcpkg
vcpkg	bootstrappé	Dépendances (OpenCV, Clipper2, …)
Qt	6.8 LTS, binaires `msvc2022_64`	Interface graphique
Espace disque	~10 Go	vcpkg compile OpenCV au premier configure

1. Installer vcpkg et définir `VCPKG_ROOT` :

```
git clone https://github.com/microsoft/vcpkg.git C:\dev\vcpkg
C:\dev\vcpkg\bootstrap-vcpkg.bat -disableMetrics
setx VCPKG_ROOT C:\dev\vcpkg
```

2. Installer Qt 6.8 (sans compte, via `aqtinstall`) et définir `QT_ROOT` :

```
python -m pip install --user aqtinstall
python -m aqt install-qt windows desktop 6.8.3 win64_msvc2022_64 -O C:\Qt -m qtimageformats
setx QT_ROOT C:\Qt\6.8.3\msvc2022_64
```

3. Configurer, compiler, tester (nouveau terminal pour prendre les variables) :

```
cmake --preset msvc
cmake --build --preset msvc-debug
ctest --preset msvc-debug
```

Les exécutables produits :

- `build\msvc\apps\desktop\Debug\openstitch.exe` — application graphique ;
- `build\msvc\apps\cli\Debug\openstitch-cli.exe` — outil en ligne de commande.

Avertissement : au **premier** `cmake --preset msvc`, vcpkg compile OpenCV et d'autres dépendances, ce qui peut prendre 10 à 30 minutes. Les compilations suivantes réutilisent le cache binaire de vcpkg. Les problèmes de compilation sont traités dans Dépannage ; l'organisation du build dans Compilation et développement.

Vérification : `ctest --preset msvc-debug` doit rapporter **100 % de tests réussis** ; `openstitch-cli.exe --version` affiche la version ; `openstitch-cli.exe info une-image.png` affiche les métadonnées d'une image.

Implémentation associée

- `.github/workflows/ci.yml`, `.github/workflows/release.yml` — builds bêta, release « latest » et installeur.
- `packaging/windows/installer.iss` — installeur Inno Setup (par utilisateur).
- `apps/desktop/main.cpp` — nom d'organisation et d'application (clé QSettings).
- `apps/desktop/autosave.cpp`, `apps/desktop/recent_files.cpp` — sauvegarde automatique et projets récents.
- `scripts/build.ps1` — voie rapide ; `CMakePresets.json` — presets `msvc` et `linux-core` ; `vcpkg.json` — dépendances verrouillées par baseline.

Guide utilisateur détaillé

Public : utilisateur débutant et avancé. Ce chapitre documente chaque menu, outil et raccourci **réellement présents** dans `apps/desktop/`. Pour un premier motif pas à pas, voir le Guide de prise en main ; pour un problème, Dépannage ; pour les mots techniques, Glossaire.

Disposition générale

L'interface place le **canevas au centre** et l'entoure de zones fonctionnelles :

```
Menus
Barre d'outils principale
Barre contextuelle (suit la sélection)
[Palette d'outils] [ CANEVAS ] [ Inspecteur ]
[Document / Ordre] [ Workflow ]
Barre de simulation (zone réservée)
Barre d'état
```

Tous les panneaux sont des **docks** redimensionnables, déplaçables et masquables (**Ctrl+Shift+P** = mode canevas). Leur disposition, la géométrie de la fenêtre, le **thème** et la **densité** sont mémorisés d'une session à l'autre (préférences d'interface, distinctes du projet `.osp`).

État d'accueil

Tant qu'aucun document n'est ouvert, le centre du canevas propose **Ouvrir une image**, **Ouvrir un projet** et **Importer un DST**, avec une courte explication et les cinq derniers projets (le menu Fichier ■ Récents en garde dix).

Ouvrir un fichier depuis l'Explorateur ou par glisser-déposer

Un fichier peut être ouvert sans passer par les menus : double-clic sur un `.osp` (ou « Ouvrir avec » OpenStitch Studio), nom de fichier passé en argument de la ligne de commande, ou glisser-déposer sur la fenêtre. L'application route par extension : `.osp` (projet), `.dst` (import machine), `.svg` et images PNG/JPEG/BMP/TIFF. Les mêmes gardes que les menus s'appliquent : si le projet courant a des modifications non enregistrées, l'application demande d'abord d'enregistrer. Une extension non prise en charge est signalée en barre d'état, sans rien ouvrir.

Le canevas

La zone centrale est un canevas dont l'unité est le **millimètre** (origine au centre, Y vers le haut). Des **règles** graduées en mm bordent le haut et la gauche ; une **grille** adaptative et le **cadre** de broderie (rectangle rouge, défini par l'utilisateur, cf. *Taille du cadre*) sont dessinés. La position du curseur en mm apparaît dans la barre d'état.

Le **mode d'interaction** vient de la palette d'outils (à gauche) :

- **Sélection** (`V`) : sélectionner une région/un objet (Maj ajoute, Ctrl bascule, glisser dans le vide trace un rectangle de sélection) ; la vue se déplace au clic molette ou à Espace + glisser.
- **Déplacer la vue** (`H`) : déplacement pur (le clic ne sélectionne pas).
- **Recadrer l'image** (`M`) : glissez le cadre à conserver ; l'image est recadrée **au relâchement** (Échap : annuler). Le message de la barre d'état le rappelle, et une confirmation est demandée si des objets existent déjà (ils ne suivraient pas l'image). Pour dessiner un rectangle, utilisez l'outil `R`.
- **Outils de dessin** (palette de gauche) : **Rectangle** (`R`), **Ellipse** (`O`, Maj = cercle), **Polygone** (`P`), **Polygone régulier** (`G`, nombre de côtés dans la palette), **Courbe de Bézier** (`B`), **Main levée** (`L`) et **Colonne satin** (`S`, clics alternés côté A / côté B). Entrée ou double-clic termine le tracé, Retour arrière retire le dernier point, Échap l'annule.
- **Couteau** (`K`) : trace une ligne qui découpe les formes traversées (voir *Menu Forme*).

- **Zoom** : molette (ancrée sous le curseur) ou barre d'outils / menu Affichage.
- **Échap** : revient à la Sélection et annule le mode fusion en cours.

Sélection multiple

Avec l'outil Sélection, **Maj + clic** ajoute un objet à la sélection, **Ctrl + clic** le bascule, et **glisser dans le vide** trace un rectangle (vers la droite : objets entièrement englobés ; vers la gauche : objets touchés ; Maj/Ctrl s'appliquent aussi au rectangle). Une sélection de plusieurs **objets vectoriels** se déplace d'un bloc (glisser), se duplique (Alt + glisser) et se supprime ensemble (clic droit ■ « Supprimer N objets »). Les entrées qui n'ont de sens que pour un seul objet (type de points, décaler, orientation) disparaissent alors du menu. **Les régions de segmentation** ont leur propre sélection multiple (voir *Menu Segmentation*) ; la dernière région cliquée est la « région active ».

La sélection d'un objet est tracée en **double contraste** (halo clair + trait d'accent), lisible sur tout fond. Le rendu est organisé en deux couches (image/vecteurs/régions d'une part, points d'autre part) pour rester fluide sur de gros motifs.

Souris et clavier

Le comportement de la souris et du clavier dans le canevas suit une **table unique** (`apps/desktop/interaction_map.cpp`) : c'est elle qui pilote le comportement du canevas, la ligne d'indications de la barre d'état et le tableau ci-dessous. Principes : la **molette** zoome sous le curseur (**Ctrl + molette** aussi ; c'est ainsi que Windows livre le pincement d'un pavé tactile), le **clic molette** ou **Espace** + glisser déplace la vue, **Maj** ajoute à la sélection, **Ctrl** ajoute ou retire, un **appui long** ou **Alt + clic** ouvre « Sélectionner dessous ». La correspondance est exacte : Maj + clic n'est pas un clic simple, et le panoramique reste disponible pendant un outil de dessin. Sous Windows, un pavé tactile se distingue mal d'une molette : le préréglage de navigation « Pavé tactile » (Affichage ■ Navigation) y met en avant Espace + glisser et Ctrl + molette.

Le tableau est **généré** par `openstitch_gesture_table --markdown` ; ne pas le modifier à la main (le test `docs_gestures_in_sync` échoue si la table du code et ce bloc divergent). Pour le régénérer, recopier la sortie de la commande entre les deux marqueurs.

Réf.	Contexte	Geste	Action
G1	Partout	Molette	Zoom au curseur
G2	Partout	Ctrl + Maj + clic molette + glisser	Zoom continu
G3	Partout	Clic molette + glisser	Panoramique
G4	Partout	Double-clic molette	Cadrer le design
G5	Partout	Espace + glisser	Panoramique (sans clic molette)
G6	Partout	Maj + molette	Défilement horizontal
G7	Partout	Alt + molette	Défilement vertical
G8	Partout	Défilement à deux doigts	Panoramique
G9	Partout	Pincement	Zoom au curseur
G10	Sélection	Clic droit	Menu contextuel
G10b	Déplacement	Clic droit	Menu contextuel
G10c	Édition de nœuds	Clic droit	Menu contextuel
G10d	Édition de points	Clic droit	Menu contextuel
G11	Partout	Suppr	Supprimer la sélection
G12	Partout	Échap	Annuler l'outil en cours
G13	Partout	Ctrl + molette	Zoom au curseur
G14	Partout	F	Ajuster au canevas
G15	Partout	Flèches	Déplacer l'objet de 0,1 mm
G15b	Partout	Maj + Flèches	Déplacer l'objet de 1 mm
P1	Déplacer la vue	Glisser	Panoramique
S1	Sélection	Clic	Sélectionner (je vide désélectionne)
S2	Sélection	Maj + clic	Ajouter à la sélection
S3	Sélection	Ctrl + clic	Basculer dans la sélection
S3b	Sélection	Ctrl + Maj + clic	Basculer dans la sélection
S4	Sélection	Appui long	Sélectionner dessous
S5	Sélection	Alt + clic	Sélectionner dessous
S6	Sélection	Glisser	Sélection par rectangle (vers la droite : englobe, vers la gauche : croise)
S7	Sélection	Maj + glisser	Rectangle : ajouter à la sélection
S8	Sélection	Ctrl + glisser	Rectangle : basculer dans la sélection
S8b	Sélection	Ctrl + Maj + glisser	Rectangle : basculer dans la sélection
S10	Sélection	Double-clic	Entrer en édition de l'objet
S11	Sélection	Survol	Surbrillance de pré-sélection
M1	Déplacement	Glisser	Déplacer l'objet
M2	Déplacement	Maj + glisser	Verrouiller l'axe
M4	Déplacement	Alt + glisser	Dupliquer en déplaçant
D1	Dessin (clics)	Clic	Ajouter un point
D2	Dessin (clics)	Double-clic	Terminer le tracé
D2b	Dessin (clics)	Entrée	Terminer le tracé
D3	Dessin (cadre)	Maj	Contraindre la forme (carré, cercle)
D4	Dessin (cadre)	Alt	Dessiner depuis le centre
D5	Dessin (clics)	Ctrl	Suspendre l'accroche
D6	Dessin (clics)	Retour arrière	Retirer le dernier point
D7	Dessin (clics)	Échap	Annuler l'outil en cours
N1	Édition de nœuds	Glisser	Déplacer le nœud
N2	Édition de nœuds	Maj + glisser	Verrouiller l'axe
N2b	Édition de nœuds	Ctrl + glisser	Suspendre l'accroche
N4	Édition de nœuds	Suppr	Supprimer les nœuds sélectionnés

Précisions sur les gestes du tableau :

- **Surbrillance** : avec l'outil Sélection, le contour en pointillés de l'objet non sélectionné situé sous le curseur est mis en évidence ; elle disparaît quand le curseur quitte le canevas ou pendant un glisser.
- **Alt** : *Alt + clic* ouvre « Sélectionner dessous », y compris sur le corps d'un objet déjà sélectionné ; *Alt tenu avant l'appui*, puis glisser le corps d'un objet **déjà sélectionné**, le **duplique** en déplaçant la copie (un seul pas d'annulation) ; la copie ne reprend **aucun objet de broderie** de l'original (il faut lui en créer un), de même que la commande « Dupliquer ». En dessin de rectangle, ellipse ou polygone régulier, *Alt* dessine le cadre depuis son centre (le point d'appui).
- **Maj** pendant un glisser verrouille l'axe dominant (objets et nœuds) ; il ne suspend **pas** l'accroche. **Ctrl** suspend l'accroche pendant le tracé d'un polygone ou d'une colonne satin et pendant le glisser d'un nœud.
- **Glisser un nœud** : le nœud ne bouge qu'après le seuil de glisser (un clic légèrement tremblé ne crée ni déplacement ni pas d'annulation) ; la barre d'état affiche ses coordonnées en mm et un cercle marque le sommet visé quand l'accroche est active ; **Échap** annule le glisser sans rien modifier.
- **Redimensionner** : les quatre poignées carrées (zone de 22 px, curseur diagonal propre à chaque coin) redimensionnent autour du coin opposé ; **Maj** conserve les proportions, un cadre en pointillés et « Taille : L × H mm » (barre d'état) montrent le résultat, **Échap** annule. La forme ne peut ni être retournée par inadvertance (miroir) ni tomber sous 5 % de sa taille.
- **Accrochage des nœuds au glisser** (Affichage ■ *Accrochage des nœuds au glisser*, réglage `edit/snapNodesOnDrag`, **désactivé par défaut**) : un nœud relâché à moins de 1 mm (et 10 px) d'un sommet d'un autre objet s'y accroche ; Ctrl pendant le glisser l'évite. La bascule **Accroche** de la barre d'état reflète et pilote ce réglage. Glisser un objet entier n'a pas d'accroche.

Menu Fichier

Action	Raccourci	Effet
Nouveau projet	Ctrl+N	Repart d'un document vierge (voir ci-dessous)
Ouvrir une image…	Ctrl+O	Charge PNG/JPEG/BMP/TIFF ou SVG puis demande la taille physique ; propose d'enregistrer le projet en cours
Enregistrer le projet	Ctrl+S	Réécrit le `.osp` courant (demande où enregistrer la première fois)
Enregistrer le projet sous…	Ctrl+Maj+S	Écrit le document dans un nouveau `.osp`, qui devient le fichier courant
Ouvrir un projet…	—	Recharge un `.osp`
Récents	—	Sous-menu des 10 derniers projets ouverts ou enregistrés (voir ci-dessous)
Vider la liste des récents	—	Efface la liste
Exporter une broderie machine…	Ctrl+E	Dialogue : format (DST, PES, JEF, EXP), options (coupes, arrêts, changements de couleur) et résumé pré-export (dimensions, points, analyse, limites du format) ; puis le fichier, avec l'extension du format (points uniquement)
Importer une broderie machine…	—	Relit un `.dst`, `.pes`, `.jef` ou `.exp` comme séquence de points, avec les couleurs de fil pour PES et JEF (propose d'enregistrer le projet en cours) ; le glisser-déposer accepte les mêmes extensions
Fiche de production…	—	Aperçu, impression ou export PDF A4 d'une fiche (dimensions, points, temps estimé, blocs de couleur, avertissements, notes) ; voir fiche de production
Importer un DXF… / Exporter en DXF…	—	Échange de contours vectoriels avec un logiciel de dessin (les points ne sont pas concernés)
Quitter	Ctrl+Q	Ferme l'application

Nouveau projet : si le document courant a été modifié, une garde propose d'enregistrer, d'abandonner les modifications ou d'annuler — la même que celle de la fermeture de la fenêtre. Le document repart à vide (aucune image, aucun objet, aucune région), la pile Annuler/Rétablir est vidée, et **tout l'état d'édition de la fenêtre est relâché** : sélections, mode d'édition des points, modes satin (barreaux et rails), guides de direction, tracé en cours (polygone, main levée, colonne satin, Bézier) et simulation. L'outil revient à Sélection. Cette réinitialisation est partagée par tous les chemins qui remplacent le document (ouvrir une image, un SVG, un projet, importer un DST).

Import : le dialogue affiche un aperçu, les dimensions en pixels, la résolution (**mm/pixel et dpi**) en direct, la taille du cadre, et **alerte si l'image dépasse le cadre**. La taille proposée d'emblée **tient dans le cadre** (96 dpi, ramenée au cadre si besoin) et **Ajuster au cadre** y revient en un clic. Avec « Conserver les proportions », les bornes des champs empêchent de dépasser la plage 1–1000 mm sans respecter le ratio ; décochée, une alerte indique de combien l'image est étirée. Le dossier du dernier import est mémorisé. **Export DST** : un **résumé** (dimensions, points, sauts, coupes, changements de couleur, fil estimé, cadre, dépassement éventuel) est présenté avant écriture, avec un rappel que le DST ne conserve pas les objets.

Enregistrer : le document retient le fichier `.osp` auquel il est rattaché (celui qu'on vient d'ouvrir ou d'enregistrer). Ctrl+S réécrit ce fichier directement ; le sélecteur n'apparaît que pour un document encore sans fichier, ou via **Enregistrer sous…** (Ctrl+Maj+S), qui adopte ensuite le nouveau fichier. Tout changement de document (Nouveau, ouvrir une image/un SVG/un projet, importer un DST) oublie ce rattachement, pour qu'un Ctrl+S ne puisse jamais écraser le projet précédent. L'écriture est **atomique** (fichier temporaire puis renommage) : une coupure en cours d'enregistrement ne détruit pas le `.osp` existant.

Le titre de la fenêtre affiche le nom du fichier courant (ou « Sans titre ») et un indicateur **modifié** (`*`) ; quitter avec des modifications non enregistrées propose **Enregistrer / Ignorer / Annuler**.

Projets récents

Fichier ■ Récents liste les derniers projets ouverts ou enregistrés (10 au plus, le plus récent en tête) et l'écran d'accueil les reprend. Un fichier qui n'existe plus est retiré de la liste à l'ouverture suivante. La liste est une préférence de l'application, pas du projet : elle est stockée avec les autres préférences (voir Installation, « Où l'application range ses données »).

Sauvegarde automatique et récupération

Toutes les **2 minutes**, si le document a été modifié et n'est pas vide, l'application écrit un instantané dans son dossier de données (le message « Sauvegarde automatique à HH:MM » apparaît brièvement dans la barre d'état). Cet instantané **ne remplace jamais votre `.osp`** : c'est un fichier à part, dans `%APPDATA%\OpenStitch\OpenStitch Studio\autosave\`. Une fermeture normale le supprime. Si l'application s'arrête anormalement, le démarrage suivant affiche **Récupération après un arrêt anormal** : **Récupérer** ouvre l'instantané comme un document modifié, non rattaché à un fichier (pensez à *Enregistrer sous…*) ; **Ignorer** le supprime. Un projet jamais enregistré est proposé comme « projet sans nom ».

Menu Édition

Action	Raccourci	Effet
Annuler	Ctrl+Z	Défait la dernière opération (le libellé nomme l'action)
Rétablir	Ctrl+Maj+Z (Ctrl+Y sous Windows)	Refait l'opération annulée
Supprimer la sélection	Suppr	Supprime la région, l'objet de broderie ou les objets vectoriels sélectionnés
Dupliquer la forme	—	Duplique la forme sélectionnée
Décaler la forme…	—	Décale ou rétrécit la forme sélectionnée
Préférences — Intelligence artificielle…	—	Configure la segmentation par IA (voir *Menu Segmentation*)
Aligner la sélection	—	À gauche, centrés, à droite, en haut, centrés verticalement, en bas : range les formes sélectionnées (au moins deux) sur la boîte de la sélection, en un seul pas d'annulation

L'annulation couvre les opérations d'image, la segmentation (segmenter, fusionner, supprimer, recolorer), la création et le déplacement de nœuds vectoriels, la création d'objets de broderie, le **changement de type**, l'**orientation** d'un remplissage, la conversion satin→tatami et le réordonnancement.

Les libellés d'historique sont des verbes à l'infinitif (« Déplacer la forme », « Modifier : Espacement des rangées »). Une **rafale** de modifications du même champ du même objet (molette d'un champ, flèches du clavier, angle d'un guide) dans une fenêtre de 0,6 s ne forme qu'**un** pas d'annulation. Le panneau **Historique** (Affichage ■ Panneaux) liste les pas : un clic revient à cet état, les pas annulés (en italique) restent rétablissables.

Menu Image

Toutes ces opérations sont non destructives (pile de transformations sur l'original) :

- Niveaux de gris ;
- Luminosité/contraste… (aperçu en direct) ;
- Débruitage léger / moyen (médian) ;
- Quantifier les couleurs… (k-means, nombre de couleurs 2–64, **aperçu en direct** sur le canevas, dernier choix mémorisé) ;
- Symétrie horizontale / verticale ;
- Rotation 90° horaire / antihoraire ;
- Recadrer l'image (glisser un cadre) — voir l'outil `M` ci-dessus.

Menu Segmentation

- Segmenter l'image… (**F6** ; nombre de couleurs, taille min de région avec son équivalent en mm², lissage — les derniers réglages sont mémorisés) ;
- Segmenter avec l'IA… (régions proposées par un modèle ; grisée tant qu'aucune image n'est ouverte) ;
- Afficher la carte des régions (bascule) et **Opacité de la carte** (glissière 20–100 %, mémorisée) : baissez-la pour juger les régions d'après la photo dessous ;
- **Fusionner la sélection** (Ctrl+M) — fusionne toutes les régions sélectionnées dans la dernière cliquée (la **région active**, qui garde sa couleur), en un seul pas d'annulation ;
- **Fusionner dans la voisine principale** (Ctrl+Maj+M) — la région rejoint la voisine avec laquelle elle partage la plus longue frontière ;
- Fusionner avec… (puis clic sur la région cible) — absorbe toute la sélection dans la région cliquée. Le mode s'annonce dans la barre d'état et par un curseur « main » ; un clic sur une région déjà sélectionnée ou dans le vide n'annule pas le mode mais explique quoi cliquer (Échap annule) ;
- **Sélectionner la même couleur**, **Sélectionner les voisines**, **Tout sélectionner** (Ctrl+A) ;
- **Recolorer la sélection…** — un sélecteur de couleur, appliqué à toutes les régions sélectionnées (un pas d'annulation) ;
- **Rétablir la couleur d'origine** — rend à chaque région sa couleur moyenne dans l'image ;
- Supprimer : voir le menu Édition (Suppr) — les régions supprimées redeviennent du fond ;
- **Vectoriser la sélection** (**F7**) — convertit **toutes** les régions sélectionnées en objets vectoriels, en un seul pas d'annulation (le niveau de détail demandé s'applique à toutes et est mémorisé). Une région déjà vectorisée n'est jamais dupliquée : la boîte propose de **remplacer** son objet, de l'**ignorer** (ou de sélectionner l'objet existant) ou d'annuler.

Fusionner ou supprimer des régions **déjà vectorisées** demande quoi faire de leurs objets (les conserver — ils ne sont pas mis à jour —, les supprimer avec les objets de broderie qui en dépendent, ou annuler), en un seul pas d'annulation. La barre d'état permanente affiche le **nombre de régions** de la segmentation (trop de petites régions : fusionnez-les dans leur voisine principale).

Survol : sur la carte des régions, la région sous le curseur est surlignée avant le clic (sauf si elle est déjà sélectionnée).

Sélection multiple de régions (carte des régions affichée) : clic = une région ; **Ctrl + clic** ajoute ou retire une région ; **Maj + clic** en ajoute une ; un **cadre** tracé sur la carte (objets vectoriels masqués) sélectionne les régions qu'il touche — vers la gauche — ou qu'il contient entièrement — vers la droite ; Ctrl/Maj + cadre ajoute ou bascule. Objets vectoriels affichés : un cadre qui ne touche aucun objet sélectionne les régions ; un cadre qui en touche saisit les objets, et un message rappelle de les masquer (menu Affichage) pour viser les régions. La liste *Régions* du panneau Document accepte aussi Ctrl/Maj + clic. L'**inspecteur** montre alors le nombre de régions, l'aire totale, la pastille de couleur de la région active (un clic ouvre le sélecteur) et les boutons des actions ci-dessus. Le **clic droit** sur une région ouvre un menu : fusionner la sélection ou « Fusionner dans… » (voisines classées de la plus proche à la plus lointaine, avec leur couleur), recolorer, sélectionner, vectoriser, supprimer.

Sélection : cliquez une région ; ses statistiques (pixels, mm², RGB) s'affichent. La dernière région cliquée est la « région active » ; fusionner, recolorer, supprimer et vectoriser s'appliquent à toute la sélection.

Segmentation par IA

La fonction est **facultative et n'est pas livrée avec les binaires de release** : elle s'appuie sur un petit programme Python séparé (le « worker », dossier `sam-worker/` du dépôt) que l'application lance et interroge. Prérequis, à préparer vous-même :

- un **environnement Python** pour le worker, avec `torch` et `sam2` installés (CPU ou CUDA) en plus de `sam-worker/requirements.txt` ;
- les **fichiers de modèle** SAM 2.1 (par exemple `sam2.1_hiera_small.pt`) placés dans le dossier des modèles ;
- soit une distribution **WSL** (Ubuntu par défaut), soit un Python natif.

Dans **Édition ■ Préférences — Intelligence artificielle…** : case *Activer la segmentation par IA*, environnement (WSL ou Python natif), distribution WSL, Python du venv, script du worker, dossier des modèles, modèle par défaut (quatre tailles de SAM 2.1, de « Tiny (rapide) » à « Large »), processeur (automatique/CPU/GPU CUDA), résolution maximale d'analyse, conservation des fichiers de diagnostic et niveau de journalisation du worker. Le bouton **Tester la configuration** démarre le worker et affiche le résultat ou l'erreur exacte dans le journal du dialogue. Lancée depuis un dépôt cloné, l'application préremplit les chemins avec `sam-worker/` ; un build installé sans dépôt demande de les saisir.

Segmenter avec l'IA : le dialogue détecte des formes (pas des couleurs) avec SAM 2. Choisissez le modèle et le profil, **Analyser** (« Annuler l'analyse » fonctionne aussi pendant le démarrage du worker), cochez les masques à garder, **protégez** ceux que le nettoyage ne doit pas absorber (les infobulles des en-têtes *IoU*, *Stabilité* et *Protéger* expliquent les colonnes ; un clic sur un en-tête trie le tableau), puis **Valider** (bouton grisé pendant le calcul). Relancer l'analyse ou fermer avec des masques en cours de revue demande confirmation ; les réglages sont mémorisés. « **Après validation, créer** » choisit le résultat : des **régions éditables** (la segmentation du document, à fusionner, recolorer puis vectoriser comme après « Segmenter l'image… »), ou directement les **objets de broderie** (avec les options *ignorer le fond* et *détail de vectorisation*). Une erreur de configuration s'affiche en rouge avec **Ouvrir les préférences…** et **Afficher le détail** ; l'analyse relancée utilise alors les nouveaux réglages.

Menu Forme

Opérations sur les formes vectorielles, façon « Pathfinder » : chacune est **un seul pas d'annulation** et garde les objets de broderie de la forme conservée.

- **Fondre deux zones** (Ctrl+Maj+F) — unit deux formes remplies de couleurs différentes et crée un dégradé de fils entre elles (voir *Remplissage directionnel*, « Fondre deux zones voisines ») ;
- **Unir** (Ctrl+Maj+U) — fusionne au moins deux formes sélectionnées (Maj + clic) en une seule ; la dernière sélectionnée garde son identité, ses réglages de point et son nom. Les autres formes (et leurs broderies) disparaissent.

- **Soustraire** (Ctrl+Alt+S) — retire de la forme la plus **basse** du document toutes les autres formes sélectionnées, qui sont supprimées. Un résultat vide est refusé.
- **Intersecter** (Ctrl+Alt+I) — ne garde que la partie commune à toutes les formes.
- **Séparer les morceaux** (Ctrl+Maj+B) — une forme composée de plusieurs morceaux disjoints devient autant d'objets.
- **Couteau** (touche **K**) — cliquez-glissez une ligne à travers une ou plusieurs formes pour les découper. Sans sélection, toutes les formes visibles traversées sont coupées ; avec une sélection, seules les formes sélectionnées le sont. Le plus grand morceau garde l'identité de la forme d'origine ; chaque autre morceau devient un nouvel objet, avec une **copie des réglages de broderie** (tatami, directionnel, auto-satin ; les satins à rails et les retouches manuelles point par point ne sont pas copiés). La coupe retire une bande de 0,02 mm, sans effet sur la couture.

Menu Texte (lettrage)

Voir Lettrage pour le fonctionnement détaillé.

- **Outil Texte** (touche **T**) — cliquez sur le canevas : le dialogue s'ouvre, le texte est posé à cet endroit (début de la ligne de base, ou centre / bord droit selon l'alignement).
- **Nouveau texte…** — idem, au centre du cadre.
- **Modifier le texte…** (**F2**, double-clic sur une lettre, ou bouton de l'inspecteur) — rouvre le dialogue ; les lettres sont régénérées (un seul pas d'annulation). Un déplacement du texte entier est conservé ; les retouches faites à la main sur une lettre sont remplacées.
- **Supprimer le texte** — retire toutes les lettres.

Dans le dialogue : police (deux polices intégrées, puis polices installées), hauteur de capitale, couleur, type de point (**Automatique** : satin pour les traits fins, tatami pour les gros ; **Satin**, **Tatami**, **Contour**), alignement, espaces entre lettres et mots, interligne, crénage, position, rotation. L'aperçu montre les contours ; les avertissements (texte sous 5 mm, trait trop fin, lettre cousue en tatami faute de satin possible, caractère absent de la police) s'affichent sous l'aperçu et dans le panneau Analyse.

Type de points pour plusieurs formes

La sélection multiple de formes (Maj + clic, Ctrl + clic, rectangle) accepte **tous les types de points d'un coup**, exactement comme pour une forme seule :

- **Inspecteur** : avec plusieurs formes sélectionnées, le bloc « Type de points » propose *Contour cousu*, *Tatami*, *Satin automatique* et *Remplissage directionnel* ; l'espacement et l'angle cochés s'appliquent aux nouveaux réglages. **Appliquer à N objets** valide.
- **Menu Broderie** : « Créer un objet de point de contour… », « Créer un remplissage tatami… » et « Créer un satin automatique… » s'appliquent à toute la sélection (un seul dialogue de réglages).
- **Clic droit** sur une forme de la sélection : « Type de points (tous) ».

Réglages communs. Quand toutes les coutures des formes sélectionnées ont le **même type** (par exemple plusieurs tatamis, plusieurs satins automatiques), l'inspecteur affiche sous le bloc « type de points » le **même formulaire que pour une forme seule**. Chaque champ modifié est appliqué à **toutes** les formes, **champ par champ** : les autres réglages de chaque forme (angle, retrait, guides…) ne sont pas touchés. Une modification est un seul pas d'annulation, nommé d'après le champ.

Guides en lot. - *Satin automatique* : choisissez un angle (relatif à l'axe de chaque forme, ou absolu dans le dessin) puis **Poser ce guide sur toutes les formes** (un guide ancré au centre de chaque forme, qui remplace les guides existants) ; **Retirer les guides de toutes** les supprime. - *Remplissage directionnel* : **Guides automatiques pour tous** calcule le guide de chaque forme d'après son axe ; **Retirer guides et ruptures de tous** les efface. Le tracé fin d'un guide reste propre à chaque forme (sélectionnez-la seule).

Une forme déjà cousue est **convertie** (toutes ses sections), une forme sans couture en reçoit une nouvelle. Les formes qui ne peuvent pas être cousues en satin sont ignorées et listées (en satin automatique, on peut

leur donner un tatami à la place). Tout le geste est **un seul pas d'annulation**.

Le calcul des points se fait **en parallèle sur plusieurs coeurs** (contours, tatamis, remplissages directionnels et squelettes de satin) ; le résultat est identique quel que soit le nombre de coeurs. La variable d'environnement `OPENSTITCH_THREADS=1` force le calcul séquentiel (mesures, diagnostic).

Menu Broderie

- **Numérisation automatique** (**F8**, grisée tant qu'il n'y a ni segmentation ni objets vectoriels) — crée des objets pour toutes les régions. Les zones remplissables deviennent des **tatamis** ; le satin se crée ensuite, région par région, par « Créer un satin automatique ». Le dialogue propose :
- **Ignorer la plus grande région (probablement le fond)** : cochée d'office seulement pour un fond quasi blanc qui encadre le motif (couleur, part de l'image et bords touchés sont affichés) ;
- **Formes pleines** (remplissages ; curseur « Détail vectorisation ») ou **Contours (dessin au trait / Line Art)** : coud les **lignes médianes** des traits en point droit au lieu de remplir les formes ; curseur de détail (bas = lignes très simplifiées, petits traits ignorés) et technique *Automatique* ou *Running (point droit)*. La barre d'état résume (objets, segments, jonctions, longueur de point droit, replis, rejets) et un dialogue liste les traits qui n'ont pas pu être cousus tels quels ;
- **Créer un objet de point de contour…** (longueur, type simple/double/triple) ;
- **Créer un remplissage tatami…** (espacement, longueur, angle) ;
- **Créer un satin automatique…** — satin par squelette et traversées orientées : aucun rail à poser. Une **fenêtre de réglages** s'ouvre (espacement, compensation de tirage, sous-couche centrale, fractionnement des traversées longues) avec l'avertissement « non validé sur machine » ; l'aperçu annonce le nombre de colonnes et la couverture estimée, et **refuse avec une raison** les formes qui ne s'y prêtent pas (disque, forme compacte) en proposant un tatami. Le contour utilisé est celui de la région segmentée. **Non validé sur machine** : vérifiez le résultat avant de broder ;
- **Convertir automatiquement en satin (expérimental)…** — **même moteur, sans fenêtre de réglages** : réglages par défaut et une seule confirmation (« Créer le satin ? annulable »). *Choisissez « Créer un satin automatique » pour régler, « Convertir automatiquement » pour aller vite* ;
- **Orientation du remplissage…** — change l'angle des fils du tatami sélectionné (aussi réglable à la souris, voir *poignée de rotation* plus bas) ;
- **Convertir les satins auto en tatami** — répare un projet dont les satins automatiques débordent ;
- **Options de génération…** — finitions du projet : voir *Coupes et finitions DST* ci-dessous ;
- **Éditer les points…** (`E`) — déplace un à un les points cousus de l'objet sélectionné (la forme source ne bouge pas) ;
- **Modifier la colonne satin (rails + guides)…** (`Maj+E`) — nœuds des deux rails et guides transversaux d'une colonne satin **à rails** ; le sous-menu **Remodelage satin (avancé)** n'affiche que les guides ou que les rails, avec **Ajouter un guide satin** (`Maj+G`) et **Supprimer le guide satin sélectionné**. Cette voie à rails manuels est **en partie archivée** (voir Colonne satin) ;
- **Guides de direction…** (`D`) / **Générer un guide depuis la forme** / **Tracer un guide de direction** / **Tracer une ligne de rupture** — orientent les fils d'un remplissage directionnel (voir Remplissage directionnel) ; pour un satin automatique, le bouton « Placer un guide sur le canevas… » de l'inspecteur pose un guide d'orientation ;
- Les **Statistiques…** (points, sauts, coupes, changements de couleur, dimensions, longueur de fil) sont dans le menu Analyse.

Avertissement : si une colonne satin dépasse la largeur recommandée, un dialogue propose de continuer ou de préférer un remplissage tatami.

Coupes et finitions DST

Broderie ■ Options de génération… règle, pour tout le projet (annulable) :

- **Finitions automatiques** (case générale : désactivée, la séquence reste telle quelle ; activée par défaut pour un nouveau projet) ;
- **Couper au-delà de** (seuil, 3 mm par défaut) : un déplacement plus long devient *point d'arrêt, coupe, déplacement, point d'arrêt* ; plus court, un simple saut ;
- **Couper avant chaque changement de fil** ;
- **Point d'arrêt** : aucun, aller-retour, triangle ou micro-zigzag, avec sa longueur et son nombre de répétitions ;
- **Fusionner les points trop courts** et la longueur minimale de point ;
- **Découper les points trop longs** et la longueur maximale de point (7 mm) ;
- **Entrée/sortie automatiques des objets** : chaque objet est cousu dans le sens qui rapproche son début de la fin du précédent (moins de sauts). Chaque objet peut suivre ou refuser ce réglage (« Entrée/sortie » dans l'inspecteur).

Réglages de moteur par objet (inspecteur)

- **Compensation du tirage** (tatami) : les rangées dépassent du contour de la valeur choisie (0 à 3 mm ; 0,2 à 0,4 mm est courant) pour compenser le rétrécissement dans l'axe du fil.
- **Sous-couche : Automatique** (tatami, directionnel, satin, auto-satin) : le moteur choisit la sous-couche selon la taille et la largeur de la forme ; les cases manuelles sont alors grisées.
- **Satin de bordure** : sélectionnez une forme, réglez *Satin de bordure* (largeur, côté centré/intérieur/extérieur, coins vifs/arrondis) puis **Créer le satin de bordure** ; un satin par contour et par trou. Changer les réglages du satin créé le régénère depuis le contour.
- Ces réglages ne sont **pas validés sur machine réelle**.

La séquence se termine toujours par une **coupe finale**, et à l'export vers une machine chaque coupe est écrite en **trois sauts de 0,1 mm non nuls** suivis du déplacement réel (certaines machines ignorent les sauts de déplacement nul : voir Format DST). **À tester sur votre machine** : ces conventions n'ont pas été validées sur une machine réelle ; si la machine ne coupe pas, essayez un seuil plus bas ou signalez le modèle.

Menu Affichage

Le sous-menu **Calques** regroupe des interrupteurs indépendants :

Calque	Effet
Image	Affiche/masque l'image de travail
Carte des régions	Affiche/masque la segmentation
Vecteurs	Affiche/masque les objets vectoriels
Broderie (points)	Affiche/masque les points générés

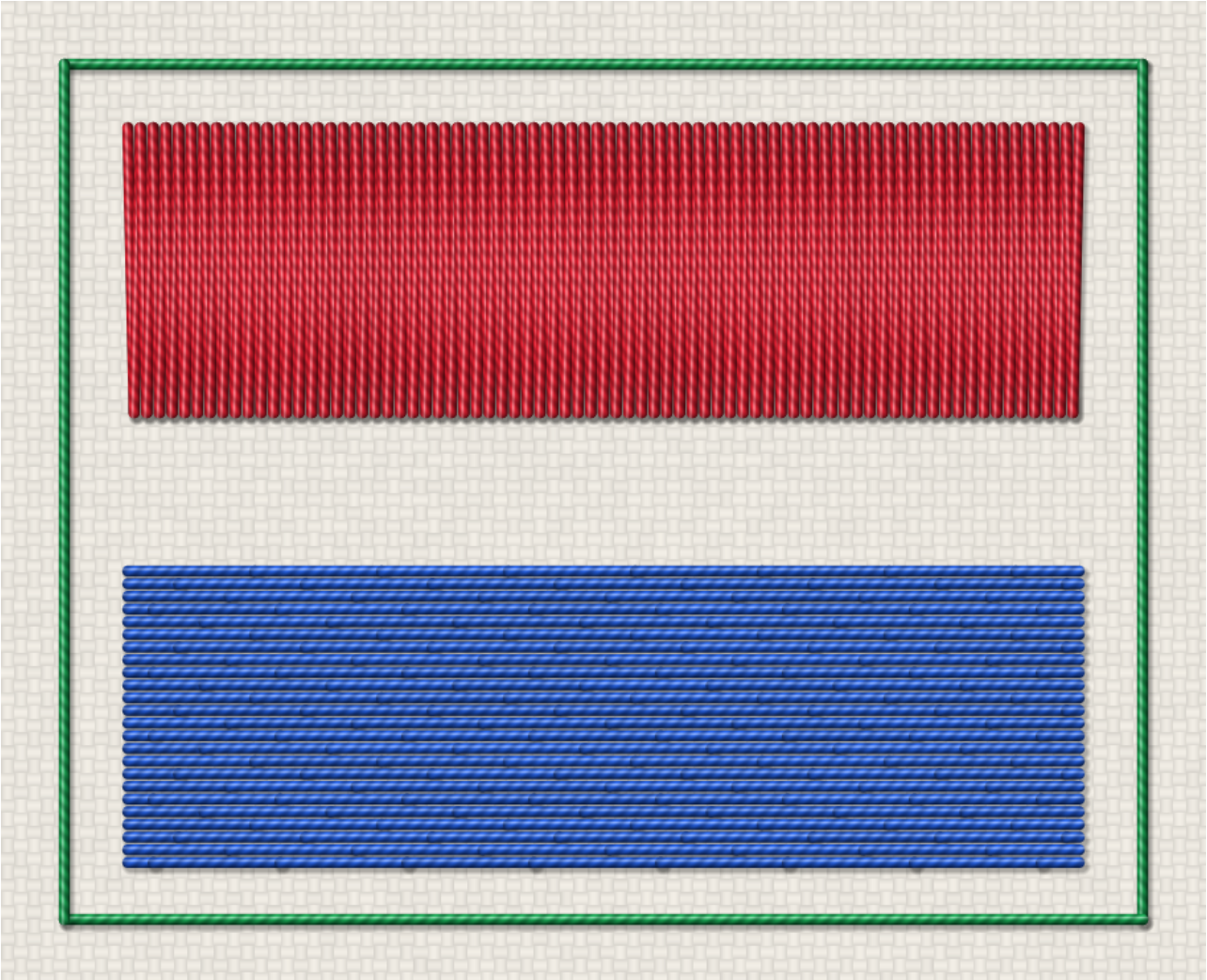
Action	Raccourci	Effet
Zoom avant	Ctrl++	Agrandit
Zoom arrière	Ctrl+-	Réduit
Ajuster au canevas	Ctrl+0 ou F	Cadre la vue sur le canevas
Rendu réaliste ■ Activer	Ctrl+Maj+R	Dessine chaque point comme un fil texturé (voir ci-dessous)
Rendu réaliste ■ Réglages…	—	Épaisseur du fil, relief, brillance, torsion, ombre, tissu, qualité
Taille du cadre…	—	Définit la zone physique de broderie (voir ci-dessous)
Thème	—	Clair / Sombre
Densité	—	Confortable / Compact
Navigation	—	Préréglage souris : **OpenStitch** (souris à trois boutons) ou **Pavé tactile** (Espace + glisser, Ctrl + molette) ; mémorisé
Accrochage des nœuds au glisser	—	Voir *Souris et clavier* (désactivé par défaut)
Panneaux	—	Affiche/masque chaque dock et **Réinitialiser la disposition**
Masquer les panneaux	Ctrl+Shift+P	Mode canevas (masque puis restaure les docks)

Cadre de broderie : **modèle** courant (liste de cadres usuels, du 50×50 au 300×400 mm, plus vos propres cadres), largeur/hauteur en mm (10–500), bouton **Pivoter** (échange largeur/hauteur) et **marge de sécurité**. La **zone brodable** est le cadre réduit de cette marge sur chaque côté : elle est affichée sur le canevas (la bande de marge est hachurée, le contour plein délimite la zone brodable) et c'est elle que l'analyse « hors cadre » et la fiche de production contrôlent. **Enregistrer ce cadre…** garde une taille sous un nom (mémorisée entre les sessions, **Supprimer** la retire) ; le dernier cadre choisi (taille et marge) est repris par les **nouveaux projets**. Le cadre est une donnée du projet (persistée dans le `.osp`, annulable ; marge à 0 pour un projet qui n'en avait pas). Accessible aussi via le bouton **« Cadre : W×H mm… »** de la barre contextuelle quand rien n'est sélectionné.

Les points cousus sont tracés **dans la couleur de fil de chaque objet** (un fil très clair est légèrement assombri pour rester visible) ; les **sauts** en pointillés orange ; des pastilles marquent les pénétrations (masquées au-delà de 4000 points pour la fluidité, et pendant la simulation).

Rendu réaliste (façon « TrueView »)

Affichage ■ Rendu réaliste ■ Activer (Ctrl+Maj+R) remplace les lignes par un aperçu proche de la broderie réelle : chaque point est dessiné comme un **fil** d'une épaisseur donnée, à section cylindrique éclairée (relief), avec reflet brillant, torsion visible, extrémités qui plongent dans le tissu et **ombre portée** sur le tissu et sur les points déjà cousus dessous. Les fils sont empilés dans l'ordre de couture réel (un point plus tardif passe par-dessus).



Affichage ■ Rendu réaliste ■ Réglages… ouvre une fenêtre non modale : chaque changement se voit tout de suite sur le canevas et est mémorisé entre deux sessions (comme les autres préférences d'affichage).

Le rendu est calculé **une seule fois** puis mis en cache : il n'est recalculé que lorsque les points, les réglages, le zoom ou la zone visible changent (avec un court délai après un zoom ou un défilement), jamais à chaque mouvement de souris. Il est limité à la zone visible, donc reste fluide sur un motif de plus de 50 000 points. Deux cas retombent volontairement sur l'affichage en **lignes colorées** : un **dézoom fort** (quand le fil fait moins de ~1,6 pixel d'épaisseur, l'ombrage n'est plus lisible) et la **simulation** de couture. Les objets masqués par les filtres d'affichage ne sont pas dessinés. Le tissu est peint autour du motif (zone englobante), pas sur tout le cadre.

Menu contextuel (clic droit)

Un **clic droit** sur une forme ouvre un menu :

- **Type de points** ■ Contour cousu / Remplissage tatami / Colonne satin (le type courant est coché) — bascule instantanée et annulable ;
- **Orientation du remplissage…** (si la forme est un tatami) ;
- **Calques** — les mêmes interrupteurs que le menu Affichage.

Poignée de rotation (orientation à la souris)

Quand un remplissage tatami est sélectionné, un **axe bleu avec une poignée** apparaît en son centre. Faites glisser la poignée pour tourner l'orientation des fils ; l'angle est appliqué au relâchement (annulable). C'est l'équivalent visuel de *Orientation du remplissage…*.

Panneau Filtres d'affichage

Dock (à droite, dès qu'il existe des objets) pour **isoler** ce qu'on regarde :

- **Types de points** : cases Contour / Tatami / Satin ;
- **Taille min. des zones** : masque les zones dont l'aire source est sous le seuil (mm²) — utile pour cacher les micro-régions ;
- **Couleurs de fil** : une case par couleur distincte, pour n'afficher que certains fils.

Ces filtres n'affectent que **l'affichage** (pas l'export ni les points générés). Ils s'appliquent aussi aux **formes vectorielles** : une forme est masquée si elle est plus petite que la taille minimale, ou si toutes ses broderies sont d'un type ou d'une couleur masqués (une forme sans broderie n'est filtrée que par sa taille). Une forme **sélectionnée** reste visible ; une forme masquée n'est ni survolée ni cliquable.

Barre d'outils principale

Actions fréquentes, icônes monochromes avec infobulle (qui rappelle le raccourci) : ouvrir image/projet, enregistrer, annuler/rétablir, zoom –/ajuster/+, analyser, aperçu des points, exporter DST. Ce sont les mêmes actions que dans les menus : une action indisponible est grisée aux deux endroits, et son infobulle (et la barre d'état) dit quoi faire pour l'activer.

Barre d'outils contextuelle

Sous la barre principale, son contenu **suit la sélection** :

- **objet de broderie** : bascule rapide du type (Contour/Tatami/Satin) + *Orientation…* si tatami ;
- **objet vectoriel** : boutons de création rapide (Contour/Tatami/Satin) ;
- **région** : aire + *Fusionner* / *Supprimer* / *Vectoriser la sélection* (les mêmes actions que les menus, avec leurs raccourcis) ;
- **aucune sélection** : résumé du motif + bouton *Cadre…*.

Inspecteur de propriétés

Dock (droite) affichant l'élément sélectionné. Pour un **objet de broderie**, il expose ses **paramètres de couture éditables après création** (contour : longueur/min/passages ; tatami : espacement/longueur/angle/retrait/décalage ; satin : espacement/compensation/sous-couche). Chaque changement passe par une commande **annulable** et régénère les points.

- **Un champ = une modification** : seul le champ touché change dans le document (les autres valeurs ne sont ni relues ni arrondies ; l'angle se règle au dixième de degré). Après une annulation ou un changement de type de points, le formulaire est reconstruit d'après le document. L'historique nomme le champ modifié.
- **Bornes** : espacement des rangées ≥ 0,1 mm ; longueurs de point ≥ 0,5 mm (contour) ou 1 mm (tatami, directionnel, satin) ; l'infobulle de chaque champ donne sa plage.
- **Molette** : un champ ne réagit à la molette que s'il a le focus (clic ou Tab) ; sinon la molette fait défiler l'inspecteur.
- **Champs dépendants** : retrait et espacement de sous-couche grisés si la case est décochée ; en auto-satin, « Longueur max des segments (y) » est bornée par Lmax et les deux sont grisées si le fractionnement est désactivé. Satin manuel et auto-satin emploient les mêmes termes : *Forme du bout* (début/fin) et *Point d'arrêt* (début/fin).
- **Guides d'orientation** (auto-satin) : l'angle est en degrés ; relatif = écart à la perpendiculaire de l'axe (0° = perpendiculaire), absolu = depuis l'horizontale du dessin, sens trigonométrique. Cliquer un guide dans la liste entoure son ancre sur le canevas.
- **Objet vectoriel** : *X*, *Y* (coin bas-gauche, Y vers le haut comme l'indicateur de curseur), *Largeur* et *Hauteur* en mm, avec « Conserver les proportions » ; le déplacement/redimensionnement est un seul pas d'annulation.
- **Plusieurs objets sélectionnés** : le bloc « Appliquer à N objets » règle le type de points (contour cousu ou tatami), l'espacement des rangées et l'angle (tatami) des coutures de tous les objets, en un seul pas d'annulation. Directionnel et satin se règlent objet par objet.

Une région affiche ses informations en lecture seule.

Panneau Document

Dock (gauche) à deux onglets, **Objets** et **Régions**, tabifié avec l'**Ordre de couture**. Sélectionner une ligne met l'élément en évidence au canevas et dans l'inspecteur ; une sélection au canevas surligne la ligne correspondante (synchronisation bidirectionnelle).

Chaque ligne d'objet commence par son **rang de couture** (« 3. Satin — … »). Deux cases : **Vis.** (visible : décocher masque l'objet, qui n'est plus ni dessiné ni cousu) et **Figé** (ordre figé, voir ci-dessous) ; un **double-clic** sur le nom le renomme ; **Suppr** supprime l'objet sélectionné ; un clic sur le nœud d'un **groupe de sections** (même forme) sélectionne toute la forme. Toutes ces actions sont des pas d'annulation.

Indicateur de workflow

Bandeau (sous Document) listant les étapes **Image** → **Régions** → **Vecteurs** → **Broderie** → **Vérification** → **Export**. L'état de chaque étape (à faire / disponible / en cours / terminé / attention) est déduit du document et rendu par pastille + libellé + mot d'état. Un clic **lance l'action de l'étape** quand elle est disponible (ouvrir une image, segmenter, vectoriser la sélection, numérisation automatique, analyser, exporter) ; sinon la barre d'état en donne la raison et le chemin de menu exact.

Menu Analyse et panneau

Analyser le motif (F5) remplit le panneau *Analyse* (dock) avec les problèmes détectés, triés par gravité.

- Chaque ligne commence par la **gravité en toutes lettres** (Erreur, Avertissement, Information), le **nom de l'objet fautif** entre guillemets, puis le message avec les longueurs en millimètres à la française (« 3,5 mm »).

- En tête du panneau : les **compteurs par gravité** et un **filtre** (Tout, Erreurs, Avertissements, Informations).
- Un **clic** ou **Entrée** sur un problème sélectionne l'objet concerné et centre la vue sur lui (y compris quand le problème se trouve à l'origine du canevas). Le clic droit propose « Sélectionner l'objet » et « Centrer la vue sur le problème ».
- Sous la liste, une **piste de correction** accompagne le problème sélectionné.
- Chaque catégorie est plafonnée à 50 problèmes ; une ligne « … et N autre(s) problème(s) » dit combien n'ont pas été listés.
- Quand le motif change, le résultat est marqué **périmé** puis recalculé après 300 ms si le panneau est visible (sinon à sa prochaine ouverture).

Panneau Ordre de couture

Onglet du panneau Document listant les objets de broderie dans l'ordre, avec les mêmes libellés que le panneau Document (rang, type, nom, masqué, ordre figé). **Monter** (Alt+Haut) et **Descendre** (Alt+Bas) sont grisés aux extrémités. **Figer l'ordre** (bouton à bascule) empêche *Optimiser l'ordre* de déplacer l'objet : ce n'est **pas** un verrou d'édition (l'objet reste déplaçable et modifiable ; l'ancien nom « Verrouiller » prêtait à confusion). Les stratégies : ordre du document, par couleur, par proximité, couleur puis proximité. Le libellé *Trajet estimé* donne la distance à vide entre objets et le nombre de changements de fil.

Panneau Fils et menu Fils

Le panneau **Fils** (menu *Fils* ou *Affichage* ■ *Panneaux*, masqué par défaut) relie le motif aux fils réels. Il a trois onglets ; une ligne en tête rappelle la sélection courante.

- **Projet** — les fils utilisés dans l'ordre de première couture (pastille, marque et référence ou `#RRGGBB` pour une couleur libre, nombre d'objets, de points, longueur de fil et durée estimée en infobulle). *Sélectionner les objets de ce fil* ; *Remplacer ce fil par celui du catalogue* (tout le motif, un seul pas d'annulation) ; *Couleurs libres → fil le plus proche* (associe chaque objet sans fil au fil le plus proche du nuancier choisi, sur la sélection ou sur tout le motif) ; **Limiter à N fils** (fusionne les couleurs les plus proches jusqu'à N, la couleur du plus grand aplat l'emporte) ; *Exporter la liste des fils (CSV)…* (tableur : ordre, marque, nuancier, référence, nom, couleur, objets, points, longueur, durée).
- **Catalogues** — un nuancier à la fois (ou tous), recherche par référence, nom ou gamme. **Un clic sur un fil l'assigne à tous les objets sélectionnés** (sélection multiple comprise ; un seul Ctrl+Z annule l'ensemble). La couleur de l'objet devient celle du fil. *Fils les plus proches de la sélection* classe les 8 fils les plus proches (distance CIEDE2000) de la couleur du premier objet sélectionné. *Importer un nuancier…* charge un fichier CSV ou JSON (format décrit dans *Fils* ■ *Format d'import des nuanciers…*) ; les nuanciers importés sont conservés d'une session à l'autre et peuvent être retirés. Le nuancier **Générique** (couleurs usuelles sans marque) est toujours disponible ; les nuanciers de marques intégrés sont des **données de démonstration fictives**, signalées comme telles : importez vos propres cartes de fils.
- **Film couleur** — les blocs de couleur dans l'ordre de couture. Glissez un bloc (ou *Monter*/*Descendre*) pour réordonner les couleurs ; les objets dont l'ordre est figé ne bougent pas. *Fusionner les blocs de même fil* regroupe les passages d'un même fil pour réduire les changements ; attention, un fil cousu plus tard passe plus tôt (ordre des couches), d'où l'annulation en un pas.

Dans la numérisation automatique, la case **Limiter à N fils** applique la même fusion des couleurs proches avant de créer les objets.

Barre de simulation

Boutons de lecture/pause et un curseur qui révèle la couture jusqu'à un index de point, avec un repère d'aiguille. Une liste **Vitesse** (×0,25, ×1, ×4, ×16) règle l'avance de la lecture ; la **pastille de couleur** et le **nom de l'objet** en cours de couture s'affichent à droite du compteur. Le dessin est incrémental : seul le tronçon nouveau est ajouté à chaque pas, la lecture reste fluide sur un gros motif. Toute modification du document **réinitialise** la simulation (elle ne continue pas sur des points qui n'existent plus). La barre occupe une **zone réservée** (toujours visible, contrôles grisés tant qu'aucune séquence n'existe) pour ne pas faire sauter la mise en page.

Erreurs et messages

Les opérations impossibles (recadrage hors image, satin non constructible, export d'une séquence vide…) affichent un message clair et **n'appliquent rien**. Les erreurs connues et les entrées invalides **couvertes par les tests** renvoient une erreur structurée au lieu de provoquer un arrêt du programme. Aucun plantage n'a été observé dans le corpus de tests actuel — ce qui ne constitue pas une garantie absolue en version 0.1.0.

Où trouver les journaux

L'application n'écrit **aucun fichier de log** : les messages (spdlog) sortent sur la **sortie d'erreur** de la console. Pour les lire, lancez `openstitch.exe` depuis un terminal (`.\openstitch.exe 2> journal.txt` pour les garder). Pour les **préférences** et la **sauvegarde automatique**, voir Installation. Le worker IA a son propre niveau de journalisation (préférences IA) et un journal visible dans le test de configuration.

Menu Aide

Le menu **Aide** compte trois entrées, chacune avec une infobulle et un texte d'aide dans la barre d'état :

- **Guide de prise en main** : une fenêtre **non modale** (elle reste ouverte pendant que vous travaillez, une seule à la fois) qui déroule les six étapes de l'image au fichier DST : ouvrir une image, segmenter, vectoriser ou numériser automatiquement, choisir le type de point (tatami, satin, contour), analyser (F5), exporter en DST. Chaque étape porte un bouton qui lance la vraie commande du menu ; il est grisé tant que la commande n'est pas disponible (par exemple « Analyser » sans motif).
- **Gestes souris et clavier** (**F1**) : fenêtre non modale qui liste, dans un tableau Contexte / Geste / Action filtrable par la recherche, tous les gestes de la souris (table de la section *Souris et clavier*) et tous les raccourcis des commandes. Elle propose aussi le choix du préréglage de navigation ; elle remplace l'ancienne boîte « Raccourcis clavier », supprimée.
- **À propos** : nom, version, licence et dépôt du code source.

La **ligne d'indications** de la barre d'état (à gauche des indicateurs d'outil et de position) rappelle en permanence les gestes de l'outil actif ; elle se met à jour quand on change d'outil, de sélection ou qu'on tient Maj, Ctrl ou Alt, et n'est jamais masquée par les messages de la barre d'état. Le préréglage de navigation se choisit aussi dans **Affichage ■ Navigation** (**OpenStitch** : souris à trois boutons ; **Pavé tactile** : Espace + glisser et Ctrl + molette mis en avant) ; le choix est mémorisé entre deux sessions.

Raccourcis

Tous lus dans `apps/desktop/main_window.cpp` et `main_window_directional.cpp` ; la liste complète et filtrable est dans **Aide ■ Gestes souris et clavier** (F1).



Attention à ne pas confondre : `G` choisit l'outil *Polygone régulier* tandis que `Maj+G` ajoute un guide satin ; `E` édite les **points cousus** tandis que `Maj+E` remodèle une **colonne satin**.

Les raccourcis standard proviennent des séquences Qt (Rétablir suit la convention de la plateforme) ; `Ctrl+0`, `F5`–`F8`, `F`, `V/H/M`, `Ctrl+Shift+P` sont définis explicitement, sans conflit avec la navigation clavier (Tab reste réservé au parcours des contrôles). Les **touches simples** des outils (V H M R O P G B L S E D F) ne sont **pas interceptées quand le focus est dans une liste ou une liste déroulante** (la frappe y sert à la recherche), et Entrée / Retour arrière / Échap restent disponibles pour les champs de saisie. Ctrl+= zoome aussi (Ctrl++ demande Maj sur un clavier AZERTY).

Interface : langue, thème, enregistrement

- Les boutons standard de Qt (Annuler, Oui, Non, « Afficher les détails »…) sont en **français** quand les traductions de Qt sont livrées avec l'application (dossier `translations`) ; sinon l'interface reste utilisable avec les libellés anglais de Qt.
- Au **premier lancement**, le thème suit celui du système (clair ou sombre) ; le choix fait ensuite dans Affichage ■ Thème est mémorisé.
- La barre d'état affiche en permanence « **Enregistré à HH:mm** », « Non enregistré » ou « Nouveau document ». Les messages d'état s'effacent d'eux-mêmes (10 s au plus).
- Le mode **Masquer les panneaux** est levé à la fermeture : le lancement suivant ne démarre jamais sans panneau. La taille de la fenêtre est bornée à l'écran disponible ; une disposition enregistrée illisible est remplacée par la disposition par défaut.
- Les calculs longs (segmentation, vectorisation, numérisation automatique) s'exécutent en arrière-plan derrière la fenêtre « Veuillez patienter », qui reste animée ; Windows ne marque plus la fenêtre « ne répond pas ».

Implémentation associée

- `apps/desktop/main_window.cpp/.hpp` — menus, barres, docks, sélection, cadre.
- `apps/desktop/app_theme.*`, `design_tokens.*` — thème et tokens.
- `apps/desktop/properties_panel.*`, `document_panel.*`, `workflow_panel.*`, `empty_state_widget.*` — panneaux.
- `apps/desktop/tools.hpp`, `ui_icons.*` — modes d'interaction et icônes.
- `apps/desktop/canvas_view.cpp` — zoom, déplacement, grille, cadre, rendu points.
- `apps/desktop/main_window_realistic.cpp`, `realistic_render_dialog.*`, `realistic_preferences.*`, `realistic_view_state.hpp` — rendu réaliste : sous-menu, fenêtre de réglages, préférences QSettings (`view/realistic/*`) et peinture du pixmap en cache. Le calcul (brins de fil, ombrage, tissu) vit dans `libs/stitch_render` (sans Qt, testé par Catch2 ; image RGBA rendue en bandes parallèles, déterministe).
- `apps/desktop/ruler.cpp` — règles en mm.
- `apps/desktop/import_dialog.cpp`, `brightness_dialog.cpp` — dialogues.
- `apps/desktop/generation_options_dialog.cpp` — options de génération (coupes, points d'arrêt).
- `apps/desktop/main_window_satin_auto.cpp` — « Créer un satin automatique » / « Convertir automatiquement ».
- `apps/desktop/autosave.*`, `recent_files.*` — sauvegarde automatique, récents.
- `apps/desktop/ai_preferences*.cpp` — préférences et test de la segmentation par IA.

Ligne de commande (openstitch-cli)

Public : utilisateur avancé, développeur, intégrateur. `openstitch-cli` est le même moteur que l'application, sans interface graphique et **sans Qt** : il sert à numériser une image en lot, à relire un DST, à comparer le résultat à une référence externe et à diagnostiquer le moteur de points. Il est construit avec le reste du dépôt (voir *Installation* ; exécutable `build\msvc\apps\cli\Release\openstitch-cli.exe`) et existe aussi sous Linux.

```
openstitch-cli [--json] [--version] <sous-commande> [options]
openstitch-cli <sous-commande> --help
```

Sous-commandes

Groupe	Sous-commande	Rôle
Commandes	`info`	Métadonnées d'une image (dimensions, canaux, dpi, taille estimée)
Commandes	`stats`	Statistiques et mesures de qualité d'un fichier DST
Commandes	`digitize`	Pipeline complet image → DST (segmentation, numérisation automatique, points)
Commandes	`production`	Fiche de production d'un `.osp` ou d'un DST (texte, JSON, HTML)
Commandes	`dst2svg`	Aperçu SVG d'un DST
Commandes	`osp2dst`	Exporte un projet `.osp` en DST par le même chemin que le bureau
Diagnostic	`osp2svg`	`[diagnostic]` Séquence effective d'un `.osp` en SVG (contours des vecteurs, un seul objet)
Diagnostic	`stitchdebug`	`[diagnostic]` Moteur de points sur une forme de référence
Diagnostic	`engine-debug`	`[diagnostic]` Tirage du tatami (`--pull`), sous-couche auto (`--underlay-auto`), satin de bordure (`--border`) et sauts avec/sans entrée/sortie auto sur une scène de référence
Diagnostic	`satin-auto-debug`	`[diagnostic]` Auto-satin par squelette sur une forme de référence ou un vecteur d'un `.osp`

Les sous-commandes **`[diagnostic]`** inspectent le moteur : leur sortie texte n'est **pas un contrat stable** (elle peut changer d'une version à l'autre). `info`, `stats`, `digitize` (avec `--json`), `dst2svg` et `osp2dst` sont les interfaces destinées à être scriptées.

Options communes

Option	Effet
`--json`	Pour `info`, `stats`, `digitize` : un **unique document JSON sur la sortie standard**. Les messages d'état et le détail lisible passent alors sur la sortie d'erreur. Acceptée avant ou après la sous-commande.
`--no-clobber`	`digitize`, `dst2svg`, `osp2dst`, `osp2svg` : refuse d'écraser un fichier de sortie existant.
`--force`	Écrase explicitement (**comportement par défaut**) ; incompatible avec `--no-clobber`.
`--version`, `--help`	Version ; aide (la racine affiche aussi le pipeline d'exemple et les codes de sortie).

Avant tout calcul coûteux, la commande vérifie que le **dossier de sortie existe** (elle ne le crée pas) et, avec `--no-clobber`, que le fichier n'existe pas.

info

```
openstitch-cli info <image> [--dpi N] [--json]
```

Affiche format, dimensions, canaux, canal alpha et la **taille estimée en mm**. La résolution est lue dans le fichier quand il la déclare (PNG `pHYs`, JPEG JFIF) et la sortie dit d'où elle vient : « lue dans le fichier », « option --dpi » ou « défaut (aucune résolution dans le fichier) » (96 dpi). `--dpi` l'emporte toujours.

JSON : `file`, `format`, `width_px`, `height_px`, `channels`, `has_alpha`, `dpi`, `dpi_source`, `width_mm`, `height_mm`.

stats

```
openstitch-cli stats <fichier.dst> [--json]
```

Points, sauts, coupes, changements de fil, dimensions, fil estimé, puis les mesures de qualité de la séquence (déplacements longs sans coupe, points courts hors points d'arrêt reconnus par leur forme, directions dominantes). JSON : `stitches`, `jumps`, `trims`, `color_changes`, `width_mm`, `height_mm`, `thread_m`, `moves`, `long_moves_without_trim`, `short_stitches`, `short_lock_stitches` (le détail lisible reste sur la sortie d'erreur).

production

```
openstitch-cli production <fichier.osp|fichier.dst> [--json] [-o fiche.html]
   [--name NOM] [--date AAAA-MM-JJ] [--notes TEXTE] [--speed PTS_PAR_MIN] [--no-clobber]
```

Fiche de production (détail) : dimensions, points, sauts, coupes, changements de fil, fil et temps estimés, blocs de couleur dans l'ordre de couture, avertissements d'analyse. Un `.osp` passe par la séquence effective (et son cadre) ; un DST n'a pas de cadre. Sans option : texte lisible. `--json` : JSON stable sur stdout (`schema`, `project`, `date`, `notes`, `size_mm`, `frame_mm`, `fits_frame`, `totals`, `estimate`, `blocks[]`, `findings[]`, `suppressed`). `-o` écrit une page HTML autonome (aperçu SVG intégré) ; le PDF, lui, s'exporte depuis le bureau (le CLI reste sans Qt). `--speed` règle l'hypothèse de vitesse (défaut 700).

digitize

```
openstitch-cli digitize <image> <sortie.dst> [options]
```

Mêmes valeurs par défaut que le dialogue **Numérisation automatique** du bureau.

Option	Défaut	Effet
`--dpi N`	96	Résolution supposée de l'image (échelle mm/px). Ne lit **pas** le dpi du fichier : le préciser pour une taille réelle exacte.
`--max-colors N`	8	Couleurs maximales (2 à 64)
`--min-region-px N`	16	Taille minimale d'une région
`--smoothing-px N`	3	Lissage des formes
`--curve-tolerance MM`	0 (polygones)	Mode formes : contours en courbes de Bézier avec cet écart maximal ; prendre ~1,5 px ou plus (ex. 0.4)
`--satin-bands`	désactivé	Mode formes : bandes étroites (anneaux, rubans, arcs) en auto-satin au lieu du tatami, repli tatami si la couverture est insuffisante
`--coherent-lines`	désactivé	Mode contours : extrait d'abord des traits de la photo (lignes cohérentes, FDoG)
`--stipple-points N`	2000	Mode stipple (`--mode stipple`) : nombre de points du semis (1..200000)
`--stipple-seed S`	1	Mode stipple : graine du tirage initial
`--stipple-invert`	désactivé	Mode stipple : points denses sur les zones claires
`--skip-background`	`auto`	`auto`, `yes` ou `no` : ignorer la plus grande région (le fond). `auto` : seulement si c'est un fond quasi blanc qui encadre le motif. Les anciennes valeurs `-1`, `0`, `1` restent acceptées.
`--trim-threshold MM`	3	Coupe au-delà de ce déplacement
`--lock`	`backforth`	Point d'arrêt : `none`, `backforth`, `triangle` ou `zigzag`
`--mode`	`shapes`	`shapes` : formes pleines ; `contours` : dessin au trait (lignes médianes en point droit) ; `stipple` : semis de points relié en un trait
`--detail 0..1`	0.5	Mode contours : niveau de détail
`--technique`	`auto`	Mode contours : `auto`, `running` ou `legacy-satin`. `legacy-satin` : ancien satin dégradé en point droit (**pas** l'auto-satin) ; l'ancien nom `satin` est accepté avec un avertissement.
`--output-svg FICHIER`	—	SVG de diagnostic en plus du DST
`--json`, `--no-clobber`, `--force`	—	voir *Options communes*

La commande **échoue sans écrire de DST** (code 1, avec une piste de correction) quand elle ne produit rien : aucune région exploitable, aucun objet de broderie ou aucun point généré. Sinon elle affiche régions, objets (satin / tatami / running), points, sauts, coupes, dimensions et mesures de qualité (surface non couverte, angles de remplissage, déplacements et points courts par source).

JSON : `image`, `dst`, `svg`, `image_width_px`, `image_height_px`, `regions`, `background_skipped`, `objects` (`total`, `satin`, `tatami`, `running`), `stitches`, `jumps`, `trims`, `color_changes`, `width_mm`, `height_mm`, `thread_m`, `moves`, `long_moves_without_trim`, `short_stitches`, `small_objects`, `uncovered_ratio` (nul si non mesuré).

Note : `digitize` n'invoque **pas** l'auto-satin par squelette ; les zones remplissables deviennent des tatamis. L'auto-satin se demande région par région dans l'application (voir *Guide utilisateur détaillé*).

dst2svg et osp2dst

```
openstitch-cli dst2svg <entree.dst> <sortie.svg>
openstitch-cli osp2dst <projet.osp> <sortie.dst>
```

`osp2dst` passe par la fonction d'export du bureau : on y voit exactement les coupes et la fin de fichier que la machine recevra (voir *Format DST*). Pour `dst2svg`, la sortie peut aussi s'écrire `--output-svg`/`--output`.

Sous-commandes de diagnostic

osp2svg

```
openstitch-cli osp2svg <projet.osp> <sortie.svg> [--outlines] [--only ID]
```

Séquence effective du projet (celle de l'aperçu, de l'export et de l'analyse). `--outlines` superpose en gris le contour des vecteurs sources (les points qui débordent se voient d'un coup d'œil) ; `--only ID` limite à un objet brodé (un id inconnu liste les ids disponibles). Anciennes formes `--osp FICHIER --output FICHIER` (ou `--output-svg`) toujours acceptées.

stitchdebug

```
openstitch-cli stitchdebug [--shape line|corner|circle|bezier|star|ring]
                           [--length MM] [--repeats 1|2|3] [--output-svg F]
                           [--underlay MASQUE] [--underpath]
```

Point droit sur une forme de référence ; `--repeats` : 1 simple, 2 aller-retour, 3 triple. `--shape ring` génère un anneau tatami (avec `--underlay` : 1 contour, 2 parallèle ; `--underpath` : liaisons cousues cachées) et **compte les coutures qui traversent le trou** : il sort avec le code **2** si ce nombre n'est pas 0.

satin-auto-debug

```
openstitch-cli satin-auto-debug [--shape NOM | --osp F.osp --vector ID]
                                [--spacing MM] [--guide x,y,angle[,1]]
                                [--pristine] [--output-svg F] [--list-shapes]
```

Auto-satin par squelette et traversées orientées : colonnes, longueurs de traversées, diagnostics (garde de rayon, guides orphelins, couverture estimée) et SVG optionnel. `--list-shapes` affiche les noms de formes valides ; une forme inconnue produit la même liste dans le message d'erreur. `--vector` et `--pristine` exigent `--osp` ; `--osp` sans `--vector` liste les ids de vecteurs du projet. Le premier bloc de la sortie rappelle la source (« forme X » ou « vecteur N de fichier.osp »).

Codes de sortie et erreurs

Les erreurs de code 1 ont toutes la forme `openstitch-cli <sous-commande> : message`, suivie d'une ligne `Piste :` qui indique quoi vérifier (formats d'image acceptés, extension attendue, ids disponibles…). Elles vont sur la **sortie d'erreur**, jamais dans le JSON.

Deux pipelines types

1. Image vers DST, avec contrôle :

```
openstitch-cli info logo.png                    # taille estimée, dpi du fichier
openstitch-cli digitize logo.png logo.dst --dpi 300 --output-svg logo.svg
openstitch-cli stats logo.dst                   # relire le DST produit
openstitch-cli dst2svg logo.dst apercu.svg      # vérifier visuellement
```

2. Traitement en lot avec JSON (sortie standard = JSON seul) :

```
Get-ChildItem *.png | ForEach-Object {
   $r = openstitch-cli digitize $_.FullName "out\$($_.BaseName).dst" --json | ConvertFrom-Json
   if ($LASTEXITCODE -ne 0) { "ECHEC $($_.Name)"; return }
   "{0} : {1} points, {2:N1} x {3:N1} mm" -f $_.Name, $r.stitches, $r.width_mm, $r.height_mm
}
```

Les messages d'état de `digitize` vont sur la sortie d'erreur ; en cas d'échec le code de sortie est non nul et aucun JSON n'est écrit.

Implémentation associée

- `apps/cli/main.cpp` — sous-commandes, options, codes de sortie, JSON.
- `tests/cli/check_cli.cmake`, `tests/cli/CMakeLists.txt` — tests de contrat (`ctest -R cli_`), image de test `tests/fixtures/cli/`.
- `scripts/acceptance-sample.ps1` — recette sur l'image d'exemple (`digitize` + `stats`).

Traitement d'image

Public : utilisateur avancé, développeur. État : **Implémenté** (non destructif).

Principe

L'image importée est convertie en une **image de travail RGBA 8 bits** (`image::Image`). L'original n'est **jamais** modifié : le document conserve l'original et une **pile d'opérations** (`std::vector<ImageOp>`) rejouée à la demande par `apply_pipeline`. C'est ce qui rend toutes les retouches annulables.

Chargement

`load_image` lit le fichier via OpenCV (`cv::imdecode` sur les octets, ce qui gère les chemins Windows non-ASCII), gère 8 et 16 bits, niveaux de gris + alpha, et normalise tout en RGBA. `read_image_info` renvoie largeur, hauteur, canaux, présence d'alpha et format.

Opérations disponibles

Chaque opération est une alternative du variant `image::ImageOp` :

Opération	Type	Paramètres	Effet
Recadrage	`CropOp`	x, y, largeur, hauteur (px)	Découpe (borné à l'image)
Symétrie	`FlipOp`	horizontal/vertical	Miroir
Rotation 90°	`Rotate90Op`	quarts de tour 1–3	Rotation
Niveaux de gris	`GrayscaleOp`	—	Gris (alpha conservé)
Luminosité/contraste	`BrightnessContrastOp`	–100..100	Ajuste (alpha conservé)
Débruitage	`MedianDenoiseOp`	force 1–2 (noyau 3/5)	Médian
Quantification	`QuantizeOp`	2–64 couleurs	k-means déterministe (RGB)
Lissage bilatéral	`BilateralDenoiseOp`	force 1–3 (noyau 5/7/9 px, tolérance de couleur 20/40/60)	Adoucit aplats et dégradés en gardant les contours ; alpha conservé
Lignes cohérentes	`CoherentLinesOp`	`sigma_c` 0,3–5 · `sigma_m` 0–10 · `rho` 0,9–1 · `tau` 0,5–1 · ETF : 0–10 passes, rayon 1–15	Photo/dessin → traits noirs sur fond blanc (FDoG) ; alpha conservé

Le lissage bilatéral est la 8■ alternative du variant (ajoutée **en fin** : les index existants ne bougent pas). Dans le `.osp`, `"bilateralDenoise"` avec `strength` ; une version antérieure refuse un projet qui le contient (« Opération d'image inconnue »). Il sert surtout avant l'analyse de couleurs d'une région pour un fondu à deux fils (*Remplissage directionnel*).

Lignes cohérentes (`CoherentLinesOp`, 9■ alternative, `"coherentLines"` dans le `.osp`). Implémentation à partir de la publication *Coherent Line Drawing* (Kang, Lee & Chui, NPAR 2007), sans code tiers : un champ de flux de tangentes (ETF) est calculé depuis le gradient puis lissé itérativement ; une différence de gaussiennes 1D est évaluée dans la direction du gradient puis lissée **le long du flux**, enfin seuillée (`F < 0`

et `1 + tanh(F) < tau`). Le résultat est binaire (noir/blanc), l'alpha d'origine est conservé et un pixel transparent n'est jamais tracé. Comparé à un seuillage de gradient, les traits sont continus et le bruit isolé disparaît. Calcul séquentiel en `float` : déterministe. À utiliser avant le mode *Contours / Line Art* (menu Image, ou `openstitch-cli digitize --mode contours --coherent-lines`). Un trait épais donne ses deux bords ; le squelette du mode Contours les fusionne ensuite.

Note : Le **rééchantillonnage** (redimensionnement de la résolution de travail) est volontairement absent de cette pile : il changerait le rapport mm/pixel. La taille physique se fixe à l'import (voir *Modèle physique*).

Déterminisme

La quantification fixe la graine du générateur aléatoire d'OpenCV (`cv::setRNGSeed`), donc deux exécutions donnent le même résultat — prérequis des tests et de la reproductibilité.

Aperçu et annulation

La luminosité/contraste offre un **aperçu en direct** (le dialogue émet un signal, la fenêtre recalcule l'affichage sans modifier le document tant que l'utilisateur n'a pas validé). Toute opération validée passe par une commande d'annulation (`AppendImageOpCommand`).

Erreurs possibles

- Recadrage hors de l'image → message, aucune modification.
- Nombre de couleurs hors bornes → refus.
- Fichier illisible/corrompu → erreur `InvalidFile`, pas de crash.

Implémentation associée

- `libs/image/include/openstitch/image/ops.hpp` — le variant `ImageOp`.
- `libs/image/src/ops.cpp` — `apply_op`, `apply_pipeline` (OpenCV encapsulé).
- `libs/image/src/load.cpp` — `load_image`, `read_image_info`, `encode_png`, `decode_image`.
- `libs/commands/include/openstitch/commands/project_commands.hpp` — `AppendImageOpCommand`.
- Tests : `tests/unit/image/test_ops.cpp`, `test_image_load.cpp`.

Segmentation

Public : utilisateur avancé, développeur. État : **Implémenté**.

But

Réduire l'image de travail en **régions de couleur connexes**, sélectionnables et éditables, qui serviront de base à la vectorisation puis aux objets de broderie.

Algorithme

1. Les pixels d'alpha < 128 sont traités comme **fond**.
2. Les pixels opaques sont convertis en **CIELAB** (espace perceptuel) ;
3. un **k-means** déterministe (graine fixe) sur un échantillon de pixels calcule jusqu'à *N* couleurs représentatives ;
4. chaque pixel est affecté au centre Lab le plus proche ;
5. par couleur, les **composantes connexes** (4-connexité, `cv::connectedComponents`) deviennent des **régions**. Deux régions de même couleur mais disjointes restent distinctes ;
6. les régions plus petites que la taille minimale sont **absorbées** par leur voisine majoritaire (nettoyage).

Identifiants stables

Chaque région a un `RegionId` = index de slot + 1. Les slots ne sont **jamais** réutilisés : un identifiant reste valide pour toute la vie de la segmentation (important pour l'undo/redo). La carte est stockée comme `labels[y*w+x]` (0 = fond) et `region_slots[]` (la région ou `nullopt` si supprimée/fusionnée).

Éditions

Action	Fonction	Effet
Sélection	`region_at(x,y)`	Renvoie la région sous un pixel
Fusion	`merge_regions(keep, absorb)`	Réétiquette `absorb` en `keep`
Suppression	`remove_region(id)`	Renvoie les pixels au **fond** (pas de fusion)
Recoloration	`recolor_region(id, rgb)`	Change la couleur représentative
Carte	`render_map(highlight)`	Image RGBA (fond transparent, sélection éclaircie)
Nettoyage	`remove_thin_parts(min_width_px)`, `merge_small_regions(min_px, excluded)`	Ouverture morphologique et fusion des fragments, sur la copie de travail de l'auto-numérisation (voir *Numérisation automatique*)
Fond présumé	`background_candidate(options)`	Couleur de la plus grande région, surface, L, *bords touchés, recommandation (voir* Numérisation automatique*)

Note : « Supprimer » fait **disparaître** la région (retour au fond) et ne la fusionne pas avec une voisine — corrigé après un retour utilisateur (voir *Limitations*). Pour transférer des pixels à une autre région, utilisez la **fusion** explicite.

Undo/redo

Les commandes `SetSegmentationCommand`, `MergeRegionsCommand`, `RemoveRegionCommand` et `RecolorRegionCommand` mémorisent les **indices de pixels** réétiquetés (pas une copie de la carte entière), ce qui rend l'annulation exacte et peu coûteuse.

Erreurs

- Image entièrement transparente → erreur `UserInput`.
- Nombre de couleurs hors bornes → refus.

Implémentation associée

- `libs/segmentation/include/openstitch/segmentation/segmentation.hpp`
- `libs/segmentation/src/segmentation.cpp` — `segment`, `merge_regions`, `remove_region`, `recolor_region`, `render_map`.
- `libs/commands/.../project_commands.hpp` — commandes de segmentation.
- Tests : `tests/unit/segmentation/test_segmentation.cpp`, `tests/unit/commands/test_undo_stack.cpp`.

Vectorisation

Public : utilisateur avancé, développeur. État : **Implémenté** (édition de nœuds limitée au déplacement).

But

Transformer une région (masque de pixels) en **contours vectoriels propres** : contour extérieur et trous, en coordonnées physiques (µm), simplifiés et nettoyés.

Algorithme

1. Construction du **masque binaire** de la région.
2. Extraction des contours et trous par `cv::findContours` (mode `RETR_CCOMP`, approximation `CHAIN_APPROX_SIMPLE`) — algorithme de Suzuki-Abe.
3. Conversion des points en **coordonnées physiques** (µm, origine au centre de l'image, Y vers le haut).
4. **Simplification** Douglas-Peucker par contour, avec une tolérance **adaptative** bornée à `périmètre / 16` : les petits trous (mât, cordage traversant une voile) ne sont pas aplatis au point de disparaître.
5. **Nettoyage** par union Clipper2 (`clean_to_path_sets`) qui reconstruit la hiérarchie extérieur/trous (règle pair-impair) et corrige les auto-intersections. Une région en plusieurs morceaux produit plusieurs `PathSet`.

Variante courbes de Bézier (`curve_tolerance`)

Option `VectorizeOptions::curve_tolerance` (> 0 ; défaut 0 = polygones, rien ne change pour les projets existants). Les étapes 1 à 3 et le nettoyage Clipper2 sont identiques, mais le contour dense n'est **pas** simplifié par Douglas-Peucker : il est ajusté par `geometry::fit_curves` *après* le nettoyage (la topologie extérieur/trous est déjà corrigée).

`fit_curves` implémente l'ajustement de Schneider (*An Algorithm for Automatically Fitting Digitized Curves*, Graphics Gems, 1990) : 1) détection des **coins** (changement de direction mesuré sur une fenêtre d'arc de part et d'autre du sommet, maximum local, seuil 55° par défaut) ; 2) par tronçon entre coins : moindres carrés sur la longueur des poignées, reparamétrisation de Newton-Raphson, découpe récursive au point d'écart maximal ; 3) l'écart est mesuré **géométriquement dans les deux sens** (données → courbe et courbe → données), pas seulement sur le paramètre. Les parties droites restent des segments sans tangente ; les jonctions internes sont des nœuds *Lisses*, les coins des nœuds *Coin*. Écrit d'après la publication (Potrace, sous GPL, n'est ni utilisé ni copié). Déterministe.

Mesures (tests `test_curve_fit`, `test_vectorize`) sur un disque rastérisé à 96 dpi : à tolérance égale de 1,5 px (0,4 mm) ou plus, le nombre de nœuds est divisé par ≥ **3** par rapport à Douglas-Peucker (critère HP-AUTO-004) et l'écart au vrai cercle reste dans la tolérance. **Limite :** le contour de pixels est un escalier de ±0,5 px ; une tolérance inférieure à ~1 px ne fait qu'ajuster ce bruit (à 1,2 px le gain tombe vers ÷2). Le raffinement par contours actifs (Álvarez–Morel 2026) et la bissection/symétrie (Zhu–You–Zhang 2023) ne sont pas implémentés.

Les remplissages (tatami, directionnel, auto-satin, mesures d'aire) lisent les positions de nœuds : `generate.cpp` aplatit les courbes avant (`geometry:: flatten_to_polygon`, tolérance 50 µm), et la couche Clipper2 de `geometry` ainsi que `signed_area_um2` font de même. Ces fonctions sont sans effet sur un chemin sans tangente. Effet connu : sur une forme réellement ronde, les rangées de tatami proches de la tangente produisent davantage de points courts (< 0,5 mm) qu'avec un contour à facettes. Option CLI : `digitize --curve-tolerance`.

Résultat

Un `document::VectorObject` contient un ou plusieurs `geometry::PathSet` (`outer` + `holes`), garde la couleur de la région et un lien vers la `RegionId` d'origine.

Formes dessinées à la main

État : Présent · Testé (QTest headless) · Validé visuellement : non (capture d'écran non disponible dans cet environnement — voir Limitations*).* Activé par défaut, aucun réglage.

Second chemin de création d'un `VectorObject`, indépendant de toute image : avant cette fonctionnalité, la seule façon d'obtenir un objet vectoriel était de vectoriser une région segmentée depuis une image importée — un logiciel de digitalisation classique (Hatch et équivalents) permet aussi de dessiner directement une forme de base sur le canevas. Quatre outils dans la palette (barre d'outils gauche) :

- **Rectangle** (`R`) : glisser un cadre élastique → rectangle à 4 coins droits.
- **Ellipse** (`O`, Maj = cercle) : glisser un cadre élastique englobant → ellipse à 4 nœuds **lisses** (tangentes de Bézier, constante de Kappa ≈ 0,5523 — approximation standard, erreur radiale relative maximale ~0,027 %). Maj enfoncée pendant le glisser : le côté le plus petit du cadre contraint l'autre → cercle.
- **Polygone** (`P`) : clics successifs posent les sommets (aperçu élastique jusqu'au curseur) ; double-clic pour fermer (minimum 3 sommets, sinon le tracé est abandonné sans rien créer) ; Échap annule le tracé en cours à tout moment, y compris en changeant d'outil.
- **Forme libre / lasso** (`L`) : glisser en continu → un point par évènement de déplacement souris pendant que le bouton reste enfoncé (aperçu élastique mis à jour à chaque point) ; au relâchement, le tracé brut est **simplifié** (Douglas-Peucker, tolérance 0,3 mm — cf. *Vectorisation*, même algorithme que la simplification des contours vectorisés) pour ne garder qu'un contour à angles droits exploitable, sans lissage. Moins de 3 sommets après simplification (tracé trop court, ou entièrement colinéaire) → rien n'est créé, message en barre de statut.

La forme créée est un `VectorObject` **sans région source** (`source_region = std::nullopt` — le champ est optionnel précisément pour ce cas), automatiquement sélectionné : l'utilisateur enchaîne avec les actions **Créer un…** existantes (menu Broderie) pour la convertir en objet de broderie, exactement comme pour une forme vectorisée depuis une image. Aucun nouveau chemin de création côté broderie : le pipeline aval (génération, routage, export DST) est intégralement réutilisé.

Géométrie construite par `libs/geometry/src/primitives.cpp` (`rectangle_path`/`ellipse_path`/`polygon_path`/`freeform_path`, testés indépendamment de Qt) ; `MainWindow` ne fait que router le geste souris (glisser pour rectangle/ellipse — mécanique de cadre élastique déjà utilisée pour le recadrage image, généralisée ; clics successifs pour le polygone ; glisser continu pour la forme libre) vers ces fonctions puis vers `AddVectorObjectCommand` (annulable, comme la vectorisation). Un cadre trop petit (glisser quasi nul) ne crée rien.

Correctif de revue — Maj = cercle, désormais testable et fiable. La détection de Maj (contrainte cercle sur l'ellipse) interrogeait `QGuiApplication::keyboardModifiers()` (état clavier **global**) au moment de traiter le signal de fin de glisser — fonctionnellement correct en usage réel (même thread, traitement synchrone), mais impossible à couvrir par un test QTest **offscreen** (`QTest::keyPress` sur une fenêtre non affichée ne met pas à jour cet état de façon fiable), donc jamais réellement vérifié bout en bout. Corrigé : les modificateurs sont désormais capturés directement sur l'évènement Qt de relâchement qui termine le glisser (`CanvasView::mouseReleaseEvent`) et transmis par le signal `boxDrawnMm` — plus de dépendance à un état global, testable en passant directement `Qt::ShiftModifier`.

Édition de nœuds

Les nœuds de l'objet sélectionné s'affichent (poignées de taille constante) et se **déplacent** à la souris ; chaque déplacement passe par `MoveNodeCommand` (annulable).

Limitation : l'**ajout/suppression de nœuds**, la **conversion anguleux/lisse** et l'édition de tangentes de Bézier ne sont **pas** exposés dans l'interface (le modèle les prévoit : `PathNode` porte `tan_in`/`tan_out` et un `NodeType`).

Robustesse

La correction de la tolérance de simplification (adaptative) évite un défaut observé : des trous trop simplifiés faisaient déborder les remplissages (voir *Tatami* et *Limitations*).

Implémentation associée

- `libs/vectorization/src/vectorize.cpp` — `vectorize_region`.
- `libs/geometry/src/simplify.cpp` — Douglas-Peucker, aire signée.
- `libs/geometry/src/clean.cpp` — `clean_to_path_sets` (Clipper2 encapsulé).
- `libs/document/include/openstitch/document/vector_object.hpp` — `VectorObject`, `NodeRef`.
- `libs/commands/.../project_commands.hpp` — `AddVectorObjectCommand`, `MoveNodeCommand`.
- `libs/geometry/include/openstitch/geometry/primitives.hpp` + `src/primitives.cpp` — `rectangle_path`, `ellipse_path`, `polygon_path`, `freeform_path` (formes dessinées à la main).
- `apps/desktop/main_window.cpp` — `addVectorPrimitive`, `onBoxDrawn`, `onCanvasDoubleClicked`, `updatePolygonPreview`/`finishPolygon`/ `cancelPolygonDraw`, `onFreeformPointAdded`/`finishFreeform`/ `cancelFreeformDraw` ; `apps/desktop/canvas_view.hpp/.cpp` — modes `setBoxDrawMode`/`setPolygonDrawMode`/`setFreeformDrawMode` (généralisation du cadre élastique de recadrage) ; `apps/desktop/ui_icons.hpp/.cpp` — `icons::freeform`.
- Tests : `tests/unit/vectorization/test_vectorize.cpp`, `tests/unit/geometry/test_simplify.cpp`, `test_clean.cpp`, `test_primitives.cpp` ; `tests/unit/desktop/test_main_window.cpp` (création, undo/redo, cadre dégénéré, tracé de polygone, annulation).

Objets de broderie

Public : utilisateur avancé, développeur. État : **Implémenté** (quatre types).

Définition

Un `document::EmbroideryObject` porte l'**intention de couture** : un type de point et ses paramètres, une couleur de fil, un lien vers l'objet vectoriel source, un état de visibilité et de verrouillage. Ses **points ne sont pas stockés** dans l'objet : ils sont régénérés à la demande (séparation intention/points).

Le variant `StitchParams`

Le type de point est un `std::variant` à quatre alternatives :

Objet	Params	Géométrie suivie	Sous-couche
Point de contour	`RunningStitchParams`	contours de l'objet vectoriel	non
Remplissage tatami	`TatamiParams`	régions pleines (avec trous)	contour et/ou rangées perpendiculaires (désactivées par défaut ; activées selon l'aire par l'auto-numérisation)
Colonne satin	`SatinParams`	deux **rails** portés par l'objet	centrale (point droit)
Remplissage directionnel	`DirectionalFillParams`	régions pleines ; points qui suivent des **guides**	contour + perpendiculaire (du tatami)

Le satin est particulier : il **porte sa propre géométrie** (deux rails éditables), car une colonne ne se déduit pas d'un simple contour.

Tableau récapitulatif

Objet	Générateur	Paramètres clés	Statut recommandé
Running (simple/double/triple)	`run_stitch` + `apply_repeats`	`stitch_length`, `min_length`, `repeats`	Implémenté
Tatami	`fill_tatami`	`row_spacing`, `stitch_length`, `angle`, `inset`, `stagger`	Partiel
Satin	`fill_satin`	`density`, `pull_compensation`, `center_underlay`	Expérimental
Directionnel	`fill_directional`	`guides`, `break_lines`, `row_spacing`, `stitch_length`, `handmade`	Expérimental

Le remplissage directionnel est détaillé dans son chapitre (*Remplissage directionnel*) ; il se crée par conversion d'un tatami (inspecteur) ou via *Type de points*.

Le tableau reflète la **qualité métier**, pas seulement la présence de code : le running stitch est solide ; le tatami est fonctionnel, sa sous-couche et son underpath caché existent mais sont désactivés par défaut (l'auto-numérisation active la sous-couche) ; le satin est une génération simple à deux rails (voir les chapitres dédiés). Aucun n'est validé sur machine réelle.

Création

Depuis l'interface (menu Broderie) sur un objet vectoriel sélectionné, ou en lot par la **classification automatique expérimentale des régions** (voir *Limitations*) qui choisit le type selon la forme :

- bande fine compatible → une ou plusieurs sections de **satin topologique** ;
- autre zone **remplissable** → **tatami**, orienté selon l'axe principal de la région et avec une sous-couche selon son aire (voir *Numérisation automatique*) ;
- petite forme → **contour** cousu (point triple) ;
- satin **automatique** naïf : désactivé par défaut car il débordait sur les formes concaves/branchues (`AutoOptions::use_naive_satin` pour le réactiver).

Le **type se change ensuite** à tout moment (clic droit sur la forme ■ *Type de points* ■ contour / tatami / satin / directionnel) via `SetStitchTypeCommand` (annulable) ; l'action **Convertir les satins auto en tatami** répare en lot un projet importé.

Limitation : cette classification est une heuristique simple (largeur moyenne = 2·aire/périmètre total) autour d'un moteur géométrique ; ce n'est pas encore un moteur d'auto-numérisation robuste comparable à un logiciel commercial. Elle produit des objets **éditables** qu'il faut vérifier et retoucher.

Régénération

`generate_sequence(project)` parcourt les objets **visibles** dans l'ordre, insère un changement de couleur entre deux objets de couleurs différentes, et appelle le générateur adapté à chaque type via `std::visit`. Le résultat est une `StitchSequence`. Chaque commande porte l'`ObjectId` de sa source : l'interface s'en sert pour afficher la broderie **dans la couleur de chaque fil** et pour **filtrer** l'affichage (par type, couleur ou taille de zone).

Implémentation associée

- `libs/document/include/openstitch/document/embroidery_object.hpp` — `EmbroideryObject`, `StitchParams`, `RunningStitchParams`, `TatamiParams`, `SatinParams`.
- `libs/stitch_generation/src/generate.cpp` — `generate_sequence`, dispatch.
- `libs/autodigitize/src/autodigitize.cpp` — choix automatique du type.
- Tests : `tests/unit/stitch/test_generate.cpp`.

Numérisation automatique

Public : utilisateur avancé, développeur. État : **Partiellement implémenté** (heuristiques, non validées sur machine réelle).

La numérisation automatique (`autodigitize::auto_digitize`) transforme une segmentation en objets de broderie **éditables** : un objet vectoriel par région, puis un objet de broderie dont le type (satin, tatami, contour) est choisi selon la forme (voir *Objets de broderie*). Ce chapitre documente les réglages qui décident de la qualité du résultat cousu, introduits après l'audit d'un cas réel (septembre 2026, ci-dessous).

Cas d'étude : la marine plein cadre

Une image « carte de segmentation » (ciel, nuages, lune, mer en trois tons, reflet de lune, rivage, deux bateaux, mouettes ; PNG sans alpha, environ 250 × 176 mm) numérisée puis exportée en DST donnait : 38 907 points, aucun point sur le ciel, tous les tatami à 0° sans sous-couche, des interstices entre remplissages voisins, 2 916 déplacements dont 1 321 de plus de 3 mm et aucune coupe, 4 946 points de moins de 0,5 mm, le reflet éclaté en 817 morceaux et aucun point d'arrêt. Chaque section ci-dessous traite une de ces causes (lots A à G de la mission).

Fond présumé (Lot A)

Une image sans canal alpha n'a aucun pixel transparent : son fond devient une région opaque comme les autres, souvent la plus grande. `AutoOptions::skip_largest_region` exclut alors la **couleur** de la plus grande région, et toutes les régions de cette couleur exacte (un fond peut être fragmenté par le motif).

Avant ce lot, l'option était cochée d'office dès que l'image n'avait pas d'alpha. Sur une image **plein cadre**, la plus grande région est un vrai élément du motif (le ciel de la marine) : elle n'était pas brodée du tout.

La recommandation vient désormais de `segmentation::background_candidate(seg)`, qui mesure pour la couleur de la plus grande région :

Mesure	Sens
`area_ratio`	part de l'image couverte par cette couleur (toutes régions)
`lightness`	clarté CIELAB L* de la couleur (`cielab_lightness`, calcul analytique D65)
`sides_touched`	nombre de bords de l'image touchés (0 à 4)
`recommended`	`lightness > min_lightness` (90) **et** `sides_touched >= min_sides_touched` (3)

Les seuils sont dans `BackgroundCandidateOptions`. Le dialogue *Numérisation automatique* coche la case seulement si `recommended`, et affiche la pastille de couleur, le pourcentage de surface, L *et le nombre de bords touchés. Le CLI (`openstitch-cli digitize`) applique la même règle avec `--skip-background auto` (défaut ; anciennement `-1`) et imprime la candidate ; `yes`/`no` (anciennement `1`/`0`) restent des choix explicites prioritaires. Sur la marine, le ciel (bleu, L* ≈ 43) n'est plus ignoré.

Orientation et sous-couche des remplissages (Lot B)

Avant ce lot, chaque tatami créé par l'auto-numérisation recevait `TatamiParams{}` : rangées à 0°, aucune sous-couche. Les valeurs par défaut de `TatamiParams` **ne changent pas** (projets `.osp` existants, objets créés à la main) : les réglages sont appliqués par une post-passe (`configure_tatami_fills`) sur les tatami que l'auto-numérisation crée, y compris les remplissages de repli d'un satin incomplet, pilotée par des champs d'`AutoOptions`.

Angle. L'orientation des rangées suit l'axe principal de la surface de l'objet (`geometry::principal_axis` : moments d'inertie du second ordre, calculés exactement sur le polygone par la formule de Green, trous déduits). Si le rapport des moments principaux est inférieur à `fill_isotropy_ratio` (1,3), la forme est jugée quasi isotrope (disque, carré) : l'angle vaut `isotropic_fill_angle` (45°).

Voisins. Deux tatami dont les régions se touchent (au sens de `segmentation::region_adjacency`) ne doivent pas avoir des rangées à moins de `min_neighbor_fill_angle_gap` (20°) l'une de l'autre, sinon la frontière disparaît à l'œil. Les tatami sont traités par aire décroissante (puis identifiant d'objet, pour un ordre total déterministe) : une grande région fixe son angle naturel, une plus petite en conflit cherche l'angle le plus proche du sien (pas de `fill_angle_search_step`, 5°) qui respecte l'écart avec tous ses voisins déjà placés ; à défaut, celui qui maximise l'écart minimal. C'est donc toujours la plus petite qui cède.

Sous-couche selon l'aire nette de l'objet :

Aire	Sous-couche
< `underlay_edge_min_area_mm2` (20 mm²)	aucune
20 à `underlay_parallel_min_area_mm2` (100 mm²)	contour rentré (`underlay_edge`)
≥ 100 mm²	contour + rangées perpendiculaires (`underlay_parallel`)

`auto_fill_angle = false` et `auto_fill_underlay = false` rétablissent le comportement historique. L'import SVG direct (`auto_digitize_vectors`) reçoit l'angle naturel et la sous-couche, mais pas la règle de voisinage (aucune segmentation, donc aucune adjacence connue) — limite documentée.

Chevauchement entre remplissages voisins et ordre en couches (Lot C)

Cause des interstices. Trois écarts s'additionnaient le long de chaque frontière entre deux tatami :

1. la vectorisation trace chaque contour par les **centres** des pixels de bord (`vectorize.cpp`) : deux régions voisines sont séparées d'un pixel entier (0,17 mm sur la marine) ;
2. chaque contour est simplifié indépendamment (tolérance 0,2 mm) ;
3. chaque tatami est rentré de `TatamiParams::inset` (0,2 mm) de son côté.

Correction (`overlap_neighbor_fills`) : la surface remplie d'un tatami devient

```
(région rentrée de inset) ∪ (région ⊕ g ∩ voisins brodés ⊕ g),  g = fill_overlap + ½ pixel
```

Le débord ne se fait donc que du côté des régions voisines **brodées**, sur `fill_overlap` (0,3 mm, réglable dans `AutoOptions`) au-delà de la vraie frontière. Les bords extérieurs du motif et le contact avec le fond ignoré (qui n'a pas d'objet vectoriel, donc n'est jamais un voisin brodé) gardent le retrait. La surface élargie remplace la géométrie de l'objet vectoriel de la région, et `inset` passe à 0 pour ne pas rentrer deux fois. Conséquence visible : l'objet vectoriel éditable déborde légèrement sur ses voisins. Toutes les surfaces sont calculées sur la géométrie d'origine avant d'être appliquées, donc le résultat ne dépend pas de l'ordre de traitement. **Désactivé par défaut (2026-10).** `fill_overlap` vaut maintenant 0 : la surface élargie était portée par l'objet vectoriel lui-même, donc tout autre type de points créé ensuite sur la région (satin, contour cousu, remplissage directionnel) débordait de sa zone (+14 à +57 % d'aire mesurés sur un projet réel). Les contours restent ceux de la segmentation, et chaque tatami garde son retrait `inset` (0,2 mm). Conséquence : un interstice réapparaît entre deux couleurs voisines ; une valeur de `fill_overlap` > 0 réactive la correction ci-dessus à la demande. Dans le bureau, créer un satin ou passer un objet en satin ramène en plus un contour agrandi par une ancienne auto-numérisation à celui de sa région (un seul pas d'annulation).

Réglage dans l'interface (2026-10) : la boîte *Numérisation automatique* (mode formes) propose « Chevauchement des remplissages » (0 à 1 mm, par pas de 0,05 mm, 0 par défaut) ; la valeur est

mémorisée entre les sessions (`autoDigitize/fillOverlapUm`) et alimente `AutoOptions::fill_overlap`. Même réserve que ci-dessus : le débord fait partie de la forme vectorielle.

Les remplissages de repli d'un satin incomplet ne sont pas concernés : ils recouvrent déjà leurs bandes satin (`kCoverageOverlap`).

Ordre en couches. Un remplissage qui déborde doit passer **sous** son voisin : les grandes zones de fond (ciel, mer) doivent être cousues avant les détails posés dessus. La nouvelle stratégie `optimization::OrderStrategy::LayeredColorThenProximity` :

- garde le regroupement par couleur (même nombre de changements de fil) ;
- ordonne les couleurs par aire de leur plus grande zone, décroissante ;
- à couleur égale, coud d'abord les grandes zones (aire ≥ `layer_large_area_ratio` = 25 % de la plus grande de la couleur) par aire décroissante, puis les petites par proximité ;
- laisse les objets verrouillés à leur place (mécanisme commun à toutes les stratégies).

L'auto-numérisation l'applique à son résultat (`order_by_layers`). L'unité d'ordre est une suite contiguë d'objets de même `source_vector`, pour que les sections satin d'une région restent contiguës : `generate_sequence` ne route ensemble que des sections contiguës. Avant ce lot, aucune numérisation automatique (desktop ou CLI) n'ordonnait son résultat : les objets suivaient l'ordre des identifiants de région (classe k-means puis balayage de l'image).

Fragments (Lot D)

Avant ce lot, le seul seuil de taille était `SegmentationOptions:: min_region_px` (16 px, soit 0,48 mm² à 146,8 dpi : indépendant de l'échelle), et toute région sous `min_fill_area_mm2` devenait un contour point triple, même au milieu d'autres régions brodées. Sur la marine, le reflet (couleur très découpée, anticrénelage) donnait 817 morceaux, dont 683 de moins de 15 points.

`auto_digitize` travaille désormais sur une **copie** de la segmentation (celle du projet n'est pas modifiée) et la nettoie avant de vectoriser :

1. **Isthmes et lamelles** (`segmentation::remove_thin_parts`) : ouverture morphologique région par région, avec un noyau elliptique de `min_feature_width_mm` (1,2 mm, converti en pixels avec `mm_per_px`). Les pixels retirés sont rendus de proche en proche à la région **voisine** majoritaire. Une partie qui ne touche aucune autre région (lamelle isolée dans le vide) reste en place.
2. **Petites régions** (`segmentation::merge_small_regions`) : toute région de moins de `min_region_area_mm2` (3 mm², converti en pixels) est fusionnée avec la voisine qui partage la plus longue frontière. La plus petite restante est toujours traitée d'abord, et la région absorbante garde sa couleur. Le **fond ignoré n'absorbe jamais** : un fragment qui n'a que lui pour voisin reste une région isolée et devient un contour point triple (s'il est sous `min_fill_area_mm2`). C'est désormais le seul cas de contour pour une petite région.

Les deux étapes sont déterministes (voisinages ordonnés, égalités tranchées par le plus petit identifiant) et désactivables (seuil à 0).

Aire vectorisée. Le seuil porte sur l'aire que la région aura une fois vectorisée, pas sur son nombre de pixels. Le contour passe par les centres des pixels de bord : une tache de N pixels et B arêtes de frontière donne un polygone d'aire N – B/2 + 1 (théorème de Pick). Sans cette correction, des régions de 3 mm² en pixels devenaient des contours de 2,5 mm² sur la marine. `merge_small_regions` reçoit donc un poids de frontière de ½ (`boundary_weight`).

Compromis connu : l'ouverture efface aussi les détails volontaires plus fins que 1,2 mm (par exemple un mât fin), rendus à la région qui les entoure. Pour les garder, baissez `min_feature_width_mm`.

Coupes automatiques et points d'arrêt (Lot E)

Avant ce lot, seules les retouches manuelles produisaient des `Trim`, et les seuls points d'arrêt étaient ceux, désactivés par défaut, du satin : la marine comptait 1 321 déplacements de plus de 3 mm sans coupe et aucun point d'arrêt. Ces finitions ne relèvent pas de l'auto-numérisation mais de la séquence : elles s'appliquent à tout projet, via `project.finishing` (`document::SequenceFinishing`) et `stitch_generation::finish_sequence`, dernière passe d'`effective_sequence` (voir *Moteur de génération de points*) :

Réglage	Défaut	Rôle
`trim_threshold`	3 mm	au-delà : arrêt de sortie, coupe, déplacement, arrêt d'entrée ; en deçà : simple saut dans un même objet ; entre deux objets, la coupe est toujours faite
`trim_before_color_change`	oui	coupe avant chaque changement de fil
`lock_type`	aller-retour	point d'arrêt à l'entrée/sortie de chaque objet et autour de chaque coupe
`lock_length`, `lock_passes`	0,8 mm, 2	taille (bornée au point voisin) et répétitions

Réglables dans le desktop (menu *Broderie* ■ *Options de génération…*, annulable) et dans le CLI (`digitize --trim-threshold <mm> --lock none|backforth|triangle|zigzag`). Un projet `.osp` antérieur est relu sans finitions (séquence identique).

Déplacements internes et points courts (Lot F)

Trajets cachés. Les tatami créés par l'auto-numérisation reçoivent `hidden_underpath = true` (`AutoOptions::fill_hidden_underpath`) : la liaison entre deux composantes de rangées devient un trajet cousu caché sous la couche supérieure (direct s'il reste court et intérieur, sinon le long du contour rentré ; voir *Remplissage tatami*) au lieu d'un saut. Le défaut de `TatamiParams` reste `false`.

Points courts. Dernier réglage de `project.finishing` : `filter_short_stitches` (activé) et `min_stitch_length` (0,5 mm). Avant la pose des verrous, `finish_sequence` retire chaque point cousu plus proche que ce seuil du point conservé précédent, ce qui revient à le fusionner avec le suivant. Aucun point n'est déplacé. Ne sont jamais retirés :

- le premier et le dernier point d'un tracé ;
- les passes de verrou (`Lock`), qui sont d'ailleurs ajoutées après le filtre ;
- les points retouchés à la main (`Manual`) ;
- un point dont le retrait ferait passer le nouveau segment hors de la région de son objet (`segment_stays_in_region` sur l'objet vectoriel suivi) — par exemple le point d'un angle intérieur.

La piqûre de longueur nulle à l'arrivée d'un saut (`Jump p0` puis `Stitch p0`) est omise quand un verrou d'entrée suit : le verrou revient piquer en `p0`.

Mesures et critères d'acceptation (Lot G)

`openstitch-cli stats fichier.dst` imprime, en plus des compteurs de base, les mesures de `stitch_analysis::sequence_metrics` (voir *Analyse et validation*) : déplacements, dont ceux de plus de 3 mm sans coupe, points de moins de 0,5 mm hors points d'arrêt, histogramme des directions. `openstitch-cli digitize` imprime en plus `project_metrics` : objets de moins de 3 mm², surface non couverte hors fond ignoré, angles de remplissage distincts, ventilation par source.

Contours / Line Art

Seconde stratégie de l'auto-numérisation, pour les **dessins au trait** : au lieu de remplir les régions, on coud les *traits*. Entrée : la même segmentation CIELAB ; sortie : des objets éditables (`AutoResult`), jamais des points stockés.

Pipeline (`libs/autodigitize/src/contour_*.cpp`) : segmentation → pour chaque couleur, squelette / lignes médianes avec largeur locale → réseau de nœuds (jonctions, extrémités) et segments (`ContourNetwork`) → nettoyage selon `detail` (branches courtes, boucles et éléments isolés trop petits, fusion de traits proches, simplification Douglas-Peucker) → classification par segment (`classify_segment` : point droit simple, point triple, satin, rejeté) → objets : une couleur = un groupe contigu (les plus claires d'abord, la plus sombre en dernier), ordre déterministe. Les lignes sont des objets Running à chemin ouvert ; l'auto-numérisation ne produit pas de satin (`satin_planning` a été supprimé) : l'utilisateur convertit ensuite en satin automatique.

Détail → seuils (`contour_thresholds`, fonction pure et monotone ; facteur `f = 4^(1 - 2*detail)`, 0,5 = réglages historiques) :

detail	f	branche min	élément isolé min	tolérance DP	fusion
0,0	4	6 mm	9,6 mm	0,40 mm	1,2 mm
0,25	2	3 mm	4,8 mm	0,28 mm	0,6 mm
0,5	1	1,5 mm	2,4 mm	0,20 mm	0,3 mm
0,75	0,5	0,75 mm	1,2 mm	0,14 mm	0,15 mm
1,0	0,25	0,375 mm	0,6 mm	0,10 mm	0,075 mm

Boucle fermée minimale : périmètre π × branche min. Chaque seuil est borné par les garde-fous physiques ci-dessous.

Garde-fous physiques (`contour_limits`, jamais abaissés par `detail`) : largeur satin min = `SatinabilityThresholds::min_satin_width` (0,8 mm) et max = `AutoOptions::satin_max_width` ; longueur min d'élément = `RunningStitchParams::min_length` ; périmètre de boucle min = 3 fois cette longueur ; tolérance de simplification min = 0,1 mm (pas DST). Heuristiques de qualité satin (variation de largeur 50 %, virage 60° sur ~1 mm) ignorées quand la technique est forcée à Satin. Un satin voulu mais impossible retombe en point droit avec un avertissement (jamais silencieux) ; les segments inférieurs à 0,5 mm entre nœuds sont rejetés avec avertissement.

Technique : Automatique (par segment : fin -> point droit, régulier -> satin, sinon repli), Running (point droit sur la ligne médiane), Satin (satin quand la largeur le permet).

Jonctions : croisements (X, T) conservés comme nœuds du réseau ; les colonnes satin aboutissant à une jonction étaient ancrées par l'ancien `satin_planning` (supprimé) ; le satin par squelette traite les jonctions par cellules (voir `satin-squelette.md`).

Routage en chemin unique (`ContourOptions::routing = SinglePath`, `digitize --single-path`, défaut `Chains` = comportement historique inchangé). L'assemblage historique (`assemble_chains`) n'apparie à une jonction que des branches « alignées » (virage < 90°) : un éventail de branches serrées

reste en autant de polylignes, donc autant de sauts et de coupes. `route_single_path` (`contour_route.cpp`) cherche à la place un **parcours eulérien** du réseau (problème du postier chinois, version gloutonne déterministe) : 1) les nœuds de degré impair sont les extrémités de polylignes inévitables (au minimum `impairs / 2`) ; 2) deux impairs distants d'au moins `trim_threshold` (3 mm, au-delà duquel un saut coûte une coupe) sont reliés en **repassant** sur le plus court chemin du réseau s'il fait au plus `max_retrace` (12 mm) ; 3) les impairs restants sont appariés au plus proche (liaison par saut) ; 4) circuit d'Euler par Hierholzer, en continuant toujours par la branche la plus alignée, coupé aux liaisons par saut. Un réseau sans nœud impair donne un seul circuit fermé. Sur une grille de traits de test : 7 -> 5 polylignes et 7 -> 5 coupes. Les idées viennent de la théorie des graphes ; l'outil libre Ink/Stitch, sous GPL, n'a pas été consulté. Limites : la paire de repassage est choisie gloutonnement (pas l'appariement optimal) ; les tronçons repassés sont cousus deux fois (visible en point simple, absorbé en point triple).

Lignes cohérentes (photo -> traits) : appliquer d'abord l'opération d'image `CoherentLinesOp` (voir Traitement d'image), ou `digitize --coherent-lines` ; le mode Contours reçoit alors un dessin au trait net au lieu d'une photo.

Métriques (`ContourMetrics`) : composantes, segments, jonctions, extrémités, polylignes (`trails`) et longueur repassée, branches courtes et éléments petits supprimés, longueur point droit / satin (mm), largeur min / moyenne pondérée / max, replis, rejets. Affichées par `openstitch-cli digitize --mode contours` et dans la barre d'état du desktop.

Utilisation : CLI `digitize image.png out.dst --mode contours --detail 0.5 --technique auto|running|legacy-satin` (`satin` : ancien nom accepté ; défaut `--mode shapes`, comportement inchangé ; voir Ligne de commande) ; desktop : dialogue « Numérisation automatique » -> « Contours », curseur Détail 0-100 (défaut 50), technique Automatique / Running / Satin. Pas d'aperçu superposé dans le dialogue pour l'instant.

Limites connues : un pincement en 8 n'est que partiellement satin (repli point droit + avertissement) ; segments entre nœuds < 0,5 mm rejetés ; images

> 4000 px non testées ; groupes de couleur non ordonnés par proximité ; sur le cas synthétique CLI, `detail` 0,9 produit des colonnes refusées sur un anneau fin (« trou entre stations ») : le segment est alors découpé en plusieurs objets au lieu de rester continu.

Satin sur les bandes (`satin_bands`)

Par défaut l'auto-numérisation ne produit que du tatami (zones) ou du contour (petites zones). `AutoOptions::satin_bands` (CLI : `--satin-bands` ; desktop : case « Satin automatique pour les bandes étroites » du dialogue, mémorisée ; désactivé par défaut) rend le satin aux **bandes étroites** : anneaux, rubans, traits épais, arcs.

Une région devient un **auto-satin par squelette** (`AutoSatinParams` par défaut, éditable ensuite) si :

- elle fait au plus 1 trou (un anneau) ;
- sa largeur inscrite maximale est <= `satin_max_width` (6 mm) et sa largeur moyenne >= 0,8 mm (seuil de `SatinabilityThresholds`) ;
- elle est allongée : longueur >= 2,5 x largeur, avec longueur = périmètre total (extérieur + trous) / 2. Le rapport de satinabilité n'est pas utilisé ici : il ignore les trous dans le périmètre et un anneau n'a pas d'arête de squelette ;
- **la génération essayée** (`generate_skeleton_satin`, déterministe) couvre au moins `satin_min_coverage` (0,9) de la région.

Sinon : tatami, comme avant. Les **formes compactes sans trou** (disques, pétales) de largeur <= `satin_compact_max_width` (12 mm, 0 = désactivé) sont cousues en satin à cordes parallèles (`AutoSatinParams::compact_axis`, voir `satin-squelette.md`), sous la même condition de couverture. Au-delà, tatami. Essai CLI (anneau 2,4 mm, arc 3,7 mm, disque 11,6 mm) : les trois en satin, 7 sauts au lieu de 12 et plus de points courts de tatami.

Limites : la case du desktop n'a pas été compilée dans l'environnement de développement Linux (Qt absent), à vérifier au prochain build Windows ; une région à plusieurs trous n'est pas tentée ; la couverture est géométrique, pas validée sur machine ; pas de satin en éventail ou concentrique pour les grands disques (voir `satin-squelette.md`).

Stippling (semis de points relié en un trait)

`libs/stipple` (Qt-free, sans dépendance GPL) transforme une image en un **unique trait de points droits** dont la densité reproduit les tons.

1. **Densité** : luminance Rec. 601 → `(1 - L)^gamma` (alpha < 128 = vide), moyenne par blocs jusqu'à 600 px max (`--stipple-invert` : zones claires denses, pour un fil clair sur fond sombre).
2. **Semis** (Secord, « Weighted Voronoi Stippling », NPAR 2002) : tirage par rejet selon la densité (générateur xorshift64 *portable, graine fixe), puis relaxations de Lloyd où chaque point va au centroïde pondéré par la densité** de sa cellule de Voronoï (cellules exactes sur la grille). On obtient un bruit bleu, dense dans les zones sombres.
3. **Tour** : ordre de Hilbert, puis 2-opt (listes de voisins, bits « don't look ») et Or-opt. Concorde n'est volontairement pas utilisé (licence académique). Le tour fermé est coupé à son arête la plus longue et aux liaisons > 8 mm : aucune liaison ne traverse un grand vide.
4. **Objets** : un `VectorObject` (chemins ouverts) + un objet brodé point droit (longueur 1,5 mm), fil noir par défaut.

Utilisation : `openstitch-cli digitize image.png out.dst --mode stipple [--stipple-points N] [--stipple-seed S] [--stipple-invert]` (défaut 2000 points). Résultat déterministe pour une graine donnée.

Limites connues : pas de couleur (une seule couche monochrome) ; pas de bouton dans le desktop pour l'instant ; le trait reste un tour de points, pas une trame de lignes de densité variable (PhotoStitch complet = HP-STI-017 partiel) ; un tour TSP heuristique est ~5 % au-dessus de l'optimum.

Implémentation associée

- `libs/segmentation/include/openstitch/segmentation/segmentation.hpp` — `cielab_lightness`, `BackgroundCandidateOptions`, `BackgroundCandidate`, `background_candidate`, `region_adjacency`, `remove_thin_parts`, `merge_small_regions` (Lot D).
- `libs/autodigitize/src/autodigitize.cpp` — `auto_digitize`, `AutoOptions::skip_largest_region`, `configure_tatami_fills` (Lot B), `overlap_neighbor_fills`, `order_in_layers` (Lot C).
- `libs/geometry/include/openstitch/geometry/boolean.hpp` — `union_polygons`.
- `libs/optimization/include/openstitch/optimization/order.hpp` — `OrderStrategy::LayeredColorThenProximity`, `OrderOptions`, `OrderItem::area_mm2`.
- `libs/geometry/include/openstitch/geometry/moments.hpp` — `principal_axis`.
- `apps/desktop/main_window.cpp` — `MainWindow::autoDigitize` (dialogue).
- `libs/autodigitize/src/contour_*.cpp` — stratégie Contours (`auto_digitize_contours`, `contour_thresholds`, `contour_limits`).
- `apps/cli/main.cpp` — `run_digitize` (`--mode contours`).
- Tests : `tests/unit/segmentation/test_segmentation.cpp` (cas « fond blanc », « plein cadre coloré », « blanc central », « deux bords »), `tests/unit/desktop/test_main_window.cpp` (`autoDigitizeDialogDoesNotSkipColoredFullFrameRegion`, `autoDigitizeDialogSkipsNearWhiteFramingBackground`), `tests/unit/geometry/test_moments.cpp`, `tests/unit/autodigitize/test_autodigitize.cpp` (cas « Lot B », « Lot C » et « Lot D »), `tests/unit/optimization/test_order.cpp`, `tests/unit/geometry/test_boolean.cpp`.

Génération de points — point droit et fondations

Public : utilisateur avancé, développeur.

> État : Présent dans le code : oui · Tests unitaires : oui · Tests visuels : oui (SVG golden) · Import/export DST : oui · Test sur machine réelle : **non** · **Statut recommandé : implémenté** — reconstruit sur des fondations de longueur d'arc, c'est le générateur le plus solide du projet.

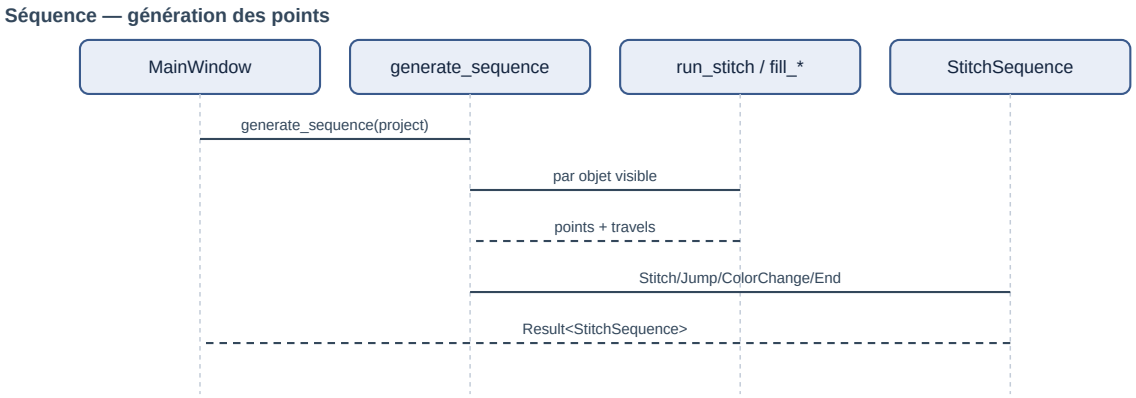


Figure — Génération de la séquence de points à partir du document.

Fondations géométriques

Avant tout type de point, deux primitives (dans `geometry`) :

- **Aplatissement de Bézier adaptatif** (`flatten`) : les segments à tangentes sont subdivisés récursivement (De Casteljau) jusqu'à ce que l'écart à la corde passe sous une tolérance. On n'échantillonne **jamais** une Bézier par pas uniforme du paramètre `t`.
- **Paramétrisation par longueur d'arc** (`cumulative_lengths`, `point_at_length`) et **ré-échantillonnage équilibré** (`resample_run`) : un tronçon de longueur L est divisé en `n = ceil(L / cible)` parts **égales**, donc chaque point est ≤ la longueur cible et il n'y a pas de segment résiduel minuscule.

Sémantique du paramètre : avec `ceil`, le champ (`RunningConfig::target_length`) est traité comme une **longueur maximale** (les points ne la dépassent jamais, mais peuvent être plus courts). Si on voulait une longueur *souhaitée* dont l'espacement colle au plus près de la consigne, `round(L / cible)` serait plus adapté. Ce choix (`ceil` plutôt que `round`) est **assumé explicitement** dans le code (`libs/geometry/src/polyline.cpp`, commentaire de `resample_run` : « Écart assumé (...) ; `ceil` garantit en plus le respect strict de la longueur maximale. ») — ce n'est plus une ambiguïté ouverte, `target_length` se comporte délibérément comme un maximum, pas une cible exacte.

Point droit (running stitch)

`run_stitch(path, config)` :

1. aplatit le chemin (courbes comprises) ;
2. détecte les **coins vifs** (angle de rotation > seuil, ~35° par défaut) ;
3. découpe le chemin en tronçons entre coins ;
4. ré-échantillonne **chaque tronçon par longueur d'arc**.

Résultat : les coins sont des pénétrations **exactes** ; les portions lisses ont un espacement **régulier**. Un cercle finement facetté ne produit donc plus un point par facette, mais des points espacés de la longueur cible.

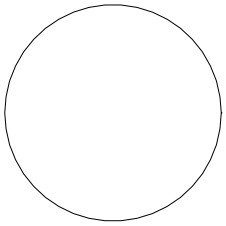


Figure — Trajectoire réelle produite par le moteur (SVG de diagnostic) : cercle de rayon 20 mm, longueur cible 3 mm. Trait = couture, orange pointillé = sauts.

Le résultat est structuré : `RunningResult { points, warnings, stats }`. Il ne plante jamais et renvoie un avertissement (chemin vide, trop court, point trop long…).

Chemins fermés, sens, phase, départ

`RunningConfig` expose : `reverse` (sens), `phase` (décalage curviligne des boucles lisses) et `start` (point de départ projeté sur une boucle fermée). Une boucle fermée revient à son point de départ sans doublon minuscule.

Répétitions

`apply_repeat_mode` implémente des modes **explicites** :

Mode	Effet
`SinglePass`	un passage
`BackAndForth`	aller complet puis retour (termine au départ)
`BeanStitch`	chaque intervalle cousu `n` fois (impair) — point triple
`Backstitch`	progression avec recouvrement

Les mouvements nuls sont supprimés ; le nombre exact de traversées est testé.

Traitement des points courts

Le running stitch **ne supprime pas** aveuglément les points sous une longueur minimale (cela déplacerait un sommet ou détruirait un détail). À la place, les **vrais coins** (angle de rotation > seuil) sont préservés comme pénétrations exactes, et chaque tronçon lisse est ré-échantillonné indépendamment par longueur d'arc — ce qui évite les rafales de micro-points sur les courbes finement facettées. Un point plus court que `min_length` peut subsister à un **coin serré** (deux coins proches) ; il est alors **signalé** par un avertissement, pas supprimé.

Limitation : ce traitement fin s'applique au running stitch. Aux **fins de rangée** du tatami et surtout aux **extrémités pointues** d'un satin, des pénétrations peuvent se rapprocher voire s'empiler ; la gestion dédiée (retrait, redistribution, terminaisons) n'est **pas** implémentée. C'est une des raisons du statut *expérimental* du satin.

Outil de diagnostic

`openstitch-cli stitchdebug --shape circle|line|corner|bezier|star|ring --length 3 --output-svg out.svg` génère les points sur une forme de référence, affiche les statistiques (points, longueur, segment min/max) et un SVG. Utilisé pour valider le moteur (voir *Tests*).

Implémentation associée

- `libs/geometry/src/polyline.cpp` — `flatten`, longueur d'arc, `resample_run`.

- `libs/stitch_generation/src/running_stitch.cpp` — `run_stitch`, `apply_repeat_mode`, `apply_repeats`, `sample_path`.
- `apps/cli/main.cpp` — sous-commande `stitchdebug`.
- Docs de conception : `docs/stitch-engine-audit.md`, `docs/stitch-engine-research.md`.
- Tests : `tests/unit/geometry/test_polyline.cpp`, `tests/unit/stitch/test_running_stitch.cpp`.

Moteur de génération de points

Public : développeur, utilisateur avancé.

> État : Présent dans le code : oui · Tests unitaires : oui (running/tatami/ satin/routage/retouches) · Tests visuels : partiels (SVG de diagnostic, golden) · Import/export DST : oui · Test sur machine réelle : **non** · **Statut recommandé : hétérogène** — la fondation commune (`stitch`, l'orchestration de `generate_sequence`, le contrat `effective_sequence`) est solide et testée ; le point droit est implémenté ; le tatami est partiel ; le satin reste expérimental (voir *Colonne satin* pour le détail). Ce chapitre documente le **mécanisme**, pas la qualité du résultat cousu — voir *Limitations* pour la synthèse par capacité.

Ce chapitre est la référence unifiée du moteur qui transforme un `document::Project` en `std::vector<stitch::StitchCommand>` exportable en DST. Il couvre l'intégralité de `libs/stitch_generation/` (module `stitch_generation`, cf. *Référence des modules*) et son vocabulaire commun dans `libs/stitch/`. Les chapitres *Génération de points — point droit et fondations*, *Remplissage tatami* et *Colonne satin* restent les références narratives (audits, historique des correctifs, chiffres mesurés) pour chaque type de point ; ce chapitre les cite plutôt que de les dupliquer, et se concentre sur ce qu'aucun des trois ne couvre en un seul endroit : le contrat d'entrée (`generate_sequence` vs `effective_sequence`), le vocabulaire commun des commandes/passes, l'algorithme exact qui transforme des rails + barreaux en points cousus (Lots 2 à 6 du satin, jamais rassemblés ailleurs), et la place de la génération dans le pipeline complet (retouches, ordre, export).

1. Vue d'ensemble : de l'objet à l'export

```
document::Project
 ■■ optimization::optimize_order()      réordonne project.embroidery_objects
      (document, AVANT génération)       (commande annulable ReorderEmbroideryCommand)
 ■■ stitch_generation::generate_sequence(project)   -- séquence BRUTE
      ■■ pour chaque EmbroideryObject visible, dans l'ordre du document :
           RunningStitchParams -> generate_running (run_stitch + apply_repeats)
           TatamiParams        -> generate_tatami  (tatami_underlay + fill_tatami)
           SatinParams         -> generate_satin ou generate_satin_group (routage)
           DirectionalFillParams -> generate_directional (directional_underlay + fill_directional)
      ■■ ColorChange entre deux objets de couleurs différentes, End final
 ■■ stitch_generation::apply_manual_overrides(sequence, project)  -- retouches Lot 8
 ■■ stitch_generation::finish_sequence(sequence, project)  -- finitions (coupes,
      points d'arrêt, points courts ; project.finishing, Lots E/F)
 = stitch_generation::effective_sequence(project)   -- SEUL point d'entrée de production
      ■■ apps/desktop : aperçu (renderStitches), simulation, statistiques (stitch::compute_stats)
      ■■ formats::write_dst_file  -- export DST (voir *Format DST*)
```

Deux fonctions pures cohabitent, et la distinction est le contrat le plus important du module :

- **`generate_sequence(project)`** (`generate.hpp`/`generate.cpp`) parcourt le document et produit la séquence **brute** : uniquement ce que la géométrie actuelle des objets dicte, sans aucune retouche manuelle.
- **`apply_manual_overrides(sequence, project)`** (`overrides.hpp`/`overrides.cpp`) patche **en place** une séquence déjà générée avec les retouches (`EmbroideryObject::overrides`, Lot 8/ADR-014 — déplacement de point, Stitch↔Jump, coupe de fil) des objets dans l'état `ManuallyEdited`.
- **`finish_sequence(sequence, project)`** (`finish.hpp`/`finish.cpp`, Lots E/F de l'audit marine plein cadre) ajoute les finitions réglées par `project.finishing` (`document::SequenceFinishing`) : tout déplacement plus long que `trim_threshold` (3 mm) devient point d'arrêt de sortie, `Trim`, déplacement, point d'arrêt d'entrée ; un changement d'objet reçoit toujours une coupe (quelle que soit la distance) et un point d'arrêt de sortie et d'entrée (passe `Lock`, `lock_stitches`, posé le long du point voisin du tracé) ; une coupe précède chaque changement de fil (`trim_before_color_change`). Un objet qui porte déjà

ses verrous (satin `lock_start`/`lock_end`) n'est pas doublé. Elle ne modifie jamais la vue brute (`raw_slice`) : les index des retouches restent valides. Un projet antérieur (sans bloc `finishing` dans le .osp) est relu avec `enabled = false` : séquence identique à avant.

- **`effective_sequence(project)`** enchaîne les trois, dans cet ordre, et rien d'autre — sa signature est volontairement identique à `generate_sequence` pour qu'un appelant existant n'ait qu'à substituer l'appel.

Le contrat : personne ne contourne les retouches

CLAUDE.md et le code sont explicites : **tout consommateur de production** (aperçu desktop, export DST, simulation, statistiques, CLI) doit appeler `effective_sequence`, jamais `generate_sequence` seul — sans quoi les retouches manuelles d'un utilisateur disparaîtraient silencieusement de l'aperçu ou de l'export. Ce contrat n'est pas qu'une convention documentée : il est **imposé structurellement** par `tests/check_no_raw_sequence_bypass.cmake`, un script CMake exécuté comme test CTest qui :

1. parcourt tous les `.cpp`/`.hpp` de `apps/` et `libs/` ;
2. ignore `libs/stitch_generation/` (implémentation interne légitime : `effective_sequence` doit bien appeler `generate_sequence`) et `tests/` (fixtures synthétiques, jamais un projet utilisateur réel) ;
3. échoue avec le fichier:ligne exact si un site d'appel restant invoque `generate_sequence(` sans qu'un commentaire `raw-sequence-ok: <raison>` apparaisse sur la même ligne ou la ligne précédente.

Deux sites — et seulement deux, à ce jour — portent cette annotation, tous deux dans `apps/cli/main.cpp` : `run_stitchdebug` (ligne 181, "Projet synthétique construit ici même, jamais chargé/sauvegardé, aucune retouche manuelle possible") et le générateur de debug du routage satin (ligne 534, même justification). Les deux construisent un `document::Project` synthétique en mémoire, uniquement pour visualiser un algorithme sur une forme de référence — un projet qui, par construction, ne peut porter aucun `StitchOverride`. C'est la seule catégorie d'exception prévue par le mécanisme : un vrai projet utilisateur (chargé d'un `.osp`, édité dans l'interface) ne doit jamais transiter par `generate_sequence` seul.

2. Vocabulaire commun (`libs/stitch/`)

`stitch::StitchCommand` (`libs/stitch/include/openstitch/stitch/sequence.hpp`) est l'unité atomique de toute séquence, qu'elle soit générée, importée d'un DST ou retouchée :

```
struct StitchCommand {
   Vec2um pos;
   CommandType type;   // Stitch, Jump, Trim, ColorChange, Stop, End
   ObjectId source;     // 0 = manuel/importé
   StitchPass pass;      // Underlay, TopStitch, Travel, Lock, Manual
};
```

`CommandType` est la sémantique **machine** (ce qui finit dans le DST, cf. *Format DST*) ; `StitchPass` est une métadonnée **logique**, ajoutée par ce moteur et **jamais sérialisée** (la séquence est toujours régénérée, ADR-014 — le DST l'ignore complètement). Elle permet d'afficher, filtrer et analyser les sous-couches indépendamment de la couche supérieure :

Passe	Origine	Exemples
`Underlay`	sous-couches, émises **avant** la couche supérieure	`tatami_underlay`, sous-couches satin (center/edge/zigzag)
`TopStitch`	couche visible finale	running stitch, `fill_tatami` hors sauts, zigzag satin principal
`Travel`	déplacement — cousu et caché, ou saut	pénétrations intermédiaires d'un underpath (tatami §15, satin routé Lot 6), et le `Jump` de tête de chaque polyligne (`emit_polyline`)
`Lock`	point de fixation	`lock_stitches` (satin, Lot 5)
`Manual`	modifié par une retouche	`apply_manual_overrides` : position déplacée ou type forcé

Point précis souvent mal lu : `Travel` couvre **deux réalités opposées** — un `Jump` (aiguille levée, aucune couture) *et* une pénétration `Stitch` cousue et cachée sous la couche supérieure (underpath). Distinguer les deux exige de regarder `CommandType`, pas seulement `StitchPass` : un point `{Travel, Jump}` est un vrai saut, un point `{Travel, Stitch}` est cousu. L'outil de diagnostic desktop (`MainWindow::buildDebugDump`, menu contextuel *Diagnostic*) imprime justement `pass=<Passe> type=<Type> pos=<x,y>` pour chaque commande d'un objet, ainsi qu'un histogramme par passe (`Underlay=… TopStitch=… Travel=… Lock=… Manual=…`) — c'est la source du vocabulaire `pass=Underlay`/`pass=TopStitch`/… utilisé informellement ailleurs dans le projet.

`stitch::compute_stats`

`libs/stitch/src/stats.cpp` dérive `StitchStats` (points, sauts, coupes, changements de couleur, longueur de fil, boîte englobante) en un seul passage séquentiel :

- `stitches`/`jumps`/`trims`/`color_changes` : compteurs directs par `CommandType`.
- `thread_length_um` : somme de `length_um(cmd.pos - prev)`, ajoutée **uniquement** sur une commande `Stitch`, où `prev` est la position de la commande précédente **quel que soit son type** (`Stitch` ou `Jump` — `prev` est mis à jour dans les deux cas). Un saut ne contribue donc jamais directement à `thread_length_um`, mais la première `Stitch` qui le suit ajoute la distance depuis le point d'atterrissage du saut. En pratique cette distance est **nulle** : `emit_polyline` saute vers `points.front()` puis coud `points[0]`, qui **est** `points.front()` — le saut et la première couture partagent exactement la même position. `thread_length_um` mesure donc bien la longueur de fil réellement cousu, jamais la distance des sauts eux-mêmes, mais ce n'est vrai que parce que chaque émetteur respecte cette convention (aucune garantie structurelle au niveau de `compute_stats` lui-même).
- `bounds` : boîte englobante des positions `Stitch` **uniquement** — un saut loin du motif (retour au point d'origine avant un `Trim`, par exemple) n'élargit pas le cadre rapporté.

3. Orchestration : `generate_sequence`

`generate_sequence` (`libs/stitch_generation/src/generate.cpp`) itère sur `project.embroidery_objects` dans l'ordre du document (l'ordre qu'*Ordre de couture* — §7 ci-dessous — a éventuellement déjà réarrangé en amont, au niveau du document, pas ici) :

1. les objets `!visible` sont ignorés (index avancé, rien émis) ;
2. pour un type non-satin, l'objet vectoriel source (`source_vector`) est recherché dans `project.vector_objects` ; absent, `generate_sequence` renvoie une erreur `ErrorCategory::Internal` nommant l'objet — la seule voie d'échec de la génération en dehors d'un document totalement vide ;
3. un `ColorChange` est inséré si l'objet précédent existait et avait une couleur RGB différente ;
4. **routage satin (§6.5)** : si l'objet est un satin *routable* (`is_routable_satin` — satin dont `SatinParams::rungs.size() >= 2`, donc issu de l'auto-satin ou d'un satin manuel avec barreaux par défaut), le moteur cherche le plus long groupe **contigu** d'objets satin routables suivants qui partagent **exactement** la même couleur (`rgb`) et le même `source_vector`. Un groupe de taille ≥ 2 part dans `generate_satin_group` (§6.5) ; un groupe de taille 1 (satin routable isolé, ou dernier de sa série) suit le chemin normal `generate_satin`. Un satin manuel/legacy sans barreaux n'est jamais groupé, quelle que soit sa couleur ;
5. sinon, `std::visit` sur `object.params` dispatch vers `generate_running`, `generate_tatami`, `generate_satin` ou `generate_directional` (§4, §5, §6 ; le directionnel a son chapitre, *Remplissage directionnel*).

Une fois tous les objets traités, si la séquence reste vide (aucun objet visible n'a rien produit — cas trivial : document vide, ou uniquement des objets masqués), l'erreur `ErrorCategory::UserInput` "Aucun objet de broderie visible : rien à générer" est renvoyée. Sinon, une commande `End` est ajoutée à la position de la dernière commande.

Les trois émetteurs de polylignes (vocabulaire interne de `generate.cpp`)

Trois fonctions statiques traduisent une géométrie déjà calculée en commandes, partagées par les trois générateurs :

- **`emit_polyline(seq, points, source, pass)`** : un `Jump` vers `points.front()`, puis chaque point en `Stitch`. Le saut est **omis** si la toute dernière commande est déjà un `Stitch` à la même position (enchaînement direct depuis une passe précédente du même objet — sous-couche → lock → satin) ; un saut est toujours émis après un `ColorChange`, un autre `Jump`, ou entre deux objets différents, même à position identique.
- **`emit_polyline_with_breaks(seq, points, jump_before, source, pass)`** : identique, mais certains points internes (indices listés dans `jump_before`, triés) sont eux aussi atteints par un `Jump` plutôt qu'enchaînés — c'est le mécanisme qui matérialise `SatinResult::jump_before` (§6.4) dans la séquence machine.
- **`emit_fill(seq, fill, source)`** : consomme un `std::vector<FillStitch>` (tatami, §5) — `fs.jump` devient `Jump`/`Travel`, `fs.travel` devient `Stitch`/`Travel` (underpath caché), sinon `Stitch`/`TopStitch`. Le tout premier point est toujours un `Jump`, même si `fs.jump` vaut `false` (`!started` force la branche `Jump`).

4. Point droit (running stitch) — synthèse

Couvert en détail par *Génération de points — point droit et fondations* ; voici la synthèse tendue nécessaire pour situer ce point dans le moteur, avec deux corrections vérifiées contre le code actuel.

`stitch_generation::run_stitch(path, RunningConfig)` (`libs/stitch_generation/src/running_stitch.cpp`) : aplatit la Bézier (`geometry::flatten`, tolérance `flatten_tolerance` = 100 µm par défaut) → détecte les coins vifs (angle de rotation > `corner_threshold`, ≈ 0,6108652 rad = 35° par défaut) → découpe en tronçons entre coins → ré-échantillonne **chaque tronçon** par longueur d'arc (`geometry::resample_run`, dans `libs/geometry/src/polyline.cpp`). `generate_running` (`generate.cpp`) appelle `sample_path` (délègue à `run_stitch`, un seul passage) puis `apply_repeats` pour chaque contour extérieur et chaque trou de l'objet vectoriel source.

Correction apportée à *Génération de points…* : ce chapitre présente le comportement "cible traitée comme un maximum" (`ceil(L/cible)` plutôt que `round`) comme *"un point d'ambiguïté à clarifier dans une future version (distinguer `target_length` de `maximum_length`)"*. À la lecture du code actuel, cette clarification a déjà eu lieu **partiellement** : le champ s'appelle désormais `RunningConfig::target_length` (plus `stitch_length`), et des champs `min_length`/`max_length` distincts existent bel et bien. Mais le choix `ceil` reste, et le commentaire à côté de `geometry::resample_run` (`libs/geometry/src/polyline.cpp`) le qualifie explicitement d'**assumé**, pas d'ouvert : *"Écart assumé (...) ; `ceil` garantit en plus le respect strict de la longueur maximale."* Le comportement observable — les points ne dépassent jamais `target_length` mais peuvent être plus courts — est donc un choix de conception délibéré et documenté dans le code, pas une ambiguïté résiduelle ; seul le nom du champ dans la doc narrative (`stitch_length`) est à mettre à jour vers `target_length`.

`apply_repeat_mode` (`RepeatMode::SinglePass`/`BackAndForth`/`BeanStitch`/ `Backstitch`) et son alias de compatibilité `apply_repeats(points, int)` restent inchangés par rapport à la description existante.

5. Remplissage tatami — synthèse

Couvert en détail par *Remplissage tatami* ; synthèse tendue, vérifiée sans divergence notable contre `libs/stitch_generation/src/tatami.cpp`.

`fill_tatami(region, TatamiParams)` : rotation virtuelle de `-angle` → balayage de rangées horizontales espacées de `row_spacing` (400 µm par défaut) → intersections pair-impair par rangée → pénétrations sur une grille de pas `stitch_length` (3 mm par défaut), déphasée d'une rangée à l'autre selon `stagger` (2 par défaut) → les segments de rangées voisines qui se chevauchent en x sont les arêtes d'un graphe, parcouru par un algorithme glouton (`current` → voisin non visité le plus proche, sinon saut vers le segment non visité le plus proche) → rotation inverse. Chaque liaison cousue (dans une rangée ou entre rangées) est

validée géométriquement par `connector_invalid` : le segment est découpé à **chaque** intersection paramétrique avec une arête du contour ou d'un trou (y compris un contact dégénéré par un sommet), et le milieu de chaque sous-segment doit rester dans la région (`in_region`, qui traite un point exactement sur une frontière comme intérieur, tolérance ~0,01 µm) — sans quoi la liaison devient un saut. Ce mécanisme est ce qui garantit qu'aucune couture ne traverse un trou.

Tatami avancé (Lot 7, tout désactivé par défaut) : `tatami_underlay` produit un contour rentré (`underlay_edge`, retrait `underlay_inset` = 600 µm — jamais sur les bords de trous, politique sûre si le retrait échoue) et des rangées perpendiculaires espacées (`underlay_parallel`, pas `underlay_spacing` = 2 mm) ; `hidden_underpath` remplace un saut par un trajet cousu caché (passe `Travel`) — direct s'il reste court (`seglen(prev, rp) <= underpathCap`, avec `underpathCap = max(6·row_spacing, 8 mm)`), sinon le long du contour extérieur rentré (`routeHighway`, qui essaie les deux sens de parcours du contour et garde le plus court sous le même plafond), toujours revalidé par `connector_invalid` segment par segment.

Aucune divergence factuelle trouvée entre *Remplissage tatami* et le code actuel — la chaîne d'implémentation `fill_tatami`/`tatami_underlay`/ `connector_invalid`/`in_region` correspond exactement à ce que ce chapitre décrit, chiffres compris.

6. Colonne satin : des rails aux points cousus (Lots 2 à 6)

Colonne satin documente en détail l'**appariement géométrique** (`ladder_correspondence`, l'algorithme "ladder" qui remplace l'ancienne correspondance par fraction d'abscisse) et tout le moteur `auto_satin` qui produit des rails et des barreaux à partir d'un squelette. Ce chapitre-ci ne reprend pas cette histoire — il documente précisément ce qui se passe **une fois que `SatinParams` (rails + barreaux) est connu**, jusqu'aux commandes machine : c'est le contenu des Lots 2 à 6 de `libs/stitch_generation/src/satin.cpp`, `lock.cpp` et `routing.cpp`, jamais rassemblé en un seul endroit ailleurs dans la documentation.

6.1. Deux chemins de génération, un seul point d'entrée

`generate_satin` (`generate.cpp`) construit un `SatinConfig` à partir de `document::SatinParams` (copie champ à champ des Lots 3/4, cf. tableau §6.6) puis choisit :

- **`fill_satin_columns(rail_a, rail_b, rungs, config)`** si `params.rungs.size() >= 2` — c'est le chemin qui implémente **l'intégralité** de `SatinConfig` (terminaisons, split, sous-couches de bord/zigzag, compensation push/pull asymétrique). Depuis `default_rungs` (voir *Colonne satin* §*Barreaux par défaut*), c'est le chemin emprunté par la quasi- totalité des satins de l'application, y compris manuels.
- **`fill_satin(rail_a, rail_b, config)`** sinon (satin legacy sans barreaux, compatibilité ascendante uniquement) — n'implémente qu'un **sous-ensemble** : densité, compensation pull **symétrique**, sous-couche centrale. Tout autre réglage exposé par l'inspecteur (terminaisons, split, sous-couches de bord/ zigzag, push, compensation asymétrique) reste silencieusement **sans effet** sur ce chemin — limitation déjà documentée dans *Colonne satin*, reconfirmée ici en lisant `fill_satin` : sa boucle d'émission ne référence ni `cap_start`/`cap_end`, ni `split_stitch`, ni `underlay_edge`/`underlay_zigzag`, ni `pull_left`/`pull_right`/`push_start`/`push_end`.

6.2. `satin_stations` : la correspondance structurelle

`fill_satin_columns` délègue d'abord à `satin_stations(rail_a, rail_b, rungs, density)` (exposée publiquement dans `satin.hpp`, réutilisée par `satin_coverage`) : aplatit les deux rails (`geometry::flatten`, tolérance `kRailFlattenTolerance` = 30 µm — sous la résolution DST de 100 µm, donc sans perte perceptible), réoriente `rail_b` si les rails sont fournis tête-bêche (`opposite_orientation`), projette chaque barreau sur les deux rails (abscisses curvilignes), **trie** les barreaux par abscisse projetée cumulée (`sa + sb`) — l'ordre du vecteur d'entrée n'est jamais signifiant — puis **fusionne** deux barreaux consécutifs après tri dont la progression sur les deux rails est inférieure à la moitié du pas de densité (`anchorMinGap = density / 2`), pour éliminer tout intervalle dégénéré.

Moins de deux barreaux après fusion → liste vide (l'appelant retombe sur `fill_satin`). Sinon, pour chaque paire de barreaux consécutifs :

- si `non_ribbon_interval(a0, a1)` est vrai (§6.4), aucune station intermédiaire n'est produite : seul le barreau d'arrivée est ajouté, marqué `jump_before = true` ;
- sinon, `ladder_correspondence` puis `resample_by_medial_spacing` (voir *Colonne satin* pour l'algorithme complet) produisent les stations intermédiaires à l'espacement `density` (400 µm par défaut), mesuré sur la **ligne médiane** — donc approximativement perpendiculaire au fil, y compris en section inclinée ou courbe, jamais mesuré le long d'un rail.

6.3. `fill_satin_columns` : ordre exact des transformations

Une fois les stations connues, chacune devient un `Thread{a, b, anchor, jump_before}`. Les transformations s'enchaînent **dans cet ordre précis** — ordre qui a des conséquences directes sur le résultat (voir la note sur les sous-couches ci-dessous) :

1. **Terminaisons** (`cap_start`/`cap_end`, `SatinCapType`, §6.6) : pour chaque fil `i`, `f = min(cap_factor(cap_start, i, cap_length), cap_factor(cap_end, nThreads-1-i, cap_length))` — le **minimum** des deux facteurs, pour qu'une colonne trop courte pour loger les deux terminaisons sans chevauchement reste cohérente (jamais deux rétrécissements indépendants qui se contrediraient). Si `f < 1`, `a` et `b` sont chacun ramenés vers le milieu du fil par interpolation (`lerp(milieu, point, f)`). `cap_factor` : `Flat` **et `Automatic`** renvoient tous deux `1.0` (aucun rétrécissement) — la branche `default:` du `switch` couvre les deux ; en l'état actuel du code, **`SatinCapType::Automatic` ne se comporte pas différemment de `Flat`**, malgré son nom qui suggère un choix adaptatif. C'est une lacune non documentée ailleurs : l'inspecteur propose les deux options sans qu'aucune différence de résultat ne soit produite. `Tapered` suit une rampe linéaire de `kMin` (0,18 — jamais 0, pour ne pas empiler sur une coordonnée unique) à 1 ; `Rounded` suit un quart de sinusoïde.
2. **Points courts** (`short_stitch`, §6.6), si activé : pour chaque fil `i ≥ 1` (jamais sur un barreau, `t.anchor` protégé), compare l'avance de chaque rail depuis le fil précédent (`advA`, `advB`) ; si le ratio `min/max < short_stitch_curvature` (0,55 par défaut) et que l'avance minimale est sous `short_stitch_min_gap` (250 µm), le virage est jugé "serré". `RemoveAndRedistribute` marque le fil `dropped` un fil sur deux (jamais deux d'affilée) ; `SingleInset`/`MultiLevelInset` rentrent le point du rail intérieur vers le milieu d'une fraction `short_stitch_inset` (0,35 par défaut), triangulaire sur `short_stitch_levels` (2 par défaut) niveaux en mode `MultiLevelInset`.
3. **Push** (`push_start`/`push_end`, §6.6) : `shiftEnd` étend/rétracte le premier et le dernier fil le long de l'axe vers leur voisin immédiat. Une rétraction (`amount < 0`) est bornée à `-(n-1)` (`n` = distance au voisin) : elle ne peut jamais dépasser la station voisine, ce qui éviterait l'auto-croisement documenté dans *Colonne satin* §*Sous-couches et compensation*. Une extension (`amount > 0`) reste **non bornée**.
4. **Milieux** (`mids`) et longueur cumulée médiane (`cumMid`) sont calculés **après** les trois étapes précédentes — donc sur la géométrie déjà terminée/rentrée/étendue.
5. **Sous-couches** (`center_underlay`, `underlay_edge`, `underlay_zigzag`, §6.6), dans l'ordre center → edge → zigzag, chacune une passe `Underlay` distincte : la sous-couche centrale est ré-échantillonnée à `underlay_spacing` (2 mm par défaut, **pas** à `density`, correctif détaillé dans *Colonne satin* §*Défaut trouvé en usage réel*) via `point_at` sur `mids`/`cumMid`. L'edge walk décale chaque point vers l'intérieur de `min(underlay_edge_inset, demi-largeur × 0,9)`. Le zigzag prend un fil sur `stride = round(underlay_zigzag_spacing / density)` (indices, pas longueur d'arc — approximation valable car les fils sont déjà espacés régulièrement par construction, sauf immédiatement après un `jump_before`).
6. **Couche supérieure** : pour chaque fil non `dropped`, si `jump_before`, l'indice courant est ajouté à `result.jump_before` (§6.4) ; **la compensation pull latérale est calculée ici, pas avant** — donc les sous-couches (étape 5) voient la largeur **avant** compensation, la couche supérieure après. `split_stitch` (§6.6) fractionne toute traversée `a→b` plus longue que `max_stitch_length` (7 mm par défaut).

6.4. Rupture de ruban : `non_ribbon_interval` et `jump_before`

Entre deux barreaux consécutifs, `non_ribbon_interval(a0, a1, b0, b1)` (`libs/stitch_generation/src/satin.cpp`) compare la **direction d'avance** de chaque rail (`a1.pa - a0.pa` vs `a1.pb - a0.pb`), pas leur largeur : si les deux avances dépassent `kJumpMinSpan` (800 µm — sous ce plancher, la variation angulaire n'est pas significative) et que l'angle entre les deux directions dépasse `kJumpDegPerMm` (10°/mm) **multiplié par la distance moyenne parcourue en mm**, la colonne cesse de se comporter comme un ruban sur cet intervalle. Le seuil est un **rapport angle/distance**, pas un angle brut — le détail de calibration (deux défauts réels distincts, un seuil brut se révélant incapable de distinguer un virage légitime étalé sur ~14 mm d'un défaut concentré sur 1,5-3,7 mm) est documenté dans *Colonne satin* §*Seuil angle/distance plutôt qu'angle brut*.

Quand c'est le cas, `satin_stations` n'interpole **aucune** station intermédiaire sur cet intervalle et marque le barreau d'arrivée `jump_before = true`. `fill_satin_columns` reporte cet indice dans `SatinResult::jump_before`. `generate_satin` transmet ce vecteur à `emit_polyline_with_breaks` (§3), qui insère un `Jump` avant ces points précis plutôt que d'enchaîner un `Stitch` continu — le fil est levé, pas forcé à traverser la forme en diagonale.

Réorientation et `jump_before` : quand `generate_satin` inverse la colonne (§6.6, entrée/sortie), les indices de `jump_before` doivent être recalculés — un index `i` dans la séquence d'origine correspond à `n - i` après inversion (`n` = taille totale), car c'est l'**arête** (pas le point) qui doit rester marquée comme sautée, et l'arête `(i-1, i)` devient `(n-i, n-i-1)` après inversion. Le code recalcule chaque indice puis retrie le vecteur (`generate.cpp`, fonction `generate_satin`) — sans ce retri, l'ordre croissant qu'`emit_polyline_with_breaks` suppose serait rompu.

6.5. Points de fixation (`lock_stitches`, Lot 5)

`lock_stitches(anchor, toward, type, length, passes)` (`libs/stitch_generation/src/lock.cpp`) construit une polyligne courte ancrée en `anchor`, orientée vers `toward` (le point voisin — pour un lock de colonne satin, c'est le point du **rail opposé**, donc la largeur locale de la colonne à cet endroit, pas un point lointain le long de la couture). La longueur effective `L` est bornée : `L = min(length, n)` où `n = |toward - anchor|`, sauf dans le cas dégénéré `anchor == toward` où `L = length` sans autre référence géométrique — correctif détaillé dans *Colonne satin* §*Entrée/sortie et points de fixation*, qui évitait qu'un lock pique hors de la matière déjà cousue sur une colonne plus fine que `lock_length` (défaut 0,8 mm). Trois types, chacun répété `passes` fois (2 par défaut) :

- **`BackAndForth`** : `anchor → P(L,0) → anchor`, répété.
- **`Triangle`** : `anchor → P(L,0) → P(L·0,5, L·0,6) → anchor`, répété — un petit triangle d'ancrage.
- **`MicroZigzag`** : `anchor` puis `2·passes` points alternant de côté (`P(L·0,4·■k/2■, ±L·0,5)`), progressant le long de l'axe, puis retour à `anchor`.

`generate_satin` émet, dans cet ordre : sous-couches → lock de début (passe `Lock`, entre `result.satin.front()` et `result.satin[1]`, si `lock_start != None`) → couche supérieure (avec ses éventuels `jump_before`) → lock de fin (entre `result.satin[n-1]` et `result.satin[n-2]`, si `lock_end != None`). Un seul lock par extrémité, jamais par sous-passe.

6.6. Entrée/sortie, et tableau des paramètres du satin

Si `entry_point` et/ou `exit_point` sont définis, `generate_satin` compare le coût "sens normal" (`|entry - satin.front()| + |exit - satin.back()|`) au coût "sens inversé" (`|entry - satin.back()| + |exit - satin.front()|`) ; le sens inversé l'emporte strictement en cas d'égalité non stricte du côté normal (`reversed < normal`). Inverser la séquence exige de recalculer `jump_before` (§6.4). Sans point d'entrée ni de sortie, l'orientation issue du générateur (celle des rails tels qu'orientés dans le document) est conservée telle quelle.



6.7. Routage multi-colonnes (`route_columns`, Lot 6)

Quand `generate_sequence` regroupe plusieurs colonnes satin routables contiguës de même couleur/source (§3 point 4), `generate_satin_group` (`generate.cpp`) construit un `RouteColumn{id, start, end, start_junction, end_junction}` par objet — `start`/`end` viennent de `column_endpoints(params)` (milieux des barreaux d'about, **avec le même décalage `push_start`/`push_end` que celui réellement appliqué par `fill_satin_columns`**, même formule de borne : reproduire un point différent de celui effectivement cousu fausserait la décision de routage, cf. le correctif détaillé dans *Colonne satin* §*Routage multi-colonnes*) — et `start_junction`/`end_junction` viennent de `SatinParams::topology` (`SatinSectionTopology::start_junction`/`end_junction`, un identifiant de jonction **local au réseau porté par `source_vector`**, absent pour une extrémité libre ou une colonne indépendante/legacy).

`route_columns(columns, origin, RoutingConfig)` (`libs/stitch_generation/src/routing.cpp`) :

1. **Ordre glouton** (`greedy_order`) : depuis la position courante, choisit l'extrémité libre la plus proche — mais une extrémité qui **partage une jonction** avec la sortie courante (même id, distance ≤ `underpath_max`) l'emporte systématiquement sur une simple proximité, même plus courte.
2. **Orientation optimale** (`best_orientation`) : pour l'ordre retenu, programmation dynamique sur les deux orientations possibles de chaque colonne (`std::array<OrientationCost, 2>`) — maximise d'abord le nombre de liaisons validées par une jonction commune, minimise ensuite la distance totale parcourue. Déterministe, résolu **exactement** (pas une heuristique).
3. **Amélioration 2-opt** (si `config.two_opt`, activé par défaut) : inversion de sous-segments de l'ordre tant que le coût diminue, avec une marge anti-oscillation de 1 µm, bornée à 64 itérations de balayage complet.
4. Pour chaque étape, le **type de liaison** (`ConnectorKind`) avec la colonne précédente est décidé par la distance entre le point de sortie précédent et le point d'entrée courant : - une **jonction commune validée** (même id des deux côtés, ET distance ≤ `underpath_max` — 8 mm par défaut) autorise un `Underpath` jusqu'à cette même borne de 8 mm ; - en son **absence**, seul un quasi-contact sous `underpath_max_without_junction` (1,5 mm par défaut, nettement plus strict) autorise un `Underpath` ; - au-delà de la borne applicable, la liaison devient un `Jump` (avec coupe implicite — `ConnectorKind::Jump` porte déjà cette sémantique dans sa propre définition, il n'existe pas de "trim" distinct à ce niveau).

`ConnectorKind::Start` marque uniquement la toute première colonne du groupe (simple pose du fil, jamais de décision Underpath/Jump). Le vocabulaire diffère légèrement de celui employé par la spécification SGSD (`libs/satin_planning/`, voir *Colonne satin* §*Phase 9*) — "travel run" y désigne ce que ce module appelle `Underpath` — mais recouvre exactement le même mécanisme ; SGSD ne réimplémente aucune logique de routage, il construit des `RouteColumn` et délègue à ce même `route_columns`.

Émission (`generate_satin_group`) : pour chaque étape du plan, la colonne est générée normalement (`generate_satin` dans une séquence temporaire, avec `entry_point`/`exit_point` imposés par `step.reversed`) puis :

- si `step.connector == Underpath` et qu'une commande précédente existe déjà, un segment `[position courante → première pénétration de la colonne]` est échantillonné par `sample_path` (cible 2,5 mm, minimum 500 µm), et **seuls les points intermédiaires** (ni le premier ni le dernier, déjà couverts respectivement par la position courante et la colonne elle-même) sont ajoutés en `Stitch`/`Travel` — puis la colonne est enchaînée **sans** son `Jump` de tête (`tmp.commands[1..]`) ;
- sinon (début de groupe, ou liaison trop longue), la colonne est ajoutée telle quelle, `Jump` de tête compris.

7. Ordre de couture et export — où ce moteur s'arrête

`optimization::optimize_order` (`libs/optimization/`, voir *Algorithmes* §*Ordre de couture*) n'est **pas** un post-traitement de la séquence de points : il réordonne la **liste d'objets du document** (`project.embroidery_objects`, via la commande annulable `ReorderEmbroideryCommand`, `MainWindow::applyOrderStrategy`), les objets verrouillés (`EmbroideryObject::locked`) restant à leur position. La génération de points est ensuite **relancée** sur ce nouvel ordre — il n'existe aucun mécanisme qui réordonnerait un `stitch::StitchSequence` déjà produit. Concrètement, dans le desktop, `applyOrderStrategy` exécute la commande de réordonnancement puis appelle `refreshImage()`, qui appelle `stitch_generation::refresh_context` (donc `effective_sequence`, §1) sur le projet déjà réordonné.

L'export DST (`formats::write_dst_file`, voir *Format DST*) consomme directement la séquence effective ainsi obtenue (`MainWindow::exportDst`, membre `sequence_` rempli par `refreshImage`) — la génération de points est donc strictement **en amont** de l'ordre optimisé (qui agit sur le document avant régénération) et **en amont** de l'export (qui ne transforme plus la séquence).

8. Retouches manuelles : ce que `apply_manual_overrides` touche exactement

Détaillé pour l'utilisateur dans *Retouche des points et de la géométrie* ; ce qui suit est le contrat exact côté moteur, nécessaire pour comprendre pourquoi `effective_sequence` est incontournable (§1).

Une retouche (`document::StitchOverride{base_index, moved_to, forced_type, trim_after}`) cible un index dans la **vue brute** d'un objet (`raw_slice(sequence, object.id)` — toutes les commandes de la séquence brute dont `source == object.id`, dans l'ordre de production, **toutes passes confondues**, qu'elles soient contiguës ou entrelacées comme dans un satin routé). Seules les entrées de passe `TopStitch` sont éligibles (`is_topstitch_entry`) — les sous-couches, trajets cachés et locks restent entièrement régénérés, jamais retouchables :

- `moved_to` exige en plus une cible `Stitch` (`is_movable_point`) — déplacer un saut n'a pas de sens, sa position est déterminée par ses deux voisins de trajet ;
- `forced_type` (bascule `Stitch`↔`Jump`, bidirectionnelle) et `trim_after` acceptent une cible `Stitch` **ou** `Jump`.

Chaque champ d'un `StitchOverride` est validé **indépendamment** : un champ invalide pour sa cible est ignoré silencieusement, sans rejeter les autres champs du même override. Une commande modifiée passe en passe `Manual`. Les insertions de `Trim` sont différées et appliquées après coup, triées par index décroissant, pour qu'aucune insertion ne décale un index restant à traiter.

`classify_edit_state(object, raw)` compare la taille et l'empreinte FNV-1a 64 bits (`fingerprint`, sérialisation little-endian explicite de position/ type/passe/source de chaque commande — reproductible bit à bit, indépendante de l'architecture hôte) de la vue brute **actuelle** contre `edited_point_count`/`edited_fingerprint` enregistrés lors de la dernière retouche réussie : identiques → `ManuallyEdited` (les retouches s'appliquent) ; différents → `Dirty` (la géométrie source a changé depuis, les retouches ne sont **plus jamais réappliquées** tant que l'utilisateur ne les abandonne pas explicitement) ; `overrides` vide → `Clean`. Rien n'est mis en cache : cet état est recalculé à chaque appel depuis le document et la séquence brute actuelle.

`edit_view`, `classify_all_edit_states` et `refresh_context` sont des compositions de ces mêmes primitives pour l'UI desktop (indicateurs Clean/ManuallyEdited/Dirty, poignées d'édition) — `refresh_context` en particulier consolide un **seul** appel interne à `generate_sequence` pour produire à la fois la séquence effective, les états par objet et la vue d'édition d'une cible, plutôt que jusqu'à trois régénérations indépendantes par rafraîchissement (`MainWindow::refreshImage`).

9. Limites connues (au niveau du moteur, au-delà de ce que documentent déjà satin.md/tatami.md)

- **`SatinCapType::Automatic` ne fait rien de différent de `Flat`** dans `cap_factor` (§6.3 pt 1) — la terminaison "automatique" promise par l'inspecteur n'a aucune implémentation propre à ce jour.
- **`center_underlay` a deux défauts différents** selon le niveau regardé : `false` dans `SatinConfig` (le type interne au moteur), `true` dans `document::SatinParams` (le type exposé par l'inspecteur) — sans conséquence pratique puisque le document fournit toujours une valeur explicite, mais un piège pour quiconque lit `SatinConfig` isolément en pensant y trouver le comportement par défaut réel d'un satin créé dans l'application.
- **Le zigzag de sous-couche satin** (`underlay_zigzag`) espace ses points par un **stride en nombre de fils** (`round(underlay_zigzag_spacing / density)`), pas par une distance réellement mesurée le long de la médiane — une approximation raisonnable tant que les fils restent régulièrement espacés, mais fausse localement juste après un `jump_before` (§6.4), où l'espacement réel entre deux fils consécutifs peut s'écarter fortement de `density`.
- **Le trajet caché d'un routage satin (`Underpath`) reste un segment direct échantillonné**, jamais un suivi du centre d'une colonne déjà cousue — la garantie de rester "sous la broderie" vient uniquement de la borne de distance (jonction validée ou quasi-contact), pas d'un vrai calcul de trajet le long de la matière (limite déjà posée dans *Colonne satin* §*Routage multi-colonnes*, confirmée ici au niveau du code d'émission).
- **Le contrat `raw-sequence-ok`** ne protège que les appels textuellement détectables par le script CMake (`generate_sequence(` sur une ligne) : un appel indirect via un alias de fonction ou un pointeur de fonction ne serait pas détecté. Aucun cas de ce genre n'existe actuellement dans le dépôt (vérifié par lecture des deux seuls sites annotés, §1), mais ce n'est pas une garantie statique complète, seulement une convention outillée.

10. Lot « moteur » : sous-couche automatique, entrée/sortie, longueurs, bordure

Tous ces réglages sont **désactivés par défaut** : un projet existant produit exactement les mêmes points tant qu'on n'y touche pas (vérifié par la suite de tests complète et ses fichiers dorés, inchangés).

Sous-couche automatique (HP-ENG-002)

`UnderlayMode::Auto` (tatami, directionnel, satin, auto-satin ; défaut `Manual`) fait choisir la sous-couche par le moteur (`underlay_auto.cpp`, fonctions pures). Mesure d'une forme : aire, périmètre et épaisseur moyenne `2·aire/périmètre` (largeur exacte d'une bande longue ; trous inclus dans le périmètre).

Objet	Condition	Sous-couche
Remplissage	aire < 6 mm² ou épaisseur < 0,8 mm	aucune
Remplissage	aire < 60 mm²	contour seul
Remplissage	aire ≥ 60 mm²	contour + rangées perpendiculaires (espacement 2 mm, 2,5 mm au-delà de 400 mm²)
Satin	largeur < 1,0 mm	aucune
Satin	1,0 à 3,5 mm	centre
Satin	3,5 à 7,0 mm	contour (deux chemins internes)
Satin	≥ 7,0 mm	contour + zigzag léger

Le retrait du contour de remplissage vaut un quart de l'épaisseur, borné à [0,3 ; 0,6] mm. En mode Auto, les cases manuelles sont ignorées (grisées dans l'inspecteur). Les nouveaux objets créés à la main restent en `Manual` : le mode Auto se choisit dans l'inspecteur (« Sous-couche : Automatique »).

Entrée/sortie automatiques (HP-ENG-010)

`SequenceFinishing::auto_join` (« Options de génération ») et, par objet, `EmbroideryObject::join` (Hérite / Automatique / Désactivée, commande annulable `SetEmbroideryJoinModeCommand`). Les points d'arrêt et coupes à l'entrée et à la sortie de chaque objet existaient déjà (`finish_sequence`) ; ce lot choisit le **sens** de couture. Pour un contour ou un remplissage, `orient_chunk` évalue les quatre combinaisons (sous-couche inversée ou non × couche supérieure inversée ou non) et garde celle qui minimise `|fin précédente → début| + ½·|fin → début suivant| + sauts internes` ; mêmes pénétrations, ordre inversé, sous-couche toujours avant la couche supérieure. Une colonne satin ou un auto-satin reçoit la fin de l'objet précédent comme point d'entrée (sens du générateur existant). Un point d'entrée explicite de l'objet est respecté. Les colonnes d'un groupe routé gardent le routage existant (origine = fin précédente). Mesure (`openstitch-cli engine-debug`, trois tatami empilés plus une bordure) : 116,0 mm de sauts → 102,6 mm. **Non fait** : l'ordre des objets (`libs/optimization`) juge encore par centres, pas par extrémités, et les contours fermés ne sont pas ré-attaqués au sommet le plus proche.

Longueurs min/max (HP-ENG-008)

`SequenceFinishing::split_long_stitches` + `max_stitch_length` (7 mm, borné à [1 ; 12,1] mm) découpent en points égaux sur la même droite tout point cousu trop long, pour **tous** les types ; les morceaux mesurent au moins la moitié du maximum. Le filtre des points courts (`min_stitch_length`) existait. Côté analyse, `stitch_analysis::options_from_project` fait juger les points avec ces limites, et `check_stitch_limits` avertit **avant génération** (catégorie `parametre-hors-limites`) d'un réglage d'objet hors plage.

Satin de bordure (HP-STI-004)

`border_satin_from_path` (`border_satin.cpp`) transforme un contour (fermé ou ouvert) en `SatinParams` à largeur constante : un barreau perpendiculaire par sommet aplati, rails décalés le long des normales, côté centré / intérieur / extérieur (la matière d'un trou est du côté opposé à son centre), coins vifs (onglet, biseau au-delà d'un rapport de 3) ou arrondis (arc, un barreau tous les 12°, éventail autour du point intérieur). Le côté intérieur du virage est ramené à la moitié de l'arête voisine : jamais de croisement de rails, mais une bordure plus large que le rayon de courbure local se déforme. Construction analytique (pas d'offset Clipper2) : déterministe et sans arrondi entier intermédiaire. Un satin par anneau (extérieur puis trous), jamais routé avec ses voisins. `BorderSatinSpec` mémorise largeur/côté/coins/anneau : l'inspecteur régénère les rails depuis le vecteur source (`regenerate_border_satin`, `SetStitchParamsCommand`) ; déplacer le vecteur déplace la bordure (`TranslateVectorObjectCommand`), la mise à l'échelle du vecteur demande de modifier un réglage de bordure pour régénérer. Création : inspecteur d'un objet vectoriel, « Créer le satin de bordure » (un seul pas d'annulation).

Implémentation associée

- `libs/stitch_generation/{border_satin,join,underlay_auto}.*`, `libs/stitch_analysis/src/limits.cpp`, `apps/desktop/main_window_engine.cpp` — lot « moteur » (§10).
- `libs/stitch/include/openstitch/stitch/sequence.hpp` — `StitchCommand`, `CommandType`, `StitchPass`, `StitchSequence`, `StitchStats`.
- `libs/stitch/src/stats.cpp` — `compute_stats`.
- `libs/stitch_generation/include/openstitch/stitch_generation/generate.hpp` + `src/generate.cpp` — `generate_sequence` (orchestration, dispatch, détection de groupe satin routable), `emit_polyline`, `emit_polyline_with_breaks`, `emit_fill`, `generate_running`, `generate_tatami`, `generate_satin`, `generate_satin_group`, `column_endpoints`, `is_routable_satin`.

- `libs/stitch_generation/include/openstitch/stitch_generation/overrides.hpp` + `src/overrides.cpp` — `effective_sequence`, `apply_manual_overrides`, `raw_slice`, `fingerprint`, `classify_edit_state`, `is_movable_point`, `edit_view`, `classify_all_edit_states`, `refresh_context` (Lot 8/ADR-014).
- `libs/stitch_generation/include/openstitch/stitch_generation/running_stitch.hpp`
- `src/running_stitch.cpp` — `run_stitch`, `sample_path`, `apply_repeat_mode`, `apply_repeats`, `RunningConfig`, `RunningResult`.
- `libs/stitch_generation/include/openstitch/stitch_generation/tatami.hpp` + `src/tatami.cpp` — `fill_tatami`, `tatami_underlay`, `segment_stays_in_region`, `connector_invalid`, `in_region`.
- `libs/stitch_generation/include/openstitch/stitch_generation/satin.hpp` + `src/satin.cpp` — `SatinConfig`, `SatinResult`, `SatinStation`, `satin_stations`, `fill_satin`, `fill_satin_columns`, `default_rungs`, `rails_from_contour`, `non_ribbon_interval`, `cap_factor`, `ladder_correspondence`, `resample_by_medial_spacing` (voir *Colonne satin* pour l'algorithme d'appariement).
- `libs/stitch_generation/include/openstitch/stitch_generation/lock.hpp` + `src/lock.cpp` — `lock_stitches`, `LockType`.
- `libs/stitch_generation/include/openstitch/stitch_generation/routing.hpp` + `src/routing.cpp` — `route_columns`, `RoutePlan`, `RouteColumn`, `RouteStep`, `ConnectorKind`, `RoutingConfig`, `greedy_order`, `best_orientation`.
- `libs/stitch_generation/src/satin_guides.cpp` — édition interactive des guides/barreaux (`satin_guide_junctions`, `move_satin_guide_group`, etc.), hors périmètre de la génération elle-même.
- `libs/document/include/openstitch/document/embroidery_object.hpp` — `RunningStitchParams`, `TatamiParams`, `SatinParams`, `SatinRung`, `SatinSectionTopology`, `SatinLock`, `StitchOverride`, `StitchPointType`, `EmbroideryObject`.
- `libs/optimization/include/openstitch/optimization/order.hpp` + `src/order.cpp` — `optimize_order`, `OrderStrategy`, `compute_cost` (réordonnancement des objets du document, en amont de la génération).
- `apps/desktop/main_window.cpp` — `refreshImage` (appel à `refresh_context`), `applyOrderStrategy` (réordonnancement puis régénération), `exportDst`, `buildDebugDump`/`showDebugDump` (dump `pass=… type=… pos=…` par commande, histogramme par passe).
- `apps/cli/main.cpp` — `run_stitchdebug`, générateur de debug du routage satin : les deux seuls sites annotés `raw-sequence-ok:` du dépôt.
- `tests/check_no_raw_sequence_bypass.cmake` — garde structurelle CTest du contrat `effective_sequence`.
- Tests : `tests/unit/stitch/test_generate.cpp`, `test_overrides.cpp`, `test_running_stitch.cpp`, `test_tatami.cpp`, `test_satin.cpp`, `test_satin_pairing_metrics.cpp`, `test_satin_guides.cpp`, `test_routing.cpp`, `test_stats.cpp`.
- Chapitres associés : *Génération de points — point droit et fondations*, *Remplissage tatami*, *Colonne satin*, *Retouche des points et de la géométrie*, *Format DST*, *Algorithmes*, *Objets de broderie*.

Satin par squelette et traversées orientées

Public : développeur, mainteneur. État : **moteur de référence de l'auto-satin**, **non validé sur machine**. Ce chapitre décrit l'implémentation réelle (`libs/auto_satin/src/skeleton_satin.cpp` et fichiers voisins) ; la spécification de conception et le registre complet des problèmes sont dans `specs/plans/satin-squelette-traversees.md`.

Pourquoi l'ancien auto-satin a été remplacé

L'ancien moteur construisait **deux rails** le long du squelette, les appariait (`ladder_correspondence`), posait des barreaux, réparait les jonctions, puis validait la couverture. Il a été retiré de l'auto-numérisation (commit `18de427`) parce que ses résultats polluaient le dessin : éligibilité trop permissive (`2A/P ≤ largeur max`), couverture partielle acceptée, mosaïque de sections satin et de pièces de tatami de repli.

Limites concrètes de cette architecture, toutes documentées dans `satin.md` (archive) : rails inversés ou croisés, barreaux non triés, ajustement paramétrique, décomposition récursive coûteuse (SGSD, 16 à 44 s sur une image réelle), formes branchées jamais complètes (`multi_neck`, `star5`, `comb`, `E`).

Un fait structurant : les « rails » étaient déjà les extrémités de cordes perpendiculaires au squelette. Le nouveau moteur garde cette primitive de mesure mais supprime la construction de rails, l'appariement et la décomposition. Les problèmes de **géométrie** (jonctions, coudes, pointes, bruit du squelette) ne disparaissent pas avec elle : ils sont traités explicitement (ci-dessous).

Principe

Pour une région Ω et son squelette (Zhang-Suen, `SkeletonGraph` élagué) :

1. Chaque chaîne d'arêtes du squelette (les nœuds de degré 2 ne sont pas des coupures) donne un **axe** lissé, paramétré par sa longueur d'arc.
2. On échantillonne l'axe. À chaque échantillon `P(s)`, une **orientation** `g(s)` définit la droite `P + t·(cos g, sin g)`.
3. La droite est intersectée avec la région ; la **corde** est l'intervalle intérieur qui contient `P`. Ses extrémités `A` et `B` sont les points de la traversée, sans rail ni appariement.
4. Les traversées d'une colonne passent par les finitions communes du satin (`finish_satin_stations` : terminaisons, points courts, sous-couches, compensation, fractionnement, sauts, verrous).

Trois grandeurs distinctes, jamais confondues : l'abscisse `s` sur l'axe, l'espacement `ρ` entre traversées, la longueur `L` d'un point de broderie.

Orientation `g(s)`

- **Défaut, par branche** : `g = α(s) + π/2` (perpendiculaire à la tangente). La spécification initiale prévoyait un angle absolu de 0° pour les squelettes « complexes » ; cette règle a été **écartée** : sur une branche parallèle à `g`, l'espacement perpendiculaire `h·|sin(g − α)|` tend vers 0 et la couverture latérale est nulle (échec sur les traits horizontaux de E, T, H, L). La distinction simple/complexe disparaît : chaque branche reçoit sa perpendiculaire.
- **Plancher** : `|sin(g − α)| ≥ 0,17` (≈ 10°). Une orientation plus proche de la tangente est ramenée au plancher et comptée (`clamped_angle`).
- **Guides** : un guide est un **point du repère du modèle** et un angle modulo π, relatif (écart à la perpendiculaire) ou absolu. Il est projeté sur l'axe à la génération ; aucun identifiant de branche n'est persisté (ils changent à chaque édition du contour). Entre deux guides l'angle est interpolé linéairement sur l'arc le plus court modulo π ; un écart d'exactement 90° tourne dans le sens positif (règle déterministe). Avant le premier guide et après le dernier : la valeur du guide le plus proche. Deux guides absolus égaux donnent un angle absolu constant. Un guide à plus de `max(2 r, 1,5 mm)` de l'axe est **orphelin** : signalé, ignoré.

Un trait tracé sur le canevas fixe un guide **absolu** (ancre = début du trait, angle = direction du trait).

Cordes et cellules

- La droite est intersectée avec tous les polygones (extérieur et trous, règle pair-impair) ; règle **demi-ouverte** aux sommets pour que la parité reste correcte quand la droite passe exactement par un sommet.
- **Cellules** : la région est partagée en cellules, une par morceau de colonne (une chaîne coupée aux coudes), selon le site le plus proche. Chaque corde est **écrêtée à sa cellule**, avec un recouvrement `ε` (0,2 mm) qui supprime les interstices. Aux jonctions, cela revient à des lignes de division bissectrices (idée d'EP0761860B1, rev. 3 à 5) et, aux coudes, à une coupe en onglet. Sans cet écrêtage, une corde au droit d'un nœud ou d'un coude traverse la branche voisine (jusqu'à 36 mm mesurés sur le tronc en «] »).
- **Coudes** : changement de cap mesuré sur une fenêtre `±W` au-dessus de 35° ; on coupe au **centre du plateau** de la mesure (couper à son premier point démarrait le morceau suivant dans la partie droite du bras, avec une tangente fausse).
- **Garde de rayon** : une corde plus longue que `2,5 r + 60 µm` (r = rayon inscrit sur l'axe) est écrêtée à cette borne et comptée (`radius_guard_hits`).
- **Bouts libres** : le squelette de Zhang-Suen s'arrête environ un rayon avant le bord. L'axe est prolongé en ligne droite tant que le point reste intérieur (marge de 1 µm), au plus `3 r + 1 mm`. Aucun refus des cordes étroites aux pointes ; longueur minimale de fil 0,3 mm.
- **Anneau** (squelette sans nœud) : axe **fermé**, périodique, point de départ canonique (plus petit `x`, puis `y`). La dernière traversée répète la première pour fermer la couture. Les cycles et boucles perdus par `build_skeleton_graph` (qui exclut le retour au pixel d'origine) sont récupérés depuis le squelette brut uniquement s'ils sont fermés ou si leurs **deux** bouts touchent le graphe ; une épine à bout libre est une branche élaguée à juste titre et ne revient pas.

Densité adaptative

Pour une traversée de demi-longueurs `a` (côté –u) et `b` (côté +u), avec `σ = sin(α − g)` et `g' = dg/ds`, l'espacement perpendiculaire à distance `t` de l'axe vaut `h·|σ + t·g'|` (dérivé). Le pas sur l'axe est

```
h(s) = ρ / max( |σ − a·g'| , |σ + b·g'| ),   h ∈ [50 µm, 3ρ]
```

c'est-à-dire que l'espacement au bord le **plus écarté** vaut `ρ` : aucun trou du côté extérieur d'un virage. Le côté intérieur, plus dense, est traité par les points courts existants. Les traversées voisines ne se croisent pas tant que `|g'|·d_côté < |sin(g − α)|` (pour une perpendiculaire : `κ·d_intérieur < 1`). Sur un demi-anneau de rayons 3 à 7 mm, l'espacement du bord extérieur est `ρ` à 8 % près (test).

Fractionnement `Lmax` / `y`

`SatinConfig::max_stitch_length` est le seuil `Lmax` ; `split_length` est la longueur de segment `y` (0 = `Lmax`, comportement historique). Si `L > Lmax`, la traversée est découpée en ■`L / y`■ segments égaux, avec décalage déterministe des pénétrations intermédiaires (décalé, ou jitter à graine fixe). `split_connecting_throws` fractionne **aussi** le trajet retour `B_i → A_{i+1}` du zigzag : le satin à rails ne fractionnait que les traversées `A → B`, laissant la moitié des points de fil à pleine largeur. Exemple testé : 20 mm, `Lmax` 7 mm, `y` 4 mm → points de 4 mm maximum.

Structures de données

`document::AutoSatinParams` (5■ alternative de `StitchParams`, ajoutée en fin : les index historiques sont inchangés) :

Champ	Rôle
`guides`	`AutoSatinGuide { anchor (µm), angle, absolute }`
`spacing`	ρ, espacement cible au bord le plus écarté
`split_stitch`, `split_threshold`, `split_length`	fractionnement (`Lmax`, `y`)
`short_stitch`, `cap_start/end`, `lock_*`, `pull_*`, `push_*`	finitions communes
`center_underlay`, `underlay_edge`, `underlay_zigzag`	sous-couches
`entry_point`, `exit_point`	sens de couture

L'objet **suit la région de son vecteur source** : les traversées sont recalculées à la demande (ADR-014), jamais stockées. Translation et mise à l'échelle du vecteur déplacent aussi les ancres de guides et les points d'entrée/sortie, avec annulation exacte. Contrairement au satin à rails, il ne porte aucune géométrie propre.

Format `.osp`

Type JSON `autoSatin`. Toutes les clés sauf `type` sont optionnelles (défauts du modèle). `kSchemaVersion` passe de 4 à 5 : un ancien exécutable refuse proprement un fichier plus récent au lieu d'échouer sur un type de point inconnu. Les anciens projets se chargent inchangés. **Les objets satin à deux rails (`satin`) se chargent et génèrent leurs points comme avant** (ils restent le satin manuel). Il n'existe pas de migration automatique d'un satin à rails vers un auto-satin : les rails édités à la main ne sont pas déduits du squelette.

Diagnostics (non bloquants)

`SkeletonSatinResult::diagnostics` : échantillons hors région, traversées trop courtes, orientations ramenées au plancher, gardes de rayon, guides orphelins, morceaux, messages de refus nommés (région compacte, squelette vide), et, sur demande (`measure_coverage`), couverture estimée, recouvrement moyen et aire non couverte. L'estimation rasterise les triangles balayés par deux traversées consécutives sur un masque de 100 µm érodé d'un pixel ; elle concorde avec une mesure indépendante par grille à 2 points près. `skeleton_satin_to_svg` et la commande `satin-auto-debug` du CLI produisent un SVG (contour, axes, traversées, zigzag). Aucun seuil de qualité textile n'est imposé avant essai machine.

Résultats sur le corpus de formes

Mesures du 2026-10 (couverture / recouvrement, `make_shape`), planchers gelés dans `tests/unit/auto_satin/test_skeleton_satin.cpp` :

Forme	Couverture	Recouvrement	Ancien moteur (source : `satin.md`)
rectangle	1,000	1,00	plancher 0,96 à 1,00
capsule, ruban, S	0,997 à 0,999	1,00 à 1,07	n/d
Y, T, croix, H	0,996 à 1,000	1,00 à 1,06	plancher 0,858 à 0,887
E, tronc en]	0,993	1,00 à 1,01	refus (coude à 90°)
trident	0,991	1,03	refus à 0,05 mm de pixel
star5, comb, multi_neck, dumbbell	0,979 à 0,997	0,98 à 1,03	non « Complete » avant correctifs
deux trous	0,994	1,06	n/d
anneau, anneau + branche	0,9998	1,00	0 région en SGSD

Formes compactes : axe synthétique (`compact_axis`)

Le squelette d'un disque se réduit à un point : sans axe, pas de traversée. Avec `SkeletonSatinParameters::compact_axis` (`AutoSatinParams::compact_axis`, faux par défaut, clé `.osp` optionnelle `compactAxis`), si aucune chaîne n'existe, le moteur construit un **axe droit synthétique** :

1. centroïde de la région (extérieur moins trous) ;
2. direction = axe principal d'inertie (`geometry::principal_axis`) ; forme quasi isotrope (anisotropie <= 1,3) : horizontal par convention, déterministe ;

3. corde intérieure qui contient le centroïde, raccourcie de 5 % à chaque bout (bouts libres, prolongés par le sampler) ; le rayon en chaque point est la demi-corde perpendiculaire.

Les traversées sont perpendiculaires à l'axe : des cordes **parallèles** de longueur variable (0 -> diamètre -> 0), comme un rond cousu à la main. Elles valent au plus le diamètre : au-delà de `split_threshold` (7 mm) elles sont fractionnées (décalé). Mesure : `disc_15mm`, `circle`, `petal` >= 90 % de couverture (test). Le message « région compacte » est remplacé par un message d'information. Sans l'option, le refus et ses tests sont inchangés.

Limites : pas de satin « en éventail » ou concentrique (le fil passe toujours en ligne droite d'un bord à l'autre) ; un grand disque donne de longs points fractionnés, d'où la borne `satin_compact_max_width` (12 mm) côté auto-numérisation ; non validé sur machine.

Limites restantes

- **Aucune validation sur machine.** La couverture géométrique n'est pas la qualité textile.
- `deep_channel` (U dont les bras font 16 mm de large) : les bras sont des blocs, pas des rubans, et l'élagage retire leurs branches ; la couverture reste autour de 65 %. Le moteur le **rapporte** (diagnostic) mais ne le résout pas.
- Disques, pétales et formes compactes : refus explicite (« région compacte ») par défaut. Option `compact_axis` (voir ci-dessous) pour les coudre en cordes parallèles. Le satin tournant par anneaux de l'ancien code a disparu.
- Anneau avec plusieurs trous et cycles sans nœud : cas non couverts.
- Le squelette reste celui de Zhang-Suen (bruit de pixel, tangentes lissées sur 600 µm). Un axe médian calculé sur le polygone (diagramme de Voronoï de segments) supprimerait cette source de bruit ; il n'est pas implémenté. La comparaison chiffrée est à faire avant d'ajouter une dépendance (Boost.Polygon, BSL-1.0).
- Le squelette est recalculé à chaque génération (aucun cache entre appels).
- Le fil peut se croiser du côté intérieur d'un virage très serré (`κ·d_intérieur ≥ 1`) : les points courts atténuent, ne garantissent pas.

Élagage à erreur bornée (2026-10-12)

Le squelette de Zhang-Suen d'un contour de pixels (ou d'un contour vectorisé) porte des rameaux parasites. L'ancienne règle de `prune_graph` retire une branche terminale si elle est courte en absolu (1,5 mm) *et* faible devant son rayon moyen ; elle ignore combien la branche dépasse réellement de la forme.

`GraphCleanupParameters::max_protrusion` / `max_protrusion_to_radius` ajoutent un critère à **erreur bornée**, dans l'esprit de l'élagage par résidu de frontière (Ogniewicz) et des mesures de stabilité du squelette (λ-axe médian, *scale axis transform*) : une branche terminale de longueur `L`, d'extrémité de rayon `r_E`, issue d'une jonction de rayon `r_J`, ne dépasse du disque de la jonction que de `h = L + r_E − r_J`. Tout disque `(c(s), r(s))` de la branche est contenu dans le disque `(J, r_J + h)` parce que `s + r(s)` ne décroît pas (le rayon varie au plus comme l'abscisse, propriété 1-lipschitzienne de la distance au bord). Retirer la branche déplace donc la forme reconstruite de **au plus `h`** : on la retire si `h <= max(max_protrusion, max_protrusion_to_radius · r_J)`. Une branche courte mais saillante est conservée ; une branche longue mais noyée dans un gros disque est retirée. Les candidates sont traitées par `h` croissant (égalité : indice d'arête), jamais jusqu'à vider le graphe, et chaque retrait est diagnostiqué (« branche terminale non significative »).

Valeurs par défaut du moteur (`stable_skeleton_analysis()`) : 0,4 mm et 15 % du rayon. `AutoSatinParameters` seul, et donc le mode Contours, gardent 0 / 0 (inchangés). Les clés de cache du squelette incluent ces champs. Mesures : corpus de formes propres, mêmes colonnes et même couverture ; contours bruités (densifiés tous les 0,4 mm, bruit ±0,2 mm) : colonnes en moins (h 19 → 15, comb 23 → 21, cross 14 → 12, E inchangé) et couverture égale ou meilleure ; ±0,3 mm : E 53 → 43. Le bruit blanc non corrélé de ces mesures est plus sévère que celui d'une photo : à confirmer sur des contours réels. Ne traite pas les boucles parasites ni les ponts de bruit (non terminaux).

Fichiers

Fichier	Contenu
`libs/auto_satin/src/chord.*`	cordes multi-intervalles, règle demi-ouverte
`libs/auto_satin/src/orientation.hpp`	`g(s)`, modulo π, clés relatives et absolues
`libs/auto_satin/src/axis.*`	axe lissé, ouvert ou fermé
`libs/auto_satin/src/axis_sampler.*`	pas adaptatif, bouts, garde de rayon
`libs/auto_satin/src/skeleton_satin.cpp`	chaînes, coudes, cellules, guides, anneaux
`libs/stitch_generation/src/generate.cpp`	`generate_auto_satin`
`apps/desktop/main_window_satin_auto.cpp`	création, guides, aperçu, squelette

Coudes serrés : croisements et éventail (2026-10)

Sur des bandes courbes à coude serré (rayon du coude inférieur à la largeur), trois défauts ont été observés puis corrigés dans `libs/auto_satin` :

- **Nœud de fils côté intérieur** : les traversées d'un coude se coupent au centre instantané de rotation de l'orientation (`t* = −σ/g'`). Chaque traversée est bornée du côté convergent (`converge_keep`), puis une passe `trim_crossings` raccourcit, à chaque croisement résiduel, la traversée qui s'étend le plus loin de son point d'axe (l'autre reste entière). Les traversées quasi confondues (recouvrement de cellules) sont dédoublonnées, la plus courte étant retirée.
- **Secteur extérieur vide** : couper un coude en deux pièces perdait la rotation située juste à la coupe. `junction_fan` ajoute, côté extérieur uniquement, des cordes autour du point de coupe dont l'angle suit la rotation de la tangente (pas angulaire `ρ/portée`).
- **Saut après échec d'échantillon** : après une corde non émise, le pas retombe à `ρ/4` au lieu de `ρ`, pour ne pas enjamber un coude.

Garantie testée : sur tout le corpus, aucune paire de traversées d'une même colonne ne se coupe (`crossing_pairs == 0`). Coût mesuré : `s` passe de 0,99 à 0,975 et `two_holes` de 0,985 à 0,980 de couverture estimée (planchers abaissés en conséquence). Non validé sur machine.

Colonne satin

> **Archive technique partielle (2026-10).** L'auto-satin décrit dans ce chapitre (deux rails, barreaux, appariement, `satin_planning`, `satin_coverage`, SGSD) a été **supprimé** et remplacé par l'auto-satin par squelette et traversées orientées, documenté dans satin-squelette.md. Restent valables ici : le satin **manuel** à deux rails (`SatinParams`, `fill_satin_columns`), les guides de barreaux, les sous-couches, compensations, points courts, découpes, verrous et le routage. Les sections sur la construction automatique de colonnes, le squelette multi-sections, `satin_planning` et la couverture géométrique décrivent du code qui n'existe plus ; ne pas s'y fier pour modifier le moteur actuel.

Public : utilisateur avancé, développeur.

> État : Présent dans le code : oui · Tests unitaires : oui · Tests visuels : partiels · Import/export DST : oui · Test sur machine réelle : **non** · **Statut recommandé : expérimental** — génération satin simple à deux rails. Plusieurs propriétés d'un générateur satin correct manquent (voir *Limitations* ci-dessous). Vérifiez impérativement le résultat avant tout passage sur machine.

Définition

Une colonne satin est un zigzag serré entre **deux rails** (bords gauche et droit) qui remplit une bande. Elle donne un effet lisse et brillant, adapté aux lettrages et aux bordures étroites.

Modèle

`SatinParams` porte `rail_a` et `rail_b` (deux `geometry::Path`), la densité (`density`), la compensation de tirage (`pull_compensation`), et l'activation d'une sous-couche centrale (`center_underlay`).

Génération

`fill_satin(rail_a, rail_b, config)` :

1. calcule une **correspondance locale monotone** entre les deux rails (`ladder_correspondence`, voir *Correction de l'appariement* ci-dessous) — les rails peuvent avoir des longueurs, courbures et échantillonnages différents ;
2. ré-échantillonne cette correspondance par pas fixe le long de sa **ligne médiane** (≈ perpendiculaire aux fils), et produit un **zigzag alterné** d'un bord à l'autre (L0, R0, L1, R1, …) ;
3. la **densité** fixe ce pas d'avancement, mesuré sur la ligne médiane (donc proche de la perpendiculaire aux fils, y compris en section inclinée ou courbe) ;
4. la **compensation de tirage** écarte les deux rails le long de leur médiatrice, pour compenser le resserrement du fil ;
5. la **sous-couche centrale** (optionnelle) est un point droit grossier sur l'axe, cousu avant le zigzag, dérivée de la même correspondance.

Le résultat (`SatinResult`) fournit les points de sous-couche, du satin, et la **largeur maximale** rencontrée (pour l'avertissement).

Paramètres

Valeurs par défaut lues dans `SatinParams`.

Paramètre	Unité	Défaut	Effet
`density`	mm	0,4	pas d'avancement le long de la colonne (plus petit = plus dense)
`pull_compensation`	mm	0	élargit la colonne (compense la traction du fil)
`center_underlay`	bool	vrai	ajoute une sous-couche centrale (point droit sur l'axe)
`max_width`	mm	9	seuil d'avertissement de largeur excessive

> Validation physique : non effectuée. La densité satin devrait idéalement se mesurer perpendiculairement au fil (voir l'avertissement ci-dessus).

Correction de l'appariement rail gauche / rail droit (audit rails, 2026-08-01)

État : Présent · Testé numériquement (fixtures géométriques dédiées) · non validé simulateur/physique.

Défaut trouvé. L'ancien appariement (`fill_satin` sans barreaux, et l'interpolation à l'intérieur d'un intervalle entre deux barreaux dans `fill_satin_columns`) associait les deux rails par la **même fraction d'abscisse curviligne** appliquée indépendamment à chacun (ou, avec barreaux, la même fraction `u` sur tout l'intervalle). Dès que les deux rails divergent en longueur ou en courbure — virage, coude, largeur variable — cette hypothèse de proportionnalité est fausse : les fils cessent d'être localement perpendiculaires au ruban. Symptôme observé : orientation des points en retard sur la direction locale des rails, éventails, quasi-croisements et densité visuelle irrégulière dans un ruban courbe ou anguleux.

Correction. Remplacé par une correspondance locale **"ladder"** (triangulation de bande par la diagonale la plus courte entre deux chaînes — technique classique de géométrie algorithmique, indépendante de tout logiciel de broderie) :

1. chaque rail est sous-échantillonné entre deux ancres exactes (barreaux, ou les deux extrémités de colonne si aucun barreau) à une résolution bornée (`maxStep`, fonction de la densité) — donne assez de détail même si les sommets d'origine sont épars (rail dessiné à la main, contour simplifié) ;
2. à chaque pas, la correspondance avance sur le rail dont le **prochain sommet forme la diagonale la plus courte** avec le point courant de l'autre rail (`ladder_correspondence`) — un garde-fou rejette une avance qui croiserait la diagonale précédente et force l'autre côté ;
3. cette correspondance dense est ré-échantillonnée par pas fixe le long de sa **ligne médiane** (`resample_by_medial_spacing`) : l'interpolation de (sa, sb) se fait dans le pas LOCAL du ladder qui encadre chaque cible, jamais sur tout l'intervalle — l'erreur reste bornée par la résolution du ladder, pas par la longueur de l'intervalle.

Invariants garantis (vérifiés par fixtures, voir ci-dessous) :

- **Monotonie** : les indices/abscisses avancent sur chaque rail dans le même sens que le parcours, jamais en arrière.
- **Absence de croisement** : deux fils consécutifs (et, mesuré sur les fixtures, deux fils quelconques) ne se croisent jamais.
- **Stabilité à l'échantillonnage** : la correspondance ne dépend pas du nombre de sommets d'origine de chaque rail (rails de résolutions très différentes acceptés).
- **Déterminisme** : aucune donnée non déterministe, résultat identique à entrée identique.
- **Complexité** : O(nA + nB) par intervalle entre deux barreaux (linéaire) — la ladder ne revisite jamais un sommet, comme la triangulation de bande classique dont elle s'inspire.

Limite assumée : à un coude **C0 franc** (angle vif sans congé), la notion de « normale locale » est elle-même discontinue ; un seul fil au sommet du coude absorbe nécessairement une déviation angulaire importante. Aucun appariement ne peut faire autrement sans mitre/congé explicite (relève de `ShortStitchMode`, hors périmètre de l'appariement de base). Ce que la correction garantit dans ce cas : la déviation reste localisée à ce fil unique (pas d'éventail étendu sur plusieurs fils) et aucun croisement n'apparaît.

Fixtures et métriques (`tests/unit/stitch/test_satin_pairing_metrics.cpp`, 13 tests, indépendants des tests de non-régression `test_satin.cpp`) : ruban droit (cas trivial), ruban en S, coude à 90° franc, largeur variable, cas combiné courbe+coude+largeur (inspiré d'un ruban capturé par l'utilisateur), plus la même correspondance via `fill_satin_columns` (barreaux), et 7 fixtures adverses (revue corrective ci-dessous). Chaque test mesure croisements (adjacents et toute paire), monotonie sur les deux rails, angle fil/normale locale (moyenne + pire cas), continuité angulaire entre fils consécutifs, régularité de la densité médiane, longueurs min/max, et déterminisme. Une réplique isolée de l'ancien algorithme (jamais utilisée en production) sert de référence comparative dans les mêmes tests. Résultats mesurés : ruban en S, angle

max fil/normale 2,7° (nouveau) contre 58,4° (ancien), angle moyen 1,2° contre 33,5° ; coude à 90°, 0 croisement (nouveau) contre 3 (ancien) ; cas combiné, angle max 4,9° contre 44,3°, 0 croisement dans les deux cas sur cette fixture précise mais éventail moyen réduit d'un facteur 12 (2,0° contre 24,0°).

Revue corrective (audit adverse, 2026-08-01)

État : Présent · Testé numériquement (fixtures adverses dédiées) · non validé simulateur/physique.

Audit ciblé de `ladder_correspondence` avec des fixtures délibérément défavorables (rails tête-bêche, échantillonnage très asymétrique avec doublons/segments nuls, longueurs très différentes + largeur quasi nulle, épingle à cheveux ~170°, barreaux désordonnés/dupliqués). Deux défauts réels trouvés et corrigés, un comportement clarifié :

- **Rails fournis tête-bêche** (bout 0 de A proche du bout N de B) : le commentaire de `fill_satin` exigeait déjà des rails « orientés dans le même sens », mais rien ne le vérifiait — un appel avec des rails inversés produisait un nœud papillon (chaque fil tend vers la diagonale opposée, croisements en O(n²)) au lieu d'un ruban. Aucun chemin de production actuel ne peut produire ce cas (`rails_from_contour` et `auto_satin::build_satin_columns` garantissent déjà le même sens), mais c'est une précondition silencieuse d'une fonction de bibliothèque publique — risquée pour un futur appelant (import, script, saisie manuelle). Corrigé par une détection bon marché (`opposite_orientation` : compare la somme des distances bout-à-bout dans les deux sens possibles) et une ré-orientation interne automatique, dans `fill_satin` et `fill_satin_columns`.
- **Barreaux non triés** : `fill_satin_columns` ne gardait un barreau que si sa projection avançait sur les deux rails par rapport au **dernier barreau GARDÉ dans l'ORDRE DU VECTEUR D'ENTRÉE** — un barreau placé en tête du vecteur mais loin le long de la colonne verrouillait cette référence et faisait rejeter silencieusement tous les barreaux suivants, repliant sur `fill_satin` sans barreaux (alors que la doc promet des barreaux toujours traversés exactement). Un barreau est une station transversale, pas un élément de séquence : il n'y a aucune raison que son ordre dans le vecteur soit signifiant. Corrigé par un tri par position projetée avant le filtre anti-croisement — le résultat ne dépend plus de l'ordre d'entrée des barreaux (vérifié : résultat bit-à-bit identique entre un vecteur trié et le même mélangé).
- **Barreaux dupliqués/quasi-dupliqués** : deux barreaux dont la projection n'avance que d'une quantité minuscule (mais non nulle) sur les deux rails créaient un intervalle dégénéré. Comme le garde-fou anti-croisement de `ladder_correspondence` n'agit qu'À L'INTÉRIEUR d'un intervalle (jamais entre deux intervalles voisins), les deux fils-ancres qui encadraient cet intervalle dégénéré n'étaient jamais comparés l'un à l'autre — s'il tombait près d'un virage serré, ils pouvaient se croiser (reproduit et corrigé, fixture "barreau dupliqué/quasi-dupliqué"). Corrigé en fusionnant (après tri) tout barreau dont la projection avance de moins de la moitié du pas de densité (`anchorMinGap = density / 2`) par rapport au dernier barreau gardé — élimine la cause (l'intervalle dégénéré) plutôt que de tenter de détecter le croisement a posteriori.
- **Échantillonnage très asymétrique / segments nuls / longueurs très différentes / largeur quasi nulle / épingle à cheveux serrée (~170°)** : déjà robustes sans modification — aucun croisement, monotonie et déterminisme préservés, aucune coordonnée dégénérée (vérifié par bornes larges plutôt que `isfinite` sur les micromètres entiers, qui ne peuvent pas représenter NaN).

Complexité vérifiée : le tri des barreaux ajouté est O(m log m) avec `m` = nombre de barreaux (petit, jamais lié à la résolution des rails) — ne change pas la complexité globale. Mesure empirique (release, ruban sinusoïdal 8 périodes, densité fixe) : temps de `fill_satin` stable (~6 ms) et nombre de fils constant pour un nombre de sommets d'entrée par rail variant de 500 à 8000 — confirme l'absence de comportement quadratique cachée dans le nombre de sommets d'origine.

Fixtures ajoutées (mêmes métriques que ci-dessus) : rails tête-bêche, longueurs très différentes + largeur quasi nulle, épingle à cheveux (mesure `crossings_total`, pas seulement `crossings_adjacent`, pour couvrir un croisement non adjacent), barreaux désordonnés (comparaison bit-à-bit avec la version triée), barreau dupliqué/quasi-dupliqué. S'ajoutent à la fixture d'échantillonnage asymétrique avec doublons/segments nuls déjà présente.

Colonne trop large

À la création, si la largeur dépasse le seuil recommandé, l'interface **prévient** et propose de préférer un remplissage tatami — la limite physique n'est jamais masquée.

Rails automatiques

`rails_from_contour(contour)` découpe un contour fermé en deux rails, coupé aux **deux sommets les plus éloignés** (les « bouts » de la colonne).

Débordement — désactivé par défaut. Cette heuristique ne connaît pas l'axe de la forme : sur un contour **concave ou branchu**, les deux rails traversent l'intérieur et les barreaux (rungs) enjambent les creux, ce qui fait **sortir les points de la région**. Mesuré sur un projet réel, jusqu'à **57 % des barreaux d'une colonne hors de sa région**. En conséquence :

- l'auto-numérisation **n'utilise plus** le satin naïf par défaut. Elle essaie d'abord `auto_satin::build_satin_columns` (`use_auto_satin`, défaut `true`) ; une forme refusée retombe sur le **tatami**. L'option `AutoOptions::use_naive_satin` (défaut `false`) ne subsiste que comme repli explicite pour essais/tests ;
- l'action **Broderie ■ Convertir les satins auto en tatami** répare un projet qui contient déjà des satins débordants ;
- le satin reste créable **manuellement** (menu Broderie, ou clic droit ■ *Type de points* ■ *Colonne satin*) — à vérifier visuellement, c'est la même heuristique de rails.

Limitation : `rails_from_contour` reste une heuristique (utilisée seulement pour la création **manuelle** de satin). La bonne méthode est le moteur géométrique ci-dessous.

Colonnes automatiques par squelette (`build_satin_columns`)

État : Présent · Testé numériquement · Validé visuellement (SVG) · non validé simulateur/physique.

`auto_satin::build_satin_columns(region, params)` construit une ou plusieurs **colonnes éditables** (`SatinColumnGeometry` : deux rails + **barreaux**) depuis une région, via le pipeline : rasterisation → transformée de distance → squelette (Zhang-Suen) → graphe élagué → satinabilité → **sections transversales** → rails → barreaux → validation.

Pour chaque branche du squelette : l'axe est lissé (Chaikin) et ré-échantillonné par longueur d'arc ; à chaque station on calcule la tangente puis la **normale**, on l'intersecte avec le contour et on retient l'**intervalle intérieur encadrant l'axe** (jamais la bounding box, jamais une association par index). Les deux extrémités donnent les rails ; la stabilité gauche/droite vient du signe de la normale (le rail A est toujours à gauche du sens de parcours). Les barreaux sont posés aux extrémités, aux virages, aux changements de largeur et à intervalle maximal. Un nettoyage anti-croisement retire les stations qui replieraient la colonne.

Extension des bouts ouverts jusqu'au bord réel

État : Présent · Testé numériquement · Validé visuellement (SVG). Activé par défaut (`SatinColumnsParameters::extend_open_ends`), rétrocompatible.

Défaut trouvé par revue (mission « auto-satin béton », audit du squelette existant confronté à la littérature — voir *Sources* ci-dessous) : un bout **ouvert** du squelette (sans jonction) s'arrête, par construction de l'amincissement (Zhang-Suen érode la forme progressivement depuis le bord), sensiblement **avant** le bord réel de la région. Démontré visuellement sur `capsule` (rectangle de 40 mm + deux demi-cercles de rayon 2,5 mm, longueur bout-à-bout réelle 45 mm) : avant correction, la colonne s'arrêtait à ~38,8 mm — les deux embouts arrondis (≈ 11 % de la longueur totale) restaient **entièrement hors couture**, un rectangle purement rendu vide au bout de chaque colonne. Le même retrait (proportionnel à la demi-largeur locale) affecte aussi un bout **carré** : sur `rectangle` (bouts plats, 40 mm), le retrait mesuré est de ~2,55 mm par bout — un défaut générique de l'amincissement, pas spécifique aux formes arrondies.

Correction (`extend_tip` dans `satin_column.cpp`) : pour chaque bout **sans jonction** (un bout de jonction reste inchangé — il doit rester exactement au nœud du squelette pour la reconciliation multi-sections), on

marche depuis la dernière station le long de la tangente sortante par pas de `station_spacing`, en ré-évaluant une section transversale réelle (intersection avec le contour, pas une approximation) à chaque pas tant qu'elle continue de **rétrécir** (tolérance 5 % contre une marche qui déborderait dans une autre partie de la forme). La fermeture finale est localisée par **bissection** sur un test point-dans-région pur (robuste même quand la section transversale devient numériquement dégénérée tout près du bord), avec une marge de sûreté de quelques µm avant l'arrondi final en micromètres entiers — sans cette marge, la bissection converge si près du bord analytique que l'arrondi peut faire retomber le point de l'autre côté du polygone discrétisé (trouvé par un test en échec, corrigé avant commit). Le point de fermeture reçoit une **largeur plancher non nulle** (`tip_min_width`, 0,05 mm par défaut) plutôt qu'un barreau littéralement nul, pour rester exploitable tel quel par `fill_satin_columns`. Marche et bissection sont toutes deux **bornées** (200 pas / 24 bissections) : jamais de boucle infinie, même sur une géométrie pathologique. Les rails **se terminent désormais sur le contour réel** → plus de débordement structurel ET plus de zone non couverte.

Vérifié : la longueur bout-à-bout d'une colonne `capsule` passe de ~38,8 mm à 45,0 mm (±1 mm) — la valeur géométrique exacte ; un bout carré (`rectangle`) est également corrigé (40,0 mm au lieu de ~34,9 mm) ; les bouts de jonction d'un réseau Y restent géométriquement identiques désactivé/activé (seuls les bouts ouverts s'allongent) ; aucun barreau dégénéré après extension ; extension déterministe ; le bascule `extend_open_ends=false` restaure l'ancien comportement (SVG avant/après dans `tests/golden/auto-satin/columns-*.svg`).

Sources. Ce mécanisme reprend, en le réimplémentant intégralement à partir des principes documentés (aucun code tiers consulté ni copié — projet Apache-2.0, jamais de dépendance à du code GPL) : le brevet Pulse Microsystems US6804573B2 *« Automatically generating embroidery designs from a scanned image »* (Google Patents), qui décrit l'extension du nœud terminal du squelette « dans la direction de la branche squelettique entrante », proportionnellement à l'épaisseur locale, pour que le tracé final couvre l'embout entier d'un objet fin. Le même brevet motive deux pistes non retenues dans ce lot (portée limitée à la couverture des bouts) : la classification fin/épais par statistiques de transformée de distance (max/moyenne/écart-type le long du squelette, plus riche que nos seuils actuels largeur min/max) et l'ancrage des jonctions sur les concavités réelles du contour plutôt que sur la seule topologie du squelette. Voir aussi Bai, Latecki & Liu, *« Skeleton Pruning by Contour Partitioning with Discrete Curve Evolution »*, IEEE TPAMI 2007 (PDF) pour un élagage de squelette plus fidèle à la forme que notre seuil longueur/rayon actuel (`graph_cleanup.cpp`) — piste de travail future, non implémentée ici.

Ancrage des jonctions sur les sommets reflex du contour

État : Présent · Testé numériquement · Validé visuellement (SVG). Activé par défaut (`SatinColumnsParameters::anchor_junction_ends`), rétrocompatible.

Défaut trouvé par revue (suite de la mission « auto-satin béton », piste « ancrage des jonctions » du même brevet). Mesuré précisément sur `y` : la largeur d'une branche reste stable (~5,0 mm) loin de sa jonction, mais **dérive nettement** sur les toutes dernières stations avant le nœud du squelette — jusqu'à ~9,2 mm sur la dernière station de la branche principale, une croissance quasi linéaire station après station. Cause : la section transversale d'une branche est calculée depuis sa **seule tangente locale** ; près d'une confluence, plusieurs branches se recouvrent physiquement, et le rayon perpendiculaire balaie alors ce **bourrelet de la confluence** — pas la ceinture réelle de cette branche seule. Conséquence démontrée : les 3 sections d'un Y ne se rejoignaient PAS au même point — jusqu'à **~5 mm d'écart** entre deux rails censés coïncider exactement à la confluence, laissant un vide ou un repli selon les branches.

Correction (`trim_and_anchor_junction_end` dans `satin_column.cpp`) en deux temps, pour chaque bout de JONCTION (jamais un bout ouvert — traité par l'extension des bouts ouverts, § précédente) :

1. **Amputation** de la queue instable : les stations terminales sont retirées tant que leur largeur dépasse de plus de 10 % celle de leur voisine plus intérieure — signature précise de la dérive mesurée (croissance nette et soudaine, à distinguer d'une variation progressive légitime ailleurs dans la colonne, jamais touchée par cette règle).
2. **Ancrage** indépendant de chaque rail sur le **sommet reflex** (concave) du contour le plus proche, dans un rayon borné (`junction_anchor_radius`, 6 mm par défaut) : les sommets reflex sont exactement les « encoches » où le contour réel bascule d'une branche à sa voisine. Les deux rails d'une même branche

touchent généralement **deux encoches distinctes** — jamais ancrés au même point. Si les deux rails convoitent le même sommet (cas réel trouvé sur un « T » où une moitié de barre n'a qu'un seul côté avec une vraie encoche voisine), seul le plus proche le garde ; l'autre cherche son propre sommet **distinct** ou renonce — sans cette règle d'exclusion, les deux rails convergeaient par erreur vers l'unique encoche, créant un barreau de largeur nulle. Sans ancre à portée, un rail conserve sa position amputée (repli sans erreur — toutes les jonctions ne sont pas des étoiles symétriques à *n* encoches pour *n* branches).

Vérifié : sur un Y symétrique (3 branches), les 6 extrémités de rail touchant la jonction se regroupent en **exactement 3 sommets, chacun partagé par exactement 2 rails** de sections différentes (`length_um` < 5 µm entre les deux) ; sur un T (topologie asymétrique, 2 encoches réelles pour 3 branches), **aucune collision** (jamais 3 rails ou plus au même sommet) et aucun barreau dégénéré ; aucune dérive de largeur résiduelle (> 1,2× la médiane) sur aucune section ; déterminisme ; le bascule `anchor_junction_ends=false` restaure l'ancienne dérive (régression volontairement reproduite pour prouver que le bascule agit réellement).

Amas de pixels de jonction et ancrage indépendant par branche (audit jonctions branchées/concaves, 2026-08-03)

État : Présent · Testé numériquement. Correctif de revue, `skeleton_graph.cpp` + `satin_column.cpp`.

Défaut trouvé par revue (capture d'une forme branchée/concave réelle, suite de la mission « ancrage des jonctions » ci-dessus). Deux causes distinctes, une même conséquence visible : les rails venant de deux branches angulairement voisines d'une même confluence ne se raccordent pas exactement — derniers barreaux en éventail, rails terminaux qui ne partagent pas le même point, zone triangulaire mal couverte près du centre.

1. **`build_skeleton_graph` ne consolidait pas les amas de pixels de jonction.** Sur une confluence à 3+ branches — surtout asymétrique —, l'amincissement Zhang-Suen laisse souvent un petit amas de PLUSIEURS pixels adjacents ayant chacun un nombre de croisement ≥ 3, plutôt qu'un unique pixel net. L'ancien code créait un `SkeletonNode` **par pixel** de cet amas, reliés entre eux par des micro-arêtes internes (`length_um` de quelques dizaines de µm) que `graph_cleanup.cpp` ne pouvait pas élaguer (son seuil ne s'applique qu'aux arêtes **terminales**, degré 1 — une micro-arête entre deux pixels de jonction a ses deux extrémités de degré ≥ 3). Chaque branche incidente se retrouvait donc ancrée sur un nœud légèrement différent selon le pixel de l'amas par lequel son tracé y entrait.
2. **`trim_and_anchor_junction_end` ancrait chaque rail indépendamment**, y compris après la consolidation ci-dessus : chaque rail de chaque branche cherchait pour son propre compte le sommet reflex le plus proche (`reflex_vertices`, liste globale du contour), sans aucune coordination avec les rails des branches VOISINES de la même confluence. Deux branches angulairement adjacentes pouvaient donc élire des sommets différents pour ce qui est géométriquement la MÊME encoche.

Correction, deux volets :

1. `build_skeleton_graph` regroupe désormais les pixels de jonction 8-connexes (union-find) en un seul amas logique avant de créer les `SkeletonNode` : position stable = le pixel de plus grand rayon dans le `DistanceField` au sein de l'amas (départagé par (y, x) croissants pour le déterminisme — toujours un pixel réel, jamais un barycentre flottant), toutes les branches incidentes pointent vers ce même identifiant, et toute micro-arête interne à l'amas (`from == to` après consolidation) est rejetée sans condition.
2. L'ancrage sur sommet reflex est sorti de `build_column` (qui ne fait plus que l'AMPUTATION de la queue instable, `trim_unstable_junction_tail`) et devient une résolution **globale par jonction** (`resolve_junction_anchors`, `satin_column.cpp`), appelée une fois toutes les colonnes construites : les branches incidentes à une jonction sont triées par angle autour de son centre, et chaque ESPACE ANGULAIRE entre deux branches adjacentes reçoit au plus une ancre (le sommet reflex le plus proche du centre dans ce secteur), appliquée **identiquement** aux deux rails qui se font face — le rail gauche (A) de la branche « avant » dans l'ordre angulaire et le rail droit (B) de la branche « après ». Une branche sans encoche à portée dans son secteur garde sa position brute (repli, comme avant — toutes les confluences n'ont pas autant d'encoches que de branches, cf. le T ci-dessus). Défaut annexe trouvé en

cours d'audit : la section transversale brute d'une branche rétrécit naturellement vers zéro du côté qui longe une encoche (géométrie réelle, pas une erreur) ; quand seul le côté OPPOSÉ de cette branche reçoit une ancre partagée, le côté non ancré restait à cette position quasi nulle — barreau terminal dégénéré. Corrigé par un plancher de largeur (`tip_min_width`, même paramètre que la fermeture de `extend_tip`) qui ne déplace QUE le côté non ancré, jamais le côté ancré (qui doit rester exactement coïncident avec la branche voisine).

Une validation finale par jonction (`validate_junctions`, appelée après la résolution des ancres) refuse proprement la génération de TOUTE la région plutôt que de renvoyer un raccord incohérent : arêtes incidentes manquantes (une branche non construite sur une confluence sinon partiellement ancrée), rails terminaux qui ne coïncident pas et s'éloignent trop du centre (éventail non résolu), croisement entre barreaux terminaux de branches voisines, ou secteur angulaire anormalement large sans aucune ancre commune (trou triangulaire). `extend_tip` reste structurellement impossible côté jonction (un bout de jonction n'emprunte jamais ce chemin, inchangé par cet audit).

Vérifié : suite `test_auto_satin` complète inchangée (`y`, `t`, `cross`, `h` toujours sans collision ni barreau dégénéré) ; nouvelle fixture `trident` (grande branche verticale épaisse, branche interne pointue — triangle effilé, pas une bande à largeur constante — et branche latérale étroite, confluence délibérément non étoilée) : raccord cohérent, aucune collision à 3+ rails, aucun barreau dégénéré, aucun croisement entre les trois barreaux terminaux, tous les barreaux dans la région ; déterminisme.

Remplacement de l'ancrage par des bridges verrouillés, sans mutation de rail (2026-08-03, suite)

État : Présent · Testé numériquement. Remplacement architectural, `satin_column.cpp` + `satin_column.hpp` + `debug_export.cpp`.

Défaut trouvé en usage réel sur `trident` (confluence à 3 branches très inégales — 6 mm / 1,2 mm / pointe effilée) : `resolve_junction_anchors` (§ ci-dessus) reste correct sur les confluences symétriques ou peu contrastées (`y`, `t`, `cross`, `h`), mais sur `trident`, l'ancre trouvée pour un espace angulaire pouvait se situer à plusieurs millimètres de la dernière station réellement stable de la branche. `set_terminal_point` déplaçait alors le dernier nœud du rail **en ligne droite** jusqu'à cette ancre — une corde artificielle traversant l'intérieur de la confluence au lieu de suivre le bord réel de la forme, visible comme une grande diagonale dans le SVG de diagnostic. Le même mécanisme (`tip_min_width` plancher, repli symétrique) souffrait du même défaut de principe : déplacer un point plutôt que construire la géométrie.

Correction : suppression complète de la mutation. `resolve_junction_anchors`, `best_anchor_in_sector`, `boundary_anchor_on_sector_bisector`, `set_terminal_point` et `reflex_vertices` sont retirés. Un bout de jonction n'est plus jamais déplacé après coup : la dernière station stable que `trim_unstable_junction_tail` (inchangée dans son principe) laisse en place devient directement le **`JunctionBridge`** verrouillé de la colonne — `rail_a_point`/`rail_b_point` (déjà des points réels du contour, `cross_section` les calcule par intersection avec le bord), `axis_point`, `tangent`, `width_um`. Deux branches angulairement adjacentes ne sont plus forcées à coïncider : le petit vide géométrique qui peut subsister entre leurs bridges respectifs (`JunctionCore`, exposé dans `SatinColumnsResult::junction_cores`, jamais dissimulé) est calculé séparément et n'entraîne PAS de refus — seul un défaut structurel réel (bridge égaré, croisement) refuse encore la génération (`validate_junction_bridges`).

Deux effets de bord de la suppression de l'ancrage, trouvés en generalisant sur `trident` puis sur un réseau en T issu d'une image segmentée réelle :

1. **Le critère de stabilité de `trim_unstable_junction_tail` devait devenir plus robuste.** L'ancien plafond de distance (lié à `junction_anchor_radius`, qui ne bornait à l'origine que le rayon de recherche d'une ancre — notion disparue) arrêtait parfois l'amputation alors que la largeur était encore en pleine dérive (bourrelet de `trident` dépassant 6 mm d'abscisse curviligne) ; supprimé, plus aucun plafond de distance. Le critère de largeur lui-même (décroissance stricte station à station) échouait sur un PALIER de largeur (deux stations quasi égales mais toutes deux énormes) — remplacé par une comparaison à `representative_station_width` : la médiane du **tiers médian** des stations de la branche (ni le premier

tiers, qui peut être contaminé par un bout OUVERT qui rétrécit légitimement vers une pointe — `trident`, branche interne effilée — ni le dernier, potentiellement gonflé par le bourrelet de jonction).

2. **Un bridge peut légitimement croiser celui d'une branche voisine sans dérive de largeur mesurable**, si l'angle entre les deux branches est proche de 90° et que l'une d'elles est nettement plus large que l'autre (la coupe transversale, perpendiculaire à la branche fine, atteint alors le bord de la branche large sans que sa PROPRE largeur ait significativement augmenté). `resolve_and_validate_junctions` recule alors itérativement d'une station RÉELLE supplémentaire (`retract_bridge_station`, jamais un déplacement) sur les deux branches en conflit, jusqu'à ce que le croisement disparaisse ou que le plancher de stations soit atteint.

Nouvelles primitives internes (diagnostic uniquement, jamais utilisées pour construire ou déplacer un rail) : `ContourPolyline`/`project_to_contour`/ `extract_contour_arc` (polygone fermé, abscisse curviligne, projection, extraction d'arc) — utilisées par `compute_junction_core` pour approximer le bord réel du noyau central entre deux bridges, et par `columns_to_svg` pour l'afficher (remplissage magenta pointillé) aux côtés des bridges terminaux (barreaux rouges épais) et des identifiants de colonne/jonction.

Vérifié : suite `test_auto_satin` complète (44 cas) ; nouveau test dédié `trident` sur la géométrie des rails (aucun segment terminal excédant 4× `station_spacing`, progression curviligne strictement monotone, `junction_cores` non vide et borné) ; test `y` symétrique réécrit (l'ancienne coïncidence exacte forcée n'est plus un invariant recherché — remplacé par « bridges locaux au centre de confluence, noyau résiduel petit ») ; suite `test_autodigitize` (réseau en T issu d'image segmentée, régression trouvée et corrigée par la retractation itérative ci-dessus) ; suite d'intégration complète (`tentabrode`) ; déterminisme.

Limite connue : la décomposition d'une branche en sous-région fermée propre (rail A + bridge + rail B inversé + autre bridge, comme dans Ink/Stitch) et le remplissage explicite du `JunctionCore` par un objet tatami distinct ne sont pas implémentés — le noyau reste un diagnostic (aire + contour approximatif) non cousu. Les corps de branche restent construits par échantillonnage de sections transversales le long de l'axe du squelette (inchangé), pas par une paire d'arcs de contour indépendants.

Optimisation globale des bridges de jonction + correctif d'un polygone auto-croisé (suite)

État : Présent · Testé numériquement. Remplacement de `resolve_junction_anchors`/`retract_bridge_station` par une recherche combinatoire, `satin_column.cpp` + `satin_column.hpp` + `debug_export.cpp`.

Défaut signalé en usage réel sur `trident` : la capture indépendante « une seule candidate par branche » (§ ci-dessus) choisissait toujours la toute dernière station de chaque branche comme bridge, sans coordination avec ses voisines — sur une confluence aussi asymétrique que `trident`, ces bridges restaient parfois francs millimètres les uns des autres, et le `JunctionCore` qui en résultait apparaissait comme un grand polygone concave recouvrant toute la confluence.

Deux défauts distincts corrigés, pas un seul :

Optimisation combinatoire des bridges (la demande initiale). Chaque branche conserve désormais plusieurs sections transversales CANDIDATES (`collect_bridge_candidates`) — des stations déjà présentes dans son rail, jamais un point fabriqué — prises dans un rayon local (`junction_core_radius`, nouveau paramètre, 4 mm par défaut) autour du nœud de squelette. `optimize_junction_bridges` évalue toutes les combinaisons raisonnables (une candidate par branche, budget total borné quel que soit le degré de la jonction) et retient celle de coût minimal — aire du `JunctionCore`, distance maximale au centre, recul total le long des branches, discontinuités de largeur — parmi celles qui interdisent tout croisement bridge/bridge (`find_crossing_bridge_pair`) et toute traversée d'une branche voisine (`bridge_crosses_other_rail`). **Aucun rail n'est modifié** : seul l'index de la station retenue varie d'une combinaison à l'autre. `retract_bridge_station` (qui tronquait réellement les rails) est supprimé — l'invariant « les rails restent ceux produits par `build_column` » est désormais strict, plus seulement documenté.

Bug préexistant trouvé EN OPTIMISANT, indépendant du point 1 : l'ordre des sommets de `compute_junction_core` reliait les mauvais côtés. `rail_a` (+N) n'est PAS toujours le côté qui fait face à la branche suivante dans l'ordre angulaire : sa convention gauche/droite est relative au sens de

parcours interne du rail (`build_column`), qui va tantôt de la jonction vers le bout ouvert, tantôt l'inverse, selon l'orientation de l'arête de squelette (`atEnd`) — alors que `tangent` (`outward_tangent`) pointe toujours correctement vers l'intérieur de la branche, quel que soit `atEnd`. L'ancien code connectait systématiquement `rail_b(cur)` à `rail_a(next)` : sur une branche où `atEnd` inverse la convention (le cas des deux bras courts de `trident`), ce n'était PAS le côté qui fait réellement face au voisin. L'arc de contour « le plus court » entre deux points qui ne se font pas face part alors dans une direction non pertinente, et le polygone assemblé s'auto-croise — la formule du lacet annule alors une partie de l'aire réelle par recouvrement, produisant un nombre artificiellement PETIT (le fameux 3,24 mm² observé) pour un polygone géométriquement INVALIDE. Corrigé par un calcul explicite, `leading_point`/`trailing_point`, dérivé de `tangent` tourné de +90° (fiable quel que soit `atEnd`) plutôt que des labels `rail_a`/`rail_b` bruts.

Garde-fous ajoutés une fois le bug de connexion corrigé : `polygon_self_intersects` détecte tout polygone auto-croisé restant ; `compute_junction_core` construit désormais le noyau en DEUX PASSES — d'abord la version repli (segments droits uniquement entre bridges successifs ; si elle-même s'auto-croise, la combinaison entière est invalide) puis, pour chaque connecteur, un remplacement par l'arc de contour réel UNIQUEMENT s'il garde le polygone simple ET ne fait pas grossir son aire (sur une confluence où 3+ branches se chevauchent mutuellement, l'arc « le plus court » au sens de la longueur de contour peut rester simple tout en empruntant un grand détour convexe).

Résultat mesuré : une fois le polygone correctement calculé (simple, non auto-croisé), l'aire réelle du noyau de `trident` est de l'ordre de 22 à 23,5 mm² selon les seuils — **plus grande**, pas plus petite, que l'ancien 3,24 mm² invalide. Ceci n'est PAS une régression : `y`, `t`, `cross` et `h` donnent des noyaux carrés/triangulaires parfaitement simples de 20 à 30 mm² une fois rendus en SVG et inspectés visuellement (bandes de test larges de 5 mm, donc recouvrement mutuel substantiel aux confluences par construction), confirmant que ces ordres de grandeur sont réalistes pour ces fixtures, pas un artefact. `JunctionCore::requires_fill` (nouveau champ, seuil `junction_core_significant_area_um2` = 0,5 mm² par défaut) signale qu'un noyau de cette taille appelle un objet de remplissage séparé plutôt qu'une zone non couturée silencieuse — la synthèse de cet objet reste hors périmètre (diagnostic seul, cf. limite connue ci-dessus).

Diagnostic enrichi : `SatinColumnsResult::junction_bridge_candidates` (nouveau, toutes les candidates évaluées par branche + laquelle est retenue) et `JunctionCore::radius_um` sont exposés et affichés par `columns_to_svg` (candidates en gris pointillé fin, bridge retenu en vert, disque de recherche/contenance en pointillé gris autour du nœud de squelette).

Vérifié : suite `test_auto_satin` complète (44 cas, déterminisme inclus) ; test `trident` étendu (rails inchangés — déterminisme + chaque nœud à moins de 0,2 mm du contour réel —, noyau local à J1 au rayon `core.radius_um` près, aire bornée à une valeur réaliste et documentée comme telle) ; inspection visuelle des SVG rendus (`y`, `cross`, `h`, `trident`) confirmant des noyaux simples, bien formés, sans auto-croisement ni grand polygone parasite.

Séparation StableBranchEnd / JunctionSeparator : le noyau n'est plus « région entière moins colonnes » (suite)

État : Présent · Testé numériquement. Remplacement de l'optimisation combinatoire de bridges (§ ci-dessus) par une partition explicite, `satin_column.cpp` + `satin_column.hpp` + `debug_export.cpp`.

Défaut architectural signalé en usage réel sur `trident` : la recherche combinatoire (§ précédente) choisit, PARMI plusieurs reculs candidats d'une même branche, celui qui minimise l'aire du `JunctionCore` — mais reculer une section transversale ne change pas où se trouve la vraie frontière géométrique entre deux branches (l'encoche réelle du contour). Sur `trident`, aucune combinaison de reculs ne pouvait donc produire un noyau petit ; le résultat mesuré (22 à 23,5 mm², quasiment toute la confluence) était réel, pas un artefact — mais architecturalement le mauvais résultat.

Correction : séparer deux notions jusqu'ici confondues.

1. **`StableBranchEnd`** — la dernière section transversale RÉELLEMENT stable d'une branche (ce qu'on appelait le « bridge »), inchangée dans son principe (`trim_unstable_junction_tail`) : point d'entrée

du traitement de jonction, jamais déplacée.

2. **`JunctionSeparator`** — la frontière locale PARTAGÉE entre deux branches angulairement adjacentes, construite DANS la zone de confluence (sommet reflex du contour le plus profond, ou repli sur le milieu curviligne de l'arc de contour reliant les deux branches) : ce n'est ni une section transversale, ni jamais un point sur un rail.

La région locale de jonction (disque de rayon adaptatif — `1,2 ×` la plus éloignée des `StableBranchEnd` au nœud, jamais une constante fixe, plafonné par `junction_core_radius`) est partitionnée en un secteur par branche (bordé par ses deux `JunctionSeparator` voisins, l'arc de contour réel, et sa propre `StableBranchEnd`). Le `JunctionCore` résiduel est calculé **comme `local_region − union(secteurs)`**, jamais comme « région entière moins colonnes » : concrètement, le polygone des `JunctionSeparator` eux-mêmes, simple par construction (sommets triés par angle autour d'un centre commun = polygone en étoile).

Deux bugs trouvés en généralisant, tous deux avec un symptôme identique (noyau anormalement grand) mais des causes distinctes :

1. **Recherche de séparateur par SECTEUR ANGULAIRE au lieu de CONTIGUÏTÉ DE CONTOUR.** La première implémentation cherchait le sommet reflex d'un espace angulaire `[angle(branche i), angle(branche i+1)]`, basé sur l'angle du rayon squelette de chaque branche. Sur `trident`, la branche verticale (6 mm) a ses points `leading`/`trailing` écartés de plus de 90° autour du nœud : l'encoche réelle entre elle et sa voisine fine tombait alors, en angle pur, dans le secteur voisin — laissant DEUX des trois espaces sans aucun candidat, repliés sur un rayon lancé depuis le nœud qui pouvait heurter le mur d'une branche sans rapport (noyau mesuré : 14,4 mm², toujours trop grand). Corrigé en cherchant plutôt sur l'arc de contour qui relie réellement `leading_point(branche i)` à `trailing_point(branche i+1)` (`ContourDirection::Shortest`) : la contiguïté de contour reflète la vraie adjacence géométrique, pas l'angle depuis un centre qui peut être trompeur pour une branche large. Résultat après correction : 0,94 mm².
2. **Une `StableBranchEnd` peut être stable en LARGEUR tout en restant géométriquement dans le corridor d'une voisine.** `trim_unstable_junction_tail` ne détecte que la dérive de largeur du bourrelet — pas sa position. Sur un réseau en T très asymétrique (bras large, branche fine, régression trouvée par `test_autodigitize`), la toute première section d'un bras pouvait avoir une largeur déjà plausible (donc jamais amputée) tout en croisant le rail de la branche voisine, un défaut que le seul critère de largeur ne peut pas voir. Corrigé par une retractation itérative bornée dans `resolve_junction` (jamais dans `build_column`) : toute paire de `StableBranchEnd` dont les coupes se croisent (`find_crossing_end_pair`/`end_crosses_other_rail`) fait reculer les deux branches impliquées d'une station réelle supplémentaire (`make_stable_branch_end` généralisé avec un paramètre `offset`), jusqu'à ce que la confluence soit cohérente ou qu'il n'y ait plus de station disponible (refus propre, comme avant).

Diagnostic enrichi : `SatinColumnsResult` expose désormais `stable_branch_ends`, `junction_separators` et `junction_sectors` (en plus de `junction_cores`, dont les champs `radius_um` unique est remplacé par `configured_radius_um`/`local_radius_um`/`actual_max_radius_um` — le premier est un plafond de sécurité, jamais le rayon réel). `columns_to_svg` affiche les `StableBranchEnd` en rouge, les `JunctionSeparator` en vert, un secteur par branche en teinte translucide distincte, le noyau en magenta, et le disque de région locale en pointillé — avec les trois rayons étiquetés séparément pour ne plus jamais laisser croire que le plafond configuré est une mesure.

Vérifié : suite `test_auto_satin` complète (44 cas) ; test `trident` étendu (3 secteurs, 3 séparateurs partagés exactement entre secteurs adjacents, aire du noyau très inférieure à l'ancien résultat invalide, aucun point du noyau au-delà du rayon local réellement calculé) ; suite `test_autodigitize` complète (réseau en T, régression trouvée et corrigée par la retractation itérative ci-dessus) ; inspection visuelle des SVG rendus (`trident`, `y`, `t`, `cross`, `h`) : noyaux petits et centrés sur `trident`/`y` (encoches réelles présentes), noyaux carrés honnêtes sur `t`/`cross`/`h` (branches de même largeur se croisant à angle droit sans aucune encoche — un noyau non trivial y est le résultat géométriquement correct, pas un défaut).

Traçage du graphe de squelette : priorité aux nœuds, exclusion de l'origine

État : Présent · Testé numériquement. Correctif de revue, `skeleton_graph.cpp`.

Défaut trouvé par revue (deux symptômes distincts, même cause racine). Le traçage d'une arête (`build_skeleton_graph`) suit une chaîne de pixels de degré 2 depuis un pixel-nœud jusqu'au prochain pixel-nœud, en choisissant à chaque pas le **prochain voisin** parmi les 8 directions. L'ancien code s'arrêtait sur le **premier candidat rencontré** dans l'ordre fixe des 8 directions, qu'il s'agisse d'un vrai nœud ou d'un simple pixel de continuation — alors qu'un pixel juste avant une jonction a souvent, en plus de la jonction elle-même, un pixel de degré 2 d'une **autre branche** dans son 8-voisinage (les branches se recouvrent physiquement près d'une confluence, cf. section précédente).

1. **Jonction "croix" ramenée à un degré 2 au lieu de 4** (le défaut initialement identifié, stable sur toutes les résolutions testées) : deux bras opposés étaient fusionnés en une seule arête traversante quand le pixel de l'un d'eux apparaissait avant le pixel de la jonction elle-même dans l'ordre de balayage.
2. **Branche entière perdue sur un réseau "y" avec arête parasite en boucle** (défaut distinct, non documenté avant cette revue) : le pixel d'**origine** de la trace pouvait être ré-atteint par un chemin de 2 pas différent de celui emprunté au départ — seul le pixel immédiatement précédent était exclu des candidats, pas le pixel d'origine de toute la trace. La tige du "y" (une branche entière) disparaissait alors du graphe, remplacée par une arête `from == to` de quelques centaines de µm sur la jonction, sans aucune arête ni diagnostic pour l'utilisateur.

Correction : priorité **absolue** à un nœud sur tout le 8-voisinage (balayage complet des 8 directions pour un nœud avant de retomber sur un pixel de continuation, au lieu du premier candidat rencontré), et exclusion du pixel d'**origine** de la trace (pas seulement le pixel précédent) pendant toute la marche.

Vérifié : la jonction d'une `cross` a désormais un degré réel de 4 dans le graphe élagué (4 arêtes, 4 extrémités, 1 jonction) ; un réseau `y` conserve ses 3 branches (aucune arête `from == to`) ; aucune régression sur les formes déjà testées (`rectangle`, `t`, `capsule`, `ribbon`, `s`, `ring`, `wide`, `circle`).

Contrat de `graph_cleanup.cpp` : aucune suppression silencieuse

État : Présent · Testé numériquement. Correctif de revue.

Défaut trouvé par revue : `graph_cleanup.hpp` documente explicitement « ne supprime rien silencieusement (chaque suppression est diagnostiquée) », mais `prune_graph` ne réinsérait un nœud dans le graphe élagué que s'il était référencé par au moins une arête vivante — un nœud du graphe brut **sans aucune arête** (isolé dès le départ, comme le point unique auquel se réduit le squelette d'un disque plein, ou devenu orphelin après élagage d'une arête terminale courte) disparaissait donc du résultat sans jamais apparaître dans `removed`, contredisant le contrat documenté.

Correction : tout nœud du graphe d'origine non repris dans le graphe élagué (aucune arête vivante incidente, quelle qu'en soit la raison) est désormais systématiquement ajouté à `removed`, avec un motif explicite. Vérifié sur `circle` (squelette réduit à un point isolé) : le graphe élagué reste vide, mais le nœud isolé apparaît dans le diagnostic.

Arête Jonction-Jonction (pont d'un "H") convertie en colonne

État : Présent · Testé numériquement. Correctif de revue, `satin_column.cpp`.

Défaut trouvé par revue : `build_satin_columns` (cas `RequiresDecomposition`) ne convertissait en colonne que les arêtes du squelette élagué touchant au moins une **extrémité** (« une colonne par branche menant à une extrémité, bras du Y/T »). Une arête reliant **deux jonctions** — le pont horizontal d'un "H", topologie absente des formes de test précédentes (Y/T/croix n'ont qu'une seule jonction) — n'était donc jamais essayée : `r.columns` restait non vide (les bras des deux barres verticales étaient bien produits) et `r.refusal` restait vide (aucun signal d'erreur), mais le pont lui-même — une portion entière et visible de la région — ne recevait **aucun point de broderie**.

Correction : toutes les arêtes du graphe élagué sont désormais essayées, sans distinction sur le type de leurs deux extrémités — `try_edge` gère déjà chaque bout indépendamment selon son type (extension si extrémité ouverte, ancrage si jonction), aucune branche de code particulière n'était nécessaire. Vérifié sur

une fixture "H" dédiée (deux barres verticales de 40 mm reliées par un pont horizontal de 5 mm) : 5 colonnes produites (les 4 demi-barres + le pont), aucune collision aux deux jonctions, aucun barreau dégénéré.

Amputation de la queue instable : décroissance stricte, pas un seuil fixe

État : Présent · Testé numériquement. Correctif de revue, `satin_column.cpp`.

Défaut trouvé par revue, révélé par le correctif de traçage ci-dessus (la branche "y" auparavant perdue expose maintenant ce second défaut, jusque-là invisible faute d'atteindre `satin_column.cpp`). L'étape 1 de l'ancrage des jonctions (§ précédente) amputait la queue instable en comparant chaque station à sa **seule voisine immédiate** (seuil relatif fixe, +10 %) : ce critère suppose que le bourrelet de confluence retombe en un seul saut net. Mesuré sur la tige du "y" : la largeur décroît **progressivement** sur plusieurs stations (9200 → 8586 → 7576 → 6566 → 5554 → 5000 µm, stable), où **chaque écart pris isolément reste sous 10 %** (le premier, 9200 vs 8586, n'est que 7 %) mais la dérive cumulée atteint +72 % — l'ancien critère s'arrêtait dès le tout premier écart local insuffisant, laissant la quasi-totalité du bourrelet en place.

Correction : le critère devient une décroissance **stricte** (avec une tolérance de 1 µm pour le bruit flottant) au lieu d'un seuil relatif fixe — toute décroissance signale qu'on est encore dans la zone d'influence de la confluence, sans hypothèse sur l'ampleur du saut d'une station à l'autre. Vérifié : la largeur de la tige du "y" (résolution 0,1 mm, où les 3 branches survivent désormais grâce au correctif de traçage) reste sous 1,2× la médiane sur toutes les stations, y compris celles immédiatement après la jonction ; aucune régression sur les cas déjà couverts (Y symétrique à 50 µm, T).

Décision : `Suitable` → une colonne (axe principal) ; `RequiresDecomposition` (Y/T) → une colonne par branche menant à une extrémité. Un anneau fin à un trou est ouvert sur une couture déterministe puis décomposé en quatre sections cycliques ; ses barreaux sont contrôlés sur plusieurs échantillons et contre les croisements locaux/globaux. `Ambiguous` (quasi circulaire sans trou) et les formes trop larges/étroites ou non appariables → **refus explicite**. Déterministe. Vérifié visuellement via `openstitch-cli auto-satin-debug --shape <forme> --output-svg` (SVG dans `tests/golden/auto-satin/`).

Intégration : la numérisation automatique et **Broderie ■ Convertir automatiquement en satin…** créent une `EmbroideryObject` par section. Toutes les sections d'un réseau gardent la même source/couleur ; le routage Lot 6 les traite donc comme un groupe multi-rail. Chaque objet reste un `SatinParams` historique à deux rails : aucune rupture du format. Les **barreaux sont stockés** dans `SatinParams.rungs` et sérialisés (`.osp` schéma v2, rétrocompatible).

Chaque section générée porte désormais une topologie explicite et déterministe : `section_index`/`section_count`, plus `start_junction` et `end_junction` quand l'extrémité touche une jonction du squelette. Les anneaux forment un cycle de quatre jonctions ; les réseaux Y/T réutilisent le même identifiant au point partagé. Ces métadonnées deviennent le bloc `SatinParams.topology` optionnel dans le document et le `.osp`. Un ancien satin sans ce bloc reste une colonne isolée. Ce lot fournit une identité de jonction fiable ; il ne modifie pas encore le routage ni l'édition coordonnée des guides.

Régressions couvertes : bande simple, réseau en T (au moins trois sections), anneau rasterisé (quatre sections, trou préservé) et pipeline complet sur `tests/fixtures/tentabrode.png`, en Debug et Release. Cette validation reste logicielle ; la qualité textile et les cas très concaves demandent encore un contrôle visuel puis machine.

Génération partielle sur formes concaves : trous silencieux dans `build_column`

État : Présent · Testé numériquement. Correctif de revue, `satin_column.cpp` (audit 2026-08-03).

Défaut trouvé par revue. `build_column` échantillonne l'axe (squelette lissé) station par station et calcule à chaque point une section transversale (`cross_section`, intersection de la normale locale avec le contour). Sur une forme **concave ou très large**, cette section transversale peut légitimement échouer pour certaines stations (normale qui rase une encoche voisine, section plus large que `max_satin_width`, milieu de l'intervalle retombant hors région) — un cas déjà connu et documenté (§ *Rails automatiques* plus haut, à propos de l'heuristique naïve). Le code de `build_column`, lui, traitait cet échec par un simple `continue` : la station disparaissait de l'axe **sans aucune trace**, et les deux stations valides encadrant le trou se

retrouvaient reconnectées directement, quelle que soit la distance réelle entre elles. Le nettoyage anti-croisement qui suit (retrait d'une station dont le quadrilatère avec la précédente s'auto-intersecte) souffrait du même défaut : les stations retirées n'étaient jamais comptées ni bornées. Sur un projet réel présentant une échancrure prononcée, ceci produisait — selon la forme — de larges zones de la région sans aucun point de broderie, des rails visiblement discontinus, des éventails ou pointes artificielles à l'endroit de la reconnexion, ou une colonne partiellement construite là où un refus propre aurait été attendu.

Correction, entièrement dans `build_column` (le squelette Zhang-Suen et le graphe de squelette ne sont pas en cause — le défaut est postérieur, dans la conversion centerline → rails) :

1. `cross_section` retourne désormais un `std::expected<..., CrossSectionFailure>` au lieu d'un `std::optional` indifférencié : la raison exacte de l'échec (axe hors région, intersection négative/positive manquante, largeur supérieure à `max_satin_width`, intervalle invalide) est conservée et propagée jusqu'aux diagnostics (`SatinColumnsResult::warnings`).
2. Chaque échantillon d'axe garde son indice et son résultat (succès ou échec) avant tout filtrage. Un échec **isolé** (une seule station, encadrée de deux succès) est comblé par **interpolation linéaire** des rails voisins, acceptée seulement si le résultat reste géométriquement plausible (intérieur à la région, largeur non aberrante, aucun croisement avec les deux stations voisines) — sinon il est traité comme un trou irrésolu. Des échecs en tête ou en queue d'axe restent un bord légitime (comportement inchangé, absorbé silencieusement comme avant : ce n'est pas un trou interne).
3. Tout trou interne non résolu par interpolation (échecs consécutifs, ou isolé mais géométriquement invalide) **refuse la colonne entière** plutôt que de la reconnecter à travers le trou. Le nettoyage anti-croisement applique le même principe : retirer une ou quelques stations sur un virage serré reste toléré (comportement préexistant, nécessaire sur `ribbon`/`y`), mais si la distance d'axe réellement pontée dépasse `max_station_gap_ratio × station_spacing`, la colonne est refusée.
4. Parce qu'un trou interne irrésolu refuse systématiquement la colonne plutôt que de conserver le plus long fragment, `st.front()`/`st.back()` restent toujours de vrais bouts d'axe au moment d'appeler `extend_tip` : aucune extrémité artificielle issue d'une coupure interne n'est jamais étendue comme s'il s'agissait d'un vrai bout du squelette.
5. Une validation finale (sur le cœur de l'axe, **avant** extension des bouts et ancrage de jonction — ces deux étapes ont leurs propres tolérances déjà testées, notamment le saut de largeur délibéré du point de fermeture `tip_min_width`) vérifie l'absence de trou résiduel, l'absence de saut de largeur incohérent entre stations adjacentes, et une couverture minimale (`min_axis_coverage_ratio`) de la longueur d'axe réellement convertie en stations. Une dernière passe, une fois les barreaux posés, rejette tout barreau retombant hors région ou toute paire de barreaux qui se croisent.

Calibration : les trois nouveaux seuils (`max_station_gap_ratio` = 5,0 × `station_spacing`, `min_axis_coverage_ratio` = 85 %, `max_adjacent_width_jump_ratio` = 75 %) ont été choisis pour ne régresser aucune fixture existante — `ribbon` et `y`, sur des virages serrés, écartent normalement 1 à 3 stations d'affilée lors du nettoyage anti-croisement, et une jonction à haut degré (`cross`, `h`) peut légitimement voir sa couverture chuter autour de 80 % tout près du nœud du squelette (bourrelet de confluence, cf. *Ancrage des jonctions* plus haut).

Vérifié : suite complète `test_auto_satin` inchangée (aucune régression sur `rectangle`, `capsule`, `ribbon`, `s`, `y`, `t`, `cross`, `h`, `ring`, `wide`) ; nouvelle fixture `notch` (bande de 40 × 5 mm entaillée d'une encoche en V profonde sur un bord, resserrant la largeur locale à 1 mm) et sa variante plus sévère `pinch` (resserrement à 0,3 mm) produisent, à chaque exécution, **soit** un refus explicite (`refusal` non vide, aucune colonne), **soit** une colonne complète sans trou entre stations consécutives, avec tous les barreaux dans la région et aucun croisement entre barreaux — jamais un résultat partiel ; déterminisme vérifié (mêmes rails à chaque exécution sur les deux fixtures).

HP-STI-018 Phase B.5b : `extend_tip` direction-aware sur les fourches Y (2026-10)

État : Présent · Testé sur le chemin corridor.

Après la collecte direction-aware de Phase B.5, `y` et `y_symmetric` échouaient encore sous `SatinColumnsParameters::use_corridor_tracing_dev_only` avec un refus `croisement entre`

`barreaux #0/1`. L'isolation `extend_open_ends=false` montrait que les trois branches étaient déjà construites : le défaut venait de l'extension du bout ouvert, pas de `trace_corridor` ni de `find_stable_corridor_end`.

La tentative naïve de réutiliser directement le partage par demi-plan de `nearest_boundary_feet_oriented` n'était pas suffisante dans `extend_tip` : la marche d'embout interroge un point très proche du cap ouvert, donc le point le plus proche de chaque demi-plan peut être le mur d'embout lui-même au lieu du rail latéral.

Correction : `extend_tip` tente d'abord une sonde latérale par cône angulaire autour de `+N` et `-N`, ce qui écarte les candidats quasi tangentiels du cap. Le résultat est raccordé à la station précédente par continuité A/B ; si la station finale au plancher `tip_min_width` croiserait immédiatement le barreau précédent, elle est simplement omise, et la dernière station saine reste le bout de colonne.

Vérifié : le test corridor Phase B.5b affirme que `y` et `y_symmetric` produisent 3 colonnes, sans refus et sans diagnostic `croisement entre barreaux`. `trident` reste une limite distincte : sa branche en pointe est un effilement géométrique réel où les deux côtés convergent vers le même point du contour.

Pointes de colonne réduites à quelques dizaines de µm : plancher franchi en un seul pas dans `extend_tip`

État : Présent · Testé numériquement (unitaire + intégration sur projet réel). Correctif d'audit (2026-08-11), retour utilisateur : *« le satin fait n'importe quoi »* sur un logo circulaire à motif de circuit imprimé finement détaillé (nombreuses formes fines à extrémité OUVERTE — pointe de piste, plot de connecteur).

Défaut trouvé en investiguant le retour utilisateur. `extend_tip` prolonge un bout ouvert de colonne au-delà de ce que le squelette aminci couvre, en marchant par pas de `stepLen` le long de la tangente sortante et en ré-échantillonnant une section transversale à chaque pas, tant que la largeur rétrécit. La boucle **poussait la station dans `ext` avant de vérifier** si sa largeur avait déjà atteint le plancher voulu (`tip_min_width`) :

```
Station st;
st.width = width;
ext.push_back(st);        // ← poussée d'abord...
...
if (width <= tipMinWidth) {
   return ext;            // ← ...le plancher n'est vérifié qu'ensuite
}
```

Or rien ne borne la largeur **au-dessus** du plancher au moment de la poussée : si un pas de marche fait chuter la largeur de, disons, 200 µm à 34 µm en une seule itération (taper serré, pas de marche relativement grossier par rapport à la vitesse de rétrécissement — le cas courant pour une petite forme pointue), c'est cette station à 34 µm qui se retrouve poussée dans `ext`, **avant** que la condition d'arrêt n'ait la main. La bissection de fermeture qui suit (laquelle, elle, respecte correctement `tipMinWidth` : `half = max(tipMinWidth * 0.5, 1.0)`) n'était alors jamais atteinte pour cette extrémité — sa station de tête restait celle, sous-plancher, déjà poussée.

Un second facteur amplifiait la fréquence du défaut sans en changer la nature : `tip_min_width` valait **50 µm par défaut**, sous le pas de quantification DST lui-même (0,1 mm, cf. `formats/dst.hpp`) — un barreau à tout juste 50 µm n'aurait de toute façon jamais représenté un point cousu exploitable, même sans le défaut d'ordre de poussée ci-dessus.

Impact mesuré sur un projet réel (logo circulaire, ~340 régions segmentées, `tests/integration/test_pipeline.cpp`, diagnostic « satin pointes fines (GISTRE) ») : **178 des 190 colonnes satin (94 %)** avaient au moins un barreau sous 0,8 mm, la plupart entre 33 et 51 µm — quasiment toute colonne à extrémité ouverte était concernée, puisque `extend_tip` s'applique à chacune. Une colonne satin dont le dernier barreau mesure quelques dizaines de µm génère, selon la machine et le fil, un point quasi nul, un bourrage de fil ou une casse — exactement le symptôme rapporté (*« ça fait n'importe quoi »*), et son omniprésence sur ce type d'image (nombreuses petites formes en pointe) explique

pourquoi il n'était pas passé inaperçu plus tôt sur des images plus simples (peu de bouts ouverts fins).

Correction, deux volets indépendants :

1. **Ordre de la boucle inversé** : la largeur est comparée au plancher *avant* de pousser la station, pas après. Une station déjà sous le plancher n'est jamais poussée — la marche s'arrête un pas plus tôt et laisse la bissection de fermeture (déjà correcte) fermer proprement au plancher exact.
2. **`tip_min_width` porté de 50 à 300 µm** : reste nettement sous `min_satin_width` (0,8 mm, le seuil « satin digne de ce nom » pour toute la colonne — la pointe reste une pointe, pas un bout à pleine largeur), tout en dépassant largement le pas DST avec une marge de sécurité (×3).

Un défaut apparenté, dans la boucle dense principale de `compute_column_stations` (partagée par `build_column` et `build_parametric_object`, donc par les deux modes de géométrie) : aucun plancher de largeur n'existait pour une station de CORPS (pas de bout), qu'elle vienne de l'axe principal ou d'une interpolation comblant un échec isolé de `cross_section` (§ *Génération partielle sur formes concaves* ci-dessus). Une région globalement fine mais localement très étroite sur une portion de son squelette pouvait ainsi passer le test d'éligibilité (`satinability.cpp` compare `max_satin_width` à la largeur **maximale** relevée sur toute la région, pas à sa largeur typique) puis dégénérer en aval. Ajout d'un nouveau cas d'échec `CrossSectionFailure::TooNarrow` (largeur sous `min_satin_width`), traité par le mécanisme d'échec déjà en place (§ *Génération partielle*) : un échec isolé est comblé par interpolation *si l'interpolée elle-même dépasse le plancher* (`minWidth` ajouté aux critères de `interpolated_station_valid`, qui ne vérifiait jusqu'ici que `maxWidth` — sans quoi une station trop étroite se faisait réaccepter telle quelle par simple interpolation de position avec ses voisines, sans jamais revalider sa largeur) ; un échec étendu refuse la colonne entière plutôt que de produire des rails quasi confondus sur toute sa longueur.

Vérifié : suite `test_auto_satin` complète inchangée (1999 assertions, 51 cas, aucune régression sur les fixtures existantes) ; nouveau cas sur la géométrie EXACTE d'une région du projet réel en cause (`tests/unit/auto_satin/test_columns.cpp`) ; diagnostic d'intégration sur le projet réel complet (`tests/integration/test_pipeline.cpp`, § GISTRE) : 0 barreau sous 0,29 mm après correctif contre 178/190 colonnes touchées avant, pire séparation observée 299 µm (au plancher voulu, à l'arrondi µm près).

Ligne de coupe manuelle pour les jonctions satin (`geometry::cut_path_set`, 2026-08-12)

État : Présent (outil `DrawSatinCutLine`) · Complément, pas remplacement, de la détection automatique de jonctions ci-dessus.

Origine : audit de la pipeline Ink/Stitch

Sur des logos complexes réels (ex. sceau circulaire à motif de circuit imprimé), le squelette automatique (`build_satin_columns`) peine parfois sur des jonctions ambiguës — nœuds en Y/T proches, branches très courtes, squelette bruité par un contour segmenté imparfaitement. Un audit de la pipeline auto-satin d'Ink/Stitch (GPL ; documentation publique consultée, aucun code repris — implémentation ci-dessous entièrement originale) a montré que cet outil ne tente PAS de détecter les jonctions automatiquement : son "Stroke to Satin" calcule une ligne centrale puis s'appuie sur des "cut lines" tracées à la main par l'utilisateur pour séparer une forme aux endroits voulus, avant conversion en colonnes. Le remplissage tatami sur formes complexes suit le même principe ("break apart" manuel documenté comme solution officielle).

Plutôt que de renoncer à la détection automatique (qui reste supérieure sur la majorité des formes — voir toutes les sections ci-dessus), l'outil de ligne de coupe est ajouté en complément : un geste manuel simple pour les cas où l'automatique reste ambigu, sans jamais devenir la voie par défaut.

Primitive géométrique : `geometry::cut_path_set`

`libs/geometry/include/openstitch/geometry/cut.hpp` / `libs/geometry/src/cut.cpp`. Signature :

```
Result<std::vector<PathSet>> cut_path_set(const PathSet& region, Vec2um a, Vec2um b,
                                          Micrometers cut_width = Micrometers{20});
```

Retire une fine bande rectangulaire (`cut_width`, quelques dizaines de µm par défaut — négligeable une fois cousu) centrée sur le segment `a`-`b`, prolongée très au-delà de la région dans les deux sens pour garantir une traversée complète quels que soient les deux points fournis (l'utilisateur n'a donc qu'à tracer un segment qui *croise* la jonction visée, pas à viser précisément les bords de la forme). Implémentation via une booléenne Clipper2 `Difference` (même encapsulation par fichier que le reste de `libs/geometry` — ADR-005 : aucun type Clipper2 ne sort de `cut.cpp`), qui garantit une vraie séparation topologique — une coupe infiniment fine laisserait parfois deux morceaux qui se retouchent en un point, toujours une seule composante connexe aux yeux d'un traitement ultérieur.

Renvoie un morceau par composante connexe résultante : 1 seul morceau (la région quasi inchangée) si la ligne ne traverse pas réellement la région ou est dégénérée (`a == b`) — à l'appelant de vérifier `size() >= 2` pour savoir si la coupe a réellement séparé quelque chose. Testé (`tests/unit/geometry/test_cut.cpp`, 6 cas / 29 assertions) : séparation d'un carré, coupe du pont d'un haltère (cas de jonction Y/T), ligne hors région sans effet, points confondus sans effet, déterminisme, largeur de bande bornée.

Outil desktop (`Tool::DrawSatinCutLine`)

Réutilise le mécanisme presser-glisser-relâcher déjà en place pour l'outil Bézier (`CanvasView::bezierPointDraggingMm`/`bezierPointCommittedMm`) — un second couple de connexions sur les mêmes signaux, sans aucune modification de `CanvasView` (les gestionnaires Bézier existants font déjà un retour anticipé hors de l'outil `DrawBezier`, donc no-op automatique quand l'outil actif est `DrawSatinCutLine`).

Séquence (`apps/desktop/main_window.cpp`, `createSatinObjectWithCutLine`) :

1. Requiert une forme vectorielle sélectionnée (`selectedObject_`) ; sinon, message dans la barre de statut plutôt qu'une boîte de dialogue bloquante.
2. Glisser trop court (< 200 µm, clic net) ignoré silencieusement — geste probablement involontaire.
3. `geometry::cut_path_set` sur `source->paths.front()` ; `< 2` morceaux -> avertissement explicite ("tracez la ligne bien d'un bord à l'autre").
4. Boîte de dialogue densité/compensation/sous-couche, identique à celle de `createSatinObject()` (création automatique) pour cohérence d'expérience.
5. Chaque morceau est passé indépendamment à `auto_satin::build_satin_columns` (mode `Parametric`) : un morceau qui ne produit aucune colonne (trop petit/dégénéré) est simplement ignoré plutôt que d'annuler toute l'opération — la coupe a pu réussir pour la jonction visée même si un fragment marginal ne l'est pas.
6. Avertissement de largeur excessive identique à §*Colonne trop large*.
7. Un seul `commands::AddObjectBatchCommand` ("Colonne satin (ligne de coupe)"), annulable en un geste comme toutes les créations de forme de cette fenêtre.

Un seul geste = une seule coupe (portée v1, pas d'accumulation multi-coupes) : succès -> retour automatique sur l'outil Sélection pour enchaîner sur la retouche du résultat ; échec -> reste sur l'outil pour retracer.

Testé (`tests/unit/desktop/test_main_window.cpp`, `satinCutLineToolSplitsSelectedShapeIntoTwoSatinColumns`) : glisser souris réel à travers un rectangle allongé sélectionné -> deux objets satin indépendants, undo/redo, retour automatique sur Sélection.

Objets satin paramétriques (mode `Parametric`, refonte 2026-08-04)

État : Présent, activable par `SatinColumnsParameters::geometry_mode` (CLI : `--satin-geometry=parametric`) · Mode par défaut toujours `Legacy` · Testé numériquement (44 cas legacy inchangés + 6 nouveaux cas parametric) · Validé visuellement (SVG) sur `rectangle`, `capsule`, `y`, `t`, `cross`, `trident`.

Défaut architectural du pipeline `Legacy`. `build_column` convertit chaque station dense (une tous les `station_spacing` = 500 µm) directement en un nœud de rail `Corner` (`rail_path`) et un barreau (`SatinRung`) — alors que `geometry::PathNode` porte déjà `tan_in`/`tan_out` (Bézier, cf. `polyline.hpp::flatten`, De Casteljau adaptatif) et que `stitch_generation::fill_satin_columns` implémente déjà l'essentiel de l'étape « génération par longueur d'arc entre guides » (`ladder_correspondence` et `resample_by_medial_spacing`, § section suivante). Le vrai problème n'était donc pas une architecture Bézier manquante, mais que **rien ne l'utilisait** : même en y injectant des `tan_in`/`tan_out`, `to_points` (dans `satin.cpp`) lisait `node.pos` brut sans jamais appeler `flatten` — corrigé (§ plus bas). Les jonctions (`StableBranchEnd`/`JunctionSeparator`/secteurs/`JunctionCore`) opéraient sur ces nœuds denses avec une contrainte de partition exacte — fragile par construction : c'est cette contrainte de partition, pas le nombre de nœuds en soi, qui produisait la parade de défauts historiques (ancre centrale, diagonale, éventail, noyau énorme, bridges égarés).

Principe du mode `Parametric` : au lieu d'une polyligne dense, un objet satin devient `ParametricSatinObject` — deux rails Bézier ÉPARS (`geometry::Path`, quelques `PathNode` à tangentes) couplés par un petit nombre de `SatinControlPair`/`SatinAngleGuide`. Les points de couture ne font PAS partie de cette représentation : ils sont dérivés à la demande par `fill_satin_columns`, qui aplatit `rail_a`/`rail_b` avant de les parcourir. La couche d'analyse dense (`compute_column_stations`, extraite de l'ancien `build_column` sans changement de comportement) reste PARTAGÉE par les deux modes — seule la finalisation diffère.

1. **Sélection des paires structurantes** (`select_structural_indices`) : union de critères déterministes — les deux extrémités, les extrema de largeur (`width_extrema_indices`), les sauts de largeur ADJACENTS (`adjacent_width_jump_indices` — cf. bug ci-dessous), les maxima de courbure (`curvature_maxima_indices`), et une simplification Douglas-Peucker de CHAQUE rail en réutilisant `geometry::simplify` (indices retrouvés par correspondance exacte de position, jamais une réimplémentation de l'algorithme). Fusion des indices trop proches (`minimum_control_pair_spacing`).
2. **Ajustement Bézier conjoint** (`fit_both_rails`/`fit_both_rails_recursive`) : un seul ensemble d'ancrages PARTAGÉ par les deux rails (jamais deux subdivisions indépendantes qui dériveraient l'une de l'autre — § étape 6, chaque paire structurante correspond exactement à une paire de points sur les deux rails). Par intervalle, `fit_cubic_segment` résout un cubique simple (Schneider, moindres carrés à tangentes fixées, sans reparamétrisation itérative) ; subdivision ADAPTATIVE (station la plus déviante, ou milieu si dépassement pur d'espacement) tant que l'erreur (distance OU tangente) ou l'espacement dépasse les seuils configurés, bornée par `minimum_control_pair_spacing` et une profondeur maximale.
3. **Lignes d'angle** (`SatinAngleGuide`) : une par paire structurante, avec son abscisse curviligne sur chaque rail APLATI (pas un paramètre Bézier brut) — consommées telles quelles par `fill_satin_columns`.

Deux bugs de fit trouvés en généralisant sur les six formes cibles, tous deux avec un symptôme identique (rail sortant de la région) mais des causes distinctes :

1. **Longueur de poignée = corde brute, pas sa PROJECTION sur la tangente.** Sur un bout OUVERT à plat (`extend_tip`, plancher `tip_min_width`), la station de fermeture saute presque uniquement en LARGEUR sur une distance d'axe minime — sa tangente fixée (avant/horizontale) est alors quasi PERPENDICULAIRE à la corde réelle. `fallbackP1 = p0 + tangente × (corde/3)` utilisait la longueur de corde totale (dominée par le saut perpendiculaire) comme échelle de poignée le long d'une direction quasi-orthogonale — la poignée partait très loin dans le mauvais axe avant de devoir rebrousser chemin, débordant largement hors la forme (démontré : rectangle, poignée à x=40775 pour un bord à x=39995, soit +780 µm de débordement). Corrigé : la longueur d'échelle des poignées est la PROJECTION de la corde sur la tangente (`scale0`/`scale1` = `|dot(p3-p0, tangente)|`), jamais la corde brute — sur un coin quasi perpendiculaire, la projection est naturellement courte, produisant une poignée courte proche d'un coin SANS code spécial pour détecter « est-ce une pointe » (§ étape 5 : « discontinuité de tangente sur un coin structurel » — la géométrie seule tranche).

2. **`width_extrema_indices` ne détecte que les pics/creux LOCAUX, pas un SAUT suivi d'un PALIER.** Une station de fermeture (largeur plancher) collée à la première vraie station de corps (largeur pleine) n'est ni un maximum ni un minimum local (elle est suivie d'un PALIER à largeur pleine, pas d'une redescente) — ce schéma passait inaperçu, laissant un seul intervalle couvrir un saut de largeur de 100×, avec la même conséquence que le bug précédent. Ajouté `adjacent_width_jump_indices` : détecte tout saut relatif entre deux stations ADJACENTES (pas seulement les extrema), complémentaire et nécessaire.

Robustesse de la subdivision adaptative : le point de coupure « idéal » (pire erreur, ou milieu) peut violer `minimum_control_pair_spacing` — typiquement collé au tout premier/dernier point d'un intervalle contenant un saut de largeur en bordure. Plutôt que d'abandonner toute subdivision (et garder un segment unique très imprécis, potentiellement débordant), le point valide le plus proche de la coupure idéale est recherché.

Validation (`validate_parametric_object`, § étape 12) : échantillonnage dense des rails APLATIS (jamais les seuls nœuds de contrôle) — région (tolérance dérivée de `bezier_fit_tolerance` : un rail trace le contour PAR CONSTRUCTION donc est légitimement sur le bord, `in_region` seul est ambigu pile sur une arête discrétisée), auto-intersection, croisement entre les deux rails, lignes d'angle non dégénérées et non croisées, correspondance monotone. Un objet refusé n'écarte QUE cette branche (`warnings`), jamais la génération entière.

Jonctions : recouvrement local, sans ancre (remplace StableBranchEnd/JunctionSeparator/secteurs/JunctionCore en mode `Parametric`)

Une branche = un `ParametricSatinObject` INDÉPENDANT (déjà vrai structurellement : `build_satin_columns` construit une colonne par arête de squelette). Aucune ancre centrale commune, aucun sommet de bissectrice, aucun déplacement de nœud terminal, aucune diagonale, aucune rampe, aucun noyau résiduel calculé par soustraction globale — l'apparat entier `StableBranchEnd`/`JunctionSeparator`/secteurs/`JunctionCore` (toujours présent et actif en mode `Legacy`) est simplement absent du chemin `Parametric`.

À la place : après amputation de la queue instable (`trim_unstable_junction_tail`, inchangée), `extend_into_confluence` (variante de `extend_tip` sans critère de convergence — la largeur PEUT légitimement croître en entrant dans le bourrelet, c'est le recouvrement voulu) ré-échantillonne quelques sections transversales réelles au-delà du dernier point stable, jusqu'à une distance bornée (`junction_overlap_target` = fraction `junction_overlap_ratio` de la largeur locale, bornée par `junction_overlap_min`/`junction_overlap_max`). S'arrête dès que `cross_section` échoue (bord réel atteint, ou — cas `t`/`cross`, voir plus bas — section perpendiculaire qui balaie la branche voisine).

Cas `t`/`cross` (branches perpendiculaires de même largeur) : recouvrement mesuré = 0, et c'est correct. Rien n'est amputé par `trim_unstable_junction_tail` (aucun bourrelet à détecter sur une confluence symétrique à largeur constante), donc chaque rail atteint déjà le bord de la confluence par son propre corps — la couverture visuelle de la confluence est déjà complète sans extension. Y prolonger échouerait de toute façon dès le premier pas : une section transversale prise DANS le carré de confluence, perpendiculaire à cette branche, balaie alors la longueur de la branche perpendiculaire et dépasse `max_satin_width` immédiatement. **Cas `y`/`trident`** (branches à angle, largeurs différentes) : une queue instable est réellement amputée, et le recouvrement mesuré est non nul (0,3 à 2,5 mm selon la largeur locale).

Ordre de couture (`SatinJunctionPlan`, `plan_junction_stitch_order`) : heuristique déterministe — branche la plus large en premier (cousue dessous), branches plus fines ensuite par largeur décroissante (cousues dessus, masquent les transitions des précédentes). Ne modifie aucune géométrie, ordonne seulement les indices déjà construits.

Résultat mesuré sur `trident` (critère explicite : 3 objets, un par branche, aucune diagonale, aucun rail traversant la confluence, aucun éventail, recouvrements locaux visibles et bornés, déterministe) : 3 objets indépendants (17/27/48 stations denses → 5/3/10 paires structurantes), aucun croisement de rail entre branches (vérifié sur les rails APLATIS, pas les seuls nœuds de contrôle), recouvrement 0,3–1 mm sur les

deux branches fines, ordre de couture `[large, moyenne, fine]`. Inspection visuelle (`trident-parametric.svg`) : rails visuellement lisses (courbure Bézier, pas une succession de petits segments), zone de recouvrement visible en jaune translucide, aucune grande diagonale, aucun grand trou central.

Branché côté `apps/desktop` ET `autodigitize.cpp` (audit satin, 2026-08-10, suite). Les trois chemins de création satin manuelle (`autoConvertToSatin`, `createSatinObject`, `setStitchType`) ET la numérisation automatique (`auto_digitize`, quand `AutoOptions::use_auto_satin` est actif — le défaut) demandent désormais `geometry_mode = Parametric` et lisent `SatinColumnsResult::parametric_columns` en priorité — repli automatique sur `columns` (mode `Legacy`) géré À L'INTÉRIEUR de `build_satin_columns` lui-même (anneaux, cas refusés), donc aucun cas ne perd de couverture. Un seul point de conversion vers `document::SatinParams` (`autodigitize::satin_params_from_column`, template sur `SatinColumnGeometry`/`ParametricSatinObject` — mêmes noms de champs, exporté depuis `libs/autodigitize` et réutilisé tel quel par `apps/desktop`) évite de dupliquer la logique entre les deux modes ET entre les deux appelants. Les commandes d'édition de rail satin existantes (`MoveSatinRailNodeCommand`, etc.) opèrent sans modification sur les rails `Parametric` : ce sont de simples `geometry::Path`, seulement plus épars.

Limites connues restantes : les anneaux (`region.holes.size()==1`) restent exclusivement `Legacy` même en mode `Parametric` demandé (`parametric_columns` vide, `columns` peuplé comme avant) ; aucune intégration éditeur DÉDIÉE (déplacer une `SatinControlPair` comme entité propre, verrouiller une `SatinAngleGuide` indépendamment du nœud de rail qu'elle partage aujourd'hui) n'est implémentée — seule l'édition de nœud de rail générique est disponible, ce qui couvre l'usage courant mais pas la manipulation fine des paires structurantes en tant que telles.

Génération par barreaux (`fill_satin_columns`, Lot 2)

État : Présent · Testé numériquement · Validé visuellement (SVG) · non validé simulateur/physique.

Quand un satin porte des **barreaux** (`SatinParams.rungs` ≥ 2), la génération utilise `fill_satin_columns` au lieu de `fill_satin` :

- **Correspondance par sections** : chaque paire de barreaux consécutifs découpe les rails en intervalles correspondants, appariés par correspondance locale **ladder** (voir *Correction de l'appariement* plus haut — rails de longueurs, nombres de nœuds et courbures différents autorisés, sans éventail). Les barreaux sont **traversés exactement**.
- **Espacement perpendiculaire** : le pas n'est plus mesuré le long d'un rail mais sur la **ligne médiane** (≈ perpendiculaire aux fils) — on ré-échantillonne la médiane de chaque intervalle par la densité demandée.
- **Séquence** L0, R0, L1, R1, … déterministe (chaque point cousu traverse la colonne). Un satin **sans barreaux** (manuel/legacy) retombe sur `fill_satin`.

Vérifié : espacement médian régulier, barreaux exacts, rails de longueurs différentes, déterminisme (`tests/unit/stitch/test_satin.cpp`) ; zigzag superposé dans les SVG `tests/golden/auto-satin/`.

Saut plutôt qu'un point continu disproportionné entre deux barreaux non-ruban (audit lettres, 2026-08-12)

État : Présent (`SatinResult::jump_before`) · Testé numériquement (géométrie synthétique + géométrie réelle) · non validé simulateur/physique.

Origine : retour utilisateur sur un projet réel (sceau "GISTRE", texte en périphérie) — le satin automatique produisait un résultat visuellement incohérent sur les lettres, en particulier au coude intérieur d'une lettre en T. Export debug de l'objet en cause (`Satin region 301 - section 1/4`) : deux barreaux consécutifs, `rung0` quasi ponctuel (301 µm, une pointe de section) immédiatement suivi de `rung1` à 5431 µm — rail A avance presque horizontalement entre les deux, rail B presque verticalement (~88° d'écart). `fill_satin_columns` interpolait quand même un ruban continu sur cet intervalle : la ré-échantillonnage par ligne médiane (`resample_by_medial_spacing`) atteint son abscisse cible sur un rail AVANT l'autre quand

les deux rails divergent autant, produisant un avant-dernier fil dont un côté est déjà au barreau cible et l'autre non — un point continu d'environ 5 mm, puis le barreau exact lui-même à la même largeur : une diagonale visible qui traverse la lettre, suivie d'un aller-retour quasi identique.

Piste écartée : un seuil de largeur absolue (barreau > X mm ⇒ saut). Les sections de lettre touchant une jonction ont légitimement des barreaux larges (5 à 6 mm, cf. *Jonctions : recouvrement local* plus haut — `extend_into_confluence` fait CROÎTRE la largeur en entrant dans le recouvrement, par conception) : un seuil absolu aurait déclenché un saut sur un ruban parfaitement valide, juste large.

Correctif — `non_ribbon_interval` (`libs/stitch_generation/src/satin.cpp`) : compare la DIRECTION d'avance de chaque rail entre deux barreaux consécutifs (`a1.pa - a0.pa` vs `a1.pb - a0.pb`), pas leur largeur. Au-delà de ~75° d'écart (et seulement si les deux avances dépassent 0,8 mm — sous ce plancher, le bruit sous-résolution ne justifie aucun saut), la colonne ne se comporte plus comme un ruban sur cet intervalle : aucun fil intermédiaire n'est interpolé, le barreau suivant devient un point de reprise après un SAUT (aiguille levée) plutôt qu'un point cousu. Une terminaison effilée ORDINAIRE (largeur qui décroît jusqu'à un point, rails toujours co-directionnels) garde des directions colinéaires et ne déclenche donc jamais de saut — seul un changement de direction franc le fait, jamais une simple variation de largeur. Pratique standard en broderie (logiciels commerciaux du métier : lever le fil plutôt que forcer un point disproportionné) plutôt qu'une invention propre au projet.

`SatinResult::jump_before` (indices dans `satin`) porte l'information jusqu'à `stitch_generation::generate.cpp`, qui émet un `Jump` (aiguille levée) au lieu d'enchaîner un `Stitch` à ces indices (`emit_polyline_with_breaks`, distincte d'`emit_polyline` pour ne pas perturber les passes sans saut — sous-couches, locks). L'orientation entrée/sortie (§ *Entrée/sortie et points de fixation*) inverse `result.satin` : les indices de `jump_before` sont recalculés (`idx_after = taille - idx_avant`, l'arête cassée reste la même arête physique, cf. commentaire dans `generate.cpp`) puis retriés.

Vérifié : `tests/unit/stitch/test_satin.cpp` (coin quasi perpendiculaire → saut, garde-fou anti-faux-positif sur une terminaison effilée ordinaire) ; `tests/unit/auto_satin/test_columns.cpp`, géométrie EXACTE de la lettre en T en cause (contour à 37 sommets) — invariant testé : aucun point cousu ne dépasse la largeur du plus large barreau RÉEL qui l'encadre (pas "aucun point large", qui serait faux pour une jonction légitimement large) ; le mécanisme de saut s'engage bien sur cette géométrie réelle (2 des 4 sections).

Seuil angle/distance plutôt qu'angle brut (deuxième cas réel, 2026-08-13)

État : Présent (`kJumpDegPerMm`) · Testé numériquement (deux cas réels distincts + garde-fous synthétiques) · non validé simulateur/physique.

Défaut trouvé en usage réel : un DEUXIÈME export debug (même sceau, `Satin region 267 - section 1/6`) montrait le même artefact — un point continu d'environ 3,3 mm faisant l'aller-retour — sur un coude à seulement **~68,5°**, sous le seuil de 75° calibré sur le premier cas (~88°). Baisser le seuil brut à 50° corrigeait ce deuxième cas mais **régressait** le test « virage » existant (Lot 3, § *Sous-couches et compensation*) : un virage LÉGITIME (rail intérieur droit et court, rail extérieur en long détour) présente un angle bout-à-bout de **~55°** — à peine 13° sous le vrai défaut à 68,5°. La fenêtre de seuil brut sûre entre ces deux cas était trop étroite (quelques degrés) pour être fiable face à un futur troisième cas.

Correctif — l'angle brut est remplacé par un rapport **angle/distance parcourue** (`kJumpDegPerMm = 10°/mm`) : ce qui distingue réellement un virage légitime d'un défaut n'est pas l'angle en lui-même, mais la BRUTALITÉ du changement de direction. Le virage légitime étale son virage sur ~14 mm (≈4°/mm) ; les deux défauts réels observés concentrent un angle comparable sur seulement 3,7 mm (≈24°/mm) et 1,5 mm (≈45°/mm) — plus d'un ordre de grandeur d'écart entre le cas légitime le plus proche et le défaut le plus proche, une marge bien plus sûre qu'un seuil d'angle brut ne pouvait l'offrir. `kJumpMinSpan` (0,8 mm) inchangé.

Vérifié : nouveau cas dans `tests/unit/stitch/test_satin.cpp`, géométrie EXACTE des rails/barreaux du deuxième export debug (7 nœuds Bézier épars par rail) ; suite complète de `test_satin.cpp`/`test_satin_pairing_metrics.cpp` sans régression, y compris le test « virage » qui avait révélé la limite du seuil brut.

Précision a posteriori (audit complémentaire, même jour) : les largeurs proches de `max_satin_width` observées sur 2 des 4 sections dès leur première station ne sont PAS un artefact de recouvrement de jonction (`extend_into_confluence`) comme d'abord supposé — vérifié en dumpant le graphe de squelette brut de cette lettre (`SkeletonGraph`, 5 arêtes/6 nœuds) : ces deux sections sont deux branches INDÉPENDANTES (aucune jonction), et leur largeur de départ élevée reflète simplement la largeur réelle de l'empattement à cet endroit. Le squelette complet a en réalité 5 arêtes ; la 5e (~28 mm, reliant le crochet latéral gauche à la jonction du bas) est REJETÉE dans son intégralité : `compute_column_stations` y mesure une largeur locale >6 mm sur la totalité de ses ~54 stations échantillonnées (vérifié : ~8,1 mm à mi-parcours, calcul indépendant par intersection de segments) — un empattement réellement trop large pour du satin à cet endroit, pas un bug de mesure. Voir *Avertissements auto-satin remontés à l'utilisateur* ci-dessous pour la suite donnée à cette branche rejetée.

Repli tatami sur une branche auto-satin rejetée, jamais de zone sans point (2026-08-13)

État : Présent (`geometry::subtract_polygons`, `AutoResult::warnings`) · Testé numériquement (chaîne complète, géométrie réelle rasterisée, vérification de couverture géométrique de bout en bout) · non validé simulateur/physique.

Défaut trouvé en usage réel (suite directe de l'audit ci-dessus) : quand `build_satin_columns` rejette une branche de squelette (ex. trop large, comme la 5e arête de la lettre en T ci-dessus), le rejet est bien diagnostiqué (`SatinColumnsResult::warnings`) mais `autodigitize.cpp` ne lisait jamais ce champ. Tant qu'au moins UNE branche de la même région réussit (`sectionCount > 0`), la numérisation automatique se poursuit normalement, satisfaite d'avoir produit *quelque chose* — la portion couverte par la branche rejetée, elle, ne reçoit RIEN (ni satin, ni tatami, ni contour), sans le moindre signal dans l'interface. Sur la lettre en T de l'audit : ~28 mm de squelette disparaissaient silencieusement à chaque numérisation automatique de ce logo.

Première itération (incomplète) : remonter `SatinColumnsResult::warnings` jusqu'à une boîte de dialogue utilisateur (`AutoResult::warnings`), sans générer de point de repli. Retour utilisateur explicite : *"il faut absolument que ça couvre toute la zone"* — un avertissement, aussi visible soit-il, laisse toujours un trou physique dans le motif cousu. Étendu en repli tatami automatique.

Primitive géométrique — `geometry::subtract_polygons(base, cutouts)` (nouveau, `libs/geometry`) : différence booléenne Clipper2 (règle **NonZero**, pas EvenOdd comme `clean.cpp`/`cut.cpp`) entre `base` et l'union de `cutouts`. NonZero choisi délibérément : des bandes de colonnes satin adjacentes qui se chevauchent légèrement (même barreau partagé) s'annuleraient localement sous EvenOdd, laissant un trou parasite au milieu d'une zone pourtant couverte. Comme NonZero est sensible à l'orientation (contrairement à EvenOdd), `base.outer`/`base.holes`/chaque `cutout` sont renormalisés en interne (CCW/CW) avant l'opération — l'orientation d'entrée n'est PAS garantie ailleurs dans `geometry/`, qui utilise EvenOdd précisément pour s'en affranchir.

Détection du besoin de repli — signal STRUCTUREL, jamais une correspondance de texte sur `warnings` : `network.debug.graph.edges.size() > sectionCount` (le squelette élagué compte plus d'arêtes que de sections produites). Une correspondance sur le texte des avertissements a été tentée puis abandonnée : `warnings` porte aussi des messages purement informatifs sans rapport avec une couverture manquante (ex. *"couture fermée : routage cyclique de quatre sections"* sur un anneau déjà parfaitement couvert), et un simple calcul d'aire du reliquat géométrique SANS ce garde-fou déclenchait à tort sur des réseaux satin entièrement réussis : un rail satin (surtout en mode Parametric, ajusté en Bézier épars) approxime toujours la forme source avec une légère tolérance même quand tout réussit, produisant plusieurs mm² de reliquat "naturel" aux pointes/jonctions — repérage régressé sur les fixtures existantes ("réseau en T", "anneau fin") avant d'ajouter ce garde-fou structurel.

Recouvrement délibéré, pas une couture pile au bord — `shrink_strips_for_cutout` rétrécit chaque bande satin de 0,4 mm avant de la soustraire de la région : le territoire de repli déborde ainsi À L'INTÉRIEUR de la bande satin plutôt que de s'arrêter exactement à son bord. Défaut trouvé par revue lors du premier essai (grandir la bande AVANT soustraction plutôt que la rétrécir) : l'effet est inversé — le repli recule et laisse un interstice, pas un recouvrement. Un recouvrement (léger double-point) est anodin ; un interstice ne l'est pas. Même principe que le recouvrement de jonction (`extend_into_confluence`

ci-dessus) et la pratique standard du métier (chevaucher plutôt que raccorder pile, cf. audit Wilcom Hatch — Column B/miter joints : *"les trous ne sont pas des échecs, c'est de la physique attendue ; il faut faire disparaître le trou par le recouvrement, pas par la géométrie seule"*).

Mise en œuvre (`libs/autodigitize/src/autodigitize.cpp`) : pour chaque section satin réussie, une bande approximative (rail A aller + rail B retour, rails aplatis) ; la région source moins l'union rétrécie de ces bandes donne le(s) reliquat(s) ; chaque reliquat ≥ 0,5 mm² (seuil délibérément bas et INDÉPENDANT de `min_fill_area_mm2` — ce dernier répond à "cette région entière vaut-elle un remplissage ?", pas à "ce reliquat déjà largement couvert mérite-t-il d'être comblé ?") devient un nouvel objet vectoriel + un objet de broderie tatami, nommés explicitement ("zone non couverte par le satin" / "Remplissage repli région"). `AutoResult::warnings` reste rempli (préfixé par la région source) : les avertissements expliquent maintenant POURQUOI une zone est en tatami plutôt qu'en satin comme le reste de la région, la boîte de dialogue desktop (`autoDigitize()`/`segmentWithAi()`) étant reformulée en conséquence.

Vérifié : `tests/unit/geometry/test_boolean.cpp` (7 cas : trou net, recouvrement toute la région, deux cutouts qui se chevauchent avec des orientations OPPOSÉES sans trou parasite, déterminisme) ; `tests/unit/autodigitize/test_autodigitize.cpp` — géométrie EXACTE de la lettre en T (37 sommets, la même que l'audit ci-dessus), **rasterisée** et passée par la VRAIE chaîne segmentation → vectorisation → auto-satin (pas `build_satin_columns` en isolation) : reconstruction indépendante de la couverture (bandes satin des sections réussies + zones de repli tatami) et vérification géométrique que la région source moins cette couverture ne laisse aucun reliquat significatif ; suite complète (506 cas) sans régression, y compris les fixtures "réseau en T" et "anneau fin" qui auraient (à tort) déclenché un repli sans le garde-fou structurel.

Barreaux par défaut pour un satin manuel (`default_rungs`)

État : Présent · Testé numériquement. Correctif de revue.

Défaut trouvé par revue : un satin créé **manuellement** (deux rails seuls, `MainWindow::createSatinObject` via **Broderie ■ Colonne satin…**, cf. `rails_from_contour`) n'avait jamais de barreau, et retombait donc sur `fill_satin` — qui n'implémente qu'un **sous-ensemble** de `SatinConfig` (densité, compensation pull symétrique, sous-couche centrale). Les autres réglages Lot 3/4 (terminaisons, split, sous-couches de bord/zigzag, compensation push/pull asymétrique) restaient **silencieusement sans effet**, alors que l'inspecteur les expose et les laisse éditer sans aucune distinction ni avertissement pour ce type de satin : un utilisateur pouvait activer « Terminaison : Effilée » sur un satin manuel, voir le champ persister dans le `.osp`, sans jamais observer le moindre changement dans le résultat cousu.

Correction : `default_rungs(rail_a, rail_b, spacing)` (nouvelle fonction publique de `satin.hpp`) génère des barreaux par défaut en réutilisant **la même correspondance ladder** que `fill_satin` (aucun nouvel algorithme de correspondance, aucun risque de régression géométrique), à l'espacement `density`. `createSatinObject` les assigne à `SatinParams.rungs`, ce qui fait désormais passer **tout nouveau satin manuel** par `fill_satin_columns` — le seul chemin implémentant l'intégralité de `SatinConfig`. Portée volontairement restreinte à ce seul point d'entrée : un projet historique chargé depuis un `.osp` sans barreaux continue de suivre `fill_satin` sans changement de comportement (rétrocompatibilité), et `fill_satin` lui-même n'est pas modifié.

Vérifié : `default_rungs` produit au moins deux barreaux sur une colonne simple (vide sur des rails dégénérés) ; avec `cap_end = Tapered`, le même rail/config produit un bout à pleine largeur via `fill_satin` (réglage ignoré, ancien comportement) contre un bout réellement effilé via `fill_satin_columns` + `default_rungs` (réglage appliqué) — démontre que le défaut est bien corrigé, pas seulement que des barreaux existent.

Édition interactive de plusieurs guides

Sélectionner une colonne satin puis activer **Broderie ■ Éditer les guides satin…** (raccourci `Maj+E`) affiche chaque barreau et deux poignées. Une extrémité peut être glissée indépendamment : elle est projetée exactement sur son rail et la colonne est régénérée. Plusieurs guides successifs imposent donc plusieurs orientations locales ; `fill_satin_columns` interpole la correspondance de façon monotone entre chaque paire de guides.

Un geste est refusé s'il ferait franchir deux guides sur un seul rail ou les rapprocherait de moins d'un demi-pas de densité. Cette validation partage l'invariant du générateur, ce qui évite qu'un guide visible soit ensuite ignoré silencieusement. Chaque glisser produit une seule commande annulable **Déplacer un guide satin**. Les guides restent les `SatinParams.rungs` historiques : la persistance `.osp` demeure rétrocompatible.

Un clic sur le segment d'un guide le sélectionne et le met en évidence. La commande **Ajouter un guide satin** (`Maj+G`) partage automatiquement le plus grand intervalle disponible sur les deux rails ; elle est refusée si le nouveau guide serait trop proche de ses voisins. **Supprimer le guide satin sélectionné** conserve toujours au moins deux guides, seuil nécessaire à une colonne paramétrique. Ajout, suppression et déplacement utilisent trois commandes distinctes et sont entièrement annulables/rétablissables. L'insertion préserve aussi l'ordre de progression croissant ou décroissant des guides, afin que les extrémités employées par le routage textile restent exactes.

Le guide terminal qui porte une **jonction de réseau** est structurel : il reste sélectionnable et explicitement marqué comme verrouillé, mais ses poignées et l'action de suppression sont désactivées. Le générateur ne couvre que l'intervalle compris entre ses guides extrêmes ; déplacer ou supprimer ce guide pourrait donc raccourcir une section et créer un vide ou un repli à la jonction. Quand ce guide verrouillé est sélectionné, **Ajouter un guide satin** crée un guide interne dans l'intervalle directement adjacent de chacune des sections incidentes — jamais dans un grand vide éloigné de la jonction. L'opération est globale : si le réseau est incomplet, une section dupliquée ou un intervalle inadmissible, aucun guide n'est ajouté. Une seule commande undo/redo couvre toutes les sections. Chaque guide interne reste ensuite déplaçable section par section ; ce mécanisme amorce l'orientation de toutes les branches sans déplacer la jonction structurelle. Celle-ci est reconnue par l'ordre géométrique des stations projetées sur les deux rails, et non par son index de stockage, y compris après un import aux barreaux inversés ou désordonnés.

Chaque groupe de guides internes créés ensemble par un ajout coordonné à une jonction porte un `link_id` (`SatinRung.link_id`, optionnel) — un identifiant local au réseau (`source_vector`), alloué de façon déterministe et monotone par `next_satin_guide_link_id` (échoue explicitement si les identifiants uint32 du réseau sont épuisés plutôt que d'en réémettre un déjà utilisé). Un guide ajouté indépendamment n'a pas de `link_id` ; les projets historiques n'en ont jamais. Persisté dans le `.osp` (`rungs[].linkId`, validation stricte : entier `uint32` exact, aucune troncature silencieuse d'une valeur négative/flottante/hors bornes).

Invariant central : un groupe lié est identifié par la paire `(source_vector, link_id)` — jamais par le seul `link_id`, qui n'est pas globalement unique entre réseaux distincts. `satin_linked_guides` énumère les membres d'un groupe (un guide par section touchée par la jonction d'origine, jamais plus), rejette la totalité si une section porte deux guides du même `link_id` (donnée corrompue), si une section déclarée manque, si les membres ne touchent pas la même jonction ou si leur progression n'est pas strictement monotone sur les deux rails, et trie le résultat par `(section_index, embroidery_id, guide_index)` — mutation et énumération restent donc déterministes indépendamment de l'ordre des objets dans le document, et un même identifiant numérique réutilisé dans un autre réseau (`source_vector` différent) n'est jamais confondu avec le groupe.

Suppression de groupe atomique. Supprimer un guide lié (`Broderie ■ Supprimer le guide satin sélectionné`) supprime désormais l'identité logique entière : tous les guides du groupe, dans toutes leurs sections, en une seule commande **Supprimer des guides satin coordonnés** (`RemoveSatinGuidesCommand`) annulable/rétablissable comme un tout. Refusée en bloc — aucune section n'est touchée — si une section tomberait à moins de deux guides restants.

Déplacement de groupe atomique — geste explicite. Un glisser ordinaire d'une extrémité de guide (sans modificateur) reste **local** : il ne modifie que l'angle de cette section, comme avant (édition d'angle intrinsèquement locale — deux sections d'un même réseau peuvent légitimement avoir des largeurs et orientations différentes). Pour déplacer tout le groupe de façon cohérente, l'utilisateur maintient **Maj** en glissant une extrémité d'un guide lié : `move_satin_guide_group` calcule, pour la section glissée, sa position normalisée `t` dans l'intervalle `[station de la jonction, station du voisin suivant]` (sur chaque rail), en déduit un delta normalisé unique à partir du point relâché, puis applique **ce même delta** à `t` dans **chaque** section du groupe — recalculée entièrement à partir de la géométrie propre de cette section (ses propres rails, sa propre densité). Aucune coordonnée ni angle brut n'est jamais recopié d'une section à l'autre ; seul le delta normalisé traverse la frontière. Le résultat est appliqué par une seule commande **Déplacer des guides satin coordonnés** (réutilise `MoveSatinGuidesCommand`), tout ou rien : si une seule section sortirait de son intervalle admissible (moins d'un demi-pas de densité de sa marge, ou guide plus adjacent à sa jonction — topologie modifiée entretemps), aucune section n'est mutée et un message explique le refus. Un Maj+clic sans déplacement est un no-op exact et ne crée pas d'entrée dans l'historique. L'infobulle du guide et le message de statut du mode documentent ce geste.

Limite actuelle : le déplacement de groupe ne s'applique qu'aux guides internes directement adjacents à leur jonction (ceux créés par l'ajout coordonné) — un guide qui aurait perdu cette adjacence (un autre guide inséré entre lui et la jonction) redevient local et refuse le geste de groupe plutôt que de deviner un intervalle arbitraire. Le guide structurel de jonction lui-même reste volontairement verrouillé (ni déplaçable ni supprimable) ; le déverrouiller exigerait un calcul garantissant simultanément la couverture, la monotonie et l'absence de repli dans toutes les sections incidentes.

Finitions : points courts, split, terminaisons (Lot 3)

État : Présent · Testé numériquement · Validé visuellement (SVG). Toutes ces options sont **désactivées par défaut** (comportement inchangé) et éditables dans l'inspecteur (section satin) ; elles sont persistées dans le `.osp`.

- **Points courts** (`ShortStitchMode`) : dans un virage serré (le rail intérieur avance beaucoup moins que l'extérieur), les pénétrations intérieures sont *rentrées* vers l'axe (`SingleInset`, `MultiLevelInset` = motif triangulaire de profondeurs) pour ne pas s'entasser sur le bord, ou allégées (`RemoveAndRedistribute` : un fil serré sur deux, jamais un barreau, jamais deux d'affilée).
- **Split** (`SplitStitchMode`) : une traversée plus longue que `max_stitch_length` reçoit des pénétrations intermédiaires. `Simple` (colinéaire régulier), `Staggered` (décalage cyclique) et `DeterministicJitter` (variation **reproductible**, graine = identifiant de l'objet) évitent une **ligne centrale** visible. Jamais d'aléa non déterministe.
- **Terminaisons** (`SatinCapType`) : `Flat` (barreau final), `Rounded` (réduction arrondie de largeur), `Tapered` (effilé vers une pointe **sans** empiler sur une coordonnée unique — largeur minimale ~18 %), `Automatic`.

Vérifié : inset modifie le rail intérieur en virage, `RemoveAndRedistribute` réduit les pénétrations, split ajoute des points décalés (staggered ≠ ligne centrale ; jitter déterministe), taper réduit la largeur au bout sans l'annuler (SVG `tests/golden/auto-satin/lot3-*.svg` : effilé 5,00 → 0,91 mm).

Sous-couches et compensation (Lot 4)

État : Présent · Testé numériquement · Validé visuellement (SVG). Toutes désactivées par défaut, éditables (inspecteur) et persistées (`.osp`).

Modèle de passes (§4) : chaque commande de la séquence porte une `StitchPass` (`Underlay`, `TopStitch`, `Travel`, `Lock`, `Manual`) — non sérialisée (la séquence est régénérée), qui permet d'**identifier, filtrer et analyser** les passes séparément. Les sous-couches sont émises **avant** la couche supérieure.

- **Center walk** : point droit sur la médiane (retrait aux extrémités).
- **Edge walk** : deux chemins internes rentrés des rails (`underlay_edge_inset`).
- **Zigzag underlay** : satin léger (largeur réduite, pas plus grand).

- Ordre : **center** → **edge** → **zigzag** → **satin** (recommandé).

Compensation (§11), appliquée à la géométrie des paires avant génération :

- **Pull latérale asymétrique** : décalage par côté = base symétrique (`pull_compensation`) + fixe gauche/droite (`pull_left`/`pull_right`) + proportionnel à la largeur, borné (`pull_max`).
- **Push longitudinale** : `push_start`/`push_end` étendent (ou rétractent) la colonne aux extrémités le long de l'axe.

Correctif de revue — rétraction bornée. Contrairement à la compensation pull (bornée par `pull_max`), `push_start`/`push_end` n'avaient **aucune borne** : une rétraction excessive (`push` très négatif, p. ex. une valeur saisie par erreur en mm au lieu de µm) faisait passer le premier (ou dernier) fil de la colonne **de l'autre côté** de son voisin, inversant leur ordre le long de l'axe — un point auto-croisé visible en tout début/fin de colonne, la sous-couche `mids`/`cumMid` étant elle-même calculée **après** ce décalage (§ sous-couches ci-dessus, potentiellement corrompue en cascade). Corrigé : une rétraction ne dépasse désormais jamais la station voisine (borne dérivée, pas un seuil arbitraire — la même logique que l'exclusion des rails à l'ancrage des jonctions plus haut). Une extension (push positif) reste volontairement non bornée : elle éloigne le fil sans jamais croiser un autre point. Vérifié : une rétraction demandée de –50 mm sur une colonne de 1 mm de densité reste proche de son point d'origine (`tests/unit/stitch/test_satin.cpp`).

Vérifié : center/edge/zigzag = 4 passes distinctes ordonnées, pull élargit **un seul côté** (asymétrique), push étend le bout, déterminisme, aller-retour `.osp` (SVG `tests/golden/auto-satin/lot4-*.svg` : sous-couches en vert).

Défaut trouvé en usage réel (2026-08-13) — sous-couche centrale à la densité du zigzag principal au lieu de `underlay_spacing` : sur un satin **avec barreaux** (`fill_satin_columns`, auto-satin ou satin manuel avec barreaux par défaut — le chemin réellement emprunté par la quasi-totalité des satins de l'application), la sous-couche centrale reprenait simplement le milieu (`mids`) de **chaque fil** du zigzag principal, lui-même espacé à `density` (souvent 0,4 mm) — au lieu d'être ré-échantillonnée à `underlay_spacing` (2 mm par défaut). `underlay_spacing` existe et est correctement utilisé par `fill_satin` (satin manuel/legacy, sans barreaux) ; `fill_satin_columns` l'ignorait silencieusement en réutilisant `mids` tel quel. Conséquence concrète : des points de sous-couche de 0,3-0,4 mm au lieu de ~2 mm, systématiquement signalés `point-court` par l'analyse (risque de casse du fil et de sur-densité) — et un nombre de points de sous-couche inutilement élevé (5x plus que nécessaire à densité 0,4 mm). Corrigé en ré-échantillonnant `mids`/`cumMid` (déjà calculés pour `keepEnd`) à `underlay_spacing` via `point_at` (même primitive que le reste du fichier), sur l'intervalle `[retract, totalMid - retract]` — bornes identiques à l'ancien filtre par indice `keepEnd`. Vérifié : `tests/unit/stitch/test_satin.cpp` (colonne de 20 mm, densité 0,4 mm, `underlay_spacing` 2 mm → ~10 points de sous-couche, tous espacés de plus de 0,5 mm — pas ~50 points à 0,4 mm comme avant le correctif).

Reste (lots suivants) : tatami avancé (Lot 7). Voir `docs/stitch-feature-gap-audit.md`.

Entrée/sortie et points de fixation (Lot 5)

État : Présent · Testé numériquement · Validé visuellement (SVG). Éditables (inspecteur) et persistés (`.osp`).

Points d'entrée/sortie (§12) : `entry_point` et `exit_point` (optionnels) orientent la couture. Le satin est **retourné** si son extrémité de départ est plus proche du point de sortie que du point d'entrée (coût = somme des distances extrémités↔points) — jamais de réordonnancement partiel, seulement le sens. Sans point, l'orientation issue du générateur est conservée.

Points de fixation (`SatinLock`, émis en passe `Lock` uniquement) : ancrent le fil au **début** et à la **fin** de l'objet, jamais par sous-passe (un lock au départ, un à l'arrivée). Chaque bout se règle indépendamment (`lock_start`/`lock_end`), avec longueur (`lock_length`) et nombre de passages (`lock_passes`) communs.

- **`None`** : aucun point de fixation.
- **`BackAndForth`** : aller-retour court le long du premier/dernier point.
- **`Triangle`** : petit triangle d'ancrage.

- **`MicroZigzag`** : micro-zigzag progressant dans l'axe de couture.

Toutes les formes sont **bornées** (`lock_length`, défaut 0,8 mm) et ancrées à l'extrémité exacte du satin (continuité : pas de saut parasite grâce à l'enchaînement de passes à position identique).

Correctif de revue — borne réelle, pas seulement nominale. `lock_length` n'était en réalité **jamais comparée** à la distance disponible : `toward` (le point voisin qui donne la direction du lock) est le point du rail **opposé** — donc la largeur locale du barreau, pas un point lointain le long de la couture. Sur une colonne fine (largeur < `lock_length`, cas courant pour du texte fin ou un petit motif), le lock piquait **hors de la matière déjà cousue**, dans du tissu vierge, sans qu'aucune borne ne l'en empêche. Corrigé : la longueur effective du lock ne dépasse désormais jamais la distance réelle vers `toward` (repli sur la longueur demandée uniquement dans le cas dégénéré où `toward == anchor`, sans référence géométrique fiable). Vérifié : sur une colonne de 0,3 mm de large avec `lock_length` par défaut (0,8 mm), tous les points du lock restent dans la largeur réelle, quel que soit le type (`tests/unit/stitch/test_satin.cpp`).

Vérifié : `None` → vide ; les autres progressent dans l'axe de couture et restent bornés ; le point d'entrée fait démarrer la couture à la bonne extrémité ; locks en passe `Lock`, exactement deux groupes (début/fin) ; aller-retour `.osp` des champs Lot 5 (SVG `tests/golden/auto-satin/lot5-lock-*.svg` : fixations en rouge).

Routage multi-colonnes (Lot 6)

État : Présent · Testé numériquement · Validé visuellement (SVG). Automatique à la génération ; aucun réglage persisté.

Une forme décomposée (Y, T, croix…) devient **plusieurs colonnes satin** de même couleur et même source. Sans routage, elles seraient cousues dans l'ordre du document, avec de longs sauts. §13 les ordonne et les oriente ensemble (`route_columns`) :

- **Ordre** : une jonction explicite commune et géométriquement admissible prime sur une proximité fortuite ; à égalité, glouton plus proche voisin depuis la position courante de l'aiguille, puis amélioration **2-opt**. Déterministe.
- **Orientation** : pour un ordre donné, le sens de chaque colonne est résolu **exactement** par programmation dynamique sur ses deux extrémités : maximum de liaisons par jonction, puis distance minimale (l'orientation choisie est imposée à `generate_satin` via entrée/sortie).
- **Liaisons** : une transition **justifiée par une jonction commune validée** tolère jusqu'à `underpath_max` (défaut 8 mm) en **trajet caché** (passe `Travel`, running stitch — pas de coupe) ; **sans** jonction, seul un quasi-contact (`underpath_max_without_junction`, défaut 1,5 mm — bien plus strict) l'autorise. Au-delà de la borne applicable, la liaison reste un **saut**. Minimise les coupes et les déplacements à découvert.

Une jonction déclarée mais séparée de plus de `underpath_max` est traitée comme incohérente et n'influence ni l'ordre ni le type de liaison. Cette garde évite qu'un `.osp` altéré force un trajet caché arbitraire. Limite actuelle : le trajet entre deux sections reste un segment échantillonné ; il ne suit pas encore le centre d'une branche déjà cousue lors d'un retour vers une jonction.

Correctif « auto-satin béton » (suite) : `route_columns` ne regardait la jonction que pour l'ORDRE et l'ORIENTATION ; le TYPE de liaison (trajet caché ou saut) ne dépendait que de la distance, sans jamais vérifier qu'elle était justifiée par cette même jonction. Deux colonnes **sans aucun lien topologique** mais distantes de 5 à 8 mm — deux lettres rapprochées, une forme en C dont les deux bouts se frôlent sans être reliés — étaient donc cousues en trajet caché à travers un espace dont rien ne garantissait qu'il était couvert de tissu. Corrigé : le type de liaison distingue désormais explicitement les deux cas (jonction validée vs proximité seule), chacun avec sa propre borne. Une jonction reste géométriquement anchrée exactement (§ ancrage des jonctions ci-dessus), donc digne de confiance jusqu'à 8 mm ; une simple proximité, sans cette garantie, ne l'est que jusqu'à 1,5 mm.

Le groupe routé est **contigu** et limité aux colonnes auto (porteuses de barreaux) de couleur et source identiques : l'ordre inter-groupes et le reste du document sont préservés. Un satin manuel isolé n'est pas réordonné.

Correctif de revue — le point de décision doit être le point réellement cousu. `column_endpoints` (extrémités représentatives d'une colonne pour le routage) utilisait le milieu **brut** des barreaux d'about, alors que la génération réelle (`fill_satin_columns`) peut **décaler** ces mêmes extrémités via `push_start`/`push_end` (§ sous-couches et compensation, Lot 4) avant de coudre. La décision Underpath/saut se fondait donc sur un point différent de celui effectivement cousu : un écart évalué juste sous le seuil pouvait, une fois le décalage réel appliqué, le dépasser (trajet caché accordé à travers un espace non garanti couvert — précisément le défaut visé par le correctif « auto-satin béton (suite) » ci-dessus, réintroduit par un chemin différent), ou inversement imposer un saut évitable. Corrigé : `column_endpoints` reproduit désormais le même décalage (et la même borne de rétraction, § Lot 4) que celui réellement appliqué à la génération. Vérifié : deux colonnes dont l'écart **brut** entre barreaux (900 µm, sous le seuil sans jonction de 1,5 mm) aurait autorisé un trajet caché, mais dont l'écart **réel** une fois `push_end` (rétraction de 700 µm) appliqué dépasse ce seuil (1,6 mm), produisent désormais un saut — pas un trajet caché injustifié (`tests/unit/stitch/test_routing.cpp`).

Vérifié : liste vide → plan vide ; une colonne → liaison de départ, aucun saut ; réordonnancement minimisant le déplacement ; orientation par l'extrémité proche ; liaison longue → saut, liaison courte → trajet caché ; **liaison proche (5 mm) SANS jonction commune → désormais un saut** (défaut corrigé) ; la même liaison justifiée par une jonction reste un trajet caché ; un quasi-contact (1 mm) sans jonction reste toléré ; à la génération, un groupe adjacent n'émet qu'un saut initial (le reste cousu), un groupe éloigné **ou sans jonction commune** conserve ses sauts (SVG `tests/golden/auto-satin/lot6-route-*.svg` : trajets cachés en bleu, sauts en rouge pointillé).

Limite : le trajet caché est un segment **direct** (échantillonné en points cousus) ; il n'est garanti *sous* la broderie que pour des colonnes reliées par une jonction (ou en quasi-contact). Un routage suivant réellement la matière viendra avec le tatami avancé (Lot 7).

Satin Coverage Analyzer : mesurer objectivement la surface couverte (2026-08-13)

État : Présent (`satin_coverage::analyze_satin_coverage`) · Testé numériquement (7 cas géométriques à résultat connu) · Observateur indépendant, n'influence encore aucune décision du générateur.

Origine

L'audit des ruptures de ruban (§ *Saut plutôt qu'un point continu...* et *Seuil angle/distance...* ci-dessus) et le repli tatami sur branche rejetée (§ précédente) partagent un point aveugle commun : aucun des deux ne répond, de façon indépendante et vérifiable, à la question « les colonnes générées couvrent-elles réellement l'intégralité de la forme d'origine ? ». Les validations existantes portent sur la cohérence des rails, du squelette, des largeurs ou des jonctions — jamais sur une comparaison de SURFACE entre la région source et ce que les colonnes balaient effectivement. Un squelette valide, des sections toutes construites sans refus, et pourtant une zone entière peut rester sans le moindre point : rien dans les validations structurelles précédentes ne le détecte directement.

Principe : comparer deux surfaces, jamais une heuristique de squelette

`satin_coverage::analyze_satin_coverage(target, columns, config)` (`libs/satin_coverage/`, nouvelle bibliothèque, dépend uniquement de `core`, `geometry`, `stitch_generation` — jamais de `auto_satin` ni `document`, pour rester un **observateur totalement indépendant du générateur**, utilisable depuis un test synthétique aussi bien que depuis `autodigitize`) calcule :

1. pour chaque colonne, la suite ordonnée de stations (L■, R■) via `stitch_generation::satin_stations` (nouvelle fonction publique, extraite sans changement de comportement de la correspondance ladder déjà utilisée par `fill_satin_columns` — cf. *Correction de l'appariement* plus haut) : la géométrie STRUCTURELLE des rails/barreaux, jamais les points de couture finaux (compensation pull, split-stitch, terminaisons), qui répondent à un besoin de couture et non à la question « les rails couvrent-ils la forme d'origine ? ». Une rupture de ruban (`SatinStation::jump_before`, § audit lettres/seuil angle-distance ci-dessus) coupe la colonne : aucun quadrilatère de couverture ne relie deux stations de part et d'autre d'un saut — le même mécanisme qui

empêche un point cousu disproportionné empêche aussi un quadrilatère de couverture artificiel ;

2. chaque intervalle entre deux stations consécutives devient un quadrilatère (L■, R■, R■■■, L■■■), décomposé en DEUX TRIANGLES (diagonale L■-R■■■) — robuste à un quadrilatère non convexe, et surtout : un garde-fou défensif (`segments_cross_strict` sur L■-R■ vs L■■■-R■■■) exclut et COMPTE tout intervalle où les deux rails se croiseraient, plutôt que de laisser un quadrilatère dégénéré s'ajouter silencieusement à l'union — l'analyseur ne fait jamais confiance aveuglément à l'invariant anti-croisement de l'appariement amont, même si celui-ci est déjà testé séparément ;

3. tous les triangles de toutes les colonnes sont réunis par une union booléenne NonZero (`geometry::union_nonzero`) → `C`. Chaque triangle est explicitement réorienté CCW avant l'union : `union_nonzero`, contrairement à `intersect_polygons`/`difference_polygons` (ajoutés par ce lot, voir plus bas), ne renormalise pas l'orientation d'entrée — deux triangles chevauchants d'orientations opposées s'annuleraient localement sous NonZero, créant un trou parasite qui masquerait précisément le genre de défaut que cet outil doit révéler (piège identifié dès la conception, pas trouvé après coup) ;

4. `C` est comparé à la région cible `P` (`geometry::PathSet`, trous compris) par intersection/différence booléennes NonZero, trous compris DES DEUX CÔTÉS (`geometry::intersect_polygons`/`difference_polygons`, nouvelles primitives de `libs/geometry` — contrairement à `subtract_polygons` existant, dont les `cutouts` sont de simples contours sans trou, ces deux nouvelles fonctions gèrent le cas d'une poche non couverte elle-même entourée de couverture de tous côtés, c'est-à-dire un TROU dans `C`) : `covered = aire(P ∩ C)`, `missing = aire(P \ C)`, `outside = aire(C \ P)`.

Composantes manquantes et profondeur des trous

Chaque composante connexe de `missing` est déjà un élément distinct du `vector<PathSet>` renvoyé par `difference_polygons` (issu directement du `PolyTree64` de Clipper2) : aucun algorithme de composantes connexes séparé n'est nécessaire. Pour chacune (`MissingRegion`) : aire, part de la cible, centroïde/bbox (barycentre des sommets), et un `max_gap_radius_mm` — le rayon du plus grand disque inscrit, obtenu par **recherche binaire d'érosions successives** (`geometry::inset_path_set`, déjà présent, déjà testé) jusqu'à disparition de la région, plutôt qu'un raster + transformée de distance OpenCV : précision continue, aucune dépendance nouvelle, et cette bibliothèque reste sans dépendance OpenCV. Distingue une frange fine de plusieurs mm² le long d'un bord (rayon petit, peu grave) d'une poche ronde de même aire en plein milieu (rayon significatif, vrai défaut).

Une **couverture du cœur** (`core_coverage_ratio`) complète la couverture brute : `P` est érodé de `boundary_tolerance_mm` (0,1 mm par défaut) avant de recalculer le manquant — absorbe les approximations de discrétisation le long du bord sans masquer une vraie poche interne. Sur une forme trop fine pour survivre à cette érosion, repli documenté sur la couverture brute plutôt qu'une division par zéro.

Rapport et seuils explicites

`SatinCoverageReport` (aires cible/couverte/manquante/hors-forme, ratios brut et cœur, régions manquantes triées par aire décroissante, rayon de trou maximal, `passed`, message texte diagnostic prêt à logger — jamais un simple « couverture insuffisante », toujours les chiffres et la raison précise du refus) et `SatinCoverageConfig` (tous les seuils : couverture brute/cœur minimales, aire/rayon de trou maximaux, ratio hors-forme maximal, tolérance de bord, résolution de la recherche de rayon — valeurs par défaut raisonnables, explicitement PAS des constantes universelles du métier, ajustables sans toucher au code de l'analyseur).

Vérifié

Sept cas géométriques à résultat connu (`tests/unit/satin_coverage/test_coverage.cpp`, 43 assertions) : rectangle entièrement couvert (~100 %, PASS) ; rectangle couvert à moitié (~50 %, FAIL, une seule région manquante de l'aire attendue) ; trou de la forme source (anneau tuilé par quatre colonnes) jamais compté comme manquant ; vraie poche non couverte entourée de matière (FAIL, rayon de trou mesuré conforme au rectangle de la poche à 0,05 mm près) ; débordement hors cible (`outside_area` > 0, cible elle-même toujours entièrement couverte) ; deux colonnes qui se chevauchent (l'union fait foi, aucun double comptage) ; et surtout **`trident`** (jonction concave asymétrique à 3 branches inégales, forme historiquement problématique pour Auto-Satin, cf. audits ci-dessus) : couverture complète (toutes branches) 88,1 %, plus grande zone manquante 18,0 mm² (le noyau de jonction résiduel déjà documenté § *Optimisation globale des bridges...* — un résidu CONNU, pas un défaut de cet audit) ; en retirant la branche PRINCIPALE (simulant un rejet amont), couverture 9,4 %, plus grande zone manquante 157,8 mm² — démonstration que l'analyseur distingue nettement « toutes les branches présentes » de « une branche absente », par une mesure de surface et non un décompte de branches/stations traitées. Suite complète (514 cas ctest) sans régression, Debug et Release, y compris la suite `stitch`/`auto_satin`/`autodigitize` complète après l'extraction de `satin_stations` (comportement bit-à-bit identique à l'ancien `fill_satin_columns`).

Portée actuelle et limites assumées : observateur pur, n'influence encore aucune décision de génération. Pas de second mode « couverture des points finaux » (§16 de la demande d'origine). `degenerate_interval_count` reste à 0 sur toutes les fixtures testées à ce jour (l'invariant anti-croisement amont tient) ; le garde-fou existe pour le jour où ce ne sera plus le cas plutôt que pour un défaut déjà observé.

Non-régression sur le corpus de formes historiques

État : Présent (`tests/unit/auto_satin/test_coverage_regression.cpp`).

Photographie la couverture géométrique actuelle de dix formes du corpus `auto_satin::shapes` (`rectangle`, `capsule`, `ribbon`, `s`, `y`, `t`, `cross`, `h`, `ring`, `trident` — `circle`/`tiny`/`notch`/`pinch` exclus, testés ailleurs pour leur comportement de refus, pas pour leur couverture) : un plancher de couverture brute par forme, mesuré en calibrant ce test puis abaissé de quelques points de marge — un vrai plancher de non-régression, **pas** un objectif de qualité (les résidus de noyau de jonction sur `y`/`t`/`cross`/`h`/`trident`, § *Optimisation globale des bridges...* et § *Séparation StableBranchEnd/JunctionSeparator...* plus haut, sont un fait déjà documenté, pas un défaut que ce test cherche à faire disparaître). Sans surprise, `rectangle`/`ring` sont quasi parfaits (96 %/99,99 %) ; les formes à jonction se situent entre 85,8 % (`cross`) et 88,7 % (`t`).

`wide` (rectangle délibéré 100 × 20 mm, `shapes.cpp` : « un vrai cas trop large pour satin ») refuse entièrement dès l'analyse de satinabilité (`Unsuitable`, `has_wide_area` : sa largeur de 20 mm dépasse `max_satin_width`, 9 mm par défaut, `satinability.cpp:76`) — **comportement voulu par construction**, pas un défaut. Exclue du corpus de ce test pour la même raison que `circle`/`tiny`/`notch`/`pinch`.

Diagnostic « wide » (2026-08-13) : confirmation, pas de défaut trouvé

Investigation demandée explicitement pour vérifier l'hypothèse ci-dessus. Trace complète : `evaluate_satinability` (`satinability.cpp:76`) calcule `maxWidthUm` à partir de la transformée de distance du raster de la région (le rayon maximal du disque inscrit, doublé) ; pour `wide` (100 × 20 mm), cette largeur maximale est bien ~20 mm, très au-delà de `max_satin_width` (9 mm, `SatinColumnsParameters::analysis.thresholds`) — `has_wide_area` passe vrai, `status` devient `Unsuitable`, message « Zone trop large pour un satin (envisagez un tatami). », et `build_satin_columns` s'arrête avant toute construction de rail (`satin_column.cpp:2977`, refus immédiat sur statut `Unsuitable`). Aucune trace d'un défaut de squelette, de section transversale ou de jonction : le refus intervient AVANT que le pipeline géométrique ne s'exécute, sur un seuil de configuration simple et correctement appliqué. **Conclusion : `wide` fonctionne exactement comme conçu** (rediriger une forme trop large vers un remplissage tatami, cf. § *Colonne trop large* plus haut) ; la note précédente (« surprenant, piste non investiguée ») provenait d'une lecture incomplète de `shapes.cpp` avant d'avoir vérifié le seuil réellement en cause — corrigée ici plutôt que laissée en l'état.

Diagnostic « trident »/pixel_size (2026-08-13) : pas de perte de couverture réelle, fragilité Legacy documentée

Investigation demandée explicitement, en plusieurs passes (deux hypothèses de correctif tentées et invalidées par la mesure avant la conclusion ci-dessous — aucune des deux n'a été committée).

Symptôme observé (`openstitch-cli auto-satin-debug --shape trident`, mode Legacy, `--satin-geometry=legacy`) : à `pixel_size = 0,05 mm` (défaut CLI), la branche latérale de `trident` échoue (« colonne refusée : saut de largeur incohérent entre stations finales #1/2 »), ce qui fait échouer la validation de jonction (« jonction 1 incomplète, 2/3 branches disponibles ») et refuse **toute la région**. À `pixel_size = 0,1 mm` (convention des fixtures de test), la même forme réussit (88,1 % de couverture).

Deux hypothèses testées, toutes deux invalidées par instrumentation directe (dump des largeurs de station réelles, `OS_DEBUG_STATIONS`, retiré avant commit) :

1. *Pointe effilée mal exemptée près du bout OUVERT.* Correctif tenté : n'appliquer le contrôle de saut de largeur qu'au tiers médian de l'axe (mêmes bornes que `representative_station_width`). **Invalidé** : la branche fautive ne compte que 5 stations après amputation de jonction, un tiers médian trop étroit pour exempter l'intervalle fautif — aucun changement observé après correctif.
2. *Amputation de jonction qui s'arrête trop tôt faute de palier stable.* Correctif tenté : étendre `trim_unstable_junction_tail` pour continuer d'amputer tant que la frontière reste incohérente avec sa voisine immédiate (même seuil que la validation séparée). **Invalidé** : l'intervalle fautif (stations #1/#2, largeurs 881 µm puis 4393 µm) se situe près du bout **OUVERT**, pas du bout de jonction — `trim_unstable_ junction_tail` n'amputant QUE côté jonction par construction, aucune amputation supplémentaire ne pouvait l'atteindre. Widths réelles observées sur cette branche : 807,5 → 881,1 → **4393,0** → 7606,2 → 9423,7 µm — un saut brutal et isolé juste après la pointe, puis une remontée globalement cohérente vers la confluence.

Cause probable (non creusée plus loin, cf. portée ci-dessous) : la station à 4393 µm correspond à la station d'axe #23, signalée ailleurs comme « interpolée » (échec isolé de `cross_section` comblé par interpolation linéaire, § *Génération partielle sur formes concaves*). Sur cette branche courte et proche d'une confluence très asymétrique, la section transversale calculée depuis la seule tangente locale peut basculer brutalement de « mesure de cette branche seule » à « balayage de la branche voisine bien plus large », dès la station suivant la pointe — un phénomène de bord plausible pour une géométrie aussi contrastée, mais dont le mécanisme exact dans `cross_section` n'a pas été tracé en détail.

Conclusion — pas de correctif nécessaire, car pas de perte de couverture réelle. Relecture de `libs/autodigitize/src/autodigitize.cpp` :

1. `autodigitize.cpp` utilise le mode **`Parametric`** par défaut (`geometry_mode = SatinGeometryMode::Parametric`, ligne 189), jamais `Legacy` — alors que le CLI `auto-satin-debug` testé ci-dessus utilise `Legacy` par défaut (`--satin-geometry=legacy`). Le symptôme observé n'a donc jamais été observé dans le chemin réellement emprunté par la numérisation automatique.
2. Même le mécanisme de refus en cascade (« jonction incohérente ») qui transforme un échec de branche isolé en refus de LA RÉGION ENTIÈRE est spécifique à `Legacy` : le mode `Parametric` n'a « aucune ancre centrale commune, aucun sommet de bissectrice... » (§ *Jonctions : recouvrement local, sans ancre* plus haut) — un échec de branche y resterait individuel, sans faire échouer les deux autres.
3. Et si malgré tout `build_satin_columns` refusait la totalité (`columns` ET `parametric_columns` vides), `autodigitize.cpp` (ligne 300-315) retombe automatiquement sur un remplissage **tatami de la région entière** dès que `sectionCount == 0` — aucune zone sans point, juste un changement de type de point (satin → tatami).

Trois filets indépendants (mode par défaut différent de celui testé, refus non cascadé en mode Parametric, repli tatami région entière en dernier recours) protègent donc la couverture livrée à l'utilisateur, même dans le pire des cas où cette fragilité de construction de colonne Legacy se manifeste. **Rien à corriger côté couverture** ; la fragilité elle-même (refus d'une branche courte proche d'une confluence très asymétrique, à certaines résolutions de raster, en mode Legacy) reste une piste d'amélioration future légitime de robustesse — pas urgente, et hors du périmètre de cet audit de couverture.

Balayage du corpus de formes (mode × pixel_size) : deux défauts réels trouvés et corrigés (2026-08-13)

État : Présent, corrigé. `geometry::inset_path_set` (`libs/geometry/src/offset.cpp`) et le câblage `--coverage-svg` du CLI (`apps/cli/main.cpp`).

Utilisation directe de l'analyseur pour balayer les 15 formes du corpus `auto_satin::shapes`, en Legacy ET Parametric, à `pixel_size` 0,05 et 0,1 mm (60 exécutions, `openstitch-cli auto-satin-debug --coverage-svg`) — pas encore automatisé en test, exploration manuelle guidée par les chiffres bruts. Deux défauts RÉELS trouvés, tous deux corrigés séance tenante, vérifiés sans régression (517 cas ctest, Debug+Release) :

1. **`geometry::inset_path_set` ne réoriente pas les trous avant Clipper2** (contrairement à `boolean.cpp`, qui le fait déjà pour `subtract_polygons` et pour `intersect_polygons`/`difference_polygons` ajoutés par ce lot). `InflatePaths` offsette chaque contour par rapport à SA PROPRE orientation : un trou de même orientation que son extérieur (au lieu de l'opposée) grandit ou rétrécit à l'envers lors d'une érosion. Repéré sur `ring` (`shapes.cpp`, dont le trou et l'extérieur sont générés par le même parcours d'angle croissant de `circle()`) : couverture brute correcte (99,99 %, les opérations d'union/différence QUE cet audit a ajoutées se réorientent déjà correctement) mais **couverture du cœur mesurée à 71,18 %** au lieu d'un ~100 % attendu — rien d'autre dans le pipeline Auto-Satin n'étant sensible à l'orientation d'un trou, ce défaut latent était invisible avant que `core_coverage_ratio` (qui appelle `inset_path_set` sur la région cible) ne l'expose. Corrigé par le même motif `oriented_as` que `boolean.cpp` (dupliqué localement, ADR-005) ; `shapes.cpp` corrigé en complément (trou de `ring` réorienté à la construction) pour que la fixture soit correcte par construction, pas seulement tolérée par une réorientation défensive en aval. Nouveau test dédié (`tests/unit/geometry/test_offset.cpp`, anneau carré dont le trou partage délibérément l'orientation de l'extérieur) vérifiant l'aire nette après érosion. Après correctif : `ring` couverture cœur 100,00 %, PASSED.
2. **Le câblage `--coverage-svg` du CLI lisait `parametric_columns` sur le seul indicateur `--satin-geometry` demandé**, pas sur la liste RÉELLEMENT peuplée par `build_satin_columns` — alors que certaines formes (anneaux, cas refusés en Parametric) retombent automatiquement sur `columns` (Legacy) À L'INTÉRIEUR de `build_satin_columns` même quand Parametric est demandé (§ *Objets satin paramétriques* plus haut). Symptôme : `ring --satin-geometry=parametric` rapportait 0 colonne, 0 % de couverture, alors que des colonnes existaient bel et bien (côté `columns`, jamais lu). Corrigé en testant `!parametric_columns.empty()` plutôt que l'indicateur demandé — même motif que `autodigitize.cpp` (`useParametric`, déjà correct). Bug de câblage CLI, jamais de défaut Auto-Satin lui-même ; le bloc pré-existant de génération du SVG `--output-svg` (`parametric_to_svg`) partage la même faiblesse mais n'a pas été touché ici (hors périmètre de cet audit, à corriger séparément).

Piste non investiguée, potentiellement plus significative que les deux ci-dessus : `notch` (bande à encoche en V profonde) montre un écart systématique et notable entre modes — couverture brute **~86-87 % en Legacy contre ~77-78 % en Parametric**, alors que Parametric est le mode utilisé par défaut dans `autodigitize.cpp` (contrairement au cas `trident` ci-dessus, aucun filet ne compense ici puisque Parametric produit bien une colonne, juste avec une couverture dégradée). Zone manquante la plus grande mesurée jusqu'à 39,4 mm² (22 % de la cible) selon la résolution de raster. Hypothèse de départ, non vérifiée : l'ajustement Bézier à paires structurantes éparses (`select_structural_indices`/`fit_both_rails`) capturerait moins fidèlement une encoche concave profonde que l'échantillonnage dense de `Legacy`. À investiguer en profondeur dans une prochaine session (tracer `fit_both_rails`/l'erreur d'ajustement autour de l'encoche, comme fait ici pour `trident`/pixel_size) avant toute tentative de correctif.

Diagnostic sur image réelle (`tentabrode.png`) : le repli tatami ne protège pas toutes les régions (2026-08-13)

État : Présent, diagnostic seul (aucun correctif) — test permanent `TEST_CASE("DIAGNOSTIC TEMPORAIRE couverture satin (tentabrode)")` (`tests/integration/test_pipeline.cpp`), toujours en échec (`CHECK(false)`), même convention que les diagnostics GISTRE existants.

Applique l'analyseur à la sortie réelle d'`autodigitize::auto_digitize` sur `tests/fixtures/tentabrode.png` (pas une forme synthétique du corpus), en regroupant par `source_region` (`document::VectorObject::source_region`) pour combiner chaque vecteur satin avec son éventuel repli tatami — le filet décrit plus haut (§ *Repli tatami sur une branche auto-satin rejetée*) ne se déclenche QUE sur le signal structurel `network.debug.graph.edges.size() > sectionCount` (branche entière rejetée), jamais sur une couverture partielle d'une branche par ailleurs acceptée.

Sur les 14 régions satin-portantes de cette image :

- **Couverture satin seule (sans repli), agrégée sur toute l'image : 21,3 %.** Ce chiffre seul est trompeur — il ignore le filet tatami — mais confirme que l'écart entre colonnes générées et surface cible n'est pas anecdotique sur une image réelle.
- **Couverture réelle (satin + repli tatami combiné par région), agrégée : 97,3 %.** Le filet comble donc l'essentiel de l'écart, comme conçu.
- **Mais 9 des 14 régions restent sous le seuil de 99,5 % même après combinaison**, dont deux sévèrement et **sans aucun repli** (le signal structurel qui déclenche le filet n'a pas été franchi) : un vecteur à **46,6 %** et un autre à **59,3 %** de couverture réelle.

Ce résultat corrobore et QUANTIFIE, sur une image réelle, la même catégorie de défaut que la piste `notch`/Parametric non investiguée ci-dessus (écart de couverture sur une branche acceptée, pas un rejet structurel) : le filet tatami actuel protège contre le cas « branche entièrement refusée » mais pas contre le cas « branche acceptée avec une géométrie de colonnes qui ne couvre pas fidèlement sa région », qui semble être la situation la plus courante en pratique sur des formes réelles bruitées. C'est le constat central qui motive la refonte d'architecture décrite ci-dessous (§ *Décomposition guidée par squelette*) : plutôt que d'élargir encore le filet de repli, traiter la décomposition d'une région branchée en sous-régions satinables comme une étape de planification à part entière, en amont du générateur actuel.

Suite (2026-08-14) : ce constat a depuis motivé le plan SGSD en entier (§ ci-dessous, 9 phases livrées) puis son branchement en production dans `autodigitize.cpp` (§ *Intégration production*, tout en bas de ce chapitre). Ce même diagnostic, réexécuté après le branchement, mesure une couverture réelle agrégée de **98,9 %** (contre 97,3 % avant) — un gain net, mais plus modeste et plus mitigé région par région que sur le corpus synthétique propre (§ *Outil CLI `sgsd-debug`*, gains de +4,6 à +7,4 points) : sur cette image réelle bruitée, SGSD s'appuie plus souvent sur le filet tatami plutôt que sur une couverture satin propre, plutôt que d'améliorer la couverture satin elle-même comme sur les formes synthétiques. Détails dans la section d'intégration.

Visualisation de debug (`coverage_to_svg`)

État : Présent · Câblé dans `openstitch-cli auto-satin-debug --coverage-svg`.

`satin_coverage::coverage_to_svg(target, columns, report)` superpose, mêmes conventions que `auto_satin::debug_export` (millimètres, Y inversé) : la couverture en vert translucide, les zones manquantes en rouge (une par composante connexe, `fill-rule="evenodd"` pour respecter leurs propres trous), le débordement en orange, le contour de la cible en gris, et pour chaque colonne ses deux rails (bleu/orange) et ses stations structurelles (points — rouges aux stations atteintes par un saut, cf. `SatinStation::jump_before`). Un commentaire d'en-tête résume le rapport en texte (aires, ratios, nombre de régions manquantes), lisible sans rendu graphique, plus une ligne par région manquante (aire, part de la cible, rayon de trou, centroïde).

`openstitch-cli auto-satin-debug --shape <forme> --coverage-svg <fichier>` calcule la couverture des colonnes réellement construites (Legacy ou Parametric selon `--satin-geometry`) et écrit à la fois le diagnostic texte sur la sortie standard et ce SVG. Vérifié en conditions réelles sur `trident` : à la résolution de raster par défaut (0,05 mm/px), le mode Legacy REFUSE entièrement la forme (« jonction 1 incomplète, 2/3 branches disponibles ») — le SVG rend alors 100 % de la cible en rouge, cohérent avec zéro colonne produite ; à 0,1 mm/px, 88,14 % de couverture avec quatre composantes manquantes (la plus grande, 18,05 mm², est le noyau de jonction déjà documenté). Cette sensibilité de `trident` à la résolution de raster en mode Legacy n'était pas documentée avant cet outil ; investiguée en profondeur ci-dessous (§ *Diagnostic « trident »/pixel_size*) — conclusion : sans impact réel sur la couverture livrée à l'utilisateur,

grâce aux filets déjà en place en amont de ce chemin CLI brut.

Testé (`tests/unit/satin_coverage/test_coverage.cpp`) : SVG bien formé (`<svg>…</svg>`), couleurs de remplissage vert/rouge et rails bleu/orange effectivement présents pour un cas de couverture partielle connu.

Décomposition guidée par squelette (SGSD) : planification globale au-dessus de l'Auto-Satin (2026-08-13)

Motivation : une forme = une colonne est une hypothèse trop restrictive

État : En cours — phase 1 sur 9 livrée (planification topologique pure, aucune génération de géométrie).

`build_satin_columns` (§ *Colonnes automatiques par squelette* plus haut) traite déjà une région branchée en construisant une colonne par arête du squelette, mais TOUJOURS par rapport au contour ORIGINAL non découpé : les sections transversales de chaque branche sont mesurées contre la même `geometry::PathSet` entière, jamais contre une sous-région dédiée. Les jonctions sont donc résolues a posteriori par des heuristiques locales (`trim_unstable_junction_tail`, `StableBranchEnd`/`JunctionSeparator` en Legacy, recouvrement borné en Parametric) qui amputent ou recouvrent, mais ne peuvent jamais réellement replanifier la forme. Le diagnostic `tentabrode.png` ci-dessus montre l'impact réel de cette limite : des régions acceptées (pas rejetées) dont la couverture reste significativement incomplète, sans qu'aucun filet ne s'en aperçoive.

L'objectif de la SGSD (*Skeleton-Guided Satin Decomposition*) est d'introduire un étage de PLANIFICATION avant l'Auto-Satin existant, qui devient un solveur LOCAL : découper une région branchée en un petit nombre de sous-régions simples (chacune proche d'un chemin squelette `endpoint – endpoint`, sans jonction interne), construire une colonne satin par sous-région avec le générateur actuel INCHANGÉ, mesurer la couverture de chaque sous-région et de l'assemblage avec `satin_coverage` (déjà générateur-agnostique, § plus haut), puis fusionner/router. L'Auto-Satin existant n'est ni supprimé ni modifié : il est réutilisé tel quel comme générateur d'une région simple, et comme oracle de qualité pour comparer des découpages candidats.

Déploiement en 9 phases, chacune testable et commitée séparément, sans jamais toucher au générateur de rails existant avant que la décomposition elle-même soit validée :

1. **Graphe de squelette + appariement de branches (diagnostic seul).**
2. **Décomposition du graphe en `SatinPath`** (chemins topologiques maximaux) — livrée avec la phase 1, § *Phase 1* ci-dessous.
3. **Génération de candidats de coupe + découpage réel du polygone en `SatinRegion`.**
4. **`SatinabilityAnalyzer`** (la région candidate est-elle satinable ?) — livrée, § *Phase 4* ci-dessous.
5. **Auto-Satin par région + Coverage Analyzer comme oracle de sélection** — livrée, § *Phase 5* ci-dessous.
6. **Recherche à faisceau (*beam search*) sur les découpages candidats** — livrée (portée réduite à une décision de coupe à la fois, § *Phase 6* ci-dessous), aucune recherche combinatoire multi-jonctions.
7. **Passe de fusion (*merge*) post-découpage pour minimiser le nombre de segments** — livrée (une seule passe, pas de fusion en cascade, § *Phase 7* ci-dessous).
8. **Recouvrements (*overlaps*) entre régions adjacentes + génération de géométrie de couture** — livrée (paires directement adjacentes uniquement, § *Phase 8* ci-dessous).
9. **Routage multi-colonnes (continuité directe / travel / jump / trim)** — livrée, pont vers le moteur `stitch_generation` existant (§ *Phase 9* ci-dessous), toutes les phases du plan sont désormais couvertes.

Phase 1 : réutilisation du `SkeletonGraph` existant + appariement de branches (`libs/satin_planning`)

État : Présent, testé, aucune régression.
`libs/satin_planning/include/openstitch/satin_planning/branch_pairing.hpp` + `src/branch_pairing.cpp` (nouvelle bibliothèque, ne dépend que de `openstitch::core` et `openstitch::auto_satin` — jamais l'inverse, la couche `auto_satin` reste indépendante de la planification).

Le graphe de squelette voulu par la SGSD (nœuds typés `Endpoint`/`Junction`, arêtes portant une centerline et des rayons locaux) EXISTAIT DÉJÀ presque intégralement avant cette session : `auto_satin::SkeletonGraph`/`SkeletonNode`/`SkeletonEdge` (`libs/auto_satin/include/openstitch/auto_satin/skeleton_graph.hpp`), construit par `build_skeleton_graph` et élagué par `prune_graph` (`graph_cleanup.hpp`), tous deux déjà exposés via `auto_satin::analyze_region(region, params).debug.graph` (graphe élagué, déterministe). La phase 1 ne réimplémente donc RIEN de cette partie — elle consomme directement ce graphe existant. Ce qui manquait réellement : une façon de décider, à chaque jonction, quelles branches se prolongent naturellement l'une l'autre plutôt que de traiter chaque arête indépendamment.

Coût de continuation (`continuation_cost`, `ContinuationCostWeights` — poids `angle=1.0`, `width=0.6`, `curvature=0.4`, tout centralisé, aucun coefficient dispersé) entre deux arêtes incidentes à une même jonction :

- `angle_cost` = `(1 + cos θ) / 2` où `θ` est l'angle entre les tangentes SORTANTES des deux branches (moyenne pondérée par longueur sur une fenêtre de `tangent_window_um` = 1,5 mm depuis la jonction, cohérent avec `GraphCleanupParameters::minimum_branch_length`) : 0 pour une continuation en ligne droite (tangentes opposées), 1 pour un repli sur soi-même.
- `width_cost` = `|log(rayonA / rayonB)|` (rayon local moyen sur la même fenêtre) — différence RELATIVE plutôt qu'absolue, pour qu'une transition 4,1 mm→4,0 mm coûte presque rien alors qu'une transition 1 mm→8 mm coûte cher.
- `curvature_cost` = stabilité de la tangente sur la fenêtre (écart entre le premier et le dernier segment échantillonné) — pénalise une branche qui tourne déjà fortement près de la jonction.

Résolution d'une jonction (`pair_branches_at_junction`) : toutes les paires d'arêtes incidentes sont testées (`C(degré, 2)` candidats, triés par coût croissant) et la moins coûteuse devient la continuation principale (`selected_pair`) ; les autres arêtes restent `detached` — chacune devient l'extrémité d'un `SatinPath` secondaire. Pour un degré 3 (T, Y), c'est un choix entre 3 appariements. Pour un degré ≥ 4 (croix), stratégie délibérément simple : une seule continuation principale, toutes les autres branches détachées individuellement — les appariements multiples simultanés (ex. `A↔C` ET `B↔D`) sont reportés à une phase ultérieure car ils peuvent produire des colonnes qui se croisent géométriquement au centre de la jonction, ce que la phase 1 (aucune géométrie générée) ne peut pas encore vérifier.

Décomposition complète (`decompose_into_paths`) : parcourt tous les nœuds `Junction`, construit la carte des continuations retenues, puis assemble des `SatinPath` maximaux en suivant les appariements de proche en proche — partition EXACTE des arêtes du graphe (chaque arête apparaît dans exactement un chemin, vérifié par test sur tout le corpus de formes). Un pont Jonction-Jonction (le cas `h`, § plus haut) qui continue naturellement aux DEUX bouts fusionne les deux montants et le pont en un seul chemin, exactement la même intuition que la conversion pont→colonne déjà en place côté générateur. `format_decomposition_report` produit un rendu texte structuré (jonction par jonction, candidats avec leurs coûts, sélection, chemins résultants) pour le debug — aucun `fprintf` direct dans la bibliothèque.

Aucune géométrie n'est générée à cette phase : ni découpage de polygone, ni appel à `build_satin_columns`. C'est une étape de planification topologique pure, décorrélée du générateur, exactement comme demandé.

Testé (`tests/unit/satin_planning/test_branch_pairing.cpp`, 12 cas) :

- Cas synthétique fait main (jonction en T à trois branches, coûts calculés à la main) vérifiant numériquement les trois formules de coût.

- Formes réelles du corpus `auto_satin::shapes` passées par le VRAI pipeline de rasterisation/squelettisation (`analyze_region`) : `rectangle` (0 jonction, 1 chemin), `t` (1 jonction degré 3, 2 chemins), `cross` (1 jonction degré 4, 1 continuation + 2 détachées, 3 chemins), `h` (2 jonctions, le pont détaché aux deux bouts confirmé par introspection du graphe, 3 chemins), `trident` (1 jonction degré 3, 2 chemins).
- Propriété de partition (chaque arête dans exactement un chemin) vérifiée génériquement sur 12 formes du corpus.
- Rendu `format_decomposition_report` non vide et contenant les marqueurs attendus.

Phase 3 : candidats de coupe + découpage réel du polygone (`libs/satin_planning/region_split`)

État : Présent, testé, aucune régression — v1 purement géométrique (pas encore d'oracle Auto-Satin, cf. phase 5). `libs/satin_planning/include/openstitch/satin_planning/region_split.hpp` + `src/region_split.cpp`.

Cette phase transforme chaque `SatinPath` (chemin topologique, phase 1) en un véritable sous-polygone (`SatinRegion`), en réutilisant tel quel l'outil de coupe manuelle déjà existant (`geometry::cut_path_set`, § *Ligne de coupe manuelle* plus haut) plutôt que d'écrire un nouveau moteur de découpe — la seule brique manquante était de choisir AUTOMATIQUEMENT où et dans quelle direction couper.

Un point important sur `cut_path_set` : ce n'est jamais une entaille locale — la coupe est toujours prolongée en une ligne complète qui traverse toute la boîte englobante de la région, quels que soient les points fournis (pensé pour un geste utilisateur qui vise la jonction sans forcément atteindre les bords exacts). Un candidat de coupe automatique doit donc être validé géométriquement avant d'être accepté, pas seulement calculé : la ligne peut en principe traverser une zone sans rapport si la forme n'est pas localement simple près de la jonction.

Recherche de candidats (`generate_cut_candidates`) : pour une branche détachée (jonction + arête, directement issues de `JunctionPairingReport::detached`), balaie des distances croissantes depuis la jonction (`search_min_um`=300 µm à `search_max_um`=1500 µm par pas de 150 µm, §11 spec SGSD — jamais exactement au pixel de jonction). À chaque distance, calcule le point et la tangente locale sur la centerline, construit la ligne de coupe selon la normale, et appelle `cut_path_set`. Rejette immédiatement (§13 spec SGSD, sous-ensemble géométrique — l'oracle Auto-Satin complet est phase 5) :

- un résultat qui n'a pas EXACTEMENT 2 morceaux (ligne qui a traversé une zone sans rapport) ;
- un fragment plus petit que `min_piece_area_mm2` (0,3 mm² par défaut).

Le premier candidat valide (le plus proche de la jonction) est retenu — un choix délibérément simple pour cette v1 géométrique pure ; la sélection par score global (§13 complet, oracle de couverture) est reportée à la phase 5/6, une fois `SatinabilityAnalyzer` et l'intégration Auto-Satin par région en place.

Découpage complet (`split_region`) : traite chaque événement de détachement (une jonction, une arête de son `detached`) dans un ordre déterministe (jonction puis arête croissantes), en localisant à chaque fois le morceau COURANT du pool qui contient le point de jonction (test point-dans-polygone par ray casting, implémenté localement — les chemins manipulés ici sont toujours polygonaux, jamais des courbes de contrôle) avant d'y appliquer la coupe retenue. Le morceau découpé est remplacé par les deux résultants dans le pool. **Aucun traitement spécial n'est nécessaire pour une branche à deux bouts de jonction** (le pont d'un « H ») : le second événement de détachement retrouve naturellement, dans le pool, le morceau laissé par le premier — la même mécanique générique s'applique. Assignation finale : chaque `SatinPath` reçoit le morceau du pool qui contient un point sonde intérieur à son propre tracé (un nœud strictement interne au chemin s'il y en a un, sinon le milieu de ses deux extrémités — jamais un point exactement sur une coupe). Une branche qu'aucune coupe valide ne sépare reste fusionnée avec son morceau parent, reportée dans `unresolved_paths` plutôt que silencieusement ignorée.

Testé (`tests/unit/satin_planning/test_region_split.cpp`, 4 cas) contre le VRAI pipeline de rasterisation/squelettisation, pas de fixtures synthétiques faites main pour cette phase — la géométrie réelle du contour est indispensable ici (contrairement à la phase 1, purement topologique) :

- `rectangle` (0 jonction) : aucune coupe tentée, la région ressort inchangée.
- `t` : une coupe à la distance minimale testée (300 µm) isole le pied (100,44 mm²) de la barre (238,84 mm²) sur les 9 candidats testés, tous valides et croissants avec la distance — confirme que le balayage se comporte de façon monotone et prévisible.
- Propriété générale sur `t`, `y`, `cross`, `h`, `trident` : **tous les chemins de ces cinq formes sont intégralement résolus** (`unresolved_paths` vide dans les cinq cas), y compris `h` (les deux coupes successives du pont fonctionnent sans logique dédiée) et `cross` (les deux branches détachées de l'unique jonction degré 4 sont isolées l'une après l'autre du même morceau restant). Aire cumulée des régions résolues toujours inférieure à l'aire d'origine (une coupe ne peut que retirer une fine bande, jamais en ajouter), `path_index` assignés uniques.
- Rendu `format_region_split_report` non vide et contenant les marqueurs attendus.

Aucune géométrie satin n'est générée à cette phase non plus — c'est un découpage de polygone pur, décorrélé du générateur.

Mise à jour (phase 4, voir ci-dessous) : la validation purement géométrique ci-dessus s'est révélée insuffisante et a été renforcée, pas remplacée — `generate_cut_candidates` intègre désormais un garde-fou de satinabilité (§ *Phase 4*) qui a directement corrigé un défaut réel trouvé sur `t`, `y` et `cross` : une coupe jugée valide par les seuls critères géométriques (exactement 2 morceaux, aucun fragment trop petit) pouvait en réalité trancher AUSSI la branche voisine près d'une confluence où plusieurs branches se chevauchent géométriquement, laissant une jonction résiduelle bien réelle dans le morceau censé être isolé.

Phase 4 : `SatinabilityAnalyzer` — et un défaut réel trouvé et corrigé dans la phase 3 (`libs/satin_planning/region_satinability`)

État : Présent, testé, aucune régression.
`libs/satin_planning/include/openstitch/satin_planning/region_satinability.hpp` et `src/region_satinability.cpp`.

Bonne surprise, même schéma qu'aux phases précédentes : la spec SGSD (§14) demande de « créer une vraie fonction `analyzeSatinability` » examinant le nombre de jonctions/extrémités du squelette, la largeur, la variation de largeur, etc. — exactement ce que `auto_satin::evaluate_satinability` / `auto_satin::analyze_region` font déjà (§ *Analyse de satinabilité* plus haut). Rien à réimplémenter : `check_region_satinability(region, params)` fait simplement tourner le VRAI pipeline existant (rasterisation → distance → squelette → graphe → `evaluate_satinability`) sur une `SatinRegion` déjà découpée, exactement comme le ferait `build_satin_columns` en interne, et classe le verdict `ready_for_generation` selon que le statut résultant est `Suitable`/`SuitableWithWarnings`. `check_all_regions` applique ça à tout un `RegionSplitReport` (phase 3), en reportant aussi automatiquement dans `not_ready` les chemins jamais isolés (`unresolved_paths`) — pas de verdict sans région, mais pas d'omission silencieuse non plus.

Défaut réel trouvé en testant bout-en-bout (décompose + découpe + vérifie) sur le corpus branché : chaque `SatinRegion` fraîchement découpée à la phase 3 était réanalysée isolément, et le résultat était systématiquement `RequiresDecomposition` (une jonction résiduelle) pour `t`, `y`, `cross` ET le pont d'un « H » — jamais `Suitable` comme attendu pour une branche qui vient d'être proprement détachée. Diagnostic en trois temps (instrumentation temporaire, retirée une fois la cause confirmée, même méthode que l'audit `trident`/pixel_size) :

1. **D'abord écarté : artefact de rasterisation.** Un balayage de `pixel_size` de 50 à 6,25 µm sur la région coupable ne change RIEN au résultat (`junctions=1` à toutes les résolutions) — élimine l'hypothèse d'un escalier de pixellisation au bord de la coupe (contrairement au cas `trident`/pixel_size documenté plus haut, qui LUI dépendait bien de la résolution).
2. **Cause confirmée : la coupe par défaut (300–1500 µm de la jonction) tombe encore dans la zone où les empreintes de deux branches voisines se chevauchent.** Sur `t`, la branche du pied (2,5 mm de demi-largeur) est testée à des distances qui restent À L'INTÉRIEUR de l'emprise en X de la barre horizontale (elles se croisent exactement à la jonction) : la ligne de coupe, bien que géométriquement

valide au sens « exactement 2 morceaux, aucun fragment trop petit », tranche EN RÉALITÉ un peu de la barre en même temps que le pied, laissant un résidu détectable comme une vraie troisième branche. Balayage direct de la distance de coupe (300 µm à 7 mm) : le morceau isolé reste branché (`RequiresDecomposition`) jusqu'à 2000 µm inclus, et devient proprement `Suitable` (0 jonction) à partir de 2600 µm — la marge par défaut de la phase 3 (max 1500 µm) n'atteignait jamais cette distance.

3. **Corrigé** : `CutCandidateParams` élargit la plage de recherche par défaut (`search_max_um` 1500→4000 µm, `search_step_um` 150→300 µm) ET `generate_cut_candidates` ajoute un garde-fou — quand le bout distal de la branche détachée est une VRAIE extrémité (pas une autre jonction), le morceau isolé par chaque candidat est réanalysé avec `auto_satin::analyze_region` et le candidat est rejeté si une jonction résiduelle subsiste, avant même de le proposer comme sélection. Le cas d'un bout de jonction (le pont d'un « H », § plus haut) est délibérément EXCLU de ce garde-fou : ce chemin a besoin de DEUX coupes successives, et le vérifier après la première le rejetterait à tort — il est encore légitimement branché à ce stade-là. Documenté comme limite connue plutôt que « corrigé » : une validation correcte du pont demanderait de ne réappliquer le garde-fou qu'après la DERNIÈRE coupe d'un chemin à deux bouts de jonction, une information que `generate_cut_candidates` (qui ne traite qu'un événement de détachement à la fois, sans mémoire du chemin complet) n'a pas.

Vérifié après correctif (`tests/unit/satin_planning/test_region_satinability.cpp`, 5 cas) :

- `rectangle` : déjà simple, prête sans découpe.
- `t`, `y`, `cross` (branches détachées se terminant toutes par une vraie extrémité) : **100 % des régions ressortent propres** (`junction_count == 0`, `ready_for_generation`) — le défaut ci-dessus est bien corrigé pour ces trois formes.
- `h` : les deux montants (jamais coupés qu'à une seule extrémité chacun) ressortent propres ; le pont (les deux bouts sur une jonction) reste honnêtement classé `not_ready` — limite connue, pas un échec silencieux.
- `trident` : la branche latérale, étroite et courte, peut ne trouver AUCUNE distance de coupe qui échappe complètement à l'emprise de sa voisine dans la plage testée — le garde-fou rejette alors TOUTES les distances candidates plutôt que d'accepter une coupe défectueuse, et `split_region` reporte honnêtement le chemin en `unresolved_paths` plutôt que de produire une région silencieusement fausse. Vérifié : quand une région EST résolue, elle est toujours effectivement propre (`junction_count == 0`).

Le coût de ce garde-fou est un `analyze_region` complet (rasterisation + squelettisation) par candidat testé sur une branche à extrémité réelle — pas gratuit, mais borné (13 candidats maximum par branche avec les paramètres par défaut) et seulement pendant la planification, jamais pendant la génération de points elle-même.

Phase 5 : Auto-Satin réel par région + Coverage Analyzer comme oracle complet (`libs/satin_planning/region_oracle`)

État : Présent, testé, aucune régression.
`libs/satin_planning/include/openstitch/satin_planning/region_oracle.hpp` et `src/region_oracle.cpp`.

Contrairement à la phase 4 (qui ne vérifie que le STATUT de satinabilité avant toute génération), cette phase construit réellement les colonnes satin sur chaque `SatinRegion` (`auto_satin::build_satin_columns`, mode `Parametric` pour reproduire le comportement de production d'`autodigitize.cpp` — repli automatique vers `Legacy` déjà géré en interne) puis mesure la couverture obtenue avec `satin_coverage::analyze_satin_coverage` — exactement l'oracle demandé par la spec SGSD (§15) : « la preuve finale reste, est-ce que l'Auto-Satin généré couvre réellement correctement la région ? ». `satin_coverage` reste totalement indépendant du décomposeur (ne connaît que des rails/barreaux génériques, jamais un type `auto_satin` ou `satin_planning`) — même garantie qu'à sa création (§ *Satin Coverage Analyzer* plus haut).

`evaluate_region_generation(region, genParams, coverageConfig, density)` convertit la sortie de `build_satin_columns` (`parametric_columns` si non vide, sinon `columns` — même sélection que `autodigitize.cpp`/le CLI, même défaut corrigé cette session, § *Balayage du corpus de formes*) en `satin_coverage::SatinColumnInput`, puis appelle l'analyseur. `evaluate_decomposition_generation` l'applique à tout un `RegionSplitReport` et agrège une couverture globale (aire couverte / aire cible sur toutes les régions).

Résultats mesurés sur le corpus branché (mode Parametric, seuils par défaut de `satin_coverage`, dont `min_core_coverage = 99,5 %`) :

- `t`, `y`, `cross` : chaque région (toutes propres depuis le correctif phase 4) atteint **97–99 % de couverture brute/cœur** — cohérent avec la limite déjà documentée des résidus de noyau de jonction/bouts (§ *Vérifié*, section Satin Coverage Analyzer) : aucune de ces régions ne franchit le seuil strict de 99,5 % (`passed = false` partout), mais ce n'est PAS un défaut nouveau — c'est la même limite connue de l'Auto-Satin actuel, simplement mesurée ici région par région plutôt que sur la forme entière.
- `h` : les deux montants (jamais coupés qu'à une seule extrémité chacun, propres dès la phase 4) couvrent aussi **~98 %** — mais **le pont (encore branché après découpe, `not_ready` en phase 4) chute à ~70 %** de couverture. L'oracle phase 5 confirme AVEC DES CHIFFRES ce que la phase 4 ne pouvait que signaler qualitativement (statut `RequiresDecomposition`) : un écart net et mesurable, pas juste un statut binaire.
- `trident` : la branche isolée avec succès couvre ~98 % ; la branche non résolue (§ phase 4, aucune coupe n'échappe à sa voisine) reste honnêtement absente du rapport de couverture plutôt que représentée par un chiffre inventé.

Ce résultat borne précisément le périmètre encore à couvrir par les phases suivantes : la phase 6 (recherche à faisceau) pourra comparer plusieurs découpages candidats en utilisant CET oracle comme fonction de score ; la phase 7 (fusion) pourra détecter qu'une paire de régions adjacentes aux couvertures individuellement bonnes redeviendrait encore meilleure fusionnée (ou, pour le pont du H, qu'une meilleure position de coupe/fusion est nécessaire avant que sa couverture ne devienne acceptable) ; aucune des deux n'est encore implémentée à ce stade.

Testé (`tests/unit/satin_planning/test_region_oracle.cpp`, 4 cas) :

- `rectangle` : couverture > 90 % sans aucune décomposition.
- Boucle complète sur `t`, `y`, `cross`, `h`, `trident` : chaque région effectivement construite produit un rapport de couverture exploitable (aire cible non nulle), que le seuil strict soit franchi ou non.
- `h` dédié : vérifie numériquement que le pont couvre nettement moins bien (< 85 %) que le meilleur montant (> 95 %) — non-régression directe sur la découverte ci-dessus.
- Rendu `format_generation_report` non vide et contenant les marqueurs attendus.

Phase 6 : recherche à faisceau guidée par l'oracle (`libs/satin_planning/beam_search`)

État : Présent, testé, aucune régression — portée réduite à une décision de coupe à la fois (voir plus bas). `libs/satin_planning/include/openstitch/satin_planning/beam_search.hpp` et `src/beam_search.cpp`.

Jusqu'ici, `split_region` retenait toujours le premier candidat géométriquement/topologiquement valide (le plus proche de la jonction, § *Phase 3/4*) — un choix arbitraire parmi plusieurs qui passaient déjà les garde-fous. La phase 6 remplace ce choix par une vraie décision de qualité, en utilisant l'oracle de la phase 5 pour ARBITRER entre plusieurs candidats plutôt que d'en accepter un au hasard.

Portée volontairement réduite par rapport à la spec (§16) : une recherche à faisceau complète explorerait des combinaisons de coupes sur PLUSIEURS jonctions simultanément (la spec avertit elle-même du risque d'explosion combinatoire). Cette phase se limite à une recherche locale, une décision de coupe à la fois — un vrai `beamWidth` (candidats évalués, meilleur retenu), mais appliqué indépendamment à chaque événement de détachement plutôt qu'à l'arbre complet des décompositions possibles. Une recherche multi-jonctions arborescente reste un travail futur, explicitement hors périmètre ici.

Mécanisme : `region_split.hpp` gagne un point d'injection, `CutCandidateSelector` (`std::function<optional<size_t>(PathSet, vector<CutCandidate>)>`), stocké dans `CutCandidateParams::selector` — vide par défaut (comportement historique inchangé, aucune régression pour les appelants existants). Ce point d'injection permet à `beam_search` de brancher une stratégie de sélection SANS que `region_split.hpp/.cpp` (qui reste un découpeur de polygone pur) n'acquière de dépendance vers `region_oracle.hpp` — c'est `beam_search` qui dépend des deux, jamais l'inverse, la même discipline de couches que partout ailleurs dans `libs/satin_planning`.

`OracleGuidedSelector` (`operator()` compatible `CutCandidateSelector`, à injecter via `std::ref`) matérialise, pour chaque événement de détachement, jusqu'à `beam_width` candidats déjà valides (les plus proches de la jonction d'abord — le pré-filtrage géométrique/satinabilité de la phase 3/4 s'applique toujours en amont, ce sélecteur ne peut jamais choisir un candidat que ces garde-fous ont déjà rejeté), construit réellement des colonnes satin sur chacun et mesure sa couverture (oracle phase 5 complet), puis retient celui dont la couverture brute est la meilleure. Chaque décision est journalisée (`decisions()`) pour le debug.

Résultat mesuré sur le pont du « H » (le cas le plus dégradé identifié en phase 5, ~70 % de couverture) : avec `beam_width = 3` et mode `Parametric`, sa couverture passe de **69,9 % à 92,3 %** — un gain net de plus de 20 points, obtenu SANS modifier le générateur de rails ni la logique de découpage elle-même, seulement en arbitrant, parmi des positions de coupe déjà valides, laquelle produit réellement le meilleur résultat une fois générée. Les deux montants (déjà propres à ~98 %) restent globalement stables (97,6 % et 98,4 %, les positions de coupe ayant légèrement bougé en conséquence).

Coût : jusqu'à `beam_width` générations + mesures complètes PAR événement de détachement, en plus des analyses de satinabilité déjà faites par `generate_cut_candidates` — réservé à un usage hors ligne (planification, CLI de debug, tests), jamais au chemin interactif.

Testé (`tests/unit/satin_planning/test_beam_search.cpp`, 4 cas) :

- `t` : résout toujours la branche, journalise une décision (au plus `beam_width` candidats évalués).
- Boucle sur `t`, `y`, `cross`, `h`, `trident` : le nombre de régions résolues avec le sélecteur guidé par l'oracle n'est jamais inférieur au comportement par défaut (le sélecteur ne peut que choisir MIEUX parmi les candidats déjà valides, jamais en révéler de nouveaux).
- `h` : deux décisions journalisées (les deux bouts du pont), au plus `beam_width` candidats chacune.
- `h`, non-régression numérique dédiée : confirme le gain de couverture du pont (< 80 % avant, > 85 % après, écart > 10 points) — verrouille la découverte ci-dessus.

Phase 7 : passe de fusion post-découpage (`libs/satin_planning/merge_pass`)

État : Présent, testé, aucune régression — une seule passe, pas de fusion en cascade (voir plus bas). `libs/satin_planning/include/openstitch/satin_planning/merge_pass.hpp` et `src/merge_pass.cpp`, plus une extension de `region_split.hpp/.cpp` (`RegionSplitReport::merge_candidates`, `MergeCandidate`).

Repérer les candidats de fusion sans reconstruction géométrique approximative — c'est le point délicat de cette phase. Une paire de régions adjacentes pourrait sembler se recombiner par une simple union Clipper2, mais la bande retirée par chaque coupe (`cut_width`, 20 µm par défaut) laisserait un interstice entre les deux morceaux : une union classique ne les refusionnerait PAS en un seul contour extérieur. La solution retenue évite le problème plutôt que de le contourner : `split_region` (phase 3) connaît déjà, à l'instant précis de chaque coupe, la géométrie EXACTE du morceau juste avant qu'elle ne soit appliquée — c'est littéralement l'union des deux résultats, sans aucun interstice. Il suffisait de la conserver. `split_region` trace maintenant, pour chaque coupe réussie, un petit arbre de filiation (quel morceau vient d'où) et expose `merge_candidates` : une entrée par coupe dont les DEUX résultats sont restés des feuilles jusqu'à la fin du découpage (jamais redécoupés par un événement ultérieur — cf. `cross`, où une seule des deux coupes qualifie, l'autre ayant son « reste » redécoupé ensuite). Chaque candidat porte directement la géométrie pré-coupe capturée en mémoire, prête à être réanalysée sans recalcul.

Décision de fusion (`evaluate_merge_pass`) : pour chaque candidat, construit réellement (oracle phase 5) les deux régions séparées ET la région fusionnée, compare la couverture obtenue (moyenne pondérée par aire des deux séparées, contre la couverture unique de la version fusionnée), et recommande la fusion si l'écart ne dépasse pas une tolérance configurable (`coverage_tolerance`, 2 points par défaut) — un seul segment vaut une petite perte de couverture, l'inverse (fusionner malgré une dégradation importante) non.

Portée volontairement réduite, même discipline que la phase 6 : chaque candidat est évalué INDÉPENDAMMENT, sans fusion en cascade (une région fusionnée n'est jamais elle-même retestée contre un troisième voisin). La spec (§16/§18) avertit explicitement du risque d'explosion combinatoire d'une passe itérative « jusqu'à stabilité » sur l'ensemble du graphe de décomposition — reportée à un travail futur.

Résultat vérifié sur `t` : la fusion est correctement **rejetée** (couverture séparée 97,68 %, couverture fusionnée 90,32 % — la version fusionnée réexpose exactement la complexité de jonction que la phase 3 avait résolue en coupant, un écart de 7,4 points bien au-delà de la tolérance). C'est la preuve que la passe ne réduit pas aveuglément le nombre de segments : elle mesure réellement, via le même oracle qu'ailleurs dans le plan SGSD, et ne fusionne QUE quand la qualité le justifie.

Testé (`tests/unit/satin_planning/test_merge_pass.cpp`, 4 cas) :

- `t` : exactement un candidat de fusion exposé, dont la géométrie fusionnée a une aire égale à l'aire d'origine (preuve indirecte qu'aucun interstice de coupe ne subsiste — la reconstruction est exacte, pas une union approximative).
- `cross` : sur les deux coupes, une seule expose ses deux enfants comme feuilles finales (l'autre a son « reste » redécoupé par l'événement suivant) — exactement un candidat de fusion, comme attendu de l'arbre de décision décrit ci-dessus.
- `t` : décision de fusion exploitable dans les deux sens (le test ne préjuge pas du sens de la recommandation, seulement de la cohérence des mesures).
- Rendu `format_merge_pass_report` non vide et contenant les marqueurs attendus.

Phase 8 : recouvrements entre régions adjacentes (`libs/satin_planning/overlap`)

État : Présent, testé, aucune régression — limité aux paires directement adjacentes (même restriction qu'à la phase 7, voir plus bas). `libs/satin_planning/include/openstitch/satin_planning/overlap.hpp` et `src/overlap.cpp`.

Le problème (§19 spec SGSD) : deux régions séparées exactement sur une ligne de coupe laisseraient, une fois cousues indépendamment, un interstice physique visible — la bande retirée par `cut_width` (20 µm par défaut). Il faut une petite extension de chaque région vers l'autre, mais SANS modifier les régions structurelles utilisées pour toute mesure de couverture (phases 4 à 7) : une géométrie dédiée à la broderie, distincte.

Méthode, entièrement construite sur des primitives déjà existantes — aucune nouvelle opération géométrique bas niveau n'a été nécessaire. Dilater une région de `overlap_distance` est un simple appel à `geometry::inset_path_set` avec un delta NÉGATIF (l'outil existant d'érosion/dilatation, déjà utilisé pour les sous-couches). Reste à garantir que l'extension « reste dans la shape originale » (§19) sans reconstruire une ligne de coupe déplacée : la région dilatée est recadrée (`geometry::intersect_polygons`) directement dans `MergeCandidate::merged_region` — la géométrie EXACTE d'avant coupe, déjà capturée sans interstice à la phase 7. Deux appels à des primitives existantes suffisent donc à produire une extension géométriquement correcte et bornée.

Résultat vérifié sur `t` (`overlap_distance` = 300 µm par défaut) : chaque région élargie est strictement plus grande que l'originale, jamais au-delà de la géométrie fusionnée de référence, et surtout — la preuve centrale de cette phase — **les deux régions élargies se chevauchent réellement désormais** (aire d'intersection mesurée : 2,95 mm²), là où les régions structurelles d'origine ne se touchaient pas (séparées par `cut_width`). L'interstice est fermé.

Limite connue, même restriction qu'à la phase 7 et pour la même raison : ne couvre que les paires DIRECTEMENT adjacentes dont les deux côtés sont restés des feuilles jusqu'à la fin du découpage (`RegionSplitReport::merge_candidates`, réutilisé tel quel — c'est déjà exactement l'ensemble des paires

dont la géométrie exacte d'avant coupe est connue). Une couture plus profonde dans l'arbre de décision (un côté redécoupé ensuite, ex. le pont d'un « H » vis-à-vis d'un montant) n'a pas encore de mécanisme de recouvrement — déterminer l'adjacence finale au-delà d'une seule coupe reste un travail futur. La densité de points dans la zone de recouvrement (§19 : « ne pas créer une surdensité énorme ») est hors périmètre de cette phase, purement géométrique — à traiter quand `stitch_generation` consommera cette géométrie dédiée.

Testé (`tests/unit/satin_planning/test_overlap.cpp`, 3 cas) :

- `t` : aire élargie strictement supérieure à l'originale pour les deux régions, jamais au-delà de la géométrie fusionnée, ET intersection non vide entre les deux régions élargies (l'invariant central : l'interstice est bien fermé).
- `cross` : un recouvrement généré par paire directement adjacente, en correspondance exacte avec `merge_candidates`.
- Rendu `format_overlap_report` non vide et contenant les marqueurs attendus.

Phase 9 : routage multi-régions (`libs/satin_planning/region_routing`)

État : Présent, testé, aucune régression — pont vers le moteur de routage existant, aucune logique de routage réimplémentée. Toutes les phases du plan SGSD (1 à 9) sont désormais livrées. `libs/satin_planning/include/openstitch/satin_planning/region_routing.hpp` et `src/region_routing.cpp`.

Encore une bonne surprise, le schéma qui revient tout au long de ce chantier : le moteur de routage que la spec réclame (§20 : continuité directe / travel run / jump, le trim comme conséquence du choix plutôt qu'une contrainte imposée au générateur) **existait déjà**, complet et testé — `stitch_generation::route_columns`/`RoutePlan` (Lot 6, § *Routage multi-colonnes* plus haut). Son vocabulaire diffère légèrement de celui de la spec (`ConnectorKind::Underpath` pour un trajet caché plutôt que « travel run », `Jump` qui porte déjà la coupe — donc le « trim » — dans sa propre définition) mais couvre exactement le même problème : ordonner et orienter des colonnes pour minimiser le déplacement, en préférant un trajet caché quand il est topologiquement justifié, un saut sinon. Rien à réimplémenter : la phase 9 est un pont, pas un nouveau moteur.

Le pont (`route_regions`) construit réellement une colonne satin par `SatinRegion` résolue (même conversion que l'oracle phase 5), la réduit à ses deux extrémités (centre des barreaux d'about, `stitch_generation::RouteColumn`) et confie l'ensemble à `route_columns`.

Limite connue, documentée plutôt que masquée : `RouteColumn::start_junction`/ `end_junction` ne sont pas renseignés. Dans le cas normal (une seule région non décomposée produisant plusieurs colonnes), ces champs portent un identifiant de jonction PARTAGÉ entre colonnes, ce qui autorise `route_columns` à valider un trajet caché avec une portée généreuse (`underpath_max`, 8 mm). Ici, chaque `SatinRegion` est réanalysée INDÉPENDAMMENT par `build_satin_columns` (son propre appel interne à `analyze_region`), avec sa PROPRE numérotation locale de jonctions — non comparable d'une région à l'autre. `route_columns` retombe donc sur son seuil de quasi-contact le plus strict (`underpath_max_without_junction`, 1,5 mm) pour toutes les liaisons entre régions SGSD : sûr (jamais de trajet caché injustifié à travers du vide), mais plus conservateur qu'il ne pourrait l'être — l'adjacence RÉELLE entre régions est pourtant déjà connue avec certitude via `RegionSplitReport::merge_candidates` (phase 7). Relier cette connaissance à la numérotation de jonctions attendue par `route_columns` reste un travail futur, qui débloquerait des trajets cachés à la portée normale précisément là où on sait déjà que deux régions étaient adjacentes avant découpage.

Testé (`tests/unit/satin_planning/test_region_routing.cpp`, 3 cas) :

- `rectangle` : une seule région, un seul pas de départ (`ConnectorKind::Start`), aucun saut.
- `t` : les deux régions sont routées, le plan contient exactement un pas de départ et le reste en trajets cachés/sauts, chaque chemin de la décomposition représenté exactement une fois (aucune région omise ni dupliquée dans le plan).
- Rendu `format_region_routing_report` non vide et contenant les marqueurs attendus.

Outil CLI `sgsd-debug` : brancher le pipeline complet, mesurer l'effet réel (2026-08-13)

État : Présent. `openstitch-cli sgsd-debug --shape <forme> [--pixel-size mm] [--beam-width n]` (`apps/cli/main.cpp`, `run_sgsd_debug`).

Les 9 phases du plan SGSD n'étaient jusqu'ici exercées que par des tests unitaires — aucun point d'entrée ne permettait de faire tourner le pipeline complet sur une forme et de voir, concrètement, si la décomposition change quoi que ce soit pour un utilisateur. `sgsd-debug` fait exactement ça, suivant le même modèle que `auto-satin-debug` (outil de diagnostic, aucune géométrie satin définitive écrite) : `analyze_region` → `decompose_into_paths` → `split_region` (avec un `OracleGuidedSelector`, phase 6, actif par défaut à `--beam-width 3`) → `evaluate_decomposition_generation` (phase 5) → `evaluate_merge_pass` (phase 7) → `generate_overlaps` (phase 8) → `route_regions` (phase 9), en imprimant le rapport texte de CHAQUE phase (les mêmes `format_*` déjà testés unitairement, rien de nouveau à faire confiance ici) puis une comparaison chiffrée : couverture agrégée du pipeline SGSD contre un appel direct à `build_satin_columns` sur la forme entière, non décomposée (l'approche historique).

Résultat balayé sur tout le corpus de formes

Forme	SGSD	Direct	Écart
`rectangle`, `capsule`, `ribbon`, `s`, `notch`	= Direct	—	0 (aucune décomposition nécessaire, comportement identique)
`y`	97,9 %	92,6 %	**+5,3**
`t`	97,7 %	90,3 %	**+7,4**
`cross`	97,8 %	90,6 %	**+7,2**
`h`	94,9 %	89,5 %	**+5,4**
`trident`	95,1 %	90,5 %	**+4,6**
`wide`, `tiny`, `pinch`, `circle`	refus (identique)	refus	0 (SGSD hérite fidèlement du refus de `build_satin_columns` par région, aucun faux négatif introduit)
`ring`	**0 %** (0 région)	**100 %**	**régression connue, voir ci-dessous**

Sur toutes les formes BRANCHÉES du corpus historique, la décomposition apporte un gain de couverture réel et mesurable (+4,6 à +7,4 points) par rapport à l'appel direct non décomposé — la validation concrète, chiffrée, que le chantier SGSD visait depuis le début. Sur les formes déjà simples ou déjà refusées, SGSD se comporte de façon strictement identique à l'existant : aucune régression silencieuse introduite par la décomposition elle-même.

Défaut réel trouvé et corrigé : `probe_point` sur un chemin sans nœud interne (`notch`)

En balayant le corpus avec cet outil, `notch` (bande à encoche concave profonde, § *Diagnostic sur formes concaves* — SANS AUCUNE jonction, un seul arc de squelette) ressortait à **0 région résolue sur 1** — un chemin pourtant trivialement résolu (aucune coupe nécessaire) rapporté comme « jamais isolé ». Cause : `probe_point` (utilisée par `split_region` pour retrouver, en fin de découpage, quel morceau du pool appartient à quel `SatinPath`) retombait, faute de nœud interne (un seul arc, donc deux nœuds seulement), sur le MILIEU DROIT entre les deux extrémités du chemin — une corde qui sort de la forme dès que le tracé courbe suffisamment autour d'une concavité profonde, faisant échouer l'assignation d'un chemin par ailleurs parfaitement valide.

Corrigé en préférant, dans tous les cas, un point pris DIRECTEMENT sur la centerline de l'arc médian du chemin — toujours intérieur à la forme par construction (c'est l'axe médian lui-même), quelle que soit la courbure ; le repli « milieu des extrémités » ne reste qu'un dernier recours théorique. Vérifié après correctif : `notch` ressort à 76,6 % de couverture, identique à l'appel direct (aucune décomposition n'était de toute façon nécessaire). Non-régression dédiée (`tests/unit/satin_planning/test_region_split.cpp`, « notch -- chemin unique sans jonction résolu malgré une forte courbure ») ; la propriété générale du corpus a aussi été renforcée pour exiger `unresolved_paths` vide sur toute forme sans jonction (`s`, `ribbon`, `wide` ajoutés au balayage général en plus de `notch`).

Limite réelle découverte, non corrigée : les anneaux (`ring`)

Le balayage a aussi révélé un vrai trou de couverture fonctionnelle du pipeline SGSD, distinct du défaut ci-dessus (pas un bug de bookkeeping, une lacune de conception) : **`ring` (anneau) n'est pas décomposé du tout** (`decompose_into_paths` renvoie zéro chemin) alors que l'appel direct le gère parfaitement via un chemin spécial dédié (`build_annular_sections`, § *Colonnes automatiques par squelette*). La raison structurelle : le squelette d'un anneau est une boucle fermée sans aucune vraie extrémité (`Endpoint`) ni jonction détectable par les règles actuelles de classification de `SkeletonGraph` — exactement le même mécanisme dégénéré déjà identifié pour `circle` lors de l'inspection initiale du code existant (§ *Première mission*), mais jamais quantifié jusqu'ici. **Documenté comme limite connue plutôt que masqué** : `sgsd-debug --shape ring` rapporte honnêtement 0 région et 0 % plutôt qu'un chiffre trompeur. Traiter la topologie annulaire dans `decompose_into_paths` (détecter un cycle fermé et le traiter comme un cas spécial, à l'image de `build_annular_sections` côté générateur) reste un travail futur.

Intégration production : SGSD dans `autodigitize.cpp` (2026-08-14)

État : Présent, seul chemin disponible (§ section suivante — le bascule `AutoOptions::use_sgsd_decomposition` mentionné ci-dessous a depuis été supprimé). `libs/autodigitize/src/autodigitize.cpp` (`try_sgsd_decomposition` à l'origine).

Le CLI `sgsd-debug` (§ ci-dessus) a permis de MESURER le gain SGSD, mais rien ne l'utilisait encore pour de vrai : `autodigitize.cpp` appelait toujours `build_satin_columns` directement sur la région entière. Cette étape branche enfin le pipeline sur le chemin réellement emprunté par l'application (numérisation automatique depuis une image).

Où et comment

Sur une région dont le squelette est réellement branché (`analyze_region(main).debug.graph.junction_count() > 0`), tente d'abord `decompose_into_paths` → `split_region` (recherche à faisceau active, `beam_width = 3`) → `build_satin_columns` RÉEL sur chaque `SatinRegion` résolue. Repli **intégral** sur l'appel direct historique dès que SGSD ne résout rien (`split.regions` vide — c'est exactement le gate qui protège les anneaux, § *Limite réelle découverte* ci-dessus : leur squelette n'a pas de jonction, donc SGSD n'est même pas tenté, et le chemin spécial `build_annular_sections` reste seul aux commandes). Les deux chemins (direct et SGSD) convergent ensuite dans le MÊME code d'émission de sections et le MÊME mécanisme de repli tatami structurel déjà en place — seule la manière dont les colonnes sont produites diffère, jamais le format de sortie ni les garanties de couverture.

Un vrai défaut trouvé en le branchant, corrigé avant d'aller plus loin

Faire tourner la suite de tests existante (`tests/unit/autodigitize`) contre ce nouveau chemin par défaut a immédiatement révélé un cas réel non couvert par les tests SGSD précédents (tous basés sur le corpus de formes propres, jamais sur le pipeline de rejet complet d'`autodigitize`) : quand SGSD isole une branche localement trop large en sa propre `SatinRegion`, cette sous-région — n'ayant plus la jonction voisine — n'est plus `RequiresDecomposition` mais passe par le chemin « colonne unique » de `build_satin_columns`, qui NE POUSSE PAS le même message « colonne refusee » que la boucle par arête du chemin direct en cas d'échec. Résultat : le remplissage de repli comblait bien la zone (aucun trou physique), mais `AutoResult::warnings` restait complètement VIDE — silence total sur la raison du repli, brisant la garantie explicite « jamais silencieux » du contrat documenté sur ce champ. Corrigé en poussant explicitement un message (incluant `built.refusal` s'il est renseigné) chaque fois qu'une sous-région SGSD ne produit aucune colonne.

Résultat mesuré

Sur le CORPUS SYNTHÉTIQUE (formes propres), l'intégration en production préserve exactement les gains déjà mesurés par `sgsd-debug` (+4,6 à +7,4 points sur les formes branchées, comportement strictement identique aux formes déjà simples ou refusées).

Sur une IMAGE RÉELLE (`tentabrode.png`, § diagnostic ci-dessus), le même diagnostic permanent, réexécuté après ce branchement, montre un résultat positif mais plus mesuré : couverture réelle agrégée (satin + repli tatami) **98,9 %**, contre 97,3 % avant l'intégration — un gain net, mais SGSD s'y appuie plus fréquemment sur le filet tatami que sur une couverture satin directement meilleure, contrairement au corpus synthétique propre. Cause probable, cohérente avec les limites déjà documentées (`notch`, branches étroites du `trident`) : les régions vectorisées depuis une vraie image sont nettement plus bruitées que les formes procédurales du corpus de test, et la recherche de coupe (phase 3, plage 300–4000 µm) échoue plus souvent à isoler proprement une branche courte ou irrégulière — auquel cas SGSD reporte honnêtement le chemin en `unresolved_paths` plutôt que de forcer une coupe défectueuse (§ phase 3/4), et le filet tatami prend le relais. La garantie de couverture reste intacte (aucune régression, gain net mesuré) ; améliorer la robustesse de la recherche de coupe sur des squelettes bruités reste un travail futur.

Testé

`tests/unit/autodigitize/test_autodigitize.cpp` : à l'introduction (avec bascule encore présente), le test historique du « réseau en T » (validant la topologie de jonction partagée du chemin direct) était conservé avec `use_sgsd_decomposition = false` explicite, à côté d'un nouveau test couvrant le comportement SGSD sur la même forme (2 sections indépendantes au lieu de 3, `topology` absent — état déjà documenté comme « colonne indépendante » dans `document::SatinParams`) ; le test de non-silence sur une lettre réelle trop large (`tests/unit/autodigitize`, fixture GISTRE) confirme le correctif ci-dessus. Depuis la suppression du bascule (§ section suivante), le test « réseau en T » côté direct pur a été retiré ici — cette garantie reste couverte indépendamment par `tests/unit/auto_satin/test_columns.cpp`, qui exerce `build_satin_columns` directement. Suite complète sans régression sur les 4 diagnostics permanents déjà connus.

SGSD exclusif : suppression du bascule + intégration UI desktop (2026-08-14)

État : Présent. SGSD n'est désormais plus une option activable/désactivable côté application : `AutoOptions::use_sgsd_decomposition` (§ section précédente) a été **supprimé** de `libs/autodigitize/include/openstitch/autodigitize/autodigitize.hpp`. Sur une région dont le squelette est branché, la décomposition guidée par squelette est désormais le SEUL chemin emprunté par l'application — plus d'échappatoire côté `AutoOptions`. L'appel direct historique à `build_satin_columns` sur la région entière reste utilisé, mais uniquement en INTERNE comme filet de sécurité structurel (région non branchée dès le départ, ou anneau — squelette en boucle fermée sans jonction détectable) : jamais une zone laissée sans le moindre point.

Point d'entrée unique partagé

Toute la logique auparavant privée à `autodigitize.cpp` (`try_sgsd_decomposition` + `sections_from_result`) a été extraite en une fonction publique, `autodigitize::build_satin_sections`, déclarée dans `autodigitize.hpp` aux côtés de `satin_params_from_column` :

```
[[nodiscard]] SatinBuildReport build_satin_sections(
   const geometry::PathSet& region,
   const auto_satin::SatinColumnsParameters& genParams,
   Micrometers density, Micrometers pullCompensation,
   bool centerUnderlay, Micrometers maxWidth,
   const std::string& warningLabel = {});
```

`SatinBuildReport` regroupe les sections construites (`BuiltSatinSection`, géométrie + `document::SatinParams`), un indicateur `used_sgsd`, un signal `structural_gap` (zone potentiellement non couverte, pour un éventuel repli tatami côté appelant), les avertissements accumulés, l'analyse de satinabilité de la région ENTIÈRE (`whole_region_report`, toujours calculée en amont, utile pour un aperçu même quand SGSD décompose ensuite en sous-régions) et un message de refus le plus pertinent si rien n'a pu être construit.

Ce point d'entrée est désormais utilisé par LES DEUX consommateurs de satin automatique de l'application, garantissant les mêmes garanties de couverture partout plutôt que de dupliquer la logique de décomposition + repli :

- `auto_digitize()` (`libs/autodigitize/src/autodigitize.cpp`), déjà branché depuis la section précédente ;
- trois des quatre points d'appel satin manuel de `apps/desktop/main_window.cpp` (§ ci-dessous).

Migration de l'UI desktop

`apps/desktop/main_window.cpp` appelait `auto_satin::build_satin_columns` directement à quatre endroits. Trois sont migrés vers `build_satin_sections` :

- **`createSatinObject()`** (« Colonne satin ») : la géométrie des rails ne dépend pas de la densité/compensation/sous-couche choisies dans le dialogue (uniquement des réglages de remplissage des points, appliqués en aval) — `build_satin_sections` est donc appelée UNE SEULE FOIS, avant le dialogue, avec les valeurs par défaut de `document::SatinParams`, puis les champs `density`/`pull_compensation`/`center_underlay` de chaque section sont réécrits avec les valeurs choisies par l'utilisateur une fois le dialogue validé — évite de relancer la recherche à faisceau (potentiellement coûteuse) une seconde fois pour la même forme. Les avertissements SGSD sont désormais remontés via `warnAboutSkippedAutoSatinBranches` (déjà utilisé après la segmentation IA), auparavant silencieux sur ce chemin.
- **Outil ligne de coupe** (`createSatinObjectWithCutLine`) : le dialogue étant affiché AVANT la boucle sur les morceaux coupés, la densité est déjà connue — chaque morceau appelle `build_satin_sections` directement avec les valeurs finales. Un morceau ne produisant aucune section reste ignoré (comportement inchangé), et les avertissements de tous les morceaux sont accumulés puis remontés en une seule fois en fin d'opération.
- **`autoConvertToSatin()`** (aperçu + confirmation avant conversion) : l'aperçu de satinabilité (statut, largeurs, nombre de branches) utilise désormais `whole_region_report` — l'analyse de la région ENTIÈRE, calculée une seule fois en amont même quand SGSD décompose ensuite en plusieurs sous-régions — plutôt que `SatinColumnsResult::report`, qui ne portait que sur l'appel direct.

`setStitchType()` (cas 2, conversion de type d'un objet existant) reste INCHANGÉ, intentionnellement : cette action exige EXACTEMENT une colonne unique (modèle « un objet, un type »), un cas explicitement hors du champ de la décomposition SGSD — commentaire déjà présent dans le code avant ce chantier. Elle continue d'appeler `build_satin_columns` directement.

Tests

`tests/unit/autodigitize/test_autodigitize.cpp` : le test « réseau en T » scindé par bascule (§ section précédente) n'était plus exprimable une fois `use_sgsd_decomposition` supprimé — la variante « SGSD désactivé » a été retirée (garantie du moteur direct seul déjà couverte indépendamment par `tests/unit/auto_satin/test_columns.cpp`), la variante SGSD renommée pour ne plus référencer un bascule qui n'existe plus. Suite complète (Debug + Release) sans régression sur les 4 diagnostics permanents déjà connus. Aucun nouveau test desktop ajouté pour la migration UI : le comportement géométrique exercé (`build_satin_sections`) est déjà couvert côté `autodigitize`, et les trois fonctions migrées n'ont pas de suite Qt dédiée préexistante à étendre.

Défaut réel : une sous-région SGSD acceptée peut laisser un reliquat massif non couvert (2026-08-14)

État : Corrigé. Signalé par une capture d'écran utilisateur montrant une zone visiblement sans le moindre point (ni satin, ni repli tatami) sur une grande partie d'une forme réelle (une lettre avec contre-poinçon — un trou proche d'une jonction de branches).

Root cause

`build_satin_sections` (§ *Intégration production* ci-dessus) déterminait `structural_gap` — le signal qui déclenche le remplissage tatami de repli — à partir de signaux purement STRUCTURELS : un chemin SGSD non isolé (`unresolved_paths`), ou une sous-région ayant produit **zéro** colonne. Ce signal ne détecte que l'absence totale de colonne sur une branche — jamais une colonne **acceptée** (au moins une colonne produite, statut `SuitableWithWarnings` ou `RequiresDecomposition`) dont la géométrie n'approxime qu'une fraction de la surface de sa propre sous-région.

Rejoué avec les coordonnées exactes exportées par l'utilisateur (`tests/unit/satin_planning/test_region_split.cpp`, nouveau test de régression) : la région (60,8 mm², un trou/boucle + 3 jonctions) se décompose en 4 chemins SGSD, **tous isolés, tous acceptés** (`regions=4, non_isoles=0`, chaque sous-région produit au moins une colonne) — succès structurel complet, donc `structural_gap` restait `false`. Pourtant, la sous-région contenant la boucle (21,2 mm², topologie proche d'un anneau) est si mal approximée par un unique ruban satin que le reliquat géométrique réel (région moins bandes réellement produites, mesuré par `geometry::subtract_polygons`) atteint **31,5 mm² sur 60,8 mm² (52 %)** — sans qu'aucun signal structurel n'ait jamais existé pour le détecter. C'est la même famille de limite que celle déjà mesurée sur `tentabrode.png` (§ *Intégration production* : « une branche **acceptée** dont la géométrie de colonne ne couvre pas fidèlement sa région »), mais ici démontrée **après** décomposition SGSD, pas seulement sur l'appel direct pré-SGSD.

Correctif

`structural_gap` est désormais calculé une seule fois, à la fin de `build_satin_sections`, à partir d'une mesure géométrique RÉELLE plutôt que d'un proxy structurel : `geometry::subtract_polygons(region, bandes des sections produites)`, avec un seuil mixte fixe **et** proportionnel (`max(1 mm², 3 % de l'aire totale)`) pour tolérer le reliquat NATUREL d'une pointe/jonction (quelques mm² même quand tout réussit, § *Intégration production*) sans le confondre avec un vrai trou structurel, quelle que soit la taille de la région. Ce calcul généralise et remplace les deux anciens signaux (arêtes du squelette vs sections produites côté direct ; chemins non isolés/refusés côté SGSD) : un chemin jamais isolé ou une sous-région refusée n'ont, par construction, aucune bande dans le calcul, donc leur surface apparaît déjà dans le reliquat mesuré.

Point de calibration important, trouvé en cours de correctif : la première version utilisait `shrink_strips_for_cutout` (le même rétrécissement de 0,4 mm par bande que la découpe du remplissage de repli lui-même, pour faire légèrement déborder le repli DANS la bande satin plutôt que de laisser un interstice) pour la MESURE du reliquat — ce qui gonflait artificiellement le reliquat mesuré d'une valeur proportionnelle au périmètre total des bandes, au point de déclencher un repli même sur le réseau en T et l'anneau fin du corpus synthétique (couverture déjà mesurée à 97,7 % et 100 % respectivement, § *Outil CLI* `sgsd-debug`) — un faux positif immédiat sur les deux tests existants. Corrigé en mesurant le reliquat sur les bandes à leur taille RÉELLE, non rétrécies ; le rétrécissement reste réservé, comme avant, à la découpe

du polygone de repli lui-même côté appelant (`auto_digitize`).

Résultat mesuré

Sur le réseau en T (corpus synthétique, jusqu'ici considéré « sans repli nécessaire » alors que sa couverture SGSD mesurée était déjà de 97,7 %, pas 100 %) : le reliquat naturel près de l'ancienne jonction est désormais correctement détecté et comblé par un remplissage tatami — comportement attendu, test mis à jour en conséquence (`tests/unit/autodigitize/ test_autodigitize.cpp`). Sur l'anneau fin (couverture 100 % mesurée) : aucun repli ne se déclenche, confirmant l'absence de faux positif sur un cas réellement complet. Sur la forme utilisateur ayant motivé ce correctif : 31,5 mm² désormais comblés en tatami plutôt que silencieusement absents.

Tests

- `tests/unit/satin_planning/test_region_split.cpp` — nouveau test (« région réelle avec boucle -- SGSD accepte chaque sous-région mais laisse un reliquat massif non couvert ») rejouant les coordonnées exactes de la forme utilisateur : vérifie le succès structurel complet (`regions=4, non_isoles=0`, chaque sous-région produit ≥ 1 colonne) **et** le reliquat géométrique réel (> 30 % de l'aire totale) malgré ce succès — la preuve que le signal structurel seul est insuffisant.
- `tests/unit/autodigitize/test_autodigitize.cpp` — le test « réseau en T » mis à jour pour vérifier explicitement l'apparition du remplissage de repli (`result->vectors.size() == 2`, un objet tatami présent) plutôt que son absence.
- Suite complète (Debug + Release) sans régression sur les 4 diagnostics permanents déjà connus.

Refonte du contrat satin : le planner récursif `create_satin_plan` (2026-08-14)

État : Présent. Changement de paradigme, pas une nouvelle optimisation locale : jusqu'ici, une région trop complexe pouvait se voir répondre « pas directement satinable » comme résultat FINAL, avec un repli tatami silencieux ou une heuristique naïve dégradée en substitut. Le contrat devient désormais : **une intention SATIN explicite (utilisateur ou classification automatique) doit toujours déclencher une recherche de partition satinable, jamais un refus direct** — `RequiresDecomposition` (et tout statut de satinabilité) redevient une information INTERNE au planner, jamais une réponse finale à l'utilisateur.

Le planner : `satin_planning::create_satin_plan`

Nouveau module (`libs/satin_planning/include/openstitch/satin_planning/ satin_plan.hpp` + `src/satin_plan.cpp`), point d'entrée unique désormais partagé par TOUS les chemins de création satin de l'application (auto-numérisation, création manuelle, ligne de coupe, conversion de type) via `autodigitize::build_satin_sections`, qui n'est plus qu'un mince adaptateur document au-dessus de lui.

```
SatinPlan create_satin_plan(const geometry::PathSet& source,
                            const SatinPlanConfig& config = {});
```

Algorithme, réellement récursif (contrairement à l'ancienne décomposition SGSD à une seule passe, § sections précédentes) :

1. **Essaie toujours le solveur local d'abord** (`auto_satin:: build_satin_columns`, inchangé — jamais patché, exactement l'esprit du chantier SGSD original) sur la région telle quelle.
2. **Mesure la couverture réellement obtenue** (`satin_coverage:: analyze_satin_coverage`, le même oracle indépendant que les phases 5 à 7 de SGSD) — jamais une règle théorique sur la topologie du squelette (nombre de jonctions, largeur agrégée…). Si la couverture passe (`SatinCoverageReport::passed`) ou que la profondeur maximale de récursion est atteinte, la région est acceptée telle quelle.
3. **Sinon, décompose** (réutilise `decompose_into_paths` + `split_region` + `OracleGuidedSelector`, exactement l'infrastructure SGSD existante, jamais réimplémentée) et **replanifie récursivement CHAQUE sous-région** — c'est ici la vraie nouveauté : une région fille encore médiocre peut à son tour être redécoupée, au lieu de s'arrêter après une seule passe.

4. Si la décomposition ne produit rien de mieux (aucune jonction, ou aucune coupe valide), le meilleur effort local est conservé tel quel plutôt que perdu.
5. **Réparation de résidu** (une fois l'arbre de récursion épuisé) : mesure la couverture AGRÉGÉE sur la région SOURCE entière et tente de replanifier chaque composante manquante significative comme une nouvelle région à part entière — jusqu'à `max_residual_repair_rounds` passes.

```
plan(region):
   candidate = try_local_satin(region)
   if candidate.couverture suffisante OU profondeur max atteinte:
       return candidate
   decompositions = decompose(region)
   if rien de mieux:
       return candidate  (meilleur effort)
   return concat(plan(sous-région) pour chaque sous-région)

# une fois la racine entièrement planifiée :
reliquat = couverture_agregee(source) - union(toutes les régions acceptées)
pour chaque composante significative du reliquat :
   tenter plan(composante) -- vraie tentative recursive, jamais un remplissage automatique
```

Ce que `create_satin_plan` ne fait JAMAIS

Ce module ne connaît même pas `document::TatamiParams` — il ne produit QUE de la géométrie satin + un résidu explicite (`SatinPlan::unresolved_residual`, une liste de `geometry::PathSet` brute, jamais transformée). Aucune décision de repli n'est prise ici : c'est strictement à l'appelant (§ section suivante) de décider quoi faire d'un résidu non nul — jamais une substitution automatique et silencieuse de l'intention SATIN.

Trois défauts réels trouvés en construisant le planner

1. Filtrer un reliquat individuellement négligeable peut masquer un vrai trou agrégé. Première version : une composante manquante sous un seuil de signifiance (aire, rayon de trou) était simplement écartée du résidu rapporté. Sur une forme réelle (lettre T d'un vrai logo, empattement large), ce filtrage a d'abord masqué un trou de **384,8 mm² sur 392,3 mm² source (98 %)** composé de nombreuses petites composantes individuellement sous le seuil. Corrigé en séparant les deux décisions que ce seuil confondait : il ne décide plus JAMAIS de ce qui est *rapporté* (`unresolved_residual` reste exhaustif, toute composante manquante y apparaît), seulement de ce qui mérite une *tentative de réparation individuelle* coûteuse (éviter de relancer `build_satin_columns` et une mesure de couverture complète sur chaque micro-résidu d'arrondi). Le filtrage de signifiance pour la décision finale (« faut-il alerter l'utilisateur ? ») vit désormais uniquement chez l'appelant (`autodigitize::build_satin_sections`, § section suivante), sur la somme totale du résidu — jamais composante par composante.

2. Une couverture invérifiable n'est pas une réussite. `analyze_satin_coverage` ne peut échouer (une vraie erreur, jamais un simple score bas) que sur une région cible dégénérée (moins de 3 sommets) — un artefact occasionnel d'un découpage ou d'une réparation de résidu. `build_satin_columns` construit parfois quand même des colonnes (souvent quasi vides) sur une telle région dégénérée. Accepter cela sans distinction d'une vraie réussite mesurée fabriquait une « réussite » invérifiable en silence : sur la même forme réelle, un plan se déclarait complet avec 4 régions acceptées alors que la couverture agrégée RÉELLE n'était que de **1,9 %**. Corrigé en routant explicitement ce cas vers le résidu plutôt que vers une acceptation aveugle — seule la profondeur de récursion maximale (dernier recours documenté, pas un raccourci général) accepte encore un résultat non vérifié.

3. Une couverture mesurée mais dérisoire n'est pas non plus une réussite. Trouvé en rejouant le diagnostic permanent `tentabrode.png` (une vraie image, § chapitre *Analyse et validation*) après les deux premiers correctifs : la couverture agrégée RÉELLE s'est effondrée à **16,0 %** (contre 98,9 % avant ce chantier), alors que le plan se déclarait sans résidu significatif. Cause : sur une région sans aucune jonction interne (un gros blob rond, aucune décomposition possible par construction), le « meilleur effort local » — une colonne unique mesurée à 1,2 % de couverture réelle — était accepté SANS aucun seuil minimal,

simplement parce que sa couverture avait pu être *mesurée* (donc distincte du défaut n°2 ci-dessus) et qu'aucune décomposition n'était possible pour faire mieux. Corrigé en ajoutant `SatinPlanConfig::min_fallback_coverage_ratio` (50 % par défaut) : un « meilleur effort » en dessous de ce seuil est désormais rapporté comme résidu au lieu d'être accepté tel quel — que ce soit au dernier niveau de récursion ou après un échec de décomposition. Ce seuil s'applique de la même façon à la boucle de réparation de résidu (§ ci-dessus), qui aurait sinon pu « réparer » un trou en y empilant plusieurs colonnes toutes individuellement dérisoires sans jamais s'apercevoir que la couverture agrégée restait mauvaise.

Intégration : `autodigitize::build_satin_sections` devient un adaptateur mince

`libs/autodigitize/include/openstitch/autodigitize/autodigitize.hpp` — `SatinBuildReport` expose désormais `unresolved_residual` (géométrie brute complète, jamais filtrée) et `aggregate_coverage` (`satin_coverage::SatinCoverageReport` sur la région source entière). `structural_gap` reste un raccourci booléen pour les appelants existants, mais recalculé à partir de la somme AGRÉGÉE du résidu (seuil mixte, fixe et proportionnel, même calibration que le correctif précédent) — jamais composante par composante, pour ne pas répéter le premier défaut ci-dessus à ce niveau.

Les deux workflows, distingués comme demandé (§24)

- **`auto_digitize()`** (auto-numérisation, classification automatique « AutoChoice » — aucun utilisateur interactif à qui proposer un choix) reste la seule voie qui crée encore un remplissage tatami de repli automatique sur un résidu significatif — mais plus jamais silencieusement : un avertissement explicite (« satin incomplet : X % de la région couverte, Y mm² comblés par un remplissage tatami de repli (classification automatique) ») est systématiquement poussé dans `AutoResult::warnings` avant toute création.
- **Les créations satin explicites côté desktop** (`createSatinObject`, l'outil ligne de coupe, `autoConvertToSatin`) ne créent plus JAMAIS de tatami de repli automatiquement. Un nouvel avertisseur partagé, `MainWindow::warnAboutIncompleteSatinCoverage` (`apps/desktop/main_window.cpp`/`.hpp`), informe l'utilisateur (couverture estimée, surface manquante, options — retoucher la coupe, créer un tatami manuellement, ou accepter le résultat partiel) au lieu de décider à sa place. `autoConvertToSatin()` va plus loin : la couverture estimée apparaît directement dans l'aperçu, AVANT la création (§23 du plan de refonte), pas seulement après coup.
- **`setStitchType()` (conversion de type d'un objet existant, cas satin)** passe désormais par le même point d'entrée unifié (`build_satin_sections`), bénéficiant de la même qualité de solveur récursif — mais reste honnêtement limitée par une contrainte STRUCTURELLE de `SetStitchTypeCommand` (remplace un seul `EmbroideryObject` par un seul jeu de paramètres, pas un lot). Sur une décomposition à plusieurs sections, la plus grande est conservée et l'utilisateur en est explicitement informé (« Cette forme nécessite plusieurs sections… ») plutôt qu'une substitution silencieuse par l'ancienne heuristique naïve dégradée.

Adjacence et recouvrement reliés au planner (§19/§20, 2026-08-14)

`satin_planning::generate_overlaps` (phase 8 du plan SGSD) existait déjà, testé et fonctionnel, depuis une phase antérieure du chantier — mais rien ne l'appelait depuis `create_satin_plan`, et `SatinPlan::adjacency` restait un vecteur systématiquement vide. Corrigé : `decompose_and_recurse` calcule maintenant, à chaque niveau de récursion, l'adjacence directe entre les sous-régions issues d'une même coupe (`RegionSplitReport::merge_candidates`) ET leur recouvrement de couture (`generate_overlaps`), puis réindexe ces paires vers leur position finale dans `SatinPlan::regions` au fur et à mesure que les résultats remontent la récursion (et, séparément, à travers la boucle de réparation de résidu, §17).

Deux nouveaux champs sur `SatinPlan` :

- `adjacency` — paires d'index dans `regions` connues comme directement adjacentes (une même coupe, les deux côtés restés des feuilles).

- `overlaps` — aligné 1:1 sur `adjacency` : la géométrie de couture dédiée (chaque côté élargi vers l'autre, `SatinPlanConfig::overlap_distance`, 300 µm par défaut) — repli sur la région non modifiée si le calcul échoue ou si `SatinPlanConfig::compute_overlaps` est désactivé, jamais un vecteur désaligné ou plus court.

Limite assumée, héritée de `generate_overlaps` lui-même : une paire dont un côté a été redécoupé ultérieurement (donc devenu plusieurs feuilles) n'est pas rapportée — déterminer l'adjacence à travers un redécoupage plus profond reste un travail futur. Le module continue de ne faire QUE calculer cette géométrie : aucune fusion ni génération de points n'y est reliée ici — c'est à un futur consommateur (génération de points, §20) d'en faire usage.

Tests dédiés dans `tests/unit/satin_planning/test_satin_plan.cpp` : vérifient sur le réseau en T que `adjacency`/`overlaps` sont bien peuplés, alignés en taille, que chaque extension de recouvrement ne rétrécit jamais la région d'origine, et que désactiver `compute_overlaps` laisse `adjacency` intact (deux informations indépendantes) tout en retombant proprement sur la géométrie identité.

Deuxieme famille de coupes candidates : JunctionSeparator reutilise (§14, 2026-08-14)

`generate_cut_candidates` ne produisait qu'une seule famille (normale au squelette, distance croissante depuis la jonction) — la cause directe du résultat catastrophique mesuré sur la lettre T réelle (1,9 % de couverture agrégée). Le plan de refonte demandait explicitement de réutiliser les `JunctionSeparatorInfo` déjà calculés par le moteur Legacy (`resolve_junction`, sommet reflex du contour à chaque confluence, ou à défaut intersection de bissectrice) plutôt que de deviner une position par balayage — c'est fait, sans réimplémenter la moindre géométrie de détection d'encoche.

Le principe : avant `split_region`, `decompose_and_recurse` appelle une fois `auto_satin::build_satin_columns` en mode Legacy FORCÉ (indépendamment du mode réellement utilisé pour la génération de rails) sur la région entière, uniquement pour récolter `junction_separators` — un appel diagnostique, jamais utilisé pour la géométrie satin produite. Pour chaque événement de détachement, `generate_cut_candidates` projette chaque séparateur de la même jonction sur la centerline de l'arête testée (distance d'arc + distance perpendiculaire) ; un séparateur suffisamment proche (`junction_separator_max_perpendicular_um`, 6 mm par défaut — calibré empiriquement sur `t`, où les vrais séparateurs mesurent 2,5 à 3,5 mm depuis la centerline, une tolérance de 2 mm initiale les rejetait tous à tort) devient une distance de coupe candidate **testée en premier**, avant le balayage régulier.

Ordre critique : `OracleGuidedSelector` (phase 6, beam search) n'évalue que les `beam_width` premiers candidats VALIDES du vecteur retourné (3 par défaut) — ajouter les candidats §14 en fin de vecteur les aurait pratiquement condamnés à ne jamais être évalués. Testés en premier, ils concourent en priorité pour le budget du beam search, cohérent avec le fait que ce sont les candidats les mieux motivés géométriquement.

Effet mesuré : sur le corpus synthétique branché (y/`cross`/h/ `trident`), la couverture agrégée est passée à 95–98 % (bien au-delà du seuil de test à 75 %). Sur la lettre T réelle en revanche, aucun changement — le moteur Legacy lui-même échoue à résoudre les mêmes jonctions incohérentes sur cette forme pathologique, donc `junction_separators` reste vide et la famille supplémentaire n'a simplement rien à proposer : dégradation honnête, jamais un crash ni une fabrication.

Effet de bord découvert en testant : le corpus `h`/`trident`/`cross` n'a plus jamais besoin d'une deuxième passe de récursion — la première décomposition, désormais meilleure, suffit. Bon signe de qualité, mais cela invalidait la prémisse du test dédié à la PROPRIÉTÉ de récursion (`create_ satin_plan : recursion reelle...`, qui vérifiait `depth > 1` quelque part dans ce corpus) : corrigé en désactivant `use_junction_separator_cuts` dans ce test spécifique — il isole désormais le mécanisme de récursion, pas la qualité d'une famille de coupes particulière.

Nouveau `SatinPlanConfig::use_junction_separator_cuts` (`true` par défaut) — désactivable pour éviter le coût d'un appel Legacy supplémentaire par niveau de récursion, ou pour isoler l'ancienne seule famille en test.

Testé (`tests/unit/satin_planning/test_region_split.cpp`) : la famille apparaît bien en premier et priorise le budget du beam search sur `t` ; un séparateur d'une autre jonction, ou trop éloigné

perpendiculairement, est correctement ignoré. Suite complète (satin_planning, autodigitize, desktop) sans régression après le fix de calibration ci-dessus.

Passe de fusion reliée au planner (§18, phase 7 SGSD, 2026-08-14)

`evaluate_merge_pass` existait déjà, testé et fonctionnel, depuis la phase 7 du chantier SGSD — mais comme `generate_overlaps` avant lui, rien ne l'appelait depuis `create_satin_plan`. La sélection de coupe (recherche à faisceau guidée par l'oracle) choisit déjà le découpage jugé le plus favorable au moment de couper, mais ne reconsidérait jamais après coup si NE PAS couper aurait donné un résultat comparable avec moins de segments (§18 : « le plus petit nombre de segments permettant une couverture et un satin corrects »).

Le principe : dans `decompose_and_recurse`, juste après `split_region`, `evaluate_merge_pass` est appelé sur le rapport de découpage complet. Pour chaque paire recommandée à la fusion, les deux régions ne sont PAS recousues individuellement — une seule récursion est lancée sur la géométrie fusionnée exacte (`MergeCandidate::merged_region`) à leur place. Chaque région issue de `split_region` participe à au plus un `merge_candidates` (celui de la coupe qui l'a créée), donc aucune ambiguïté sur quelles paires fusionner. Nouveau `SatinPlanConfig::use_merge_pass` (`true` par défaut) et `merge_pass_coverage_tolerance` (2 %, même défaut que `MergePassParams`).

Effet mesuré, honnêtement rapporté : sur tout le corpus de formes délibérément branchées (`t`/`y`/`cross`/`h`/`trident`), §18 ne recommande **aucune** fusion — le nombre de régions final est strictement identique avec ou sans la passe. Attendu : ces formes sont conçues pour nécessiter réellement une découpe, donc défusionner dégraderait la couverture au-delà de la tolérance. Le chemin « fusion appliquée » du wiring restait donc totalement inexercé par le corpus existant — pour le tester honnêtement (plutôt que de le laisser sans preuve), le test dédié force la recommandation via une tolérance de 100 % (`merge_recommended` devient vrai dès que la région fusionnée construit ne serait-ce qu'une colonne, indépendamment de la comparaison réelle de couverture) : confirme que le nombre de régions diminue réellement (2→1 sur `t`) quand la recommandation est positive — sans dépendre d'une forme du corpus où §18 gagnerait par hasard.

Testé (`tests/unit/satin_planning/test_satin_plan.cpp`) : fusion forcée réduit réellement le nombre de régions ; suite complète (satin_planning, autodigitize, desktop) sans régression.

Coupes concavité→concavité et concavité→bord opposé (§14, suite, 2026-08-14)

Les deux familles précédentes (normale au squelette, JunctionSeparator) sont toutes deux ancrées sur un événement de détachement à une JONCTION du squelette. Une région SANS jonction interne — un seul chemin topologique, comme `notch`/`pinch` du corpus (bande entaillée d'un profond V) — n'avait donc **rien** à quoi elles puissent s'accrocher : `decompose_and_recurse` renonçait immédiatement dès que `junction_count() == 0`, même quand le solveur local échouait clairement.

Le principe (nouveau module `libs/satin_planning/concavity_cuts`) : pure géométrie de CONTOUR, sans aucun graphe de squelette. - `concavité→concavité` : chaque PAIRE de sommets reflex (concaves) du contour extérieur, coupée en ligne droite entre les deux — le cas d'un sablier (deux entailles se faisant face). - `concavité→bord opposé` : chaque sommet reflex seul, coupé selon la direction perpendiculaire moyenne des deux arêtes qui s'y rejoignent — orientée **empiriquement** vers l'intérieur de la matière (testée point par point via un test point-dans-polygone, jamais supposée depuis le sens de parcours du contour, qui peut différer d'une concavité à l'autre sur une forme non convexe) — le cas d'une entaille unique.

Chaque candidat est filtré par les mêmes règles que les familles précédentes (exactement 2 morceaux, aucun trop petit), puis le meilleur est choisi en construisant réellement les deux moitiés et en mesurant leur couverture combinée (même principe que le beam search phase 6).

Effet mesuré, honnêtement rapporté :

Forme	Avant	Après
`pinch` (encoche à 0,3mm du bord opposé)	0 région, **0%** (échec total)	4 régions, **86,5%**
`notch` (encoche à 1mm du bord opposé)	1 région, 76,6% (repli dégradé)	3 régions, **79,9%**

Deux garde-fous trouvés par des régressions réelles en construisant cette famille (`tests/unit/autodigitize/test_autodigitize.cpp`) : 1. **Jamais pendant la réparation de résidu** (`fromResidualRepair`) : un fragment de résidu est une géométrie de DÉCOUPE (Clipper2, `subtract_polygons` sur la couverture déjà produite), souvent petite et irrégulière par construction — un sommet reflex « parasite » issu de ce bruit géométrique fragmentait le résidu en éclats au lieu de le laisser honnêtement rapporté (le réseau en T voyait son reliquat mesuré augmenter malgré l'« acceptation » de ces éclats). 2. **Jamais sur une région avec un trou** : un anneau a déjà son propre solveur local dédié et excellent (`build_annular_sections`, appelé inconditionnellement dès qu'il y a exactement un trou) — la famille concavité ne considère que le contour EXTÉRIEUR, jamais les trous, donc une coupe rectiligne y ignore complètement la topologie du trou. Sur un anneau réel (segmentation → vectorisation, contour légèrement irrégulier), un sommet reflex marginal du contour tranchait l'anneau en 9 fragments au lieu des 4 sections annulaires propres — préféré inconditionnellement (même règle que le chemin basé sur le squelette : la décomposition l'emporte sans comparaison de qualité dès qu'elle produit quoi que ce soit).

Nouveau `SatinPlanConfig::use_concavity_cuts` (`true` par défaut), `concavityCutParams`, `concavity_cut_beam_width` (6 par défaut).

Troisième défaut trouvé, en validation manuelle après les deux garde-fous ci-dessus : la famille concavité→concavité génère une paire de candidats par PAIRE de sommets reflex (O(n²)), chacune une vraie coupe Clipper2 — un contour issu de vectorisation réelle peut compter des dizaines de sommets « reflex » au sens strict mais géométriquement négligeables (quasi rectilignes, bruit de tracé), faisant exploser le temps de calcul sans qu'aucun ne représente une vraie encoche. Corrigé par un filtre d'angle minimal (`min_reflex_turn_deg`, 15° par défaut) appliqué AVANT toute construction de candidat, plus une borne dure de défense en profondeur (`max_reflex_vertices`, 12 par défaut — garde les concavités les plus prononcées, triées par magnitude de virage).

Testé (`tests/unit/satin_planning/test_concavity_cuts.cpp`, nouveau fichier) : aucune concavité sur un rectangle convexe ; `notch` n'a qu'**une seule** concavité réelle (contrairement à l'intuition visuelle — vérifié empiriquement : les « épaules » du V sont convexes, pas concaves, seule la pointe l'est) donc seule la famille bord-opposé s'y applique ; une forme « sablier » construite à la main (deux notches en vis-à-vis) exerce spécifiquement concavité→concavité. Suite complète (satin_planning, autodigitize, desktop) sans régression après les deux garde-fous ci-dessus.

Dialogue à choix multiples pour un satin incomplet (§23, 2026-08-15)

`warnAboutIncompleteSatinCoverage` était une simple `QMessageBox::information` — un résumé en prose des options (« retoucher la coupe, créer un tatami, ou accepter tel quel ») dont AUCUNE n'était réellement actionnable depuis la boîte elle-même. Remplacée par `askAboutIncompleteSatinCoverage`, qui retourne un choix réel (`ContinuePartial` / `UseTatami` / `Cancel`) via une `QMessageBox` à boutons personnalisés (« Continuer avec satin partiel » / « Utiliser tatami pour le reliquat » / « Annuler »), le détail par zone disponible via le bouton « Détails » que Qt ajoute automatiquement (`setDetailedText`) — couvrant « Voir le problème » sans un quatrième bouton dédié.

Changement d'architecture nécessaire : les deux appelants historiques (`createSatinObject`, `createSatinObjectWithCutLine`) committaient déjà l'`AddObjectBatchCommand` AVANT d'afficher l'ancien avertissement — un « Annuler » n'aurait donc pu qu'annuler une commande déjà appliquée. Le choix est désormais demandé AVANT toute création ; `Cancel` retourne sans avoir rien muté ; `UseTatami` construit des paires `VectorObject`+`EmbroideryObject` supplémentaires (même schéma que le repli automatique d'`autodigitize.cpp`, `appendTatamiFallbackObjects`, jamais réimplémenté différemment) et les inclut dans le MÊME `AddObjectBatchCommand` que les sections satin — un seul geste annulable, satin et repli tatami ensemble.

`autoConvertToSatin()` avait déjà son propre gate Oui/Non pré-création avec un aperçu de couverture, mais pas d'option tatami — unifié pour utiliser le même dialogue quand le reliquat est significatif (le Oui/Non simple reste le geste de confirmation le plus léger pour une conversion propre, sans interrompre inutilement l'utilisateur).

Défaut trouvé en écrivant les tests : le helper de test existant (`autoDismissModalDialogs`) appelait `QDialog::accept()` directement plutôt que de cliquer un bouton — invisible pour une `QMessageBox` à

boutons `QDialogButtonBox::Ok/Yes` standards (accept() en est l'équivalent), mais `askAboutIncompleteSatinCoverage` n'utilise QUE des boutons personnalisés (`addButton(text, role)`) : `clickedButton()` restait `nullptr` après `accept()`, lu à tort comme « Annuler ». Corrigé en cliquant réellement le bouton par défaut (`QMessageBox::defaultButton()->click()`), routant par la vraie machinerie de clic de Qt comme un appui sur Entrée. Nouveau `clickModalDialogButton(widget, texte)` pour choisir explicitement un bouton par son texte à travers plusieurs boîtes modales successives (densité puis §23) dans un test.

Testé (`tests/unit/desktop/test_main_window.cpp`, fixture `buildPinchShapeFixture` — encoche en V sans jonction de squelette, ~86 % de couverture réelle après §14, un reliquat largement significatif) : les trois choix bout en bout depuis `createSatinObject()` — satin seul sans tatami, satin + tatami de repli (un seul geste annulable), et document totalement inchangé sur annulation. Suite complète (61 tests desktop, +3) + Debug/Release (582 tests, mêmes 4 diagnostics permanents connus, aucune régression).

Recouvrement consommé par la génération de points (§20, 2026-08-16)

`generate_overlaps` (phase 8 SGSD) était relié au planner depuis le 2026-08-14 (`SatinPlan::adjacency/overlaps` peuplés) mais rien n'utilisait encore cette géométrie — les colonnes de chaque `SatinPlanRegion` restaient générées sur la géométrie structurelle seule, laissant l'interstice de coupe entre deux régions voisines visuellement ouvert malgré l'information déjà calculée.

Le principe : après la réparation de résidu (§17) et juste avant la mesure finale d'`aggregate_coverage`, `extend_columns_into_known_overlaps` reconstruit les colonnes de chaque région ayant un voisin direct connu (`SatinPlan::adjacency`) sur sa géométrie de recouvrement DÉDIÉE (`SatinPlan::overlaps`, jamais la géométrie structurelle — qui reste inchangée, seule référence pour toute mesure de couverture, comme documenté dès la création de ce champ). Garde-fou central : la reconstruction n'est acceptée que si la couverture mesurée sur la région STRUCTURELLE ne se dégrade pas par rapport aux colonnes d'origine — repli automatique et silencieux sur les colonnes d'origine sinon (échec de construction sur la géométrie étendue, ou couverture moindre) — jamais une dégradation acceptée au nom de fermer un interstice.

Effet mesuré, honnêtement rapporté : sur le réseau en T, la couverture d'une des deux régions passe de 98,3 % à 99,8 % (l'autre reste inchangée — son extension n'a simplement rien apporté), la couverture agrégée de 98,3 % à 99,0 %. Une amélioration réelle mais modeste, cohérente avec ce que fait la fonctionnalité : fermer un interstice de `cut_width` (20 µm) déjà petit, pas résoudre un déficit de couverture majeur (ça, c'est le rôle des quatre familles de coupes et de la réparation de résidu).

Nouveau `SatinPlanConfig::extend_columns_into_overlap` (`true` par défaut, sans effet si `compute_overlaps` est désactivé).

Testé (`tests/unit/satin_planning/test_satin_plan.cpp`) : garde-fou — chaque région couvre au moins autant avec l'extension qu'sans elle, jamais moins ; et effet réel — la couverture agrégée s'améliore réellement sur le réseau en T (pas seulement « ne se dégrade pas »), preuve que le recouvrement est réellement consommé et non un no-op. Suite complète (satin_planning, autodigitize, desktop) + Debug/Release sans régression.

Ce qui reste hors périmètre (limites connues, honnêtement documentées)

(État au 2026-08-16 : aucune limite logicielle connue à cette date. Voir la section « Durcissement du contrat du SatinPlanner » plus bas, datée du 2026-08-17, pour un audit délibérément adverse qui en a trouvé plusieurs — la philosophie de documentation change à partir de là : une liste vide n'est plus le but recherché, cf. §26 de cette mission.)

Champ EmbroideryIntent et regroupement du panneau d'objets (§21/§24, 2026-08-16)

La distinction AutoChoice/ForcedUserChoice (§24) restait implicite dans le CODE (quel chemin d'appel a créé l'objet), jamais un concept explicite du modèle de document — et les sections d'un même plan satin multi-régions apparaissaient en liste plate dans le panneau d'objets (§21), sans regroupement visuel.

Modèle de données : nouveau `document::EmbroideryIntent` (`AutoChoice` / `ForcedUserChoice`) sur `EmbroideryObject::intent`, `AutoChoice` par défaut (comportement historique, absent d'un `.osp`

antérieur ⇒ `AutoChoice`, jamais une supposition différente — champ purement additif, aucun bump de `kSchemaVersion` nécessaire, même convention que `overrides`/`topology`). Fixé à chaque site de construction : `AutoChoice` dans `autodigitize.cpp` (classification automatique, sans utilisateur interactif à qui proposer un choix) ; `ForcedUserChoice` dans les trois créations satin manuelles côté desktop (`createSatinObject`, `createSatinObjectWithCutLine`, `autoConvertToSatin`), le repli tatami explicite du dialogue §23 (`appendTatamiFallbackObjects`), et les créations manuelles de contour/tatami (`createRunningStitchObject`/`createTatamiObject`). `SetStitchTypeCommand` marque `ForcedUserChoice` à `apply()` (toute conversion de type via cette commande est une action explicite de l'utilisateur, menu contextuel ou inspecteur) et restaure l'intention précédente à `revert()` — undo/redo exacts, comme `params_`.

Regroupement du panneau d'objets : `DocumentPanel::objectsList_` migré de `QListWidget` à `QTreeWidget` — les `EmbroideryObject` partageant le même `source_vector` (les sections d'un plan satin) sont regroupées sous un nœud parent, mais SEULEMENT quand il y en a plus d'une ; un objet seul reste un simple item de premier niveau, visuellement identique à l'ancienne liste plate. API publique (`refresh`/`syncSelection`/signaux) inchangée — migration contenue entièrement dans `document_panel.hpp/.cpp`.

Défaut trouvé en écrivant les tests : `source_vector.value == 0` est la valeur *invalide* de `ObjectId` (`Id::valid()`, `ids.hpp`) — grouper naïvement par `source_vector` aurait mélangé à tort tous les objets qui n'en ont jamais reçu (ex. les deux objets indépendants de la fixture historique de `test_document_panel.cpp`, tous deux à `source_vector` par défaut). Corrigé en ne comptant/regroupant QUE les `source_vector` valides.

Testé (`tests/unit/desktop/test_document_panel.cpp`, 4 nouveaux cas ; `tests/unit/project_io/test_intent_persistence.cpp`, nouveau fichier ; `tests/unit/commands/test_undo_stack.cpp`) : regroupement correct (3 sections plus 1 objet autonome ⇒ 1 groupe plus 1 item de premier niveau, jamais 4 items plats) ; sélectionner une section groupée émet son propre id ; sélectionner le nœud de groupe lui-même n'émet rien (aucun ObjectId propre) ; `syncSelection` retrouve un enfant groupé en descendant dans l'arbre ; round-trip exact de `intent` (API publique) et absence ⇒ `AutoChoice` (fichier forgé à la main) ; `SetStitchTypeCommand` marque/restaure `intent` sur undo/redo. Suite complète (desktop, project_io, commands, autodigitize) + Debug/Release (585 tests, mêmes 4 diagnostics permanents connus, aucune régression).

Coupes polygonales (§14, dernier élément, 2026-08-16)

Dernière limite honnêtement documentée du plan de refonte (§14) : les quatre familles de coupe candidates (normale au squelette, JunctionSeparator, concavité→concavité, concavité→bord opposé) ne suivent qu'une ligne DROITE entre leurs deux extrémités — aucune ne gère une coupe en plusieurs segments (un « coude »). Contrairement aux trois familles précédentes, chacune motivée par un cas RÉEL en échec dans le corpus de test, **aucune forme du corpus connu ne déclenche ce besoin à ce jour** : les coupes polygonales ont donc été implémentées et validées sur une forme synthétique construite spécifiquement pour exercer honnêtement le mécanisme, pas sur une régression de corpus. Travail repris à ce stade précis après plusieurs tours explicites de validation de la portée (l'équipe a confirmé vouloir ce dernier élément malgré l'absence de cas réel connu).

Le principe : quand la coupe DROITE entre deux concavités (famille concavité→concavité) échoue parce que son point médian tombe hors de la région — la ligne « coupe dans le vide » plutôt qu'à travers la matière —, `find_elbow_waypoint` cherche un point de passage intérieur (recherche bornée, 12 pas le long de la perpendiculaire au segment direct depuis son milieu, deux sens) tel que les deux segments obtenus (concavité→coude, coude→concavité) restent chacun entièrement dans la région (vérifié par échantillonnage : le point de passage et le milieu de chaque sous-segment). Si un tel point existe, `try_polyline` construit et mesure la coupe en coude exactement comme `try_segment` le fait pour une ligne droite (mêmes règles d'acceptation : deux morceaux exactement, aucun trop petit) ; sinon, comportement inchangé (rejet « point médian hors de la région »). Désactivable via `ConcavityCutParams::try_elbow_cuts` pour isoler le comportement lignes-droites seul (comparaison A/B, non-régression).

Trois défauts géométriques réels trouvés en construisant la forme de preuve (aucun n'était visible sur le corpus existant, puisque cette famille n'y était jamais exercée) :

1. *Coin non coupé côté extérieur du coude* — les deux quadrilatères de coupe (un par segment) ne se touchaient qu'en un seul point exact (le coude), laissant un triangle de matière intacte du côté convexe du virage dès que l'angle n'était pas parfaitement rectiligne. Corrigé par un petit carré centré au point de coude (`make_square`, 3× la demi-largeur de coupe) qui referme ce coin quel que soit l'angle, sans mitre à calculer.
2. *Extension trop longue aux deux extrémités* — les segments d'entrée/sortie étaient prolongés très au-delà de la région (même marge que la famille concavité→bord opposé, 1 m) dans la direction du segment ENTRANT ; pour un coude, cette direction n'a aucun rapport avec la géométrie locale de la concavité visée et pouvait raser l'épaule voisine de la même encoche avant de ressortir par un bord sans rapport, y découpant un fragment parasite. Corrigé en réutilisant la bissectrice locale déjà calculée par la famille concavité→bord opposé (`inward_bisector`, extraite en fonction partagée) — la seule direction garantissant de « sortir proprement » de la matière près de ce sommet précis — avec une marge courte (200 µm) au lieu d'une extension arbitrairement longue.
3. *Sens de l'extension inversé* — première tentative de correctif du point précédent : la bissectrice pointe VERS L'INTÉRIEUR de la matière (c'est sa définition), mais le quadrilatère doit au contraire dépasser LÉGÈREMENT dans le VIDE de l'encoche pour que la soustraction traverse réellement le contour au lieu d'y être tangente (sans quoi la coupe ne sépare rien du tout — constaté empiriquement : un seul morceau en sortie). Corrigé en inversant le signe (`sub` plutôt que `add`).

Testé (`tests/unit/satin_planning/test_concavity_cuts.cpp`) : une forme synthétique dédiée (`elbow_obstructed_hourglass_shape`, un ruban 100×8 mm entaillé de deux notches pointus sur des bords opposés mais désalignés, plus une obstruction rectangulaire physique entre les deux qui bloque spécifiquement la ligne droite sans toucher aux notches) prouve que (a) la coupe droite échoue bien pour cette paire (point médian dans le trou de l'obstruction), (b) la coupe polygonale réussit et sépare la région en deux moitiés comparables, (c) avec `try_elbow_cuts=false`, la même paire est rejetée proprement sans qu'aucun candidat « (polygonale) » n'apparaisse (ancre de non-régression pour le comportement lignes-droites seul). Suite `test_satin_planning` complète (695 assertions, 67 cas), suites `test_autodigitize` et `test_main_window`, et Debug/Release complets (mêmes 4 diagnostics permanents connus, aucune régression) : le mécanisme est inerte sur tout le corpus existant (aucune forme ne le déclenche), confirmant qu'il s'agit d'un ajout pur, pas d'un changement de comportement pour les formes déjà gérées.

Tests

- `tests/unit/satin_planning/test_satin_plan.cpp` (nouveau) — corpus complet (rectangle, T, y/cross/h/trident, anneau), propriété de non-décomposition inutile sur une forme simple, preuve de récursion réelle (profondeur > 1 atteinte sur le corpus branché), garde-fou de profondeur maximale, et régression dédiée sur la lettre T réelle (couverture agrégée mesurée, résidu honnêtement non nul, jamais de région acceptée sans mesure de couverture).
- `tests/unit/autodigitize/test_autodigitize.cpp` — tests réseau en T et anneau rendus robustes à la présence ou non d'un petit repli tatami (détail de calibration interne, plus un compte exact d'objets) en vérifiant directement la garantie qui compte : couverture de bout en bout (satin seul, ou satin + repli) sans trou résiduel significatif.
- `tests/unit/satin_planning/test_concavity_cuts.cpp` — forme synthétique dédiée aux coupes polygonales (§14, dernier élément) : succès du coude là où la ligne droite échoue, et ancre de non-régression pour `try_elbow_cuts=false`.
- Suite complète (Debug + Release) sans régression sur les 4 diagnostics permanents déjà connus.

Durcissement du contrat du SatinPlanner (mission adverse, 2026-08-17)

État : Présent, partiellement. Le chantier précédent (§1-33) a livré l'architecture — `create_satin_plan` comme cœur récursif commun. Cette mission repart du principe inverse : « l'implémentation existe, donc il faut essayer de démontrer qu'elle NE respecte PAS encore son contrat », pas la considérer terminée parce que les tests passent. Elle a délibérément cherché des contre-exemples plutôt que d'ajouter de la couverture de tests pour la forme.

Phase 1 — Audit architectural (aucune modification de code)

Traçage des points d'entrée réels (pas la documentation) sur `desktop → autodigitize → satin_planning → auto_satin`. Quatre réponses, avec preuves de call sites :

- **`create_satin_plan` a un unique appelant** en tout et pour tout dans le code de production : `build_satin_sections` (alors dans `libs/autodigitize`). Le binaire desktop devait donc lier `autodigitize` — et transitivement `segmentation`/`vectorization`, deux bibliothèques d'auto-classification d'image — uniquement pour créer un satin MANUEL.
- **`build_satin_sections` était localisée dans `autodigitize` par accident historique** confirmé par son propre commentaire de tête (« seul point d'appel restant vers `libs/satin_planning` ») — écrite pour l'auto-numérisation, réutilisée telle quelle par le desktop, jamais déplacée vers une couche neutre alors qu'elle ne dépend d'aucune primitive d'auto-classification.
- **Deux workflows contournaient réellement `create_satin_plan`** : `createSatinObject()` et `setStitchType()` (cas satin) basculaient sur `stitch_generation::rails_from_contour` — une heuristique naïve, documentée ailleurs comme débordante sur les formes concaves/branchues et **désactivée par défaut** côté auto-numérisation (`AutoOptions:: use_naive_satin{false}`) — dès que le planner renvoyait zéro section, sans garde-fou équivalent sur les chemins `ForcedUserChoice`. Le résidu affiché dans le dialogue §23 pouvait de plus devenir obsolète après cette bascule (calculé avant, jamais recalculé après).
- **La conversion de type (`setStitchType`) utilise le même planner en RECHERCHE, mais pas en RESTITUTION** : `SetStitchTypeCommand` ne sait porter qu'UN seul `StitchParams` (contrainte du modèle de commande, pas du planner) — une décomposition multi-régions réussie était donc tronquée à la plus grande section, avec avertissement explicite mais perte réelle de territoire déjà résolu.

Phase 2 — Contrat de résultat explicite

`SatinPlan` gagne un champ `status` (`SatinPlanStatus::Complete`/ `Incomplete`/`Impossible`) calculé une seule fois en fin de `create_satin_plan`, jamais à déduire de `regions.empty()`. Critères A-E :

- **A** (résidu) — aucune pièce d'`unresolved_residual` ne dépasse le même seuil de significativité que la réparation de résidu.
- **B** (colonnes) — déjà garanti par construction (`accept_as_leaf` n'accepte jamais une région sans colonnes), re-vérifié explicitement (diagnostic `RegionWithoutColumns` si jamais violé).
- **C** (couverture globale) — **délibérément distincte** du seuil strict de l'analyseur de couverture (`coverageConfig.min_core_coverage`, 99,5 %, calibré pour la décision interne « redécouper ou accepter » PENDANT la récursion). Défaut réel trouvé en écrivant le tout premier test : un simple RECTANGLE, sans aucun défaut de génération, ne franchissait pas ce seuil (0,98995 mesuré — résidu naturel aux coins). Nouveau seuil dédié `SatinPlanConfig::complete_min_raw_coverage` (0,95, couverture BRUTE, moins sensible à ce bruit ponctuel).
- **D** (trou local) — `max_gap_radius_mm` de la couverture agrégée contre le seuil de l'analyseur.
- **E** (géométrie invalide) — réutilise `degenerate_interval_count` déjà calculé par l'analyseur de couverture indépendant (rails croisés).

Budgets d'exploration explicites (§18) : `max_total_regions`, `max_planning_iterations` et `max_oracle_evaluations`. Ils sont tous fondés sur des compteurs déterministes : la vitesse de la machine et la charge CPU ne peuvent plus changer le point d'arrêt ni le plan émis. Toute limite atteinte produit `Incomplete`/`Impossible` avec diagnostic `SearchBudgetExceeded`, jamais un faux `Complete`.

Phase 3 — Extraction de l'adaptateur générique, suppression des bypass

`BuiltSatinSection`/`SatinBuildReport`/`build_satin_sections` déplacés vers `libs/satin_planning` (`satin_sections.hpp/.cpp`) — `satin_planning` gagne `document` en dépendance PUBLIQUE (aucun cycle : `document` ne dépend d'aucun module de planification). `autodigitize` appelle désormais `satin_planning::build_satin_sections` comme n'importe quel autre consommateur ; le desktop lie `openstitch::satin_planning` directement et n'a plus besoin d'`autodigitize` pour créer un satin manuel (il le lie séparément, pour SA fonctionnalité propre : l'auto-numérisation).

Les deux bascules `rails_from_contour` (Phase 1) sont **supprimées** sur les chemins `ForcedUserChoice` : un plan sans la moindre section déclenche désormais un dialogue honnête (statut du planificateur affiché, choix tatami-ou-annuler), jamais une substitution silencieuse par une géométrie de moindre qualité.

Phase 4-5 — Corpus de torture et tests bout en bout

Douze nouvelles fixtures (`libs/auto_satin/shapes.cpp`) : `star5`, `asymmetric_star`, `comb`, `E`, `deep_recursive`, `multi_neck`, `dumbbell`, `deep_channel`, `two_holes`, `ring_branch`, `junction_with_hole`, `polygonal_cut_fixture` (portage exact de la forme qui a validé les coupes polygonales, exposée ici pour un test bout en bout du planner COMPLET). Nouveau fichier `tests/unit/satin_planning/test_torture_corpus.cpp` : invariants génériques sur tout le corpus (source jamais modifiée, aucune région n'explique une surface hors source, chaque feuille produit des colonnes, `Complete` implique l'absence de résidu significatif), preuve de récursion réelle à profondeur ≥ 3 (`deep_recursive`), déterminisme (§19, répétitions), invariance à la translation (§20).

Phase 7 — Défauts réels trouvés (et sort de chacun)

Le corpus de torture a immédiatement trouvé plus de défauts que l'architecture précédente n'en avait révélé en plusieurs semaines de corpus stable — signe que la philosophie « chercher activement les contre-exemples » fonctionne.

1. **Rectangle jamais `Complete`** (Phase 2, ci-dessus) — corrigé (seuil de complétude dédié, distinct du seuil interne strict).
2. **`comb` (6 jonctions en série) et `star5` (une jonction à 5 branches) prenaient chacun 16 à 44 secondes** pour une seule tentative de décomposition (mesures Debug, cf. mise en garde ci-dessous). Isolé précisément (§33 : diagnostic avant correctif) : le solveur local seul (`auto_satin::build_satin_columns`) reste rapide (moins d'une seconde, refuse ou réussit proprement) — c'est la génération de candidats de décomposition de `satin_planning` qui est coûteuse PAR ITÉRATION sur ces topologies. Cause probable : `OracleGuidedSelector`, le sélecteur PAR DÉFAUT de chaque décomposition, est documenté par son propre fichier source comme « réservé à un usage hors ligne (CLI de debug, tests), jamais au chemin interactif » — utilisé pourtant sans amortissement par `create_satin_plan`. Corrigé **partiellement** : `beam_width` effectif réduit à 1 au-delà d'un seuil de complexité locale ET d'un budget global d'évaluations à pleine qualité (`SatinPlanConfig::beam_width`, `max_junctions_for_full_beam_ search`, `max_oracle_evaluations_at_full_beam_width`) — a réduit le coût sans l'éliminer (le vrai goulot, la génération de candidats de `split_region` elle-même sur une jonction à haut degré, est identifié mais délibérément NON corrigé, cf. §33 : pas de patch opportuniste du générateur/planner sous la pression d'une seule fixture). Le budget déterministe d'oracle borne le nombre d'ITÉRATIONS de la récursion, mais **ne préempte pas un appel unique** à `split_region`/`generate_concavity_cut_candidates` déjà en cours : le budget n'est revérifié qu'au retour de cet appel, pas pendant. Sur une jonction pathologique où une seule génération de candidats prend déjà plusieurs dizaines de secondes (mesuré isolément, ci-dessus), le coût réel d'un appel peut donc rester élevé avant que le plan ne s'arrête. Constaté concrètement en exécutant la suite `test_satin_planning` complète (nombreux appels à `create_satin_plan` sur les formes lentes, cumulés par les tests d'invariants qui parcourent tout le corpus) : durée totale de plusieurs dizaines de minutes. Le budget garantit la terminaison EN NOMBRE D'ITÉRATIONS, pas un plafond de temps strict par appel.

Mise en garde vérifiée, pas supposée (§37) : ceci mélange DEUX limitations distinctes que les mesures Debug seules ne permettent pas de séparer. Un test Release identique (2026-08-17) montre

que `star5`, `comb` et `E` terminent tous les trois **sans jamais toucher le budget wall-clock** (aucun diagnostic `SearchBudgetExceeded`) — la génération de candidats, en code optimisé, va assez vite pour que la récursion s'achève naturellement. Et pourtant, aucun des trois n'atteint `Complete` même dans ce cas : la couverture/le résidu agrégés restent insuffisants, une limitation de QUALITÉ de la famille de coupes actuelle sur les jonctions à haut degré/branches nombreuses — indépendante de la vitesse, et donc réelle en production (Release), pas seulement un artefact de test. Les deux observations (lenteur Debug, qualité insuffisante même rapide) partagent la même cause structurelle (génération de candidats de coupe pas assez raffinée sur ces topologies), mais seule la seconde est une limitation logicielle de production à documenter comme telle ; la première n'est qu'un ralentissement de build à ne jamais coder en dur dans un test (cf. point 7 ci-dessous, où l'inverse se produit : une lenteur Debug PURE, sans aucune limitation de qualité réelle en Release). 3. **`E`, `multi_neck`, `deep_channel`, `asymmetric_star` partagent la même limitation** que `comb`/`star5` (branches ou concavités multiples) — trouvés en élargissant le corpus, pas corrigés individuellement (même cause racine). 4. **Régions à 2+ trous sans AUCUNE stratégie de décomposition** (`two_holes`) — ni le solveur annulaire dédié (`build_annular_sections`, qui ne gère qu'EXACTEMENT un trou) ni la famille concavité (gardée par `region.holes.empty()`) ne s'appliquaient. Corrigé **partiellement** : le garde-fou concavité passe de `holes.empty()` à `holes.size() != 1` (n'exclut plus que le cas où le solveur annulaire dédié, strictement meilleur, s'applique déjà) — insuffisant pour la fixture `two_holes` elle-même, dont le contour EXTÉRIEUR est un simple rectangle convexe sans sommet reflex à exploiter (aucune famille de coupe actuelle ne peut agir sur un contour convexe) : une vraie famille « couper ENTRE deux trous » resterait à écrire, hors de portée de cette mission. 5. **`polygonal_cut_fixture` échoue en bout en bout malgré une coupe valide** — le mécanisme géométrique fonctionne (`generate_concavity_ cut_candidates` trouve un candidat polygonal valide, prouvé en moins d'une milliseconde), mais `select_best_concavity_cut` (qui CHOISIT en construisant et mesurant réellement des colonnes sur chaque morceau) échoue à construire une colonne satisfaisante sur les morceaux résultant de cette géométrie précise — défaut déjà connu et volontairement descopé plus tôt dans le développement des coupes polygonales (couche `satin_coverage`/`auto_satin`, pas la coupe elle-même), maintenant redécouvert par un test bout en bout plutôt que caché par un test isolé. 6. **Double comptage d'aire entre régions lors de la réparation de résidu** (`notch`, `pinch`, `deep_recursive`, `junction_with_hole`) — trouvé par le tout premier lancement complet du nouveau test d'invariant « aucune région n'explique une surface arbitrairement extérieure » (§8/§23) : jusqu'à 9 % de surface source en trop dans la somme des régions. Cause : la géométrie « manquante » (`missing.region`, issue de `source \ colonnes-émises`) peut recouper le territoire STRUCTUREL d'une région déjà acceptée dès que cette dernière a une couverture interne imparfaite — normal, jamais 100 % (cf. `complete_min_raw_coverage`) — et se retrouvait replanifiée telle quelle comme une NOUVELLE région structurellement chevauchante. **Corrigé PUIS REVERTÉ** (§37 : vérifier qu'un correctif est une amélioration RÉELLE, pas seulement qu'il fait taire un test) — un premier correctif recoupait (`geometry::difference_polygons`) la géométrie manquante contre les régions déjà acceptées avant replanification, ne gardant que le reliquat réellement non revendiqué : l'invariant d'aire passait, mais `test_autodigitize` (réseau en T RÉEL, pas une fixture synthétique) s'est mis à laisser 33,8 mm² de tissu réellement non point-piqué (repli tatami compris), contre moins de 0,5 mm² avant — recouper produit des lambeaux fins le long des frontières entre régions, bien plus difficiles à point-piquer correctement que la géométrie manquante brute. Un vrai résidu de couverture est un défaut bien plus grave qu'un chevauchement de comptabilité interne entre le polygone structurel d'une région et celui de son patch de réparation — le correctif a donc été **reverté**, et c'est le TEST d'invariant qui a été corrigé à la place : il distingue désormais les régions de la décomposition primaire (dont la somme ne doit jamais déborder la source, vraie partition géométrique) des régions issues de la réparation de résidu (`SatinPlanRegion:: from_residual_repair`, chevauchement avec leur parent toléré par conception, borné à une marge large plutôt qu'à zéro). 7. **Le budget par défaut (calibré pour la réactivité interactive) est trop étroit pour les mêmes formes en build Debug non optimisé** — trouvé sur `cross`, puis `h` (formes ordinaires du corpus, présentes bien avant cette mission) : le test « budget généreux » (censé prouver que le budget par défaut ne se déclenche jamais sur le corpus habituel) échouait de façon parfaitement reproductible en Debug (~10,5 à 11 s pour un seul appel de génération de candidats sur une jonction), *alors qu'un test Release identique résout les deux formes en `Complete` net, sans le moindre diagnostic de budget*. Vérifié explicitement avant de conclure (§37 : ne pas supposer) plutôt que documenté comme une limitation

architecturale de plus — c'est un artefact de configuration de build (code géométrique/Clipper2 non optimisé en Debug), pas une propriété structurelle de `cross`/`h` elles-mêmes. Distinction importante avec le point 2 : `star5`/ `comb`/`E` ne touchent PAS non plus le budget en Release (même constat), mais eux n'atteignent quand même jamais `Complete` — une limitation de QUALITÉ réelle, indépendante du build. `cross`/`h` n'ont AUCUNE limitation résiduelle en Release (`Complete` net, aucun diagnostic) : leur cas est un pur artefact de vitesse Debug, sans contrepartie de qualité. **Corrigé** en distinguant clairement les deux usages du budget : le défaut de production (`max_oracle_evaluations=32`) reste borné et déterministe, tandis que le test qui vérifie « la décomposition réussit avec des ressources suffisantes » utilise un budget d'oracle explicitement généreux. Ce test n'a jamais eu vocation à valider un plafond de temps, seulement la logique de décomposition elle-même. 8. **`unresolved_residual` pouvait se vider à tort quand le budget s'épuisait PENDANT une réparation de résidu** — trouvé en investiguant le revert du point 6 ci-dessus (`test_autodigitize`, réseau en T réel, 33,8 mm² non signalés). Hypothèse initiale erronée : supposer que c'était le MÊME artefact Debug que `cross`/`h` (point 7) et simplement remonter `max_planning_wall_clock_ms` à 20 000 — invalidée en vérifiant (§37) : le temps écoulé réel à l'échéance SUIT le plafond configuré (~15,4 s à 10 s de plafond, ~29,6 s à 20 s de plafond) au lieu de se stabiliser, signe d'un coût qui CROÎT avec le budget disponible (même famille que `comb`/`star5`/`E`), pas d'un simple appel ponctuel un peu lent — remonter le plafond global n'aidait donc pas ce cas (juste plus lent avant le même échec) et ralentissait inutilement toute la suite sur les cas déjà pathologiques. Cause racine réelle, isolée par instrumentation directe (`std::cerr` temporaire dans `autodigitize.cpp`, retiré après diagnostic) : quand le budget s'épuise PENDANT un appel de réparation, `plan_recursive` applique délibérément son repli « meilleur effort local » (`adequateFallback`, §7 défaut du 2026-08-14 : ne jamais perdre une pièce) et accepte une région dont la couverture interne est à peine au-dessus de `min_fallback_coverage_ratio` — cette acceptation faisait alors disparaître la pièce de la liste de résidu accumulée round par round, sans qu'aucun round suivant n'ait la chance de re-mesurer et re-signaler ce qui restait réellement non couvert À L'INTÉRIEUR de cette région à peine acceptée (le round suivant s'arrête immédiatement sur `budget.exceeded`). Un résidu bien réel disparaissait donc silencieusement du rapport final. **Corrigé** à la source : `create_satin_plan` dérive désormais `unresolved_residual` de la mesure de couverture FINALE (déjà calculée, authoritative, sur l'état réellement émis) plutôt que du bookkeeping accumulé round par round pendant la boucle — qui peut devenir périmé exactement dans ce cas de bord. Le plafond wall-clock décrit dans cette investigation a depuis été supprimé au profit du compteur déterministe `max_oracle_evaluations`. Non-régression : `test_autodigitize` complet + `test_satin_planning` complet, Debug ET Release.

Phase 19 — Déterminisme

Répétitions (5×) sur `polygonal_cut_fixture`, `two_holes` : nombre de régions, statut, couverture agrégée, `regions_explored`/ `oracle_evaluations` identiques à chaque exécution. Le filet wall-clock qui rendait auparavant `deep_recursive`, `comb` ou `star5` sensibles à la charge machine a été supprimé : les limites de recherche sont maintenant des compteurs déterministes. Les formes pathologiques peuvent toujours terminer en `Incomplete`, mais leur point d'arrêt et leur sortie ne changent plus selon la vitesse du runner.

Limitations connues (honnêtement documentées, trois catégories)

Limitations logicielles connues et reproduites (corpus de torture, `tests/unit/satin_planning/test_torture_corpus.cpp`) :

- Couverture/résidu agrégés insuffisants pour atteindre `Complete` sur les régions à branches ou concavités nombreuses au-delà de ~4 événements de détachement : `star5`, `asymmetric_star`, `comb`, `E`, `multi_neck`, `deep_channel` — limitation de QUALITÉ de la famille de coupes actuelle, confirmée indépendante du build (Debug ET Release, cf. Phase 7 point 2) : ce n'est jamais un problème de lenteur en soi. En Debug (code géométrique non optimisé), la génération de candidats par itération est de surcroît assez lente pour épuiser le budget wall-clock AVANT même d'atteindre ce plafond de qualité ; en Release, la décomposition va assez vite pour s'achever naturellement, mais le résultat reste `Incomplete` pour la même raison de fond. Cause isolée (génération de candidats de `satin_planning::split_region`/`concavity_cuts`), correctif de fond délibérément reporté (§33).

- Aucune stratégie de décomposition pour une région à 2+ trous dont le contour extérieur est convexe (`two_holes`).
- Les coupes polygonales (§13 du plan de refonte précédent) fonctionnent géométriquement mais peuvent échouer à produire des colonnes satisfaisantes sur les morceaux résultants pour certaines géométries (`polygonal_cut_fixture`) — limite de la couche `auto_satin`/ `satin_coverage`, pas du mécanisme de coupe.

Limitations textile/physiques connues : aucune (ce chantier ne touche pas à la génération de points elle-même).

Zones non encore validées : **aucun passage sur machine à broder réelle** — statut expérimental déjà affiché en tête de ce chapitre, inchangé par cette mission. Sensibilité à la résolution de rasterisation (§21 de la mission) non explorée faute de temps. Fuzzing procédural (§22) non implémenté.

Tableau de comparaison (corpus complet, 2026-08-17)

Mesures **Release** (données Debug écartées : la Phase 7 a montré qu'un sous-ensemble de ces formes a un statut sensible au build — cf. points 2 et 7 — un tableau Debug aurait donc affiché des `Impossible`/`Incomplete` partiellement artefactuels). Machine de développement — les colonnes `Régions`/`Explorées` restent indicatives, non comparables à un budget CPU de production.

Forme	Statut	Régions	Explorées
rectangle	Complete	1	1
capsule	Incomplete	1	1
ribbon	Incomplete	1	1
s	Incomplete	1	1
notch	Incomplete	2	3
pinch	Incomplete	2	3
t	Complete	2	3
y	Complete	2	3
cross	Complete	3	4
h	Complete	3	4
trident	Complete	4	7
ring	Complete	1	1
star5	Incomplete[1]	7	13
asymmetric_star	Incomplete[1]	6	15
comb	Incomplete[1]	0	10
E	Incomplete[1]	0	10
multi_neck	Incomplete[1]	0	4
deep_recursive	Incomplete (profondeur≥3)	2	6
dumbbell	Incomplete	1	4
deep_channel	Incomplete[1]	0	2
two_holes	Impossible[2]	0	3
ring_branch	Complete	1	1
junction_with_hole	Incomplete	3	8
polygonal_cut_fixture	Incomplete[3]	0	2

[1] Limitation de QUALITÉ (couverture/résidu agrégés insuffisants avec la famille de coupes actuelle, cf. Phase 7.2-3) — confirmée indépendante du build (ni un artefact Debug, ni résolue par plus de budget). `cross`/`h` n'apparaissent PLUS dans cette catégorie : leur `Incomplete` mesuré en Debug (tableau précédent, avant vérification Release) était un artefact de build pur, corrigé en Phase 7 point 7 — les deux atteignent `Complete` net dès qu'on mesure correctement. [2] Aucune famille de coupe applicable (contour convexe, 2+ trous, cf. Phase 7.4). [3] Coupe valide mais construction de colonne insatisfaisante sur les morceaux (cf. Phase 7.5).

Critères de succès de cette mission

Atteints : contrat générique unique (A), statut explicite jamais confondu avec un succès (F), le planner ne connaît toujours aucun type Tatami (G), `ForcedUserChoice::Satin` ne change jamais silencieusement d'intention (H, déjà garanti par `SetStitchTypeCommand`), budgets bornés avec `Incomplete` honnête plutôt qu'un faux succès (I), corpus adversarial qui a réellement trouvé et documenté des limitations (J).

Partiellement atteints : tous les workflows explicites passent par le contrat commun (B) — vrai pour la recherche, pas pour la restitution multi-régions de `setStitchType` (limite structurelle déjà documentée en Phase 1, non corrigée — changerait le contrat de `SetStitchTypeCommand`, hors périmètre). Récursion réelle jusqu'aux feuilles (C) — prouvée (profondeur ≥ 3), mais le budget peut désormais interrompre cette récursion avant son terme naturel sur les formes complexes. Coupes polygonales exercées bout en bout (D) — le mécanisme l'est, la sélection finale échoue sur la fixture dédiée (Phase 7.5). Formes cycliques/trouées testées (E) — testées, et une lacune réelle trouvée (2+ trous) plutôt que masquée.

Tests

- `tests/unit/satin_planning/test_torture_corpus.cpp` (nouveau) — corpus de torture complet, invariants génériques, déterminisme, invariance à la translation.
- `tests/unit/satin_planning/test_satin_plan.cpp` — tests dédiés au contrat de statut (`SatinPlanStatus`, diagnostics, budgets).
- Suite complète (satin_planning, autodigitize, desktop) + Debug/Release sans régression sur les 4 diagnostics permanents déjà connus.

Implémentation associée

- `libs/document/.../embroidery_object.hpp` — `SatinParams`, `SatinRung`.
- `libs/stitch_generation/src/satin.cpp` — `fill_satin`, `fill_satin_columns` (par barreaux), `rails_from_contour`, `ladder_correspondence` + `resample_by_medial_spacing` (correspondance locale rail A/rail B, audit rails 2026-08-01).
- `libs/stitch_generation/src/satin_guides.cpp` — projection sur rail et validation monotone partagées par l'éditeur de guides.
- `libs/stitch_generation/src/generate.cpp` — `generate_satin` (route vers `fill_satin_columns` si barreaux présents, oriente par entrée/sortie, émet les locks).
- `libs/stitch_generation/src/lock.cpp` — `lock_stitches`, `LockType` (Lot 5).
- `libs/stitch_generation/src/routing.cpp` — `route_columns`, `RoutePlan` (ordre/orientation/liaisons, Lot 6) ; `generate_satin_group` dans `generate.cpp` (émission des groupes routés).
- `libs/auto_satin/.../satin_column.hpp` + `src/satin_column.cpp` — `build_satin_columns`, `SatinColumnGeometry`, `SatinRung`, `JunctionCore` (moteur géométrique) ; `extend_tip` (bouts ouverts) et `trim_unstable_junction_tail`/`representative_station_width` (amputation de la queue instable d'un bout de jonction jusqu'à sa dernière section réellement stable) ; `cross_section`/`CrossSectionFailure`, `interpolate_station`/`interpolated_station_valid` (assemblage des stations tolérant aux trous, audit génération partielle 2026-08-03) ; `collect_junction_branches`/`StableBranchEnd`/`make_stable_branch_end`/ `reflex_vertices`/`build_separator`/`build_sector`/`resolve_junction` (partition StableBranchEnd/JunctionSeparator/secteurs par jonction, sans mutation de rail, noyau calculé comme

`local_region – union(secteurs)` — remplace l'optimisation combinatoire de bridges, 2026-08-04) ; `station_point`/`outward_tangent_at` (variantes à `offset` de `terminal_point`/`outward_tangent`, retractation itérative d'une `StableBranchEnd` qui croise une branche voisine) ; `polygon_self_intersects` ; `ContourPolyline`/`project_to_contour`/`extract_contour_arc` (utilitaires de contour, diagnostic uniquement, aussi utilisés pour la recherche de séparateur par contiguïté de contour). Tout ce paragraphe est le moteur `Legacy` (`SatinGeometryMode::Legacy`, mode par défaut), inchangé.

- `libs/auto_satin/.../satin_column.hpp` + `src/satin_column.cpp` — mode `Parametric` (refonte 2026-08-04) : `ParametricSatinObject`, `SatinControlPair`, `SatinAngleGuide`, `SatinJunctionPlan`, `SatinGeometryMode` ; `compute_column_stations` (couche d'analyse dense PARTAGÉE avec `Legacy`, extraite de l'ancien `build_column`) ; `select_structural_indices`/`width_extrema_indices`/ `adjacent_width_jump_indices`/`curvature_maxima_indices`/ `douglas_peucker_indices` (sélection des paires structurantes) ; `fit_cubic_segment`/`cubic_eval`/`cubic_point_distance` (moindres carrés à tangentes fixées, longueur de poignée = projection de la corde sur la tangente) ; `fit_both_rails`/`fit_both_rails_recursive`/`JointRailSegment`/ `RailFit` (ajustement conjoint des deux rails, ancrages partagés, subdivision adaptative) ; `rail_bezier_path`/`build_control_pairs_and_guides` ; `build_parametric_object` (finalisation paramétrique complète d'une branche) ; `extend_into_confluence`/`junction_overlap_target` (recouvrement local de jonction, remplace `resolve_junction` pour ce mode) ; `plan_junction_stitch_order` (§ étape 10) ; `validate_parametric_object`/`distance_to_polys` (§ étape 12).
- `libs/auto_satin/src/skeleton_graph.cpp` — `build_skeleton_graph` : consolidation des amas de pixels de jonction 8-connexes en un seul `SkeletonNode` (`DisjointSet`), suppression des micro-arêtes internes à l'amas (audit jonctions branchées/concaves, 2026-08-03).
- `libs/auto_satin/src/debug_export.cpp` — `columns_to_svg` (mode `Legacy`) ; `parametric_to_svg` (mode `Parametric`, 2026-08-04 : rails aplatis, poignées Bézier, paires structurantes, recouvrement, ordre de couture, statistiques par objet en commentaire).
- `libs/auto_satin/.../shapes.hpp` + `src/shapes.cpp` — corpus de formes de test (`make_shape`), dont `notch`/`pinch` (encoche concave, audit génération partielle 2026-08-03) et `trident` (jonction concave asymétrique à 3 branches inégales, audit jonctions branchées/concaves, 2026-08-03).
- `apps/cli/main.cpp` — `run_auto_satin_debug` : option `--satin-geometry= legacy|parametric` (2026-08-04, défaut `legacy`).
- Tests : `tests/unit/stitch/test_satin.cpp` (dont « rail Bezier eparse -> points suivent la courbe, pas la corde », 2026-08-04 — vérifie que `fill_satin_columns`/`to_points` aplatit réellement un rail à tangentes), `tests/unit/stitch/test_satin_pairing_metrics.cpp` (fixtures et métriques de l'appariement, audit rails 2026-08-01), `tests/unit/auto_satin/test_columns.cpp` (§ « parametrique : ... », six nouveaux cas 2026-08-04 : simplification réelle, embouts arrondis dans la région, trident à 3 objets sans croisement, recouvrement borné, correspondance monotone, lignes d'angle non croisées), `tests/unit/auto_satin/test_pipeline.cpp`.
- `libs/satin_coverage/include/openstitch/satin_coverage/coverage.hpp` + `src/coverage.cpp` — `analyze_satin_coverage`, `SatinColumnInput`, `SatinCoverageReport`, `MissingRegion`, `SatinCoverageConfig` (Satin Coverage Analyzer, 2026-08-13, § ci-dessus).
- `libs/stitch_generation/include/openstitch/stitch_generation/satin.hpp` + `src/satin.cpp` — `SatinStation`, `satin_stations` (correspondance structurelle rail A/rail B exposée publiquement, extraite sans changement de comportement de `fill_satin_columns`, 2026-08-13).
- `libs/geometry/include/openstitch/geometry/boolean.hpp` + `src/boolean.cpp` — `path_set_area_um2`, `intersect_polygons`, `difference_polygons` (booléens NonZero généralisés `vector<PathSet> × vector<PathSet>`, trous compris des deux côtés, 2026-08-13).
- `libs/geometry/src/offset.cpp` — `inset_path_set`, réoriente désormais chaque contour (`oriented_as`) avant Clipper2 (correctif balayage auto-satin, 2026-08-13).
- `libs/auto_satin/src/shapes.cpp` — trou de `ring` réorienté à la construction (2026-08-13).

- Tests : `tests/unit/satin_coverage/test_coverage.cpp` ; `tests/unit/auto_satin/test_coverage_regression.cpp` (non-régression sur le corpus de formes historiques, 2026-08-13) ; `tests/unit/geometry/test_offset.cpp` (érosion d'un trou de même orientation que son extérieur, 2026-08-13) ; `tests/integration/test_pipeline.cpp` (« DIAGNOSTIC TEMPORAIRE couverture satin (tentabrode) », combinaison satin + repli tatami par `source_region` sur une image réelle, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/branch_pairing.hpp` + `src/branch_pairing.cpp` — `continuation_cost`, `pair_branches_at_junction`, `decompose_into_paths`, `format_decomposition_report`, `SatinPath`, `JunctionPairingReport` (SGSD phase 1, appariement de branches au-dessus du `SkeletonGraph` existant, 2026-08-13, § ci-dessus) ; `find_edge`/`find_node` (recherche par id, exposées publiquement pour réutilisation par `region_split`).
- `libs/satin_planning/include/openstitch/satin_planning/region_split.hpp` + `src/region_split.cpp` — `generate_cut_candidates`, `split_region`, `format_region_split_report`, `SatinRegion`, `CutCandidate`, `CutAttempt` (SGSD phase 3, candidats de coupe + découpage réel du polygone via `geometry::cut_path_set`, 2026-08-13, § ci-dessus) ; garde-fou de satinabilité intégré à `generate_cut_candidates` + plage de recherche élargie (`CutCandidateParams::search_max_um` 1500→4000µm, correctif phase 4, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/region_satinability.hpp` et `src/region_satinability.cpp` — `check_region_satinability`, `check_all_regions`, `format_satinability_report`, `RegionSatinabilityVerdict` (SGSD phase 4, réanalyse de satinabilité d'une `SatinRegion` déjà découpée via `auto_satin::analyze_region`, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/region_oracle.hpp` et `src/region_oracle.cpp` — `evaluate_region_generation`, `evaluate_decomposition_generation`, `format_generation_report`, `RegionGenerationVerdict` (SGSD phase 5, `build_satin_columns` réel + `satin_coverage::analyze_satin_coverage` par région, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/region_split.hpp` — ajoute `CutCandidateSelector` + `CutCandidateParams::selector` (point d'injection phase 6, vide par défaut, comportement historique inchangé, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/beam_search.hpp` et `src/beam_search.cpp` — `OracleGuidedSelector`, `BeamSearchParams`, `BeamCandidateScore` (SGSD phase 6, recherche à faisceau locale guidée par l'oracle phase 5, gain mesuré de 69,9→92,3 % sur le pont du « H », 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/region_split.hpp` — ajoute `MergeCandidate` + `RegionSplitReport::merge_candidates`, tracés par un petit arbre de filiation interne à `split_region` (phase 7, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/merge_pass.hpp` et `src/merge_pass.cpp` — `evaluate_merge_pass`, `format_merge_pass_report`, `MergeDecision`, `MergePassParams` (SGSD phase 7, décision de fusion guidée par l'oracle phase 5, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/overlap.hpp` et `src/overlap.cpp` — `generate_overlaps`, `format_overlap_report`, `RegionOverlap`, `OverlapParams` (SGSD phase 8, dilatation `geometry::inset_path_set` recadrée dans `MergeCandidate::merged_region`, aucune nouvelle primitive géométrique bas niveau, 2026-08-13, § ci-dessus).
- `libs/satin_planning/include/openstitch/satin_planning/region_routing.hpp` et `src/region_routing.cpp` — `route_regions`, `format_region_routing_report`, `RoutedRegion`, `RegionRoutingReport` (SGSD phase 9, pont vers `stitch_generation::route_columns` existant, aucune logique de routage réimplémentée, 2026-08-13, § ci-dessus).
- Tests : `tests/unit/satin_planning/test_branch_pairing.cpp`, `tests/unit/satin_planning/test_region_split.cpp`,

`tests/unit/satin_planning/test_region_satinability.cpp`, `tests/unit/satin_planning/test_region_oracle.cpp`, `tests/unit/satin_planning/test_beam_search.cpp`, `tests/unit/satin_planning/test_merge_pass.cpp`, `tests/unit/satin_planning/test_overlap.cpp`, `tests/unit/satin_planning/test_region_routing.cpp`.

- `apps/cli/main.cpp` — `run_sgsd_debug`, sous-commande `sgsd-debug --shape <forme> [--pixel-size mm] [--beam-width n]` (pipeline SGSD complet 1-9 + comparaison chiffrée contre l'appel direct à `build_satin_columns`, 2026-08-13, § ci-dessus).
- `libs/satin_planning/src/region_split.cpp` — `probe_point` corrigée (centerline de l'arc médian plutôt que corde droite entre extrémités, défaut réel trouvé sur `notch` via `sgsd-debug`, 2026-08-13, § ci-dessus).
- Tests : `tests/unit/satin_planning/test_region_split.cpp` (« notch -- chemin unique sans jonction résolu malgré une forte courbure », 2026-08-13).
- `libs/autodigitize/include/openstitch/autodigitize/autodigitize.hpp` — `AutoOptions::use_sgsd_decomposition` introduit puis retiré le même jour (2026-08-14, § *Intégration production* puis *SGSD exclusif* ci-dessus) ; remplacé par l'API publique partagée `BuiltSatinSection`, `SatinBuildReport`, `build_satin_sections` (point d'entrée unique pour `auto_digitize()` et les créations satin manuelles côté desktop).
- `libs/autodigitize/src/autodigitize.cpp` — `try_sgsd_decomposition` + `BuiltSection`/`sections_from_result` privés consolidés dans l'implémentation publique de `build_satin_sections` (branchement SGSD dans le pipeline de numérisation automatique, repli intégral sur l'appel direct si SGSD ne résout rien, 2026-08-14, § ci-dessus).
- `apps/desktop/main_window.cpp` — `createSatinObject()`, `createSatinObjectWithCutLine()`, `autoConvertToSatin()` migrées vers `autodigitize::build_satin_sections` ; `setStitchType()` (cas 2) laissée inchangée (sémantique « un objet, un type » incompatible avec la décomposition, 2026-08-14, § *SGSD exclusif* ci-dessus).
- Tests : `tests/unit/autodigitize/test_autodigitize.cpp` — le test « réseau en T » a d'abord été scindé en variante `use_sgsd_decomposition = false` + nouvelle variante par défaut (correctif de silence sur refus isolé vérifié par « branche squelette localement trop large », 2026-08-14), puis la variante « désactivé » a été retirée à la suppression du bascule (garantie équivalente déjà couverte par `tests/unit/auto_satin/test_columns.cpp`).
- `libs/satin_planning/include/openstitch/satin_planning/satin_plan.hpp` + `src/satin_plan.cpp` (nouveau, 2026-08-14, § *Refonte du contrat satin* ci-dessus) — `SatinPlanConfig`, `SatinPlanRegion`, `SatinPlan`, `create_satin_plan`, `format_satin_plan` : planner récursif unifié, seul point d'appel restant vers ce module depuis `libs/autodigitize`.
- `libs/autodigitize/include/openstitch/autodigitize/autodigitize.hpp` + `src/autodigitize.cpp` — `build_satin_sections` réécrit pour déléguer entièrement à `create_satin_plan` ; `SatinBuildReport` gagne `unresolved_residual` et `aggregate_coverage`, `structural_gap` recalculé sur la somme agrégée du résidu. `AutoOptions::use_auto_satin` redocumenté comme classification « AutoChoice » (§24 du plan de refonte), distincte d'une intention satin explicitement forcée ailleurs dans l'application.
- `libs/autodigitize/CMakeLists.txt` — `openstitch::satin_coverage` ajouté en dépendance PUBLIQUE (le header expose désormais `satin_coverage::SatinCoverageReport`).
- `apps/desktop/main_window.hpp/.cpp` — nouvel avertisseur partagé `warnAboutIncompleteSatinCoverage`, appelé par `createSatinObject()`, l'outil ligne de coupe et (sous forme d'aperçu pré-création plutôt que post-création) `autoConvertToSatin()` ; `setStitchType()` (cas satin) migré vers `build_satin_sections`, conserve la plus grande section sur une décomposition multi-régions avec message explicite plutôt qu'un repli silencieux sur `rails_from_contour`.
- Tests : `tests/unit/satin_planning/test_satin_plan.cpp` (nouveau, corpus complet + régression lettre T réelle) ; `tests/unit/autodigitize/ test_autodigitize.cpp` (tests réseau en T et anneau

rendus robustes à la présence ou non d'un petit repli tatami, vérifient la couverture de bout en bout plutôt qu'un compte exact d'objets) ; `tests/unit/satin_planning/ test_region_split.cpp` (régression région87 mise à jour, seuil de reporting du résidu recalibré).

Correctif « trace d'arête sans retour arrière » (2026-08-21)

Signalement utilisateur : une région convexe (décagone ~4,8×3,7 mm) refusée pour le satin, dialogue « Satin impossible… statut du planificateur : Incomplete ». Vérifié avant de conclure quoi que ce soit (§37 : calculer les produits vectoriels aux 10 sommets confirme un polygone STRICTEMENT CONVEXE — aucun sommet reflex, donc rien à décomposer par les familles de coupe existantes) : le refus lui-même s'est avéré correct et déjà documenté (« cercle plein/forme large refusés », `limitations.md`) — une forme ronde n'a structurellement pas d'axe satin bien défini. Mais l'investigation demandée par l'utilisateur (« il reste des bugs sur le satin ») a mis au jour un défaut réel, distinct du refus lui-même.

Défaut trouvé : rejouer la géométrie exacte à travers `auto_satin::analyze_region` montrait un amincissement (Zhang-Suen) parfaitement connexe (31 pixels de squelette, deux extrémités détectées), mais `build_skeleton_graph` produisait **zéro arête** entre les deux nœuds — `estimated_length_mm` retombait à 0 (une valeur factice, jamais mesurée), et le diagnostic « Ambiguous » se fondait sur des métriques de secours plutôt que sur une vraie mesure. Cause racine, isolée en rejouant l'algorithme pixel par pixel contre la grille réelle : sur un « escalier » de squelette à 2 px de large (deux pixels valides au même pas, artefact normal de l'amincissement sur une diagonale), la marche gloutonne de `build_skeleton_graph` (aucun retour arrière) pouvait choisir le mauvais pixel, s'échouer dans une impasse quelques pas plus loin, et laisser les pixels visités marqués `used` **de façon permanente** — bloquant au passage toute tentative ultérieure depuis l'AUTRE extrémité, qui traverse le même territoire. Un défaut de la même famille que les deux défauts de `skeleton_graph.cpp` déjà corrigés lors de l'« audit satin adversarial » (voir plus haut, priorité de nœud / boucle sur l'origine), mais jamais couvert par ces correctifs-là.

Impact réel pour CETTE région : nul — recalculée correctement (`estimated_length_mm=1,75 mm`, `min_width_mm≈2,77 mm`, élongation ≈0,63, très en dessous du seuil de 2,5), elle reste honnêtement classée `Ambiguous`/refusée, exactement comme avant. **Impact réel plus large** : ce défaut peut frapper N'IMPORTE QUELLE forme, y compris une forme réellement allongée et satinable, dès que son squelette traverse un escalier de 2 px à un angle défavorable — une classification « Ambiguous » ou une arête manquante pouvaient donc être de FAUX négatifs géométriques, pas seulement affecter les formes rondes.

Corrigé : la marche devient une recherche en profondeur AVEC retour arrière explicite (pile de « frames », candidats précalculés par pixel, `used` levé et le pixel marqué « mort pour cette tentative » au retour arrière plutôt que rendu disponible sans mémoire). Un second défaut, DÉCOUVERT EN MESURANT APRÈS COUP plutôt que supposé correct (§37) : la toute première version de ce retour arrière allouait un buffer de la taille du masque À CHAQUE tentative de direction de départ (jusqu'à 8 par nœud), un coût qui semblait plausible mais s'est avéré, une fois mesuré, ne PAS être le facteur dominant — un chronométrage isolé de `analyze_region` seul, comparé par `git stash` à la version d'avant tout correctif, a montré un profil de temps STRICTEMENT IDENTIQUE avec et sans le correctif de performance, prouvant que le temps déjà long observé sur certaines formes (`t`, `y`, `cross`, `h`… jusqu'à la limite du budget wall-clock en Debug) était un comportement PRÉEXISTANT, sans rapport avec ce correctif — la même limite Debug-vs-Release déjà documentée en Phase 7 point 7, simplement plus répandue qu'établi à l'époque. Le remplacement d'allocation (compteur de génération partagé plutôt qu'un buffer réalloué) a été conservé malgré son effet mesuré nul : il reste une amélioration correcte et sans coût, pas un correctif de performance nécessaire.

Régression permanente : nouvelle forme nommée `thick_diagonal_blob` dans `auto_satin::make_shape` (géométrie exacte de la région utilisateur, volontairement non minimisée — la marche gloutonne est sensible à l'ordre exact des pixels), testée dans `tests/unit/auto_satin/test_pipeline.cpp` (vérifie `raw_graph.edges.size() >= 1` et `estimated_length_mm > 0`, au pixel_size par défaut réel de l'application — 50 µm, pas les 100 µm plus rapides utilisés par le reste de ce fichier de test).

Un troisième défaut trouvé EN VALIDANT le correctif (§37 : ne jamais supposer qu'un correctif est une amélioration nette sans faire tourner tout le corpus) — `junction_with_hole` (déjà dans le corpus de

torture, un trou près d'une confluence à 3 jonctions) passait de 4 régions acceptées/93,93 % de couverture à **0 région, 0 %**, en Debug ET en Release (donc pas l'artefact Debug habituel — vérifié explicitement par `git stash` avant de conclure). Le graphe de squelette de la région ENTIÈRE s'est avéré STRICTEMENT IDENTIQUE avant/après correctif (mêmes 6 nœuds, mêmes 6 arêtes, mêmes longueurs) — la régression ne venait donc pas d'un changement de topologie au niveau supérieur. Cause racine, isolée en lisant `region_split.cpp` : `split_region` valide chaque coupe candidate en ré-analysant le morceau isolé et en exigeant `junction_count == 0` (aucune jonction résiduelle). Avant le correctif, une jonction résiduelle AUTHENTIQUE dont la trace d'arête échouait silencieusement (même défaut que ci-dessus) se retrouvait élaguée par `prune_graph` comme nœud orphelin sans arête vivante — `junction_count` retombait alors À TORT à zéro, validant des coupes qui n'isolaient PAS réellement une branche propre. Le correctif expose donc une limitation RÉELLE déjà présente : pour cette géométrie précise, aucune coupe candidate n'isole en vérité un morceau sans jonction résiduelle — l'ancienne « réussite » (93,93 % de couverture) était construite sur une vérification cassée, pas sur une décomposition réellement propre ; le nouveau résultat (`Incomplete`/ `Impossible` honnête, jamais un faux `Complete`) est donc le comportement CORRECT, même s'il est numériquement moins bon. Non corrigé (§33 : pas de patch opportuniste du générateur/de la vérification de coupe sous la pression d'une seule fixture) — `junction_with_hole` ajoutée à `shapes_hitting_known_limitation()` (`test_torture_corpus.cpp`), pour une troisième raison distincte de `two_holes` (aucune famille de coupe applicable) et des formes à branches nombreuses (limitation de qualité de la génération de candidats).

Non-régression : `test_auto_satin` (2235 assertions), `test_satin_ planning` et `test_autodigitize` complets, Debug ET Release.

Portée de coupe automatique bornée au squelette voisin, pas au rayon local (2026-08-22)

Suite directe du signalement utilisateur ci-dessus (« il reste des bugs sur le satin, certaines régions restent non satinables ») — mission : pousser la promesse centrale de `satin_planning::create_satin_plan` (toute région non dégénérée devrait être satinable, via décomposition récursive) au-delà de la liste de limitations connues existante, en cherchant activement d'autres défauts réels plutôt qu'en acceptant cette liste comme définitive (§37).

Défaut trouvé (`comb`, fixture du corpus de torture) : `region_split.cpp` détache une branche à une jonction en découpant `piece` avec `geometry::cut_path_set(piece, a, b, cut_width)` — une ligne de coupe qui s'étend délibérément jusqu'à la diagonale de la boîte englobante de TOUTE la région (+1 mm de marge), conçue à l'origine pour l'outil MANUEL de ligne de coupe (l'utilisateur vise la jonction, pas les bords exacts). Sur `comb` (un tronc + 6 dents parallèles partageant la même plage en Y), cette ligne « infinie » traverse TOUJOURS les 6 dents à la fois, quelle que soit la distance de coupe testée le long d'une dent — `generate_cut_ candidates` rejetait donc INCONDITIONNELLEMENT chaque candidat (« la coupe n'a pas produit exactement 2 morceaux »), et `comb` restait bloqué à `Impossible`/0 % de couverture.

Premier correctif, insuffisant : nouvelle primitive `geometry:: cut_path_set_bounded(region, a, b, reach_um, cut_width)` (`cut.cpp` factorisé en une fonction commune `cut_band`, `reach` devenant un paramètre au lieu d'être calculé en interne), avec `reach_um` déterminé au point de coupe par le rayon local du squelette (transformée en distance) + une marge fixe. Corrige bien `comb` (0 → 6/7 branches isolées, 93,64 % de couverture) mais casse silencieusement DEUX autres cas, chacun trouvé en faisant tourner le corpus complet plutôt qu'en supposant la correction suffisante (§37) :

- **Lettre réelle avec contre-forme** (fixture `tests/unit/satin_planning/ test_region_split.cpp`, coordonnées exportées d'une vraie région utilisateur) : le rayon local (plus proche bord dans N'IMPORTE quelle direction) mesurait la distance jusqu'au TROU plutôt que jusqu'au vrai bord EXTÉRIEUR dès que le trou était plus proche — la coupe s'arrêtait donc au bord du trou, laissant non coupée la matière qui continue au-delà, et la pièce restait connexe malgré la coupe (`1 == 4` régions au lieu de `4`, à TOUTES les distances testées).
- **`t`** (le cas le plus simple du corpus — une seule jonction, un seul barreau) : la coupe réussissait bien géométriquement, mais laissait un sommet reflex parasite sur le morceau « reste » juste au-delà du

renflement de la confluence (le bras large de la barre, non atteint par une portée trop courte) — suffisant pour franchir le seuil de 15° de `ConcavityCutParams::min_reflex_turn_deg` et déclencher À TORT une redécomposition par `try_concavity_decomposition`, fragmentant le morceau « reste » en deux pièces à faible couverture (~70 % chacune) au lieu d'un seul morceau à 98 %. `create_satin_plan(shape("t"))` régressait de couverture agrégée >90 % avec adjacence peuplée, à 72 % sans aucune adjacence.

Root cause commune, isolée en comparant explicitement les deux échecs (§37 : bissection empirique de la portée plutôt qu'un raisonnement géométrique non vérifié — un premier raisonnement « la portée en excès est sans effet, `Clipper2::Difference` ne fait qu'écrêter à la matière réelle » s'est avéré FAUX, contredit en forçant `reach=100000` sur `t`, qui reproduit EXACTEMENT le résultat historique (`179,640 mm²`) là où `reach=3000` donnait `179,940 mm²` avec le sommet reflex parasite) : un simple rayon NE PEUT PAS être à la fois assez court pour ne jamais atteindre une branche SANS RAPPORT (`comb`) et assez long pour toujours atteindre le vrai bord extérieur au-delà d'un renflement de confluence ou d'un trou (`t`, la lettre) — ces deux exigences sont directement contradictoires pour une distance scalaire unique, quelle que soit la formule (rayon local, rayon vers le bord extérieur seul, marge fixe ou proportionnelle — chacune testée et chacune cassant l'un des deux cas).

Correctif retenu : la portée redevient généreuse PAR DÉFAUT (même calcul que l'ancien `cut_path_set` non borné — diagonale de la boîte englobante de la pièce), mais plafonnée par la distance au squelette d'une VRAIE branche voisine (`distance_to_other_branches`, nouvelle fonction dans `region_split.cpp`) — en excluant explicitement les arêtes incidentes à LA MÊME jonction que celle traitée. Cette exclusion est la clé : sur `t`, les arêtes de la barre partagent la jonction du barreau détaché — jamais des « voisines » à éviter, seulement la matière naturelle de la confluence — donc rien ne plafonne, et la portée généreuse s'applique sans risque, EXACTEMENT comme avant tout ce correctif. Sur `comb`, chaque dent voisine a sa PROPRE jonction, à quelques milliers de µm — la distance plafonne donc correctement avant d'atteindre la dent suivante, exactement comme le premier correctif (rayon local) le faisait, mais sans son défaut de sous-estimation.

Un second plancher reste nécessaire en complément (retrouvé en revalidant le cas de la lettre avec ce nouveau plafond seul : régression identique à avant, `siblingDistance` plafonnant à ~3400 µm quand ~5500-7000 µm étaient nécessaires pour atteindre le vrai bord extérieur au-delà du trou) — `ray_boundary_distance(piece, point, direction)`, un rayon simple vers le bord EXTÉRIEUR de la pièce (delibérément PAS les trous, qui ne représentent jamais une branche sans rapport). La portée finale combine les trois signaux : `reach = min(diagonale_généreuse, max(rayon_extérieur, distance_branche_voisine) + marge)` quand une branche voisine existe, `diagonale_généreuse` telle quelle sinon. Validé exact (byte-identique aux valeurs historiques `179,64 mm²/98,0-99,8 %` de couverture sur `t`, `4/4` régions isolées sur la lettre) tout en conservant (et même améliorant encore, à 96,17 %) le gain sur `comb` (`Incomplete`, jamais `Complete` — cf. `test_torture_corpus.cpp`, qui vérifie explicitement que ce statut reste honnête plutôt que de forcer un faux succès).

État mesuré du reste des limitations connues après ce correctif (§37 : revalidé plutôt que supposé stable) — comparaison `git checkout` ciblée entre avant/après sur les 6 fixtures de `shapes_hitting_known_performance_limit()`, avec `prod_config()` (les mêmes paramètres que les tests dédiés) : `star5` (6→9 régions, 92,8 %→91,1 % — décomposition plus complète, couverture globale comparable), `asymmetric_star` (4→9 régions, 84,1 %→92,0 % — amélioration nette), `comb` (0→5 régions, 0 %→96,17 %, déjà détaillé ci-dessus). `E`/`multi_neck`/ `deep_channel` restent, AVANT et APRÈS ce correctif, à 0 région acceptée/ 0 % de couverture — confirmé identique par la même comparaison ciblée, pas seulement supposé : ces trois fixtures ne partagent donc PAS la cause corrigée ici, cohérent avec leur documentation existante (limitation de PERFORMANCE — génération de candidats coûteuse par itération — plutôt que de portée de coupe).

Propagation : `CutCandidate::reach_um` (nouveau champ) porte la portée choisie jusqu'aux TROIS sites consommateurs de a/b (`generate_cut_ candidates` qui la calcule, `OracleGuidedSelector::operator()` dans `beam_search.cpp` qui doit ré-évaluer chaque candidat avec la MÊME portée que celle qui a servi à le construire, et l'application finale de la coupe choisie dans `split_region`) — une incohérence entre ces trois sites reproduirait exactement le défaut initial de manière

plus subtile (portée correcte à la génération, mais différente à l'évaluation ou à l'application).

Limite assumée : `libs/satin_planning/src/concavity_cuts.cpp` (famille de coupe ancrée sur les sommets reflex du CONTOUR, sans graphe de squelette) appelle toujours l'ancien `geometry::cut_path_set` non borné — en principe exposée au même défaut que `comb` si une forme cumule plusieurs concavités proches partageant la même plage perpendiculaire. Non corrigé ici : `star5`/`E`/`multi_neck`/`deep_channel` (`shapes_hitting_known_performance_ limit()`) restent des limitations DOCUMENTÉES de PERFORMANCE (génération de candidats de décomposition coûteuse par itération, avant épuisement du budget) plutôt que de correction géométrique — aucune preuve empirique (bissection, `git stash`) qu'elles partagent la root cause corrigée ici, et §33 de la mission interdit un patch opportuniste du générateur de candidats sous la pression d'une fixture non confirmée. À réexaminer si un signalement réel implique concrètement `concavity_cuts.cpp`.

Non-régression : `test_satin_planning` (90 cas de test, 3825 assertions en Release, 3819 en Debug) et `test_geometry` (58 cas, 291 assertions) complets, Debug ET Release.

Appariement des nœuds Continuation dans `decompose_into_paths` (2026-08-28)

Suite directe de la mesure ci-dessus : `comb` franchit désormais le seuil de couverture (96,17 % > 95 %) mais reste `Incomplete` pour une raison DIFFÉRENTE — le critère « aucun trou local disproportionné » (`max_gap_radius_mm=0,52 mm`, seuil `0,50 mm`) — à cause d'une dent qui reste en permanence fusionnée au tronc. Root cause isolée en lisant `graph_cleanup.cpp`/`branch_pairing.cpp` plutôt que supposée : le court renflement de tronc à gauche de cette dent (~4 mm, quasi carré) a un squelette dont la LONGUEUR d'arête est quasi nulle (une forme carrée n'a pratiquement pas d'axe médian) malgré une aire non négligeable — `prune_graph` l'élague donc comme « branche terminale courte » (comportement voulu et documenté, pas un défaut en soi : la longueur de squelette reste le bon signal pour la vraie parasite). Cet élagage fait retomber le degré de la jonction voisine à 2, et `prune_graph` la reclasse alors en `Continuation` (comportement également documenté : « les jonctions redevenues de degré 2 deviennent des continuations »).

Défaut réel, distinct de l'élagage lui-même : `decompose_into_paths` (`branch_pairing.cpp`) ne traite QUE les nœuds `Junction` — il appelle `pair_branches_at_junction` pour chacun d'eux et peuple une table `(nœud, arête) -> arête partenaire` qui permet à la marche de continuer tout droit à travers ce nœud. Pour un nœud `Continuation` (degré 2 par construction, puisque c'est la SEULE façon dont ce type existe — `build_skeleton_graph` ne crée jamais de nœud pour un pixel de simple continuation), cette table restait vide : la marche s'arrêtait donc À TORT à chaque nœud `Continuation`, coupant un unique chemin physique continu en DEUX `SatinPath` non reliés — sans qu'aucune coupe réelle ne sépare jamais la matière entre eux (seuls les nœuds `Junction` produisent un événement de détachement dans `region_split.cpp`). `split_region` assignait alors les DEUX chemins au MÊME morceau non coupé (un seul accepté, l'autre marqué non isolé), et la dent restait fusionnée en permanence — pas parce que la coupe échouait, mais parce qu'elle n'était même jamais TENTÉE.

Corrigé : `decompose_into_paths` apparie aussi les deux arêtes incidentes de chaque nœud `Continuation` (appariement trivial — un seul choix possible, aucun coût à comparer, contrairement à `pair_branches_at_junction` qui doit choisir entre plusieurs paires possibles à une vraie jonction) — fidèle au NOM du type `Continuation`, jamais traité comme un point de coupe. Corrige `comb` sans toucher à `graph_cleanup.cpp` (l'élagage et sa reclassification par degré restent inchangés, comportement voulu conservé) — la portée du correctif reste localisée à la façon dont `decompose_into_paths` INTERPRÈTE un nœud déjà démoté, pas à la décision de démotion elle-même. Non-régression : `test_satin_planning` (90 cas, 3824 assertions Release, 3817 Debug), `test_auto_satin` (54 cas, 2235 assertions, inchangé — ce module ne dépend pas de `branch_pairing.cpp`) et `test_autodigitize` (11 cas, 87 assertions) complets.

Repli de portée sur le bord extérieur réel (2026-08-28)

Troisième et dernier correctif de cette même mission : `comb` était désormais `Incomplete` pour une raison RÉSIDUELLE (`max_gap_radius_mm= 0,52 mm`, seuil `0,50 mm`) après les deux correctifs précédents. Investigation élargie au reste du corpus de limitations connues (`E`/`multi_neck`/`deep_channel`) pour comprendre si elles partagent une cause commune : `E` (lettre à 3 branches du même côté d'un montant) révèle un TROISIÈME défaut distinct dans la formule de portée.

Défaut trouvé : le squelette RÉEL de `E` (vérifié dans le graphe AVANT tout élagage — pas un artefact de `prune_graph`) n'a qu'UNE seule jonction à 3 branches : les montants haut et bas se combinent chacun avec leur portion de tronc adjacente en un seul arc continu de ~32 mm, et rejoignent la barre du milieu au même point. Détacher la barre du milieu exige une coupe VERTICALE (perpendiculaire à sa tangente horizontale) — or les deux autres branches, elles, s'étendent PRÉCISÉMENT dans cette même direction verticale, partageant la MÊME jonction que la barre détachée. `distance_to_other_branches` (corrigé le 2026-08-22 pour `comb`/`t`) exclut délibérément les arêtes incidentes à LA MÊME jonction — correct pour `t` (le renflement à traverser est la matière naturelle de la confluence), mais FAUX ici : ces deux arêtes sont des branches à part entière, jamais un simple renflement, et la portée généreuse (utilisée faute de toute AUTRE jonction dans tout le graphe) les tranchait toutes les deux (« 4 morceaux » au lieu de 2) à chaque distance testée au-delà du tronc.

`t` et `E` partagent exactement le même signal topologique observable (une seule jonction dans tout le graphe, donc `distance_to_other_branches` toujours infinie) tout en exigeant des stratégies de portée OPPOSÉES — un rayon simple vers le bord extérieur (`ray_boundary_distance`) aurait suffi ici (le vrai bord extérieur, au-delà de la barre du milieu, se trouve à quelques millimètres seulement dès qu'on s'éloigne du tronc), mais ce même rayon est ce qui, à l'inverse, sous-dimensionnait `t` avant le correctif du 2026-08-22.

Corrigé : plutôt que de choisir UNE formule de portée, `try_distance` tente désormais la portée courante (généreuse/plafonnée par `distance_to_other_branches`) EN PREMIER, et seulement si elle est REJETÉE (mauvais nombre de morceaux ou jonction résiduelle) retente, pour la MÊME distance, avec `ray_boundary_distance` seul (strictement plus court, sinon la seconde tentative n'a aucune chance de réussir là où la première a déjà échoué). Ne change RIEN pour `t` (sa portée généreuse réussit dès la première tentative, jamais de repli déclenché) ni pour `comb` (son plafond par squelette voisin réussit déjà). Pour `comb`, ce filet de sécurité supplémentaire résout la dernière dent bloquée : **`comb` atteint désormais `Complete`** (96,52 % brut / 97,16 % cœur, cinq régions acceptées) — retirée de `shapes_hitting_known_performance_limit()`, son test dédié réécrit en test de succès.

Limite assumée, partielle sur `E` : le repli débloque bien la génération de candidats (7 candidats géométriquement valides trouvés, contre zéro avant), mais `E` reste `Incomplete` : `OracleGuidedSelector` rejette encore chacun de ces candidats (`build_succeeded=false`).

Root cause identifiée (2026-08-28, poursuite de la même investigation) : la pièce isolée à `d=2100` µm (la plus favorable des 7, un rectangle PROPRE de 10,89 × 4 mm — vérifié nœud par nœud, aucun artefact de coupe) est classée `Ambiguous` par `evaluate_satinability` avec le refus « direction ambiguë (forme quasi circulaire) ». Cause : l'élongation y est mesurée via la longueur du SQUELETTE (l'axe médian, qui exclut les deux embouts de largeur/2 à chaque extrémité) divisée par la largeur — soit ≈(10890-4000)/4000 = 1,72 — plutôt que le ratio VISUEL longueur/largeur du rectangle lui-même (2,72 : 1, nettement au-dessus du seuil de satinabilité de 2,5). Les 6 autres candidats, tous plus courts (branche décroissante avec `d`), échouent nécessairement au même critère. Root cause CONFIRMÉE distincte de tout ce qui précède dans cette section — un défaut de calibration de la métrique d'élongation dans `evaluate_satinability` (`libs/auto_satin`), pas de la portée de coupe (`libs/satin_planning`). Non corrigé ici : modifier ce seuil ou cette métrique reclasse potentiellement CHAQUE forme du corpus (portée bien plus large qu'un correctif localisé au générateur de candidats de coupe), à n'entreprendre qu'avec une preuve empirique sur plusieurs fixtures réelles, jamais sous la pression d'une seule (§33). `E`/`multi_neck`/`deep_channel` restent donc des limitations connues, désormais confirmées ne PAS relever de la portée de coupe.

Le test dédié à `comb` reçoit un budget d'oracle explicitement généreux (`max_oracle_evaluations=1000`) afin de vérifier la qualité atteignable sans confondre cette vérification avec le plafond de production.

Non-régression : `test_satin_planning` (90 cas, 3831 assertions Release, 3813 Debug) complet.

Décomposition topologique : jonctions de première classe, candidats multiples, fonction de coût (2026-08-30)

Refonte demandée explicitement par l'utilisateur : remplacer l'hypothèse « une branche de squelette = une colonne satin » par un pipeline « forme → analyse topologique → candidats de décomposition multiples → fonction de coût explicite → colonnes ». Analyse préalable (exigée avant tout code) : la majeure partie de cette architecture existe déjà sous le nom **SGSD** (section « Décomposition guidée par squelette » plus haut, 2026-08-13) — ce travail est donc une extension ciblée de SGSD, jamais un second moteur de décomposition parallèle, déployée en 6 étapes incrémentales (degré ≤2 → T → Y → X → trident/multi-jonctions → boucles). Ce qui suit couvre les étapes 1 à 3.

Étape 1 — `SatinColumn`/`satin_column_view` (`libs/auto_satin`) : façade en LECTURE SEULE sur `SatinColumnGeometry`/`ParametricSatinObject`, qui élimine la logique dupliquée « `parametric_columns` si non vide sinon `columns` » déjà présente indépendamment dans `region_oracle.cpp`, `satin_sections.cpp` et `region_routing.cpp`. `start_width`/`end_width` dérivés de la longueur du premier/dernier barreau RÉEL (jamais fabriqués). Vérifié avant implémentation : `stitch_generation::fill_satin_columns` ne dépend d'aucun type `SatinColumnGeometry`/`ParametricSatinObject` — zéro changement en aval. `SatinColumn` reste une projection pure, jamais consommée par `build_satin_columns` lui-même.

Étape 2 — `topology.hpp` (`libs/satin_planning`) : `RegionClass` (Régulier/Singulier), `JunctionType` (T/Y/X/AcuteFork/Merge/LoopConnection/ Complex), `SatinJunction`, et surtout `enumerate_decomposition_candidates` — généralise `decompose_into_paths` en exposant TOUTES les variantes dignes d'être construites+mesurées (jusqu'à `max_candidates_per_junction`, défaut 3 : chaque paire de `JunctionPairingReport::candidates`, déjà calculée mais jusqu'ici jetée sauf l'argmin, plus une variante finale « aucun tronc, toutes les branches indépendantes », jamais générée avant cette refonte). `classify_junction` (degré 3 : écart dominant entre les deux meilleurs coûts de continuité → `T`, sinon coûts proches par symétrie → `Y` ; degré 4 : partition en 2 paires disjointes comparée à la paire naturelle → `X` ou `Complex`) sert de signal STRUCTUREL, réutilisé comme porte de déclenchement à l'étape 3 (voir plus bas). Non-régression propre à cette étape : `t`/`cross`/`rectangle` classés correctement, et le candidat 0 d'`enumerate_decomposition_candidates` reproduit TOUJOURS l'argmin de `decompose_into_paths` (auto-cohérence, jamais un second calcul divergent pour le même cas).

Étape 3 — `decomposition_cost.hpp` (`libs/satin_planning`) : 8 termes de coût nommés (`coverage`/`continuity`/`angle`/`width`/`overlap`/ `density`/`rail_quality`/`routing`, poids explicites dans `DecompositionCostWeights`), dont seuls `coverage_cost` et `continuity_cost` sont RÉELLEMENT mesurés dans cette première intégration — les 6 autres restent à 0,0, documentés comme tels plutôt que simulés. `evaluate_decomposition_cost` construit RÉELLEMENT chaque candidat (`split_region` puis `evaluate_decomposition_generation`, mêmes fonctions que le reste de SGSD) plutôt que d'utiliser un proxy pré-construction seul.

Défaut trouvé en écrivant cette fonction : première version calculant `coverage_cost = 1 - aggregate_coverage_ratio` produisait des valeurs absurdes (jusqu'à -96,68). Cause : `aggregate_coverage_ratio` (`region_oracle.cpp`) est un POURCENTAGE 0-100, pas une fraction [0,1]. Corrigé (`/100.0`), avec un commentaire explicite contre la même confusion.

Preuve que le mécanisme sait faire gagner autre chose que l'argmin : la fixture `y` existante (bras à 45°/135°/270°) a déjà un appariement dominant sous la nouvelle fonction de coût — ne démontre rien de neuf. Nouvelle fixture `y_symmetric` (`shapes.cpp`, 3 bras strictement identiques à 120°) construite spécifiquement pour ce test : avec des `CutCandidateParams` par défaut, les 3 appariements ont un coût de continuité quasi identique par symétrie, et le candidat runner-up (continuité 0,433, moins bon que l'argmin à 0,296) gagne réellement sur le coût total (0,566 contre 0,674) grâce à une meilleure couverture mesurée (`test_decomposition_cost.cpp`).

Branchement production (`decompose_and_recurse`, `satin_plan.cpp`, `SatinPlanConfig::use_topology_multi_candidate`) : pour une région à UNE seule jonction de degré

EXACTEMENT 3, compare réellement 2-3 décompositions candidates au lieu de l'argmin seul. Deux défauts réels trouvés en l'intégrant :

Régression de budget (`polygonal_cut_fixture` : `Incomplete`/ 53,30 % → `Impossible`/38,70 %, `SearchBudgetExceeded` à 14,4 s/10 s). Premier essai : déclencher la comparaison coûteuse via un simple écart numérique entre les deux meilleurs coûts de continuité (`< 0,15`) — cet écart se produit aussi INCIDEMMENT sur des jonctions degré-3 issues des coupes concavité/polygonales de cette fixture, sans rapport avec un vrai Y symétrique, déclenchant la comparaison bien plus souvent que nécessaire. Corrigé en remplaçant ce seuil par la classification structurelle de l'étape 2 : `classify_junction(...).type == JunctionType::Y` — `polygonal_cut_fixture` produit alors un résultat identique avec/sans le nouveau chemin.

`y_symmetric` bout en bout ne gagne PAS de couverture supplémentaire — contrairement à la mesure isolée ci-dessus. Cause : `decompose_ and_recurse` enrichit TOUJOURS ses `CutCandidateParams` des coupes « séparateur de jonction » du moteur Legacy (§14, sommet reflex RÉEL du contour, calculées indépendamment du choix topologique) avant même de comparer les candidats — avec ces coupes réelles, les deux décompositions candidates de `y_symmetric` convergent vers un résultat final identique (couverture=0,136/continuité=0,296 pour l'argmin contre couverture=0,081/continuité=0,433 pour le runner-up, mais 0,568 contre 0,595 au total → l'argmin regagne). Le mécanisme est donc vérifié correct et sans régression, mais pas encore démontré comme un gain de couverture bout en bout sur le corpus actuel : attendu plutôt des étapes 4/5 (croix/trident), où le choix topologique influence la construction bien au-delà des seules coupes de jonction. `use_topology_ multi_candidate` reste activé par défaut : infrastructure correcte, sans coût de qualité mesuré, posant les bases des étapes suivantes.

Fausse alerte écartée en validant bout en bout : une exécution complète de `test_satin_planning` a montré `y_symmetric` avec une couverture DIFFÉRENTE selon que le drapeau soit actif ou non (0,904 contre 0,928) — en apparence une régression réelle. Vérifié AVANT de conclure (§37) : 5 appels isolés répétés dans le même process, puis un fort préchauffage (24 plans lourds calculés juste avant, dans le même process) donnent tous les deux `oracle_evaluations=16` et une couverture IDENTIQUE à 6 décimales — ni contamination inter-tests ni non-déterminisme du mécanisme lui-même. La seule exécution déviante coïncidait avec une compilation Debug lancée en tâche de fond CONCURRENTE sur la même machine : le filet de sécurité `PlanningBudgetState::max_planning_ wall_clock_ms` (§18, déjà réel — cf. `SearchBudgetExceeded` ci-dessus) a biaisé la mesure sous contention CPU réelle, un comportement attendu du budget existant, pas un défaut de cette étape. Retenu comme rappel : même pattern déjà en place pour `comb`/« réseau en T » (budget wall-clock explicitement généreux dans les tests dédiés) — un test qui mesure une couverture bout en bout doit soit tolérer une marge généreuse, soit préférer une preuve indépendante du temps réel (ex : `oracle_evaluations`, un compte, jamais soumis à la contention CPU).

Non-régression : `test_satin_planning` (113 cas, 3968 assertions Release, 111 cas/3956 assertions Debug) et `test_auto_satin` (58 cas, 2505 assertions Release) complets.

Étape 4 (croix) : infrastructure topologique posée, limite architecturale réelle trouvée avant tout branchement (2026-08-30)

Suite directe de l'étape 3 ci-dessus : `decompose_into_paths` documente depuis l'étape 2 que, pour une jonction de degré ≥4, « les appariements multiples (ex : A↔C et B↔D simultanément) sont laissés à une phase ultérieure : ils peuvent produire des colonnes qui se croisent géométriquement au centre de la jonction, ce que la phase 1 ne vérifie pas encore ». Cette étape visait à lever cette limitation pour une croix (4 branches, degré 4, classée `JunctionType::X`).

Infrastructure posée (correcte et testée, sans changement de comportement historique) : `JunctionPairingReport::secondary_pair` et `JunctionOverride::forced_secondary_pair` (`branch_pairing.hpp`) — généralisent le mécanisme d'override de l'étape 2 pour représenter DEUX paires simultanées traversant la même jonction, `secondary_pair` restant TOUJOURS vide pour tout appelant historique (T/Y, degré 3) et pour le calcul naturel de `pair_branches_at_junction` lui-même (qui ne retient toujours qu'une seule paire — seul un override explicite peut le renseigner). `decompose_into_paths` traite

`secondary_pair` exactement comme `selected_pair` dans sa carte de continuation (la marche produit alors 2 `SatinPath` au lieu de 3, un par paire traversante). `enumerate_ decomposition_candidates` génère désormais, pour un nœud de degré EXACTEMENT 4, une variante supplémentaire « deux traversées simultanées » (la meilleure des 3 partitions possibles en 2 paires disjointes, réutilisant le même calcul que `classify_junction` — factorisé en `disjoint_pair_partitions`/`best_ dual_through_partition`), en réduisant d'autant le nombre de variantes « un seul trunk » testées pour rester dans le même budget par défaut (3 candidats : 1 trunk simple + 1 deux-traversées + 1 aucun trunk). `compute_continuity_cost` (`decomposition_cost.cpp`) compte `secondary_pair`, quand présent, comme un terme SUPPLÉMENTAIRE dans sa moyenne, jamais à la place du premier — n'affecte aucune jonction T/Y existante.

Défaut réel trouvé en tentant de brancher cette variante en production (avant d'écrire le moindre test de non-régression prétendant que « la croix s'améliore ») : `split_region` (`region_split.cpp`) dérive la TOTALITÉ de ses événements de coupe de `JunctionPairingReport::detached` — une coupe par arête détachée, amputée du « reste ». Un candidat « deux traversées simultanées » ne détache RIEN par construction (les 4 arêtes appartiennent à l'une des deux paires retenues) : `split_region` ne génère alors AUCUNE coupe, et toute la région est traitée comme UNE seule pièce indivise que `try_local_satin` construit très mal (un « + » entier comme un seul rail). Mesuré sur `cross` : le candidat « deux traversées » obtient couverture=0,094/continuité=0/total=0,189, contre couverture=0,021/continuité=0/total=0,043 pour le trunk simple — un facteur 4, pas un cas limite. Ce n'est pas un défaut de réglage mais exactement la limite prédite par le commentaire cité en ouverture : réaliser cette variante correctement exige que chaque paire devienne sa PROPRE colonne construite sur la région ENTIÈRE non coupée, avec un recouvrement DÉLIBÉRÉ au centre (même principe que `overlap.hpp`, mais comme stratégie de CONSTRUCTION plutôt que de correction a posteriori) — une nouvelle `RailConstructionMethod`, hors de portée d'une étape purement topologique.

Décision (§33 : jamais de patch opportuniste sous la pression d'une seule fixture) : la variante « deux traversées simultanées » N'EST PAS branchée dans `decompose_and_recurse` — brancher une comparaison qui ne peut aujourd'hui QUE perdre serait un coût de budget pur sans aucun bénéfice possible, l'inverse de la discipline « construire puis mesurer » de ce fichier. `cross` continue de passer par le chemin historique (`decompose_into_paths`, argmin de continuité), inchangé : toujours `Complete` en Release, marginal en Debug (déjà documenté, cf. le test « budget généreux » plus haut). Un test dédié (`evaluate_decomposition_cost : cross -- deux traversées simultanées perd`, `test_decomposition_cost.cpp`) fige ce résultat comme garde-fou explicite — si le trunk simple cessait de gagner nettement sans qu'une vraie méthode de construction par recouvrement n'ait été ajoutée, ce serait un signal à examiner, pas une amélioration à accepter telle quelle.

Reporté : la réalisation réelle de « deux traversées simultanées » (construction par recouvrement plutôt que par coupe disjointe) est reportée aux étapes 5/6 du plan de refonte, qui anticipent déjà ce même besoin de recouvrement délibéré pour les régions singulières (trident/boucles).

Non-régression : `test_satin_planning` (114 cas, 3999 assertions Release, 3989 assertions Debug) complet.

Étape 5 (trident/multi-jonctions) : le « résidu » recouvre en réalité deux problèmes distincts (2026-08-30)

Avant toute implémentation (§21/§22 du brief de refonte : analyser d'abord), mesure de la géométrie RÉELLE du résidu non couvert (`SatinPlan::unresolved_residual`, dérivé de la mesure finale de couverture — jamais une supposition) sur les formes ciblées par cette étape :

Forme	Statut	Résidu	Répartition
`star5`	Incomplete	6,5 %	39 éclats, le plus gros 5,3 mm²
`comb`	Complete	3,5 %	12 éclats, le plus gros 2,0 mm²
`trident`	Complete	4,4 %	9 éclats, le plus gros 4,8 mm²
`E`	Incomplete	**100 %**	1 pièce — la forme entière, rien construit
`multi_neck`	Incomplete	**100 %**	idem
`two_holes`	Impossible	**100 %**	idem

Deux problèmes différents, pas un seul. Sur `star5`/`comb`/`trident`, le résidu est réel mais MARGINAL (3-6,5 % de la surface, jamais une seule pièce mais des dizaines de petits éclats ANGULAIRES) — chaque jonction est déjà résolue en plusieurs colonnes qui, individuellement, couvrent bien leur propre branche ; il ne manque que la matière triangulaire au voisinage immédiat de la confluence, là où 3 branches ou plus se rejoignent. Sur `E`/`multi_neck`/`two_holes`, RIEN n'est construit du tout : le planificateur tente de faire tenir la forme ENTIÈRE (par exemple les 2 montants + la barre du milieu de `E`) sur une seule interprétation d'axe, ce qui échoue catastrophiquement (13 % de couverture brute mesurée sur `E`, `sgsd-debug --shape E` : aucune coupe de la barre du milieu n'est acceptée par le sélecteur, malgré des candidats géométriquement valides). La correspondance de contour (DTW) anticipée par le plan de refonte cible spécifiquement ce SECOND cas — un travail nettement plus risqué et encore non résolu par cette étape (voir « Reporté » plus bas).

Corrigé ici (premier cas — les éclats de jonction) : `generate_overlaps` (phase 8 SGSD) et sa consommation dans `create_satin_plan` (`extend_columns_into_known_overlaps`) ne couvraient que les paires de régions ISSUES D'UNE MÊME COUPE (`RegionSplitReport::merge_candidates`) — une jonction à 3+ branches laisse un résidu triangulaire entre des « cousins » du même nœud de jonction, jamais adjacents par une seule coupe binaire (l'arbre de décomposition est binaire, la jonction ne l'est pas). `extend_toward` (déjà le bloc de construction du recouvrement pairwise — dilate de `overlap_distance` puis recadre dans `bounds`) est désormais exposée publiquement et réutilisée avec `bounds` = la FORME SOURCE ENTIÈRE plutôt que la seule géométrie d'avant-coupe d'UN voisin — strictement plus généreux (`source` ⊇ `merged_region` d'une paire), jamais une invention de matière (le recadrage Clipper2 garantit de rester dans le tissu réel). Appliqué à CHAQUE région feuille, après le recouvrement pairwise existant (chaque région part de son meilleur état connu), avec la même règle « jamais de dégradation silencieuse » (repli sur l'original si la reconstruction échoue ou couvre moins bien).

Gain mesuré, sans régression de statut sur le corpus complet (Debug ET Release) : `trident` 95,65 %→98,08 %, `star5` 93,50 %→95,79 %, `comb` 96,52 %→97,33 %. `t`/`y`/`cross`/`h` (déjà `Complete`) s'améliorent aussi marginalement, sans effet sur leur statut. `polygonal_cut_fixture` (limitation connue, sans rapport avec une jonction à haut degré) reste inchangée, comme attendu.

Reporté : le second problème (`E`/`multi_neck`/`two_holes`, refus total) reste entier — c'est lui la vraie justification de la correspondance de contour prévue par le plan de refonte, pas les éclats corrigés ici. Root cause partielle déjà isolée (`sgsd-debug`) : sur `E`, la coupe qui isole la barre du milieu est rejetée à toute distance courte (jonction résiduelle) et, une fois géométriquement valide à distance plus longue, rejetée quand même par le sélecteur — la pièce « reste » qui en résulterait (le reste de la lettre, toujours pas une géométrie simple) échoue elle aussi localement. Nécessite une méthode de construction qui ne suppose JAMAIS un axe unique sur la région entière — hors périmètre de cette étape, prototypage isolé requis avant tout branchement (§ risque #1 du plan de refonte).

Non-régression : `test_satin_planning` (116 cas, 4024 assertions Release, 4010 assertions Debug) complet.

Root cause précise du refus total sur « E » : un coude à 90° fait exploser la largeur mesurée, pas une histoire de jonction (2026-08-30)

Poursuite directe de l'investigation ci-dessus. Hypothèse testée isolément (§37 : mesurer, jamais supposer) avant d'aller plus loin : et si le vrai blocage n'était PAS la coupe de la barre du milieu, mais le tronc lui-même une fois isolé ? Construit directement la forme en «] » (montant + barre haute + barre basse de `E`, SANS la barre du milieu) et appelé `build_satin_columns` dessus, indépendamment de toute coupe ou de tout planificateur.

Résultat sans appel : refus total, même sans aucune jonction (`jonctions_squelette=0` — un simple ruban courbé n'a pas de nœud de squelette). `compute_column_stations` rapporte : `colonne refusee : trou de 2471 um entre stations axe #23 et #28 (largeur superieure a max_width)`. Ce n'est donc PAS un problème de jonction, PAS un problème de topologie à 3+ branches — c'est un défaut géométrique dans la méthode axe→normale→intersection elle-même : `CrossSectionFailure:: TooWide` est déjà un cas nommé et documenté (« normale quasi parallèle au bord ») exactement pour ce scénario. À

chaque coude à 90°, la tangente locale est calculée par différence centrée (`axis[i+1] - axis[i-1]`, `satin_column.cpp`), qui devient une direction DIAGONALE quand la fenêtre de calcul chevauche le coude — la normale perpendiculaire correspondante balaie alors quasi parallèlement à l'un des deux bords, parcourant une distance bien plus grande que la largeur RÉELLE de la branche avant de toucher le bord opposé. Le défaut touche PLUSIEURS stations consécutives (4 dans le cas mesuré, pas une seule) — au-delà de la fenêtre de la différence centrée autour du sommet.

Pourquoi ce n'est pas corrigé ici. `compute_column_stations` documente déjà, en commentaire, une décision délibérée et déjà coûteuse à apprendre : un échec ISOLÉ (une seule station) est comblé par interpolation (`interpolate_station`), mais un GROUPE d'échecs consécutifs refuse la colonne ENTIÈRE plutôt que de reconnecter silencieusement deux stations lointaines — cette reconnexion silencieuse est exactement ce qui produisait rails discontinus, éventails et morceaux partiels sur les formes concaves lors d'un audit antérieur (cf. plus haut dans ce document). Étendre naïvement le pontage à un groupe de plusieurs stations reviendrait à défaire ce garde-fou pour TOUTE forme du corpus, pas seulement `E` — cette fonction est partagée par absolument tous les objets satin, Legacy et Parametric. Le bon correctif ciblé (probablement une direction de section transversale en biseau/bissectrice au coude, plutôt que la tangente par différence centrée brute) doit être prototypé et mesuré isolément contre le corpus complet (`test_columns.cpp`, 28 cas, ET le corpus de torture) avant tout branchement — même discipline que la correspondance de contour elle-même (§ risque #1 du plan de refonte), pas une correction ponctuelle sous la pression d'une seule fixture (§33).

Portée réelle de cette découverte : distincte du cas `multi_neck` (vérifié séparément : 0 jonction, 1 seul arc de squelette — un pur problème de variation de largeur le long d'un ruban simple, 40,69 % de couverture brute mesurée en isolant `sgsd-debug`, causé par les transitions largeur fine/large des étranglements, PAS par un coude) et de `two_holes` (aucune famille de coupe applicable à 2+ trous, un problème encore différent). Les trois formes partagent le même symptôme de surface (100 % de résidu) mais relèvent de TROIS causes distinctes — aucun correctif unique ne les résoudra toutes les trois.

Deux hypothèses de correctif pour le coude à 90° testées et REJETÉES (2026-09-01)

Prototypage isolé (§21/§22 : mesurer avant de brancher) de deux correctifs plausibles pour la root cause ci-dessus, chacun testé DIRECTEMENT contre le tronc en «] » isolé de `E` avant tout branchement dans `build_satin_columns`.

Hypothèse 1 — tangente à sens unique en repli : `station_cross_section` tente la tangente par différence centrée (comportement historique) puis, uniquement si `TooWide`, retente avec la tangente entrante seule puis sortante seule. **Résultat : AUCUN changement** — échec identique à l'octet près (« trou de 2471 µm entre stations axe #23 et #28 »). Les deux tangentes à sens unique échouent elles aussi en `TooWide` à CHAQUE station du groupe : le problème n'est donc pas la direction de la tangente au point d'échantillonnage.

Hypothèse 2 — désactiver le lissage Chaikin de l'axe (`axis_smoothing_iterations=0`) : si le lissage étalait le coude sur plusieurs échantillons avant même le calcul de section, le désactiver devrait révéler un sommet net et resoudre le probleme. **Résultat : AUCUN changement non plus** (trou quasi identique, 2468 µm). Le lissage de l'axe n'est donc pas la cause.

Ce que ces deux rejets apprennent réellement : le problème n'est ni la tangente locale ni le lissage de l'axe, mais plus probablement une propriété structurelle de `cross_section` elle-même près d'une transition de largeur en T (spine 4 mm → barre 18 mm) — le rayon perpendiculaire, quelle que soit sa direction locale, traverse l'encoche concave interne du coude et ressort loin sur le bord convexe externe, mesurant une largeur qui n'a plus de rapport avec la largeur réelle du ruban à cet endroit. Une piste plus prometteuse mais non testée : plafonner localement `max_width` près d'un coude détecté (angle de tangente au-delà d'un seuil) par la largeur des stations voisines déjà valides, plutôt que par le plafond global — mais **cela reste une hypothèse non vérifiée**, à prototyper isolément comme les deux précédentes avant tout branchement. Les deux tentatives ci-dessus ont été retirées intégralement (`git checkout`, aucune trace en production) — rien n'est resté branché sans preuve de fonctionnement.

Aucune régression possible à signaler (les deux tentatives ont été testées puis retirées avant tout commit de code de production).

Troisième hypothèse : elle RÉSOUT « E » mais casse « trident » (2026-09-04)

Troisième prototype isolé, suite directe des deux rejets ci-dessus. Constat qui a orienté cette hypothèse : dans les deux tentatives précédentes, la normale reste PERPENDICULAIRE À LA TANGENTE (qu'elle soit calculée par différence centrée ou à sens unique) — jamais remise en cause elle-même. Or `cross_section` cherche l'intersection la PLUS PROCHE de chaque côté sur toute la droite portée par cette normale ; si cette droite précise traverse l'encoche concave du coude avant de ressortir sur le bord convexe externe de l'autre bras, AUCUN choix de tangente ne change la droite qui échoue — il faut changer l'ANGLE de la normale elle-même.

`station_cross_section`, v3 : sur un échec `TooWide` de la normale stricte, teste une série d'angles voisins (±5° à ±40° par pas de 5°, 16 candidats) et garde le résultat le plus ÉTROIT parmi tous les succès (pas le premier) — la mesure la plus proche de la largeur locale réelle est la plus plausible. `st.tangent` suit la normale réellement retenue (cohérence railA/railB/barreaux en aval).

Résultat sur le tronc isolé de « E » : succès complet. Plus aucun refus — 1 objet construit. `create_satin_plan("E")` complet passe de 100 % de résidu à **`Complete`, 98,03 % de couverture**. Progression mesurée en resserrant le pas angulaire : à ±10/20/30° (6 candidats), le trou residuel tombe de 2471 µm (rejet total) à 2572 µm après nettoyage anti-croisement — juste AU-DESSUS du seuil `max_station_gap_ratio × station_spacing` (2500 µm par défaut) ; resserrer à ±5° passe sous ce seuil et la colonne entière passe.

Mais régression réelle sur `trident` (`test_auto_satin`, corpus complet) — 3 échecs sur 58 cas, tous sur la même forme : `satin_column_view` (mode Legacy), `test_columns.cpp` (« exactement 3 objets independants »), `test_coverage_regression.cpp`. Message Legacy : `colonne refusee : trou de 5260 um entre stations axe #11 et #22 (largeur inferieure a min_satin_width)` — un refus `TooNarrow`, PAS `TooWide` (`station_cross_section` ne déclenche sa recherche d'angles voisins que sur `TooWide`, jamais directement sur `TooNarrow`) : la régression est donc un effet de bord INDIRECT — choisir un angle différent à une station donne des rails positionnés différemment, ce qui change la forme du quadrilatère avec la station suivante et peut déclencher `TooNarrow` ou le nettoyage anti-croisement plus loin sur l'axe, sur une branche qui s'effile légitimement vers une pointe (exactement le cas de `trident`, cf. le commentaire existant sur le plancher `min_satin_width` : « les stations proches d'un bout OUVERT rétrécissant légitimement vers zéro… ne sont PAS concernées », sauf que cette hypothèse-ci les concerne bel et bien indirectement).

Décision : retirée intégralement (`git checkout`, aucune trace en production) — `trident` est un garde-fou explicite du plan de refonte (« doit rester Complete »), une régression dessus n'est jamais acceptable même pour résoudre une limitation connue ailleurs. Piste retenue pour une suite éventuelle : restreindre la recherche d'angles voisins aux stations dont le VOISINAGE (avant/après) a une tangente stable (pas de rétrécissement en cours) — distinguerait un vrai coude à angle droit (E) d'un effilement légitime en pointe (trident), mais **reste une hypothèse non vérifiée**, à prototyper et mesurer contre le corpus complet (`test_auto_satin` ET `test_satin_planning`, Debug ET Release) avant tout nouveau branchement.

Aucune régression committée : la tentative a été testée puis intégralement retirée avant tout commit de code de production, comme les deux précédentes.

Étape 6 (boucles/complexe) : « two_holes » a déjà un repli complet et correct, sans correspondance de contour (2026-09-04)

Avant tout prototypage (§21/§22 : analyser d'abord) — investigation de `two_holes` (rectangle 60×30 mm, 2 trous circulaires de 5 mm séparés), la forme ciblée par l'étape 6.

Root cause du statut `Impossible` : `sgsd-debug --shape two_holes` révèle un squelette à 2 jonctions, 0 extrémité, dont un chemin qui EST UNE VRAIE BOUCLE (« chemin 1 : arcs [0, 1], nœuds [0, 1, 0] » — parcourt un nœud puis y revient), topologie totalement absente des étapes précédentes. Chaque coupe

candidate testée détache une branche de ~1000 mm² (essentiellement la moitié du rectangle autour d'un trou) — jamais une VRAIE fine branche satin — et `split_region`/le sélecteur rejette chacune (`aucune coupe valide, branche non isolee`, même symptôme que « E », cf. plus haut). Root cause RÉELLE, distincte de « E » : cette forme n'a structurellement AUCUNE branche satin-shaped à extraire — un rectangle large percé de trous est intrinsèquement un candidat TATAMI (remplissage de surface autour d'obstacles), pas satin (rubans étroits le long d'un axe). Aucune méthode de décomposition satin, aussi sophistiquée soit-elle, ne peut produire de « colonnes satin partielles » significatives sur une géométrie qui n'en contient tout simplement pas.

Vérifié séparément : le repli tatami existant dans l'application (§23 du plan de refonte satin, `MainWindow::askAboutIncompleteSatinCoverage` → `appendTatamiFallbackObjects`) est gated UNIQUEMENT sur `unresolved_residual`/l'aire de la région — jamais sur `SatinPlanStatus`. Il se déclenche donc IDENTIQUEMENT que le statut soit `Incomplete` ou `Impossible`. Mesuré au niveau bibliothèque (`tests/unit/stitch/ test_generate.cpp`, un objet `TatamiParams` directement sur la géométrie `two_holes`, trous inclus) : `generate_sequence` produit une séquence complète et correcte — couverture réelle de la forme entière, **zéro point à l'intérieur de l'un ou l'autre trou** (vérifié explicitement, jamais supposé). Concrètement, pour l'utilisateur : créer un satin sur `two_holes` déclenche déjà le dialogue §23 (0 % converti en satin, 100 % résiduel), et choisir « Utiliser tatami pour le reliquat » remplit CORRECTEMENT et COMPLÈTEMENT toute la forme — un résultat pleinement utilisable, déjà disponible aujourd'hui, sans le moindre changement de code.

Conclusion : l'étape 6 telle que formulée dans le plan de refonte (« `two_holes` doit passer d'`Impossible` à au moins `Incomplete` avec colonnes partielles ») ne correspond pas à un manque RÉEL côté utilisateur — c'est une limitation de vocabulaire interne (`SatinPlanStatus::Impossible` au niveau `satin_planning`) déjà totalement absorbée par le repli existant au niveau application. La correspondance de contour dédiée (le travail le plus risqué du plan, jamais entamé faute de cible réelle qui la justifie clairement — cf. l'investigation « E » ci-dessus, où le vrai blocage s'est avéré être un défaut de mesure de largeur aux coudes, pas un besoin de correspondance de contour non plus) reste donc un investissement à ne déclencher que lorsqu'une forme réelle et mesurée en a authentiquement besoin, jamais pour satisfaire un gate formulé avant l'investigation.

Non-régression : `test_stitch` (187 cas, 1520 assertions) complet, nouveau test dédié inclus.

`auto_digitize_vectors` : AutoChoice sans segmentation, pour l'import SVG direct (2026-09-12)

Demande utilisateur, suite à l'import SVG natif (`libs/formats/src/ svg_import.cpp`, § plus haut dans ce document) : pouvoir lancer « Numérisation automatique » (§24, `AutoChoice`) directement sur les objets vectoriels d'un SVG importé, sans jamais passer par une segmentation pixel — « comme si on avait skip la partie segmentation ». Avant ce correctif, `MainWindow::autoDigitize()` refusait purement et simplement tant que `project_.segmentation` était absent, message « Segmentez d'abord l'image » — un message trompeur pour un document sans image du tout.

Root cause du couplage : `autodigitize::auto_digitize(seg, ids, options)` mélangeait deux responsabilités dans une seule boucle — (1) itérer les régions d'une `Segmentation`, les vectoriser (`vectorization::vectorize_region`), et (2) classifier chaque morceau vectorisé en satin/tatami/contour (§24) puis construire les objets de broderie correspondants, avec repli tatami sur reliquat satin non couvert. Seule (1) a réellement besoin d'une `Segmentation` ; (2) ne prend en entrée qu'un `geometry::PathSet` déjà vectorisé — exactement ce qu'un `document::VectorObject` importé depuis un SVG fournit déjà.

Correctif : extraction de (2) dans une fonction libre `classify_and_build_embroidery(main, RegionSource, ids, options, result)` (`libs/autodigitize/src/autodigitize.cpp`, namespace anonyme) — `RegionSource` porte l'id du vecteur, sa couleur, un libellé pour les noms/avertissements (« Région 3 » ou le nom de l'objet importé), et un `RegionId` optionnel (présent seulement côté segmentation, reporté sur un éventuel vecteur de repli pour que `tests/integration/test_pipeline.cpp` continue de pouvoir regrouper repli et couverture satin par région d'origine). `auto_digitize` devient un simple appelant de cette fonction après vectorisation ; nouvelle fonction publique `auto_digitize_vectors(vectors, ids, options)` l'appelle directement sur des `VectorObject` déjà existants — **aucun changement de**

comportement pour la voie segmentation (même fonction partagée, mêmes chemins de code, seule la source du morceau à classifier change).

Différence assumée entre les deux entrées : `auto_digitize` crée toujours un nouvel objet vectoriel pour la géométrie qu'il vient de vectoriser (§ comportement historique) ; `auto_digitize_vectors` n'en crée JAMAIS pour son entrée (elle existe déjà dans le document de l'appelant) — seul un reliquat satin non couvert peut encore ajouter un vecteur de repli, avec la même garantie de couverture que la voie segmentation.

Câblage `apps/desktop` : `MainWindow::autoDigitize()` détecte l'absence de `project_.segmentation` et, si `project_.vector_objects` n'est pas vide, appelle `auto_digitize_vectors` au lieu de refuser. La boîte de dialogue « fond présumé » (`skip_largest_region`) ne s'affiche que côté segmentation — un SVG n'a pas de notion de fond de pixels. Le message d'erreur mis à jour mentionne désormais les deux chemins d'entrée possibles (« Segmentez d'abord l'image... ou importez un fichier SVG »).

Non-régression : `test_autodigitize` (17 cas, 6 nouveaux pour `auto_digitize_vectors`, dont un test parité forme-par-forme avec les cas segmentation équivalents), `test_pipeline` (vérifié manuellement que le regroupement par `source_region` des replis tatami reste correct — les tests DIAGNOSTIC TEMPORAIRE de ce fichier échouent tous par construction, `CHECK(false)` délibéré, non liés à ce correctif), nouveau test bout en bout `autoDigitizeAfterOpenSvgClassifiesVectorObjectsDirectly` (`tests/unit/desktop/test_main_window.cpp`) : import SVG réel puis `autoDigitize()` sans dialogue, vérifie une classification satin correcte sans jamais traverser image/segmentation.

Remplissage tatami

Public : utilisateur avancé, développeur.

> État : Présent dans le code : oui · Tests unitaires : oui (invariants de routage) · Tests visuels : partiels (SVG de diagnostic) · Import/export DST : oui · Test sur machine réelle : **non** · **Statut recommandé : partiel** (scanline + routage validé géométriquement ; **sous-couches, underpath caché et entrée** ajoutés au Lot 7 ; motifs de phase avancés non implémentés).

Définition

Le tatami remplit une zone pleine par des **rangées parallèles** de points, avec un décalage régulier des pénétrations d'une rangée à l'autre pour éviter un sillon visible. C'est le remplissage de référence pour les surfaces larges.

Génération par balayage (scanline)

`fill_tatami(region, params)` :

1. la géométrie (extérieur + trous) est virtuellement **tournée** de `-angle` pour travailler avec des rangées horizontales ;
2. pour chaque rangée `y`, on calcule les **intersections** avec tous les contours et on forme les **segments intérieurs** par paires (règle pair-impair) — une rangée peut donc comporter **plusieurs segments** (formes concaves, trous) ;
3. sur chaque segment, les pénétrations sont posées sur une **grille** de pas `stitch_length`, décalée d'une rangée à l'autre selon `stagger` ;
4. la géométrie est retournée dans le repère d'origine.

Routage qui contourne les trous

C'est le point délicat. Les segments de rangée sont les **nœuds d'un graphe** : deux segments de rangées voisines qui **se chevauchent en x** sont considérés comme **candidats** à une liaison. Un **parcours glouton** suit ces candidats et ne saute (aiguille levée) que vers un segment non relié.

Le chevauchement n'est **qu'une heuristique d'adjacence** : il ne prouve pas qu'une liaison entre les extrémités des deux segments reste dans la région. Une corde entre deux points posés sur les bords peut longer l'extérieur (encoche, poche concave d'une forme en U, pont au-dessus d'un trou). **Chaque trajet cousu est donc validé géométriquement** contre la région et ses trous (`connector_invalid`) : le connecteur `[a,b]` est **découpé** à chaque intersection paramétrique avec une arête (extérieur ou trou) — y compris un contact **dégénéré par un sommet** (le connecteur touche exactement un coin sans « croiser » franchement une arête) — puis le **milieu de chaque sous-segment** ainsi obtenu doit rester dans la région :

- une arête **colinéaire** au connecteur (suivi de bord/frontière) ne produit aucune découpe → le suivi de bord reste cousu, quelle que soit son orientation ;
- toute autre intersection (croisement franc, **ou simple contact par un sommet**) découpe le connecteur ; si un sous-segment tombe hors de la région → saut. Ce test est **indépendant de l'écart en x** du connecteur : un connecteur **parfaitement vertical** qui traverse un trou de part en part en touchant exactement ses deux sommets (haut et bas) est donc détecté, alors qu'une version antérieure de `connector_invalid` — qui excluait les contacts sommet/extrémité du test de croisement **et** ne sondait l'intérieur que si l'écart en x dépassait `2 × row_spacing` — le laissait passer à tort (corrigé courant Lot 7, cf. `segment_stays_in_region` pour la validation exposée).

En l'absence de trajet intérieur valide, le moteur utilise un **saut**. Chaque point est un `FillStitch { pos, jump }` (`jump = true` = aiguille levée). Ainsi aucune couture ne traverse un trou ni ne sort de la région.

Note : Ce comportement corrige plusieurs défauts successifs signalés en revue (les remplissages débordaient sur les régions à trou ; puis la justification « chevauchement ⇒ liaison intérieure » était trop

optimiste ; puis les contacts par sommet échappaient à la validation géométrique). Vérification via `openstitch-cli stitchdebug --shape ring` : sur un anneau, 0 couture traverse le trou et le contournement ne coûte que ~2 sauts. Des tests couvrent aussi une forme concave en **U** (aucune couture ne traverse l'encoche), une forme en **L** — où l'on échantillonne chaque segment cousu à **cinq angles** (0/20/45/90/135°) pour vérifier qu'**aucun ne sort de la région** (le test tolère les coutures qui *longent* le bord) et où l'on vérifie spécifiquement qu'un contact au **coin rentrant** ne laisse pas passer une couture vers l'encoche —, et des trous en **losange** (sommets alignés verticalement, pluralité de trous) qui exercent précisément le cas des contacts sommet. C'est ce filet qui garantit que router les zones vers le tatami — plutôt que vers le satin naïf — supprime le débordement (voir *Colonne satin*).

Point subtil de `in_region` : le test pair-impair est ambigu **exactement sur une frontière**. Pour un point sur le bord de l'**extérieur**, il se résout correctement en « intérieur » ; pour un point sur le bord d'un **trou**, une implémentation naïve le classerait à tort « dans le trou » (parité déclenchée par l'arête opposée du trou), ce qui rejetterait un suivi de bord pourtant légitime. `in_region` traite donc explicitement tout point **sur** une frontière (distance quasi nulle à une arête, tolérance ~0,01 µm — une marge numérique, pas géométrique) comme faisant partie de la région, avant le test pair-impair.

Choix de l'auto-numérisation : comme le satin naïf déborde, **toute zone remplissable est désormais numérisée en tatami** par défaut (fines bandes comprises). Le tatami est le remplissage sûr tant que le vrai moteur satin (squelette) n'est pas branché.

Tatami avancé (Lot 7)

État : Présent · Testé numériquement · Validé visuellement (SVG). Tout est **désactivé par défaut**, éditable (inspecteur) et persisté (`.osp`, rétrocompatible). Chaque commande porte une `StitchPass` (§4) : sous-couches `Underlay`, couche supérieure `TopStitch`, trajet caché `Travel`.

Sous-couches (`tatami_underlay`), cousues **avant** la couche supérieure :

- **Contour rentré** (`underlay_edge`) : running le long du bord **extérieur**, retrait `underlay_inset`. Les **bords de trous ne sont pas** longés (l'inset d'un trou le rétrécit vers son centre ; en longer le bord ferait passer la sous-couche au-dessus du trou — sûreté). **Politique sûre explicite** si le retrait échoue ou fait disparaître la forme (pièce trop petite pour `underlay_inset`) : **aucune** sous-couche de contour n'est émise pour cette forme — jamais de repli silencieux sur le bord **brut** (coudre exactement sur l'arête finale n'offre aucune marge de stabilisation et peut déborder une fois la compensation de bord de la couche supérieure appliquée par-dessus). Un retrait **explicitement nul** (`underlay_inset` = 0) est une intention distincte, pas un échec : le bord brut est alors bien celui voulu.
- **Rangées parallèles espacées** (`underlay_parallel`) : balayage tatami **perpendiculaire** aux rangées supérieures, pas `underlay_spacing`. Réutilise le routage (contourne les trous). Évite l'affaissement de la surface.

Underpath caché (`hidden_underpath`) : au lieu de sauter, une liaison est **cousue et cachée** quand un trajet valide existe — soit **direct** (s'il reste intérieur et court), soit **le long du contour extérieur rentré** (« autoroute » qui **contourne les trous**). Les pénétrations intermédiaires sont taguées `Travel`. Au-delà d'un plafond de longueur (`max(6·pas, 16 mm)`), on **saute** (un déplacement à découvert long serait pire qu'une coupe). Toujours borné par la validation géométrique : **jamais** de trajet caché à travers un trou.

Entrée (`entry_point`, optionnel) : le remplissage démarre par le segment le plus proche du point d'entrée, puis chaque nouvelle composante est atteinte par le segment non visité le plus proche.

Vérifié : sous-couche de contour rentrée dans la forme ; sous-couche parallèle espacée ; underpath convertit au moins un saut en trajet cousu **sans jamais** traverser le trou (invariant préservé) ; déterminisme ; l'entrée oriente le démarrage ; aller-retour `.osp` ; retrait de contour impossible (pièce trop petite) → aucune sous-couche émise (pas de repli sur le bord brut) ; retrait explicitement nul → bord brut (intention voulue). SVG : `openstitch-cli stitchdebug --shape ring --underlay 3 --underpath` (contour et parallèle en vert, couche supérieure en gris, underpath en bleu, sauts en rouge).

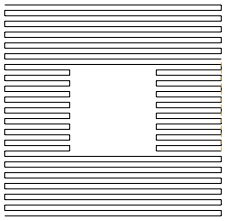


Figure — Trajectoire réelle (SVG de diagnostic) : anneau à trou central rempli en tatami. Le remplissage contourne le trou ; aucune couture ne le traverse (seulement ~2 sauts).

Déplacements internes (audit marine plein cadre, 2026-09-22)

Trois défauts mesurés sur le tatami d'un grand ciel percé de nuages, avec les outils de mesure du Lot G (*Analyse et validation*) :

- **Composante suivante.** Quand une composante de rangées est épuisée, `fill_tatami` reprenait au premier segment non visité **dans l'ordre de balayage**, souvent à l'autre bout de la forme. Il reprend désormais au segment non visité le **plus proche** du point courant (même règle que pour un point d'entrée), ce qui raccourcit le saut et permet plus souvent un trajet caché.
- **Liaisons cousues trop longues.** La liaison entre deux segments de rangées voisines (arête du graphe) était validée géométriquement mais pas bornée en longueur. Deux segments qui ne se chevauchent que sur une petite portion étaient reliés par un seul point de 12 à 62 mm, un fil flottant visible. Une liaison plus longue que `stitch_length` est maintenant découpée en pénétrations cachées (passe `Travel`) d'au plus `stitch_length`.
- **Sous-couche en rangées.** Elle était toujours générée sans trajet caché (625 sauts sur la marine). Elle hérite désormais de `hidden_underpath` : sous la couche supérieure, une liaison cousue de sous-couche est invisible.

Optimisation des déplacements (2026-10-12) : le trajet caché est désormais un **graphe de visibilité** (plus court chemin, Dijkstra) sur les sommets du contour rentré de la **région entière** -- extérieur rentré **et trous agrandis** --, borné par `underpathCap` (16 mm) et validé segment par segment (`connector_invalid`) : les trous sont contournés par leurs propres sommets, plus seulement par le contour extérieur. Sur une image de 380 x 290 mm (`sample/`), avec un plafond de 8 mm les déplacements passent de 1 619 à 860 ; le plafond est porté à **16 mm** (675 déplacements, 616 coupes ; décision utilisateur du 2026-10-12). Les îlots d'un même objet sont cousus du plus proche au plus loin (boîte englobante) et non dans l'ordre de stockage. Limite restante : au-delà de `underpathCap` on saute ; les coutures cachées jusqu'à 16 mm sont plus longues donc plus visibles que celles de 8 mm.

Paramètres

Valeurs par défaut lues dans `TatamiParams` (`libs/document/.../embroidery_object.hpp`).

Paramètre	Unité	Défaut	Effet quand on augmente	Effet quand on diminue
`row_spacing` (densité)	mm	0,4	remplissage plus léger, risque de jours	remplissage plus dense, plus rigide
`stitch_length`	mm	3,0	points plus longs le long des rangées	points plus courts, plus de pénétrations
`angle`	° (rad interne)	0	oriente les rangées	—
`inset` (retrait de bord)	mm	0,2	remplissage plus rentré	remplissage plus au bord (risque de débord)
`stagger`	rangées	2	pénétrations décalées sur plus de rangées	sillon plus visible
`underlay_edge`	bool	off	sous-couche de contour (Lot 7)	—
`underlay_inset`	mm	0,6	contour de sous-couche plus rentré (échec/disparition → aucune sous-couche émise, jamais le bord brut ; 0 = bord brut voulu)	contour plus proche du bord final
`underlay_parallel`	bool	off	sous-couche perpendiculaire espacée (Lot 7)	—
`underlay_spacing`	mm	2,0	rangées de sous-couche plus espacées	plus denses
`hidden_underpath`	bool	off	coud et cache les liaisons courtes (Lot 7)	plus de sauts
`entry_point`	µm	—	démarre le remplissage près de ce point (Lot 7)	—
`pull_compensation`	mm	0	chaque rangée dépasse du contour de cette longueur, dans l'axe du fil (HP-ENG-001) ; plage [0 ; 3] mm	aucune compensation
`underlay_mode`	Manuelle / Automatique	Manuelle	Automatique : le moteur choisit la sous-couche (voir *Moteur de génération de points*), les réglages `underlay_*` sont ignorés	réglages manuels appliqués tels quels

Compensation du tirage (HP-ENG-001)

Le fil tire dans sa propre direction : une zone cousue en rangées rétrécit dans l'axe des rangées, pas perpendiculairement. `pull_compensation` allonge donc chaque rangée de `pull` **aux deux bouts**, dans l'axe du fil, quel que soit l'`angle`. Seules les pénétrations d'extrémité de rangée dépassent ; la validation des liaisons (cousues cachées ou sauts) et l'adjacence des rangées se font sur les bornes réelles de la région (`prevC`/`rpC` dans `fill_tatami`), donc le nombre de sauts ne change pas. Près d'un trou, les rangées dépassent dans le trou d'au plus `pull` (le trou se rétrécit à la couture comme le reste). La sous-couche en rangées reste dans la forme (`pull` forcé à 0). À 0 (défaut), la sortie est identique octet pour octet à celle d'avant ; la valeur est bornée à 3 mm à la génération et à la lecture du .osp. Il n'existe pas de notion de tissu dans le modèle : le réglage est manuel (0,2 à 0,4 mm est courant). Le remplissage directionnel n'a pas encore cette compensation (HP-ENG-001 reste partiel). Tests : `tests/unit/stitch/test_engine_points.cpp` (« tirage : … »).

Le paramètre principal de **densité** est `row_spacing` (entre rangées) — à ne pas confondre avec `stitch_length` (le long d'une rangée). Le retrait de bord est appliqué **en amont** (offset intérieur du polygone) par `generate_tatami`. Tous ces paramètres, y compris `underlay_inset` et `underlay_spacing`, sont éditables dans l'inspecteur (`PropertiesPanel`), avec undo/redo générique (`SetStitchParamsCommand`).

Orientation éditable (angle des fils)

L'`angle` se règle à la création, puis se **modifie après coup** : soit numériquement (menu *Broderie* ■ *Orientation du remplissage…*, ou clic droit sur la forme), soit **directement dans la scène** en faisant glisser une **poignée de rotation** (un axe bleu au centre du remplissage). Le nouvel angle est appliqué au relâchement via une commande annulable (`SetFillAngleCommand`) ; les points sont régénérés (ADR-014). L'angle est modulo 180° (orientation d'une droite).

> Validation physique : non effectuée. Ces valeurs sont des points de départ logiciels, pas des réglages éprouvés sur machine.

Limitation réelle (au-delà du Lot 7, cf. ci-dessus qui **est** implémenté) : les **motifs de phase avancés** (staggering non uniforme, densité variable) restent **prévus, non implémentés**.

Implémentation associée

- `libs/document/.../embroidery_object.hpp` — `TatamiParams`.
- `libs/stitch_generation/include/openstitch/stitch_generation/tatami.hpp` — `FillStitch`, `fill_tatami`, `tatami_underlay`, `segment_stays_in_region` (validation géométrique exposée pour test).
- `libs/stitch_generation/src/tatami.cpp` — scanline, graphe de routage, `connector_invalid` (validation géométrique des trajets cousus, découpe paramétrique robuste aux contacts sommet), `in_region` (frontière traitée comme intérieure, tolérance numérique).
- `libs/stitch_generation/src/generate.cpp` — `generate_tatami`, `emit_fill`.
- `libs/geometry/src/offset.cpp` — `inset_path_set` (retrait de bord).
- `apps/desktop/properties_panel.cpp` — inspecteur : tous les champs `TatamiParams`, y compris `underlay_inset`/`underlay_spacing`.
- Tests : `tests/unit/stitch/test_tatami.cpp` (invariants : aucune couture dans le trou ; aucune couture hors région sur une forme en L à tous les angles ; contacts sommet — trou en losange, multi-trous, coin rentrant d'un L ; politique sûre de `tatami_underlay` sur retrait impossible/nul), `tests/unit/geometry/test_offset.cpp`.
- `libs/commands/.../project_commands.hpp` — `SetFillAngleCommand` (orientation), `ConvertFillsToTatamiCommand`, `SetStitchTypeCommand`.

Remplissage directionnel

Public : utilisateur avancé, développeur. État : **Implémenté (Phases 1 à 3)** · Tests unitaires : oui · Tests visuels : non · Test sur machine réelle : **non** · **Statut recommandé : expérimental**.

Objectif

Le tatami couse des rangées **droites** à angle fixe. Le remplissage directionnel est une **alternative** (le tatami n'est pas modifié) dont les points **suivent la forme**, pour imiter la broderie main — passé empiétant, peinture à l'aiguille. Exemple de référence : une vague dont les points partent de la crête et descendent en éventail en suivant le rouleau, avec des secteurs aux directions différentes (chevrons) et des points de longueurs irrégulières qui s'imbriquent.

Modèle de données

`document::DirectionalFillParams`, quatrième alternative du variant `StitchParams` (ajoutée **en fin** de variant : les index des types historiques ne changent pas).

Champ	Défaut	Rôle
`guides`	vide	courbes guides (polylignes ou Béziers, ouvertes), µm, repère du modèle
`break_lines`	vide	lignes de rupture (secteurs indépendants)
`row_spacing`	0,4 mm	écart entre lignes de couture (densité)
`density_gradient`	absent	dégradé de densité (voir ci-dessous) ; remplace `row_spacing` pour le tracé
`spacing_regularity`	0	régularité de l'espacement [0 ; 1] (voir *Régularisation*) ; 0 = tracé brut
`stitch_length`	3 mm	longueur cible, **bornée à [1 ; 7] mm** à la génération
`edge_weight`	0	influence de la tangente du bord le plus proche, [0 ; 1]
`inset`	0,2 mm	retrait de bord (comme le tatami)
`stagger`	2	lignes avant répétition de la phase des pénétrations
`underlay_edge` / `underlay_parallel`	non	sous-couches (générateurs du tatami)
`underlay_inset` / `underlay_spacing`	0,6 / 2 mm	réglages des sous-couches
`hidden_underpath`	oui	liaisons cousues cachées plutôt que des sauts
`sector_overlap`	0,25 mm	chevauchement le long des ruptures
`handmade` / `handmade_intensity`	non / 50 %	aspect fait main
`seed`	id de l'objet	graine de l'aspect fait main

Les guides sont exprimés dans le **même repère que le contour** de l'objet vectoriel source : `TranslateVectorObjectCommand` et `ScaleVectorObjectCommand` les déplacent / redimensionnent avec la forme (undo exact).

Dégradé de densité (`DensityGradient`, HP-STI-010) : l'écart entre lignes varie **linéairement** le long de l'axe `from` → `to` (µm, repère du modèle), de `spacing_from` à `spacing_to`, constant au-delà des extrémités. Les écarts sont bornés à **[0,1 ; 4] mm** à la génération ; un axe nul (`from == to`) donne un écart uniforme `spacing_from`. Sans dégradé le tracé est identique octet pour octet à l'ancien. L'axe suit la forme (déplacement, redimensionnement) comme les guides ; les écarts, longueurs physiques, ne sont pas mis à l'échelle. `row_spacing` reste la référence des seuils de parcours (liaisons cousables, trajet caché), non du tracé.

Format `.osp` : type `"directional"`, toutes les clés sauf `type` sont optionnelles (défauts du modèle) ; le dégradé est l'objet optionnel `densityGradient` (`fromX`, `fromY`, `toX`, `toY`, `spacingFrom`, `spacingTo`, µm entiers). `schemaVersion` n'a pas changé : un projet ancien se relit à l'identique. Un projet contenant un remplissage directionnel n'est pas lisible par une version antérieure (« Type de point inconnu »).

Algorithme

`libs/stitch_generation/src/directional_fill.cpp`, fonctions pures et déterministes. `generate_sequence` l'appelle comme le tatami : retrait de bord (repli sur la forme brute si le retrait la fait disparaître), sous-couches, puis couche supérieure.

1. Secteurs (Phase 2)

Chaque ligne de rupture est convertie en bandes de 20 µm (prolongées de `max(espacement, 1 mm)` aux extrémités pour qu'une rupture tracée « presque » jusqu'au bord la traverse quand même) et soustraite de la région (`geometry::subtract_polygons`) : chaque composante est un **secteur**. Les miettes (< 4 espacement²) sont ignorées. Chaque secteur a son propre champ, ses propres lignes et sa propre grille de séparation ; il est tracé dans une zone élargie de `sector_overlap` (offset puis intersection avec la région), d'où le chevauchement qui évite les interstices le long des ruptures. Les guides sont densifiés (pas ≤ 0,5 mm) puis **coupés** par secteur : un guide qui traverse une rupture n'influence que ses deux morceaux respectifs.

Limitation : une rupture qui ne traverse pas la forme (ne la sépare pas) n'a aucun effet — le champ s'interpole alors à travers elle.

2. Champ de directions

En chaque nœud d'une grille (pas = espacement, borné à [0,25 ; 1] mm) :

- pour chaque guide du secteur, point le plus proche et tangente de son segment ; pondération `1 / (d² + h²)` avec `h` = espacement (pas de singularité sur un guide) ;
- les tangentes sont moyennées sur l'**angle doublé** `(cos 2θ, sin 2θ)` : une orientation et son opposée donnent le même vecteur, le sens de tracé d'un guide est donc indifférent et deux guides quasi opposés ne s'annulent pas ;
- sans guide (ou en cas d'annulation exacte) : **axe principal** du secteur (`geometry::principal_axis`) ;
- influence du bord : `α = edge_weight · exp(−d_bord / bande)`, bande = `max(2 mm, 5 × espacement)`, mélange `(1 − α)·guides + α·tangente du bord`, toujours sur l'angle doublé.

Entre les nœuds, interpolation bilinéaire des vecteurs doublés.

3. Lignes de courant (Jobard & Lefer)

Algorithme de *Creating Evenly-Spaced Streamlines of Arbitrary Density* (Jobard & Lefer, 1997) :

- distance de séparation `d_sep` = espacement, distance de test `d_test` = 0,5 × espacement ; avec un dégradé de densité, ces deux distances sont **locales** (`d_sep(p)` évalué au point considéré) : un écart plus grand à un bout fait naître moins de lignes. Pas d'intégration `0,2 × d_sep_min`, fenêtre d'auto-proximité `3 × d_sep_max`, cellule de la grille = moyenne géométrique de `d_sep_min` et

`d_sep_max` ;

- intégration RK2 à pas fixe 0,2 × `d_sep`, dans les deux sens depuis la graine, le sens étant maintenu d'un pas à l'autre ;
- arrêt : sortie de la zone ou entrée dans un trou (le point de bord exact est retrouvé par dichotomie, pour que la couture atteigne le contour), approche à moins de `d_test` d'une autre ligne (ou d'elle-même hors d'une fenêtre de 3 `d_sep`), virage > 40° en un pas (point singulier du champ) ;
- propagation : graines candidates à ± `d_sep` perpendiculairement à chaque ligne, tous les `d_sep`/2 ; file FIFO ;
- **comblement** : balayage final d'une grille au pas `d_sep`/4 ; tout point resté à plus de 0,9 `d_sep` de toute ligne devient une graine — aucune zone vide ni îlot oublié.

Chaque ligne porte une coordonnée d'arc `u` héritée de sa ligne mère (même valeur au point de la graine) et un **rang** (± 1 par rapport à la mère) : ce sont eux qui alignent les pénétrations d'une ligne à sa voisine.

Structures d'accélération : `RegionIndex` (grille dont chaque case connaît ses arêtes et l'état intérieur/extérieur de son centre — point intérieur, segment intérieur et bord le plus proche en temps quasi constant) et `SeparationGrid` (cases de `d_sep`).

3 bis. Régularisation de l'espacement (optionnelle)

Quand les directions convergent, Jobard & Lefer coupe les lignes qui se rapprochent trop mais laisse un espacement irrégulier (mesuré sur un éventail de deux guides : écart au plus proche voisin, dispersion 0,23 de l'écart demandé). Avec `spacing_regularity > 0`, les sommets des lignes sont déplacés par **une résolution linéaire creuse** (Liu et al., CGF 2023, §3.4), après le tracé et avant la découpe en points. Énergie sans dimension (distances divisées par l'écart local `s`) :

- **densité** : pour chaque sommet, son plus proche voisin d'une autre ligne de chaque côté (à peu près en face) doit se trouver à `±s` selon la normale du champ — l'ensemble des paires est la version « voisins » de l'arête de Delaunay de l'article, sans triangulation ;
- **direction** : chaque arête garde sa direction de tracé ;
- **étirement** : un sommet ne glisse pas le long de sa ligne (poids fort) ;
- **rappel** (poids 0,1) vers la position tracée.

Le curseur `spacing_regularity` règle le poids de densité de 0,1 à 2,5 face au rappel fixe. Résolution par gradient conjugué préconditionné (Jacobi), 400 itérations au plus, mono-thread : déterministe. Garde-fous : déplacement plafonné à `s/2` et sommet ramené (dichotomie) dans la zone de traçage s'il en sortait. Le nombre de lignes ne change pas.

Calibrage mesuré (éventail de deux guides, 30 × 20 mm, écart 0,4 mm) :

`spacing_regularity`	dispersion de l'écart	écart à la direction
0 (brut)	0,232	0,2°
0,5	0,158	1,9°

Au-delà, trop de sommets butent sur le plafond de déplacement et la géométrie se dégrade : l'optimum est vers 0,5. Coût : proportionnel au nombre de sommets (synchrone, comme le reste de la génération) ; réservé aux objets qui en ont besoin.

4. Découpe en points

- Pénétrations sur une grille de pas `L` (longueur cible bornée à [1 ; 7] mm) en coordonnée `u`, déphasée de `L · (rang mod stagger) / stagger` — effet brique comme le tatami ;
- aucun point < 0,5 mm aux extrémités ; aucun > 7 mm (scission en parts égales) ;
- une corde dont la flèche dépasse 0,15 mm, ou qui sortirait de la zone (bord concave), est scindée en deux (fidélité aux courbes).

5. Parcours

Par secteur, d'un seul tenant ; secteur suivant = celui dont une extrémité est la plus proche. Dans un secteur, choix glouton de la ligne suivante :

1. parmi les liaisons **cousables** (≤ `max(4 × espacement, 1,5 mm)` et intérieures), règle de **Warnsdorff** : la ligne qui a le moins de voisines libres d'abord — une ligne courte coincée est cousue au passage au lieu de rester orpheline ;
2. sinon, l'extrémité la plus proche.

Le départ est une ligne **extrême** du faisceau (le moins de voisines). Le sens de chaque ligne suit l'extrémité la plus proche : une ligne sur deux est parcourue en sens inverse. Liaisons : point cousu si cousable ; sinon, avec `hidden_underpath`, trajet caché direct (≤ `max(6 × espacement, 8 mm)`) ou le long du contour extérieur rentré (mêmes règles que le tatami) ; sinon saut.

6. Sous-couches

`directional_underlay` réutilise `tatami_underlay` : contour rentré, et rangées droites perpendiculaires à la **direction moyenne** du champ (moyenne des vecteurs doublés sur une grille intérieure).

7. Aspect fait main (Phase 3)

Intensité `I` = `handmade_intensity` / 100 (0 si `handmade` est faux ; à `I` = 0 le résultat est identique au mode régulier) :

- chaque pénétration bouge de ± 20 % × `I` × `L` : l'écart entre deux pénétrations varie de ± 40 % × `I`, puis est reborné à [1 ; 7] mm ;
- alternance court/long des **premier et dernier** points de chaque ligne selon la parité du rang (court = `(1 − 0,6 I) · L`) : les extrémités s'imbriquent au lieu de s'aligner ;
- ondulation de direction ≤ 6° × `I`, **bruit lissé** (valeurs tirées aux nœuds d'un réseau de 5 mm, interpolation smoothstep) — jamais aléatoire point par point ;
- tout tirage passe par un hachage splitmix64 de (graine, clés) — jamais `std::uniform_*_distribution`, dont la sortie dépend de la bibliothèque standard : même projet, même résultat sur toute plateforme.

Alternatives écartées

Graines par divergence (sources et puits d'après la divergence de ρ·v, arbre k-d, affectation puits ↔ ligne ; Liu et al., CGF 2023, §3.1–3.3). Elle garantit le nombre de lignes de courant d'un champ de densité variable. Mesure faite avant de la retenir : sur un dégradé de 20 mm (écart de 0,4 mm à 0,8, 1,6 ou 3 mm), le tracé Jobard & Lefer actuel donne **35, 22 et 15 lignes pour 34,7, 23,1 et 15,5 attendues** (intégrale de 1/s), soit moins de 5 % d'écart, avec ou sans régularisation (test de non-régression `directional density gradient keeps the row count at the integral of the density`). Elle exigerait en plus un champ **orienté** (vecteurs, alors que le champ est défini modulo 180°) et un solveur d'affectation. Coût et risque sans gain mesurable : non implémentée. À reconsidérer si un cas de lignes qui naissent ou meurent mal apparaît (champs très divergents).

Parcours par arbre couvrant + profondeur d'abord (même article, §3.5 : arbre couvrant minimal sur les arêtes de Delaunay entre lignes, parcours qui double chaque arête de l'arbre, d'où un fil continu sans saut). Mesure faite avant de la retenir : sur des formes à branches (U, T, L, lignes horizontales ou verticales) le parcours actuel — Warnsdorff plus liaisons cousues, trajet caché le long du contour rentré — laisse **0 à 2 sauts, 10 à 22 mm au total** (test `directional routing leaves at most two jumps on branching shapes`). L'arbre ne supprimerait ces sauts qu'en recousant la même distance en retour sur une zone déjà cousue (surépaisseur, fil visible en haute densité) : c'est déjà ce que fait le trajet caché quand il est possible, avec un plafond de 8 mm voulu. Pas de gain mesurable : non implémenté.

Fondu de couleurs à deux fils

Idée de Liu et al. (*Directionality-Aware Design of Embroidery Patterns*, CGF 42(2), 2023, §4.3), reprise sans code tiers. Une région dont les couleurs s'étalent entre deux teintes se coud en **deux passes** : un fond uni de couleur s1, puis un remplissage de couleur s2 dont l'écart entre lignes varie. Avec `b` la largeur visible du fil de s2 et `1/ρ` l'écart entre ses lignes, une période montre `b` de s2 et `1/ρ − b` de s1 : la proportion de s2 est `t = b·ρ`, donc **`ecart = b / t`** (borné à [`min_spacing` ; `max_spacing`]).

`autodigitize::analyze_two_color_blend(samples, options)` (`libs/autodigitize/src/two_color_blend.cpp`) :

1. ACP des couleurs en **CMY** (première composante, itération de la puissance, signe canonique) ; s1 et s2 sont les extrémités de l'axe après rejet de 2,5 % de valeurs aberrantes de chaque côté ;
2. s1 (fond) est la couleur la mieux représentée : si `t` moyen dépasse 0,5, les rôles s'échangent ;
3. plan `t(x, y)` par moindres carrés → un `DensityGradient` (axe + écarts) directement utilisable comme `DirectionalFillParams::density_gradient` du remplissage de dessus ; sous `min_ramp` (0,05) de variation, l'axe est nul et l'écart uniforme `b / t`■.

Le choix des fils réels (`thread_palette::best_thread_pair`) et le lissage préalable (`BilateralDenoiseOp`) complètent la chaîne : `autodigitize::analyze_region_blend` lisse l'image de travail, échantillonne les pixels sous la forme (`sample_region_colors`, au plus 20 000, repère du modèle), analyse, puis choisit la paire de fils dans le nuancier fourni.

Action de l'interface (`MainWindow::applyTwoColorBlend`, menu Broderie et barre contextuelle d'un objet tatami ou directionnel) : l'objet sélectionné devient le **fond** (il garde son rang, donc il est cousu avant ; son dégradé éventuel est retiré) et un **nouvel objet directionnel** de la seconde couleur, à densité variable et sans sous-couche, est ajouté par-dessus. Les fils viennent du nuancier sélectionné dans le panneau Fils ; sans nuancier, les couleurs analysées restent libres. Une seule annulation défait l'ensemble. **Orientation des lignes** : l'écart entre lignes ne peut varier que perpendiculairement aux lignes ; le dessus hérite pourtant du guide du tatami (angle d'origine). Si les lignes courent le long de l'axe de couleur, le dégradé ne peut espacer presque rien et le fondu est écrasé (défaut trouvé 2026-10 : sur une rampe horizontale avec un guide horizontal, il restait plus de 80 % des lignes d'un remplissage uniforme). L'action remplace donc les guides du dessus par un guide droit perpendiculaire à l'axe (`stitch_generation::align_guides_across_density_gradient`, test `blend: rows are turned across the colour axis…`). Déterministe : aucune graine, itération dans l'ordre d'entrée. Limites : un seul plan (un dégradé linéaire par région, pas de courbe), deux couleurs maximum par région.

Fondre deux zones voisines

Forme ■ Fondre deux zones (Ctrl+Maj+F), avec exactement deux formes sélectionnées, chacune avec un remplissage (tatami ou directionnel) : les deux formes sont **unies** (comme *Unir* : la première sélectionnée, A, survit et garde son remplissage comme **fond**), et un remplissage directionnel de la couleur de B est ajouté **dessus**, sur toute la forme unie. Son dégradé de densité (`stitch_generation::density_gradient_between`) va du centre de A vers celui de B sur l'étendue des deux zones : écart large (4 mm) côté A, serré (0,4 mm) côté B ; les lignes sont perpendiculaires à cet axe. Contrairement au fondu à deux fils, les couleurs sont **celles des deux zones**, pas une analyse de l'image. Une seule annulation défait l'ensemble. Limite : rampe linéaire sur toute l'étendue (pas de zone de transition réglable).

Interface

- **Conversion** : bouton « Convertir en remplissage directionnel » de l'inspecteur d'un tatami, ou « Type de points ■ Remplissage directionnel » (menu contextuel, barre contextuelle). Les réglages du tatami sont repris et un guide droit à son angle, traversant la forme, reproduit l'orientation initiale (`directional_from_tatami`). `ConvertFillGroupCommand`, annulable.
- **Inspecteur** : espacement, longueur (1 à 7 mm), influence des bords, retrait, décalage, chevauchement des secteurs, sous-couches, liaisons cachées, aspect fait main (intensité, « Autre tirage » = nouvelle

graine déterministe), résumé du nombre de guides et de ruptures.

- **Mode « Guides de direction »** (Broderie ■ Guides de direction…, touche D, ou « Éditer les guides… » dans l'inspecteur) : affiche l'aperçu du champ (petits traits de direction, avant génération), les guides (traits pleins), les ruptures (tirets) et une poignée par point. Glisser une poignée déplace le point ; clic droit : supprimer le point ou le guide entier. « Tracer un guide de direction » / « Tracer une ligne de rupture » : clics successifs, Entrée / double-clic / ✓ pour terminer, Retour arrière pour retirer le dernier point, Échap pour annuler. Un guide devient une courbe lisse passant par les points cliqués (`geometry::smooth_open_path`, Catmull-Rom → Béziers) ; une rupture reste une polyligne (angles vifs des chevrons).
- **Générer un guide depuis la forme** (`directional_guides_from_region`, axe médian du pipeline auto-satin, sans aucun satin) : le chemin le plus long du squelette donne le guide principal ; chaque branche restante d'au moins `min_path_length` (1,5 mm) reçoit son propre guide, ancré sur le squelette déjà couvert (composante déconnectée : sur sa plus longue arête), dans la limite de `max_guides` (8). Glouton, ordonné par (longueur décroissante, id) : déterministe. Avant (2026-10), un seul guide était produit : les lobes d'une forme ramifiée n'avaient aucun guide proche et restaient mal couverts. L'action **remplace** les guides existants — y compris le guide droit posé par la conversion depuis un tatami — au lieu de s'y ajouter ; les lignes de rupture sont conservées.
- Toute édition passe par `EditDirectionalFillCommand` (ou `SetStitchParamsCommand` depuis l'inspecteur) : undo/redo exact.

Tests

`tests/unit/stitch/test_directional_fill.cpp` :

- guide horizontal → même densité qu'un tatami à 0° (± 5 %) ;
- guide en arc → toutes les cordes à moins de 10° du champ ;
- guides convergents → aucune paire de lignes à moins de 0,5 × espacement, aucune zone vide de plus de 2 × espacement ;
- déterminisme octet pour octet de `generate_sequence` ;
- repli sur l'axe principal, influence du bord, trous évités, bornes de longueur, conversion depuis le tatami, sous-couche, parcours sans saut sur forme convexe ;
- secteurs : champ discontinu de part et d'autre de la rupture, lignes cantonnées à leur secteur au chevauchement près, chevauchement effectif, un seul passage d'un secteur à l'autre ;
- fait main : identique au régulier à 0 %, longueurs variées dans [1 ; 7] mm, alternance court/long, dépendance à la seule graine, ondulation ≤ 6° et lisse.

Aussi : `test_roundtrip.cpp` (sérialisation), `test_undo_stack.cpp` (translation/mise à l'échelle des guides, conversion), `test_primitives.cpp` (`smooth_open_path`), `test_properties_panel.cpp` et `test_main_window.cpp` (conversion, tracé de guides et de ruptures, undo/redo, sortie du mode).

Limites connues

- Parcours : une forme dont les lignes se scindent en branches (fond en U, coins d'un éventail) impose une liaison entre branches ; au-delà du plafond du trajet caché (8 mm), c'est un saut — comme le tatami. Dans la bande de chevauchement d'un secteur, de courtes lignes peuvent encore être rejointes en fin de secteur par un saut.
- Les trajets cachés passent parfois sur une zone déjà cousue (même limite que le tatami).
- Pas d'édition des poignées de tangente des guides (seulement les points).
- Aucune validation sur machine réelle.
- Phase 4 (fondu de couleurs) : disponible via **Broderie ■ Fondu de couleurs à deux fils** (voir la section du même nom). Limites : un dégradé linéaire par région ; les deux objets ne sont pas « liés » (modifier l'un ne met pas l'autre à jour) ; un objet de dessus est ajouté en fin de liste, donc cousu après les objets intermédiaires.

Implémentation associée

- `libs/document/include/openstitch/document/embroidery_object.hpp` — `DirectionalFillParams`.
- `libs/stitch_generation/{include/.../directional_fill.hpp, src/directional_fill.cpp}`.
- `libs/stitch_generation/src/generate.cpp` — `generate_directional`.
- `libs/project_io/src/json_serialize.cpp` — type `"directional"`.
- `libs/commands/include/openstitch/commands/project_commands.hpp` — `EditDirectionalFillCommand`, guides dans Translate/Scale.
- `libs/geometry/src/primitives.cpp` — `smooth_open_path`.
- `apps/desktop/main_window_directional.cpp`, `properties_panel.cpp`.

Lettrage (texte brodé)

Le lettrage transforme un texte tapé par l'utilisateur en objets vectoriels (contours des glyphes) et en objets de broderie **éditables**, régénérés comme les autres objets. Il couvre les fiches HP-TXT-001, 003, 004 (en partie), 007 (en partie), 009 et 013 (en partie) de la feuille de route (`docs/roadmap-parite-hatch.md`).

Architecture et justification (ADR lettrage)

Décision : un objet « texte » dans le document + des lettres dérivées matérialisées.

- `document::TextObject` (`Project::text_objects`) porte l'**intention** : texte UTF-8, police (référence), hauteur de capitale, espacements, interligne, alignement, crénage, origine, rotation, couleur, type de remplissage, largeur maximale de satin, densité.
- Les **lettres** sont des objets ordinaires : un `VectorObject` annoté `text_owner` (id du texte) par type de point nécessaire à la lettre, et un `EmbroideryObject` qui le suit. Elles passent donc par toute la chaîne existante (aperçu, sélection, ordre de couture, analyse, export DST) sans cas particulier.
- Les **points ne sont jamais stockés** (ADR-014) : ils sont dérivés par `effective_sequence` comme pour tout objet.
- Éditer le texte **remplace atomiquement** le texte et ses lettres (`commands::SetTextObjectCommand`) : un seul pas d'annulation, les nouvelles lettres reprennent la place d'ordre de couture des anciennes, l'annulation restitue exactement l'état précédent (retouches comprises). `RemoveTextObjectCommand` supprime le texte et toutes ses lettres.

Pourquoi matérialiser les lettres plutôt que les régénérer à chaque ouverture ? (1) Le projet reste affichable et exportable **sans la police** (autre machine, police désinstallée) : seule l'édition du texte est bloquée, avec un message explicite. (2) Aucune nouvelle voie de génération parallèle : zéro risque de contourner `effective_sequence` ou les retouches. (3) Le coût est un contour vectoriel par lettre, déjà le régime des objets issus de l'auto-numérisation.

Contrepartie assumée : modifier le texte **ré-génère** ses lettres ; les retouches faites à la main sur une lettre (nœuds, retouches de points) sont remplacées. Déplacer le texte entier au canevas puis l'éditer conserve la position visible (`lettering::text_displacement`).

Format `.osp` (schéma v6)

Clé `textObjects` (absente si le projet n'a pas de texte) et `textOwner` sur les objets vectoriels. Tous les champs sont lus avec une valeur par défaut ; les énumérations hors plage sont bornées. Un lecteur v5 refuse un fichier v6 (message « version plus récente ») ; un fichier v1..v5 se charge, marqué migré (`LoadInfo::migrated`). Voir `project-format.md`.

Bibliothèque `libs/lettering` (sans Qt)

- `Font` : police TrueType/OpenType via **FreeType** (licence FTL ; l'option GPLv2 n'est pas utilisée), encapsulée, jamais exposée. Contours aplatis (Béziers, tolérance 15 µm), hauteur de capitale mesurée sur « H », crénage de la table `kern`, `inspect_font_file` pour le catalogue.
- `layout_text` : positionnement (crénage, espacement des lettres et des mots, interligne, alignement gauche/centre/droite/justifié, lignes multiples, rotation). Les contours sont unis par la règle non nulle (`geometry::union_nonzero`) : trous des glyphes (A, O, e…) et composants qui se chevauchent (accents composés) sont corrects. Caractère absent de la police : omis et signalé (`glyphe-absent`), jamais un carré silencieux. UTF-8 invalide : U+FFFD.
- `build_text_objects` : choix du point **par morceau de glyphe** (un « ä » = corps satin + deux points tatami) :

Cas	Point	Avertissement
Petite forme pleine sans trou (point, trèma)	tatami	non
Trait moyen < 1 mm	contour	`trait-trop-fin`
Trait moyen > largeur max satin (6 mm par défaut)	tatami	`lettre-trop-large` (si satin demandé)
Sinon	**auto-satin par squelette**, vérifié : couverture mesurée ≥ 92 %	—
Squelette insuffisant (boucles, jonctions complexes)	tatami	`satin-impossible`

Épaisseur moyenne de trait = 2 × aire / périmètre. Tatami / contour forcés par l'utilisateur sont appliqués tels quels. - Petites lettres (HP-TXT-013, partiel) : sous 8 mm de hauteur, pas de compensation de tirage, pas de retrait de bord ni de sous-couche de bord en tatami ; sous-couche centrale du satin seulement si le trait fait au moins 1,6 mm ; contour avec des points de 1,5 mm. - Taille minimale (HP-TXT-009) : `check_text_size` et la règle d'analyse `stitch_analysis::analyze_text_objects` (< 5 mm : satin fragile ; < 3 mm : illisible ; lettres de trait < 1 mm cousues en contour), affichée dans le panneau Analyse.

Interface

Outil **Texte (T)** : un clic sur le canevas pose l'origine et ouvre le dialogue. Menu **Texte** : Nouveau texte, Modifier le texte (F2), Supprimer le texte. Double-clic sur une lettre, ou bouton « Modifier le texte… » de l'inspecteur : même dialogue. Le dialogue (police, hauteur, couleur, type de point, alignement, espacements, interligne, crénage, position, rotation) montre un aperçu des contours et les avertissements ; la création vérifie le satin lettre par lettre. Polices : deux intégrées (Bitstream Vera Sans Gras et Roman, `resources/fonts/`, licence permissive du même dossier) et les polices installées (dossiers de polices du système, énumérés par Qt, fichier passé à la bibliothèque).

Limites connues

- Texte sur ligne droite seulement (bases arc/cercle/chemin : HP-TXT-005, enveloppes : 006).
- Pas de HarfBuzz : crénage `kern` seulement (pas GPOS), pas de ligatures ni d'écritures complexes/RTL (HP-TXT-012 non traité). Les glyphes absents sont signalés.
- Pas d'édition d'une lettre isolée conservée à la régénération (HP-TXT-008), pas de crénage manuel par paire, pas de connecteurs optimisés entre lettres (HP-TXT-014) : l'ordre est celui de la lecture, le routage est celui des objets ordinaires.
- Pas de polices de broderie prénumérisées (HP-TXT-002).
- Auto-satin : les lettres à boucle fermée avec jonction (4, 6, 9, R en Vera) retombent en tatami avec avertissement.
- Les polices à variations (variable fonts) sont lues à leur instance par défaut.

Tests

`tests/unit/lettering/` (police, mise en page, construction, commandes, alphabet complet A–Z, a–z, 0–9, accents français : 100 % des lettres cousues, points dans la boîte du glyphe, déterminisme), `tests/unit/project_io/test_text_persistence.cpp` (v5 → v6), `tests/unit/stitch_analysis/test_text_rules.cpp`, `tests/unit/desktop/test_lettering_ui.cpp` (catalogue, dialogue, outil, annulation).

Retouche des points et de la géométrie

Public : utilisateur avancé, développeur.

Ce qui est éditable aujourd'hui

Niveau	Retouche disponible	État
Image	pile d'opérations non destructive	Implémenté
Segmentation	fusion, suppression, recoloration	Implémenté
Vecteurs	**déplacement** de nœuds	Implémenté
Objets de broderie	paramètres (longueur, densité, angle, type)	Implémenté
Objets de broderie	**changer le type** (contour/tatami/satin), clic droit	Implémenté
Objets de broderie	**orientation** des fils (poignée dans la scène)	Implémenté
Ordre de couture	monter/descendre, figer l'ordre	Implémenté
Points générés	**déplacement** d'un point (mode d'édition dédié), indicateurs `Clean`/`ManuallyEdited`/`Dirty`, abandon des retouches	Partiel (Lot 8.2 — déplacement seul ; Stitch↔Jump et coupe de fil restent cœur seul, sans UI, cf. Lot 8.3)

Régénération vs édition manuelle

Les points sont **régénérés** à chaque modification du document (fonction pure), puis patchés par les retouches manuelles éventuelles de l'objet (`stitch_generation::effective_sequence`, ADR-014). Le **cœur** (Lot 8.0/8.1) implémente le modèle complet : déplacer un point, convertir Stitch↔Jump, ajouter/retirer une coupe de fil, abandonner les retouches d'un objet — quatre commandes annulables (`MoveStitchPointCommand`, `SetStitchPointTypeCommand`, `SetStitchTrimCommand`, `DiscardOverridesCommand`) et une persistance `.osp` (schéma v3). Le **Lot 8.2** expose le **déplacement** dans le canevas (mode d'édition dédié, poignées, indicateurs `Clean`/`Dirty`/`ManuallyEdited`, abandon des retouches) ; `SetStitchPointTypeCommand`/`SetStitchTrimCommand` restent, pour l'instant, uniquement accessibles par usage programmatique (CLI, tests) — prévu au sous-lot 8.3 (voir `docs/lot8-manual-editing-design.md`).

Cas particulier : une séquence **importée d'un DST** est traitée comme la vérité (le DST n'a pas d'objets), elle n'est donc pas régénérée tant qu'aucune image n'est chargée.

Édition de nœuds vectoriels

Sélectionnez un objet vectoriel : ses nœuds s'affichent en poignées. Un déplacement passe par `MoveNodeCommand` (annulable) et **régénère** les points de tout objet de broderie qui suit cet objet.

Limitation : l'ajout/suppression de nœuds, la conversion anguleux/lisse et l'édition de tangentes ne sont pas exposés (voir *Vectorisation*).

Changer le type de points (clic droit)

Un **clic droit** sur une forme ouvre un menu contextuel *Type de points* : Contour cousu / Remplissage tatami / Colonne satin (le type courant est coché). Le changement passe par `SetStitchTypeCommand` (annulable) ; pour le satin, les rails sont reconstruits depuis le contour source (`rails_from_contour`). Le menu donne aussi accès à l'orientation (si tatami) et aux calques.

Orientation des fils dans la scène

Quand un remplissage tatami est sélectionné, une **poignée de rotation** (axe bleu) s'affiche à son centre. La faire glisser réoriente les fils ; l'angle est validé au relâchement (`SetFillAngleCommand`, annulable). Détail d'ergonomie important : la commande est **différée** d'un tour de boucle d'événements, car `refreshImage()` reconstruit la scène et détruirait la poignée pendant son propre événement souris (le même soin s'applique aux poignées de nœuds).

Mode d'édition des points (Lot 8.2)

Un objet de broderie sélectionné et non `Dirty` peut entrer en mode « Éditer les points » (menu Broderie, raccourci **E**, ou barre contextuelle) : mode **exclusif**, lié à l'unique objet actif à l'activation (changer de sélection, charger un autre projet, ou faire passer l'objet à `Dirty` en fait sortir proprement, sans confirmation). Chaque point de couture réel (passe `TopStitch`, type `Stitch` — ni saut, ni sous-couche) affiche une poignée dédiée (couleur distincte des poignées de nœuds vectoriels) ; la glisser construit une `MoveStitchPointCommand` unique au relâchement (différée d'un tour de boucle d'événements, comme la poignée de rotation ci-dessus — même raison). Aucune commande n'est créée si le point n'a pas bougé.

Au-delà d'un seuil de points déplaçables (2000, coût mémoire/événements d'un `QGraphicsItem` par poignée), l'activation est refusée avec un message explicite en barre de statut plutôt que de laisser le mode actif sans aucune poignée visible ; réduire la densité ou la longueur de point de l'objet permet de repasser sous le seuil.

Un objet retouché (`ManuallyEdited`) affiche un indicateur (✎) dans le panneau Document, la barre contextuelle et l'inspecteur, avec un bouton « Abandonner les retouches » (confirmation, `DiscardOverridesCommand` annulable). Un objet `Dirty` (géométrie source changée depuis les dernières retouches, cf. `docs/lot8-manual-editing-design.md` §1) affiche un avertissement (■) à la place : ses retouches ne sont plus appliquées et le mode d'édition lui reste fermé tant qu'elles n'ont pas été abandonnées.

Implémentation associée

- `apps/desktop/node_handle.hpp` — poignée de nœud (réutilisée pour la rotation et pour les points de couture, Lot 8.2).
- `apps/desktop/main_window.cpp` — sélection, poignées, menu contextuel, poignée de rotation, mode d'édition des points (`onStitchEditModeToggled`, `discardOverrides`, gating dans `updateActions`).
- `apps/desktop/canvas_view.cpp` — signal `canvasContextMenu` (clic droit).
- `apps/desktop/document_panel.cpp`, `apps/desktop/properties_panel.cpp` — indicateurs `Clean`/`ManuallyEdited`/`Dirty` (Lot 8.2).
- `libs/commands/.../project_commands.hpp` — `MoveNodeCommand`, `SetFillAngleCommand`, `SetStitchTypeCommand`, `ConvertFillsToTatamiCommand`, et (Lot 8.1, exposées dans l'UI pour le déplacement seul depuis le Lot 8.2) `MoveStitchPointCommand`, `SetStitchPointTypeCommand`, `SetStitchTrimCommand`, `DiscardOverridesCommand`.
- `libs/stitch_generation/include/.../overrides.hpp` — `effective_sequence`, `apply_manual_overrides`, `raw_slice`, `fingerprint`, `classify_edit_state`, et (Lot 8.2) `is_movable_point`, `edit_view`, `classify_all_edit_states`, `refresh_context` (un seul `generate_sequence` interne pour la séquence effective, les états par objet et la vue d'édition d'une cible, réutilisé par `MainWindow::refreshImage` à chaque rafraîchissement).

- `libs/document/include/.../embroidery_object.hpp` — `StitchOverride`, `StitchPointType`, champs `overrides`/`edited_fingerprint`/`edited_point_count`.

Recherche brevets broderie

Public : développeur, mainteneur. État : **R&D préliminaire** (mission 2026-10).

Cette page est l'index et la synthèse de la recherche sur 32 références de brevets de broderie. Les brevets sont traités comme des **sources techniques historiques**, jamais comme du code, une licence, ni une certification de liberté d'exploitation. Les fiches détaillées sont dans quatre chapitres :

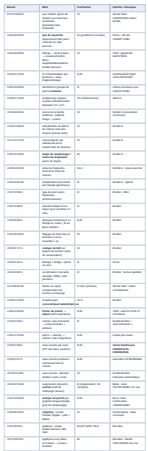
- Satin, squelette et jonctions (EP0761860B1, US6390005B1, US6690988B2, US6397120B1, US6253695B1, US6587745B1)
- Auto-numérisation (famille Goldman/SoftSight, Brother, Pulse, Shima Seiki)
- Sous-couches, contours auto-intersectés, points cachés
- Stippling, analyse de points, appliqué

Limites de la recherche (à lire d'abord)

- Sources : les fiches ont d'abord été écrites d'après des résumés automatiques de pages Google Patents, puis **les 32 brevets ont été relus sur leur texte primaire** (PDF téléchargés depuis Google Patents ; texte extrait, équations et tableaux abîmés par l'OCR relus sur rendu image ; les deux scans japonais lus visuellement). Les corrections sont dans la section « Errata après lecture des PDF » de chaque fiche et priment sur le corps. Les figures n'ont été regardées que lorsqu'une équation en dépendait. Les PDF ne sont pas versionnés dans le dépôt.
- Toute équation est étiquetée « du brevet » ou « notre proposition ». Plusieurs formules sont **ambiguës ou probablement fautives dans le brevet lui-même** : exposants de distance de recherche de la famille Goldman (rayons énormes, signe de l'exposant de fusion de bifurcation non tranché entre relectures, `c` jamais défini), règle de sélection « B>D » de US6167823B1 (choisit le plus éloigné alors que le texte dit le plus proche), Bézier de US6968255B1 (`/3` final, `m_len` non défini), seuil 20 % / 30 % de US6510360B1, `p,q,r` réutilisés dans US6390005B1. Ne rien implémenter littéralement : redériver et calibrer chez nous.
- Les statuts juridiques sont ceux affichés par Google (« hypothèse, pas une conclusion juridique »). Des anomalies sont signalées (US8219238B2 : taxes payées après l'expiration affichée ; US8532810B2 : deux dates de délivrance ; US7386361B2 : deux dates d'expiration). **Aucun avis juridique.**
- Un brevet expiré sert de référence technique ; son expiration n'est pas une preuve de liberté d'exploitation mondiale. Les continuations et brevets ultérieurs cités (US7457683B2, US6502006B1, US5740055A, US6804573B1, US7016756B2, US7587256B2) n'ont **pas** été examinés.
- On ne reproduit ni texte ni figures des brevets : descriptions originales et liens seulement.

Index des 32 références (catégories corrigées)

Les catégories de départ du catalogue étaient souvent inexactes. La colonne « Réel » donne ce que le brevet traite vraiment. Pertinence : H = haute, M = moyenne, B = basse pour OpenStitch.



Familles et doublons

- EP0761860B1 = US6010238A (Shima Seiki).
- Famille Goldman/SoftSight : **six brevets de la liste partagent la même description** (priorité 1998-08-17) et ne diffèrent que par leurs revendications : US6836695B1, US6947808B2, US7016757B2, US8219238B2, US8532810B2, US9200397B2 (plus US6804573, US7016756, US7587256 hors liste).
- Famille Brother « contour auto-intersecté » : US5823127A, US5934209A, US5957068A (CIP), JP3922316B2 (équivalents JP3629854B2, JP3760541B2).
- US6510360B1 cite US6247420B1 ; US6253695B1 et US6247420B1 ont les mêmes inventeurs (Chan, Leung, Wang).
- 32 références ≈ **19 familles distinctes** : le catalogue surestime la diversité.

Ce que les brevets soutiennent, et ne soutiennent pas

Le moteur « satin guidé par squelette » du brief P0 (axe échantillonné, traversées orientées, angle par défaut, guides interpolés, `Lmax`, pas `h`) **n'est contenu dans aucun brevet du corpus** :

Élément du P0	Source
Squelette/axe médian comme référence	EP0761860B1 rev. 1-3
Orientation ■ à l'axe aux extrémités, interpolée entre points de consigne	EP0761860B1 rev. 6-8 (interpolation décrite en mots, forme non précisée)
Orientation ■ à la corde entre intersections de contour au branchement	EP0761860B1 rev. 8
Découpe aux branchements par bissectrices (« closures »), rotation anti-collision	EP0761860B1 rev. 3-5
Contrôle de l'espacement inter-point en courbe	US6390005B1 (mécanisme différent : inset de densité, distance ■ au point précédent)
Échantillonnage régulier de l'axe, 0° par défaut en squelette complexe, interpolation d'angle mod 180° par guides, `Lmax`/y, `h` adaptable	**aucun : notre proposition**

Conséquence juridique et technique : ces éléments sont notre conception propre. C'est favorable à la liberté d'exploitation, mais ils ne s'appuient sur aucune source externe et doivent être validés empiriquement sur machine.

État réel du code (audit 2026-10, HEAD 8dec3f7)

Voir `docs/roadmap-parite-hatch.md` (HP-STI-018) pour le détail. Résumé factuel tiré du code, pas de la doc :

- **Existe** : squelette Zhang-Suen + `SkeletonGraph` (`auto_satin`) ; échantillonnage de l'axe (`compute_column_stations`, 500 µm) ; intersection perpendiculaire à un seul intervalle (`cross_section`) ; découpe des longs points (`SatinSplit`, défaut désactivé) ; points courts ; satin tournant par anneaux `IsoOffsetRing` (expérimental) ; champ d'angle à guides (`directional_fill.cpp`, pondération sur l'angle **doublé**, donc périodique mod 180°) ; oracle de couverture (`satin_coverage`).
- **Absent** : aucune fonction angle(s) par clés le long d'un axe ; aucune intersection droite/région **multi-intervalle** orientée ; aucun pas `h(s)` adaptatif ; aucun lien entre satin et remplissage directionnel ; pas d'appliqué ; pas de suppression/analyse de points cachés ; pas de modification de densité sur points existants ; pas de remplissage radial, concentrique ou spiral.

Objectif directeur : réintégrer l'auto-satin dans l'auto-numérisation

Décision utilisateur (2026-10) : réintroduire un **auto-satin** dans `autodigitize`, fondé sur ces brevets, parce que l'ancien (retiré par `18de427`) était de mauvaise qualité. Il ne revient que s'il est **mesuré meilleur que le repli tatami/contour**, derrière un réglage explicite, avec repli diagnostiqué.

Constat de l'audit et de la revue indépendante (vérifiés dans le code) :

- Le moteur existant couvre déjà l'essentiel des « traversées » : rails et barreaux persistés (`SatinParams`), rééchantillonnage à espacement médian (`resample_by_medial_spacing`), découpe des longs points (`SatinSplit`), mode paramétrique, planificateur SGSD, oracle `satin_coverage`.
- Les éléments du P0 sans source brevet (échantillonnage d'axe, 0° par défaut, interpolation mod 180°, `Lmax`/`y`, `h`) sont notre conception et ne sont à construire que si un corpus prouve un manque.
- La cause probable du retrait est l'**éligibilité trop permissive** et l'acceptation d'une **couverture partielle** (mosaïque satin + pièces de tatami), pas l'absence de traversées orientées (voir RD-PAT-000).

Chaîne retenue : (1) banc de comparaison satin vs repli sur régions réelles (RD-PAT-003) ; (2) critère d'éligibilité strict, extension de l'idée de statistiques de transformée de distance (RD-PAT-004) ; (3) auto-satin minimal sur rubans simples via le planificateur existant, satin entier ou repli entier (RD-PAT-002) ; (4) moteur de traversées orientées seulement si le banc prouve un manque (RD-PAT-001).

Priorités (après revue du plan)

1. **P0 : RD-PAT-003**, banc de comparaison ; **RD-PAT-002**, auto-satin minimal protégé, derrière **RD-PAT-004** (éligibilité, P1).
2. **P1 conditionnel : RD-PAT-001**, moteur de traversées, si le manque est prouvé.
3. **P2** : critère alternatif d'angle de tatami (US8219238B2, un choix automatique existe déjà), analyse des points recouverts (US6633794B2), sous-couche en trois cas (JP3922316B2), élagage et ordre de couture par arbre.
4. **Hors objectif directeur** : appliqué (HP-SPEC-001), remplissage polaire, stippling.
5. **Écartés** : voir `docs/roadmap-rd-brevets.md` (tableau des raisons).

Garde-fous

- Satin automatique : jamais silencieux. Activé par un réglage explicite, protégé par un verdict de couverture, avec repli tatami/contour diagnostiqué (voir « Objectif directeur »).
- Ne jamais supprimer automatiquement une sous-couche ou un trajet structurel parce qu'il est visuellement recouvert (US6633794B2).
- Toute implémentation est indépendante, calibrée sur nos corpus ; aucune constante de brevet n'est reprise sans vérification et justification.

Plan de lots

L'ordre définitif est dans `docs/roadmap-parite-hatch.md` (entrées citant ces brevets) et dans la section R&D complémentaire de cette roadmap.

1. **Documentation et traçabilité brevets** : cette page et ses quatre fiches.
2. **Provenance des rails** : `RailConstructionMethod` (livré).
3. **HP-STI-018 Phase B.5b** : `extend_tip` direction-aware (livré).
4. **Post-mortem de l'ancien auto-satin** (RD-PAT-000, analyse faite).
5. **Banc de comparaison satin vs repli** (RD-PAT-003), puis éligibilité (RD-PAT-004), puis auto-satin minimal protégé (RD-PAT-002).
6. **Moteur de traversées** (RD-PAT-001) seulement si le banc prouve un manque ; **P2/P3** : voir la liste de priorités ci-dessus.

Questions ouvertes

- Statuts juridiques à confirmer hors Google Patents avant toute communication externe ; brevets de continuation non examinés.
- Le satin `IsoOffsetRing`, le remplissage directionnel et le futur moteur de traversées restent sans validation machine réelle.
- `multi_neck`, `two_holes`, `trident` et plusieurs formes multi-branches restent des gaps connus ; aucune famille de brevets ne justifie de masquer ces limites.

Fiches brevets : satin, squelette et jonctions

> Fiches techniques de travail, issues de la lecture de pages Google Patents (résumés automatiques, pas les PDF). Statuts juridiques préliminaires. Voir l'index et les priorités dans Recherche brevets.

Méthode : texte lu via Google Patents (WebFetch, résumé automatique du texte de la page, pas le PDF). Limites : les pages donnent le texte sans numéros de paragraphes ; les équations de US6390005 sont issues d'un OCR dégradé ; les 10 k derniers caractères de US6397120 et ~35 k de US6587745 n'ont pas été relus en intégral (la partie 2 de US6587745 a été relue, elle contient les revendications). Tout ce qui est marqué "FROM PATENT" est une formule présente dans le texte ; "OUR PROPOSAL" est une idée à nous, jamais attribuée au brevet. Statuts légaux = affichage Google, PRÉLIMINAIRE, pas un avis juridique.

Constat transversal important (catégories)

- Les deux brevets censés motiver le "moteur satin guidé par squelette" ne disent PAS ce que la note de cadrage leur prête :
- EP0761860B1 (Shima Seiki) : axe médian + points de branchement + découpe en "closures" + orientation perpendiculaire à l'axe, interpolée. Pas de pas d'échantillonnage, pas de Lmax, pas de modulo 180, pas de 0 degré par défaut, pas de "guides" (au sens courbes utilisateur).
- US6390005B1 (Pulse) : ce n'est PAS un brevet de squelette ; il traite l'espacement inter-points constant d'un satin sur courbe (inset de densité). Catégorie à corriger : "calcul de la position des points d'un satin tournant / compensation de densité dans les courbures".
- US6397120B1 : pas de la "généralisation de squelette" mais de l'UI de gestion des singularités (jonctions) + base d'apprentissage. Catégorie : interprétation des jonctions du squelette / UX.
- US6253695B1 : modification de densité sur un jeu de points existant (post-traitement). Catégorie : édition de densité.
- US6587745B1 : remplissage tatami à lignes courbes par transformation bilinéaire XY<->UT. Catégorie : fill à direction courbe (pas satin).
- US6690988B2 : bitmap -> squelette (Zhang-Suen) -> objets par parcours de graphe. Catégorie correcte.
- Doublons de familles : US6010238A = contrepartie US d'EP0761860B1 (même famille, ne pas compter deux fois). US6587745B1 cite US6390005B1. Aucun autre doublon entre les six.

---

1. EP0761860B1 - Embroidery data generating system

(1) Biblio. Titulaire Shima Seiki Mfg Ltd ; inventeur Kenji Kotaki. Priorité JP 213128/95 du 22-08-1995 ; dépôt 22-08-1996 (EP96306137) ; délivrance 22-03-2000. Désignés : DE, ES, FR, GB, IT. Statut Google : Expired - Lifetime, expiration prévue 22-08-2016 ; déchéances pour taxes dans plusieurs États. Famille : EP0761860A2/A3, US6010238A, KR100406774B1, AU711370B2, DE69607268T2, ES2143720T3, TW396227B. Cité par US6836695B1, US8095232B2, JP2007175087A, JP2013146366A, US9164503B2.

(2) Revendications pertinentes (10) : - 1 (appareil, indépendante) : générateur de contour, générateur de "closures", déterminateur d'orientation, déterminateur de points d'aiguille ; ajoute calcul de lignes caractéristiques internes, recherche des points de branchement/extrémités, lignes de division près des branchements, orientations déduites des lignes caractéristiques. - 2 : lignes caractéristiques = lignes médianes. 3 : lignes de division = bissectrices des espaces entre branches de l'axe médian près du branchement. 4 : intersections lignes de division / contour. 5 : si la 1re intersection n'est pas le contour (autre ligne), on fait pivoter la ligne de division autour du point de branchement jusqu'au contour (Fig. 7, mode 3). - 6 : orientations fixées en au moins 2 points de l'axe d'une closure, au moins une dérivée de l'axe voisin, les autres interpolées. 7 : orientation ~ perpendiculaire à l'axe. 8 : première orientation perpendiculaire à l'axe en un point, seconde parallèle à la droite joignant les deux intersections contour près du branchement, reste interpolé. - 9 : version méthode ; 10 : méthode de broderie. Figures : Fig. 2 (étapes s1-s11), Fig. 3 (parcours de l'axe, t1-t12), Fig. 4 (1 branchement, 3 closures A1-A3), Fig. 5 (2 branchements,

closures B1-B3), Fig. 6 (axe circulaire), Fig. 7 (collision des lignes de division).

(3) Algorithme (reconstruit, sans équation dans le brevet) : - Contour obtenu depuis l'image (s2-s3). Axe médian M (s4) : l'exemple donné est "bandes étroites posées vers l'intérieur depuis le contour jusqu'à contact" ; l'axe est à égale distance des portions de contour opposées, son orientation étant la moyenne des leurs. Variante : rapports de pas non centrés (ex. 2:1) par région, sinon axe médian. - Parcours de l'axe depuis un sommet de départ pour repérer branchements P et extrémités (Fig. 3). - Closures (s7) : à chaque branchement, ligne de division = bissectrice de l'espace entre branches ; intersection avec le contour ; on relie ces intersections au point P. Cela découpe la forme en sous-régions simples (une closure = une région sans fourche). L'utilisateur peut décaler le point de départ d'une ligne de division. Points de coude traités comme branchements ou points d'orientation utilisateur (exemple lettre F). - Orientations (s8) : perpendiculaire à l'axe aux extrémités (inclinaison permise ; tolérance citée ~ +/-10 degrés) ; au branchement, parallèle à la droite joignant les deux intersections contour (ou perpendiculaire à partir de P au contour) ; autres positions interpolées. Axe circulaire (Fig. 6) : plusieurs points d'orientation, un nouveau chaque fois que l'orientation change de plusieurs dizaines de degrés. - Points d'aiguille (s9) : à partir closures + orientations + "embroidery density data" en RAM (aucune formule). FROM PATENT : aucune équation. L'interpolation est mentionnée en mots seulement ("interpolate", "smoothly interpolated") ; la forme (linéaire ou autre) n'est pas donnée.

Réponse précise aux points demandés : - Squelette comme axe de référence : OUI (axe médian, revendications 1-3), mais le brevet ne parle pas d'échantillonnage régulier de l'axe. - Croisements orientés = intersection ligne orientée / région, longueur de point : NON explicite. Le brevet détermine "orientation" et "points d'aiguille" sans détailler la génération des croisements. - Angle par défaut perpendiculaire à la tangente pour squelette simple : OUI en substance (revendication 7, extrémités). Angle absolu 0 degré pour squelette complexe : NON, absent. À la place : découpage en closures (chaque closure est simple) + orientation parallèle à la corde entre intersections au branchement. - Guides = fonction d'angle par interpolation linéaire, périodique mod 180 : PARTIELLEMENT. Interpolation entre orientations en au moins 2 points (rev. 6, 8) ; "linéaire" non précisé ; modulo 180 non mentionné ; "guides" utilisateur au sens courbes absents (seuls les points d'orientation utilisateur existent). - Lmax / découpage en points de longueur y : NON. Pas de longueur de point. - Pas h adaptable : NON (seulement "embroidery density data").

(4) Cas limites : collision de lignes de division (rotation autour de P, rev. 5) ; axe circulaire ; coudes. Coût : non donné ; de l'ordre calcul d'axe + parcours linéaire de l'axe.

(5) Pertinence OpenStitch : MOYENNE. Nouveau pour nous : (a) découpage systématique en closures par bissectrices aux branchements avec rotation anti-collision ; (b) orientation interpolée entre une valeur "perpendiculaire à l'axe" et une valeur "parallèle à la corde" au branchement. On a déjà squelette, ancrage de jonctions, rails/guides. Ce qui motive le moteur guidé par squelette est surtout le principe global ; les détails (Lmax, mod 180, 0 degré) sont notre proposition. OUR PROPOSAL (à ne pas attribuer au brevet) : interpolation linéaire de l'angle sur l'axe avec différence minimale modulo 180 ; 0 degré par défaut pour squelette complexe ; découpe Lmax ; pas h adaptable.

(6) Légal : expiré (taxes + terme 2016) selon Google ; idée seulement, ne rien copier du texte. Préliminaire.

---

2. US6390005B1 - Satin à espacement inter-points constant (Pulse)

(1) Biblio. Inventeurs Benito Chia, Brian Goldberg, Niranjan Mayya, Anastasios Tsonis ; cessionnaire courant Pulse Microsystems (cession 2001-04-02). Priorité/dépôt 1998-05-14 (US 09/079,011) ; délivré 2002-05-21 ; Google : Expired - Lifetime, expiration 2018-05-14. Aucune autre famille listée. Cité par US6587745B1 (Wilcom). Titre exact : "Method of filling an embroidery stitch pattern with satin stitches having a constant interstitch spacing".

(2) Revendications (24 ; indépendantes 1 et 17) : 1 = remplir une forme courbe en satin tournant en gardant un espacement prédéfini, en faisant varier dynamiquement l'inset de densité de chaque point suivant ; 2 = inset fonction de la longueur et de l'angle du point ; 9 = inset dépendant du point précédent et de la variation de forme entre extrémités ; 17 = chaque point créé avec son extrémité à une distance perpendiculaire prédéfinie de la droite du point précédent, mesurée le long de la courbe ; 24 = distance constante. Figures :

3-8 art antérieur, 9-11 boucle, 12-13 modèle de distance, 14-19 inset, 20-22 spirale test.

(3) Algorithme : - Boucle (Fig. 9-11) : point initial avec extrémité p1 sur la courbe ; fixer isd (inter-stitch distance) ; résoudre le paramètre t ; si t dépasse la fin, arrêt ; sinon poser le point suivant avec extrémité p2 à isd de p1 ; recommencer. isd constant ou variable par paire. - FROM PATENT (texte OCR, à revérifier sur le PDF) : (x1,y1,x2,y2) = f(t), t=0 = point précédent ; fx1(t)=a t^3+b t^2+c t+d, fy1=e t^3+f t^2+g t+h, fx2=i t^3+j t^2+k t+l, fy2=m t^3+n t^2+o t+p ; isd = sqrt(p*x1*+*q*y1+r) (distance perpendiculaire de (x1,y1) au segment précédent) ; isd^2 = (pa+qe)t^3+(pb+qf)t^2+(pc+qg)t+(pd+qh+r) ; résolution cubique directe ou approximation par quadratique/linéaire (isd^2 = m t + b). Remarque : les symboles p,q,r sont réutilisés pour deux choses dans le texte (coefficients de distance et coefficient de f), ambiguïté à signaler. - Inset de densité : exemples du brevet pour d=0,4 mm : longueur 0,4 mm / angle 30 deg -> 26,8 % ; 0,28 mm / 45 deg -> 58,6 % ; 0,23 mm / 60 deg -> 100 %. Aucune formule générale lue ; ne pas en déduire une. - Courbe test : spirale (t cos t, t sin t) (Fig. 20-22).

Réponse aux points demandés : squelette, angle par défaut, guides, interpolation, Lmax, découpage : TOUS ABSENTS. Le brevet ne parle que de position des points d'un satin sur contour paramétré et d'espacement. Seul lien : pas h variable/adaptatif = contrôle d'espacement.

(4) Cas limites : forme où t dépasse la fin ; racines multiples du cubique (choix de racine non détaillé lu) ; virages serrés (inset jusqu'à 100 %). Coût : une résolution polynomiale degré <= 3 par point.

(5) Pertinence : MOYENNE-BASSE pour le moteur de squelette, MOYENNE pour la qualité de satin dans les courbes : idée de placer chaque nouveau point à distance perpendiculaire constante du précédent (au lieu d'un pas le long du rail), ce qui évite la sur-densité du côté intérieur. Notre satin a probablement un pas le long des rails ; vérifier nos rails/rungs avant d'adopter. Nouveau : critère de distance perpendiculaire à la droite du point précédent, inset variable.

(6) Légal : expiré (terme 2018) selon Google ; préliminaire.

---

3. US6690988B2 - Description objet depuis un bitmap (VSM / Kaymer, Bysh)

(1) Biblio. Titulaire d'origine VSM Group AB (courant KSIN Luxembourg II selon Google). Priorité GB0120472A 2001-08-22 ; dépôt 2002-08-22 ; publication demande US20030074100A1 2003-04-17 ; délivrance 2004-02-10. Expiré (taxes 2016). Famille : US20030074100A1, GB2379454A/B.

(2) Revendications (22) : 1 = squelette du bitmap, analyse en noeuds et chemins, parcours, série d'objets. 2 = parcours depuis un noeud, chaque chemin parcouru deux fois. 3-5 = 1er passage type de point A (linéaire), 2e passage type B (linéaire ou remplissage). 6 = contour définissant une partie de la frontière des objets du second type. 7-8 = amincissement Zhang-Suen. 9 = agrandissement préalable du bitmap. 15-22 = processeur équivalent.

(3) Algorithme : - Isoler le sujet, bitmap binaire. Agrandir (x5, minimum x3) par interpolation préservant la courbure pour que le squelette reste loin du contour. - Contour : pixel actif avec voisin 4-connexe inactif ; suppression des redondances d'escalier. - Squelette : suppression des pixels "fluff" (connectivité < 2 et < 3 voisins actifs), puis Zhang-Suen à deux sous-passes (conditions de connectivité 1, 2-6 voisins actifs, et deux tests de voisins inactifs ; deuxième sous-passe symétrique) jusqu'à stabilité ; plafond d'itérations (sujet trop épais -> abandon). Re-suppression d'escaliers. - Noeuds/chemins : matrices 3x3 de noeuds (jonctions 3 et 4 branches, noeud isolé/extrémité), chemins stockés en tableaux de pixels. - Filtrage d'artefacts (Fig. 6-8) : despiking (courts chemins vers noeud isolé partant d'un noeud de valeur 3 près d'un angle vif du contour), debowing (chemins qui dévient puis convergent vers un angle vif, distances au contour divergentes), deforking (deux dents + noeud commun supprimés, chemin prolongé jusqu'au contour). Prolongement des chemins terminaux jusqu'au contour ; suppression des chemins courts insignifiants. - Points de contrôle (Fig. 9) : à chaque jonction, intersections du contour avec les bissectrices des chemins, changements de direction brusques proches, points du contour les plus proches ; objets bornés par segments de contour + segments reliant les points de contrôle au noeud ; deux points de contrôle joints directement -> objet triangulaire. - Parcours récursif : choisir un chemin non parcouru, le suivre en génération running ; quand tous les chemins du noeud ont été faits, repasser le dernier chemin en satin/remplissage. Ainsi chaque chemin est vu deux fois et l'aiguille reste dans le contour. FROM PATENT :

aucune équation, que des conditions de pixels.

(4) Cas limites : sujet trop épais (plafond d'itérations), artefacts de squelette (pointes, arcs, fourches), forme avec jonctions de 4. Coût : non indiqué ; Zhang-Suen O(N x itérations) sur bitmap agrandi x5 (donc 25x de pixels).

(5) Pertinence : MOYENNE (squelette déjà fait). NOUVEAU : (a) double parcours de chaque arête (running aller, satin/fill retour) pour un trajet sans coupe ; (b) filtres explicites despike/debow/defork ; (c) points de contrôle de jonction par bissectrices -> même idée que les closures d'EP0761860 ; (d) agrandissement x5 avant amincissement. Comparer à notre élagage SkeletonGraph.

(6) Légal : expiré (taxes) selon Google. Préliminaire.

---

4. US6397120B1 - Gestion des singularités (Goldman / Soft Sight)

(1) Biblio. Inventeur David A. Goldman ; cédé à Soft Sight (1999-12-30), puis Vistaprint/Cimpress. Dépôt/priorité 1999-12-30 (US 09/476,245) ; délivré 2002-05-28 ; Expired - Lifetime, expiration 2019-12-30. Incorpore la demande mère 09/134,981 (non lue, non couverte ici). Pas d'autre famille listée. Cité par US7587256B2, US8706286B2.

(2) Revendications (20 ; indépendantes 1, 7, 13) : détection d'une singularité, génération d'une interprétation initiale via un mécanisme SPM, modification pour meilleur rendu (1) ; validation temps réel (2-3) ; interface "choix suivant" (5) ; base d'apprentissage (6, 17-20) ; ensemble d'interprétations ordonné (15) ; métriques structurelles (16).

(3) Algorithme (aucune équation, tout en prose) : - Pipeline (Fig. 1a, méthode 400) : lissage, segmentation couleur (objets < 6 pixels fusionnés), code chaîne, classification mince/épais par moyenne et écart-type de la transformée de distance le long du squelette (faible écart + faible épaisseur = trait satin), branche épaisse : choix parmi 16 angles de balayage celui qui donne le moins de fragments, puis ordonnancement minimisant les coupes ; branche mince : lignes ajustées, squelette en branches, étiquetage contours extérieur/intérieur/squelette, SPM, lissage des colonnes (variation des normales), points satin aux extrémités des normales. - SPM (Fig. 1b) : à une singularité (jonction), toutes les paires de régions régulières donnent des interprétations candidates ; reconstruction des frontières occultées par minimisation d'énergie (transition lisse) ; classement ; la première est par défaut. Métriques extraites : nombre de branches, vecteurs squelette -> points caractéristiques de contour (CEP), différences angulaires successives, angles internes et excentricité du polygone des CEP, angles de concavité (Fig. 4), épaisseur, stats. Récupération de cas stockés par index, fusion avec les candidats selon similarité et compteur d'usage ; dédoublonnage. UI : icône "suivant" (Fig. 2a), édition de frontière avec validation en temps réel (Fig. 2b), stockage normalisé des corrections. - Idée réutilisable pour nous : choix de 16 angles de balayage minimisant le nombre de fragments (fill), et le critère mince/épais par stat de la distance transform.

(4) Cas limites : interprétations multiples d'une jonction (lettre B, Fig. 2a/3) ; frontières hors contour (rubber-band). Coût : combinatoire en nombre de branches (paires).

(5) Pertinence : FAIBLE-MOYENNE. Nouveau : énumération d'interprétations ordonnées des jonctions + UI "suivante" ; heuristique mince/épais ; sélection d'angle de fill par comptage de fragments. Pas d'algorithme exact reproductible pour l'énergie (non donnée). Apprentissage = hors périmètre pour l'instant.

(6) Légal : expiré (terme 2019) selon Google, mais assignation à Vistaprint ; préliminaire. Une partie du texte (fin de page) non relue.

---

5. US6253695B1 - Changer la densité d'un groupe de points

(1) Biblio. Inventeurs Tik Yuen Chan, King Wa Leung, Wei Wang ; "Individual". Priorité provisoire 60/099,341 du 1998-09-08 ; dépôt 1999-09-08 (09/391,392) ; délivré 2001-07-03 ; Expired - Fee Related (déchéance 2005), expiration 2019-09-08. Pas d'autre famille.

(2) Revendications (7) : 1 (indépendante) reconnaissance de points frontière gauche/droite/normaux, classement, séparation en groupes, calcul d'une lecture de densité ; 2 normal/running ; 3 copier les lignes

relatives voisines pour augmenter ; 4 supprimer pour diminuer ; 5 déplacer ; 6 réarranger les groupes running ; 7 variation d'angle de couture pour préserver la forme.

(3) Algorithme : prose seulement ; pas d'équation ; formule de densité non donnée (dimensions du groupe et nombre de rangées). Les détails des Fig. 6-7 n'étaient pas lisibles dans le texte. (4) Cas limites : points de changement d'angle protégés. Coût non donné. (5) Pertinence : FAIBLE. Opération de post-traitement sur points déjà générés ; or nos points sont dérivés (jamais stockés), donc l'équivalent chez nous est un paramètre de densité en amont. Rien de génériquement nouveau. (6) Légal : expiré. Préliminaire.

---

6. US6587745B1 - Remplissage à lignes courbes (Wilcom)

(1) Biblio. Wilcom Pty Ltd ; inventeurs Alexander Polden, William Brian Wilson. Priorité AU AUPP5766A 1998-09-07 ; dépôt 1999-09-07 (US 09/786,799) ; délivré 2003-07-01 ; Expired - Fee Related (déchéance 2007), expiration 2019-09-07. Famille : WO2000014319A1, EP1112402A4, JP2002524170A. Cité par WO2005061774A1 (Wilcom, fill tournant) etc. Cite US6390005B1.

(2) Revendications (53 ; indépendantes 1 méthode, 19 système, 34 commandes de points) : 1 = zone, type de remplissage, courbe de définition de point ; lignes de remplissage et lignes de motif courbées par la courbe, pas par la forme de la zone. 5-11 = seconde courbe (indépendante, générée par décalage/translation/dilatation, ou dérivée d'une seule courbe), plus de deux courbes pour régions différentes. 12/29 = remplir une zone plus grande puis rejeter. 13-16 = lignes droites calculées puis transformées ; transformation vers un rectangle. 17 = ajustement post-traitement. Figures 3-9 (exemple 1), 10-15 (propriétés), 16-19 (effets, trou triangulaire, courbe unique, trois courbes).

(3) Algorithme (Exemple 1, deux courbes) : 1. Type + paramètres (ex. tatami, longueur 4,0 mm, espacement 2,0 mm, décalages A,B = 0,25). 2. Contour fermé, trous possibles. 3. Deux courbes encadrant la zone. 4. Découpage en quadrilatères non croisés entre les courbes (droites joignant premiers/derniers points et lignes intermédiaires). 5. Contour converti en petits segments avec sommets aux intersections de chaque côté de quadrilatère. 6. Chaque quadrilatère XY -> rectangle UT : U longueur le long de la 1re courbe, T distance le long de la ligne de jonction ; hauteur commune Tmax (moyenne pondérée des hauteurs de tranches), largeur = moyenne des largeurs haut/bas. 7. Contour transformé en UT ; 8. remplissage droit standard en UT (tatami, program split, motif) ; 9. transformation inverse : les droites deviennent des courbes. Équations (texte source dégradé, reconstitué par le résumeur ; à vérifier sur le PDF) : - FROM PATENT : u = *(u-u0)/(u1-u0), t* = (t-t0)/(t2-t0) ; g(u,*t*) = q0 + u*(q1-q0)* + *t*(q2-q0) + u*t*((q0+q3)-(q1+q2)) (inverse UT -> XY, interpolation bilinéaire du quadrilatère q0..q3). - Transformation directe XY -> UT : pas de forme close ; affine A envoyant q0,q1,q2 vers (1,0),(0,0),(0,1) ; A(g)=(u+*k ut, t*+l u*t*) avec (1+k,1-l)=A(q3) ; u*t* solution d'une quadratique ; puis u*, t*. Les définitions de k, l et le choix de racine doivent être confirmés sur le PDF. - Courbe unique : F(t)+h((1-a)n(t)+a e), a dans [0,1] (a=0 décalage pur, a=1 translation), e = médiane du champ de normales ; dilatation P+(1+a)(F(t)+b n(t)-P). FROM PATENT. Compensation de distorsion : pour program split, recalcul des points de coupe comme intersections avec les lignes inverse-transformées. Non couvert dans les parties lues : traitement exact des trous, pondération Tmax, validité des quadrilatères non convexes.

(4) Cas limites : courbes qui doivent encadrer la forme ; continuité entre quadrilatères adjacents (Fig. 15) ; distorsion de longueurs de point. Coût : transformée par sommet O(1), quadratique pour l'inverse.

(5) Pertinence : MOYENNE-HAUTE pour notre fill directionnel (direction par courbe), BASSE pour le satin. NOUVEAU : paramétrage UT par grille curviligne + remplissage droit puis transformation (réutilise tout notre tatami) ; courbes F(t) +/- h n(t). Un point d'attention : la longueur de point change après transformation (compensation à prévoir).

(6) Légal : expiré (2007 déchéance, terme 2019) selon Google ; ne pas copier ; préliminaire.

Synthèse pour le moteur satin guidé par squelette

Ce que les brevets soutiennent réellement : EP0761860B1 (axe médian, bissectrices aux branchements, orientation perpendiculaire interpolée) et US6690988B2 (jonctions + points de contrôle sur le contour, double parcours). Ce qu'ils ne soutiennent pas : pas d'échantillonnage régulier de l'axe, angle 0 degré, modulo 180, guides à interpolation linéaire, Lmax / découpage en points, pas h adaptatif (seul US6390005 touche à l'espacement, sur un autre mécanisme). Ces éléments restent donc notre conception propre (OUR PROPOSAL), ce qui est bon pour la liberté d'exploitation mais ne s'appuie sur aucune source.

Errata après lecture des PDF (2026-10)

Ces fiches ont été écrites à partir de résumés de pages web. Les 32 brevets ont ensuite été relus sur leur **texte primaire** (PDF Google Patents ; équations et tableaux abîmés par l'OCR relus sur rendu image ; scans japonais lus visuellement). Les points ci-dessous **corrigent ou précisent** le corps de cette page ; en cas de conflit, ils priment. Les figures n'ont été regardées que lorsqu'une équation en dépendait.

- **EP0761860B1 (texte complet lu)** : la fiche est confirmée. Précisions : l'axe médian est obtenu en posant des bandes étroites depuis le contour jusqu'à contact ([0017]) ; un cercle parfait réduit l'axe à un point et l'orientation devient radiale ([0019]) ; une tolérance d'environ ±10° autour de la perpendiculaire est jugée sans importance car la couture est en zigzag ([0022]) ; hors de l'extrémité, le changement d'orientation peut être extrapolé ([0022]) ; pour un axe circulaire (trou), l'utilisateur désigne des points d'orientation ou on en crée un nouveau à chaque variation de plusieurs dizaines de degrés ([0066]). Le mot « linéaire » n'apparaît jamais ; aucun modulo 180°, échantillonnage, longueur de point ni pas.
- **US6390005B1 (texte complet lu)** : confirmé. `isd` est la distance perpendiculaire entre l'extrémité du nouveau point et la droite du point précédent ; `t` se résout par un cubique (approximation quadratique ou linéaire possible). Les exemples 26,8 % / 58,6 % / 100 % (30° / 45° / 60°) ne sont accompagnés d'aucune formule générale. La notation `sqrt(p·x1+q·y1+r)` réutilise p, q, r pour deux usages : ne pas la transcrire telle quelle.
- **US6587745B1** : l'application affine `A` envoie `q0, q1, q2` sur `(0,0), (1,0), (0,1)` (la fiche écrivait `(1,0), (0,0), (0,1)`). `A(q3) = (1+k, 1+l)` : le « 1–l » imprimé est une coquille du brevet. La formule bilinéaire `g(u*,t*)` est confirmée ; la formule imprimée de `A` a un signe faux, il vaut mieux résoudre un système linéaire 2×2. Sommets en ordre Z (`q0` et `q3` opposés). Absents du brevet : choix de la racine de la quadratique, poids de `Tmax`, quadrilatères non convexes, coût. La fiche omettait le critère de translatabilité (champ de tangentes tournant de moins d'un demi-tour), les revendications 18 et 44-53 et la compensation de distorsion.
- **US6690988B2** : la deuxième sous-passe de Zhang-Suen imprimée ne répète pas les tests de connectivité (omission probablement rédactionnelle). Seuils absents : plafond d'itérations, longueur « courte » du despike. Critères omis : despike = chemin parti à mi-angle entre deux autres et d'épaisseur décroissante vers le noeud isolé ; defork = segment entre les dents entièrement dans le contour. Le parcours est un retour arrière. La mention GB2379454 n'est pas imprimée sur la page de titre. Revendications 10 à 14 omises.
- **US6397120B1** : aucun cessionnaire imprimé (« Soft Sight / Vistaprint » vient de Google, non confirmé). La « priorité 1999-12-30 » est la date de dépôt ; la mère 09/134,981 n'est qu'incorporée par référence. Seules constantes : 6 pixels et 16 angles.
- **US6253695B1** : la densité d'origine est estimée par « analyse statistique de l'espacement entre rangées » (figure 7), sans formule.

Fiches brevets : auto-numérisation (famille Goldman et Brother)

> Fiches techniques de travail, issues de la lecture de pages Google Patents (résumés automatiques, pas les PDF). Statuts juridiques préliminaires. Voir l'index et les priorités dans Recherche brevets.

Methode et limites (a lire d'abord) - Source unique : pages patents.google.com/patent//en lues via un outil qui renvoie un resume produit par un petit modele (pas le texte brut). Chaque page fait ~190-200 k caracteres ; les ~100 premiers k (description) et la zone suivante (revendications, evenements juridiques) ont ete lus en deux passes. Aucun PDF ni autre miroir n'a ete consulte. - Consequence : les formules sont de seconde main. Elles sont marquees "A VERIFIER SUR LE PDF". Aucune equation n'est inventee ; tout ce qui n'est pas dans le brevet est etiquete NOTRE PROPOSITION. - Les revendications ci-dessous sont des paraphrases courtes. Les dates de statut juridique sont celles affichees par Google (qui precise que ce n'est pas une conclusion juridique) et presentent des incoherences internes signalees ci-dessous. - Aucun texte copie ; aucun code.

0. Vue d'ensemble, doublons et correction de categorie

Famille Goldman / SoftSight (puis Vistaprint, Cimpress). Priorite commune : 1998-08-17 (US 09/134,981). Chaine de continuations indiquee : 09/134,981 (US6836695) -> 09/950,521 (US6804573, hors liste) -> 10/806,880 (US7587256, hors liste) -> 12/507,588 (US8219238) -> 13/490,194 (US8532810) -> 14/011,452 (US9200397). Les demandes 10/806,607 (US6947808), 10/806,629 (US7016756, hors liste) et 10/806,863 (US7016757) sont des continuations deposees le 2004-03-23 de 09/950,521. Les six brevets Goldman de la liste PARTAGENT LA MEME DESCRIPTION (memes figures 1-16, meme texte) ; seules les revendications different. Les lire comme un seul corpus technique, six jeux de revendications.

Categorie "numerisation automatique" : correcte pour les six Goldman (pipeline image scannee -> objets -> points). A nuancer pour les autres : - US8532810 = ordonnancement de fragments de remplissage (optimisation d'ordre / coupes de fil), pas de la numerisation en soi. - US9200397 = simplification de polylignes (filtre triangulaire), brique geometrique. - US6947808 = classification de points de contour pres des noeuds du squelette (ancres satin), brique satin. - US5740056 (Brother) = choix du type de point selon l'epaisseur, depuis un scan ; numerisation automatique tres rudimentaire (traits fins). - US5576968 (Brother) = choix de la direction/angle de remplissage et du point de depart/fin pour UNE region fermee ; plutot "generation de remplissage / planification", pas de la segmentation d'image.

Statuts : tous expires a la date du jour (2026-10) selon toute vraisemblance (duree 20 ans depuis le depot parent de 1998-08-17, plus ajustements de duree), mais voir les anomalies : US8219238 montre des paiements de taxes en 2020 et 2023 alors que Google affiche une expiration en 2019 ; US8532810 montre une date de grant incoherente entre deux lectures (2013-08-21 vs 2013-09-10). A confirmer par un conseil en brevets ; non critique ici car la duree maximale couvre tous ces cas, mais aucun avis juridique n'est donne.

---

1. US6836695B1 (famille racine)

(1) Identification - Titre : Automatically generating embroidery designs from a scanned image. Inventeur : David A. Goldman. Cessionnaire initial : Soft Sight Inc ; puis Vistaprint (2010) ; Vistaprint Schweiz (2013) ; Cimpress Schweiz GmbH (2015). - Depot et priorite : 1998-08-17. Delivre : 2004-12-28 (la page indique aussi "granted 2004-12-09" dans un evenement, ecart de date d'evenement vs publication, normal). - Statut Google : Expired - Lifetime ; expiration ajustee 2021-02-26 (ajustement de duree). Taxes payees : 4e an 2008, 8e an 2012, surtaxe 2014, 12e an 2016. - Juridictions : seuls des documents US ressortent ; je n'ai pas vu d'equivalents EP/JP (non verifie, la liste de famille de la page ne montre que des US). - Cite par : 21 a 66 documents (Brother, Vistaprint, Levi Strauss, Zazzle). Cite : Brother, Pulse Microsystems, Aisin ; litterature non-brevet : Borgefors 1986 (transformee de distance), Kwok 1988 (amincissement).

(2) Revendications utiles - Il n'y en a que deux, independantes : revendication 1 (systeme) et 2 (procede). Elles sont tres larges : entree d'une image couleur en pixels, segmentation en objets, classification mince/epais, localisation et interpretation des regions regulieres et singulieres, calcul d'un ordre de couture optimal pour au moins une colonne, generation d'un fichier. Note : la rev. 2 dit "le systeme" dans son preambule (incoherence redactionnelle). - C'est le brevet "parapluie" ; les details sont dans les continuations.

(3) Algorithme (description commune, numeros de figures 1-16, etapes 402-432) Entree : bitmap 24 bits, 300 dpi, un bitmap par couleur. FROM PATENT sauf mention contraire. a. Segmentation (etape 404) : lissage selectif (moyenne ponderee uniquement dans les voisinages a faible contraste ; zones a fort contraste intactes) ; croissance de regions depuis des graines a faible contraste ; absorption des pixels non classes vers un objet voisin selon proximite et similarite de couleur ; suppression des objets de moins de 6 pixels (leurs pixels reaffectes au plus proche objet plus grand). b. Codage en chaine et DT (etape 406) : un seul balayage raster produit les contours fermes (exterieur + trous) en chaine a 8 directions (fig. 13). Transformee de distance chamfer entiere "3-4" calculee pendant la generation des contours ; distance normalisee = valeur DT / 3. Amincissement -> squelette (le squelette est calcule en meme temps). c. Classification (etape 408) : statistiques de la DT sur les pixels du squelette : max, moyenne mu, ecart-type sigma. Exemple de regle donne dans le texte : "objet predominamment regulier si 2*sigma < mu < max/2" (FROM PATENT, exemple ; formule rendue par un resume, A VERIFIER SUR LE PDF). Un objet est "mince" si max et mu sont inferieurs a des seuils predefinis (seuils non chiffres, "choisis experimentalement"). Sinon epais. La classification par branche est mentionnee mais "non utilisee actuellement". d. Voie epais (remplissage) : approximation polygonale par filtrage triangulaire (voir US9200397) ; choix de l'angle parmi 16 angles de balayage = celui qui donne le moins de fragments (voir US8219238) ; fragmentation par balayage de lignes modifie ; ordonnancement recursif des fragments (voir US8532810) ; entree/sortie sur le contour exterieur ; ligne mediane par fragment pour les deplacements en point droit. e. Voie mince (satin) : voir US6947808 pour ancres, fusion, normales, lissage, chemin. f. Sortie : angles, fragments, points de depart/sortie, colonnes.

Formules de recherche (FROM PATENT mais seconde main, A VERIFIER) : - Distance de recherche d'ancre de jonction : (valeur DT du noeud) x 10^(2*pi - theta), theta = angle entre les deux vecteurs de branches. Une premiere lecture avait donne "(DT)x10/(2pi-theta)" ; les deux lectures divergent, donc ne pas implementer avant d'avoir lu le PDF. - Distance de recherche de fusion de bifurcation : (valeur DT) x 10^(*cpi + theta), "c" illisible. Constante 10 "choisie empiriquement". - Critere moindres carres des normales : pas de formule explicite ; angle du nouveau segment vs normales du contour qu'il relie + terme proportionnel a la largeur moyenne de colonne.

(4) Cas limites et cout : bruit < 6 px, objets a trous, objets formes de ruban quasi regulier vs lettres (serifs codes en dur, ex. "A"), points sharp < ~30 degres, coupure de colonne < ~45 degres. Complexite : DT et squelette lineaires en pixels ; fragmentation lineaire x 16 angles ; planification de chemin recursive (exponentielle dans le pire cas, non borne par le texte).

(5) Pertinence OpenStitch : MOYENNE a faible comme exigence, mais utile comme feuille de route. OpenStitch a deja segmentation CIELAB, regions, squelette, rails/rungs, tatami. Ce que la description apporte de neuf : la classification mince/epais par statistiques de DT sur le squelette (regle simple, implementable independamment), qui pourrait nourrir la "satinabilite" ; mais la politique du projet interdit le satin automatique en autodigitize, donc seulement comme avertissement/suggestion.

(6) Caveats : expire, mais verifier avec conseil. Ne rien copier du texte. Les formules sont probablement des constantes empiriques ; en reimplementer des equivalents calibres par nos propres donnees (NOTRE PROPOSITION).

---

2. US6947808B2 (revendications : classification des points de contour pres des noeuds du squelette)

(1) Identification : meme famille ; demande 10/806,607 deposee 2004-03-23 ; publication US20040243272A1 (2004-12-02) ; delivre 2005-09-20 (evenement grant 2005-08-31) ; Google : Expired - Lifetime, expiration ajustee 2018-11-15. Taxes : 4e an payee 2009 avec surtaxe ; la lecture est tronquee (derniers evenements non vus). Continuation de 09/950,521. Cessions identiques aux autres.

(2) Revendications : indep. 1 (procede), 14 (systeme), 27 (support). Procede : obtenir les donnees de contour d'un objet, classer automatiquement les points de contour proches d'un noeud du squelette (noeud d'extremite ou de jonction), generer les donnees de broderie a partir de ces points classes. Dependantes : relation avec squelette (2), ancres d'extremite/de jonction (3-4), singularite/discontinuite/concavite (5), suppression de singularite (6), type/angle/chemin de point (7), planification de chemin avec noeud de depart choisi par l'utilisateur et parcours recursif (8-10), normales de trait entre points de contour opposes, supprimer les discontinuites (11-13), image bitmap (40-42).

(3) Algorithme satin (etapes 422-432) : - Stockage : contours externes/internes en polylignes ; branches du squelette en segments minimaux ; noeuds avec degre et connexions. - Ancre d'extremite : prolonger le bout du squelette selon sa direction d'une longueur proportionnelle a l'epaisseur locale (DT moyenne sur la fin de branche) ; trouver le sommet de contour le plus proche ; si son angle de courbure < ~30 degres -> pointe unique. Sinon chercher, de chaque cote, jusqu'a 4 sommets voisins et dans une distance max, une paire gauche/droite dont le vecteur de liaison correspond le mieux a l'angle de depart (direction) et a l'epaisseur locale (norme). - Ancre de jonction : angles delimitant les branches consecutives en chaque noeud de degre >= 3 ; chercher des concavites du contour dans une distance fonction de la DT du noeud et de l'angle theta (formule ci-dessus) ; apparier chaque concavite du squelette a la concavite de contour dont la direction est la plus proche de son centre angulaire. - Fusion d'artefacts : deux noeuds qui se projettent sur la meme paire de sommets caracteristiques -> fusionnes a leur milieu. Regles codees en dur pour empattements (serifs) : noeud de degre 3 avec deux branches courtes finissant en noeuds de degre 1 dont les ancres d'extremite sont reliees par un seul segment de contour, jonction non concave aigue. - Codage : singularites (jonctions) resolues en reconstruisant les frontieres occultees par minimisation d'energie (pas de formule dans le texte lu). Normales de trait : segment de depart au noeud, puis chaque normale choisie par moindres carres (angle vs normales de contour + terme de longueur lie a la largeur moyenne), interpolation entre sommets. - Lissage de colonnes : ajuster une normale incoherente avec ses voisines ; ajouter une normale si un point de contour est orphelin ; lisser premiere/derniere normale ; couper la colonne aux virages < ~45 degres (cousues separement). - Chemin : parcours recursif depuis un noeud d'entree ; une branche n'est terminee qu'apres les autres branches du noeud ; boucles cousues en continu ; score d'un chemin = longueur cumulee des autres colonnes cousues pendant les interruptions d'une longue colonne ; garder le moindre score ; au moins un chemin sans coupe de fil est garanti (selon une lecture). - NOTRE PROPOSITION : le squelette et la detection des extremites/jonctions existent deja dans OpenStitch (junction anchoring). Rien d'a coder tel quel.

(4) Cas limites/cout : jonctions en Y, T, X, serifs ; boucles ; lignes qui se croisent a angle aigu (fig. 10). Cout dominant : appariements locaux, faible.

(5) Pertinence : MOYENNE. OpenStitch possede deja ancrage de jonction et rails/rungs ; le seul apport possible est de comparer nos heuristiques (4 sommets, 30/45 degres, serifs) aux notres pour trouver des cas oublies, et la regle de chemin "score = longueur des colonnes cousues pendant l'interruption" pour le routage satin. Satin auto interdit par politique -> usage en "assistance" uniquement.

(6) Caveats : expire ; description identique a celle de la racine, donc risque nul de "doublon" a analyser deux fois.

---

3. US7016757B2 (revendications : classification par regularite + type de point)

(1) Identification : demande 10/806,863, deposee 2004-03-23, publiee US20040243274A1 ; delivre 2006-03-21 (evenement 2006-03-01). Expire - Lifetime, expiration anticipee 2018-08-17 selon Google. Taxes : 4e an 2009, 8e an 2013 (surtaxe 2014), 12e an 2017 (coherent avec un brevet maintenu jusqu'a son terme). Continuation de 09/950,521.

(2) Revendications : indep. 1 (procede), 10 (systeme), 19 (support). Procede : recevoir des donnees de transformee de distance, classer un objet de l'image en extrayant une "caracteristique de regularite" liee a la variation d'epaisseur, generer les donnees de broderie selon cette classification. Dependantes : statistiques de DT (2), regularite derivee des statistiques (3), ou de la DT + donnees de squelette (4), objet irregulier ou predominamment regulier (5), type de point choisi selon la regularite (6), remplissage ou colonne (7), objet parmi plusieurs ou partie d'un (8-9). Les rev. 11-18 et 20-27 les dupliquent.

(3) Algorithme : voir racine, etape c. FROM PATENT : statistiques max, mu, sigma de la DT sur les pixels du squelette ; regle exemple 2*sigma < mu < max/2 pour "regulier" (A VERIFIER). Mince si max et mu sous seuils (valeurs non donnees). Epais sinon -> remplissage ; mince et regulier -> satin (colonnes).

(4) Cas limites : objets mixtes (une branche epaisse + une branche mince) ; la classification par branche est mentionnee mais non utilisee dans l'implementation decrite. Cout : un parcours des pixels du squelette.

(5) Pertinence : MOYENNE. Idee implementable independamment : utiliser mu/sigma/max de la distance au bord le long de l'axe median pour classer "ruban regulier / zone large". OpenStitch a deja squelette + satinabilite ; apport = un critere supplementaire peu couteux, NOTRE PROPOSITION : seuils a calibrer sur nos propres jeux de test, ne pas reprendre les valeurs du brevet.

(6) Caveats : expire ; description non protegee par le simple fait que des idees generiques (statistiques d'epaisseur) existent aussi dans le domaine public (ex. Borgefors/Kwok cites dans le dossier).

---

4. US8219238B2 (revendications : angle de remplissage = moins de fragments)

(1) Identification : demande 12/507,588 deposee 2009-07-22 ; publiee US20100191364A1 (2010-07-29) ; delivre 2012-07-10 (evenement "patent grant" 2012-06-20). Continuation de 10/806,880 (US7587256). Cessionnaire : Vistaprint Technologies -> Vistaprint Schweiz -> Cimpress. Google : "Expired - Lifetime", expiration 2019-10-13 (ajustee). ANOMALIE : taxes 4e an (2016), 8e an (2020-01-10) et 12e an (2023-12-14) enregistrees ; paiement de taxes ne prolonge pas la duree, mais l'ecart avec "expire 2019" est a faire verifier. Une duree de 20 ans a partir du 1998-08-17 donne 2018 + ajustements ; les taxes pourraient etre un artefact. Sans certitude : statut a confirmer aupres d'un conseil.

(2) Revendications : indep. 1 (procede), 9 (systeme), 18 (support). Procede : identifier un objet contigu dans les donnees d'image ; tester plusieurs angles de couture ; compter les fragments par angle ; choisir l'angle donnant le moins de fragments ; generer les donnees. Dependantes : objet epais (2) ; fragment = sous-region cousable en continu (3), separee par coupe de fil ou points droits (12/21 d'apres la lecture) ; angles = angles de balayage de l'image (4) ; point d'entree/sortie des fragments sur le contour (5-6) ; ligne mediane du fragment (7) ; point droit ou coupe entre sortie d'un fragment et entree du suivant (8).

(3) Algorithme (FROM PATENT) : 16 angles de balayage candidats ; pour chaque angle, fragmentation par balayage de lignes modifie ; nombre de fragments compte ; angle du minimum retenu (fig. 6a/6b : exemple 2 fragments vs 1). Ligne mediane = milieu entre vertex gauche et droit. Details de la fragmentation (traitement des concavites/trous, regle de creation d'un nouveau fragment) : voir le texte complet, non lu en detail ici. NOTRE PROPOSITION : le critere "minimiser le nombre de composantes connexes des lignes de balayage" est generalisable ; pour OpenStitch, tester N angles (pas forcement 16) et minimiser fragments, puis desempartager par longueur de saut.

(4) Cas limites : egalite de fragments entre angles ; formes avec trous ; angle contraint par l'utilisateur. Cout : 16 x (fragmentation lineaire en nombre de lignes de balayage).

(5) Pertinence : ELEVEE pour tatami. OpenStitch a un fill directionnel et l'optimisation de l'ordre des points ; l'angle automatique base sur le comptage de fragments est genuinement nouveau et simple (autodigitize tatami/contour).

(6) Caveats : expire selon Google (voir anomalie) ; l'idee generale "choisir l'angle minimisant les coupes" est peu inventive isolee, mais ne rien copier du texte.

---

5. US8532810B2 (revendications : fragments recursifs + ordre de couture)

(1) Identification : demande 13/490,194 deposee 2012-06-06 ; publiee US20120245726A1 (2012-09-27) ; delivre en 2013 (deux dates divergentes dans mes lectures : 2013-08-21 evenement "patented case" et 2013-09-10 publication ; la seconde est probablement la date officielle de publication). Google : Expired - Fee Related ; expiration anticipee 2018-08-17 en premiere lecture, mais evenements : taxes 4e an 2017, 8e an 2021-02-11, rappel 2025-04-28, EXPIRE pour non-paiement 2025-10-13 (lapsed ; aucune reinstauration vue). Interpretation : le brevet semble avoir ete maintenu bien au-dela de 2018, ce qui est incoherent avec une duree de 20 ans depuis 1998 ; soit une date d'expiration terminale ajustee, soit une erreur de donnees Google. DANS TOUS LES CAS le statut a CONFIRMER aupres d'un conseil ; pas de conclusion ici.

(2) Revendications : indep. 1 (procede), 5 (appareil), 9 (support). Procede : segmenter en objets ; diviser recursivement un premier objet en fragments en enregistrant COMMENT chaque fragment a ete cree (un petit fragment est relie a son fragment parent plus grand au niveau de son bord / ligne de balayage de depart) ; definir l'ordre de couture a partir de cet historique de creation pour reduire les coupes de fil ou augmenter la connectivite de chemin ; sortir le fichier. Dependantes : ligne mediane par fragment (2), trajet de deplacement entre fragments via cette mediane (3), plusieurs petits fragments issus d'un grand (4) ; rev. 13/14 : minimiser coupes / maximiser connectivite.

(3) Algorithme : FROM PATENT, structure arborescente des fragments (arbre de scission) avec lien enfant -> parent au premier bord/ligne ; l'ordre de couture suit l'arbre pour chainer les fragments voisins avec points droits au lieu de coupes. Details de l'ordre exact (DFS ? priorites ?) : non verifies dans le texte lu ; A VERIFIER SUR LE PDF. NOTRE PROPOSITION : representer chaque region tatami comme un arbre de fragments, cout de saut = distance entre sortie et entree, mediane pour le trajet interne.

(4) Cas limites : fragments sans parent commun, trous, region disjointe. Cout : lineaire en nombre de fragments pour l'arbre.

(5) Pertinence : MOYENNE a ELEVEE. OpenStitch optimise deja l'ordre de points ; l'arbre de scission avec trajet par ligne mediane peut reduire les sauts et les coupes dans le tatami complexe.

(6) Caveats : voir statut ; revendication 9 sans lien explicite avec l'enregistrement de la creation (structure plus large que 1 et 5).

---

6. US9200397B2 (revendications : simplification de contour par triangle)

(1) Identification : demande 14/011,452 deposee 2013-08-27 ; publiee US2014/0094952A1 ; delivre 2015-12-01 (lecture alternative d'evenement : 2015-11-11). Cessionnaire initial indique : Cimpress Schweiz GmbH. Statut Google : Expired - Fee Related ; expiration anticipee 2018-08-17 en premiere lecture, mais evenements : rappel 2019-07-22, EXPIRE pour non-paiement des taxes 2020-01-06, "lapsed" effet 2019-12-01. Il n'y a pas de reinstauration vue : lapsed 2019-12-01 (donc brevet tombe avant le terme probable). Interpretation prudente : aucun doute pratique, expire/lapsed ; a confirmer.

(2) Revendications : indep. 1 (procede), 7 (appareil), 13 (support). Procede : identifier les sommets dans un ensemble de points de chaine (contour d'un objet) ; pour chaque sommet, former le triangle avec ses voisins et comparer sa hauteur a un seuil ; supprimer le sommet si la hauteur est plus petite ; generer les donnees de broderie avec les sommets restants. Dependantes : codes de chaine differentiels (2) ; sommets = points dont le code differe du precedent (3) ; sommet fixe sur le bord exterieur ou un objet voisin change (4) ; egalite de hauteur -> supprimer le sommet le plus proche d'un point de reference (5) ; angle de couture et fragments bases sur les sommets filtres (6).

(3) Algorithme (FROM PATENT, fig. 5) : filtrage triangulaire iteratif sur sommets de chaine ; seuil de hauteur NON chiffre. Sommets fixes aux jonctions entre objets adjacents (frontieres partagees, fig. 15b) pour eviter les trous/chevauchements entre regions de couleurs voisines. NOTRE PROPOSITION : c'est essentiellement un critere de type Visvalingam-Whyatt / Ramer-Douglas-Peucker local ; OpenStitch a deja une simplification de chemins dans `geometry`. Elements genuinement utiles : (a) sommets fixes aux frontieres partagees et (b) departage deterministe des egalites.

(4) Cas limites : egalites, boucles fermees courtes, sommets fixes consecutifs. Cout : O(n) par passe, O(n log n) avec file de priorite.

(5) Pertinence : FAIBLE a MOYENNE. Tout a fait generique ; seule la garantie de frontieres partagees apporte quelque chose si OpenStitch ne l'a pas deja dans la vectorisation (a verifier dans `vectorization`).

(6) Caveats : brevet lapsed ; la technique est generique et anterieure (algorithmes classiques de simplification), mais ne rien copier.

---

7. US5740056A (Brother : type de point selon epaisseur du trait)

(1) Identification : titre "Method and device for producing embroidery data for a household sewing machine". Inventeur : Masao Futamura. Cessionnaire : Brother Industries (Brother Kogyo KK). Demande 08/545,436 deposee 1995-10-19 ; priorites 1994-10-11 (parente US 08/321,222, devenue US5515289) et 1994-10-19 (JP 253599/94). Delivre 1998-04-14. Statut : Expired - Lifetime ; expiration anticipee 2014-10-11. Famille : continuation-in-part de US 08/321,222 ; je n'ai pas verifie d'autres membres (EP/JP possibles, non confirmes).

(2) Revendications (19) : indep. 1 (dispositif), 5 (dispositif), 10 (dispositif), 15 (procede). Idee : image d'entree -> lignes fines -> une mesure de "forme/epaisseur" par composante -> donnees de broderie selon cette mesure + lignes fines. Rev. 2 : mesure basee sur le nombre de passes d'amincissement ; rev. 3/10 : conversion en distance ; rev. 4 : largeur du zigzag selon la mesure.

(3) Algorithme (FROM PATENT, valeurs chiffrees) : - Mode 1 : binarisation (scanner a main, noir=1). Extraction des composantes connexes (4 ou 8-connexite). Amincissement sequentiel jusqu'a 1 pixel d'epaisseur, en comptant le nombre de passes N par composante. Vectorisation de la ligne fine : echantillonnage a intervalle, test du vecteur vs vecteur de reference -> segments courts. - Regle de type de point par N : 1 <= N < 3 : triple point (point de renfort) ; 3 <= N < 5 : zigzag de 1,2 mm ; N >= 5 : zigzag de 1,8 mm. - Zigzag : aiguille alternee de part et d'autre du segment a +/- demi-largeur ; triple : positions successives le long du segment. - Mode 2 : conversion en distance (bord = 1, croissant vers l'interieur), valeur max stockee par composante ; type de point determine par ce max a la place de N ; variantes citees : proportion de pixels au-dessus d'un seuil, moyenne des grandes valeurs. - Aucune equation formelle dans le texte ; seuils ci-dessus uniquement.

(4) Cas limites : composantes d'epaisseur variable (un seul N par composante : perte d'information) ; connexite. Cout : lineaire.

(5) Pertinence : FAIBLE (technologie 1994, trop simple). Deja largement depasse par la DT + squelette d'OpenStitch. L'idee "epaisseur mesuree -> largeur de satin / type de point" est la plus proche de notre domaine ; NOTRE PROPOSITION : utiliser la DT du squelette pour proposer largeur de rail, sans satin automatique.

(6) Caveats : expire (2014), faible risque.

---

8. US5576968A (Brother : direction de couture et point de depart/fin d'une region)

(1) Identification : titre "Embroidery data creating system for embroidery machine". Inventeurs : Masahiro Mizuno, Masao Futamura, Yukiyoshi Muto. Cessionnaire : Brother Industries. Demande 08/394,633 deposee 1995-02-27 ; priorite JP6-117919, 1994-05-31 ; delivre 1996-11-19. Statut : Expired - Lifetime ; expiration anticipee 2015-02-27. Aucune famille listee ; cite par 25 documents (Brother, Janome, Softsight/Goldman).

(2) Revendications : indep. 1 (systeme, critere de distance), 11 (systeme, critere d'aire), 17 (procede). Idee : machine qui coud seulement dans deux sens opposes ; region fermee ; point d'arrivee (= depart) ; frontiere perpendiculaire a la direction de couture passant par ce point ; on choisit le cote dont le point le plus eloigne est le plus loin (rev. 1) ou dont l'aire est la plus grande (rev. 11), puis la direction de couture adaptee ; la region est coupee par une ligne reelle passant par le point avec le meme angle et des sens de couture opposes de chaque cote ; le depart coincide avec l'arrivee (une seule coupe de fil double). Rev. 20 : chaque sous-region cousue depuis le point de contour le plus eloigne du point d'arrivee.

(3) Algorithme (FROM PATENT, mode 1) : contour ferme P0..Pn ; point d'arrivee = point le plus bas en y ; ligne EL verticale ; LX = |x(arrivee) - x(point le plus a gauche)|, RX = |x(point le plus a droite) - x(arrivee)| ; si LX > RX direction A (gauche->droite, angle 45 degres sens horaire par rapport a la frontiere) sinon B (sens anti-horaire). Ligne NL a 45 degres passant par l'arrivee ; de chaque cote, trouver le point le plus eloigne, point droit de l'arrivee vers lui, puis remplissage finissant a l'arrivee. Mode 2 : meme chose avec les aires SL/SR (formule non reproduite dans le texte lu : "image" absente) ; si SL > SR direction A sinon B. Angle de couture de reference : 45 degres par rapport a la frontiere (esthetique).

(4) Cas limites : NL passant par une arete (une sous-region vide) ; polygones concaves (les partielles peuvent etre disjointes ; non traite en detail). Cout : O(n) pour les extremums.

(5) Pertinence : FAIBLE. Contrainte machine specifique (couture dans deux sens). Idee generalisable : choisir depart = arrivee pour reduire les coupes de fil, et couper en deux sous-regions de sens opposes ; marginal pour tatami moderne.

(6) Caveats : expire (2015).

Synthese : ce qui est reellement nouveau et implementable pour OpenStitch

1. (ELEVEE) Choix de l'angle tatami en testant N angles et en minimisant le nombre de fragments (US8219238). Simple, deterministe, pur fonction d'un snapshot immuable -> compatible avec la regle "generation = fonction pure".
2. (MOYENNE-ELEVEE) Arbre de scission des fragments et ordre de couture a partir de l'historique de creation, trajet interne via ligne mediane (US8532810).
3. (MOYENNE) Classification epais/mince/regulier par statistiques de la DT sur le squelette (mu, sigma, max) comme avertissement ou suggestion (US6836695, US7016757). Pas pour auto-satin.
4. (FAIBLE-MOYENNE) Sommets fixes sur frontieres partagees lors de la simplification (US9200397), a verifier vs `vectorization`.
5. (FAIBLE) Heuristiques d'ancres satin (US6947808) : seulement comparaison avec notre code existant ; US5740056 et US5576968 : interet historique.

Points a verifier au PDF avant toute implementation : regle exacte de classification, formules 10^(...) des distances de recherche, details de la fragmentation et de l'ordre de couture, seuil de hauteur du filtre triangulaire, statuts juridiques (surtout US8219238 et US8532810).

Suite : brevets Brother, Pulse et Shima Seiki (lot B2)

Source unique : pages Google Patents (/en) lues via WebFetch (résumeur automatique, donc pas le texte brut). Limites générales : - Le texte renvoyé est un résumé produit par un petit modèle ; les formules rendues en image (##EQU##) sont absentes. - Pour 6356648, 5343401, 4849902, 6937919, 7386361 la page dépasse 100 000 caractères ; j'ai lu la suite par offset, mais certaines parties restent non vérifiées (signalé ci-dessous). - Tous les statuts affichés sont « Expired - Lifetime » (ou Fee Related) selon Google, qui précise que c'est une hypothèse et non une conclusion juridique. - Convention : DU BREVET = fidèle à la source ; NOTRE PROPOSITION = idée d'implémentation indépendante, jamais attribuée au brevet.

Vue d'ensemble et correction de catégorie

Brevet	Catégorie annoncée	Catégorie réelle
US5839380A	auto-digitizing	Semi-auto : bitmap -> découpe interactive en zones -> type de point par zone (fill vs ligne médiane). Pertinent.
US6356648B1	auto-digitizing	Prétraitement bitmap : amincissement, suivi de contour, élagage des branches, détection de boucles. Pertinent.
US5283747A	auto-digitizing	PAS de la numérisation : ordonnancement/routage de points de bâti (running stitch) dans un graphe de sections polygonales pour éviter les fils croisés sur machine multi-aiguilles. Lié à l'optimisation d'ordre.
US5343401A	auto-digitizing	PAS de la numérisation : système de poinçonnage interactif (PC) et transformation de lettrage « bridge » (déformation de texte sur arc). Transformation de contours.
US4849902A	auto-digitizing	Numérisation manuelle assistée (1986) : image caméra, saisie de points de contour au crayon optique, découpe en blocs polygonaux, satin par blocs. Historique.
US7386361B2	auto-digitizing	PAS de la numérisation auto : dessin à main levée à stylet, points d'aiguille générés en temps réel selon pression/courbure, plus simulation d'éclairage.
US6937919B1	auto-digitizing	PAS de la numérisation : génération de motifs de remplissage concentriques/radiaux/elliptiques dans une région. Nouveau type de remplissage.

Doublons de famille dans la liste : aucun doublon strict entre les 7. Liens de parenté : 5839380, 6356648 et 6937919 sont tous Brother ; 6356648 et 6937919 partagent l'inventeur Taguchi et 5839380 / 6937919 partagent Muto. 6937919 cite US6587745B1 (Wilcom, remplissage ligne courbe). US4849902A : voir réserve de fiabilité plus bas.

---

1. US5839380A — Method and apparatus for processing embroidery data

(1) Titulaire Brother Industries ; inventeur Yukiyoshi Muto. Priorité 27/12/1996 (JP 8-350275), dépôt 17/12/1997 (US 08/991,873), délivrance 24/11/1998. Statut : Expired - Lifetime (hypothèse Google), expiration prévue 17/12/2017. Famille : JPH10179964A (publié 07/07/1998). Classes D05B19/00-08. Cité par ~96 documents (dont Softfoundry, Goldman « Automatically generating embroidery designs from a scanned image »). Cite JPH07136361A (Brother, choix auto du type de point par transformée en distance + statistiques) et US5559711A.

(2) Claims utiles : indépendantes 1 (méthode) et 6 (appareil) ; 2-3 (fill = contour + points intérieurs ; ligne = amincissement + points le long), 4-5 (fill = satin ou tatami ; ligne = zigzag ou running), 7 (pixels connexes de même densité), 8-9 (division interactive par l'opérateur avec affichage des lignes de coupe), 10-13 (équivalents appareil). Figures 3, 4A/4B (extraction), 6, 7A/7B (drapeaux de frontière), 8A/8B, 9A/9B. Je n'ai pas le texte exact des revendications, seulement leur paraphrase.

(3) Algorithme (DU BREVET, sans aucune équation) : - Bitmap binaire. Deux plans de drapeaux par pixel : « examiné » et « frontière ». - L'opérateur trace des lignes de coupe (typiquement en travers des parties allongées) ; rastérisation par tracé de droite standard, drapeau frontière = 1 sur ces pixels. - Extraction de composantes connexes par balayage gauche-droite/haut-bas : pixel noir, non examiné, non frontière = graine. Propagation récursive en 4-voisinage (variante 8-voisinage). Un pixel frontière est inclus dans la zone mais la propagation depuis lui ne continue que vers d'autres pixels frontière : la zone ne déborde pas de l'autre côté. Variante : mettre à 0 la densité des pixels de la ligne. - L'opérateur affecte un type de point à chaque zone. Si type « remplissage » (satin/tatami) : suivi de contour -> chaîne fermée 1 pixel -> vectorisation par échantillonnage de points significatifs -> points de couture intérieurs (blocs de 4 points renvoyés à une méthode antérieure). Si type « ligne » (zigzag/running) : amincissement standard par suppression itérative de pixels de bord -> vectorisation -> points le long de la ligne médiane. - Variante citée : choix automatique du type via transformée en distance et statistiques (renvoi à JPH07136361A, non lu). - Pas de paramètres numériques autres que voisinage 4/8 et largeur 1 pixel.

(4) Cas limites/coût : récursion pixel par pixel (risque de dépassement de pile, à faire itératif). Division manuelle donc non déterministe vis-à-vis de l'utilisateur. Rien sur la qualité de la jonction entre zones adjacentes (chevauchements, trous).

(5) Pertinence OpenStitch : MOYENNE-FAIBLE. Les composantes connexes et le contour existent déjà ; l'idée « une zone allongée devient une ligne médiane, une zone épaisse devient un fill » est déjà couverte par l'extraction de squelette et la satinabilité. Ce qui est éventuellement nouveau : le mécanisme de coupe par ligne-frontière (couper une région par une polyligne utilisateur avant vectorisation) comme outil d'édition ; et la classification fill vs ligne par géométrie (la source d'origine est JPH07136361A, à lire séparément). NOTRE PROPOSITION : outil « diviser la région » opérant sur la géométrie vectorielle (Clipper2) plutôt que sur pixels.

(6) Juridique (préliminaire) : brevet expiré (fin 2017) selon Google. Pas de risque d'exploitation ; reste à ne pas copier le texte. Citations ultérieures ne créent pas de droits.

---

2. US6356648B1 — Embroidery data processor

(1) Brother ; inventeur Shoichi Taguchi. Priorité 20/02/1997 (JP 9-036673 / JPH10230088A), dépôt 18/02/1998 (US 09/025,569), délivrance 12/03/2002. Expired - Lifetime, expiration prévue 18/02/2018 ; frais payés aux années 4, 8, 12. Famille ID 12476385 : JPH10230088A. Cité par ~72 documents (Wilcom US6587745B1, Brother, DRAWstitch US10132018B2, Cimpress, Levi Strauss...). Voir aussi USRE38718E1 dans les citations.

(2) Claims : indépendantes 1 (appareil : unité d'amincissement, extracteur de région délimitée, unité de suivi de ligne fine, processeur de boucles fermées), 12 (méthode), 16 (support), 20 (appareil dont le processeur de boucle détache une boucle interne en supprimant sa branche de liaison). Dépendantes 2-5 (données de couture de région tatami + contour running/satin), 6-8 (détection et suppression des branches ; une branche = inversion de direction de 180 degrés ; branches reliant contour et boucle interne), 9-11 (boucle détectée quand un pixel est suivi deux fois ; découpe en contour interne ; décision selon les sens de suivi aller/retour). Figures 10-17. Lecture complète du corps du brevet : les derniers ~35 000 caractères de la description n'ont pas tous été vérifiés, mais les claims ont été lus.

(3) Algorithme (DU BREVET, sans équation) : - Pipeline : amincissement (Hilditch, Deutsch, Tamura cités comme exemples, n'importe quelle méthode à ligne 1 pixel) -> vectorisation par suivi et échantillonnage (intervalle non précisé) -> région à remplir désignée par l'utilisateur -> extraction -> élagage des branches par suivi du contour. - Suivi de contour sur ligne fine, 8-voisinage : point de départ P = pixel noir de coordonnées extrêmes (Xmax, Ymax, « en haut à droite »). Contours internes parcourus anti-horaire (premier essai vers la gauche), externes horaires (premier essai vers la droite). - Ordre de recherche du prochain pixel : si le chemin précédent est orthogonal, candidats dans l'ordre avant-droite, avant, avant-gauche, arrière (4 essais, compteur jusqu'à 3) ; si le chemin précédent est diagonal : droite, avant-droite, avant, avant-gauche, gauche, arrière (6 essais, compteur jusqu'à 5). Le premier pixel noir trouvé est retenu. - Arrêt quand on retombe sur P. Si le chemin suivant est l'inverse (180 degrés) du précédent : branche (cul-de-sac), le chemin précédent est supprimé (dépilé). Mémoires de chemins et de pixels visités. - Boucle : si le pixel suivant a déjà été visité, comparer le chemin de sortie à celui de la première visite ; différence de 180 degrés = boucle fermée. Une boucle reliée au contour par un « cou » est détachée et traitée comme contour interne ; une boucle en contact direct est conservée. - Exemple : moustaches et cils (branches) supprimés, nez (boucle) devenu contour interne, tatami généré ensuite.

(4) Cas limites : trous internes multiples, boucles à cou, spur pruning par retour arrière. Coût linéaire en nombre de pixels du squelette. Pas de seuil de longueur de branche : toute branche est supprimée (donc détails fins perdus).

(5) Pertinence : MOYENNE. OpenStitch fait déjà squelette + vectorisation sur régions, donc la chaîne exacte n'est pas utile telle quelle ; l'apport conceptuel est l'élagage topologique des branches et la détection de boucles à col sur un squelette 1 pixel (cela pourrait servir à nettoyer les squelettes satin, voir docs/source/satin.md pour savoir si déjà fait). NOTRE PROPOSITION : élagage par longueur de branche + conversion de petites boucles en trous, sur le graphe du squelette plutôt qu'en suivi de pixels. Peu de gain si le squelette est déjà un graphe.

(6) Juridique : expiré (2018). Faible risque ; ne pas reprendre la rédaction.

---

3. US5283747A — Embroidery pattern data processor

(1) Brother ; inventeurs Kyozi Komuro, Atsuya Hayakawa, Hideaki Shimizu. Priorité 28/06/1989 (JP 1-167875), dépôt 18/06/1990, délivrance 01/02/1994. Expired - Lifetime (expiration prévue 01/02/2011). Famille : JPH0684585B2, JPH0333255A, DE4020463A1/C2, GB2235991A/B. Cite 7 brevets, cité par ~12-15.

(2) Claims : 19 revendications, indépendantes 1 (processeur pour machine multi-aiguilles : détermination section extrémité/fourche, bâti running de la fourche vers une extrémité, puis broderie en retour vers la fourche), 7 (variante colonne principale / colonnes branches), 14 (méthode). Dépendantes : broderie après ses points de bâti avant tout autre point ; sections polygonales à sommets ; bâti terminé à un sommet de la section d'extrémité ; passage par le barycentre de chaque section intermédiaire. Figures 4-8 (aire fermée E découpée en sections a-q ; cartes mémoire).

(3) Algorithme (DU BREVET, sans équation) : - Entrée : aire fermée découpée en sections polygonales (quadrilatères : sommets 1-2 côté début, 3-4 côté fin), ordre arbitraire ; données de densité. - Adjacence : pour chaque section, chercher les autres sections partageant une paire de sommets (côté) -> liste d'adjacentes + données de bordure. - Classification par nombre d'adjacentes : 1 = section d'extrémité ; 2 = intermédiaire ; 3 ou plus = fourche ; 0 = traitement standard. - Parcours depuis la première section avec

compteur CNT de fourches ouvertes et pile BB[CNT] ; drapeaux « traité », « bordure effacée ». Quand on atteint une extrémité de branche : calcul du bâti running passant par les barycentres des sections de la branche et finissant à un sommet de l'extrémité non partagé ; puis on brode la branche en sens inverse vers la fourche (échange des sommets début/fin pour garder la direction cohérente). - Contrôle de validité à la fourche : une seule bordure non effacée doit rester, sinon l'aire non traitée serait scindée ; si échec, choisir une autre bordure adjacente. - Exemple : colonne principale a-f, branches depuis les fourches c et j. - Aucun paramètre numérique ; barycentre = moyenne des sommets (c'est la définition usuelle, non une équation du brevet).

(4) Cas limites : sections en anneaux (cycles) non traitées explicitement ; plusieurs fourches imbriquées via pile. Coût O(n) après adjacence (O(n^2) naïf).

(5) Pertinence : MOYENNE pour le routage des réseaux de satin multi-sections (auto_satin « multi-section networks » et routage dans satin.md) : l'idée « aller le long d'une branche en bâti puis broder au retour pour finir à la fourche, sans fil croisé » est un schéma d'ordonnancement d'arbre. À comparer avec ce qui existe déjà dans stitch_generation/optimization avant de décider ; je n'ai pas lu le code. NOTRE PROPOSITION : parcours en profondeur du graphe de sections avec pile de fourches.

(6) Juridique : expiré (2011) ; aucun risque notable.

---

4. US5343401A — Embroidery design system (Pulse Microsystems)

(1) Pulse Microsystems Ltd ; inventeurs Brian J. Goldberg, Anastasios Tsonis. Priorité = dépôt 17/09/1992 (US 07/946,753), délivrance 30/08/1994. Expired - Lifetime (expiration prévue 17/09/2012) ; garantie PNC 1999 sans lien. Lié à US5270939 (Method for Modifying Embroidery Design Programs, 1993). Cite Childs US4352334, Wilcom US4821662, Tokyo Juki US4720795, Melco US5056444, Goldman US4149246 ; cité par ~47-67 (Pulse, Brother, Zazzle, Melco).

(2) Claims (lus au second passage, paraphrase) : 1 = conversion d'un motif alphanumérique en format bridge (point de départ + second point pour la largeur + hauteur ; mise à l'échelle des points du contour ; l'utilisateur définit un arc ; l'échelle y de chaque point dépend de son x). 4 pourcentage de « bridge rise » signé ; 5 espacement entre caractères mis à l'échelle ; 6 chaînage de formats (vertical, rotation, arc, cercle, trois lettres, arc vertical, trois types de bridge) ; 9 version à partir d'un cercle et de deux points. L'essentiel du document (>150 000 caractères) décrit l'interface menu d'un poinçonneur PC, sans intérêt algorithmique ; les ~2 000 derniers caractères n'ont pas été lus.

(3) Algorithme : seules trois équations sont rendues (DU BREVET, telles qu'écrites) : Flat Top newy = ybaseline + ypcnt*(y - ybaseline) ; Flat Bottom newy = lefty - ypcnt*(lefty - y) ; Double newy = yhalfway + ypcnt*(y - yhalfway). ypcnt, xpcnt et Leftbrdg dépendent d'équations NON RENDUES (placeholders) : je ne peux pas les reconstruire, ne pas les inventer. Description verbale : mise à l'échelle X = largeur totale des lettres + espacements sur largeur voulue, Y = hauteur de police sur hauteur voulue ; le facteur y est interpolé entre un pourcentage au bord gauche et au bord droit de la lettre ; le côté de la lettre est traité comme une droite et non un vrai arc. Les contours sont stockés comme points regroupés en segments/groupes, les points de couture sont générés ensuite à taille/densité demandées (ce qui évite de transformer chaque point).

(4) Cas limites : approximation de l'arc par segment droit ; déformation en bordure de lettre.

(5) Pertinence : FAIBLE. Hors numérisation auto. Utile seulement si OpenStitch ajoute du texte déformé (enveloppes). La transformation d'objets contour avant génération de points est déjà l'architecture d'OpenStitch (points dérivés, jamais stockés).

(6) Juridique : expiré (2012). Les formats de menu et libellés de produit sont de l'expression, à ne pas imiter.

---

5. US4849902A — Stitch data processing apparatus for embroidery sewing machine

(1) Brother ; inventeurs Masaaki Yokoe, Yoshikazu Kurono, Kouji Hayashi, Miho Hashimoto. Priorité 21/11/1986 (JP 61-279434 et 61-286273), dépôt 19/11/1987 (US 07/122,765), délivrance 18/07/1989. Expired - Lifetime (expiration prévue 19/11/2007). Famille : DE3739647C2, GB2199165B. Cité par ~54. RESERVE : le second passage de lecture (offset 100000) a renvoyé des données d'un autre document (Bernina US8219237B2 / US20080022910A1, citations Janome, Mitsubishi) ; ces éléments sont des « brevets similaires » du bas de page, à ignorer. Je n'ai donc pas pu confirmer la fin de la page, ni la présence d'équations.

(2) Claims : 1 (appareil : support d'original, lecteur, affichage, saisie de points de contour, affichage de la ligne de contour, saisie de sommets découpant l'aire en blocs polygonaux, saisie de densité, calcul des données de point à partir des sommets et de la densité, mémorisation), 2 (zoom), 3 (CRT + crayon optique), 4 (caméra TV), 6-7 (photo / motif déjà brodé), 8 (indépendante : direction de référence et ligne de référence affichée).

(3) Algorithme (DU BREVET ; équations EQU1/EQU2 rendues en image, donc exactes non disponibles) : - Échelle : un point de référence F = origine ; rapports X/Y égaux ; exemple 400 points d'écran = 80 mm donc 0,2 mm par point ; point D1 (40,380) -> C1 (8,76) mm (multiplication par 0,2). - Satin par bloc quadrangulaire (sommets 101-104, densité N fils/mm) : milieux M1-2 (101-102) et M2-3 (103-104) ; distance l_B1 entre eux ; nombre de divisions m = N x l_B1 ; diviser les côtés 101-103 et 102-104 en m parties ; parcours alternant 101 -> 102 -> 110 -> 121 -> 112 -> 104... (les points sont visités en zigzag d'un côté à l'autre). - Bord courbe : polynôme f(X,Y) = A(X+aY+Ca)^n + B(X+bY+C_B)^(n-1) + ... + C ajusté par substitution des coordonnées des sommets (cubique ou plus avec trois points) ; deuxième calcul m2 = N x l' via milieux additionnels MA, MB, MC pour mieux estimer la longueur ; les courbes sont divisées en m1 ou m2 segments. L'énoncé « pentagonal » semble une erreur de transcription pour polynomiale. - Méthode de la fente de référence : angle theta fixe ; une droite de référence passe par chaque point de contour choisi, on calcule ses intersections avec les autres segments, on ne garde que le segment jusqu'à la première intersection ; l'ensemble découpe la forme en blocs carrés ou triangulaires (triangle = deux sommets confondus) donnant une direction de couture uniforme. - Mentionné comme alternative : détection de contour par saut d'intensité.

(4) Cas limites : blocs triangulaires, nombre m arrondi (non précisé), polynôme de haut degré lent.

(5) Pertinence : FAIBLE. Satin par division égale d'un quadrilatère = base de ce qu'OpenStitch fait déjà avec rails et barreaux. La « fente de référence » (découpe en blocs à direction constante) est proche des directional fills/guides auto. NOTRE PROPOSITION : rien de neuf à prendre.

(6) Juridique : expiré depuis 2007.

---

6. US7386361B2 — Embroidery data creation device, method, program (Shima Seiki)

(1) Shima Seiki Mfg. ; inventeurs Takeuchi Nobuyuki, Okubo Atsushi, Nishioka Hisataka. Priorité 15/10/2003 (JP2003-355163), PCT/JP2004/013108 (09/09/2004), US 10/575,787, délivrance 10/06/2008 (publ. US20070129840A1). Statut : Expired - Fee Related ; événements juridiques : brevet expiré par défaut de paiement de maintenance (07/2020, effet 10/06/2020). Remarque : le premier résumé annonçait une date ajustée 2024-11-05 qui contredit les événements juridiques ; je retiens l'expiration 2020, à vérifier au registre USPTO. Famille : WO2005038118A1, EP1676945B1, JP4153859B2, KR101099609B1, CN1867723B, US20070129840A1. Cité par (Nike, Janome, CreateMe...).

(2) Claims : 1 (appareil : dispositif de dessin donnant position, pression, vitesse, inclinaison ; unité de points d'aiguille générant des points au fil du tracé selon largeur/densité/angle ; détection de courbure et règle de correction croissante avec la courbure ; affichage en temps réel), 3 (largeur diminue avec la pression), 5 (interpolation selon la densité, aiguilles à angle et largeur donnés), 6 (ordre des traits, suppression de points dans les zones de recouvrement), 7-8 (simulation, éclairage monotone), 10 (méthode), 14 (support).

(3) Algorithme (DU BREVET, aucune équation) : échantillonnage du stylet (position, pression, inclinaison par décalage pointe/base, vitesse par différences) ; points intermédiaires insérés à espacement fixé par la densité (exemple 5 points entre P0 et P1) avec interpolation de la pression ; chaque point = centre d'un point de couture, deux aiguilles de part et d'autre à angle theta (exemple <= 30 degrés pour bord doux) et largeur modulée par la pression ; correction de courbe : courbure = taux de variation de la direction ; petit angle de tracé -> réduire la largeur côté intérieur (légère réduction extérieur) pour éviter l'amas, grand angle -> augmenter la largeur ; chevauchement de traits : numéro de trait, suppression des points de l'ancien ou du nouveau selon règle, points supprimés mémorisés, ajout de points si point de liaison trop long ; éclairage : source par hauteur theta et azimut phi, luminosité monotone le long du point.

(4) Cas limites : lissage optionnel (spline ou moyenne) des tracés tremblés ; recouvrement dans une boucle fermée.

(5) Pertinence : FAIBLE pour l'auto-digitizing ; MOYENNE si un mode « pinceau satin » à main levée ou la compensation de courbure du satin (réduire la largeur côté intérieur) est visé. La simulation d'éclairage est un apport de rendu seulement. NOTRE PROPOSITION : compenser la densité du satin sur courbure serrée côté intérieur (à comparer avec la gestion existante dans satin.md).

(6) Juridique : expiré selon l'historique (2020) ; brevets étrangers de la famille peuvent exister dans d'autres pays (EP, CN, KR, JP) ; statut non vérifié.

---

7. US6937919B1 — Embroidery data processing apparatus (Brother)

(1) Brother ; inventeurs Muto, Mizuno, Suzuki, Taguchi, Wakayama. Priorité 31/03/2004 (JP 2004-107034 / JP2005287763A), dépôt 01/02/2005, délivrance 30/08/2005. Expired - Lifetime, expiration prévue 01/02/2025 (si c'est bien le cas, déjà passée à la date d'aujourd'hui). Famille ID 34858533. Citée par US20070233309A1, US7693598B2 (Brother), US12091797B2 (Tajima). Cite JPH02133647A, US5775240A (Janome), US6587745B1 (Wilcom), US6629015B2 (Brother).

(2) Claims (32) : indépendantes 1 (concentrique), 10 (radial), 16 (elliptique), 17, 26, 32 (supports). Dépendantes : 2 (passage en coordonnées theta-R), 3-5 (espacement des cercles variable avec la distance au centre, monotone ou selon motif utilisateur), 6-9 (multi-couleurs, motifs différents, sans chevauchement, motifs inversés), 11-15 radial équivalents, 18-25 et 27-31 versions support. Figures 15-35 ; la figure 35 (équations de transformation polaire) n'est pas rendue.

(3) Algorithme (DU BREVET) : - Entrées : région, centre C. - Concentrique : l'utilisateur dessine une courbe de densité d = g(t), t dans [0,T] (densité = cercles par longueur). Densité moyenne d', distance G de C au point le plus éloigné de la région, nombre de cercles N = G x d'. Rayons Ri placés pour que chaque bande ait la même aire sous la courbe, R0 = 0 (la formule exacte n'est pas donnée, seulement ce critère). - Méthode 1 : cercles autour de C, garder cercles/arcs dans la région, placer les points dessus (variante ellipses). - Méthode 2 : contour passé en (theta, R) avec C pour origine ; cercles = droites parallèles à l'axe theta ; segments dans le contour transformé joints en une ligne L2 ; transformation inverse x = R cos(theta) + cx, y = R sin(theta) + cy ; points placés sur L. - Méthode 3 : points placés sur L2 dans l'espace theta-R puis transformés. - Méthode 4 : multi-couleurs, motifs décalés pour que les cercles de couleurs différentes ne se chevauchent pas. - Radial : même schéma avec d = h(s), s dans [0,360], N = 360 x d', angles à aire égale, demi-droites depuis C par rapport à une direction de référence SD ; mêmes méthodes 1-4. - Elliptique : ellipses déformées des cercles.

(4) Cas limites : point de départ/arrivée sur les arcs coupés (jonction des segments L2), région non étoilée par rapport à C (plusieurs segments par cercle), centre hors région, couture continue vs sauts.

(5) Pertinence : MOYENNE à ÉLEVÉE comme NOUVEAU type de remplissage (OpenStitch n'a que tatami/contour en autodigitize). Implémentable indépendamment : motif concentrique/radial/elliptique via intersection de courbes avec un polygone (Clipper2 + paramétrage polaire). Le compromis technique est la manière de lier les arcs en une couture continue. Reste utilitaire/décoratif, pas de l'auto-digitize. NOTRE PROPOSITION : ajouter un type « remplissage polaire » dans DirectionalFillParams avec centre et profil de densité. À noter : le remplissage curviligne existe déjà chez Wilcom (US6587745B1) mais n'est pas l'objet de ce brevet.

(6) Juridique : expiré ou en expiration (2025) ; faible risque. Vérifier la date exacte.

Bilan

Seuls 5839380 et 6356648 touchent la chaîne image -> régions (déjà très couverte par OpenStitch). Les plus utiles à un développement : 6937919 (nouveau type de remplissage, haute pertinence) et 5283747 (ordonnancement par arbre de sections, moyenne). 5343401, 4849902, 7386361 : faible. Tous les statuts affichés sont expirés selon Google, hypothèse non juridique. Avertissement : texte non primaire ; formules rendues en image absentes (5343401 ypcnt/xpcnt, 4849902 EQU1/2, 6937919 fig. 35) et je n'ai pas inventé leur forme.

Errata après lecture des PDF (2026-10)

Ces fiches ont été écrites à partir de résumés de pages web. Les 32 brevets ont ensuite été relus sur leur **texte primaire** (PDF Google Patents ; équations et tableaux abîmés par l'OCR relus sur rendu image ; scans japonais lus visuellement). Les points ci-dessous **corrigent ou précisent** le corps de cette page ; en cas de conflit, ils priment. Les figures n'ont été regardées que lorsqu'une équation en dépendait.

Famille Goldman / SoftSight et Brother (lot B1)

- **Exposants de recherche** (US6836695B1 et US6947808B2, descriptions identiques mot pour mot) : ancres de jonction = `DT × 10^(2π − θ)` (signe moins confirmé sur rendu image ; la lecture `DT×10/(2π−θ)` est fausse). Fusion de bifurcation = `DT × 10^(2π + θ)` dans US6947808B2 (le texte dit que la distance croît avec l'angle) ; US6836695B1 imprime `10^(cπ+θ)` avec un `c` jamais défini. **Désaccord entre relectures** : une relecture de US8532810B2 et US9200397B2 trouve `2π−θ` dans trois occurrences sur quatre et une seule fois `2π*θ`. Le sens physique est douteux partout, les rayons sont énormes (`10^(2π)` ≈ 1,9·10■) et les radians sont une inférence (~85 %). **Ne pas implémenter sans plafond propre.**
- **Règle de régularité** (US7016757B2) : `2·σ < μ < ½·max`, confirmée exactement, calculée sur les valeurs de distance chamfer 3-4 (normalisée par 3) des pixels du squelette. C'est un exemple (« may »). Elle exclut un ruban parfaitement uniforme (μ = max, σ = 0) : préférer un coefficient de variation σ/μ aux seuils calibrés chez nous. Les seuils de « mince » ne sont pas chiffrés. La statistique par branche est explicitement non employée.
- **US8219238B2** : fragment = « aire maximale remplissable à un angle donné sans repositionner l'aiguille », obtenue par un balayage de lignes modifié dont le texte ne donne aucun détail. Seize angles, un seul angle pour tout l'objet ; liste des angles, départage et critère secondaire **absents**. Ajustement de durée de 422 jours et terminal disclaimer (explique en partie l'anomalie de statut).
- **US8532810B2** : la revendication 9 porte bien la limitation « information de création / lien au bord de départ ». L'ordre est un traitement post-ordre depuis le fragment du point de sortie, points droits par ligne médiane ; le mot « arbre » vient de la fiche. Absents : fonction de coût, priorités, choix du point d'entrée. Délivrance officielle 2013-09-10.
- **US9200397B2** : seuil de hauteur du triangle **absent** (texte, revendications, figure 5). Départage des égalités = sommet le plus proche de l'origine. Le caractère itératif est une inférence. Délivrance 2015-12-01.
- **US6836695B1 / US6947808B2** : ajustements de durée de 924 et 90 jours ; la fiche retrouve ses échéances par calcul. Précisions absentes de la fiche : voisins de sommet à au plus 2× la distance du plus proche ; paire d'ancre rejetée si les vecteurs ne sont pas opposés ; angle de branche mesuré à distance = DT du noeud ; la fusion de bifurcation se fait sur les mêmes sommets caractéristiques, pas sur le recouvrement de cercles de DT ; une branche n'est supprimée que si sa longueur est inférieure à la distance de recherche maximale.
- **US5576968A** : formule d'aire récupérée mais non générale : `SR = n²(√2+1)/2`, `SL = 3n²(√2+1)/2` décrivent l'octogone régulier de la figure 10. Si `LX = RX` ou `SL = SR`, direction B. Ambiguïté du brevet : « le plus éloigné du point de fin » ou « de NL ».

- **US5740056A** : seuils confirmés ; absents : métrique de distance, quantum d'une passe, résolution du scanner (donc aucune conversion en millimètres).

Brevets Brother, Pulse et Shima Seiki (lot B2)

- **US5343401A** : la fiche le jugeait sans intérêt algorithmique, ce qui est faux. Le texte contient une densité modulée par la largeur de colonne (exemple imprimé : 56,4 spi donne 28,2 spi pour 2 mm et 84,6 spi pour 30 mm), un nettoyage des points courts, des paramètres de sous-couche, un satin courbe par triplets de points et des décalages de tatami en pourcentage. Les équations `xpcnt`, `ypcnt`, `Leftbrdg` sont imprimées mais abîmées ; `lefty` n'est défini nulle part et les étiquettes Flat Top / Flat Bottom semblent permutées : **redériver, ne pas reprendre**. La revendication 9 définit un cercle (centre, rayon, début, fin). 9 revendications.
- **US6356648B1** : algorithme confirmé (4 voisins / compteur 3 si chemin orthogonal, 6 voisins / compteur 5 si diagonal). Élagage en cascade ; branches testées avant les boucles ; `Xmax` et `Ymax` mis à jour indépendamment. Date de priorité : l'OCR lit « Sep. 20, 1997 » alors que la fiche indiquait février, à vérifier sur l'image.
- **US5283747A** : conforme. La fourche est cousue après ses branches ; le choix de la branche traitée en premier est arbitraire ; les cycles ne sont pas traités. Un terminal disclaimer fixe l'échéance effective au 29/09/2009.
- **US5839380A** : conforme ; la classification automatique est renvoyée à JP 7-136361, absent du texte.
- **US4849902A** : `EQU1 : l_B1 = √((X_Ma−X_Mb)² + (Y_Ma−Y_Mb)²)` (distance entre milieux de côtés) ; `EQU2 : N × l_B1 = m` (nombre de divisions, arrondi non précisé). Le mot « pentagonal » est bien imprimé (la fiche y voyait une erreur). 8 revendications. Rien de neuf pour OpenStitch.
- **US7386361B2** : la correction de courbure et au moins une grandeur de stylet (pression, vitesse, inclinaison) figurent dans les trois revendications indépendantes ; la revendication 3 signifie que la largeur diminue quand la pression diminue. Aucune équation ni seuil (exemple de 30° seulement). Alternative omise : détection de courbe par orientation X/Y, décalage du centre du point, lissage optionnel.
- **US6937919B1** : figure 35 lue : `θ` est l'angle entre l'axe x et la droite du point au centre `C(cx,cy)` ; `R = √((x−cx)² + (y−cy)²)` ; inverse `x = R·cos θ + cx`, `y = R·sin θ + cy` (convention `atan2` non précisée). Placement des rayons : `N = G × d'`, `R■ = 0`, et l'aire sous la courbe de densité est sensiblement égale pour chaque bande ; **aucune formule fermée n'est imprimée**. L'expression `R■ = F■¹(i)` (F primitive de g) est une inférence de confiance moyenne. Les cas limites listés par la fiche (région non étoilée, centre hors région, jonction des segments) **ne sont pas traités** par le brevet. Méthode multi-couleurs : un profil par couleur, décalé ou inversé.

Fiches brevets : sous-couches, contours auto-intersectés et points cachés

> Fiches techniques de travail, issues de la lecture de pages Google Patents (résumés automatiques, pas les PDF). Statuts juridiques préliminaires. Voir l'index et les priorités dans Recherche brevets.

Méthode : texte Google Patents (WebFetch, résumé par modèle rapide, donc pas de lecture brute). Limites : les figures ne sont accessibles que par légendes ; les Tables 1 et 2 de US6510360B1 sont absentes/illisibles ; les 56 k derniers caractères de US5957068A et 27 k de US6510360B1 ont été lus uniquement via un résumé de revendications. Aucune équation n'est donnée dans ces sept brevets (tous procéduraux). Toute formule ci-dessous est donc "NOTRE PROPOSITION". Statuts juridiques : tels qu'affichés par Google, "hypothèse et non conclusion juridique".

Correction des catégories

- US5934209A n'est PAS de la reconstruction ni de l'optimisation : c'est de l'édition de contour auto-intersecté (découpe en zones fermées + attributs par zone). Même famille que US5823127A / US5957068A / JP3922316B2.
- Famille Brother "contour auto-intersecté" (4 brevets + JP) : US5823127A (priorité JP 1996-12-20), US5934209A (JP 1996-12-27), US5957068A (CIP, JP 1997-01-13), JP3922316B2 (1997-09-19). Doublons de famille : US5957068A est CIP de US5823127A et US5934209A ; JP3629854B2 = équivalent de US5823127A ; JP3760541B2 = équivalent de US5934209A ; JP3922316B2 est indépendant (pas d'équivalent US listé).
- US6510360B1 (stitch -> objets) et US6247420B1 (stitch -> contour par bitmap) sont de la rétro-ingénierie de fichiers de points ; US6633794B2 est de l'optimisation (suppression de points cachés). US6510360B1 cite US6247420B1.

---

1. US6510360B1 : fichier de points vectoriel -> description par objets

1. Inventeurs Kaymer, Bysh ; cessionnaire initial Viking Sewing Machines AB (VSM Group, puis KSIN Luxembourg II). Priorité GB9921013, 1999-09-06 ; dépôt 2000-09-01 ; octroi 2003-01-21 ; expiré (échéance 2020-09-01 supposée). Famille : GB2353805A/B, SG87168A1 ; apparenté GB2379454B (bitmap -> objets). Cite US6247420B1, US5474000A, US5559711A, US5865134A, EP0545773A1, GB2334264A. Cité par ~25 (Brother, Melco, Bailie, Nikon...).

2. Revendications utiles : 1 (méthode : points de bord où la direction s'inverse, liste ordonnée, groupes consécutifs bornant des régions de remplissage, un objet par région) ; 2 (bords simples = angle sous seuil, et bords complexes = paire dont l'angle agrégé est sous seuil) ; 3 (conditions de fin de groupe) ; 4 (FILL si moins de 30 % des bords sont à indices de points consécutifs, sinon SATIN) ; 5-6 (angle de remplissage = moyenne sur la partie centrale) ; 7-16 (fusion de groupes voisins, critères FILL/SATIN) ; 17-21 (appareil, supports, fichiers). Incohérence relevée : la description cite 20 % pour la classification, les revendications 30 %. Revendications 1-2 lues via résumé uniquement : ambiguïté possible.

3. Algorithme (FROM PATENT, mots uniquement, valeurs numériques non récupérées sauf mention) : - Prétraitement : coordonnées absolues, suppression des sauts utilisés pour allonger un point, suppression des points de longueur nulle, angle signé de chaque point entre ses voisins (-180..180). - Liste de bords : "bord simple" = angle < seuil M(longueur) (Table 1, valeurs inconnues). "Bord complexe" = deux points consécutifs dont les angles somment à environ 180 deg ; on garde le plus petit angle ; angle agrégé = 180 moins la somme ; à moins de 10 deg de 180 : statut "free-ride". Un "nouveau bord" n'est précédé ni d'un bord ni d'un point de course. Recalcul des angles/longueurs sur la liste de bords puis re-test (Table 1 pour simples, Table 2 pour complexes). - Groupage : séquence de bords tant que le signe de l'angle alterne ; rupture si même signe, écart de longueur de plus d'un pourcentage donné, va-et-vient (comparaison aux indices béta-2 et béta-4, pour détecter les arrêts/tie-offs), ou nouveau bord. - Éliminer un groupe si >40 % d'angles nuls, <6 bords, ou motif simple-complexe-simple avec angles

simples <= 9 deg sur >30 % du groupe. - Catégorie : FILL / SATIN (voir seuil ci-dessus). Les SATIN sont étendus en retestant les points adjacents avec un seuil de 90 deg. Angle FILL = moyenne des angles des droites de bord sur le tiers central ; pour un bord simple la droite relie le point au milieu de ses voisins. Densité = moyenne des normales entre droites de bord successives, mesurées sur un groupe remis à l'échelle (2 cm) avec un facteur empirique. Un FILL dont le rapport angles pairs/impairs vaut 2:1 ou 1:2 devient SATIN avec numéro de motif. - Contours : FILL = indices impairs dans l'ordre puis indices pairs en sens inverse -> polygone fermé. SATIN = un point sur n (n pouvant valoir 8) de chaque parité -> deux rails. - Fusion : FILL si angles à <=5 deg, densités à <=2 mm, pas de recouvrement (sauf petit), pas de changement de couleur, points de connexion valides (angle entre lignes, angle au sens du remplissage, intercept normal, longueur mini). SATIN si peu de points entre contours, recouvrement <=20 % du nombre de bords, densités à <=1 mm si plus de 2 points entre eux, angles <90 deg, longueurs des lignes de connexion à <=50 %, normale de 2 mm de l'une qui coupe l'autre. Test de torsion (lignes de bout qui se croisent -> raccord croisé). - Nettoyage : suppression de pointes (point plus loin de ses voisins qu'ils ne le sont entre eux -> milieu), réduction de points quasi-colinéaires, lissage satin par projection sur la droite vers le point 4 rangs plus loin, désactivé si >40 % d'écarts de longueur >=40 % ("feather"). - Création d'objets : FILL/SATIN avec angle et densité ; autres points convertis un par un ; sauts <4 mm convertis en points.

4. Cas limites : points multiples de tie-off, satin en plume, objets rétrécis, courbes serrées (angles faibles), gros bruit de numérisation. Coût : linéaire sur la liste de points sauf fusion (comparaison de groupes voisins).

5. Pertinence OpenStitch : MOYENNE-FAIBLE. Utile seulement si on veut importer un DST/PES existant en objets éditables (fonction absente aujourd'hui : l'app vise raster -> DST). Les heuristiques de classification (alternance de signes, 20/30 %) sont exploitables comme idée de base, mais les seuils sont à calibrer par nous (OUR PROPOSAL). Nouveau et implémentable : import "DST -> objets" (reverse digitizing), en module `formats`/nouveau `reverse_stitch`, sans Qt.

6. Juridique (préliminaire) : brevet expiré (durée 20 ans écoulée à 2020), donc libre d'usage ; on ne copie ni texte ni tableaux ; on réimplémente. Vérifier tout de même les brevets cités par la suite (Melco US20140366788A1) si on s'en inspire.

---

2. US5934209A : contour auto-intersecté -> zones fermées avec attributs

1. Inventeur Mizuno ; Brother. Priorité JP 8-350274 (1996-12-27) ; dépôt US 1997-12-15 ; octroi 1999-08-10 ; expiré (échéance 2017-12-15). Famille : JP3760541B2 (JPH10179963A) ; CIP US5957068A ; cite US5179520A, US5474000A, US5740057A, US5823127A.

2. Revendications : 1 (dispositif de division qui détecte les auto-intersections, stockage d'attributs par zone, générateur), 11 (dispositif), 16 (support) ; dépendantes 2-10, 12-15, 17-18 (attributs de ligne/intérieur, couleur, type, partage de segments communs). Revendication 18 apparemment dépendante d'elle-même (erreur de rédaction, signalée par la source).

3. Algorithme (FROM PATENT) : - S1 : pour chaque segment t de 1 à K-2, tester contre les segments r de t+2 à K (enroulement : le suivant de K est 1) ; stocker les croisements. O(K au carré). - S2 : découper le contour aux croisements. - S3 : compter les zones N : drapeau "visité" par point ; parcours depuis un point non visité, en sautant à l'autre segment à chaque croisement (règle de l'exemple : on prend le candidat de numéro le plus élevé) ; fermeture du cycle -> N+1. - Attributs par zone (sélection à la souris, multi-sélection possible) : intérieur (satin/tatami, densité, angle, couleur, pas et déviation pour tatami) ; ligne (course/zigzag, pas, largeur). Exemple : 5 fils/mm 60 deg bleu ; 6 fils/mm 150 deg noir. - Génération de points par zone avec une méthode classique, non détaillée.

4. Limites : le brevet ne traite ni intersections tangentes/colinéaires, ni self-overlap de trous. Coût O(K au carré).

5. Pertinence : FAIBLE-MOYENNE. OpenStitch a déjà Clipper2 (union/XOR/évaluation des régions via règles de remplissage) qui donne des zones fermées plus robustement. Seule idée utile : attributs par sous-zone d'un contour auto-croisé (UI), à faire via Clipper2 "NonZero/EvenOdd" et pas par balayage

O(n au carré).

6. Juridique : expiré. Pas de risque pratique ; ne pas copier le texte.

---

3. US6247420B1 : reconnaissance de contour de broderie (points -> bitmap -> contour)

1. Inventeurs Chan, Leung, Wang ; Capital Automation (cession 2001-03-21). Priorité prov. 60/099,341 (1998-09-08) ; dépôt 1999-09-08 ; octroi 2001-06-19 ; expiré pour non-paiement de taxes (2005-06-19) ; échéance supposée 2019. Aucune famille étrangère listée. Cité par US6510360B1.

2. Revendications : seulement 3. Claim 1 (points de bord gauche/droite/normaux, groupage, densité, conversion bitmap, reconnaissance de contour) ; 2 (filtre image de lissage) ; 3 (points hors alignement). Texte très peu précis ; "claim ambiguity" : aucun seuil.

3. Algorithme (FROM PATENT, qualitatif) : (0) classer points et séparer course vs normaux ; (1) angle des rangées depuis la séquence ; points gauche/droite = bouts de rangée et sommets où l'angle change ; (2) rastériser les lignes entre rangées ; (3) élargir en rectangles dont l'épaisseur dépend de la densité du groupe, union en une surface pleine ; (4) analyse statistique le long des bords + lissage de courbe ; (5) re-stylisation par l'utilisateur. Seuils donnés par le brevet : remplissage > 3 mm de large, pas courant 4-5 mm (< 12 mm), satin < 12 mm (~12,7 mm antériorité), course < 5 mm d'épaisseur. Pas d'équation, résolution de la grille, formule d'épaisseur ni lissage donnés. OUR PROPOSAL pour une réimplémentation : rasteriser les segments épaissis d'un demi-pas (paramètre) dans un masque, fermeture morphologique (OpenCV dans `image`), puis `vectorization` existante -> contour. Cela réutilise nos briques.

4. Limites : densité variable, trous, courbes. Coût O(pixels).

5. Pertinence : FAIBLE-MOYENNE (même cas d'usage qu'US6510360B1 : import). Pas de contenu nouveau précis ; l'idée "bitmap des points puis vectorisation" est triviale avec nos modules.

6. Juridique : expiré (taxes impayées + durée). Risque faible.

---

4. US5957068A : sous-couche par zone d'un contour auto-intersecté (CIP)

1. Inventeur Mizuno ; Brother. Priorité JP 1997-01-13 ; dépôt 1998-01-12 ; octroi 1999-09-28 ; expiré (échéance 2017-12-15 pour Google). CIP de US5823127A et US5934209A (doublons de famille). Cité ~10-17 fois (Brother, Adobe...).

2. Revendications : 1 (division aux intersections + contour de sous-couche par zone + données), 2 (début/fin par zone), 3-4 (ordre de couture dans/entre zones), 5 (connectivité + points auxiliaires), 6-8 (activer/désactiver la sous-couche par zone), 10-13 (méthode), 14-20 (support : hiérarchie, premier segment). Revendications lues via résumé.

3. Algorithme (FROM PATENT, texte) : détection et comptage comme US5934209A ; drapeau "valide" par intersection ; hiérarchie de connectivité en partant de la zone contenant le point final E ; boucle : choisir une zone n'ayant qu'une intersection valide, rejoindre son point de départ par une course à travers les zones connectées, construire le contour de sous-couche, départ = point de contour le plus proche du point courant, fin = le plus proche de l'intersection suivante ou de E ; sous-couche en sens latéral, remplissage perpendiculaire ; invalider l'intersection. Pas d'équation ni de distance.

4. Limites : il suppose un arbre de zones (pas de cycles). Coût O(K au carré) + tri.

5. Pertinence : MOYENNE-FAIBLE. OpenStitch a déjà sous-couches ; ce qui serait nouveau est le routage des sous-couches par zones d'une offset auto-croisée (cas des formes étroites/ en "pincement"). À vérifier sur nos tatami : si l'inset Clipper2 produit déjà des polygones disjoints, il n'y a rien à faire.

6. Juridique : expiré.

---

5. US5823127A : sous-couche et connexion hors de la zone de broderie

1. Inventeur Mizuno ; Brother. Priorité JP 1996-12-20 ; dépôt 1997-12-15 ; octroi 1998-10-20 ; expiré (échéance 2017-12-15). Famille : JP3629854B2 ; CIP US5957068A. Cite US5299514A (qui découpe sans traiter la connexion). Cité 15 fois.
2. Revendications : 1 (contour de sous-couche intérieur auto-intersecté, découpe, données par segment + données auxiliaires de liaison) ; 2 (courses à pas donné) ; 3 (intersections = départ/fin) ; 5 (décalage depuis le contour, segments entièrement dedans / partiellement dehors) ; 6-8 (segments dedans, courses de liaison contenues) ; 10-16 (support : décalage, désignation, point auxiliaire, pas de course).
3. Algorithme (FROM PATENT) : offset intérieur du contour (q_i), détection d'auto-intersections (mêmes boucles t, r), découpe en segments U1..., ordre d'après la connectivité, points d'intersection = fin d'un segment / début du suivant ; test "tous les points du segment dans le contour"; si un segment sort : pour chaque q_i entre les deux intersections, tracer la droite q_i -> p_i correspondant, calculer son intersection avec le contour (la plus proche de p_i), prendre le milieu B_i entre p_i et le point d'intersection, puis courses A1 -> B1 -> B2 -> A2 en suivant le contour de broderie. Distance d'offset et pas de course non donnés. Aucune équation.
4. Limites : cas où plus de deux intersections ; "milieu" suppose un contour convexe localement.
5. Pertinence : MOYENNE-FAIBLE (même remarque que ci-dessus). Idée implémentable avec Clipper2 : inset + intersection avec le contour d'origine (intersection booléenne) pour garder uniquement l'intérieur, relier les morceaux par un chemin contenu dans le contour (pas par un segment direct). OUR PROPOSAL : routage par plus court chemin à l'intérieur du polygone.
6. Juridique : expiré.

---

6. JP3922316B2 : sous-couche, décision par segment (dedans / liaison / exclu)

1. Inventeur Mizuno (■■ ■■) ; Brother. Priorité/dépôt 1997-09-19 ; publication JPH1190069A 1999-04-06 ; octroi 2007-05-30 ; expiré (échéance 2017-09-19). Pas de famille US listée ; cité par JP2005211206A. Texte anglais machine ; revendications traduites, lecture non vérifiée par un humain.
2. Revendications : 1 (appareil) et 2 (support). Contenu : contour de sous-couche intérieur ; si auto-intersecté, découpe en contours partiels ; trois cas par région : dedans -> sous-couche normale ; sort mais relie directement/indirectement au moins deux régions dedans -> points auxiliaires de liaison ; sort et ne relie rien -> aucune sous-couche.
3. Algorithme (FROM PATENT) : S1 offset, S2 test d'auto-intersection, S3 découpe en P1..P4, S4 intersections = fin/début de régions voisines, boucle S7 : décision de la figure 4 (tableau ci-dessus), S11 remplissage principal. Exemple : P2 sort mais relie P1 et P3 -> course A->B ; P4 sort et ne relie que P3 -> ignoré. Aucun nombre. OUR PROPOSAL : graphe des régions (noeuds = régions, arêtes = intersections) ; une région "dehors" est conservée ssi elle est sur un chemin entre deux régions "dedans" (nœud d'articulation / arête de pont), sinon supprimée. Les feuilles pendantes sont élaguées.
4. Limites : une région partiellement dehors qui contient aussi du fond utile est ignorée (perte de sous-couche) ; le brevet n'en parle pas.
5. Pertinence : MOYENNE. C'est la règle la plus claire et la plus testable de la famille (décision en trois cas + élagage des feuilles), et elle évite des points inutiles sous les bords minces. À appliquer dans notre génération de sous-couche si l'inset de contours étroits produit des morceaux sortant de la forme.
6. Juridique : expiré ; la traduction automatique ne remplace pas une lecture du texte japonais si on devait aller plus loin.

---

7. US6633794B2 : suppression des points de sous-couche cachés

1. Inventeur Brian D. Bailie (personne physique). Priorité/dépôt 2001-02-23 (US 09/792,538) ; publication US2002/0120358A1 2002-08-29 ; octroi 2003-10-14 ; expiré (échéance supposée 2021-07-20 ; taxes 2007/2011/2015). Famille à un seul membre ; même inventeur ensuite : US7457683B2, US2007/0198119A1.

2. Revendications : 1 (identifier les points partiellement/entièrement cachés par un autre point, modifier), 2-7 (grille, suppression si entièrement couvert, point de liaison, taille de cellule = diamètre du fil modulé par l'étirement), 8-16 (affichage par cellule), 17-21 (système), 22 (couche par couleur : traiter chaque couleur contre les couleurs ultérieures, supprimer les suites d'au moins deux points cachés et insérer un point de liaison), 23-24 (point partiellement couvert -> découpe en sous-points, suppression des totalement couverts).

3. Algorithme (FROM PATENT, procédural) : cellule = largeur du fil (exemple 0,25 mm pour fil 40 ; règle de pouce citée : ~4 fils pour 1 mm) ; grille binaire par couche de couleur ; pour la couche en cours, dessiner dans la grille tous les points cousus APRES cette couche (cellule=1 si un fil passe) ; un point est caché si toutes les cellules qu'il traverse valent 1 ; on supprime le point, on mémorise ses extrémités et un drapeau ; au prochain point visible on insère un point de liaison entre la dernière fin visible et le prochain début visible ; un point caché isolé n'est pas supprimé (le remplacement serait équivalent) ; sorties : vues avant/après, nouveau fichier. L'ordre est essentiel : seuls les points couverts par des points cousus plus tard (couches ultérieures) sont redondants selon le brevet. Un point couvert par un point cousu AVANT n'est pas concerné. Séquences réellement redondantes selon le brevet : les points (typiquement de remplissage/sous-couche d'une couleur) dont toute la trajectoire est recouverte par des fils de couleurs postérieures, à condition de conserver la continuité du fil par un point de liaison. AVERTISSEMENT OpenStitch : ne jamais supprimer automatiquement de sous-couche ou de course structurelle (travel run) simplement parce qu'elle est visuellement couverte. Le brevet décrit un outil de nettoyage manuel d'un fichier existant (suppression à la demande, aperçu avant/après), pas un nettoyage automatique à l'export. La sous-couche sert à stabiliser, soulever et empêcher le tissu de transparaître : le fait d'être couverte est sa raison d'être. Les courses de déplacement servent à l'ancrage et au verrouillage.

4. Limites : résolution de grille (une cellule 1 ne garantit pas qu'il n'y a pas d'espace sous la cellule), points inclinés, couvrance par un seul fil vs par plusieurs, fils épais. Le critère "cellule couverte" est une approximation par raster ; le seuil de couverture n'est pas défini pour les points partiels. Coût : O(nombre de points x cellules traversées) par couche, mémoire O(grille).

5. Pertinence : FAIBLE comme suppression automatique ; MOYENNE comme règle d'analyse. Nouveau et implémentable sans risque : une règle de pré-export (module `stitch_analysis`) en lecture seule qui estime le pourcentage de points entièrement recouverts par des couches postérieures (carte de couverture rasterisée par couleur), en avertissement ou en rapport, avec marquage explicite des sous-couches/courses structurelles comme "exemptés" ; une suppression ne serait proposée que pour des points de type remplissage visible d'objets masqués (par exemple, un objet entièrement recouvert par un objet ultérieur), via un objet commande undoable et jamais en silence. Les points d'un objet sous-couche (rôle structurel dans notre document) ne doivent pas être candidats. Pas de réécriture de séquence dans `effective_sequence` (respecter la garde de contournement).

6. Juridique : expiré (taxes + durée). Les brevets suivants de Bailie (US7457683B2) sont à examiner séparément avant toute reprise, car peut-être encore en vigueur.

Synthèse

Ce qui est réellement NOUVEAU et implémentable : 1. Règle de décision de JP3922316B2 (dedans / pont entre régions dedans / exclu) pour la sous-couche de formes étroites : à tester en premier. 2. Connexion intérieure des courses de liaison (US5823127A) via plus court chemin contenu dans le contour. 3. Règle de pré-export en lecture seule "points entièrement recouverts" (US6633794B2), avec exemption de la sous-couche et des courses structurelles. 4. Import DST -> objets (US6510360B1 / US6247420B1) : fonction hors périmètre actuel, à planifier plus tard ; seuils à calibrer sur nos propres corpus. Peu de valeur : découpe d'auto-intersections par O(n au carré) (Clipper2 fait mieux). Tous les brevets sont en fin de vie ou expirés selon Google ; analyse préliminaire, pas un avis juridique.

Errata après lecture des PDF (2026-10)

Ces fiches ont été écrites à partir de résumés de pages web. Les 32 brevets ont ensuite été relus sur leur **texte primaire** (PDF Google Patents ; équations et tableaux abîmés par l'OCR relus sur rendu image ; scans japonais lus visuellement). Les points ci-dessous **corrigent ou précisent** le corps de cette page ; en cas de conflit, ils priment. Les figures n'ont été regardées que lorsqu'une équation en dépendait.

- **US6510360B1** : tableaux lus avec confiance haute. Tableau 1 (longueur → angle) : ∞→0, 170→3, 140→6, 130→8, 120→9, 90→10, 70→12, 60→13, 45→14, 40→15, 30→24, 25→29, 20→37, 15→50, 10→55, 5→60, 0→70. Tableau 2 : ∞→0, 500→6, 110→8, 90→10, 45→13, 40→15, puis comme le tableau 1 de 30 à 0. Règle : `M(L)` est l'angle de la plus grande longueur du tableau ≤ L (fonction en escalier) ; l'unité annoncée est le millimètre mais les ordres de grandeur suggèrent 0,1 mm (inférence). L'incohérence 20 % (description) / 30 % (résumé, revendications 4, 22, 23) est **réelle** et non tranchée : en faire un paramètre. L'angle FILL est mesuré par rapport à la verticale et sur le tiers central. 25 revendications. Absents : pourcentage d'écart de longueur, facteur de correction de densité, densité par défaut.
- **US5934209A** : l'auto-référence est la revendication 17 (non 18). Un `t = 1` avec `r = K` donnerait une fausse intersection en implémentation littérale (segment `PK-P1` adjacent à `P1-P2`).
- **US6247420B1** : aucun cessionnaire imprimé sur le document ; 3 revendications ; aucun seuil dans l'invention elle-même (ceux de la fiche viennent de l'art antérieur cité).
- **US5957068A** : CIP de **trois** demandes (08/990,816 ; 08/990,814 = US5823127 ; 08/990,733 = US5934209). Étape S10 omise par la fiche : sous-couche oui/non par zone. L'ordre est un élagage de feuilles depuis les extrémités vers la zone du point final ; le test « arbre sans cycle » est une déduction de la fiche.
- **US5823127A** : le brevet **traite** plus de deux intersections (étapes S93/S94 prennent la plus proche de `pi`) ; « le milieu suppose un contour convexe » ne vient pas du brevet. `Bi` est le milieu de `[pi, pi']`, traversée du goulot d'un bord à l'autre. Choix du chemin `A1 → A2` « automatique » sans règle.
- **JP3922316B2** (scan japonais, 13 pages, lisible) : contenu de la fiche confirmé (trois cas, aucun nombre). Enregistré le 2007-03-02, gazette le 2007-05-30. La formulation « noeud d'articulation / pont / feuilles pendantes » est **notre proposition** (équivalente pour une chaîne ou un arbre ; le brevet est muet pour un graphe avec cycles). L'exclusion de `P4` tient à ce qu'une zone fine en bout n'a pas besoin de sous-couche ([0052]). Le critère « sort de la zone » n'a aucun algorithme dans ce brevet. Lecture non juridique : la portée exacte des revendications n'est pas garantie.
- **US6633794B2** : fiche exacte. Revendications indépendantes 1, 8, 17, 22 ; ajustement de durée 147 jours ; seuil de recouvrement partiel absent du texte.

Fiches brevets : stippling, analyse de points et appliqué

> Fiches techniques de travail, issues de la lecture de pages Google Patents (résumés automatiques, pas les PDF). Statuts juridiques préliminaires. Voir l'index et les priorités dans Recherche brevets.

Source unique : Google Patents (pages /en), lues via un outil qui résume la page (pas de texte brut intégral ; pour US6167823B1 les 14 000 derniers caractères n'ont pas été lus ; les figures ne sont pas visibles, seulement décrites par le texte). J-PlatPat non consulté. Tout ce qui est marqué NOTRE PROPOSITION n'est pas dans les brevets.

Corrections de catégorie

- US6167823B1 n'est PAS un brevet de « remplissages décoratifs ». C'est un brevet d'ANALYSE/ÉDITION de données de points via graphiques (points en fonction du temps). Il n'y a aucune génération de remplissage.
- US6968255B1 n'est PAS de l'analyse : c'est la dérivation automatique de motifs de STIPPLING (remplissage décoratif de type méandre fractal) par clipping dans un polygone.
- US5438520A et JP3769602B2 : appliqué (deux brevets Barudan), conforme.

---

1. US6167823B1 — Method and system for computer aided embroidery

(1) Biblio. Buzz Tools, Inc. ; inventeurs John S. Laufer, Lisa A. Laufer. Priorité et dépôt 21/07/1999 ; délivré 02/01/2001. Statut Google : expiré (échéance estimée 21/07/2019, hypothèse non juridique). Famille : PCT/US2000/019976 (WO2001007700A1), AU6364400A, continuation US 09/707,668 (US6502006B1). CPC D05B19/xx. Citations non lues (tronquées).

(2) Revendications clés. 13 revendications. 1 : affichage avec le numéro de point (ou le temps) sur un axe et la longueur de point sur l'autre. 2 : idem avec l'angle de point. 3 : méthode de sélection d'une région dans le graphe temporel, où des groupes de points sont séparés par un long point ; sélection au curseur. 4-5 : surbrillance correspondante dans une vue « points dans l'espace », en couleur différente. 6 : sélection par glissé. 9-10 : graphes de propriétés dérivées (angle) ; 11 : barre de couleurs proportionnelle au nombre de points ; 12-13 : éléments adjacents et événement de changement de couleur.

(3) Algorithme (FROM PATENT, sans équation formelle sauf l'angle). - Modèle : suite ordonnée de coordonnées 1..n entrelacée de codes de contrôle (changement de fil, saut, fin). - Graphe de longueur : une barre par point, hauteur proportionnelle à la longueur (ou à son log), couleur du fil. Les longs sauts marquent les frontières d'éléments. - Graphe d'angle : chaque point est un vecteur vers le suivant ; deux vecteurs consécutifs ramenés à une origine commune, angle entre eux ; si > 180°, remplacé par 360° - angle, soit [0,180]. Signatures décrites : points de course au plancher, zigzag vers 90°, satin vers 180°, tatami en motif complexe avec demi-tours. - Barre de couleurs : un rectangle par série de points de même couleur, longueur proportionnelle au nombre de points. - Sélection « accrochée aux longs points » : longueur-curseur L déduite de la position verticale ; chercher en avant le point le plus proche de longueur > L (distance B) et en arrière (distance D) ; le texte rapporté dit « si B>D choisir A, sinon C », ce qui semble inversé par rapport à « le plus proche » : AMBIGU, à vérifier sur le texte intégral. Idem pour le point final. Tous les points entre les deux bornes sont sélectionnés, surlignés dans les deux vues. - Sélection alternative par glissé (indices min/max) et par couleur (voisin le plus proche de couleur différente). - Manipulations sur la sélection : couleur, transformation, suppression, réordonnancement, insertion d'arrêt couleur. **NOTRE PROPOSITION** : rien d'autre n'est donné ; implémentation naïve O(n) par graphe.

(4) Cas limites / coût. Sélection O(n) par balayage. Cas non traités dans le texte lu : design sans long point, doublons de longueur. Le terme « temps » dépend de la vitesse machine (modèle non donné).

(5) Pertinence OpenStitch : MOYENNE-BASSE. Idée utilisable : un panneau d'analyse (longueur/angle par point, détection de frontières d'éléments) pour `stitch_analysis` et le débogage, côté `libs/` sans Qt (calcul) + widget. Principalement de l'UI ; le calcul est trivial (longueurs, angle entre vecteurs). Rien de nouveau à générer. Le dépôt n'a pas de graphe de ce type (non vérifié en détail).

(6) Juridique (préliminaire). Expiré (2019) selon la source ; revendications d'interface et de méthode d'affichage. Rien n'interdit l'idée aujourd'hui, mais ne pas copier de texte ; vérifier le statut des continuations (US6502006B1) si on s'inspire de ses revendications.

---

2. US6968255B1 — Stippling automatique (Pulse Microsystems / Tajima Software Solutions)

(1) Biblio. Inventeurs Christos Dimaridis, Niranjan Mayya, Thanasis Triantafyllidis, Yanchun Wang. Assignataire initial Pulse Microsystems ; actuel Tajima Software Solutions (fusion 2022). Priorité provisoire 22/10/2004 (60/621,340) ; dépôt 27/10/2004 (US10/974,839) ; délivré 22/11/2005. Statut Google : expiré, échéance ajustée 07/11/2024 (hypothèse). Famille : un seul membre listé. Cite US5430658A (Pulse, fractal auto-généré) et US6807456B1.

(2) Revendications clés. 20 revendications, indépendantes 1, 12, 16, 20. 1 : chemin unique sans auto-intersection dans une frontière géométriquement symétrique, conformé à un polygone asymétrique, points sélectionnés, stockés, segments de points créés. 3 : chemin issu d'une forme fractale itérée. 12 : choix d'une forme fractale, bordure symétrique, itérations, conformation, conversion en données de broderie. 13 : déplacement aléatoire des points. 14-15 : courbes de Bézier puis points de course. 16 : intersections avec le contour, découpe des segments, conservation de ceux à l'intérieur. 20 : combinaison. Remarque : la revendication 11 cite une « bordure de polygone édité » sans antécédent clair.

(3) Algorithme. 1. Boîte englobante du polygone avec marge prédéterminée (FROM PATENT). 2. Chemin : fractale définie par axiome + règles, n itérations ; description à base de Hilbert, segments selon deux directions orthogonales, chaque point visité une fois ; densité réglée par n (FROM PATENT). 3. Clipping (Table 1) : intersections entre polyligne motif et courbes de forme par droites paramétriques L1(t)=A+t·V0, L2(m)=B+m·V1 ; cas non parallèle : m = N0·(A-B)/(N0·V1), t = N1·(B-A)/(N1·V0) avec N0, N1 perpendiculaires à V0, V1, accepté si 0<=t<=1 et 0<=m<=1 (FROM PATENT, équations rapportées par la source). Cas parallèle/colinéaire par projections par produit scalaire, bornées à [0,1]. Trier les paires (k,t), supprimer les doublons, couper le motif, garder l'intérieur, conserver les informations de connexion. 4. Points sur/près du chemin ; longueur max de point définie par l'utilisateur (exemple 2 mm), subdivision au-delà. 5. Aléa (Table 2) : dx = m_len·(rand/RAND_MAX - 0.5)·RandomFactor, idem dy ; « m_len » est illisible dans la source, probablement une longueur de segment : INCERTAIN. 6. Lissage Bézier : pour chaque point intérieur, v1=P[i-1]-P[i], v2=P[i+1]-P[i], angle θ ; points de contrôle donnés par formules (1/3 P[i-1] + 2/3 P[i]) puis divisés par 3, qui paraît une erreur de transcription ; $\theta' = \pi/2 - \theta/2$ (corrigé de π si $|\theta'| >= \pi/2$) ; rotation de $\pm\theta'$ autour de P[i]. À ne PAS reprendre telle quelle : le texte est corrompu. 7. Re-clipping de la courbe lissée à l'intérieur du polygone. La source mentionne aussi du texte parasite (« nutritional blend ») et deux numéros de brevet pour le même renvoi : texte de qualité faible.

(4) Cas limites / coût. Coût O(S·C) naïf (segments du motif × segments du contour) ; accélérable par grille. Problèmes : le clipping coupe le chemin continu en plusieurs morceaux, donc il faut relier les morceaux (travel/jump) : le brevet mentionne des « informations de connexion » mais sans détail lu. Polygones à trous non discutés (dans le texte lu).

(5) Pertinence : MOYENNE (HP-STI-013 stipple/méandre, P2, À faire ; HP-STI-006 motif fill). Nouveau et implémentable : motif de remplissage = courbe de remplissage d'espace (Hilbert/Peano ou autre) clippée au polygone, avec jitter déterministe (graine fixe pour respecter le déterminisme exigé) et lissage. Mais la plupart des briques sont génériques (clipping segment/polygone via Clipper2 dans `geometry`, courbe de Hilbert classique). NOTRE PROPOSITION : utiliser Clipper2 pour le clipping, une graine PRNG explicite, et un chaînage glouton des fragments en minimisant les sauts.

(6) Juridique (préliminaire). Expiré selon la source (2024) ; Hilbert et clipping sont de l'art public ; ne pas copier les formules Bézier douteuses. Revérifier avec un juriste si on se rapproche des revendications 1/16 de façon littérale (aucun risque attendu après expiration, sous réserve de la fiabilité du statut Google).

---

3. US5438520A — Method of creating applique data (Nippon Denpa / Barudan)

(1) Biblio. Assignataires d'origine Nippon Denpa Co., Ltd. et Barudan ; inventeurs Masaaki Satoh, Akinori Kuroda, Masashi Asai ; Google affiche Raytheon comme cessionnaire actuel (données douteuses selon Google). Priorité JP 02/04/1993 (JP5-100442) + priorités 17/04/1993, 22/04/1993, 14/03/1994 ; dépôt US 01/04/1994 (08/221,926) ; délivré 01/08/1995. Statut : expiré, échéance 01/04/2014 ; taxes de maintien payées. Famille : DE4411364C2, JPH06294065A, JP3810095B2, JP3345614B2, JP3878228B2.

(2) Revendications clés. 14. 1 (indépendante) : contour synthétique + bords de chevauchement de deux pièces, contours individuels, données de broderie par pièce. 2 : données de coupe. 3 : décomposition en points de positionnement provisoires, bâti, satin. 4 : départ des points provisoires aux intersections avec les bords de chevauchement. 5-6 : circonférences intérieure/extérieure pour bâti/satin. 7 : changement de fil sur la pièce supérieure entre bord de chevauchement et reste. 8 : choix automatique du côté par lignes de recherche et comptage d'intersections. 9 : zones à deux courbes (ligne frontière, rotation progressive). 10-14 : arrêt arrière (>= 4 points de retour), pas constant, saut, pas grossier, point de fin qui revient sur le dernier retour.

(3) Algorithme (aucune équation dans le brevet). - Entrée : points essentiels R sur le contour combiné ; départ S un peu avant, fin E un peu après pour que la coupe se recouvre. Bords de chevauchement Da (pièce basse), Db (haute). D1a = Dt coupé à Da ; D1b = Dt coupé à Db. Coupe D2a/D2b. - Positionnement provisoire (courses) D3 suivant le contour, avec départ aux intersections des bords de chevauchement ; bâti D5 (E-stitch en zone libre, course aux bords) ; direction et nombre de points saisis au clavier. - Circonférences intérieure D4 et extérieure D6 par décalage ; satin D7 par segments élémentaires perpendiculaires aux courbes, échantillonnés sur les points du contour. - Choix automatique du côté intérieur : lignes de recherche depuis un point du contour, comptage du nombre X d'intersections avec le contour ; parité (0/pair vs impair) combinée à la direction de parcours (montante/descendante) donne le côté (Table 1 : montante + pair => intérieur à droite pour recherche vers la gauche ; impair inverse ; descendante inverse ; recherche vers la droite inverse). C'est un test point-dans-polygone par parité d'intersections. FROM PATENT. - Zones à deux courbes : intersection D4bx / D6bx, ligne fantôme frontière 43 ; la direction du satin devient progressivement parallèle à cette frontière à son approche, ailleurs perpendiculaire au contour. - Échange de fil : bord de chevauchement de la pièce haute dans la couleur de la pièce, reste en couleur contrastée. - Arrêt arrière : aller vers le bord puis retour d'au moins 4 points (modes d'exemple : 15, 15 et 9 points ; 9-15 dans la description), 3 variantes (pas constant ; pas constant + 1 saut ; pas constant puis pas grossier), puis point final qui revient partiellement sur le dernier retour. La pièce dépasse du bord de 1 à 2 mm (jusqu'à ~4 mm) pour accrocher le tissu. Valeurs FROM PATENT, exemples. - Séquence de couture : coupe, positionnement, bâti, satin (bord de chevauchement de la haute d'abord).

(4) Cas limites / coût. Pièces concaves ou à contour non simple : d'où la parité ; décalage de courbes à angles vifs non traité dans le texte lu ; deux pièces seulement (superposées). Coût linéaire ou quadratique naïf pour le test de parité.

(5) Pertinence : HAUTE pour l'appliqué (HP-SPEC-001 P1, HP-SPEC-002 P2 : « rien » existe dans le code métier, vérifié par le roadmap l.1693 ; grep des libs sur « motif/appliqué » ne retourne que des faux positifs « applique »). Nouveau et implémentable : type d'objet appliqué avec passes ordonnées ligne de placement, bâti/tack-down, satin de couverture, arrêts machine (`Stop`), export du contour de coupe via `formats`. L'idée « une saisie, plusieurs sorties » est exactement l'objectif du roadmap. Les détails d'arrêts arrière sont en revanche anciens et spécifiques aux machines Barudan ; peu utiles. Le choix du côté par parité est inutile chez nous (Clipper2 gère l'orientation et les offsets signés).

(6) Juridique (préliminaire). Expiré en 2014 selon la source. Techniques d'appliqué (placement/tack-down/satin) sont communes dans l'industrie. Faible risque ; revalider si on cite le texte.

---

4. JP3769602B2 — Dispositif de création de données de coupe et de broderie d'appliqué (Barudan)

(1) Biblio. Barudan Co., Ltd. ; inventeur Shin Iwata (■■ ■). Priorité 25/01/1993 (JP5-29945) ; dépôt 30/01/1993 (JP03424093A) ; publication A 27/09/1994 (JPH06272151A) ; délivré 26/04/2006. Statut : expiré, échéance 2021-04-26 (hypothèse Google). Famille : AU659042B2, DE4401948C2, US5740055A (publié 14/04/1998). Traduction automatique Google : page lue, original japonais non consulté (J-PlatPat non vérifié).

(2) Revendication. Une seule revendication indépendante (dispositif) : moyen d'entrée du contour du tissu d'appliqué (D1) ; une unité commune qui crée automatiquement les données de coupe et de broderie ; les données de broderie sont dérivées des données de périphérie intérieure (D4) et extérieure (D5), puis des données de point satin (D7).

(3) Algorithme (procédural, sans équation). Une seule saisie du contour D1 (coins C1-C6, arcs R1-R3, S avant C1 et E après C6 pour chevauchement de coupe) → coupe D2 → bâti de positionnement D3 à pas fixe → périphérie intérieure D4 → extérieure D5 → points de fixation D6 (E-stitch ; direction et nombre saisis) → satin D7 en divisant la bande entre D4 et D5 en sections avec direction variant progressivement pour éviter les chevauchements aux coins. Sortie : disquette FD1 coupe, FD2 broderie. Ordre de couture : bâti, fixation, satin.

(4) Cas limites. Coins : variation progressive de direction par section ; pas de détail numérique. Coupe : chevauchement S/E.

(5) Pertinence : HAUTE comme pendant simplifié du précédent (un seul tissu) ; confirme le flux « contour unique → découpe + broderie ». C'est le plus proche de HP-SPEC-001. Rien de plus que US5438520A, sauf la décomposition de la bande satin en sections à direction progressive aux coins (idée déjà couverte par la logique satin du dépôt, cf. docs/source/satin.md, non relue ici).

(6) Juridique (préliminaire). Expiré (2021 estimé) ; le brevet US5740055A de la famille n'a pas été lu et son statut non vérifié ; l'idée générique est ancienne.

Synthèse rapide

- Vraiment nouveau et implémentable : (a) objet appliqué multi-passes avec arrêts + export de contour de coupe (brevets 3 et 4) ; (b) stipple par courbe d'espace clippée avec jitter déterministe (brevet 2) ; (c) panneau d'analyse longueur/angle (brevet 1), faible priorité.
- Limites de ce travail : résumés par outil (pas de texte brut), figures non vues, citations de 6167823 non lues, texte de 6968255 corrompu à plusieurs endroits.

Errata après lecture des PDF (2026-10)

Ces fiches ont été écrites à partir de résumés de pages web. Les 32 brevets ont ensuite été relus sur leur **texte primaire** (PDF Google Patents ; équations et tableaux abîmés par l'OCR relus sur rendu image ; scans japonais lus visuellement). Les points ci-dessous **corrigent ou précisent** le corps de cette page ; en cas de conflit, ils priment. Les figures n'ont été regardées que lorsqu'une équation en dépendait.

- **US6167823B1** : l'ambiguïté « B>D » est levée : le signe est « > » partout (colonnes 10 et 12, figure 11C), mais la règle imprimée choisit le point le plus éloigné alors que le texte dit « le plus proche ». C'est une erreur du brevet ; à corriger côté implémentation. Le double-clic entre deux longs points (figure 11E) et les manipulations sur la sélection manquaient. L'accrochage n'est pas revendiqué (revendications 1 à 13).
- **US6968255B1** : le texte imprime Pulse Microsystems seul (« actuel Tajima » non confirmé). Les formules Bézier sont imprimées exactement comme transcrites (avec le `/3` final et `theAngle += -PI`) : **ce n'est pas une corruption d'OCR, c'est probablement une bévue du brevet**, donc à ne pas reprendre. `m_len` est lisible mais **non défini** dans le texte (lecture plausible, confiance faible : une

longueur de référence qui met l'amplitude de l'aléa à l'échelle). Le cas parallèle de la table 1 est mal parenthésé à l'impression.

- **US5438520A** : le texte imprime « Ippon Denpa » (la fiche écrivait Nippon) ; Raytheon n'y apparaît pas. Quatre priorités JP lues sur l'image. Retour d'au moins 4 points, de préférence 6 puis 10 (description). La revendication 1 inclut la broderie effective. **Aucune distance de décalage D4/D6** n'est donnée. Tables 1 et 2 cohérentes avec la fiche.
- **JP3769602B2** : enregistrement le 17/02/2006 (bulletin du 26/04/2006) ; un recours contre rejet (■■2003-7905, 07/05/2003) était omis. La revendication unique va de D1 à D4 et D5 puis D7 ; elle ne mentionne ni bâti, ni arrêt, ni E-stitch. L'arrêt D6 vient de D4/D5 avec direction et nombre de points saisis au clavier. Aucun pas, largeur ni nombre de sections. Le rendu image de la description est blanc (polices CJK non embarquées) ; le texte a été lu dans la couche texte du PDF.

Palettes et fils

Public : utilisateur avancé, mainteneur. État : **Implémenté / partiel** (fils reliés au document et à l'interface ; nuanciers de marques à fournir par l'utilisateur).

Ce qui existe

- Chaque objet de broderie porte une **couleur RGB** (`std::array<uint8_t,3>`), reprise de la région d'origine, et peut référencer un **fil de nuancier** (`EmbroideryObject::thread`, HP-THR-004) : voir *Fil d'un objet* ci-dessous.
- Un **changement de couleur** est inséré automatiquement entre deux objets consécutifs de couleurs différentes lors de la génération.
- La recoloration d'une région est disponible (choix d'une couleur libre).
- **Catalogue de fils** (`libs/thread_palette`, `openstitch::thread_palette`) : bibliothèque cœur sans Qt ni dépendance tierce, ne liant que `openstitch::core` (C-S1-01). Voir *Catalogue* ci-dessous.

Catalogue (`libs/thread_palette`)

Types valeur (`include/openstitch/thread_palette/thread.hpp`) :

- `ThreadKey { chart_id, code }` — identité stable d'un fil, comparable et ordonnable, sans pointeur vers le nuancier. Destinée à être embarquée par un futur consommateur (`stitch`/`document`/`formats`/`stitch_analysis`, hors périmètre S1) sans tirer le registre entier.
- `Thread { key, brand, range, name, rgb }` — une entrée d'un nuancier.
- `ThreadChart { chart_id, display_name, source_note, threads }` — `source_note` porte la traçabilité du sourçage (fabricant, gamme, URL, date de consultation), en dur dans le binaire.

Registre (`include/openstitch/thread_palette/catalog.hpp`) :

- `all_charts()` — tous les nuanciers compilés, dans l'ordre textuel des appels de `src/catalog.cpp` (déterministe entre deux exécutions et entre plateformes).
- `find_chart(chart_id)` — `nullptr` si l'identifiant est inconnu.
- `find_by_code(chart_id, code)` — `std::optional<Thread>` vide si le nuancier ou le code sont inconnus.
- `search_by_name(needle)` — sous-chaîne insensible à la casse, résultats dans l'ordre du registre.

Deux nuanciers de fabricant sont chargés aujourd'hui, dans cet ordre de déclaration : **Madeira Polyneon** (`madeira_polyneon`) puis **Isacord 40** (`isacord_40`).

> **DONNÉES PLACEHOLDER.** Les deux fichiers de données (`libs/thread_palette/data/madeira_polyneon.cpp`, `libs/thread_palette/data/isacord_40.cpp`) contiennent actuellement des entrées **inventées** (codes, noms, RGB plausibles mais fictifs), pas une transcription du vrai nuancier publié par le fabricant : l'environnement d'implémentation initial n'avait pas d'accès web pour sourcer les valeurs réelles selon S1-POLICY-1. Chaque `source_note` le dit explicitement et pointe vers la page officielle à consulter. **À faire avant toute utilisation hors développement/test** : remplacer ces deux fichiers par une transcription réelle depuis la carte de couleurs officielle de chaque fabricant.

Procédure pour ajouter un nuancier (S1-POLICY-1/2/3) :

1. Sourcer uniquement depuis la page produit ou la fiche technique **publiée par le fabricant lui-même** — jamais depuis une table déjà agrégée par un logiciel ou plugin tiers. Seuls les faits (code, nom commercial, RGB publié) sont repris.
2. Ajouter `libs/thread_palette/data/<nuancier>.cpp`, qui définit une factory `detail::make_<nuancier>_chart()` déclarée dans `src/chart_data.hpp` et retournant un `ThreadChart` construit depuis un tableau de `Thread` compilé (pas de lecture JSON/CSV à l'exécution,

C-S1-02).

3. Enregistrer l'appel de la factory dans `src/catalog.cpp` (`registry()`), dans l'ordre de déclaration souhaité, et ajouter la ligne CMake du nouveau fichier `.cpp` dans `libs/thread_palette/CMakeLists.txt`.

4. Remplir `source_note` : fabricant, gamme, URL source, date de consultation. Ajouter la ligne correspondante dans la sous-section « Nuanciers de fils » de `THIRD_PARTY_LICENSES.md` (une ligne par nuancier, jamais fusionnée).

Distance perceptuelle (`include/openstitch/thread_palette/color_distance.hpp`, module séparé du registre — un consommateur qui ne veut que lister les fils ne paie pas le coût de la conversion couleur) :

- `to_cielab(rgb)` — conversion sRGB → CIELAB (D65, observateur 2°), écrite depuis les formules standard, indépendante de `cv::cvtColor` (`libs/segmentation`, approximation OpenCV non spécifiée bit à bit, et `thread_palette` ne peut lier aucune bibliothèque tierce).
- `ciede2000(a, b)` — distance perceptuelle CIEDE2000, validée en test contre les 34 paires de référence publiées par Sharma, Wu & Dalal (2005).
- `nearest_threads(rgb, chart, top_n)` — top-N fils les plus proches d'une couleur, triés par distance CIEDE2000 croissante, égalités départagées par l'ordre de déclaration du nuancier (déterministe).
- `wcag_contrast(a, b)` — contraste relatif WCAG 2.x (luminance relative sRGB), dans [1 ; 21].
- `best_thread_pair(s1, s2, chart, w = 10)` — meilleure paire de fils pour un fondu à deux fils : minimise `dE00(s1, t1) + dE00(s2, t2) + w · max(C(s1, s2) − C(t1, t2), 0)` sur toutes les paires du nuancier (recherche exhaustive, O(n²)). Le terme de contraste empêche de choisir deux fils trop proches l'un de l'autre quand les couleurs d'origine sont contrastées. Égalités : premier fil puis second dans l'ordre de déclaration. Critère de Liu et al., *Directionality-Aware Design of Embroidery Patterns*, CGF 42(2), 2023.

Bibliothèque de fils active (`ThreadLibrary`)

`include/openstitch/thread_palette/thread_library.hpp` — type valeur, sans E/S, déterministe. `ThreadLibrary::with_builtin()` contient, dans l'ordre : le nuancier **Générique** (`generic_basic_chart()`, 28 couleurs usuelles sans marque, libres de droits), puis les nuanciers compilés de `all_charts()` (Madeira/Isacord, **données de démonstration fictives**, `is_demo_chart`). `add_chart` ajoute un nuancier utilisateur (identifiant unique, codes uniques, jamais de remplacement silencieux) ; `remove_chart` ne retire que les nuanciers utilisateur. `search(needle, chart_id, limit)` cherche dans le code, le nom, la gamme et la marque ; `nearest(rgb, chart_id, n)` classe les fils par distance CIEDE2000 (tous les nuanciers si `chart_id` est vide).

Licences et nuanciers de marques (HP-THR-002)

Les cartes de couleurs des fabricants ne sont **pas redistribuées** : l'application ne livre que le nuancier Générique (valeurs choisies pour ce projet) et laisse l'utilisateur charger les siennes. Madeira/Isacord intégrés restent des données fictives tant qu'une transcription sourcée n'existe pas (HP-THR-002 reste partiel).

Format d'import (menu *Fils* ■ *Importer un nuancier*, `project_io:: read_thread_chart_file`, extension `.csv` ou `.json`, UTF-8) :

- **CSV** — séparateur `,` ou `;` détecté sur l'en-tête, guillemets doubles acceptés, lignes vides et lignes `#…` ignorées. En-tête **obligatoire** (insensible à la casse) : `code,name,hex` (`hex` = `#RRGGBB`) ou `code,name,r,g,b` (0 à 255) ; colonnes facultatives `brand`, `range` (synonymes : `reference`/`ref`, `nom`, `marque`, `gamme`). Le nom du fichier donne le nom du nuancier.
- **JSON** — `{"name": "Mes fils", "source": "carte 2024", "threads": [{"code": "1001", "name": "Rouge", "rgb": "#C8102E" ou [200,16,46], "brand": "…", "range": "…"}]}`.

L'identifiant du nuancier est dérivé du nom (`user_` + minuscules `[a-z0-9_]`) : un fichier ne peut pas se faire passer pour un nuancier intégré. Toute ligne invalide (code vide ou en double, couleur illisible) fait échouer l'import avec son numéro de ligne : jamais de nuancier à moitié lu. L'application copie le fichier dans son dossier de données (`thread_charts/`) et le recharge à chaque lancement ; les nuanciers ne sont **jamais** stockés dans le `.osp`.

Fil d'un objet (HP-THR-004)

`EmbroideryObject::thread` (`std::optional<ThreadKey>`) : le fil assigné, vide pour une couleur libre. `rgb` reste la **source de rendu** ; assigner un fil copie sa couleur dans `rgb`. `commands::SetObjectThreadCommand` (header `libs/commands/.../thread_commands.hpp`) assigne un fil, ou une couleur libre (HP-OBJ-018, version RGB), à un ou plusieurs objets en **un seul pas d'annulation** ; les états précédents (fil + couleur) sont restitués exactement. Persistance : champ `thread` du `.osp`, schéma **v6**, projet plus ancien = couleurs libres (voir *Format de projet*). `color_blocks` renseigne `ColorBlock::thread_key` depuis le fil de l'objet.

API « fils du design » (`stitch_analysis::thread_usage`)

`libs/stitch_analysis/include/openstitch/stitch_analysis/thread_usage.hpp` — Qt-free, déterministe, destinée au panneau Fils **et** à la future fiche de production imprimable (une seule dérivation des fils utilisés). Toujours appelée avec la séquence **effective** (`stitch_generation::effective_sequence`).

- `thread_usage(project, sequence, library*, options)` → un `ThreadUsage` par fil, dans l'ordre de première utilisation : `identity` (fil de nuancier, sinon RGB libre), `brand`/`code`/`name` (résolus dans la bibliothèque si fournie), `objects` (ordre de couture), `stitch_count`, `length_mm` (fil cousu piqûre à piqûre, sauts exclus), `color_blocks` (passages), `estimated_seconds` (piqûres ÷ vitesse + changements de fil
- coupes ; hypothèses dans `ThreadUsageOptions` : 700 pts/min, 30 s par changement, 3 s par coupe). Deux objets partagent un fil si leurs fils sont égaux, ou, sans fil, si leur RGB est égal. Les points sans objet source (DST importé) tombent dans une couleur libre noire, comme `color_blocks`.
- `thread_label(usage)` : « Marque Réf — Nom » ou `#RRGGBB`.
- `thread_usage_csv(usage)` : export tableur (`;`, nombres au point décimal).
- `objects_using_thread(project, identity)` : sélectionner / remplacer un fil.

Film couleur (HP-THR-005)

`color_film(project, sequence)` liste les blocs de couleur dans l'ordre de couture (`ColorFilmBlock` : fil, objets, points, longueur, `locked`). `reorder_film_blocks(project, film, from, to)` et `merge_same_thread_blocks(project, film)` renvoient le **nouvel ordre complet des objets** (à passer à `ReorderEmbroideryCommand`, un pas d'annulation). Les objets **figés** et les objets absents du film (masqués) gardent leur emplacement absolu ; seuls les objets libres sont redistribués dans les emplacements libres. Fusionner les blocs de même fil réduit les changements mais modifie l'empilement des couches (le fil cousu plus tard passe plus tôt) : l'interface le dit et l'opération reste annulable.

Réduction de palette (HP-THR-011)

`thread_palette::reduce_colors(couleurs pondérées, N)` : fusion agglomérative par distance CIEDE2000 (les deux couleurs les plus proches fusionnent, la plus lourde garde sa teinte exacte), déterministe (égalités départagées par l'ordre d'entrée). Deux usages : `AutoOptions::max_threads` (« Limiter à N fils » de la numérisation automatique : les régions sont recolorées avant la création des objets, la géométrie est inchangée, le fond ignoré ne compte pas) et le bouton *Limiter à N fils* du panneau Fils sur un motif existant (un pas d'annulation).

Panneau Fils

`apps/desktop/thread_panel.*` (présentation seule) + `main_window_threads.cpp` (câblage, aucune règle métier) : onglets *Projet*, *Catalogues* et *Film couleur*, menu *Fils* (voir le *Guide utilisateur*). Clic sur un fil du catalogue = assigner à la sélection (objets de broderie sélectionnés, ou ceux des formes sélectionnées).

Ce qui n'existe pas encore

- Pas de filtrage par gamme possédée / nuancier personnel éditable (HP-THR-007) ;
- pas de coloris multiples (HP-THR-006), de poids/type de fil (HP-THR-009) ni de pipette (HP-THR-012) ;
- les noms/références de fils sont portés par le `.osp`, la liste CSV et la future fiche de production ; **aucun format machine implémenté (DST) ne les porte** ;
- aucune carte de marque réelle n'est livrée (licences) : seuls le nuancier Générique et deux nuanciers de démonstration fictifs sont intégrés, les autres marques citées par HP-THR-002 (Robison-Anton, Sulky, Gunold, Marathon, Floriani, Coats, Brother, Janome, Pantone approximé…) passent par l'import utilisateur.

Conséquence pour le DST

Le format DST ne stocke de toute façon **pas** les couleurs réelles, seulement des arrêts « changement de fil ». L'ordre des couleurs est porté par le document `.osp`, pas par le DST (voir *Format DST*).

Implémentation associée

- `libs/thread_palette/include/openstitch/thread_palette/thread.hpp` — types `ThreadKey`/`Thread`/`ThreadChart`.
- `libs/thread_palette/include/openstitch/thread_palette/catalog.hpp` et `src/catalog.cpp` — registre et recherche.
- `libs/thread_palette/include/openstitch/thread_palette/color_distance.hpp` et `src/color_distance.cpp` — sRGB→CIELAB, CIEDE2000, `nearest_threads`.
- `libs/thread_palette/data/madeira_polyneon.cpp`, `libs/thread_palette/data/isacord_40.cpp` — nuanciers compilés (données placeholder, voir avertissement ci-dessus).
- `libs/thread_palette/.../thread_library.hpp`, `chart_import.hpp`, `color_reduction.hpp` — bibliothèque active, import CSV, réduction de palette.
- `libs/project_io/.../thread_chart_io.hpp` — import de fichiers CSV/JSON.
- `libs/document/.../embroidery_object.hpp` — champs `rgb` (rendu) et `thread`.
- `libs/commands/.../thread_commands.hpp` — `SetObjectThreadCommand`.
- `libs/stitch_analysis/.../thread_usage.hpp` — `thread_usage`, film couleur.
- `apps/desktop/thread_panel.*`, `apps/desktop/main_window_threads.cpp` — panneau.
- Tests : `tests/unit/thread_palette/test_library.cpp`, `tests/unit/commands/test_thread_commands.cpp`, `tests/unit/project_io/test_thread_persistence.cpp`, `tests/unit/stitch_analysis/test_thread_usage.cpp`, `tests/unit/autodigitize/test_thread_limit.cpp`, `tests/unit/desktop/test_thread_panel.cpp`.
- `libs/stitch_generation/src/generate.cpp` — insertion des `ColorChange`.
- `libs/segmentation/src/segmentation.cpp` — `recolor_region`.

Simulation de couture

Public : utilisateur. État : **Implémenté** (base).

But

Rejouer visuellement la couture pour repérer les sauts, l'ordre et la trajectoire de l'aiguille avant l'export.

Fonctionnement

La barre de simulation apparaît dès qu'une séquence de points existe. Elle offre :

- un bouton **Lecture / Pause** ;
- un **curseur** qui révèle la couture jusqu'à un index de point ;
- un **compteur** « point courant / total » ;
- un **repère d'aiguille** (cercle rouge) à la position courante.

Pendant la lecture, un minuteur (~60 pas/seconde) avance l'index proportionnellement à la taille du motif. Seule la **couche des points** est reconstruite à chaque image (l'image de fond et les vecteurs ne sont pas recalculés), pour rester fluide.

Limitation : la simulation est **point par point** via le curseur. Le réglage de vitesse explicite, l'avance objet-par-objet ou couleur-par-couleur, et l'affichage de l'objet actif ne sont **pas** exposés (le modèle en garde l'information : chaque `StitchCommand` porte son `ObjectId` source).

Implémentation associée

- `apps/desktop/main_window.cpp` — `buildSimulationToolbar`, `toggleSimulation`, `onSimTick`, `onSimSliderMoved`, `renderStitches`.
- `libs/stitch/include/openstitch/stitch/sequence.hpp` — `StitchSequence`, `StitchCommand` (avec `source`).

Analyse et validation

Public : utilisateur avancé, développeur. État : **Implémenté** (règles de base).

But

Détecter, avant l'export, les problèmes courants de broderie sans jamais appliquer de correction automatique silencieuse.

Règles détectées

`analyze(sequence, options)` renvoie une liste de `Finding` triés par gravité :

Catégorie	Condition (par défaut)	Gravité
`vide`	aucun point	Erreur
`point-court`	segment cousu < 0,5 mm, hors points d'arrêt (passe `Lock`)	Avertissement
`point-long`	segment cousu > 7 mm	Avertissement
`saut-long`	déplacement > 30 mm	Avertissement
`saut-sans-coupe`	déplacement > `trim_threshold` (3 mm) sans `Trim`	Avertissement
`hors-cadre`	point hors du cadre (si fourni)	Erreur
`trop-de-points`	> 100 000 points	Avertissement
`couches-superposees`	épaisseur de fil > `max_layer_thickness` (3,0 couches de remplissage dense) sur un bloc d'au moins 2 × 2 cases de 2,5 mm	Avertissement

Épaisseur de fil (`couches-superposees`). Grille de `layer_cell` (2,5 mm) ; dans chaque case, épaisseur = longueur de fil × `thread_width` (0,4 mm) / surface de la case, en « couches de remplissage dense » : un remplissage dont l'écart entre rangées égale la largeur du fil vaut 1, une sous-couche espacée de 2 mm vaut environ 0,2. Deux remplissages superposés avec leurs sous-couches font ~2,4 (les bords ~2,6), trois ~3,6 ; le seuil de 3,0 les départage. Les passes `Lock` et `Travel` ne comptent pas. Seule une zone contenant un bloc de 2 × 2 cases en excès est signalée : le bord d'un objet, ses connexions de rangées et deux objets voisins ou empiétant d'1 mm (pratique courante contre les interstices) ne déclenchent rien. Un seul résultat par zone, localisé sur sa première case. Motivation : Liu et al. (CGF 2023) ne superposent que deux couches pour limiter la déformation du tissu. `max_layer_thickness = 0` désactive la règle. Les seuils sont des heuristiques d'ingénierie, pas des valeurs de l'article.

Chaque problème porte une **gravité** (`Info`/`Warning`/`Error`), un **message**, une **localisation** et l'**objet** concerné. Un plafond par catégorie évite l'inondation ; le résultat est **déterministe**.

Mesures de qualité (Lot G, audit marine plein cadre)

En plus des règles, deux fonctions **mesurent** une séquence (`openstitch-cli stats` et `digitize` les impriment) :

- `sequence_metrics(sequence, options)` (`metrics.hpp`), calculable sur un DST relu : nombre de points, de déplacements (une suite de sauts = un déplacement) et de coupes ; **déplacements plus**

longs que `trim_threshold` sans coupe ; **points de moins de 0,5 mm hors points d'arrêt**, les points d'arrêt courts étant comptés à part. Sur un DST relu, les passes sont perdues : avec `infer_locks`, un point qui revient exactement sur la position d'il y a deux piqûres (aller-retour) est reconnu comme point d'arrêt. S'y ajoute l'**histogramme des directions** des points d'au moins 1 mm, modulo 180°, par tranches de 5°. Une piqûre de longueur nulle à l'arrivée d'un saut compte comme point court : c'est un enregistrement de 0 mm dans le fichier machine. `length_tolerance` évite de compter comme « long » un déplacement que la quantification DST (pas de 0,1 mm, erreur jusqu'à ~0,07 mm) a fait passer d'un cheveu au-dessus du seuil ; `openstitch-cli stats` relit un DST avec une tolérance d'une unité DST (0,1 mm).

- `project_metrics(project, sequence, options)` (`project_metrics.hpp`), qui a besoin du projet : **objets brodés de moins de 3 mm²** (objet vectoriel suivi, compté une fois), **angles de remplissage** des tatami, **part de l'image non couverte** par les points et une ventilation des déplacements et des points courts par type d'objet et passe.

La couverture se calcule sur les pixels de la segmentation, hors fond transparent et hors couleur du fond ignoré (`excluded_rgb`). Un pixel est couvert si son centre est à moins de `coverage_width / 2` (0,25 mm) d'un segment cousu. Le repère est le repère vectoriel centré sur l'image, comme à la vectorisation. Cette mesure n'est pas calculable depuis un DST seul (origine décalée, pas d'image).

`point-long` fantôme juste après un saut (2026-08-12)

Défaut trouvé en usage réel (export debug utilisateur, objet satin) : la règle `point-long` comparait chaque point cousu à `prevStitch`, la position du DERNIER point cousu — sans jamais tenir compte d'un `Jump` intercalé entre les deux. Un `Jump` lève l'aiguille : le fil n'est plus continu, donc la distance entre le point cousu juste avant le saut et celui juste après n'a AUCUN sens en tant que « longueur de point cousu ». Or `emit_polyline` (§ *Génération de points*) fait systématiquement suivre un `Jump` d'un `Stitch` à la position d'atterrissage (point de « pinning », distance nulle) : la comparaison portait donc en réalité sur la longueur du SAUT lui-même, rejouée comme un second avertissement `point-long` trompeur en plus du `saut-long` déjà émis pour le même saut — quasi systématique dès qu'un saut dépassait 7 mm (le seuil `point-long`), qui est bien plus bas que le seuil `saut-long` (30 mm).

Corrigé en réinitialisant `hasPrevStitch` sur tout `Jump` : la continuité du fil (et donc la comparaison `point-court`/`point-long`) ne traverse plus un saut. Vérifié : `tests/unit/stitch_analysis/test_analyze.cpp` (absence de `point-long`/`point-court` fantôme après un saut, y compris à distance d'atterrissage nulle).

Dans l'interface

Analyse → **Analyser le motif** (F5) remplit le dock *Analyse* ; un double-clic sur un problème centre la vue sur sa localisation. Le cadre utilisé est le cadre courant (100 × 100 mm par défaut).

Limitation : l'analyse **spatiale de densité** (carte de chaleur, superposition de couches, satin trop large/étroit dédié, alignement excessif des pénétrations) et les **corrections automatiques proposées** sont **prévues**, non implémentées.

Implémentation associée

- `libs/stitch_analysis/include/openstitch/stitch_analysis/analyze.hpp` — `Finding`, `Severity`, `AnalysisOptions`.
- `libs/stitch_analysis/src/analyze.cpp` — `analyze`.
- `apps/desktop/main_window.cpp` — `buildAnalysisPanel`, `runAnalysis`.
- Tests : `tests/unit/stitch_analysis/test_analyze.cpp`.

Fiche de production

Public : utilisateur, développeur. État : **Implémenté** (HP-PROD-001 ; durée estimée : première version de HP-PROD-002).

But

Une page A4 portrait à imprimer ou à joindre à un devis / un ordre de broderie : tout ce qu'un brodeur doit savoir avant de lancer la machine, sans ouvrir OpenStitch.

Contenu

1. **En-tête** : nom du projet et date.
2. **Aperçu du motif** : rendu réaliste (`libs/stitch_render`, mêmes réglages que *Affichage* ■ *Rendu réaliste*) ; sur un très grand motif, où le fil texturé serait illisible, des lignes colorées. L'image porte un **texte alternatif** complet.
3. **Caractéristiques** : dimensions réelles en mm (largeur × hauteur), taille du cadre et indication « le motif tient dans le cadre / dépasse du cadre », nombre de points, de sauts, de coupes, de changements de couleur, de couleurs, longueur de fil estimée, temps de broderie estimé.
4. **Blocs de couleur dans l'ordre de couture** : n°, pastille **et code hexadécimal** (la couleur n'est jamais portée par la seule teinte), référence de fil, points, sauts, longueur de fil, objets concernés.
5. **Avertissements** : résultats de `stitch_analysis::analyze` (gravité en toutes lettres, objet nommé, piste de correction ; les problèmes au-delà du plafond par catégorie sont comptés).
6. **Notes** : texte libre saisi dans le dialogue.

Hypothèses du temps estimé

```
minutes = points / vitesse + changements de fil × 30 s + coupes × 2 s
```

La vitesse par défaut est de **700 points/min** (plage usuelle 600–800), réglable dans le dialogue (100–2000) ou par `--speed` en ligne de commande ; les 30 s et 2 s sont les valeurs de `ProductionOptions`. Les hypothèses sont **rappelées sur la fiche**. C'est un ordre de grandeur : la vitesse réelle dépend de la machine, du fil et du motif. La **longueur de fil** est le fil du dessus (somme des segments cousus de chaque bloc) : ni canette, ni fil de saut.

Référence de fil

Le bloc affiche « marque gamme code - nom » quand son `ColorBlock::thread_key` est connu (nuancier de `libs/thread_palette`). Aujourd'hui les objets du document ne portent que leur couleur RVB : le champ reste vide (« — ») et la fiche n'invente jamais de référence. Il se remplira sans changement de format dès que les blocs porteront leur clé de fil.

Architecture

- `libs/stitch_analysis/…/production_sheet.hpp` : `make_production_sheet(project, sequence, options)` produit un `ProductionSheet` pur (aucun Qt, aucune horloge : la date est fournie) à partir de la séquence **effective** et de `color_blocks` (la table de blocs unique). `production_to_json` (JSON stable, schéma 1) et `production_to_html` (page autonome écrite dans le sous-ensemble de HTML/CSS de `QTextDocument`) vivent dans la même bibliothèque.
- `apps/desktop/production_dialog.*`, `main_window_production.cpp` : dialogue **Fichier ■ Fiche de production…**. Il édite nom, date, vitesse, notes, affiche le document HTML, l'**imprime** (`QPrinter`, avec aperçu `QPrintPreviewDialog`) ou l'**exporte en PDF** (`QPdfWriter`, A4 portrait, marges 15 mm). Le document imprimé est celui de la bibliothèque : aucun calcul dans l'interface.

- `openstitch-cli production` : voir ligne de commande.

Limites

- Le PDF n'est produit que par le bureau (Qt) ; le CLI écrit du HTML autonome (aperçu SVG intégré), imprimable depuis un navigateur.
- Pas de champ « client » dédié : à écrire dans les notes.
- Un design importé (DST) n'a ni couleur ni référence de fil : tous les blocs sont noirs.
- Gabarit 1:1 (HP-PROD-003), consommation par couleur et canette (HP-PROD-005) : non faits.

Tests

`tests/unit/stitch_analysis/test_production_sheet.cpp` (totaux, blocs, temps, cadre, référence de fil, JSON/HTML déterministes et échappés, séquence vide) ;
`tests/unit/desktop/test_production_dialog.cpp` (champs → fiche, PDF écrit).

Partie II

Formats, architecture et développement

Format DST (Tajima)

Public : utilisateur avancé, développeur, mainteneur. État : **Implémenté** (encodeur + décodeur maison, testés en aller-retour).

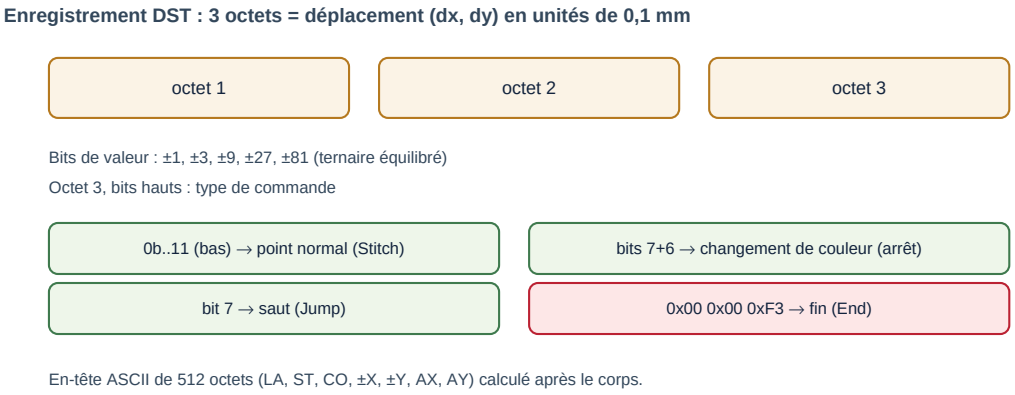


Figure — Structure d'un enregistrement DST de 3 octets et des commandes.

Rôle du format

Le DST (Tajima) est le format d'échange le plus répandu pour la broderie machine. C'est un fichier **binaire** contenant un en-tête ASCII puis une suite de **déplacements relatifs** de l'aiguille. Le module `formats` implémente son encodage et son décodage **sans dépendance externe** (le format est documenté publiquement ; la bibliothèque libembroidery n'a servi que de référence croisée).

Unités

Le pas du DST est **0,1 mm**. En interne, le logiciel travaille en **micromètres** (1 µm = 1/1000 mm) ; la conversion vers le DST se fait par division par 100, et la quantification n'intervient **qu'à l'export**.

En-tête (512 octets ASCII)

Champs terminés par CR, complétés d'espaces jusqu'à 512 octets. Calculés **après** le corps (donc cohérents avec les octets réellement produits) :

Champ	Signification
`LA:`	nom du motif (16 caractères)
`ST:`	nombre d'enregistrements
`CO:`	nombre de changements de couleur
`+X:` `-X:` `+Y:` `-Y:`	étendue du motif (unités 0,1 mm)
`AX:` `AY:`	position finale relative au départ
`MX:` `MY:` `PD:`	multi-motif (valeurs par défaut)

Corps : enregistrements de 3 octets

Chaque enregistrement encode un déplacement (dx, dy) dans [–121, +121] unités de 0,1 mm, par bits de valeurs ±1, ±3, ±9, ±27, ±81 (ternaire équilibré) répartis sur les trois octets. Les bits hauts de l'octet 3 donnent le **type** :

Commande interne	Encodage DST
`Stitch` (point)	bits bas `11`, point normal
`Jump` (saut)	bit 7 de l'octet 3
`ColorChange` / `Stop`	bits 7 + 6 de l'octet 3
`Trim` (coupe)	convention : N sauts (défaut 3) — de délta nul (historique) ou de 0,1 mm non nuls en triangle (export machine)
`End` (fin)	`0x00 0x00 0xF3`

Encodage (sans dérive)

`encode_dst` quantifie les **positions absolues** au pas de 0,1 mm puis en dérive les deltas : `delta_i = round(pos_i/100) − round(pos_{i-1}/100)`. Cela borne l'erreur à ±50 µm **sans dérive cumulative**. Tout déplacement dépassant ±121 est **subdivisé** en sauts intermédiaires. La sortie est **déterministe**.

Coupes (`Trim`) : `trim_jumps` (3) sauts de délta nul, suivis du déplacement éventuel. Les coupes automatiques de la séquence effective (Lot E, *Moteur de génération de points*) sont relues comme des coupes par `decode_dst`, déplacement long compris (vérifié par test). Un **saut sous la résolution** (délta nul une fois quantifié) ne porte aucune information : l'encodeur n'en garde qu'un par série, faute de quoi trois sauts minuscules consécutifs seraient relus comme une coupe fantôme.

Coupes à sauts non nuls (export machine). Les sauts de délta nul sont ignorés ou supprimés par certaines machines et certains logiciels de transfert : la coupe est alors perdue et le fil reste tendu d'un objet à l'autre. L'export vers une machine (`export_machine_file`, donc le bureau et la CLI) écrit chaque coupe comme trois sauts de 0,1 mm **non nuls** formant un triangle `(+1,0) (0,+1) (−1,−1)` suivis du déplacement réel, corrigé du décalage cumulé (position d'arrivée exacte). `DstWriteOptions::trim_jumps_with_movement` (faux par défaut : octets historiques inchangés) active cette forme ; `decode_dst` relit les deux formes comme une coupe.

Coupe finale. La séquence effective se termine par une coupe (`finish_sequence`) : sans elle la machine s'arrête fil attaché au dernier point. Le fichier finit donc par la coupe puis `00 00 F3`.

À valider sur machine : la forme à sauts non nuls est la convention la plus répandue mais n'a pas été essayée sur votre machine ; le seuil de trois sauts (`trim_jumps`) peut devoir être augmenté selon le modèle.

Décodage (tolérant)

`decode_dst` lit l'en-tête de façon **laxiste** (la vérité est le corps), décode chaque enregistrement, et réinterprète une rafale de sauts nuls en `Trim`. La lecture **s'arrête au marqueur de fin `00 00 F3`** et **ignore tout octet suivant** : certains logiciels (p. ex. **Hatch**) ajoutent un octet de fin DOS `0x1A` — voire du padding — après le marqueur, ce qui ne doit pas faire échouer l'import. Les cas d'erreur **couverts par les tests** (fichier vide, tronqué, sans marqueur de fin, octets en trop après `F3`) renvoient une erreur structurée (ou décodent proprement) au lieu de provoquer un arrêt du programme.

Exemple interprété (motif minimal)

Un carré de 5 mm : après l'en-tête, une commande `Jump` amène au premier coin, puis quatre points `Stitch` de 5 mm parcourent les côtés, et l'enregistrement `0x00 0x00 0xF3` termine. Sur ce motif, `+X:` et `+Y:` valent 50 (5 mm en unités de 0,1 mm) et `CO:` vaut 0.

Limites et informations perdues

Avertissement : Un DST **ne conserve pas** les objets éditables de haut niveau (contours, remplissages, colonnes satin, paramètres). Il ne stocke pas non plus les **couleurs réelles** (seulement des arrêts). Après un export DST, seul le **projet `.osp`** permet de retrouver la géométrie et les paramètres.

Tests

- **Aller-retour** : séquence → encode → decode → séquence' (mêmes types, positions à ±50 µm) — dont un test sur **tous les deltas** de −121 à +121.
- **Subdivision** d'un déplacement > 12,1 mm.
- **Fichiers invalides** (vide, tronqué, sans fin) refusés proprement.
- **Déterminisme** octet à octet.

Implémentation associée

- `libs/formats/include/openstitch/formats/dst.hpp` — API.
- `libs/formats/src/dst.cpp` — `encode_dst`, `decode_dst`, `write_dst_file`, `read_dst_file`.
- `apps/cli/main.cpp` — `stats`, `dst2svg`.
- Tests : `tests/unit/formats/test_dst.cpp`.

Formats PES, JEF et EXP

Public : utilisateur avancé, développeur, mainteneur. État : **Implémenté, non validé sur machine réelle** (HP-FMT-002 à 005). Les codecs sont testés en aller-retour et sur des octets calculés à la main, mais **aucun fichier produit ici n'a été ouvert par une machine Brother, Janome ou Melco ni par un logiciel tiers** : voir *Limites*.

Où ça vit

- `libs/formats/src/pes.cpp`, `jef.cpp`, `exp.cpp` : un codec par format, qui ne fait que la sérialisation propre au format. Toute la normalisation (découpage des déplacements, quantification sans dérive, représentation des coupes) vient de `normalize_for_machine` (HP-FMT-001, voir Format DST).
- `libs/formats/src/codec_common.*` : application des options machine, boîte englobante, vérification des limites. `machine_palettes.*` : tables de fils.
- Inscription au registre : une entrée par format dans `format_registry.cpp` (`pes`, `jef`, `exp` ; DST inchangé). `FormatInfo` gagne `encode_ex` / `decode_ex` (avec options et couleurs) et `carries_colors`.
- `formats::check_export_limits` : analyse pré-export propre au format (voir plus bas).
- `project_io::export_machine_file(projet, format, chemin, options)` et `import_machine_file` restent génériques ; l'export fournit les couleurs des blocs (`stitch_analysis::color_blocks`) aux formats qui en portent, l'import les relit.

Options machine (`MachineExportOptions`)

Option	Valeurs	Effet
`trims`	`Native`, `Drop`	coupe codée à la façon du format, ou remplacée par un saut
`stops`	`Native`, `AsColorChange`, `Drop`	arrêt machine propre au format, traité comme un changement de couleur, ou ignoré
`color_changes`	`Emit`, `Drop`	changements de couleur écrits ou supprimés (motif monochrome)
`block_colors`	RGB par bloc	couleurs réelles (PES, JEF) ; vide = palette par défaut déterministe
`design_name`	texte	nom du motif (PES : 16 caractères ASCII)
`jef_timestamp`	`AAAAMMJJhhmmss`	horodatage JEF, **fixe par défaut** (sortie octet-exacte)

Dans la CLI : `osp2dst projet.osp sortie.pes [--no-trims] [--no-color-changes] [--stops native|as-color-change|drop]` (le format vient de l'extension). Dans le bureau : **Fichier ■ Exporter une broderie machine…** ouvre un dialogue (format, coupes, arrêts, changements de couleur, résumé recalculé), puis le choix du fichier avec l'extension du format.

EXP (Melco / Bernina)

Flux sans en-tête ni marqueur de fin, paires d'octets signés (dx, dy) en 0,1 mm, **axe Y vers le haut** (comme le DST). Contrôle : `0x80` suivi d'un code : `0x04` saut (dx dy), `0x80` coupe (`07 00`), `0x01` changement de couleur ou arrêt (`00 00`). Delta max ±127 : les grands déplacements sont découpés en sauts. Pas de couleur dans le fichier (le fichier compagnon `.inf` n'est pas écrit, voir HP-FMT-009). Un arrêt devient un changement de couleur.

JEF (Janome)

En-tête de `0x74` octets : décalage des points, `0x14`, horodatage (14 caractères), nombre de couleurs, nombre de paires d'octets du flux, code de cadre (50x50 = 1, 126x110 = 3, 140x200 = 2, 200x200 = 4, repli 0), étendues du motif (demi-largeur, demi-hauteur), puis quatre groupes de marges par cadre (110x110, 50x50, 140x200, cadre personnalisé ; `-1` si le motif dépasse). Table de fils : un indice de la palette Janome (78 entrées) par bloc, `0` = arrêt machine, puis un mot `0x0D` par entrée. Flux : `dx dy` signés ; `80 01 dx dy` changement/arrêt ; `80 02 dx dy` saut ; trois sauts nuls = coupe ; `80 10` fin. **Axe Y vers le haut**, delta max ±127. Le motif est écrit **centré** sur l'origine machine (premier déplacement = positionnement, retiré à la relecture). Deux fils différents consécutifs ne reçoivent jamais le même indice.

PES (Brother / Babylock / Bernette)

Écriture : PES version 1 « complète » : `#PES0001`, décalage du bloc PEC, section PES (objets `CEmbOne` / `CSewSeg`, sections de points cousus ou de sauts avec l'indice de fil, journal des couleurs), puis bloc PEC :

- en-tête de 512 octets : `LA:` + nom sur 16, taille des vignettes (6 octets x 38), nombre de couleurs - 1, **indices de la palette Brother à 64 entrées** (fil le plus proche, CIEDE2000, via `thread_palette`) ;
- bloc de points : `00 00`, longueur 24 bits, `31 FF F0`, largeur, hauteur, `0x01E0`, `0x01B0`, puis flux : forme courte (1 octet, 7 bits, |d| <= 63) ou longue (2 octets, 12 bits, drapeau saut `0x10`, coupe `0x20`), `FE B0 xx` changement de couleur (`xx` alterne 02/01), `FF 00` fin ;
- vignettes 48x38 (une d'ensemble + une par bloc de couleur), 1 bit par pixel, bit de poids faible d'abord.

Axe Y vers le bas dans le fichier (converti). Un arrêt est un changement de couleur vers le **même** indice de fil ; à la lecture, un changement vers le même indice que le précédent est donc relu comme arrêt. Lecture : PES de toute version (le décalage du bloc PEC est lu dans l'en-tête ; la section PES n'est pas interprétée, les couleurs viennent de la table PEC). Pas d'écriture PES v6 (couleurs RGB, noms de fils).

Aller-retour et déterminisme

Sortie octet-exacte d'une exécution à l'autre. Aller-retour **exact** (types et positions au pas de 0,1 mm) pour les motifs dont les déplacements tiennent dans un enregistrement ; sinon les sauts intermédiaires du découpage réapparaissent à la relecture et un second tour redonne les **mêmes octets** (idempotence, testée). La relecture place le point de départ du motif à l'origine (comme `decode_dst`).

Analyse pré-export

`formats::check_export_limits(sequence, format, options)` retourne des `ExportIssue` (Erreur / Avertissement / Information) : plus de 255 blocs de couleur en PES, étendue au-delà de 3276,7 mm en PES (coordonnées 16 bits) -- ces deux cas sont **bloquants** et l'encodeur renvoie la même erreur claire ; déplacements qui seront découpés ; cadres connus (Janome 200 x 200, Brother 360 x 200) ; couleurs perdues en EXP/DST. Le dialogue d'export les affiche et désactive « Choisir le fichier » en cas d'erreur.

Sources consultées et licences

Aucun code GPL consulté ni repris (ni Ink/Stitch, ni Embroidermodder). Les structures des formats ont été recoupées avec **pyembroidery** (licence MIT) : lecture des modules `PesWriter`, `PecWriter`, `PecReader`, `PecGraphics`, `JefWriter`, `JefReader`, `ExpWriter`, `ExpReader`, pour *comprendre* le format ; le code est entièrement réécrit en C++. Les **tables de 64 fils PEC et de 78 fils JEF** (valeurs RGB factuelles) en sont transcrites : attribution dans `THIRD_PARTY_LICENSES.md`. Les valeurs de nos tests sont synthétiques, aucun fichier propriétaire.

Limites

- **Rien n'est validé sur machine réelle ni avec un visualiseur tiers** (critères d'acceptation des HP-FMT-002/004/005, HP-PHYS-001) : conventions d'axe Y (PES bas, JEF/EXP haut), centrage, octets de la section PES, ordre des bits des vignettes et sens exact des codes sont issus de la documentation publique, non vérifiés ici.
- PES v6, noms/marques de fils, fichier `.inf` EXP, profil de cadre (HP-HOOP-001) : absents.
- Le code de cadre JEF est déduit de la taille du motif, pas choisi par l'utilisateur.
- Pas de corpus de fichiers réels ; fuzzing = octets tronqués/altérés/aléatoires (aucun plantage), pas encore HP-QA-003.

Format de projet `.osp`

Public : développeur, mainteneur. État : **Implémenté**.

Nature

Un projet `.osp` est une **archive ZIP** (via minizip-ng) contenant :

Entrée	Contenu
`project.json`	tout le document, en JSON versionné
`original.png`	l'image source, encodée PNG (sans perte)
`segmentation.u32`	la carte des labels, en binaire little-endian (si présente)

Le JSON et le binaire sont séparés pour ne pas alourdir le JSON (la carte des labels peut faire plusieurs mégaoctets).

Compression (2026-10) : l'image PNG, déjà compressée, est stockée telle quelle dans le ZIP ; le JSON et les étiquettes sont en DEFLATE niveau rapide. Le fichier reste un ZIP standard, lisible par les versions précédentes.

Contenu de `project.json`

- `schemaVersion` (entier) et un objet `document` ;
- `mmPerPx` (résolution de travail) et `objectIdLast` (compteur d'ids) ;
- `canvas` : taille du cadre de broderie (`width`, `height` en µm) et marge de sécurité optionnelle (`margin`, µm, écrite seulement si > 0 : la zone brodable est le cadre réduit de cette marge). **Optionnel et rétrocompatible** : un projet antérieur sans ce champ retombe sur 100×100 mm, et sans `margin` sur une marge nulle ;
- `ops` : la pile d'opérations d'image (chaque opération porte son `type`) ;
- `segmentation` : dimensions et régions (id, rgb, nombre de pixels ; les labels sont dans `segmentation.u32`) ;
- `vectorObjects` : id, nom, couleur, visibilité, région source, `paths` (chemins avec nœuds et tangentes optionnelles) ;
- `embroideryObjects` : id, nom, couleur, visibilité, vecteur source, et `params` (variant `running` | `tatami` | `satin` | `directional` | `autoSatin`). Le variant `autoSatin` (schéma v5) ne stocke **aucun rail** : colonnes et traversées sont dérivées du contour source ; il porte espacement, seuil `Lmax` et longueur `y`, réglages de finition, entrée/sortie et la liste de **guides** (`anchor` en µm, `angle`, `absolute`). Le `satin` porte ses deux rails et, depuis le schéma v2, ses **barreaux** (`rungs` : liste de segments `{ax, ay, bx, by}` en µm), ses réglages de finition (points courts, split, terminaisons), de sous-couche/compensation, et de **fixation/entrée-sortie** (`lockStart`/`lockEnd`, `lockLength`, `lockPasses`, `entryPoint`/`exitPoint`) — tous **optionnels** et rétrocompatibles (clés absentes → valeurs par défaut). Une section issue d'un réseau multi-rail peut aussi porter `topology` : `sectionIndex`, `sectionCount` et les identifiants optionnels `startJunction`/`endJunction`. Ces identifiants sont locaux au réseau défini par `sourceVector` ; l'absence de `topology` désigne un satin isolé/historique. Le `tatami` porte de même ses réglages avancés (Lot 7) : `underlayEdge`, `underlayParallel`, `underlayInset`, `underlaySpacing`, `hiddenUnderpath` et `entryPoint` — optionnels et rétrocompatibles. Le `directional` (remplissage directionnel, voir *Remplissage directionnel*) porte `guides` et `breakLines` (listes de chemins ouverts), `rowSpacing`, `stitchLength`, `edgeWeight`, `inset`, `stagger`, les réglages de sous-couche du tatami, `hiddenUnderpath`, `sectorOverlap`, `handmade`, `handmadeIntensity`, `seed`, `spacingRegularity` (réel [0 ; 1], absent = 0 ; borné à la lecture) et l'objet `densityGradient` (`fromX`, `fromY`, `toX`, `toY`, `spacingFrom`, `spacingTo`, µm entiers ; absent = écart uniforme) — tous optionnels (défauts du modèle) ; son ajout n'a

pas changé `schemaVersion`. Depuis le schéma v3, un objet peut aussi porter ses **retouches manuelles** (Lot 8.1, ADR-014) : `overrides` (**tableau** JSON obligatoire de `{index, pos?, type?, trimAfter}` — `index` désigne une position dans la vue brute de l'objet et doit être **unique** dans le tableau, `pos` un déplacement `{x, y}` en µm, `type` `"stitch"` ou `"jump"` ; chaque entrée doit porter au moins une modification effective — `pos`, `type`, ou `trimAfter: true`), `editedFingerprint` (empreinte FNV-1a 64 bits de la vue brute au moment de la dernière édition, entier **exact**, jamais passé par un `double`) et `editedPointCount` — **obligatoires et explicites** dès que `overrides` n'est pas vide (aucune valeur zéro implicite). Absents, ou `overrides` vide (métadonnées éventuellement présentes mais alors ignorées) → objet `Clean` (comportement actuel inchangé).

- `finishing` : finitions de la séquence (coupes automatiques, points d'arrêt, points courts -- `document::SequenceFinishing`) : `enabled`, `trimThreshold` (µm), `trimBeforeColorChange`, `lockType` (0 aucun, 1 aller-retour, 2 triangle, 3 micro-zigzag), `lockLength` (µm), `lockPasses`, `filterShortStitches`, `minStitchLength` (µm). **Bloc absent** (projet antérieur) → finitions désactivées, séquence identique à avant ; clé absente dans un bloc présent → valeur par défaut. Ajout sans changement de `schemaVersion`. Clés ajoutées par le lot « moteur » (absentes = désactivé) : `splitLongStitches` (bool, défaut faux), `maxStitchLength` (µm, défaut 7000, borné à [1000 ; 12100]), `autoJoin` (bool, défaut faux ; entrée/sortie automatiques, HP-ENG-010).
- Lot « moteur de points », clés additives d'un objet de broderie, écrites seulement hors défaut et bornées à la lecture : `join` (0 hérite du projet, 1 automatique, 2 désactivé) ; dans `params` : tatami `pullCompensation` (µm, [0 ; 3000], HP-ENG-001) et, pour tatami, directionnel, satin et auto-satin, `underlayMode` (0 manuelle, 1 automatique, HP-ENG-002) ; satin `border` (`width` µm [500 ; 20000], `side` 0 centré / 1 intérieur / 2 extérieur, `corner` 0 vifs / 1 arrondis, `pathSet`, `ring` : anneau suivi du vecteur source, HP-STI-004). Aucun changement de `schemaVersion` : un ancien lecteur ignore ces clés.

Fil de nuancier d'un objet (schéma v6)

Un objet de broderie peut référencer un fil : `"thread": {"chart": "<id du nuancier>", "code": "<référence fabricant>"}` (= `thread_palette::ThreadKey`). Le champ est **facultatif** : absent → couleur libre. Le `rgb` de l'objet reste la **seule source de rendu** et de blocs de couleur ; assigner un fil copie la couleur du fil dans `rgb`. Un fil dont le nuancier n'est pas chargé (nuancier importé retiré) n'altère donc ni l'affichage ni l'export : seul son nom n'est plus résolu. **Migration** : un projet v1 à v5 se charge sans `thread` (couleurs libres) ; il est enregistré en v6 à la prochaine sauvegarde (copie `.vN.osp.bak` conservée, comme pour les autres migrations). Un `thread` sans `chart` ou `code` texte est refusé (`InvalidFile`). Les **nuanciers eux-mêmes** ne sont jamais stockés dans le `.osp` (voir *Palettes et fils*).

Versionnement et validation

`schemaVersion` vaut **6** (v5 → v6 : fil de nuancier `thread` des objets de broderie (`{chart, code}`, voir *Palettes et fils*) ; objets texte du lettrage, clé `textObjects` (id, text, fontFamily/fontFile/fontBuiltin/faceIndex, capHeight, letterSpacing, wordSpacing, lineSpacing, kerning, align, justifyWidth, origin, rotation, rgb, fill, maxSatinWidth, density ; µm entiers, rotation en radians) et `textOwner` sur les objets vectoriels, voir `lettering.md` ; v1 → v2 : cadre `canvas` et barreaux satin `rungs` ; v2 → v3 : retouches manuelles `overrides`/`editedFingerprint`/ `editedPointCount` par objet de broderie, Lot 8.1 ; v3 → v4 : intermédiaire ; v4 → v5 : variant `autoSatin`, auto-satin par squelette ; v5 → v6 : fil de nuancier `thread` d'un objet de broderie, HP-THR-004). La lecture est **rétrocompatible** : un fichier v1 ou v2 se charge (cadre 100×100 par défaut si absent, aucun barreau, aucune retouche → état `Clean`). Une version **supérieure** à celle du binaire est refusée proprement (`UnsupportedFormat`) ; un JSON invalide, une topologie satin incohérente (`sectionCount == 0` ou `sectionIndex >= sectionCount`), une valeur hors bornes (index négatif, non entier, ou au-delà de `numeric_limits<size_t>::max()`, coordonnée au-delà d'un `int32`, compteur au-delà d'un `uint32`, type de point inconnu), un `overrides` qui n'est pas un tableau, un `index` en double, une entrée sans modification effective, des métadonnées `editedFingerprint`/`editedPointCount` manquantes pour un tableau non vide, ou une carte de labels incohérente renvoie une erreur utile (`InvalidFile`), jamais une troncature silencieuse.

Sauvegarde atomique

`save_project` écrit d'abord un fichier temporaire `.osp.tmp` puis le **renomme** : un plantage pendant l'écriture ne corrompt jamais le fichier existant.

Aller-retour

Le test d'intégration vérifie que `save` puis `load` restitue exactement l'image (PNG sans perte), les opérations, les labels de segmentation, les tangentes de nœuds et les paramètres des trois types de points.

Limitation : la **sauvegarde automatique**, la **récupération après crash** et les **migrations** entre versions de schéma sont **prévues**, non implémentées. Le cache des points générés n'est pas stocké (il est recalculé au chargement).

Implémentation associée

- `libs/project_io/include/openstitch/project_io/project_io.hpp` — `save_project`, `load_project`, `kSchemaVersion`.
- `libs/project_io/src/json_serialize.cpp` — sérialisation du document.
- `libs/project_io/src/archive.cpp` — lecture/écriture ZIP (minizip-ng encapsulé).
- `libs/project_io/src/project_io.cpp` — orchestration, écriture atomique.
- Tests : `tests/unit/project_io/test_roundtrip.cpp`, `tests/unit/project_io/test_overrides_persistence.cpp` (retouches v3, migration v1/v2, topologie satin optionnelle/rétrocompatible, validation stricte).

Architecture logicielle

Public : développeur, mainteneur.

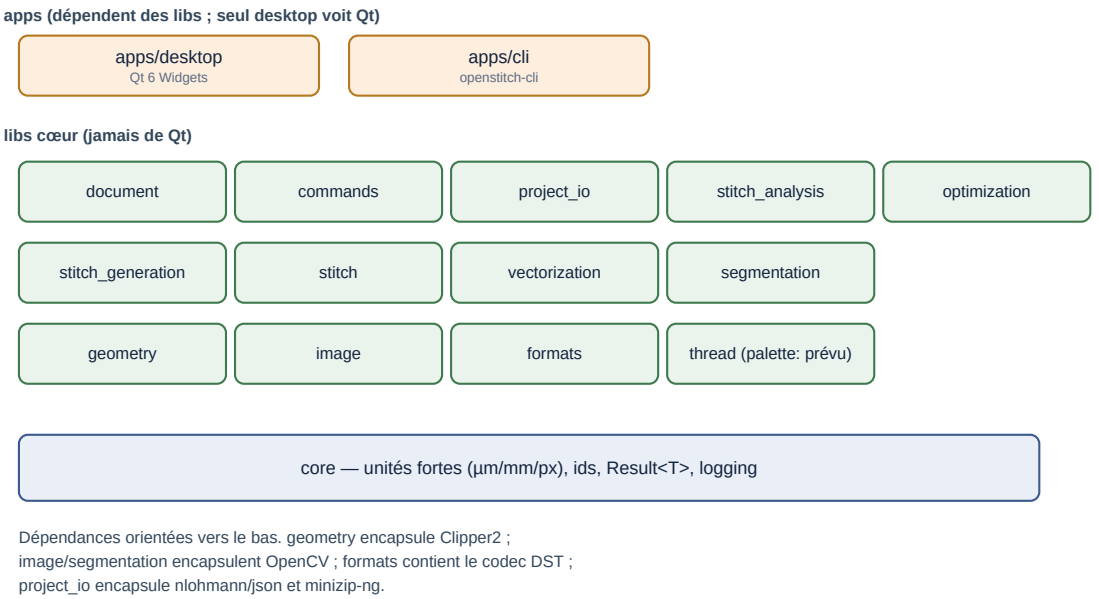


Figure — Les couches : applications, libs cœur, socle `core`.

Principes

1. **Le cœur ne dépend pas de Qt.** Seule `apps/desktop` voit Qt ; tout le reste se compile sans interface (vérifié par la CLI et par un build Linux du cœur).
2. **Dépendances orientées vers le bas** : `apps → libs`, et entre libs un graphe **acyclique**.
3. **Une lib = une cible CMake** `openstitch::<nom>`, avec ses tests.
4. **Encapsulation des tiers** : Clipper2 dans `geometry`, OpenCV dans `image`/`segmentation`/`vectorization`, le codec DST dans `formats`, json et minizip dans `project_io`.

Couches

- **Applications** : `apps/desktop` (Qt), `apps/cli`.
- **Métier / génération** : `document`, `commands`, `project_io`, `stitch_generation`, `stitch`, `stitch_analysis`, `optimization`, `autodigitize`, `auto_satin`, `formats`.
- **Traitement** : `image`, `segmentation`, `vectorization`, `geometry`.
- **Socle** : `core` (unités, ids, `Result`, logging).

Séparation cœur / interface

L'application ne contient **aucune logique métier** : elle câble des actions, affiche, et recalcule l'aperçu. Le chargement d'image, le placement physique, les transformations, la génération de points, l'analyse, l'ordre et les formats vivent tous dans les libs. La règle est vérifiable : la CLI utilise les mêmes modules sans Qt.

Modèle de document et rendu

Le `document::Project` est la source de vérité (voir *Modèle de données*). Les **points** ne sont pas stockés : ils sont recalculés par `generate_sequence`, puis patchés par les **retouches manuelles** de l'objet le cas échéant (Lot 8.1, ADR-014). `stitch_generation::effective_sequence(project)` est le **point d'entrée unique** que tout consommateur de production (aperçu, export, analyse, simulation) doit utiliser — il enchaîne les passes ; `effective_sequence` applique ensuite les **finitions** du projet (`finish_sequence` : coupes automatiques, points d'arrêt, points courts — `project.finishing`). `generate_sequence` reste appelable séparément (implémentation interne, tests, générateurs synthétiques). Une garde CTest structurelle (`tests/check_no_raw_sequence_bypass.cmake`) échoue si un nouveau site de production appelle `generate_sequence` directement sans annotation explicite. Le rendu (côté desktop) est organisé en **deux couches** persistantes — base (image/vecteurs/régions) et points — pour ne reconstruire que le nécessaire, notamment pendant la simulation.

Commandes et undo/redo

Toute mutation du document passe par une **commande** (`ICommand`) empilée dans un `UndoStack`. C'est le patron *Command* : chaque commande sait s'appliquer et se révoquer exactement.

Tâches asynchrones

Limitation : il n'y a **pas** d'infrastructure de tâches asynchrones (pool de threads, annulation, progression) implémentée. Les traitements lourds (segmentation, remplissage) s'exécutent de façon **synchrone** (avec un curseur d'attente). Les fonctions de génération sont **pures** sur des instantanés, donc compatibles avec une exécution en arrière-plan future.

Diagrammes de séquence

- Import d'une image : voir *Introduction* et la figure du chapitre.
- Génération des points : voir *Génération de points*.
- Export DST : voir *Format DST*.

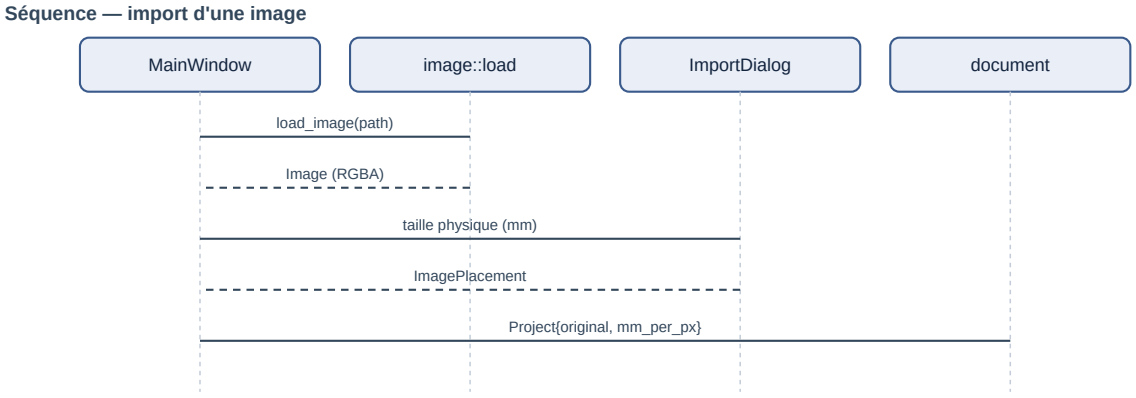


Figure — Import d'une image (du menu au document).

Implémentation associée

- `CMakeLists.txt` (racine), `libs/*/CMakeLists.txt` — cibles et dépendances.
- `apps/desktop/main_window.cpp` — câblage interface.
- `libs/commands/` — `ICommand`, `UndoStack`.
- `docs/phase0/02-architecture.md` — étude d'architecture d'origine.

Référence des modules

Public : développeur. Ce chapitre aide à trouver **où** modifier une fonctionnalité. Chaque module est une cible `openstitch::<nom>` sous `libs/`.

Tableau de synthèse

Module	Responsabilité	Dépendances	Tests
`core`	unités fortes, ids, `Result`, logging	—	`tests/unit/core`
`geometry`	chemins, simplification, booléens/offsets, longueur d'arc	core (+Clipper2)	`tests/unit/geometry`
`image`	chargement, prétraitement non destructif	core (+OpenCV)	`tests/unit/image`
`segmentation`	quantification CIELAB, régions connexes	core, image (+OpenCV)	`tests/unit/segmentation`
`vectorization`	régions → contours vectoriels	core, geometry, segmentation	`tests/unit/vectorization`
`document`	modèle métier (projet, objets)	core, geometry, image, segmentation	via commands/project_io
`stitch`	commandes machine, statistiques	core	`tests/unit/stitch`
`stitch_generation`	running / tatami / satin / directionnel	core, geometry, stitch, document	`tests/unit/stitch`
`stitch_analysis`	règles de validation, fiche de production (`production_sheet`)	core, stitch	`tests/unit/stitch_analysis`
`stitch_render`	rendu réaliste des points (brins de fil, ombrage, tissu, image RGBA ; sans Qt)	core, stitch	`tests/unit/stitch_render`
`optimization`	ordre de couture	core	`tests/unit/optimization`
`autodigitize`	image → objets éditables	vectorization, stitch_generation, stipple	`tests/unit/autodigitize`
`stipple`	image → semis de points (Voronoï pondéré, Lloyd) relié par un tour TSP (2-opt, Or-opt)	core, image	`tests/unit/stipple`
`lettering`	texte → contours de glyphes (FreeType) → lettres satin/tatami/contour éditables ; voir `lettering.md`	core, geometry, document, auto_satin (+FreeType)	`tests/unit/lettering`
`auto_satin`	squelette → satinabilité → auto-satin par traversées orientées (axe, chord, guides)	core, geometry (+OpenCV)	`tests/unit/auto_satin`
`commands`	undo/redo	document	`tests/unit/commands`
`formats`	codec DST, import DXF/SVG, export DXF/SVG diagnostic	core, geometry, stitch (+pugixml)	`tests/unit/formats`
`project_io`	format `.osp`	core, document, image	`tests/unit/project_io`

Points d'extension

- **Ajouter un type de point** : nouvelle alternative de `StitchParams` (`document`), un générateur dans `stitch_generation`, une branche `std::visit` dans `generate.cpp`, un dialogue et une action dans `apps/desktop`.
- **Ajouter un format d'export** : une fonction dans `formats` (encapsulée derrière une interface interne), une sous-commande CLI et une action Fichier.
- **Ajouter une règle d'analyse** : une catégorie dans `analyze.cpp`.
- **Ajouter une opération d'image** : une alternative de `ImageOp` (`image`), un cas dans `apply_op`, une action menu.
- **Ajouter une commande annulable** : une classe `ICommand` dans `commands`.

Où trouver quoi (raccourci)

Je veux modifier…	Fichier
l'échantillonnage des points	`libs/stitch_generation/src/running_stitch.cpp`, `polyline.cpp`
le remplissage/routage tatami	`libs/stitch_generation/src/tatami.cpp`
le remplissage directionnel (champ, lignes de courant, secteurs, fait main)	`libs/stitch_generation/src/directional_fill.cpp`
l'outil de guides de direction (canevas)	`apps/desktop/main_window_directional.cpp`
la colonne satin	`libs/stitch_generation/src/satin.cpp`
le satin par squelette et ses sections	`libs/auto_satin/src/`
la mesure de couverture du satin automatique	`libs/auto_satin/src/skeleton_satin.cpp` (`measure_coverage`)
le choix de type auto (tatami/satin), avec ou sans segmentation	`libs/autodigitize/src/autodigitize.cpp` (`auto_digitize/auto_digitize_vectors`)
l'édition (type, orientation, filtres, calques)	`apps/desktop/main_window.cpp`
l'encodage DST	`libs/formats/src/dst.cpp`
l'import SVG (contourne image/segmentation)	`libs/formats/src/svg_import.cpp`
le format projet	`libs/project_io/src/`
les menus	`apps/desktop/main_window.cpp`
les unités	`libs/core/include/openstitch/core/units.hpp`
les fichiers récents (HP-FILE-003)	`apps/desktop/recent_files.hpp/.cpp`
la sauvegarde automatique et la récupération après plantage (HP-FILE-004)	`apps/desktop/autosave.hpp/.cpp`

Implémentation associée

Voir les chapitres thématiques pour le détail de chaque module.

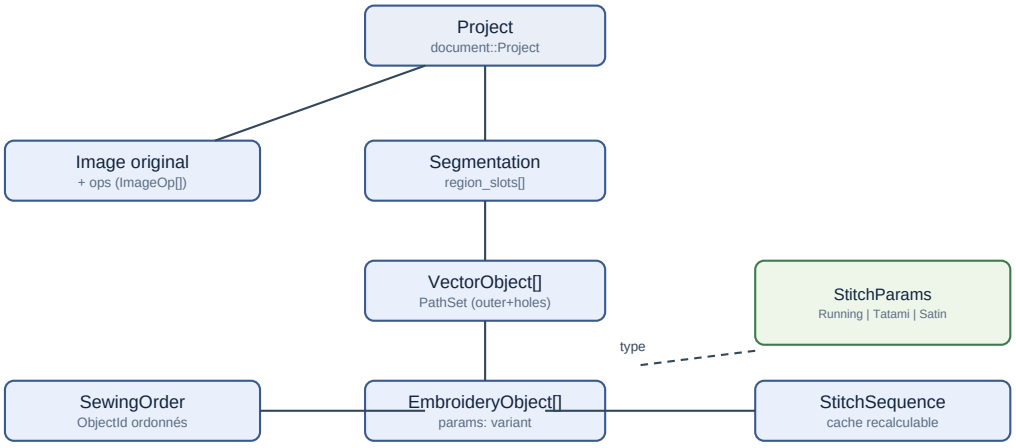
Préférences et données applicatives persistantes (`apps/desktop`)

Deux emplacements distincts, à ne pas confondre :

- `QSettings` (`ai_preferences.hpp`, `recent_files.hpp`, géométrie/état des panneaux dans `main_window.cpp`) : petites préférences clé-valeur, namespace par préfixe (`ui/*`, `ai/*`, `recent/*`).
- `QStandardPaths::AppDataLocation` : dossier de données applicatives persistantes, distinct de `QSettings`. Premier usage dans `apps/desktop/autosave.hpp/.cpp` (HP-FILE-004, sous-dossier `autosave/`) — le seul précédent dans `apps/desktop` avant cette entrée, `ai_segmentation_dialog.cpp`, n'utilise que `QStandardPaths::TempLocation` (fichiers de travail éphémères, pas de persistance voulue). Un périmètre futur qui a besoin d'un dossier de données applicatives persistant (pas seulement une préférence clé-valeur) suit ce même étendard plutôt que d'en introduire un troisième.

Modèle de données

Public : développeur.



La géométrie éditable (VectorObject) et les points générés (StitchSequence) sont séparés : régénérer ne détruit pas la source.

Figure — Relations principales du document.

Unités et coordonnées

Type	Fichier	Rôle	Unité
`Micrometers`	`core/units.hpp`	coordonnée interne (int32)	µm
`Millimeters`	`core/units.hpp`	affichage / paramètres UI	mm (double)
`Pixels`	`core/units.hpp`	domaine image	px (double)
`Angle`	`core/units.hpp`	angles	radians
`Vec2um`	`core/units.hpp`	point 2D	µm

Invariant : **aucune coordonnée n'est un `double` nu** ; l'unité interne est le micromètre entier (déterminisme, pas de dérive), Y vers le haut. La conversion vers/depuis les pixels exige une résolution explicite (mm/px).

Identifiants

`ObjectId`, `RegionId`, `ColorId`, `ThreadId` sont des types **forts distincts** (`Id<Tag>`), générés par un `IdGenerator<T>` monotone (jamais réutilisés). Fichier : `core/ids.hpp`.

Document

`document::Project` (fichier `document/project.hpp`) contient :

Champ	Type	Rôle
original	image::Image	image source intacte
mm_per_px	Millimeters	résolution de travail
canvas	Canvas	cadre de broderie (largeur/hauteur, défaut 100×100 mm, marge de sécurité → zone brodable)
ops	vector<ImageOp>	pile de prétraitements
segmentation	optional<Segmentation>	carte de régions
vector_objects	vector<VectorObject>	géométrie éditable
embroidery_objects	vector<EmbroideryObject>	objets de broderie (ordre = ordre de couture)
object_ids	IdGenerator<ObjectId>	compteur partagé

Types géométriques

- `geometry::PathNode` : `pos` (Vec2um), `type` (Corner/Smooth), `tan_in`/`tan_out` optionnelles.
- `geometry::Path` : nœuds + `closed`.
- `geometry::PathSet` : `outer` + `holes`.
- `geometry::Polyline` : polyligne aplatie (travail par longueur d'arc).

Points et commandes

- `stitch::CommandType` : `Stitch`, `Jump`, `Trim`, `ColorChange`, `Stop`, `End`.
- `stitch::StitchCommand` : `pos` (Vec2um absolue), `type`, `source` (ObjectId).
- `stitch::StitchSequence` : `vector<StitchCommand>`.
- `stitch::StitchStats` : compteurs, longueur de fil, bornes.

Sérialisation

Voir *Format de projet* : tout ce qui précède se sérialise en JSON (+ PNG + binaire), sauf les points générés (recalculés).

Éléments non présents dans le modèle

Limitation : `profil machine`, `cadre` paramétrable riche (formes, marges), `fil`/`palette`, `avertissement` persistant et `commande undo` sérialisée ne sont pas (ou peu) présents. Le `Canvas` porte la **taille du cadre**, réglable et persistée, mais reste un simple rectangle.

Implémentation associée

- `libs/core/include/openstitch/core/{units,ids,error}.hpp`
- `libs/document/include/openstitch/document/{project,vector_object,embroidery_object,canvas}.hpp`
- `libs/geometry/include/openstitch/geometry/{path,polyline}.hpp`
- `libs/stitch/include/openstitch/stitch/sequence.hpp`

Algorithmes

Public : développeur. Ce chapitre décrit les algorithmes **réellement** utilisés, avec un pseudo-code reflétant l'implémentation (et non une copie du C++).

Quantification (segmentation)

- **Objectif** : réduire l'image à N couleurs perceptuellement cohérentes.
- **Principe** : conversion RGB→CIELAB, k-means déterministe sur un échantillon, affectation au centre le plus proche.
- **Complexité** : ~O(pixels · N) pour l'affectation.
- **Cas limites** : image entièrement transparente (refus), N hors bornes.

```
segment(image, N):
 opaques = pixels d'alpha >= 128 en Lab
 centres = kmeans(échantillon(opaques), N, graine fixe)
 pour chaque pixel opaque: label = argmin_c ||Lab(pixel) - centres[c]||
 régions = composantes_connexes(label)   # 4-connexité, par couleur
 absorber les régions < taille_min dans la voisine majoritaire
```

Extraction et simplification de contours

- **Contours** : Suzuki-Abe (`cv::findContours`, RETR_CCOMP).
- **Simplification** : Douglas-Peucker, tolérance **adaptative** (≤ périmètre/16) pour préserver les petits trous.
- **Nettoyage** : union Clipper2 (règle pair-impair) → hiérarchie extérieur/trous.

Aplatissement de Bézier + longueur d'arc

```
flatten(path, tol):
 pour chaque segment à tangentes: subdiviser (De Casteljau) tant que
     distance(contrôles, corde) > tol
 fusionner les points identiques

resample_run(points, cible):
 L = longueur(points)
 n = max(1, ceil(L / cible))         # parts égales, <= cible, sans résidu
 renvoyer points aux abscisses i*L/n, i=0..n
```

Running stitch

```
run_stitch(path, cfg):
 poly = flatten(path, cfg.tol)
 coins = sommets d'angle de rotation > seuil (+ extrémités si ouvert)
 pour chaque tronçon entre coins: concat(resample_run(tronçon, cible))
```

- **Complexité** : O(points).
- **Cas limites** : chemin vide/trop court → avertissement, pas de crash.

Tatami (scanline + routage)

```
fill_tatami(région, params):
 tourner la géométrie de -angle
 pour chaque rangée y: segments = intervalles intérieurs (pair-impair)
 graphe: segments de rangées voisines qui se chevauchent en x sont adjacents
 parcours glouton: suivre une arête (couture) ; sinon sauter (déplacement)
 retourner dans le repère d'origine
```

- **Garantie** : aucune couture ne traverse un trou (adjacence ⇒ bande intérieure).
- **Complexité** : ~O(segments²) au pire pour l'adjacence locale par rangée (voisines uniquement), acceptable aux tailles usuelles.

Satin

L'auto-satin (squelette, traversées orientées, pas adaptatif) est décrit dans `satin-squelette.md` ; le pseudo-code ci-dessous ne concerne que le satin manuel à deux rails.

```
fill_satin(railA, railB, cfg):
 pour u de 0 à 1 par pas dérivé de la densité:
   L = point(railA, u*longueurA) ; R = point(railB, u*longueurB)
   appliquer compensation (écarter L et R le long de LR)
   émettre zigzag alterné L,R
 sous-couche centrale = point droit sur (L+R)/2
```

Encodage / décodage DST

```
encode(seq):
 pour chaque commande: delta = round(pos/100) - round(prev/100)
     subdiviser si |delta| > 121 (sauts)
     émettre 3 octets (ternaire équilibré + bits de type)
 en-tête calculé après le corps
```

- **Propriété** : déterministe, erreur ≤ 50 µm, sans dérive.

Ordre de couture

```
optimize_order(items, stratégie):
 items libres (non verrouillés) réarrangés:
   ByColor: regrouper par couleur (ordre d'apparition)
   ByProximity: plus proche voisin
   ColorThenProximity: groupes de couleur puis proximité
   LayeredColorThenProximity: groupes de couleur triés par aire max
     décroissante ; dans chaque groupe, grandes zones (>= 25 % de la plus
     grande) par aire décroissante, puis petites par proximité
 réinsérer dans les emplacements libres, verrous à leur place
coût = distance de déplacement + 50000 * changements de couleur
```

Implémentation associée

Voir chaque chapitre thématique et `docs/stitch-engine-research.md` pour les références (Douglas-Peucker, scanline fill, k-means, ré-échantillonnage par longueur d'arc).

Compilation et développement

Public : développeur, mainteneur.

Chaîne de build

- **CMake** (≥ 3.27) avec `CMakePresets.json`.
- **MSVC** (Visual Studio 2022+) sous Windows ; générateur par défaut du preset.
- **vcpkg** en mode manifeste (`vcpkg.json`, baseline verrouillée).
- **Qt 6.8** hors vcpkg (variable `QT_ROOT`).

Presets

Preset	Cible
`msvc` (configure)	Windows, application complète
`msvc-debug` / `msvc-release` (build/test)	Debug / Release
`linux-core`	Cœur + CLI **sans Qt** (garde-fou de portabilité)

Commandes

```
$env:VCPKG_ROOT = "C:\dev\vcpkg"
$env:QT_ROOT    = "C:\Qt\6.8.3\msvc2022_64"

cmake --preset msvc                 # configuration (premier run long : vcpkg)
cmake --build --preset msvc-debug   # compilation Debug
ctest  --preset msvc-debug          # tests
cmake --build --preset msvc-release # Release
```

Options CMake

- `OPENSTITCH_BUILD_DESKTOP` (ON) — construit l'application Qt.
- `OPENSTITCH_BUILD_TESTS` (ON) — construit les tests (Catch2).
- Cible d'interface `openstitch::warnings` : `/W4 /permissive- /utf-8` (MSVC), `-Wall -Wextra -Wconversion` (GCC/Clang), liée en PRIVATE par chaque cible.

Déploiement des DLL (Windows)

L'application lie dynamiquement Qt et OpenCV. `windeployqt` copie les DLL Qt à côté de l'exécutable (étape POST_BUILD) ; vcpkg copie ses propres DLL.

Linux (cœur uniquement)

Le preset `linux-core` compile les libs cœur et la CLI **sans Qt** — c'est un garde-fou pour éviter toute dépendance Windows/Qt involontaire dans le cœur. L'interface graphique n'est pas construite sous Linux à ce stade.

Erreurs fréquentes

- « Qt6 introuvable » → vérifier `QT_ROOT`.
- Erreur de toolchain vcpkg → vérifier `VCPKG_ROOT` et le bootstrap.
- Compilation OpenCV très longue → normal au premier configure (cache ensuite).

- Édition de lien de l'exe échouée (`LNK1168`) → l'application est en cours d'exécution ; la fermer.

Intégration continue

Un workflow GitHub Actions (`.github/workflows/ci.yml`) est présent : build MSVC Debug/Release + tests + artefacts, build Linux du cœur, et vérification du formatage ; sur `main` elle publie aussi la release « latest » (voir Installation).

Implémentation associée

- `CMakeLists.txt`, `CMakePresets.json`, `vcpkg.json`.
- `.github/workflows/ci.yml`, `.clang-format`.
- `docs/source/installation.md` — installation et compilation Windows (`docs/build-windows.md` y renvoie).

Guide du développeur

Public : développeur contributeur.

Conventions

- **C++23**, RAII, `std::optional`/`std::variant`/`std::span`/`std::filesystem`, `std::expected` (via `Result<T>`).
- **Types forts** pour les unités : jamais de coordonnée en `double` nu.
- **Const-correctness**, warnings élevés (`/W4 /permissive-`).
- **Formatage** : `.clang-format` (base LLVM, 4 espaces, 100 colonnes).
- **En-tête de licence** : `// SPDX-License-Identifier: Apache-2.0`.

Pièges connus (spécifiques à ce dépôt)

Avertissement : Quelques écueils rencontrés lors du développement :

- les **noms de tests Catch2** doivent être en **ASCII** (les accents ne survivent pas à l'encodage console Windows lors de la découverte des tests) ;
- l'en-tête spdlog est `stdout_color_sinks.h` ;
- un membre nommé `slots` est interdit (macro Qt) — utiliser un autre nom ;
- déclarer les classes Qt en **forward declaration au scope global** ;
- pas de `M_PI` sous MSVC → utiliser `std::numbers::pi` (`<numbers>`) ;
- cibles vcpkg exactes : `Clipper2::Clipper2`, `MINIZIP::minizip-ng`.

Ajouter une fonctionnalité (recettes)

- **Nouveau type de point** : voir *Référence des modules → Points d'extension*.
- **Nouvelle opération d'image** : une alternative de `ImageOp`, un cas dans `apply_op`, un test, une action menu.
- **Nouvelle commande annulable** : dériver `ICommand` (`apply`/`revert`/`name`), l'exécuter via `UndoStack::execute`. Toute mutation du document doit passer par une commande.

Undo/redo

`UndoStack` conserve deux piles (`undo`/`redo`). Une nouvelle commande vide la pile `redo`. Les commandes de segmentation mémorisent les **indices** de pixels modifiés pour une annulation exacte et légère.

Gestion des unités

Toujours convertir aux frontières : pixels→µm à l'import (résolution explicite), µm→mm pour l'affichage, µm→0,1 mm à l'export DST. Les algorithmes internes peuvent utiliser des `double`, mais les entrées/sorties des modules sont en types forts.

Multithreading

Il n'y a pas encore d'infrastructure asynchrone. Écrire les nouveaux traitements comme des **fonctions pures** sur des instantanés, pour rester compatible avec un futur passage en tâche de fond.

Implémentation associée

- `.clang-format`, `CMakeLists.txt` (cible `openstitch::warnings`).
- `libs/commands/` — `ICommand`, `UndoStack`, commandes du projet.
- `docs/phase0/` — décisions d'architecture (ADR).

Tests

Public : développeur, mainteneur.

Structure

- **Framework (cœur, cli)** : Catch2 v3 (via vcpkg), intégré à CTest.
- **Framework (UI desktop)** : Qt6::Test (QTest/QSignalSpy), intégré à CTest, toujours exécuté **headless** (`QT_QPA_PLATFORM=offscreen`) — voir Tests UI (Qt, desktop).
- **Tests unitaires** : `tests/unit/<lib>/test_*.cpp` (un exécutable par lib).
- **Test d'intégration** : `tests/integration/test_pipeline.cpp` (chaîne complète).
- **Golden (SVG de diagnostic)** : `tests/golden/stitch-generation/`.
- **Garde structurelle CI** : `tests/check_no_raw_sequence_bypass.cmake` (invoquée via `add_test`, cf. Lot 8.1) — échoue si un site de production hors `libs/stitch_generation/` appelle `generate_sequence()` directement sans annotation `raw-sequence-ok:`, contournant les retouches manuelles.
- Total au dernier passage vérifié : **398 tests CTest**, 100 % réussis.

Encadré de traçabilité (dernier passage vérifié)

Un simple « 398/398 » devient vite périmé ; voici le contexte exact du dernier passage vérifié manuellement. Régénérez ces valeurs avant toute publication.

Élément	Valeur
Commit (état du code testé)	`e4ceaa2` — formes dessinees a la main : forme libre + correctif Maj=cercle
Compilateur	MSVC toolset 14.50 (Visual Studio 2026)
CMake	4.4.0-rc3
Configurations	Debug **et** Release
Résultat CTest	398 / 398 réussis
Tests désactivés	0
Fichiers de tests d'intégration	1 (`tests/integration/test_pipeline.cpp`)
Suites Qt (UI desktop)	5 exécutables CTest (`tests/unit/desktop/`), 54 fonctions de test QTest
Tests sur machine réelle	0
Couverture de code	non mesurée

Note : chaque `TEST_CASE` Catch2 est enregistré séparément par `catch_discover_tests`. Côté Qt, chaque exécutable QTest est un seul test CTest (`add_test`) qui contient plusieurs fonctions. Le nombre d'**assertions** est supérieur au nombre de tests CTest (398, garde structurelle CI incluse).

Exécution

```
ctest --preset msvc-debug                # tous les tests
ctest --preset msvc-debug -R tatami      # filtre par nom
build\msvc\tests\unit\stitch\Debug\test_stitch.exe "[nom]"   # un exécutable
```

Correspondance modules ↔ tests

Module	Tests
core	`tests/unit/core/test_units.cpp`, `test_ids.cpp`
geometry	`test_simplify.cpp`, `test_clean.cpp`, `test_offset.cpp`, `test_polyline.cpp`, `test_primitives.cpp`
image	`test_image_load.cpp`, `test_ops.cpp`
segmentation	`test_segmentation.cpp`
vectorization	`test_vectorize.cpp`
stitch / stitch_generation	`test_stats.cpp`, `test_running_stitch.cpp`, `test_generate.cpp`, `test_tatami.cpp`, `test_satin.cpp`, `test_satin_pairing_metrics.cpp`, `test_routing.cpp`, `test_overrides.cpp`
autodigitize	`test_autodigitize.cpp`
auto_satin	`tests/unit/auto_satin/test_pipeline.cpp`, `test_columns.cpp`
stitch_analysis	`test_analyze.cpp`
optimization	`test_order.cpp`
commands	`test_undo_stack.cpp`
formats	`test_dst.cpp`, `test_svg.cpp`
project_io	`test_roundtrip.cpp`, `test_overrides_persistence.cpp`
desktop (UI, Qt)	`tests/unit/desktop/test_canvas_view.cpp`, `test_node_handle.cpp`, `test_document_panel.cpp`, `test_properties_panel.cpp`, `test_main_window.cpp`
intégration	`tests/integration/test_pipeline.cpp`

Tests UI (Qt, desktop)

Fondation ajoutée pour vérifier réellement l'interface (et pas seulement le cœur) : la sélection au canevas, le déplacement d'un nœud, la synchronisation canevas/liste/inspecteur, sans jamais comparer de pixels ni dépendre d'un écran.

Architecture de testabilité

`apps/desktop/CMakeLists.txt` sépare les sources en une bibliothèque statique `openstitch_desktop_widgets` (tout sauf `main.cpp`), liée à la fois par l'exécutable `openstitch` et par les tests. Pour rendre `MainWindow` elle-même instanciable en test (deuxième tranche, voir plus bas), trois seams minimaux s'y ajoutent, sans changer son comportement en production : `QSettings()` par défaut (au lieu d'une organisation/application codées en dur) lit l'organisation/application déjà posées sur `QCoreApplication` dans `main.cpp` — un test peut donc les rediriger vers un fichier `.ini` temporaire avant de construire `MainWindow`, sans jamais toucher au registre Windows réel ; `MainWindow::applyLoadedProject(Project)` (**privée**, appelée depuis `loadProject()` et réutilisée telle quelle) applique un projet déjà construit sans passer par `QFileDialog` — accessible aux tests via `friend class MainWindowTest` (déclaré dans `main_window.hpp`, `MainWindowTest` vit dans `openstitch::desktop`), pas via une méthode publique ajoutée uniquement pour les tests (revue corrective, voir plus bas) ; `QAction::setObjectName(...)` sur une poignée d'actions (`action_undo`, `action_redo`, `action_deleteRegion`, `action_createStitch`) pour les retrouver par `findChild` sans dépendre du texte traduit. Aucun membre n'est rendu public au-delà de l'API de production existante. Chaque suite est un exécutable QTest (`QTEST_MAIN`), exécuté par CTest avec `QT_QPA_PLATFORM=offscreen` (propriété `ENVIRONMENT`) et le dossier `bin` de Qt ajouté au `PATH` (propriété `ENVIRONMENT_MODIFICATION` — Qt n'est pas déployé à côté de chaque exécutable de test comme il l'est pour `openstitch` via `windeployqt`). Les objets métier non enregistrés comme type Qt (`ObjectId`, `RegionId`, `StitchParams`) sont observés par connexion directe (lambda) plutôt que par `QSignalSpy`, pour ne pas avoir à leur ajouter `Q_DECLARE_METATYPE` (le cœur ne dépend jamais de Qt) ; `QSignalSpy` reste utilisé pour les signaux à types Qt natifs (`QPointF`, `QRectF`).

Matrice couverte

Composant	Scénario testé	Résultat métier vérifié
`CanvasView`	zoom avant/arrière, `fitCanvas`	échelle (`pixelsPerMm`) change dans le bon sens, `viewChanged` émis
`CanvasView`	clic hors mode recadrage	`canvasClickedMm` émis avec la position scène exacte
`CanvasView`	mode recadrage actif	clic simple **n'émet plus** `canvasClickedMm`
`CanvasView`	glisser-rectangle en mode recadrage	`cropSelectedMm` émis avec le rectangle réellement dessiné (régression, voir plus bas)
`NodeHandleItem`	glisser une poignée	callbacks `onMoved`/`onReleased` reçoivent la position scène exacte, l'item a bougé
`NodeHandleItem`	clic sans glisser	`onReleased` appelé une fois, position inchangée
`DocumentPanel`	`refresh(project)`	listes Objets/Régions peuplées, id stocké (`Qt::UserRole`) correct
`DocumentPanel`	sélectionner une ligne Objets	`embroiderySelected` émis avec le bon `ObjectId`
`DocumentPanel`	`syncSelection` (retour canevas -> panneau)	**ne réémet aucun signal** (évite la boucle de sélection)
`PropertiesPanel`	`showEmbroidery`	formulaire peuplé aux bonnes valeurs, **aucun** `paramsEdited` pendant la construction
`PropertiesPanel`	éditer un champ du formulaire	`paramsEdited` émis avec le champ modifié à jour, les autres champs préservés
`PropertiesPanel`	changer de sélection (`showInfo` après `showEmbroidery`)	l'ancien formulaire est bien détruit (pas de contrôles fantômes)
`MainWindow`	clic canevas sur un objet vectoriel	`DocumentPanel` (liste Objets) et `PropertiesPanel` (spin boxes) se synchronisent sur le point de contour rattaché ; `action_createStitch` s'active
`MainWindow`	sélection région (liste) puis sélection objet (canevas)	`action_deleteRegion`/`action_createStitch` s'activent et se désactivent en opposition, selon la priorité de `resolveSelectedEmbroidery`/`onCanvasClicked`
`MainWindow`	suppression d'une région (`action_deleteRegion`, sans dialogue) puis undo/redo (`action_undo`/`action_redo`)	la liste Régions de `DocumentPanel` reflète la suppression **et** sa restauration (pas seulement `undoStack_`/le modèle) ; les actions annuler/rétablir s'activent en conséquence
`MainWindow`	sélection d'une broderie dans un projet, puis chargement d'un second projet (`applyLoadedProject`) réutilisant le même `ObjectId` de broderie (régression, voir plus bas)	ni `DocumentPanel` (sélection liste Objets) ni `PropertiesPanel` (formulaire) ne montrent la broderie du projet précédent tant qu'aucune sélection explicite n'a été faite dans le nouveau

`MoveNodeCommand` (la commande undo/redo réellement appliquée quand une poignée est relâchée) est déjà couverte côté cœur, sans Qt, par `tests/unit/commands/test_undo_stack.cpp` (cas « AddVectorObject et MoveNode ») — non dupliqué ici.

Revue corrective de la deuxième tranche (2026-08-01)

Deux défauts trouvés indépendamment sur les commits `d49e660`/`37ffc27`, corrigés sans toucher au reste du périmètre de la deuxième tranche :

1. **Seam de test dans l'API publique de production.** `MainWindow::loadProjectForTests(Project)` avait été ajouté **public** uniquement pour les tests, ce qui contredit la règle du projet (aucune API de production élargie pour le seul bénéfice des tests). Corrigé : `applyLoadedProject` (déjà privée) reste privée ; `MainWindowTest` est forward-déclarée dans `main_window.hpp` et déclarée `friend` de `MainWindow`, et vit dans `openstitch::desktop` (comme `MainWindow` elle-même) pour que le friend s'applique. Aucune donnée interne supplémentaire n'est exposée ; `loadProjectForTests` a été supprimée.
2. **Fuite de sélection de broderie entre deux projets.** `applyLoadedProject` réinitialisait `selectedRegion_` et `selectedObject_` mais oubliait `selectedEmbroidery_` — défaut déjà présent avant la deuxième tranche (queue de l'ancien `loadProject()`, seulement déplacée telle quelle dans `applyLoadedProject`). Si le nouveau projet réutilise le même `ObjectId` qu'une broderie sélectionnée dans l'ancien (cas réaliste : les projets de test allouent des ID déterministes à partir d'un compteur remis à zéro), `resolveSelectedEmbroidery()` la retrouvait silencieusement dans le nouveau document — l'inspecteur, la sélection dans `DocumentPanel` et la barre contextuelle affichaient alors une broderie jamais choisie explicitement par l'utilisateur. Corrigé par un `selectedEmbroidery_.reset()` ajouté à côté des deux autres réinitialisations. Test de régression : `embroiderySelectionDoesNotLeakAcrossProjectLoadWithReusedId` — vérifié en retirant temporairement la ligne corrigée : le test échoue bien sans le correctif (`objectsList(*docPanel)->currentRow()` reste à `0` et le formulaire de l'inspecteur reste peuplé au lieu de revenir à « Aucune sélection »).

Audit ciblé du reste de l'état de session réinitialisé par `applyLoadedProject` : `undoStack_.clear()`, `sequence_.reset()` et `sequenceImported_ = false` étaient déjà corrects. Les filtres d'affichage temporaires (`hiddenColors_`, `showType_`, `minAreaMm2_`) et `contextSig_` persistent délibérément d'un chargement à l'autre (préférences d'affichage de session, pas des données de document) ; aucun défaut démontré à leur sujet, donc non modifiés.

Bug découvert et corrigé par ce lot

Le test du glisser-rectangle a révélé que `CanvasView` utilisait `fromScenePoint`/`toScenePoint` du signal `QGraphicsView::rubberBandChanged`, qui accusent un retard d'une étape sur `viewportRect` (constaté en forçant un glisser à seulement deux évènements de déplacement espacés) : le rectangle final capturait la position du glisser **précédent**, pas la position réelle au relâchement. Invisible en usage réel (le pointeur génère des évènements quasi continus, l'écart est sub-pixel), mais faux pour un glisser rapide à peu d'évènements (trackpad, remote desktop). Correctif : reconvertir `viewportRect` lui-même (`mapToScene(viewportRect).boundingRect()`), toujours à jour. Test de régression : `cropModeRubberBandEmitsCropSelectedMm`.

Obstacles de testabilité identifiés

- **`MainWindow` reste un god-object** : constructeur de ~250 lignes construisant menus/docks/barres d'outils/panneaux en bloc, logique de sélection/filtre/simulation imbriquée dans des lambdas privées (`objectPassesFilter`, `embroideryCentroid`…). La deuxième tranche (voir ci-dessous) n'a **pas** décomposé la classe — elle a percé trois seams minimaux (QSettings injectable, `applyLoadedProject` accessible par `friend class MainWindowTest`, `objectName` sur quelques actions) qui suffisent à instancier `MainWindow` et à observer son état sans dialogue modal. Toute action qui ouvre une boîte de dialogue (`QFileDialog`, `QMessageBox`, `QInputDialog`, `QColorDialog`, `QMenu::exec` — `openImage`, `saveProject`, `segmentImage`, `onCanvasContextMenu`…) reste hors de portée d'un test QTest offscreen automatisé.
- **Sélection rectangulaire d'objets métier absente** : le `RubberBandDrag` actuel de `CanvasView` ne sert **que** le mode recadrage (`cropSelectedMm` → recadre l'image). Il n'existe aucun mécanisme qui sélectionne les objets vectoriels/de broderie contenus dans un rectangle glissé — fonctionnalité demandée par l'utilisateur mais pas encore codée (hors périmètre de la deuxième tranche, sur consigne explicite). Rien n'a donc pu être testé « avec succès » sur ce point ; c'est une fonctionnalité **manquante**, pas un test manquant.
- **Déplacement global d'un objet absent** : seul le déplacement d'un nœud individuel existe (`NodeHandleItem` → `MoveNodeCommand`, testé). Il n'y a pas de commande ni d'interaction pour translater un objet entier (tous ses nœuds à la fois) ; pas de `MoveObjectCommand` dans `libs/commands` (hors périmètre de la deuxième tranche, sur consigne explicite).

Deuxième tranche (`MainWindow` — actions/menus, synchronisation, undo/redo)

Les trois points de la roadmap ci-dessous marqués **fait** ont été couverts sans instancier `MainWindow` via un scénario nécessitant un dialogue modal : mutation via `action_deleteRegion` (`RemoveRegionCommand`, aucune boîte de dialogue) sur un document minimal construit directement en mémoire (`applyLoadedProject`, pas de fichier `.osp` ni d'image chargée depuis le disque).

Roadmap (scénarios non automatisables aujourd'hui)

But visé par l'utilisateur, dans l'ordre de valeur/risque :

1. **Sélection rectangulaire d'objets** (prérequis produit avant tout test, toujours non codé) : ajouter un mode canevas dédié (distinct du recadrage) qui émet les `ObjectId`/`RegionId` sous le rectangle ; puis test QTest reprenant le même schéma que `cropModeRubberBandEmitsCropSelectedMm`.
2. **Déplacement global d'un objet** (toujours non codé) : `MoveObjectCommand` (cœur, Catch2, translation de tous les nœuds + undo/redo — même schéma que `MoveNodeCommand`), puis interaction canevas (glisser l'intérieur d'un objet sélectionné) testée en QTest comme `NodeHandleItem`.
3. **fait** — **Synchronisation sélection canevas ↔ liste ↔ inspecteur** : clic canevas (`CanvasView::canvasClickedMm` appelé directement, sans simulation souris pixel-exacte — déjà couverte par `test_canvas_view.cpp`) → `MainWindow::onCanvasClicked` → `resolveSelectedEmbroidery` (logique de priorité broderie > vecteur, extraite de la triple duplication `updateContextToolbar`/`updateInspector`/`syncDocumentSelection`) → `DocumentPanel::syncSelection` + `PropertiesPanel::showEmbroidery`.
4. **fait** — **Undo/redo restaure modèle et UI** : le modèle était déjà couvert (Catch2, `test_undo_stack.cpp`) ; on vérifie maintenant que la liste `DocumentPanel` (pas seulement `project_`/`undoStack_`) se **rafraîchit** après un undo puis un redo déclenchés depuis `action_undo`/`action_redo`.
5. **fait** — **Menus/actions selon l'état** (`updateActions`) : `action_createStitch` (dépend de la sélection d'un objet vectoriel) et `action_deleteRegion`/région (dépend de la sélection d'une région **et** d'une segmentation présente) togglent correctement, y compris quand une sélection en chasse une autre (priorité canevas > liste, cf. `onCanvasClicked`).

Types de vérifications notables

- **DST aller-retour** sur tous les deltas de −121 à +121, déterminisme octet à octet.
- **Tatami** : invariant « aucune couture ne traverse le trou » d'un anneau, peu de déplacements ; et « aucune couture hors région » sur une forme concave en **L** échantillonnée à cinq angles (filet anti-débordement).
- **Auto-numérisation** : une bande fine devient un **tatami** par défaut (satin naïf désactivé), et un satin uniquement si `use_naive_satin` est activé.
- **Auto-satin géométrique** (`build_satin_columns`) : formes simples produisent une colonne, milieu des barreaux **intérieur à la région**, Y/T → plusieurs sections portant un identifiant de jonction commun, anneau fin → quatre sections formant un cycle, cercle plein/forme large refusés, déterminisme des rails et de la topologie ; SVG dans `tests/golden/auto-satin/`.
- **Extension des bouts ouverts** (`extend_tip`, mission « auto-satin béton ») : un bout arrondi (`capsule`) atteint le bord réel (longueur bout-à-bout 45 mm ±1 mm, contre ~38,8 mm avant) ; un bout carré (`rectangle`) aussi (40 mm contre ~34,9 mm) ; les jonctions d'un réseau Y restent géométriquement identiques bascule activée/désactivée (seuls les bouts ouverts s'allongent) ; aucun barreau dégénéré après extension ; déterminisme ; le bascule `extend_open_ends=false` restaure l'ancien comportement.
- **Ancrage des jonctions** (`trim_and_anchor_junction_end`, mission « auto-satin béton ») : sur un Y symétrique (résolution où les 3 branches survivent), les 6 extrémités de rail touchant la jonction se regroupent en **exactement 3 sommets partagés par exactement 2 rails chacun** ; sur un T (topologie asymétrique, 2 encoches réelles pour 3 branches) **aucune collision** (jamais 3+ rails au même sommet) et aucun barreau dégénéré ; aucune dérive de largeur résiduelle (> 1,2× la médiane) sur aucune section ; déterminisme ; le bascule `anchor_junction_ends=false` restaure la dérive (régression reproduite pour prouver que le bascule agit réellement).
- **Formes dessinées à la main** (`rectangle_path`/`ellipse_path`/`polygon_path`/`freeform_path`, `tests/unit/geometry/test_primitives.cpp`) : rectangle d'aire exacte quels que soient les coins fournis (ordre indifférent) ; ellipse à 4 nœuds lisses, aire aplatie proche de π·rx·ry (< 1 % d'écart), cercle quand les côtés sont égaux ; polygone reliant les sommets dans l'ordre, chemin vide si moins de 3 sommets ; forme libre (`freeform_path`) simplifie un tracé bruité (20 points colinéaires par bord d'un carré) en un contour exploitable d'aire correcte, chemin vide si moins de 3 points bruts OU si le tracé simplifié retombe sous 3 sommets (points colinéaires) ; cas dégénérés (largeur/rayon nul) sans crash ; déterminisme. Côté UI (`tests/unit/desktop/test_main_window.cpp`) : les quatre outils créent un `VectorObject` annulable (undo/redo exact) ; un cadre trop petit ne crée rien ; le tracé d'un polygone accumule les sommets (aperçu élastique tenu à jour), se ferme au double-clic (≥ 3 sommets, sinon abandon propre) sans ajouter de sommet fantôme pour la seconde pression du double-clic, et s'annule proprement en changeant d'outil ; le tracé d'une forme libre accumule les points captés pendant le glisser (aperçu tenu à jour), se ferme au relâchement du bouton (< 3 points → message clair, rien créé) et s'annule proprement en changeant d'outil ; **Maj = cercle sur l'ellipse est désormais directement testable** (modificateurs passés en paramètre du signal `boxDrawnMm`, capturés sur l'évènement Qt réel plutôt que relus depuis un état clavier global non fiable en QTest offscreen — correctif de revue, l'ancien test avait dû être écarté à la création de la fonctionnalité).
- **Persistance des réseaux satin** : aller-retour exact de l'index/nombre de sections et des jonctions, ancien satin sans `topology` encore lisible, index hors réseau refusé proprement.
- **Appariement rail gauche/rail droit (audit rails, 2026-08-01)** : fixtures géométriques dédiées (ruban droit, S, coude 90°, largeur variable, cas combiné) mesurant croisements, monotonie sur les deux rails, angle fil/normale locale, continuité angulaire, régularité de densité, déterminisme — avec comparaison chiffrée à l'ancien algorithme (réplique isolée, test uniquement). Voir `docs/source/satin.md` § *Correction de l'appariement*.
- **Satin par barreaux** (`fill_satin_columns`) : espacement **médian régulier** (colonne droite), barreaux **traversés exactement**, rails de longueurs différentes, repli sur `fill_satin` si < 2 barreaux, déterminisme.

- **Édition des guides satin** : QTest headless sur le clic de sélection, l'ajout au plus grand intervalle, la suppression avec plancher de deux guides, le glisser d'une extrémité projetée sur son rail, le verrouillage visible d'un guide terminal de jonction et les parcours undo/redo ; tests cœur sur l'espacement minimal, les ordres de guides croissant/décroissant et la détection géométrique d'une jonction malgré des barreaux inversés. La collecte multi-section est testée sur l'isolation par objet source, l'ordre déterministe et le refus d'un réseau incomplet ; la commande coordonnée vérifie undo/redo et l'absence de mutation partielle avec cible obsolète ou dupliquée. Un parcours QTest clique réellement le guide structurel, ajoute un guide interne aux deux branches dans leur intervalle adjacent (fixture asymétrique 0/2/10 mm), puis vérifie l'undo/redo unique sans temporisation ni comparaison de pixels. Deux tests cœur couvrent aussi l'ordre inverse, le guide non structurel et l'intervalle trop court.
- **Groupes de guides liés — édition atomique** (`satin_linked_guides`, `move_satin_guide_group`, `RemoveSatinGuidesCommand`) : tests cœur sur l'énumération triée et déterministe d'un groupe, l'isolation par réseau (même `link_id` numérique réutilisé dans un `source_vector` différent, aucune fuite), le refus d'un `link_id` dupliqué dans une même section (donnée corrompue), le rejet total hors de l'intervalle admissible (marge de densité), le rejet d'une section glissée hors du groupe, le rejet d'un guide qui a perdu l'adjacence à sa jonction (topologie modifiée entretemps), le rejet d'un réseau incomplet, de membres liés à des jonctions différentes et d'une progression croisée entre rails, ainsi que le calcul exact du delta normalisé partagé sur deux sections aux géométries de rail **différentes** (longueurs 10 mm/20 mm) — vérifie que le résultat de chaque section vient bien de sa propre géométrie (2 mm vs 6 mm) et non d'une coordonnée recopiée. Côté commandes : suppression de groupe tout-ou-rien avec undo/redo exact, rejet d'une cible obsolète/dupliquée/vide, réinsertion correcte de plusieurs guides dans un même objet (ordre descendant à la suppression puis ascendant à la restauration), et un test de revue corrective vérifiant qu'un document modifié entre `apply()` et `revert()` (section ayant perdu sa géométrie satin) interdit une restauration partielle — la première section reste elle aussi non restaurée même si sa propre cible était encore valide. Côté UI : Maj+glisser une extrémité d'un guide lié déplace les deux sections d'un coup (même résultat exact dans les deux, géométrie identique) en une seule commande annulable, supprimer un guide lié supprime le groupe entier en une seule commande, et un glisser SANS Maj sur le même guide reste local (seule la section glissée bouge) — non-régression du comportement d'édition d'angle pré-existant. Un Maj+clic sans mouvement ne crée aucune commande undo. Aucune temporisation, aucune comparaison de pixels.
- **Satin — finitions (Lot 3)** : points courts (inset modifie le rail intérieur, remove réduit les pénétrations), split (staggered ≠ ligne centrale, jitter déterministe), terminaisons (taper réduit la largeur au bout sans l'annuler) ; aller-retour `.osp` des modes.
- **Satin — sous-couches + compensation (Lot 4)** : center/edge/zigzag = 4 passes distinctes ordonnées ; pull élargit **un seul côté** (asymétrique) ; push étend le bout ; le générateur tague les sous-couches `Underlay` et le satin `TopStitch` ; aller-retour `.osp`.
- **Satin — entrée/sortie + lock (Lot 5)** : `lock_stitches` (`None` → vide, les autres progressent dans l'axe de couture et restent bornés) ; les locks sont émis en passe `Lock` en **exactement deux groupes** (début/fin) ; le point d'entrée fait démarrer la couture à la bonne extrémité ; aller-retour `.osp` des champs Lot 5 (lock, entrée/sortie).
- **Satin — routage multi-colonnes (Lot 6)** : `route_columns` (liste vide → plan vide, une colonne → départ sans saut, réordonnancement minimisant le déplacement, orientation par l'extrémité proche, liaison courte → trajet caché / longue → saut) ; une jonction commune admissible prime sur une extrémité plus proche sans relation topologique, tandis qu'une jonction trop distante est ignorée ; le test de génération vérifie que `SatinParams.topology` atteint le plan. Un groupe adjacent n'émet qu'un saut initial (le reste cousu en passe `Travel`), un groupe éloigné **ou sans jonction commune** conserve ses sauts.
- **Routage — trajet caché non validé (correctif, revue « auto-satin béton » (suite))** : avant correction, seule la distance (`underpath_max`, 8 mm) décidait d'un trajet caché, sans jamais vérifier `step.junction` — deux colonnes proches (5 mm) mais **sans aucune jonction commune** auraient été cousues en trajet caché à travers un espace non garanti couvert. Quatre tests verrouillent le correctif (seuil à deux paliers, `underpath_max` si `step.junction` a une valeur,

`underpath_max_without_junction` sinon) : liaison à 5 mm sans jonction → **saut** (avant : trajet caché, défaut) ; quasi-contact à 1 mm sans jonction → trajet caché toujours toléré (seuil strict non pénalisant) ; la même liaison à 5 mm **avec** jonction commune validée reste un trajet caché jusqu'à 8 mm (jonction non pénalisée par le nouveau seuil strict) ; au niveau génération, deux sections à 5 mm sans jonction ne produisent plus aucun point `Travel` (0 trajet caché, 2 sauts). Aucun test préexistant modifié : tous leurs écarts étaient déjà soit quasi-contact, soit jonction-justifiés, soit très supérieurs à 8 mm.

- **Audit satin adversarial (2026-08-02)** : 11 nouveaux tests verrouillant les 8 défauts trouvés par 4 audits indépendants du pipeline complet.
- `tests/unit/auto_satin/test_pipeline.cpp` : jonction d'une `cross` correctement détectée à degré 4 réel dans le graphe élagué (comptage d'arêtes incidentes, pas seulement le type déclaré du nœud) ; réseau `y` conserve ses 3 branches, aucune arête `from == to` (auparavant : une branche entière disparaissait) ; nœud isolé d'un `circle` (squelette réduit à un point) apparaît dans `removed_branches` au lieu de disparaître silencieusement.
- `tests/unit/auto_satin/test_columns.cpp` : nouvelle fixture `"h"` (`shapes.cpp` — deux barres verticales reliées par un pont horizontal, 2 jonctions dont une arête Jonction-Jonction) — le pont est bien converti en colonne (5 colonnes au total), aucune collision aux deux jonctions, aucun barreau dégénéré ; jonction à 4 branches (`cross`, désormais correctement détectée) — aucune collision de sommet reflex entre les 4 colonnes.
- `tests/unit/stitch/test_satin.cpp` : `default_rungs` produit au moins deux barreaux sur une colonne simple (vide sur des rails dégénérés) et débloque un réglage ignoré par `fill_satin` (`cap_end = Tapered` sans effet sans barreaux, effectif avec) ; rétraction `push_start` demandée à –50 mm reste bornée près de son origine (pas d'auto-croisement) ; lock sur une colonne de 0,3 mm avec `lock_length` par défaut (0,8 mm) reste dans la largeur réelle, quel que soit le type.
- `tests/unit/stitch/test_routing.cpp` : un écart brut de 900 µm (sous le seuil sans jonction) devient un saut une fois le `push_end` (rétraction de 700 µm) réellement appliqué pris en compte (écart réel 1,6 mm, au-dessus du seuil) — plus aucun trajet caché fondé sur un point non réellement cousu. Aucun test préexistant modifié dans cette revue (seules des fixtures/tests nouveaux) — voir `docs/source/satin.md` pour le détail de chaque défaut et son correctif.
- **Tatami avancé (Lot 7)** : sous-couche de contour rentrée dans la forme ; sous-couche parallèle espacée ; underpath caché convertit au moins un saut en trajet cousu **sans jamais** traverser le trou (invariant préservé) et ne l'augmente jamais ; déterminisme ; le point d'entrée oriente le démarrage ; le générateur tague la sous-couche `Underlay` et le remplissage `TopStitch` ; aller-retour `.osp` des champs Lot 7.
- **Retouches manuelles — cœur pur (Lot 8.0, corrigé)** : `fingerprint` (FNV-1a 64 bits) stable pour une vue brute identique, sensible à la position, au type de commande, à la passe et à l'ordre pris séparément ; `raw_slice` reconstruit correctement un objet sur une séquence contiguë et sur une séquence entrelacée (synthétique, et via un routage satin réel à trajet caché) ; `apply_manual_overrides` valide chaque champ d'un override indépendamment sur les entrées `TopStitch` : `moved_to` exige une cible `Stitch`, `forced_type` et `trim_after` acceptent une cible `Stitch` **ou** `Jump` (Stitch↔Jump bidirectionnel — correctif : `is_overridable_entry` n'acceptait auparavant qu'une cible `Stitch`, rendant Jump→Stitch impossible malgré le cadrage et le rapport initial de Lot 8.0 ; aucun test ne couvrait cette direction), un champ invalide pour sa cible est ignoré sans rejeter les autres champs du même override ; laisse un objet `Dirty` strictement inchangé (par empreinte **et**, indépendamment, par compteur de points), isole chaque objet retouché des autres, résout déterministement les doublons d'overrides (dernière entrée du vecteur, en bloc), les overrides vides et les index hors bornes ; déterminisme du résultat (séquence patchée et liste d'objets `Dirty` triée) vérifié sur deux exécutions indépendantes. Aucune commande undo/redo, aucune persistance `.osp`, aucune UI dans ce sous-lot (cf. `docs/lot8-manual-editing-design.md`).
- **Retouches manuelles — commandes, persistance, point d'entrée unique (Lot 8.1)** : les 4 commandes (`MoveStitchPointCommand`, `SetStitchPointTypeCommand`, `SetStitchTrimCommand`, `DiscardOverridesCommand`) valident `ObjectId`/`base_index`/état avant toute mutation, refusent un objet déjà `Dirty` sans y toucher, restaurent exactement l'entrée précédente au `revert` (y compris la

transition `Clean → ManuallyEdited` et son annulation), et partagent une seule entrée d'`overrides` quand deux commandes ciblent le même `base_index` (chaque `revert` ne restaure que son propre champ). `SetStitchTrimCommand(false)` sur un index sans override existant est un no-op exact (aucune entrée vide créée, aucune transition `Clean → ManuallyEdited` fantôme) ; si elle efface le dernier champ effectif d'une entrée préexistante (`trim_after` était son seul champ posé), l'entrée est retirée plutôt que laissée vide, et `revert` la réinsère exactement (`end_stitch_edit` gère ce cas). Persistance `.osp` v3 : `overrides`/`editedFingerprint`/`editedPointCount` survivent à un aller-retour exact, y compris un `editedFingerprint` **au-delà de 2■³** (aucune perte de bits via `double` — nlohmann conserve un entier positif en `number_unsigned`) ; un fichier v1/v2 sans ces champs charge un objet `Clean` ; un index négatif/non entier ou au-delà de `numeric_limits<size_t>::max()`, une coordonnée au-delà d'un `int32`, un compteur au-delà d'un `uint32`, un type de point inconnu ou une entrée sans `index` renvoient une erreur `InvalidFile`, jamais une troncature silencieuse ou un plantage. **Revue corrective (2026-08-01)** : `overrides` doit être un tableau JSON (objet/scalaire refusé) ; un tableau non vide exige la présence **explicite** d'`editedFingerprint`/`editedPointCount` (plus de valeur zéro implicite sur un document malformé) ; un tableau vide accompagné de ces mêmes clés est normalisé en `Clean` sans erreur (métadonnées orphelines ignorées, non significatives quand `overrides` est vide) ; un `index` en double dans le tableau est refusé (`InvalidFile`) au lieu d'être fusionné champ à champ (le cadrage initial l'autorisait, décision revue — aucun producteur réel n'écrit jamais deux entrées pour le même index, cf. `docs/lot8-manual-editing-design.md` §4) ; une entrée sans aucune modification effective (ni `pos`, ni `type`, ni `trimAfter: true`) est également refusée. `effective_sequence` (enchaîne `generate_sequence` puis `apply_manual_overrides`) est vérifiée identique au brut pour un objet `Clean`, patchée pour un objet `ManuallyEdited`, déterministe sur deux appels indépendants, et propage sans plantage une erreur de génération (projet sans objet visible). Une garde CTest structurelle (`check_no_raw_sequence_bypass`) fait échouer la suite si un nouveau site de production appelle `generate_sequence()` sans passer par `effective_sequence()`. Toujours aucune UI dans ce sous-lot.

- **Retouches manuelles — mode d'édition des points, UI desktop (Lot 8.2)** : côté cœur, `is_movable_point` (prédicat d'éligibilité exporté, plus de duplication dans l'UI), `edit_view`/`classify_all_edit_states` (état dérivé

- vue brute/empreinte/compteur pour un objet) et `refresh_context` (un seul `generate_sequence` interne pour produire à la fois la séquence effective — contenu **identique** à `effective_sequence`, mêmes deux passes —, les états par objet et la vue d'édition d'une cible optionnelle) sont vérifiés cohérents entre eux et propagent sans plantage une erreur de génération. Côté UI (QTest, offscreen, événements souris réels sur poignées visibles à deux niveaux de zoom, `QSignalSpy`) : activation/désactivation du mode selon la sélection et l'état `Dirty` (aucun second signal `toggled` émis par une sortie forcée, garde-fou vérifié via `QSignalSpy`) ; glisser une poignée exécute **une seule** `MoveStitchPointCommand` (undo ramène `canUndo()` à faux) dont l'annulation/le rétablissement restaurent exactement la même entrée d'override ; un clic sans déplacement ne crée aucune commande ; un changement de document pendant la fenêtre où la commande différée (`QTimer::singleShot(0)`, nécessaire pour ne pas détruire sa propre poignée) est en file d'attente fait abandonner cette commande sans muter le nouveau projet (régression du compteur `documentGeneration_`) ; abandonner les retouches d'un objet `Dirty` (confirmation auto-acceptée) est annulable et restaure exactement l'état antérieur ; un objet dont le nombre de points déplaçables dépasse le seuil d'affichage refuse proprement l'activation, avec message explicite en barre de statut, plutôt que de laisser le mode actif sans aucune poignée.

- **Tatami — correctif contacts sommet** : `segment_stays_in_region` détecte un connecteur **parfaitement vertical** traversant un trou en losange de part en part en touchant exactement ses deux sommets (défaut qu'une version antérieure laissait passer, faute de sonder l'intérieur pour un petit écart en x) ; même vérification avec **deux trous** (le couloir entre eux reste cousable) et sur le **coin rentrant** d'une forme en L (contact sommet vers l'encoche rejeté) ; non-régression : un suivi de bord colinéaire reste cousu ; `fill_tatami` sur un trou en losange ne produit aucun point cousu à l'intérieur.

- **Tatami — politique sûre de `tatami_underlay`** : un retrait de contour (`underlay_inset`) impossible (pièce trop petite) ne produit **aucune** sous-couche de contour (jamais de repli silencieux sur le bord brut) ; un retrait **explicitement nul** longe bien le bord brut (intention distincte).

- **Running** : espacement par longueur d'arc (cercle), coins préservés, courbes Bézier suivies, résultats déterministes.
- **Undo/redo** : « undo total = état initial », restauration exacte des labels ; aller-retour exact des commandes d'objet (`SetFillAngle`, `SetStitchType`, `SetStitchParams`, `ConvertFillsToTatami`, `SetCanvas`).
- **Projet** : aller-retour complet (image, ops, **cadre**, segmentation, tangentes, params).

Ajouter un test

Ajoutez un `TEST_CASE` (nom **ASCII**) dans le `test_*.cpp` du module, ou un nouveau fichier référencé dans le `CMakeLists.txt` du dossier de tests. Les golden SVG ne sont **jamais** réécrits par les tests : ils se régénèrent explicitement via `openstitch-cli stitchdebug`.

Pour l'UI desktop : ajoutez une fonction de test dans une suite existante de `tests/unit/desktop/`, ou un nouveau fichier + `openstitch_add_qt_test(...)` dans son `CMakeLists.txt`. Chaque test doit vérifier un résultat métier observable (modèle, sélection, `enabled`/`checked`, valeur d'un signal) — pas seulement l'absence de crash — et rester déterministe (pas de `sleep`, pas de comparaison de pixels, pas de dépendance à la résolution/au thème).

Implémentation associée

- `tests/` (arborescence complète), `tests/unit/*/CMakeLists.txt`.
- `tests/unit/desktop/` — tests Qt (QTest) de la couche UI.
- `apps/desktop/CMakeLists.txt` — bibliothèque `openstitch_desktop_widgets`.
- `CMakeLists.txt` (racine) — `enable_testing`, `Catch`.

Guide de contribution

Public : contributeur.

Prérequis

- Les outils de compilation : voir Installation (voie rapide `scripts■uild.ps1`, qui demande confirmation avant d'installer quoi que ce soit, ou voie manuelle).
- **Python 3.10+** et `pip` pour régénérer la documentation. Le script `docs\scripts■uild-docs.ps1` crée lui-même un environnement virtuel local `docs\.venv` (ignoré par Git, voir `.gitignore`), y installe `docs/requirements.txt` et produit le PDF ; ne l'installez pas globalement.
- `clang-format` pour la vérification de formatage de la CI (`git ls-files '*.cpp' '*.hpp' | xargs clang-format --dry-run --Werror`).

Flux

1. Cloner le dépôt et créer une **branche** dédiée (le développement se fait sur `main` en local ; ne pas committer directement sans branche pour une fonctionnalité).
2. Compiler et exécuter les tests (`ctest --preset msvc-debug`).
3. Ajouter/mettre à jour les **tests** couvrant la modification.
4. Formater (`clang-format`) et vérifier les warnings.
5. Rédiger un message de commit clair (voir ci-dessous).

Messages de commit

Le dépôt utilise des messages descriptifs en français, terminés par une ligne de co-auteur. Exemple observé :

```
Corrige le debordement des remplissages hors des regions

<description détaillée>

Co-Authored-By: ...
```

Style C++

- Respecter `.clang-format` et les warnings élevés.
- Types forts pour les unités ; pas de logique métier dans les widgets Qt.
- Toute mutation du document via une **commande** (undo/redo).
- En-tête SPDX Apache-2.0 en tête de fichier.

Ajouter une dépendance

- La déclarer dans `vcpkg.json` (ou documenter l'installation si hors vcpkg).
- **Vérifier la licence** : compatible Apache-2.0, permissive de préférence. **Aucun code GPL** ne peut être copié dans ce dépôt (voir *Licences*).
- L'encapsuler derrière une interface interne (ne pas exposer ses types).
- La documenter dans `THIRD_PARTY_LICENSES.md`.

Signaler un bug / proposer une fonctionnalité

Sur le dépôt GitHub du projet (https://github.com/ElZacho200a/OpenStitch), onglet *Issues* s'il est activé. Dans tous les cas, documentez le problème avec un cas minimal reproductible (le projet `.osp` si possible) et, pour un contributeur, un test qui échoue.

Documentation

Toute fonctionnalité visible doit être reflétée dans `docs/source/` puis le PDF régénéré (`.\docs\scripts■uild-docs.ps1`, voir *docs/README.md*) ; une option ou un code de sortie de `openstitch-cli` va dans Ligne de commande. Un lien vers un autre chapitre est un lien Markdown relatif vers son fichier `.md` : le script le transforme en ancre cliquable dans le PDF et échoue sur une cible inconnue. Les affirmations doivent rester **vérifiables** dans le code (section « Implémentation associée »).

Implémentation associée

- `.clang-format`, `THIRD_PARTY_LICENSES.md`, `LICENSE`.
- `docs/README.md` — régénération de la documentation.

Dépannage

Public : utilisateur (première partie), développeur (seconde partie). Pour chaque problème : symptôme, cause probable, diagnostic, solution. Les messages d'erreur de l'application et de la ligne de commande disent en général quoi vérifier ; la ligne de commande préfixe les siens par `openstitch-cli <sous-commande> :` et ajoute une ligne « Piste : » (voir Ligne de commande).

Partie 1 — Utilisation

Installation et lancement

Symptôme	Cause probable	Solution
Windows affiche « a protégé votre PC » / « Éditeur inconnu » (SmartScreen)	binaires non signés	**Informations complémentaires** puis **Exécuter quand même** (voir Installation)
« `Qt6Core.dll` / `opencv_*.dll` / `VCRUNTIME140.dll` est introuvable »	ZIP non extrait en entier, `openstitch.exe` copié sans ses voisins, ou Visual C++ Redistributable absent	Extraire tout le ZIP (ou utiliser l'installeur) ; installer le Redistributable Visual C++ 2015-2022 x64
L'application ne s'ouvre pas, aucun message	erreur au démarrage visible seulement en console	Lancer `openstitch.exe` depuis un terminal : les messages sortent sur la sortie d'erreur
Disposition cassée, panneaux introuvables	préférences de disposition corrompues ou fenêtre hors écran	Affichage ■ Panneaux ■ Réinitialiser la disposition ; sinon supprimer `HKCU\Software\OpenStitch`

Image, régions et formes

Symptôme	Cause probable	Diagnostic	Solution
Image non chargée	format/fichier corrompu	`openstitch-cli info fichier`	Réexporter l'image en PNG
Segmentation « incorrecte »	trop/peu de couleurs, petites régions	carte des régions	Ajuster N et la taille min, fusionner/supprimer
Motif hors cadre	taille physique trop grande	règles / cadre rouge	Réduire la taille à l'import, ou déplacer ; ajuster le cadre (Affichage ■ Taille du cadre)
Le fond est brodé, ou une partie du motif manque	mauvais réglage de « Ignorer la plus grande région »	dialogue de numérisation (couleur et part de l'image affichées)	Relancer avec la case cochée (fond) ou décochée (image plein cadre)
Remplissage qui déborde de la forme	contour vectorisé trop approximatif	zoom sur la zone ; SVG `openstitch-cli osp2svg --outlines`	Re-numériser avec un niveau de détail plus élevé ; ajuster le retrait du tatami

Satin et auto-satin

Symptôme	Cause probable	Solution
« Satin impossible » : l'auto-satin refuse la forme	forme compacte (disque, blob), trop large, ou sans axe exploitable : le moteur par squelette n'a pas pu construire de colonne	Accepter le **tatami** proposé ; ou simplifier/épaissir la forme ; le message donne la raison (voir *Satin par squelette*)
Colonne satin trop large (avertissement)	largeur au-delà de la recommandation	Préférer un tatami, ou fractionner les traversées longues
Le satin est irrégulier ou orienté bizarrement	guides ou squelette de la forme	Poser un guide d'orientation (inspecteur ■ « Placer un guide sur le canevas… ») ; sinon tatami. L'auto-satin est **expérimental et non validé sur machine**
Satins automatiques qui débordent dans un ancien projet	colonnes satin générées par une ancienne version	Broderie ■ Convertir les satins auto en tatami

Points, couleurs et export

Symptôme	Cause probable	Diagnostic	Solution
Aucun point généré	objet sans géométrie, tous invisibles	barre d'état / Statistiques	Vérifier qu'un objet vectoriel source existe et est visible
Points trop longs	longueur de point élevée	Analyse (F5)	Réduire la longueur de point dans l'inspecteur
Export DST refusé	séquence vide	message	Générer des points d'abord
Fichier DST « vide »	aucun objet visible	`openstitch-cli stats fichier.dst`	Vérifier les objets, leur visibilité et l'ordre
La machine ne lit pas le DST	extension/format attendu par la machine différent, support (clé USB, FAT32), nom trop long, ou en-tête/convention non gérée par ce modèle	`openstitch-cli stats` relit le fichier ; `dst2svg` l'affiche	Essayer un nom de fichier court sur une clé vide formatée comme l'attend la machine ; ouvrir le DST dans un autre logiciel de transfert pour confirmer qu'il est valide ; **signaler** le modèle de machine : aucune validation sur machine réelle n'a encore été faite (voir Limitations)
La machine ne **coupe pas** le fil entre les objets ou en fin de motif	certaines machines ignorent ou suppriment les sauts de coupe de déplacement nul	voir *Format DST* (coupes à sauts non nuls, coupe finale)	Régler le seuil de coupe (Broderie ■ Options de génération…) ; **tester sur la machine** : le seuil de trois sauts peut devoir être augmenté selon le modèle
Couleurs incorrectes à la broderie	l'application ne gère que des couleurs RGB par objet, pas de références de fil	—	Choisir les fils sur la machine dans l'ordre des changements de couleur affiché (voir Palettes et fils)
Projet impossible à ouvrir	`.osp` corrompu / version inconnue	message d'erreur	Vérifier le fichier ; version de schéma supportée = `1` ; essayer la sauvegarde automatique (voir ci-dessous)
Crash	entrée inattendue	—	Aucune entrée utilisateur ne devrait faire planter ; signaler avec un cas minimal et le projet `.osp`

Perte de travail et sauvegarde automatique

Où est l'autosave ? Dans `%APPDATA%\OpenStitch\OpenStitch Studio\autosave\` (fichiers `<empreinte>.osp` et `.json`), toutes les 2 minutes quand le document est modifié. Au démarrage suivant un arrêt anormal, un dialogue **Récupération après un arrêt anormal** propose **Récupérer** ou **Ignorer** (l'entrée traitée n'est plus reproposée). Une fermeture normale supprime la sauvegarde, et « Ignorer » la supprime aussi : enregistrez régulièrement avec **Ctrl+S**.

Segmentation par IA

Symptôme	Cause probable	Solution
« La segmentation par IA n'est pas activée »	case désactivée dans les préférences	Édition ■ Préférences — Intelligence artificielle…
L'IA ne démarre pas / « le worker n'a pas pu démarrer »	worker Python introuvable ou incomplet : chemins vides, distribution WSL absente, `torch`/`sam2` non installés dans le venv, dossier de modèles sans checkpoint	Dans les préférences IA, remplir tous les chemins puis **Tester la configuration** (le journal affiche l'erreur exacte). Prérequis détaillés dans le Guide utilisateur ; la fonction n'est **pas** livrée avec les binaires de release
Le test dit « Configuration incomplète »	un chemin du formulaire est vide	Renseigner Python du venv, script du worker et dossier des modèles
Très lent	exécution sur CPU, grosse résolution	Choisir un modèle plus petit, un GPU (CUDA) ou baisser la résolution maximale d'analyse

Journaux

L'application et la ligne de commande écrivent leurs messages (spdlog) sur la **sortie d'erreur** de la console : **aucun fichier de log persistant**. Pour les voir, lancez `openstitch.exe` ou `openstitch-cli.exe` depuis un terminal (`openstitch.exe 2> journal.txt` pour les garder). Le worker IA a son propre niveau de journalisation dans les préférences IA.

Partie 2 — Compilation (développeurs)

Symptôme	Cause probable	Solution
« Qt6 introuvable »	`QT_ROOT` absent/incorrect	Pointer sur `...\msvc2022_64`, reconfigurer
Erreur toolchain vcpkg	`VCPKG_ROOT` absent, bootstrap manquant	Définir la variable, relancer `bootstrap-vcpkg.bat`
`build.ps1` refuse d'installer (« non confirmée »)	console non interactive	Relancer avec `-Yes`, ou avec `-SkipBootstrap` après installation manuelle
DLL manquante au lancement d'un build local	déploiement Qt/OpenCV incomplet	Lancer depuis le dossier `Debug`/`Release` (windeployqt a copié les DLL)
`LNK1168` à l'édition de lien	l'application tourne encore	Fermer `openstitch.exe` puis recompiler
OpenCV très longue à compiler	premier configure vcpkg	Normal ; les runs suivants utilisent le cache
Génération du PDF de documentation impossible	Python/venv absent	Python 3.10+ ; `docs\.venv` est créé par `docs\scripts\build-docs.ps1` (voir Guide de contribution)
`ctest` échoue sur les tests du bureau	affichage absent	Les tests Qt tournent en **headless** : `QT_QPA_PLATFORM=offscreen`

Implémentation associée

- `libs/core/src/log.cpp` — initialisation du logger (sortie d'erreur).
- `apps/desktop/autosave.cpp`, `apps/desktop/main_window.cpp` — sauvegarde automatique et récupération.
- `apps/desktop/ai_preferences_dialog.cpp` — préférences IA et test de configuration.
- `libs/*/src/*` — les erreurs renvoyées via `Result<T>` avec un message montrable.

Limitations et état des fonctionnalités

Public : tous. Ce chapitre distingue explicitement l'état de chaque fonctionnalité, vérifié dans le code.

Tableau des fonctionnalités

Fonctionnalité	État	Interface	Module	Tests	Limitations
Import PNG/JPEG/BMP/TIFF	Implémenté	Fichier	image	oui	—
Prétraitement non destructif	Implémenté	Image	image	oui	pas de resize
Segmentation CIELAB + régions	Implémenté	Segmentation	segmentation	oui	4-connexité
Édition de régions	Implémenté	Segmentation	segmentation	oui	—
Vectorisation	Implémenté	Segmentation	vectorization	oui	—
Formes dessinées à la main (rectangle/ellipse/polygone/forme libre)	Présent - testé	palette d'outils	geometry/desktop	QTest	pas de capture d'écran manuelle possible dans cet environnement de dev
Édition de nœuds	Partiel	canevas	document	—	déplacement seul
Édition d'objets broderie	Implémenté	inspecteur/clic droit	commands	oui	changer le **type** et tous **les paramètres** après création ; orientation à la poignée
Interface (thème, panneaux, workflow)	Implémenté	desktop	desktop	(vue)	thème clair/sombre + densité, inspecteur, panneau Document, workflow, état d'accueil, persistance UI (QSettings)
Point double/double/triple	Implémenté	Broderie	stitch_generation	oui	—
Tatami	Présent - testé - SVG	Broderie	stitch_generation	oui	scanline + routage **sous-couches (contour + parallèle), underpath caché, entrée** (Lot 7) ; orientation éditable
Satin (génération par barreaux)	Présent - testé - SVG	Broderie / inspecteur	stitch_generation	oui	`fill_satin_colu`ams ; sections, espacement **perpendiculaire**, guides sélectionnables/ajustables/supprimables/déplaçables avec undo/redo, points courts / split, terminaisons (Lot 3), **sous-couches (contour/zigzag/zig-zag) + compensation pull/push** (Lot 4, passes distinctes), **lock + entrée/sortie** (Lot 5), **routage multi-colonnes** (Lot 6)
Modèle de passes	Présent - testé	(générateur)	stitch	oui	`StitchPass` ; par commande (Underlay/TopSatin/TravelLock/Manual) ; affichage/toggle par passe dans l'UI à venir
Auto-satin par squelette et traversées orientées	Présent - testé (craquoi) - SVG - **non essayé sur machine**	Satin auto, Convertir en satin	auto_satin, stitch_generation	oui	remplace l'ancien moteur rails/barreaux (supprimé) ; formes à branches (V/T/croix/H/E/peigne/étoile/arrneau) ; 0.98 de couverture estimée sur le corpus, guides d'orientation éditables, retus motteils ; traversées contournent (disque, pilliers) ; limite connue : bras très larges (`deep_channel` = 0.0/0) ; détails dans `satin-squelet.md`. Ancien libellé : formes simples, Y/T multi-sections, croix à 4 branches (jonction degré 4 correctement dirac/ée) et **étoile/H** avec en 4 sections **bouts ouverts dessus jusqu'au bord réel** et **bouts de jonction ancrés sur les sommets reflex du contour** (rotation + auto-satin bilton v) ; pont Jonction-Jonction (ex. un "H") converti en colonne ; cercle plein/forme large refusés
Classification auto des régions	**Expérimental**	Segmentation	autodigitize	oui	bandes/ flons → moteur topologique par défaut (`sink_auto_satin`) ; refus → tatami ; moteur naïf désactivé
Filtres d'affichage / calques	Implémenté	Affichage	desktop	(vue)	affichage seulement (couleur, type, taille ; image/régions/vecteurs/brodérie)
Ordre de couture	Implémenté	dock	optimization	oui	2-opt entre objets non implémenté (il existe dans le routage interne d'un objet)
Analyse	Implémenté	Analyse	stitch_analysis	oui	pas de carte de densité
Simulation	Implémenté	barre	desktop	—	pas de réglage de vitesse
Export/Import DST	Implémenté	Fichier/CLI	formats	oui	limites du format
Export SVG diagnostic	Implémenté	CLI	formats	oui	—
Format projet `.stsp`	Implémenté	Fichier	project_io	oui	suivi « modifié » + garde partagée (fermeture et **Nouveau projet**, Ctrl+N) ; autosave récupérable (Ctrl+S réécrit le fichier, Ctrl+Maj+S = Enregistrer sous), écriture atomique ; pas de fichiers récents ni d'autosave
Cadre de brodérie	Implémenté	Affichage	document	oui	taille réglable et persistée ; rectangle simple (pas de profils/formes)
Palette de fils	Partiellement implémenté	—	thread_palette	oui	fil assigné par objet, panneau Fils, filtre « n'utiliser que » limiter à la fils », import CSV/JSON (HP-THR-001/002/004, partiel 005/011) ; nuanciers de marques à importer par l'utilisateur, intégrés = démo (HP-THR-002) ; aucun format machine (DST) ne porte les noms de fils
Édition manuelle des points	Partiel (Lot 8.2)	canevas	desktop/commands	QTest	déplacement point par point + undo/redo ; Stitch/Jump/Trim UI restent à faire
Remplissages courbes/radiaux/concentriques	Non implémenté	—	—	—	prévus
Profils machine/cadres avancés	Non implémenté	—	—	—	cadre = rectangle simple (taille réglable)
Compensation directionnelle	Partiel	—	—	—	satin uniquement
Tâches asynchrones	Non implémenté	—	—	—	traitements synchrones

Dette technique connue

- Le compte CTest courant est **814** sous Linux sans Qt (2026-10-06, hors tests desktop) ; éviter de figer ce nombre dans les pages d'introduction sans le mettre à jour avec la CI.
- Le satin dispose désormais d'un moteur géométrique par **squelette** (`auto_satin::build_satin_columns`, Lot 1) et d'une **génération par barreaux** (`fill_satin_columns`, Lot 2), points courts / split / terminaisons (Lot 3) et **sous-couches (center/edge/zigzag) + compensation pull/push** (Lot 4, passes distinctes), **points d'entrée/sortie + points de fixation (lock)** (Lot 5) et **routage multi-colonnes** (Lot 6 : ordre/orientation + trajets cachés). Le **tatami** reçoit ses **sous-couches (contour + parallèle), l'underpath caché et le point d'entrée** (Lot 7). Le satin **auto naïf** reste désactivé ; le vrai moteur topologique est désormais celui essayé par défaut dans `autodigitize`.
- Les sections d'un réseau satin portent maintenant des indices et identifiants de jonction explicites, déterministes et persistés. Le routage privilégie ces jonctions quand l'écart reste compatible avec un trajet caché. La collecte déterministe des extrémités, la transaction multi-section atomique et l'ajout UI d'un guide interne dans chaque branche incidente sont présents. Les guides ainsi liés (`link_id`, scopé par `(source_vector, link_id)`) se déplacent et se suppriment désormais en **groupe atomique** : Maj+glisser une extrémité déplace toute l'identité logique (delta normalisé partagé, jamais de coordonnée/angle recopié entre sections) et supprimer un guide lié supprime le groupe entier, chacun via une seule commande undo/redo tout-ou-rien (`move_satin_guide_group`, `RemoveSatinGuidesCommand`). Un glisser SANS Maj reste un geste local (édition d'angle propre à une section, toujours possible et volontairement non propagé). Un groupe incomplet, associé à des identifiants de jonction différents ou dont la progression se croise entre les deux rails est refusé en bloc. Un Maj+clic sans déplacement ne crée aucune commande d'historique. Les guides terminaux de jonction restent verrouillés (ni déplaçables ni supprimables). Un guide lié qui perdrait son adjacence à sa jonction (un autre guide inséré entre les deux) refuse le geste de groupe plutôt que de deviner un intervalle. De plus, un retour textile vers une jonction reste rectiligne au lieu de retracer le centre d'une branche.
- **Correctif Lot 7** : `connector_invalid` ignorait les contacts sommet/extrémité et ne sondait l'intérieur d'un connecteur que si l'écart en x dépassait `2 × row_spacing`, laissant passer un connecteur quasi vertical traversant un trou de part en part en touchant exactement ses sommets. Corrigé par une découpe paramétrique du segment à chaque intersection (`segment_stays_in_region` exposé pour test). `tatami_underlay` retombait aussi silencieusement sur le bord **brut** si le retrait de contour échouait/disparaissait ; politique sûre désormais : aucune sous-couche de contour dans ce cas. `underlay_inset` et `underlay_spacing` sont exposés dans l'inspecteur (`PropertiesPanel`).
- **Correctif « auto-satin béton » — extension des bouts ouverts** : un bout de colonne sans jonction s'arrêtait, par artefact générique de l'amincissement (Zhang-Suen), sensiblement avant le bord réel — un embout arrondi restait entièrement hors couture (~11 % de la longueur d'une capsule de test), un bout carré retractait aussi (~2,55 mm). Corrigé (`extend_tip`, activé par défaut) : marche le long de la tangente sortante en ré-évaluant des sections transversales réelles tant qu'elles rétrécissent, fermeture par bissection (largeur plancher non nulle). Voir `docs/source/satin.md` § *Extension des bouts ouverts*, sources (brevet Pulse Microsystems, littérature d'élagage de squelette) citées dans ce même paragraphe.
- **Correctif « auto-satin béton » — ancrage des jonctions** : le défaut inverse du précédent. La section transversale d'une branche, calculée depuis sa seule tangente, dérive nettement en approchant d'une jonction (jusqu'à ~9,2 mm mesuré sur "y" contre ~5,0 mm nominal) car le rayon balaie le bourrelet de la confluence, pas la ceinture réelle de la branche — les sections d'un Y ne se rejoignaient pas au même point (jusqu'à ~5 mm d'écart). Corrigé (`trim_and_anchor_junction_end`, activé par défaut) : amputation de la queue instable (croissance > 10 % par rapport à la station voisine) puis ancrage indépendant de chaque rail sur le sommet reflex (concave) du contour le plus proche, avec exclusion mutuelle si les deux rails d'une branche convoitent le même sommet (défaut réel trouvé sur "T", corrigé avant commit). Voir `docs/source/satin.md` § *Ancrage des jonctions sur les sommets reflex du contour*. La limite alors non corrigée (topologie de squelette incorrecte sur la forme "croix", nœud de degré 2 au lieu de degré 4) a depuis été corrigée — voir le correctif « audit satin adversarial » plus bas.

• **Correctif « auto-satin béton » (suite) — trajet caché non validé** : `route_columns` décidait un trajet caché (`ConnectorKind::Underpath`, fil caché sous la broderie, sans coupe) sur la seule distance (`gap ≤ underpath_max`, 8 mm), sans jamais regarder si `step.junction` — déjà calculé juste au-dessus pour ce même pas — avait une valeur : deux colonnes satin géométriquement proches mais **sans aucun lien topologique réel** (deux lettres rapprochées, une forme en C dont les deux bouts se frôlent sans être reliés) pouvaient donc être cousues en ligne droite à travers un espace dont rien ne garantissait qu'il était couvert de tissu. Corrigé par un seuil à deux paliers : `underpath_max` (8 mm, inchangé) ne s'applique plus que si `step.junction` a une valeur — désormais fiable grâce à l'ancrage exact des jonctions ci-dessus — sinon un nouveau seuil, bien plus strict, `underpath_max_without_junction` (1,5 mm) ne tolère qu'un quasi-contact (arrondi de rastérisation, extrémités coïncidentes) ; au-delà, la liaison redevient un saut. Voir `docs/source/satin.md` § *Routage multi-colonnes*.

• **Audit satin adversarial (2026-08-02)** : revue systématique de tout le pipeline satin (squelette, colonnes, remplissage, guides, routage, lock), demandée explicitement par l'utilisateur (« vraiment check et recheck »). Quatre audits indépendants ont trouvé 8 défauts réels, tous corrigés et verrouillés par test :

• **`skeleton_graph.cpp`** : le traçage d'arête s'arrêtait sur le premier voisin rencontré au lieu de prioriser un nœud sur tout le 8-voisinage — cause racine du bug "croix" (degré 2 au lieu de 4) **et** d'un second défaut non documenté (une branche entière de "y" disparaissait avec une arête parasite en boucle sur la jonction, faute d'exclure le pixel d'origine de la trace, pas seulement le précédent). Les deux corrigés.

• **`graph_cleanup.cpp`** violait son propre contrat (« ne supprime rien silencieusement ») : un nœud sans aucune arête (isolé dès le départ) disparaissait sans apparaître dans `removed`. Corrigé.

• **`satin_column.cpp`** : une arête reliant deux jonctions (le pont d'un "H") n'était jamais convertie en colonne — zone non cousue, aucun avertissement. Corrigé (toutes les arêtes sont désormais essayées).

• **`satin_column.cpp`** : l'amputation de la queue instable à l'ancrage d'une jonction comparait chaque station à sa seule voisine immédiate (+10 %), insuffisant si le bourrelet décroît progressivement sur plusieurs stations (mesuré : +72 % cumulé, chaque écart local restant sous 10 %). Corrigé par un critère de décroissance stricte.

• **`satin.cpp`** : tout satin créé manuellement (sans barreaux) retombait sur `fill_satin`, qui n'implémente qu'un sous-ensemble de `SatinConfig` — terminaisons/split/sous-couches de bord/compensation asymétrique restaient silencieusement sans effet malgré l'inspecteur. Corrigé par `default_rungs` (barreaux par défaut, même correspondance ladder), appliqué à la création manuelle (`createSatinObject`).

• **`satin.cpp`** : `push_start`/`push_end` n'avaient aucune borne (contrairement à `pull_max`) — une rétraction excessive croisait le premier fil de la colonne sur lui-même. Corrigé (borné à la station voisine).

• **`lock.cpp`** : la longueur du lock n'était jamais comparée à la distance réelle disponible — sur une colonne fine, l'aiguille piquait hors de la matière cousue. Corrigé.

• **`generate.cpp`** : la décision de routage (trajet caché/saut) se fondait sur le milieu brut des barreaux, pas sur le point réellement cousu une fois `push_start`/`push_end` appliqué — pouvait réintroduire le défaut du correctif précédent par un chemin différent. Corrigé. Détails et preuves de test pour chacun dans `docs/source/satin.md`. **398 tests CTest** verts Debug/Release (383 avant + 11).

• **Formes dessinées à la main (rectangle/ellipse/polygone/forme libre)** : demande utilisateur en cours de mission « auto-satin béton » — jusqu'ici, la seule façon d'obtenir un `VectorObject` était de vectoriser une région depuis une image importée, comme dans un logiciel de digitalisation classique (Hatch et équivalents) qui permet aussi de dessiner directement une forme de base. Quatre outils dans la palette (rectangle glissé, ellipse glissée avec Maj = cercle, polygone à clics successifs fermé par double-clic, **forme libre/lasso à glisser continu, simplifiée par Douglas-Peucker à la fermeture**) créent un `VectorObject` sans région source, immédiatement sélectionné : l'utilisateur enchaîne avec les actions **Créer un…** existantes, sans nouveau chemin côté broderie. Voir `docs/source/vectorization.md` § *Formes dessinées à la main*. Testé par QTest (création, undo/redo, cadre dégénéré, tracé de polygone/forme libre, annulation) ; **pas de validation visuelle manuelle** dans cet environnement de développement (capture d'écran de l'application non disponible).

- **Finition « formes dessinées à la main » (2026-08-02)**, demandée explicitement par l'utilisateur (« finis d'implém celle existante » + ajout d'une forme libre). Deux volets : 1. **Défaut trouvé par revue — Maj = cercle jamais réellement testé bout en bout.** La contrainte cercle (ellipse + Maj) interrogeait `QGuiApplication::keyboardModifiers()` (état clavier global) au moment de traiter la fin du glisser — correct en usage réel, mais impossible à couvrir par un test QTest offscreen (état clavier non fiable à rejouer), donc jamais vérifié par un test qui aurait pu échouer si le comportement se cassait. Corrigé : les modificateurs sont capturés directement sur l'évènement Qt qui termine le glisser (`CanvasView::mouseReleaseEvent`) et transmis par le signal `boxDrawnMm` — plus de dépendance à un état global, testable directement en passant `Qt::ShiftModifier`. 2. **Nouvel outil « forme libre » (lasso, raccourci `L`)** : glisser continu capté point par point (`CanvasView::freeformPointMm`/ `freeformStrokeFinished`), simplifié à la fermeture par la même fonction Douglas-Peucker que la vectorisation d'image (`freeform_path`, réutilise `simplify()` — aucun nouvel algorithme de correspondance). Tolérance de 0,3 mm (même ordre de grandeur que la densité satin par défaut) : lisse le tremblement de la souris sans effacer l'intention du tracé. Moins de 3 sommets après simplification → aucun objet créé, message clair (au lieu d'un échec silencieux, vérifié même quand le tracé brut a suffisamment de points bruts mais s'avère colinéaire). Voir `docs/source/vectorization.md` § *Formes dessinées à la main* pour le détail. **398 tests CTest** verts Debug/Release (398 avant + 4 nouveaux `TEST_CASE` Catch2 pour `freeform_path` ; les 4 nouvelles fonctions QTest côté desktop s'ajoutent dans le même exécutable `test_main_window`, déjà compté comme un seul test CTest, sans changer ce total).
- **Correction de l'appariement rail gauche/rail droit (2026-08-01)** : l'ancien appariement (`fill_satin` sans barreaux, et l'interpolation intra-intervalle de `fill_satin_columns`) associait les deux rails par la même fraction d'abscisse curviligne appliquée indépendamment à chacun — faux dès que les rails divergent en longueur/courbure (virage, coude, largeur variable), causant éventails, quasi-croisements et densité irrégulière sur un ruban courbe ou anguleux. Remplacé par une correspondance locale monotone (« ladder », diagonale la plus courte, garde-fou anti-croisement, O(n) par intervalle) — voir `docs/source/satin.md` § *Correction de l'appariement*. Limite assumée : au sommet d'un coude C0 franc (sans congé), un seul fil absorbe nécessairement une déviation angulaire importante (mitre/congé = `ShortStitchMode`, hors périmètre). Fixtures et métriques dédiées : `tests/unit/stitch/test_satin_pairing_metrics.cpp` (13 tests).
- **Revue corrective ladder_correspondence, audit adverse (2026-08-01)** : fixtures délibérément défavorables (rails tête-bêche, échantillonnage très asymétrique/segments nuls, longueurs très différentes + largeur quasi nulle, épingle à cheveux ~170°, barreaux désordonnés/dupliqués). Deux défauts réels corrigés : rails fournis tête-bêche (précondition non vérifiée, nœud papillon en $O(n^2)$) → détection + ré-orientation interne automatique ; barreaux dépendants de l'ordre du vecteur d'entrée (un barreau en tête mais loin le long de la colonne faisait rejeter silencieusement tous les suivants, repli sans barreaux) → tri par position projetée avant filtrage, plus fusion des barreaux quasi-dupliqués (source d'un croisement isolé près d'un virage serré, faute de garde-fou entre deux intervalles voisins). Voir `docs/source/satin.md` § *Revue corrective (audit adverse)*. Aucun chemin de production actuel n'atteint le cas tête-bêche (`rails_from_contour`/`build_satin_columns` garantissent déjà le même sens). L'UI édite désormais les barreaux, mais maintient leur progression monotone et leur écart minimal ; les imports désordonnés restent acceptés et triés à la génération. Ces gardes défensives restent nécessaires à la fonction de bibliothèque publique.
- Aucune validation sur machine à broder réelle.

Niveaux de validation

« Implémenté » signifie ici *présent et testé logiciellement* — pas *prêt pour la production*. Pour un logiciel de broderie, il faut distinguer :

Niveau	Signification	État du projet
Présent	le code existe	oui (pipeline complet)
Testé numériquement	les invariants logiciels passent	oui (217 tests)
Validé visuellement	les trajectoires paraissent cohérentes	partiel (SVG de diagnostic, aperçu)
Validé sur simulateur	vérifié dans un visualiseur tiers	non
Validé physiquement	broderies réelles examinées	**non**

Un générateur peut passer 217 tests sans produire une bonne broderie : les tests vérifient des **invariants** (pas de couture dans un trou, espacement par longueur d'arc, aller-retour DST exact…), pas la **qualité textile**.

Statut de validation

Le logiciel constitue un **prototype fonctionnel** couvrant l'ensemble du pipeline principal. Ses composants sont **testés logiciellement** et vérifiés **visuellement** (SVG de diagnostic), mais la **qualité de broderie produite n'est pas encore validée sur machine réelle**. Ne pas considérer les résultats comme prêts pour la production sans essais machine.

Implémentation associée

Voir chaque chapitre thématique et l'annexe *Audit du dépôt*.

Roadmap

Public : mainteneur, contributeur. Vue d'ensemble **courte** ; **aucune date n'est annoncée**. Revue le 2026-10-06 après un audit complet contre le code.

Trois documents, un rôle chacun — ne pas dupliquer leur contenu ici :

Question	Document
Qu'est-ce qui manque par rapport à Hatch, et où en est chaque entrée (■ ■ ■) ?	`docs/roadmap-parite-hatch.md` — **source de vérité des statuts** (256 entrées HP-*, tableau de bord en tête)
Dans quel ordre le livrer (vagues, scopes S1…S15, dépendances, plans) ?	`specs/implementation-roadmap.md`
Où va le projet, en gros ?	ce chapitre

Déjà livré (depuis la roadmap d'origine)

- Auto-satin par squelette et traversées orientées (`satin-squelette.md`, branche de recherche brevets, **non fusionné sur `main`**, à essayer sur machine) ;
- Moteur satin topologique historique (supprimé, remplacé ci-dessus) : réseaux Y/T/anneaux, guides éditables, jonctions ancrées, sous-couches, underpath caché, coupes et points d'arrêt, filtre de points courts (`satin.md`, `tatami.md`, `auto-numerisation.md`).
- Auto-numérisation durcie (Lots A à G : fond présumé, angles et sous-couches des remplissages, chevauchement, fragments, finitions, métriques) et mode **Contours / Line Art** (traits → running/satin par couleur, curseur de détail, 2026-10-05).
- Fichiers : Nouveau projet, Enregistrer / Enregistrer sous, fichiers récents, sauvegarde automatique avec récupération après plantage.
- Bibliothèque de fils (`thread_palette`) reliée au document et à l'interface (fil par objet, panneau Fils, film couleur, import de nuanciers CSV/JSON) ; nuanciers de marques intégrés encore en données de démonstration.
- Distribution : installateur Windows (Inno Setup), workflow de release sur tag, release « latest » publiée par la CI — **installateur non signé**.

En cours

- **PR #5** — couche de normalisation machine (HP-FMT-001), préalable à PES/JEF/EXP.
- **PR #3** — plan de code de la sélection, du presse-papiers et des transformations (S3).
- **Satin guidé** : restent la propagation géométrique coordonnée des angles et des déplacements de guides sur un réseau, puis les retours textiles suivant le réseau plutôt qu'un segment direct. Ne jamais accepter silencieusement une gerbe ou un croisement comme satin valide.

Court terme (P0 restants)

Formats machine (PES, JEF, EXP) · fils par objet et film couleur · sélection multiple, presse-papiers, rotation, miroir · tracés ouverts et satin de bordure (outils) · lettrage · longueurs min/max appliquées à tous les générateurs · tâches de fond avec annulation · rendu réaliste des points · fiche de production · installateur signé · protocole de validation sur machine réelle · jeu de référence et métriques pour l'auto-numérisation. La liste exacte et l'ordre sont dans les deux documents ci-dessus.

Moyen et long terme

- Remplissages avancés (concentrique, spirale, radial, motifs), édition de points complète, simulateur, profils machine/cadres/tissus, appliqué, bibliothèques de motifs.
- Autres formats (VP3, HUS, …), import SVG/DXF plus complet, migrations `.osp` systématiques.
- Internationalisation (infrastructure puis anglais) : volontairement **en toute fin**, un seul balayage `tr()` une fois le reste fonctionnel.
- Portage macOS et publication publique.

Sources

- `docs/phase0/08-roadmap-adr.md` — roadmap d'origine (13 phases), **historique**.
- `docs/stitch-engine-audit.md` — défauts et priorités du moteur de points.
- `docs/roadmap-parite-hatch.md`, `specs/implementation-roadmap.md` — état courant.

Implémentation associée

Les fonctionnalités « Non implémenté » du chapitre *Limitations* constituent la liste de travail ; chacune indique le module cible.

Glossaire

Pour débuter

Les mots que vous croisez dans l'application et dans le Guide de prise en main, sans nom de code.

Terme	Signification
Point	Une piqûre d'aiguille. Un motif est une longue suite de points
Saut	Déplacement de l'aiguille **sans couture** d'un endroit à un autre
Coupe	Le fil est coupé (entre deux objets éloignés, ou à la fin). Réglée par un **seuil** : un déplacement plus long devient une coupe, plus court un simple saut. Le fichier se termine toujours par une coupe finale. **À tester sur votre machine**
Point d'arrêt	Quelques petits points qui bloquent le fil avant et après une coupe, pour qu'il ne se défasse pas
Changement de couleur	Arrêt de la machine pour passer au fil suivant
Tatami	Remplissage d'une surface par rangées parallèles de points
Satin	Zigzag serré qui couvre une bande étroite (lettre, bordure) : brillant, mais seulement adapté aux formes allongées
Auto-satin	Satin construit **automatiquement** à partir de la seule forme, sans tracer de rails. Expérimental, non validé sur machine ; il refuse les formes qui ne s'y prêtent pas (disque, forme compacte)
Squelette	Ligne médiane d'une forme (son « axe »), calculée par l'auto-satin pour savoir dans quelle direction coudre
Traversée	Un trait de satin d'un bord de la forme à l'autre ; le satin est une suite de traversées serrées
Guide	Repère que vous posez pour orienter les fils d'un satin ou d'un remplissage dans une direction précise
Sous-couche	Points posés **sous** le satin ou le tatami pour le stabiliser et le soutenir ; invisibles une fois la couche visible cousue
Tirage (compensation de tirage)	Le fil tire sur le tissu et rétrécit les formes ; on élargit légèrement la forme pour compenser
Densité	Écart entre deux rangées ou deux traversées : plus il est petit, plus la couture est serrée
Région	Zone de couleur d'une image **segmentée** (la segmentation découpe l'image en régions)
Région active	La région actuellement **sélectionnée** : une seule à la fois ; les commandes Fusionner, Supprimer, Recolorer et Vectoriser lui sont appliquées
Objet vectoriel	Contour éditable (avec ses nœuds) obtenu à partir d'une région
Objet de broderie	Intention de couture attachée à une forme : contour cousu, tatami ou satin
Numérisation automatique	Création automatique des objets de broderie pour toutes les régions
Projet (.osp)	Votre travail complet et éditable. Le DST, lui, ne contient que les points
DST	Format de fichier de broderie lu par beaucoup de machines (Tajima)
Cadre	Zone de broderie du tambour (rectangle rouge sur le canevas)

Termes techniques

Terme français, terme anglais utilisé dans le code, définition, et type/classe associé lorsqu'il existe.

Français	Anglais (code)	Définition	Type / fichier
Point	stitch	Pénétration d'aiguille cousant le fil	`CommandType::Stitch`
Saut	jump	Déplacement de l'aiguille sans couture	`CommandType::Jump`
Coupe-fil	trim	Coupe du fil	`CommandType::Trim`
Changement de couleur	color change	Arrêt pour changer de fil	`CommandType::ColorChange`
Arrêt	stop	Arrêt machine	`CommandType::Stop`
Fin	end	Fin du motif	`CommandType::End`
Point droit / courant	running stitch	Points le long d'un chemin	`run_stitch`
Point triple	bean stitch	Chaque segment cousu 3×	`RepeatMode::BeanStitch`
Point arrière	backstitch	Progression avec recouvrement	`RepeatMode::Backstitch`
Remplissage	fill	Couture d'une surface	`fill_tatami`
Tatami	tatami	Remplissage par rangées parallèles	`TatamiParams`
Colonne satin	satin column	Zigzag entre deux rails	`SatinParams`, `fill_satin`
Rail	rail	Bord d'une colonne satin	`SatinParams::rail_a/b`
Sous-couche	underlay	Couche cousue avant la couche visible	`center_underlay`
Compensation de tirage	pull compensation	Élargissement pour compenser la traction	`pull_compensation`
Densité	density / row_spacing	Espacement des fils/rangées	`density`, `row_spacing`
Longueur de point	stitch length	Longueur d'un point	`stitch_length`
Angle	angle	Orientation du remplissage	`Angle`
Contour	contour / outer	Bord d'une région	`PathSet::outer`
Trou	hole	Contour intérieur	`PathSet::holes`
Chemin	path	Polyligne/courbe	`geometry::Path`
Région	region	Zone de couleur connexe	`segmentation::Region`
Objet vectoriel	vector object	Contours éditables	`document::VectorObject`
Objet de broderie	embroidery object	Intention de couture	`document::EmbroideryObject`
Séquence de points	stitch sequence	Commandes machine	`stitch::StitchSequence`
Cadre / tambour	hoop / canvas	Zone de broderie	`document::Canvas`
Palette	palette	Ensemble de couleurs/fils	bibliothèque `thread_palette` (nuancier Générique libre + import utilisateur ; nuanciers de marques intégrés = démo), panneau Fils
Fil	thread	Fil de broderie	`thread_palette::Thread` ; assigné à un objet par `EmbroideryObject::thread`
Ordre de couture	sewing order	Ordre des objets	`optimization`, `SewingOrder`
Micromètre	micrometer	Unité interne (1/1000 mm)	`Micrometers`
Déplacement (interne)	travel	Liaison non cousue d'un remplissage	`FillStitch::travel`
Squelette	skeleton	Axe médian d'une forme (auto-satin)	`auto_satin`
Traversée	crossing	Segment d'un bord à l'autre d'une colonne satin	`skeleton_satin.hpp`
Guide	guide	Repère d'orientation des fils	`AutoSatinGuide`, guides de direction
Auto-satin	auto-satin	Satin par squelette et traversées orientées	`generate_skeleton_satin`
Coupe finale	final trim	Coupe qui termine la séquence	`finish_sequence`
Point d'arrêt	lock stitch	Points de verrouillage autour d'une coupe	`LockStitch`

Licences

Public : mainteneur, contributeur.

Licence du logiciel

OpenStitch Studio est distribué sous **Apache-2.0** (permissive, avec clause de brevets). Fichier : `LICENSE`. Chaque fichier source porte `// SPDX-License-Identifier: Apache-2.0`.

Conséquence importante, formulée avec prudence : intégrer directement du code sous licence **copyleft** (comme la GPL) dans ce projet imposerait vraisemblablement de distribuer l'ensemble dérivé sous une licence compatible avec cette licence copyleft, ce qui empêcherait probablement de conserver le tout **uniquement** sous Apache-2.0. La compatibilité exacte dépend de la version de la licence et de la manière dont le code est combiné.

En pratique, pour **conserver la distribution du projet sous Apache-2.0**, aucun code sous licence copyleft incompatible avec cet objectif ne doit être intégré directement. Les **idées algorithmiques** peuvent être réimplémentées indépendamment à partir de sources publiques, sans reprise de code ni de structure expressive protégée. **Toute réutilisation de code tiers doit faire l'objet d'une vérification de licence spécifique.** Voir `docs/stitch-engine-research.md`.

Dépendances externes

Bibliothèque	Version	Rôle	Licence	Intégration	URL
fmt	via vcpkg	formatage	MIT	vcpkg	github.com/fmtlib/fmt
spdlog	via vcpkg	journalisation	MIT	vcpkg	github.com/gabime/spdlog
CLI11	via vcpkg	args CLI	BSD-3-Clause	vcpkg	github.com/CLIUtils/CLI11
OpenCV (core/imgproc/imgcodecs)	via vcpkg	image	Apache-2.0	vcpkg	opencv.org
libpng / libjpeg-turbo / libtiff / zlib	transitives	codecs	zlib / IJG / libtiff / zlib	via OpenCV	—
Clipper2	via vcpkg	booléens/offsets	BSL-1.0	vcpkg (encapsulée)	github.com/AngusJohnson/Clipp
nlohmann/json	via vcpkg	JSON projet	MIT	vcpkg (encapsulée)	github.com/nlohmann/json
minizip-ng	via vcpkg	ZIP projet	zlib	vcpkg (encapsulée)	github.com/zlib-ng/minizip-ng
Catch2 v3	via vcpkg	tests	BSL-1.0	vcpkg (dev)	github.com/catchorg/Catch2
Qt 6.8 LTS (Widgets/Gui/Core)	binaires officiels	interface	**LGPL-3.0**	liaison dynamique	qt.io

Les versions exactes sont verrouillées par la **baseline vcpkg** (`vcpkg.json`).

Qt et la LGPL-3.0

Qt est utilisé en **liaison dynamique** avec les DLL officielles non modifiées : elles sont remplaçables par l'utilisateur, aucune source Qt n'est modifiée, et seuls des modules LGPL sont utilisés (pas de module GPL-only ni commercial). Ce mode est compatible avec la distribution d'un logiciel Apache-2.0.

Licences de la chaîne documentaire

La documentation est générée avec `python-markdown` (BSD), `xhtml2pdf` (Apache-2.0) et `reportlab` (BSD), `svglib` (LGPL, utilisée comme outil externe, non liée au produit). Ces outils ne sont pas distribués avec le logiciel.

Implémentation associée

- `LICENSE`, `THIRD_PARTY_LICENSES.md`.
- `docs/phase0/03-bibliotheques-licences.md` (analyse d'origine, ADR-002).
- `docs/stitch-engine-research.md` (point juridique Ink/Stitch).

Annexe — Audit du dépôt

Cet audit est la **base factuelle** de toute la documentation : chaque affirmation technique du présent document est vérifiable dans le code réel du dépôt. Il a été établi par inspection directe de l'arborescence, des fichiers CMake, des en-têtes, des sources, des tests et des configurations.

Identité du projet

Élément	Valeur	Source
Nom	OpenStitch Studio (temporaire, remplaçable)	`libs/core/include/openstitch/core/app_i` (`kAppName`)
Version	0.1.0	même fichier (`kAppVersion`), `CMakeLists.txt` (`project(... VERSION 0.1.0)`)
Licence	Apache-2.0	`LICENSE`, `docs/phase0/03-bibliotheques-licences.m` (ADR-002)
Langage principal	C++23	`CMakeLists.txt` (`CMAKE_CXX_STANDARD 23`)
Plateforme	Windows 10/11 (cœur portable Linux)	`CMakePresets.json` (presets `msvc`, `linux-core`)
Interface	Qt 6 Widgets (LGPL, liaison dynamique)	`apps/desktop/CMakeLists.txt`
Gestion des dépendances	vcpkg (manifeste)	`vcpkg.json`, `CMakePresets.json`

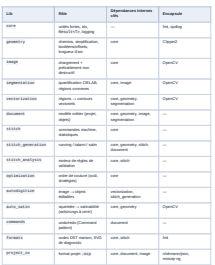
Exécutables produits

Cible	Type	Fichier	Dépend de Qt
`openstitch`	Application de bureau	`apps/desktop/`	Oui
`openstitch-cli`	Outil en ligne de commande	`apps/cli/main.cpp`	Non

Sous-commandes CLI réellement définies (`app.add_subcommand`, `apps/cli/main.cpp`) : `info`, `stats`, `dst2svg`, `stitchdebug`.

Bibliothèques internes (14)

Chaque dossier de `libs/` est une cible CMake `openstitch::<nom>` :



Dépendances externes

Depuis `vcpkg.json` et `apps/desktop/CMakeLists.txt` : `fmt`, `spdlog`, `cli11`, `clipper2`, `nlohmann-json`, `minizip-ng`, `opencv4` (features `core`, `png`, `jpeg`, `tiff`), `catch2` (tests) ; **Qt 6.8** hors vcpkg (binaires officiels, variable `QT_ROOT`). Voir le chapitre *Licences*.

Principaux espaces de noms et types

- Espace racine `openstitch` ; sous-espaces `geometry`, `image`, `segmentation`, `vectorization`, `document`, `stitch`, `stitch_generation`, `stitch_analysis`, `optimization`, `autodigitize`, `commands`, `formats`, `project_io`, `desktop`.
- Unités fortes : `Micrometers`, `Millimeters`, `Pixels`, `Angle`, `Vec2um` (`libs/core/include/openstitch/core/units.hpp`).
- Identifiants : `ObjectId`, `RegionId`, `ColorId`, `ThreadId`, `IdGenerator<T>` (`libs/core/include/openstitch/core/ids.hpp`).
- Erreurs : `Result<T>` = `std::expected<T, Error>` (`.../core/error.hpp`).
- Document : `Project`, `VectorObject`, `EmbroideryObject`, `StitchParams` (variant `RunningStitchParams | TatamiParams | SatinParams`), `Canvas` (`libs/document/include/openstitch/document/`).
- Points : `StitchCommand`, `CommandType`, `StitchSequence`, `StitchStats` (`libs/stitch/include/openstitch/stitch/sequence.hpp`).

Formats pris en charge

Format	Import	Export	Module
PNG, JPEG, BMP, TIFF	Oui	— (export projet uniquement)	`image` (OpenCV)
DST (Tajima)	Oui	Oui	`formats/src/dst.cpp`
SVG (diagnostic)	—	Oui	`formats/src/svg.cpp`
`.osp` (projet)	Oui	Oui	`project_io`

Interface graphique réellement implémentée

Menus construits dans `apps/desktop/main_window.cpp` (`buildMenus`, `buildAnalysisPanel`, `buildSimulationToolbar`, `buildOrderPanel`) : Fichier, Édition, Image, Segmentation, Broderie, Affichage, Analyse, plus un panneau d'ordre de couture (dock), un panneau d'analyse (dock) et une barre de simulation. Le détail est donné dans le *Guide utilisateur détaillé*.

Types d'objets de broderie

`RunningStitchParams` (point droit/double/triple), `TatamiParams` (remplissage), `SatinParams` (colonne satin), portés par le variant `StitchParams` (`libs/document/include/openstitch/document/embroidery_object.hpp`).

Algorithmes présents (vérifiés dans le code)

Conversion RGB↔CIELAB et k-means (`segmentation`), composantes connexes 4-conn (`segmentation`), extraction de contours Suzuki-Abe (`vectorization`), simplification Douglas-Peucker (`geometry/simplify.cpp`), union/offset via Clipper2 (`geometry/clean.cpp`, `geometry/offset.cpp`), aplatissement Bézier adaptatif + longueur d'arc (`geometry/polyline.cpp`), running stitch par longueur d'arc avec découpe aux coins (`stitch_generation/running_stitch.cpp`), tatami scanline + routage par graphe (`stitch_generation/tatami.cpp`), satin à deux rails (`stitch_generation/satin.cpp`), règles d'analyse (`stitch_analysis/analyze.cpp`), ordre de couture (`optimization/order.cpp`), codec DST (`formats/dst.cpp`).

Tests présents

25 fichiers `test_*.cpp` sous `tests/unit/<lib>/` et un test d'intégration `tests/integration/test_pipeline.cpp`. Total : **161 tests** au dernier passage (`ctest`). Framework : Catch2 v3. Voir le chapitre *Tests*.

Exemples et ressources

- SVG de diagnostic de référence : `tests/golden/stitch-generation/` (formes de running stitch et remplissage anneau).
- Aucun jeu d'images d'exemple versionné à la racine `examples/` à ce jour (*Information non déterminée dans le dépôt* : dossier `examples/` non peuplé).

Fonctionnalités expérimentales / partielles / prévues

- **Prévu (architecture mentionnée, non implémenté)** : palette de fils réelle (`thread_palette`), profils de machine/cadres avancés, remplissages courbes / radiaux / spirale / motifs, sous-couches tatami, édition manuelle des points.
- **Partiel** : classification auto des régions (heuristique de largeur) — essaie désormais le vrai moteur `auto_satin` par squelette puis retombe sur le **tatami** ; le satin **automatique** naïf reste désactivé par défaut (`use_naive_satin`). Rails, barreaux et décomposition multi-sections sont présents. Compensation (satin uniquement) ; routage tatami (graphe + validation géométrique, underpath caché non implémenté — les liaisons hors-région sont des sauts).
- **Expérimental** : `openstitch-cli stitchdebug` / `auto-satin-debug` (outils de diagnostic du moteur).

Marqueurs, incohérences

- Aucun `FIXME` critique repéré ; le `README.md` mentionne un compte de tests d'un jalon antérieur alors que le compte réel est **161** — incohérence mineure, à resynchroniser dans le README.
- Documents de conception : `docs/phase0/` (étude de cadrage, 14 ADR), `docs/stitch-engine-audit.md` et `docs/stitch-engine-research.md` (refonte du moteur de points).

Éléments non déterminés dans le dépôt

- URL de dépôt public : aucune (dépôt local uniquement).
- Fichier de logo : aucun logo bitmap/vectoriel dédié trouvé ; la couverture du PDF n'en utilise donc pas.
- Couverture de code chiffrée : non configurée.